

# AMD Instinct MI300 指令集架构 中文译稿

| 缩写           | 英文全称                                               | 中文含义                        |
|--------------|----------------------------------------------------|-----------------------------|
| AMD          | Advanced Micro Devices                             | 美国超威半导体公司                   |
| CDNA         | AMD CDNA (architecture name)                       | AMD 的计算架构名称；本文对应第 3 代       |
| ISA          | Instruction Set Architecture                       | 指令集架构                       |
| GPU          | Graphics Processing Unit                           | 图形处理器                       |
| CPU          | Central Processing Unit                            | 中央处理器                       |
| APU          | Accelerated Processing Unit                        | 加速处理器                       |
| CU           | Compute Unit                                       | 计算单元                        |
| ALU          | Arithmetic Logic Unit                              | 算术逻辑单元                      |
| SALU         | Scalar ALU                                         | 标量算术逻辑单元                    |
| VALU         | Vector ALU                                         | 向量算术逻辑单元                    |
| GPR          | General-Purpose Register                           | 通用寄存器                       |
| SGPR         | Scalar General-Purpose Register                    | 标量通用寄存器                     |
| VGPR         | Vector General-Purpose Register                    | 向量通用寄存器                     |
| AGPR         | Accumulator General-Purpose Register               | 累加器通用寄存器                    |
| LDS          | Local Data Share                                   | 本地数据共享存储                    |
| GDS          | Global Data Share                                  | 全局数据共享存储                    |
| GWS          | Global Wave Sync                                   | 全局波前同步                      |
| SIMD         | Single Instruction, Multiple Data                  | 单指令多数据                      |
| DPP          | Data-Parallel Processor / Data Parallel Primitives | 第 1 章指数据并行处理器；指令修饰语境指数据并行原语 |
| DMA          | Direct Memory Access                               | 直接内存访问                      |
| MMIO         | Memory-Mapped I/O                                  | 内存映射输入输出                    |
| PC           | Program Counter                                    | 程序计数器                       |
| EXEC         | Execute Mask                                       | 执行掩码                        |
| SCC          | Scalar Condition Code                              | 标量条件码                       |
| VCC          | Vector Condition Code                              | 向量条件码                       |
| EXECZ / VCCZ | EXEC Zero / VCC Zero                               | EXEC / VCC 为零的状态位           |
| NOP          | No Operation                                       | 空操作                         |
| MFMA         | Matrix Fused Multiply-Add                          | 矩阵融合乘加                      |
| FMA          | Fused Multiply-Add                                 | 融合乘加                        |

| 缩写          | 英文全称                                              | 中文含义                  |
|-------------|---------------------------------------------------|-----------------------|
| MAC         | Multiply-Accumulate                               | 乘加累积                  |
| SMFMAC      | Sparse Matrix Fused Multiply-Accumulate           | 稀疏矩阵融合乘加累积            |
| FP          | Floating Point                                    | 浮点；后缀数字通常表示位宽         |
| BF16 / BF8  | Brain Floating Point 16 / 8                       | 16 位 / 8 位 Brain 浮点格式 |
| NaN         | Not a Number                                      | 非数                    |
| QNaN / SNaN | Quiet / Signaling NaN                             | 安静非数 / 信号非数           |
| ULP         | Unit in the Last Place                            | 最低有效位的单位              |
| MSB / LSB   | Most / Least Significant Bit                      | 最高 / 最低有效位            |
| OOB         | Out of Bounds                                     | 越界                    |
| ECC         | Error-Correcting Code                             | 纠错码                   |
| IEEE        | Institute of Electrical and Electronics Engineers | 电气与电子工程师协会            |
| SDWA        | Sub-Dword Addressing                              | 子双字寻址                 |
| VRAM        | Video Random-Access Memory                        | 显存                    |
| VMEM / SMEM | Vector / Scalar Memory                            | 向量 / 标量内存访问单元或指令      |
| TBA / TMA   | Trap Base Address / Trap Memory Address           | 陷阱程序基址 / 陷阱内存地址       |
| TTMP        | Trap Temporary Register                           | 陷阱临时寄存器               |
| SPI         | Shader Processor Interpolator                     | 着色器处理器插值器             |
| ATC         | Address Translation Cache                         | 地址转换缓存                |
| DX          | DirectX                                           | 微软图形与计算接口系列           |

## 译稿说明

原文为 AMD《AMD Instinct MI300 Instruction Set Architecture: Reference Guide》，封面日期 2025 年 8 月 5 日，共 561 个 PDF 页面，正文页码为 1-553。译稿对应用户提供的本地文件。

本文件为非官方中文机器译稿。已统一主要技术术语，校对前言、执行模型、PC、EXEC 和部分反复出现的指令说明，并抽查排版；其余内容未逐条完成技术审校。实现指令或判断边界条件时，应对照英文原文。

正文逐页保留原版图表、指令名、寄存器名、数值和伪代码。页脚页码与原文一致；前置缩写表和说明页不计入原文页码。目录和书签用于定位原章节。

图片内无法直接替换的英文在下方提供中文对照。代码中的英文行注释已尽量翻译，代码本身及公式保持原样。过长、无法在原位置以可读字号放下的译文列入文末“排版补充译文”，原位置给出编号。

CDNA 3 为 MI300 系列的架构名称。译稿沿用规范原有型号范围；涉及 MI300A 与 MI300X 的差异时，需依据具体型号的资料核实。

# 图中英文对照

## 原文第 3 页（PDF 第 11 页）：CDNA 处理器框图

| 英文标注                                                                                | 中文含义                                 |
|-------------------------------------------------------------------------------------|--------------------------------------|
| Host CPU / Host Application / Compute Driver                                        | 主机 CPU / 主机应用程序 / 计算驱动程序             |
| MMIO ACCESS                                                                         | 内存映射输入输出访问                           |
| System Memory / Device Memory                                                       | 系统内存 / 设备内存                          |
| Commands / Instructions and Constants / Inputs and Outputs                          | 命令 / 指令和常量 / 输入和输出                   |
| Return Buffers / Private Data                                                       | 返回缓冲区 / 私有数据                         |
| MI300 Series Processor / Command Processors                                         | MI300 系列处理器 / 命令处理器                  |
| Ultra-Threaded Dispatch Processor                                                   | 超多线程派发处理器                            |
| Memory Controller / DMAs                                                            | 内存控制器 / DMA 单元                       |
| L2 R/W Cache / L1 R/W Cache                                                         | L2 读写缓存 / L1 读写缓存                    |
| Global Wave Sync / Local Data Share                                                 | 全局波前同步 / 本地数据共享                      |
| Compute Unit / Program Counter / DPP Array                                          | 计算单元 / 程序计数器 / 数据并行处理器阵列             |
| Instruction Cache / Constant Cache                                                  | 指令缓存 / 常量缓存                          |
| Discrete GPU - Physical Device Memory; APU - Region of system for GPU direct access | 独立 GPU：物理设备内存；APU：供 GPU 直接访问的系统内存区域。 |

## 原文第 6 页（PDF 第 14 页）：内存层次结构

| 英文标注                                              | 中文含义                                 |
|---------------------------------------------------|--------------------------------------|
| Memory-side Cache / CrossBar                      | 内存侧缓存 / 交叉开关互连                       |
| L2 Read/Write Cache / Texture R/W Cache L1 Per CU | L2 读写缓存 / 每个 CU 的 L1 纹理读写缓存          |
| LDS - 64kB per CU                                 | LDS：每个 CU 为 64 kB；其他寄存器数量、位宽和字段保留原图。 |

## 原文第 22 页（PDF 第 30 页）：分叉与汇合

| 英文标注                                                | 中文含义                                                                                   |
|-----------------------------------------------------|----------------------------------------------------------------------------------------|
| FORK / JOIN                                         | 分叉 / 汇合                                                                                |
| Every Fork/Join block requires three scalar GPRs... | 每个分叉/汇合块需要三个标量 GPR：两个保存执行掩码，以便进入该块前保存、离开后恢复；另一个保存三位的栈指针。图中的 mov64、mov、join 和 cb 保留原标识。 |

## 原文第 64 页（PDF 第 72 页）：缓冲区地址公式

| 英文标注                                                                                                                   | 中文含义                                                            |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Base / baseOffset / Inst_offset / Voffset / Stride / Vindex / TID                                                      | 基址 / 基址偏移 / 指令偏移 / 向量偏移 / 步长 / 向量索引 / 线程编号                      |
| Voffset is ignored when instruction bit OFFEN == 0                                                                     | 指令位 OFFEN == 0 时, 忽略 Voffset。                                   |
| Vindex is ignored when instruction bit IDXEN == 0                                                                      | 指令位 IDXEN == 0 时, 忽略 Vindex。                                    |
| TID is a constant value (0..63) unique to each thread in the wave. It is ignored when resource bit ADD_TID_ENABLE == 0 | TID 是波前内每个线程独有的常量, 范围为 0..63。资源位 ADD_TID_ENABLE == 0 时, 忽略 TID。 |

## 原文第 68 页（PDF 第 76 页）：线性缓冲区地址计算

| 英文标注                                                     | 中文含义                           |
|----------------------------------------------------------|--------------------------------|
| Address Calculation for a Linear Buffer / Full equations | 线性缓冲区地址计算 / 完整公式               |
| Offset / Index / ThreadID / Instr. / soffset             | 偏移 / 索引 / 线程编号 / 指令字段 / 标量偏移   |
| ADDRESS / Buffer_Offset                                  | 地址 / 缓冲区偏移。其余公式变量、运算符和字段名保留原文。 |

## 原文第 70 页（PDF 第 78 页）：缓冲区重排示例

| 英文标注                                                                      | 中文含义                                      |
|---------------------------------------------------------------------------|-------------------------------------------|
| Original Buffer / Swizzled Buffer                                         | 原始缓冲区 / 重排后的缓冲区                           |
| Stride = 24 bytes / Element Size = 4 bytes                                | 步长 = 24 字节 / 元素大小 = 4 字节                  |
| const_index_stride = 8 // how many consecutive indices to group together  | const_index_stride = 8: 每组包含多少个连续索引。      |
| const_element_size = 4 bytes // the size of a single element, in bytes    | const_element_size = 4 字节: 单个元素的大小, 以字节计。 |
| which simplifies to...                                                    | 化简为……（公式保持原样）。                            |
| Note that because we are dealing with dwords, offset%4 is always == 0.    | 这里处理的是双字, 因此 offset%4 始终等于 0。             |
| Index_stride span #0 / #1 / #2                                            | 索引步长覆盖区域 #0 / #1 / #2                     |
| If the const_element_size had been 8                                      | 如果 const_element_size 为 8。                |
| An alternate way to visualize Swizzled Buffers / byte address / elem_size | 重排缓冲区的另一种图示 / 字节地址 / 元素大小                 |

## 原文第 73 页（PDF 第 81 页）：内存加载到 LDS

| 英文标注                                | 中文含义                   |
|-------------------------------------|------------------------|
| HW-Alloc / Instr. / bytes-per-dword | 硬件分配 / 指令字段 / 每个双字的字节数 |
| Zero - no vgpr / 4 bytes / Zero     | 零, 不使用 VGPR / 4 字节 / 零 |



| 英文标注                                      | 中文含义                                            |
|-------------------------------------------|-------------------------------------------------|
| LDS_ADDR / MEM_ADDR / TIDinWave / soffset | LDS 地址 / 内存地址 / 波前内线程编号 / 标量偏移。公式中的字段名与位范围保留原文。 |

## 原文第 76 页（PDF 第 84 页）：缓冲区数据格式表

| 英文标注                          | 中文含义                          |
|-------------------------------|-------------------------------|
| Buffer formats / value / enum | 缓冲区格式 / 数值 / 枚举值              |
| DATA_FORMAT / NUM_FORMAT      | 数据格式字段 / 数值格式字段；枚举常量及其数值保留原表。 |

## 原文第 84、85 页（PDF 第 92、93 页）：数据共享示意图

| 英文标注                                                | 中文含义                         |
|-----------------------------------------------------|------------------------------|
| WorkGroup / Work-Item / Private Memory              | 工作组 / 工作项 / 私有内存             |
| Global/Constant Memory / Frame Buffer / Host Memory | 全局/常量内存 / 帧缓冲区 / 主机内存        |
| Host / Compute Unit / Buffers and Images            | 主机 / 计算单元 / 缓冲区和图像           |
| Texture L1 (Per Compute-Unit) / Texture L2 (Global) | 每个计算单元的 L1 纹理缓存 / 全局 L2 纹理缓存 |
| Global Memory / VRAM                                | 全局内存 / 显存                    |

## 编码图中的通用标注

| 英文标注                               | 中文含义                                |
|------------------------------------|-------------------------------------|
| src / dst / sgpr-pair / clmp / imm | 源操作数 / 目标操作数 / SGPR 寄存器对 / 限幅 / 立即数 |
| sgpr / vgpr / src or dst           | SGPR / VGPR / 源或目标                  |
| 其他助记符、字段名、枚举值和位编号                  | 均按规范原样保留；对应字段说明在第 13 章提供中文译文。       |



# AMD Instinct MI300 指令集 架构 参考指南

2025 年 8 月 5 日

## 规范使用协议

本规范协议（“协议”）是 Advanced Micro Devices, Inc.（“AMD”）与作为所附 AMD 规范（“规范”）接收者的“您”之间的法律协议。如果您在为另一方执行工作时访问本规范，则您承认您有权要求该方遵守本协议的条款和条件。如果您以任何方式访问本规范或以其他方式使用本规范或提供有关本规范的反馈（定义如下），则您同意本协议中规定的条款和条件。如果您不同意本协议中规定的条款和条件，则您无权使用本规范；请勿使用、访问本规范或提供有关本规范的反馈。考虑到您对本规范（全部或部分）的使用或访问，且已确认其收到和充分性，您同意如下：

1. 您可以查看本规范，仅供 (a) 作为参考，以帮助您规划和设计您的产品、服务或技术（“产品”），以便按照规范中规定的要求与 AMD 产品进行交互，以及 (b) 向 AMD 提供有关规范中披露的信息的反馈。
2. 除第 1 款明确规定的情况外，本规范的所有权利均由 AMD 保留。本协议不授予您任何 AMD 专利、版权、商标或其他知识产权的任何权利。您不得 (i) 复制本规范的任何部分；(ii) 删除本协议或规范中的任何通知，或 (iii) 将规范的任何部分提供给任何其他人，或者将您在本协议下的权利转让或以其他方式提供。
3. 您同意您不得使用或促使他人使用本协议的内容来 (i) 修改任何现有专利或专利申请或创建任何专利或专利申请的任何延续、部分延续或其他扩展，也不得 (ii) 分析、评估任何专利或专利申请（其中应包括创建或修改任何专利权利要求图表或侵权分析）。
4. 本规范可能包含初步信息、错误或不准确之处，或者可能不包含某些必要信息。此外，AMD 保留随时终止或更改规范及其产品的权利，恕不另行通知。该规范完全“按原样”提供。AMD 不提供任何形式的保证，并否认所有明示、默示和法定保证，包括但不限于适销性、特定用途适用性、非侵权、所有权或这些保证的默示保证作为交易过程或贸易惯例而产生。无论采取何种行动，AMD 均不对任何类型的直接、间接、继发、特殊、附带、惩罚性或惩戒性损害（包括业务损失、信息或数据损失、利润损失、资本损失、商誉损失）承担责任无论是合同、侵权行为（包括疏忽）和严格产品责任还是其他规定，即使已被告知发生此类损害的可能性。
5. 此外，AMD 的产品并非设计、预期、授权或保证用于手术植入体内的系统的组件，或用于支持或维持生命的其他应用，或 AMD 产品的故障可能造成人身伤害、死亡或严重财产或环境损害的任何其他应用。
6. 您没有义务向 AMD 提供与本规范相关的任何建议、意见或反馈（“反馈”）。但是，AMD 可以使用您自愿提供的任何反馈，而不受任何限制、费用或保密义务。因此，如果您确实就本规范的任何版本向 AMD 提供反馈，则您同意 AMD 可以在任何产品中自由使用、复制、许可、分发您的反馈以及以其他方式将您的反馈商业化，并且有权再许可第三方执行同样的操作。此外，您不会向 AMD 提供您可能合理相信的任何反馈 (i) 受到任何第三方的任何专利、版权或其他知识产权索赔或权利的约束；(ii) 受许可条款的约束，该许可条款要求将反馈或任何产品或其他 AMD 知识产权纳入或派生的任何产品或知识产权许可给任何第三方或以其他方式提供给任何第三方。
7. 您应遵守所有适用的美国、欧洲和其他出口法律，包括但不限于美国出口管理条例（“EAR”）、（15 C.F.R. 第 730 至 774 条）和欧盟。2009 年 5 月 5 日理事会条例 (EC) 第 428/2009 号。此外，根据 EAR 第 740.6 条，您特此证明，除非根据美国商务部工业与安全局授予的许可证或根据美国出口管理条例（“EAR”）下的许可证例外情况另有许可，否则您不会 (1) 向该国家/地区的国民出口、再出口或放行您根据本协议收到的 D:1、E:1 或 E:2 组任何受限制的技术、软件或源代码，或 (2) 向 D:1、E:1 或 E:2 组国家出口此类技术或软件的直接产品，前提是此类外国生产的直接产品受到商业管制清单（目前见 EAR 第 774 部分补充 1）中规定的国家安全控制。有关最新的国家/地区组列表，或有关 EAR 的其他信息或您在这些法规下的义务，请参阅美国工业和安全局的网站 <http://www.bis.doc.gov/>。本第 7 节仅适用于规范，不适用于公开发布的任何规范。
8. 如果您是美国政府的一部分，则本规范将附带 FAR 52.227-14 商业计算机软件限制权利条款 (c) (1) 和 (2) 项或 DFARS 技术数据和计算机软件权利条款 (c) (1)(ii) 项中规定的“限制权利” 252.277-7013（如适用）。
9. 本协议受加利福尼亚州法律管辖，不考虑其法律选择原则。涉及该争议的任何争议必须提交至加利福尼亚州圣克拉拉县对此类争议有管辖权的法院，并且您放弃允许争议在其他地方提起诉讼的任何抗辩和权利。如果本协议的任何部分无法执行，则将被视为进行必要的修改以使其

可执行，其余部分继续有效。AMD 未能执行本协议项下授予的任何权利或在发生任何违反本协议的情况下对您采取行动，不应被视为 AMD 放弃后续执行权利或在未来发生违约行为时采取后续行动。本协议是您与 AMD 之间关于本规范的完整协议；只能通过您和 AMD 授权代表签署的书面文件进行更改。

## 免责声明

本文档中提供的信息仅供参考，可能包含技术错误、遗漏和印刷错误。本文包含的信息可能会发生变化，并且可能因多种原因而变得不准确，包括但不限于产品和路线图变化、组件和主板版本变化、新型号和/或产品版本、不同制造商之间的产品差异、软件变化、BIOS 刷新、固件升级等。任何计算机系统都存在无法完全预防或减轻的安全漏洞风险。AMD 不承担更新或以其他方式更正或修改此信息的义务。然而，AMD 保留修改此信息并不时对其内容进行更改的权利，而 AMD 没有义务将此类修订或更改通知任何人。此信息按“原样”提供。AMD 对本文内容不做任何陈述或保证，并且对本信息中可能出现的任何不准确、错误或遗漏不承担任何责任。AMD 特别否认对非侵权、适销性或任何特定用途的适用性的任何默示保证。在任何情况下，AMD 均不对任何人因使用此处包含的任何信息而产生的任何依赖、直接、间接、特殊或其他后果性损害承担责任，即使 AMD 已明确告知发生此类损害的可能性。

AMD、AMD 箭头徽标及其组合是 Advanced Micro Devices, Inc. 的商标。本出版物中使用的其他产品名称仅用于识别目的，可能是其各自公司的商标。

版权所有 © 2024 Advanced Micro Devices, Inc. 保留所有权利。



超微半导体公司  
分段阅读\_第 2485 章  
补1  
[www.amd.com](http://www.amd.com)

# 目录

|                       |      |
|-----------------------|------|
| 前言                    | · 1  |
| 关于本文档                 | · 1  |
| 读者对象                  | · 1  |
| 文档结构                  | · 1  |
| 符号约定                  | · 2  |
| 联系信息                  | · 2  |
| 1. 简介                 | · 3  |
| 1.1. 术语               | · 4  |
| 2. 程序组织               | · 5  |
| 2.1. 计算着色器            | · 5  |
| 2.2. 数据共享             | · 5  |
| 2.2.1. 本地数据共享 (LDS)   | · 6  |
| 2.2.2. 全局波前同步 (GWS)   | · 6  |
| 2.3. 设备内存             | · 6  |
| 3. 核函数状态              | · 8  |
| 3.1. 状态概览             | · 8  |
| 3.2. 程序计数器 (PC)       | · 9  |
| 3.3. EXEC 执行掩码        | · 9  |
| 3.4. 状态寄存器            | · 9  |
| 3.5. 模式寄存器            | · 10 |
| 3.6. GPRs 与 LDS       | · 11 |
| 3.6.1. 越界行为           | · 11 |
| 3.6.2. SGPR 分配与存储     | · 12 |
| 3.6.3. SGPR 对齐        | · 12 |
| 3.6.4. VGPR 分配与对齐     | · 12 |
| 3.6.5. LDS 分配与边界限制    | · 13 |
| 3.7. M0 内存描述符         | · 13 |
| 3.8. SCC: 标量条件码       | · 13 |
| 3.9. 向量比较: VCC 与 VCCZ | · 13 |
| 3.10. 陷阱与异常寄存器        | · 14 |
| 3.10.1. 陷阱状态寄存器       | · 15 |
| 3.11. 内存访问违规          | · 15 |
| 3.12. 硬件 ID 寄存器       | · 16 |
| 3.13. GPR 初始化         | · 17 |
| 4. 程序流程控制             | · 18 |
| 4.1. 程序控制             | · 18 |
| 4.2. 分支               | · 18 |
| 4.3. 工作组              | · 19 |
| 4.4. 数据依赖处理           | · 19 |

|                                    |    |
|------------------------------------|----|
| 4.5. 手动插入等待状态 (NOP)                |    |
| 4.6. 任意分歧控制流                       | 21 |
| 5. 标量 ALU 操作                       | 24 |
| 5.1. SALU 指令格式                     | 24 |
| 5.2. 标量 ALU 操作数                    | 24 |
| 5.3. 标量条件码 (SCC)                   | 26 |
| 5.4. 整数算术指令                        | 26 |
| 5.5. 条件指令                          | 27 |
| 5.6. 比较指令                          | 27 |
| 5.7. 按位运算指令                        | 27 |
| 5.8. 访问指令                          | 29 |
| 6. 向量 ALU 操作                       | 31 |
| 6.1. 微码编码                          | 31 |
| 6.2. 操作数                           | 32 |
| 6.2.1. 指令输入                        | 32 |
| 6.2.2. 指令输出                        | 33 |
| 6.2.3. 越界 GPR                      | 34 |
| 6.3. 指令                            | 34 |
| 6.4. 非规格化数与舍入模式                    | 36 |
| 6.5. ALU 限幅位的使用                    | 37 |
| 6.6. VGPR 索引寻址                     | 37 |
| 6.6.1. 索引寻址指令                      | 37 |
| 6.6.2. VGPR 索引寻址细节                 | 38 |
| 6.7. 打包数据运算                        | 38 |
| 7. 矩阵算术指令                          | 40 |
| 7.1. 矩阵融合乘加 (MFMA)                 | 40 |
| 7.1.1. 符号约定                        | 41 |
| 7.1.2. 稠密 MFMA 指令列表                | 42 |
| 7.1.3. 使用示例                        | 43 |
| 7.1.4. 通用输入输出布局                    | 46 |
| 7.1.5. 数值广播                        | 48 |
| 7.2. BF8 与 FP8 格式及转换               | 50 |
| 7.3. 浮点处理细节与格式                     | 52 |
| 7.4. 稀疏矩阵                          | 52 |
| 7.4.1. 稀疏结构细节                      | 53 |
| 7.5. 依赖处理：所需的独立指令                  | 55 |
| 8. 标量内存操作                          | 59 |
| 8.1. 微码编码                          | 59 |
| 8.2. 操作                            | 60 |
| 8.2.1. S_LOAD_DWORD, S_STORE_DWORD | 60 |
| 8.2.2. 标量原子操作                      | 61 |

|                                        |    |
|----------------------------------------|----|
| 8.2.3. S_DCACHE_INV, S_DCACHE_WB ..... | 61 |
| 8.2.4. S_MEMTIME .....                 | 61 |
| 8.2.5. S_MEMREALTIME .....             | 62 |
| 8.3. 依赖检查 ·                            | 62 |
| 8.4. 对齐与边界检查 ·                         | 62 |
| 9. 向量内存操作 ·                            | 63 |
| 9.1. 向量内存缓冲区指令 ·                       | 63 |
| 9.1.1. 简化缓冲区寻址 ·                       | 64 |
| 9.1.2. 缓冲区指令 ·                         | 64 |
| 9.1.3. VGPR 的使用 ·                      | 65 |
| 9.1.4. 缓冲区数据 ·                         | 66 |
| 9.1.5. 缓冲区寻址 ·                         | 67 |
| 9.1.6. 16 位内存操作 ·                      | 71 |
| 9.1.7. 对齐 ·                            | 71 |
| 9.1.8. 缓冲区资源 ·                         | 71 |
| 9.1.9. 从内存缓冲区加载到 LDS ·                 | 72 |
| 9.1.10. 内存作用域与时间局部性控制 ·                | 73 |
| 9.1.11. 数据格式 ·                         | 75 |
| 9.2. 浮点内存原子操作 ·                        | 76 |
| 9.2.1. 浮点原子操作的舍入 ·                     | 77 |
| 9.2.2. 非规格化数（次正规数）处理 ·                 | 77 |
| 9.2.3. NaN 处理 ·                        | 77 |
| 10. 平坦内存指令 ·                           | 79 |
| 10.1. 平坦内存指令 ·                         | 79 |
| 10.2. 指令 ·                             | 81 |
| 10.2.1. 顺序约束 ·                         | 81 |
| 10.2.2. 重要时序要求 ·                       | 81 |
| 10.3. 寻址 ·                             | 81 |
| 10.3.1. 原子操作 ·                         | 82 |
| 10.4. Global 全局内存 ·                    | 82 |
| 10.5. Scratch 暂存内存 ·                   | 82 |
| 10.6. 数据 ·                             | 83 |
| 10.7. 暂存空间（私有） ·                       | 83 |
| 11. 数据共享操作 ·                           | 84 |
| 11.1. 概览 ·                             | 84 |
| 11.2. 内存层次中的数据流 ·                      | 85 |
| 11.3. LDS 访问 ·                         | 85 |
| 11.3.1. 数据共享存储的索引访问与原子访问 ·             | 85 |
| 11.4. GWS 编程限制 ·                       | 87 |
| 12. 指令 ·                               | 88 |
| 12.1. SOP2 指令 ·                        | 88 |

|                                 |     |
|---------------------------------|-----|
| 12.2. SOPK 指令                   |     |
| 12.3. SOP1 指令                   | 107 |
| 12.4. SOPC 指令                   | 125 |
| 12.5. SOPP 指令                   | 130 |
| 12.5.1. 发送消息                    | 138 |
| 12.6. SMEM 指令                   | 138 |
| 12.7. VOP2 指令                   | 161 |
| 12.7.1. 采用 VOP3 编码的 VOP2 指令     | 178 |
| 12.8. VOP1 指令                   | 178 |
| 12.8.1. 采用 VOP3 编码的 VOP1 指令     | 206 |
| 12.9. VOPC 指令                   | 206 |
| 12.9.1. 采用 VOP3A 编码的 VOPC 指令    | 254 |
| 12.10. VOP3P 指令                 | 254 |
| 12.11. VOP3A & VOP3B 指令         | 285 |
| 12.12. LDS & GWS 指令             | 410 |
| 12.13. MUBUF 指令                 | 450 |
| 12.14. MTBUF 指令                 | 469 |
| 12.15. FLAT、Scratch 与 Global 指令 | 474 |
| 12.15.1. Flat 指令                | 474 |
| 12.15.2. Scratch 指令             | 489 |
| 12.15.3. Global 指令              | 495 |
| 12.16. 指令限制                     | 510 |
| 12.16.1. DPP                    | 510 |
| 12.16.2. SDWA                   | 510 |
| 13. 微码格式                        | 512 |
| 13.1. 标量 ALU 与控制指令格式            | 513 |
| 13.1.1. SOP2                    | 513 |
| 13.1.2. SOPK                    | 515 |
| 13.1.3. SOP1                    | 516 |
| 13.1.4. SOPC                    | 518 |
| 13.1.5. SOPP                    | 520 |
| 13.2. 标量内存格式                    | 520 |
| 13.2.1. SMEM                    | 520 |
| 13.3. 向量 ALU 格式                 | 522 |
| 13.3.1. VOP2                    | 522 |
| 13.3.2. VOP1                    | 524 |
| 13.3.3. VOPC                    | 526 |
| 13.3.4. VOP3A                   | 531 |
| 13.3.5. VOP3B                   | 537 |
| 13.3.6. VOP3P                   | 539 |
| 13.3.7. SDWA                    | 542 |



- 13.3.8. SDWAB ..... 543
  - 13.3.9. DPP ..... 544
- 13.4. LDS 与 GWS 格式 · 545
  - 13.4.1. DS ..... 545
- 13.5. 向量内存缓冲区格式 · 547
  - 13.5.1. MTBUF ..... 547
  - 13.5.2. MUBUF ..... 549
- 13.6. Flat 格式 · 550
  - 13.6.1. FLAT ..... 550
  - 13.6.2. GLOBAL ..... 552
  - 13.6.3. SCRATCH ..... 552

# 前言

## 关于本文档

本文档介绍 AMD CDNA Instinct MI300 设备的执行环境、组织方式和程序状态，详细说明该处理器系列向程序员和编译器开放的原生指令集及微码格式。

本文档规定各类指令及其格式，并说明相关程序状态如何与指令交互。部分指令字段相互依赖，字段的任意组合并非都合法；本文档列出有效的组合。

本文件的主要目的是：

1. 规定语言结构和行为，包括各类指令的文本语法和二进制格式。
2. 提供指令操作参考，帮助编译器开发者充分发挥处理器性能。

## 读者对象

本文档面向应用软件和系统软件开发者，涉及操作系统、编译器、加载器、链接器、设备驱动程序和系统工具。读者应了解相关编程方法，并正在开发计算密集型并行应用程序（流式应用程序）。

## 文档结构

第 1 章概述 AMD CDNA 处理器的硬件与编程环境。第 2 章介绍 CDNA 程序的组织方式。第 3 章介绍需要维护的程序状态。第 4 章介绍程序流程。第 5 章介绍标量 ALU 操作。第 6 章介绍向量 ALU 操作。第 7 章介绍向量矩阵 ALU 操作。第 8 章介绍标量内存操作。第 9 章介绍向量内存操作。第 10 章介绍平坦内存指令。第 11 章介绍数据共享操作。第 12 章介绍指令细节，先按所属微码格式分类，再按字母顺序排列。第 13 章详细规定各类微码格式。

# 符号约定

本文档中使用以下约定：

|         |                                        |
|---------|----------------------------------------|
| 等宽字体    | 文件名、文件路径或代码。                           |
| *       | 代码格式、参数或指令的名称中包含任意数量的字母数字字符。           |
| < >     | 尖括号表示流。                                |
| [1,2)   | 包含最左边的值（在本例中为 1）但不包括最右边的值（在本例中为 2）的范围。 |
| [1,2]   | 包含最左边和最右边值的范围。                         |
| {x   y} | 列出的多个选项之一。在这种情况下，X 或 Y。                |
| 0.0     | 单精度（32 位）浮点值。                          |
| 1011b   | 二进制值，在此示例中为 4 位值。                      |
| 7:4     | 位范围，从位 7 到 4（含）。首先显示高位。                |
| 斜体字或短语  | 首次使用有助于理解流计算的术语或概念。                    |

# 联系信息

有关 AMD 加速并行处理开发的信息，请参阅：<http://developer.amd.com/>

# 第 1 章 简介

AMD CDNA 处理器实现了并行微架构，旨在为通用数据并行应用提供出色的平台。需要高带宽或计算密集型的数据密集型应用程序适合在 AMD CDNA 处理器上运行。

下图为 AMD CDNA 一代系列处理器的框图

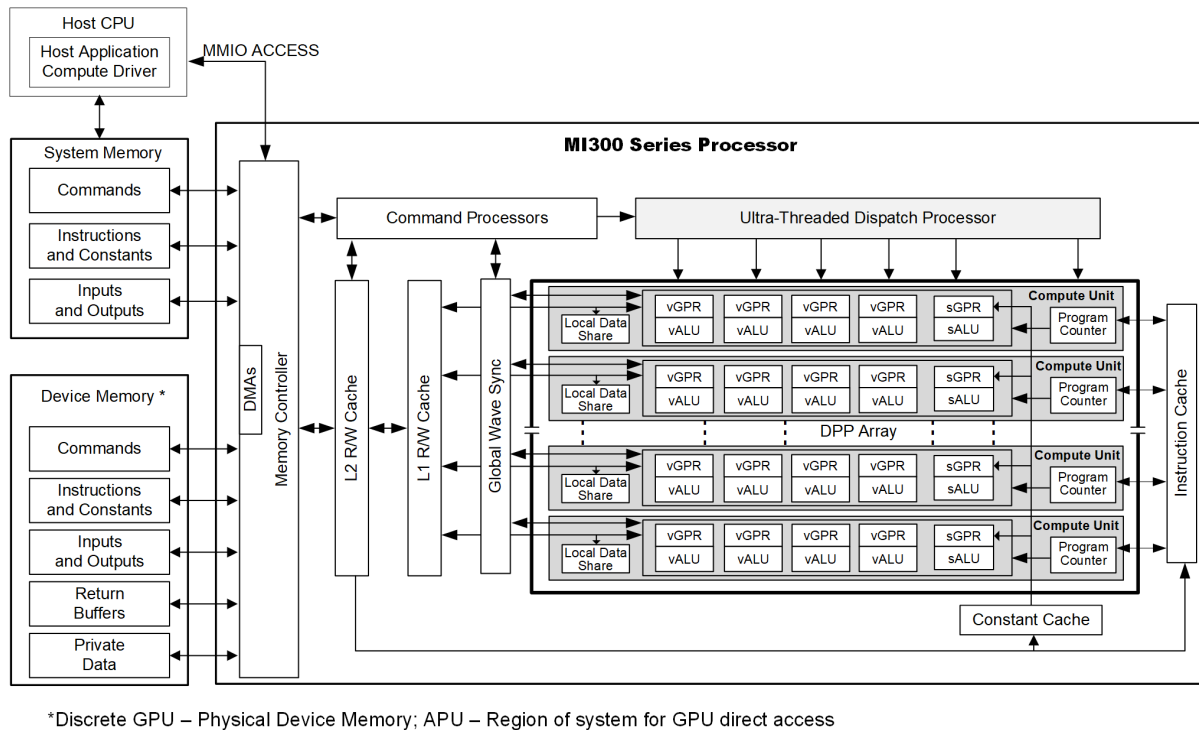


图 1. AMD CDNA 一代系列框图

CDNA 器件包括数据并行处理器(DPP)阵列、命令处理器、内存控制器和其他逻辑(未示出)。CDNA 命令处理器读取主机写入系统内存地址空间中的内存映射 CDNA 寄存器的命令。当命令完成时，命令处理器向主机发送硬件生成的中断。CDNA 内存控制器可以直接访问所有 CDNA 器件内存和主机指定的系统内存区域。为了满足读写请求，内存控制器执行直接内存访问 (DMA) 控制器的功能，包括根据内存中请求的数据格式计算内存地址偏移量。在 CDNA 环境中，一个完整的应用程序包括两部分：

- 在主机处理器上运行的程序，以及
- 名为核函数的程序在 CDNA 处理器上运行。

CDNA 程序由主机命令控制

- 设置 CDNA 内部基址和其他配置寄存器，
- 指定 CDNA 加速器要运行的数据域，
- 使 CDNA 加速器的缓存失效，并执行缓存刷新；
- 使 CDNA 加速器开始执行程序。

CDNA 驱动程序运行在主机上。

DPP 阵列是 CDNA 处理器的核心，由多条相互独立的计算单元流水线组成，并行处理浮点或整数数据流。计算单元流水线可以处理数据，也可以通过内存控制器在处理器与内存之间传输数据。流水线中的计算和向内存写入结果都可以按条件执行。

收到请求后，计算单元流水线从内存加载指令和数据，开始执行，直到核函数结束。核函数运行期间，CDNA 硬件自动将指令从内存取入片上缓存，无需 CDNA 软件参与。CDNA 核函数可以将片外内存中的数据加载到片上通用寄存器（GPRs）和缓存中。

AMD CDNA 设备能够检测浮点异常并产生中断。其中，硬件可检测 IEEE 浮点异常，并记录这些异常，供程序执行后分析。前图从命令处理器指向主机、标作软件中断的路径，表示由硬件产生的中断，用于通知命令完成及相关管理事件。

CDNA 处理器旨在通过跟踪不同执行阶段中潜在的数百个工作项以及通过将计算操作与内存访问操作重叠来隐藏内存延迟。

# 1.1. 术语

表 1. 基本术语

| 术语           | 说明                                                |
|--------------|---------------------------------------------------|
| CDNA 处理器     | 着色器处理器是标量和向量 ALU，能够代表波前运行复杂的程序。                   |
| 派发           | 调度向 CDNA 处理器阵列启动 1D、2D 或 3D 工作网格。                 |
| 工作组          | 工作组是波前的集合，具有快速相互同步的能力；他们还可以通过本地数据共享共享数据。          |
| 波前           | 64 个工作项的集合，在单个 CDNA 处理器上并行执行。                     |
| 工作项          | 工作的单个元素：调度网格中的一个元素，或者图形中的一个像素或顶点。                 |
| 字面量常量        | 放置在指令流中的 32 位整数或浮点常量。                             |
| 标量 ALU（SALU） | 标量 ALU 对每个波前一个值进行操作并管理所有控制流。                      |
| 向量 ALU（VALU） | 向量 ALU 维护每个工作项唯一的向量 GPRs 并执行算术<br>每个工作项上的操作都是唯一的。 |
| 微码格式         | 微码格式描述了用于编码指令的位模式。每条指令要么是 32 位，要么是 64 位。          |
| 指令           | 指令是核函数的基本单元。指令包括：向量 ALU、标量 ALU、内存传输和控制流操作。        |
| 缓冲区资源（V #）   | 缓冲区资源描述符描述内存中的缓冲区：地址、数据格式、步幅等。                    |

## 第 2 章 程序组织

CDNA 核函数是由 CDNA 处理器执行的程序。从编程模型看，每个工作项独立执行核函数；实际执行时，CDNA 处理器把 64 个工作项组成一个波前，在一次执行过程中为这 64 个工作项运行核函数。

CDNA 处理器包括：

- 标量 ALU：每个波前操作一个由所有工作项共用的值。
- 向量 ALU：操作各工作项各自的值。
- 本地数据存储，允许工作组内的工作项进行通信和共享数据。
- 标量内存单元：通过缓存在 SGPRs 与内存之间传输数据。
- 向量内存单元：在 VGPRs 与内存之间传输数据。

核函数的所有控制流均由标量 ALU 指令处理，包括 if/else、分支和循环。标量 ALU (SALU) 指令及标量内存指令作用于整个波前，可操作最多两个 SGPRs，以及字面量常量。

向量内存指令和向量 ALU 指令同时作用于波前中的所有工作项。为支持分支和条件执行，每个波前都有 EXEC 执行掩码，指定当前哪些工作项处于活动状态、哪些处于休眠状态。活动工作项执行向量指令，休眠工作项将该指令视为空操作 (NOP)。标量 ALU 指令可以随时修改 EXEC 掩码。

向量 ALU 指令最多接收三个参数，可来自 VGPRs、SGPRs，或指令流中的字面量常量。指令作用于 EXEC 掩码使能的所有工作项。向量比较指令和带进位输出的加法指令会向 SGPRs 返回掩码，每个工作项对应一位，分别表示比较结果是否为真或是否产生进位。

向量内存指令在 VGPRs 与内存之间传输数据。每个工作项提供各自的内存地址，并提供或接收各自的数据。这些指令也受 EXEC 掩码控制。

### 2.1. 计算着色器

计算核函数（着色器）是在 CDNA 处理器上运行的通用程序：从内存读取数据，处理后将结果写回内存。一次派发使 CDNA 处理器为一维、二维或三维数据网格中的所有工作项执行核函数。CDNA 处理器遍历网格并生成波前，由波前运行计算核函数。初始化时，每个工作项获得自己在网格中的唯一地址（索引），再据此计算待处理数据的地址，并决定如何处理结果。

### 2.2. 数据共享

AMD CDNA 流处理器允许不同工作项共享数据。数据共享能够显著提高性能。下图展示每个工作项可使用的内存层次结构。

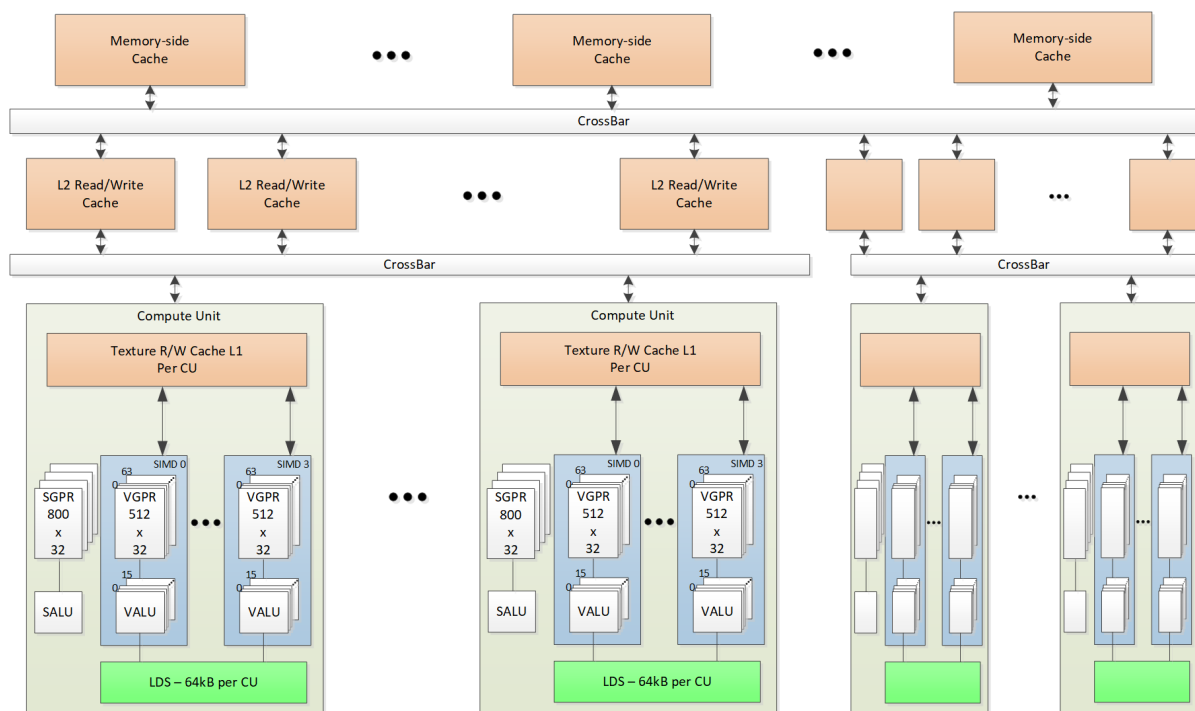


图 2. 共享内存层次结构

## 2.2.1. 本地数据共享 (LDS)

每个计算单元具有 64kB 内存空间，可实现工作组内的工作项或波前内的工作项之间的低延迟通信；这是本地数据共享 (LDS)。该内存配置有 32 个存储体，每个存储体有 512 个 4 字节条目。共享内存包含 32 个整数原子单元，旨在实现快速、无序的原子操作。该内存可用作可预测地重用数据的软件缓存、工作组工作项的数据交换机，或作为一种协作方式来实现对片外内存的高效访问。

## 2.2.2. 全局波前同步 (GWS)

AMD CDNA 器件包含一个全局同步单元 (GWS)，能够跨器件同步工作组。使用此功能的指令有时称为 “GDS”。

## 2.3. 设备内存

AMD CDNA 器件提供了多种从每个计算单元内的处理元件 (PE) 访问片外内存的方法。在主读取路径上，该器件由多个通道的 L2 读写缓存组成，为每个计算单元的 L1 缓存提供数据。特定的无缓存加载指令可以强制在执行加载子句期间从设备内存中检索数据。子句中重叠的加载请求会相互缓存。输出缓存由两级缓存组成：第一级用于写组合缓存（收集分散和存储操作并将它们组合起来以提供良好的内存访问模式）；第二个是具有原子单元的读/写缓存，它允许每个处理元素完成返回初始值的无序原子访问。每个处理元素提供原子操作作用的目标地址、原子操作中要使用的数据以及

读/写原子单元的返回地址，用于将预操作值存储在内存中。每个存储或原子操作都可以设置为在返回值（目的地的前原子操作值）存储到设备内存的写入确认后，向请求的 PE 返回确认。

此确认有两个目的：

- 使 PE 能够在收到写确认确认后通过从其返回地址执行无缓存加载来从原子操作中恢复预操作值，以及
- 使系统能够维持宽松的一致性模型。

从给定 PE 到给定内存通道的每次分散写入都会保持顺序。该确认使一个处理元件能够实现栅栏，以通过确保所有写入在完成后续写入之前已被发布到内存来维持串行一致性。通过这种方式，系统可以在系统上运行的所有并行工作项之间维持宽松的一致性模型。



# 第 3 章 核函数状态

本章描述了着色器程序可见的核函数状态。

## 3.1. 状态概览

下表显示了着色器程序可读或可写的所有硬件状态。

表 2. 可读和可写硬件状态

| 缩写           | 名称         | 大小 (位) | 说明                                         |
|--------------|------------|--------|--------------------------------------------|
| PC           | 程序计数器      | 48     | 指向下一条要执行的着色器指令的内存地址。                       |
| V0-V255      | VGPR       | 32     | 向量通用寄存器 ( “架构 VGPRs” ) 。                   |
| AV0-AV255    | AccVGPR    | 32     | 矩阵累加向量通用寄存器。                               |
| S0-S103      | SGPR       | 32     | 向量通用寄存器。                                   |
| LDS          | 本地数据共享     | 64kB   | 本地数据共享是一款具有内置算术功能的暂存 RAM，允许在工作组中的线程之间共享数据。 |
| EXEC         | 执行掩码       | 64     | 每个线程一位的位掩码，应用于向量指令并控制线程执行和忽略指令。            |
| EXECZ        | EXEC 为零    | 1      | 单个位标志，指示 EXEC 掩码全为零。                       |
| VCC          | 向量条件码      | 64     | 每个线程一位的位掩码；它保存向量比较操作的结果。                   |
| VCCZ         | VCC 为零     | 1      | 单个位标志指示 VCC 掩码全为零。                         |
| SCC          | 标量条件码      | 1      | 标量 ALU 比较指令的结果。                            |
| FLAT_SCRATCH | 平面暂存地址     | 64     | 暂存内存的 64 位基地址，位于 NumSGPRs-5 和 -6 中。只读。     |
| XNACK_MASK   | 地址转换失败。    | 64     | 地址转换失败的线程的位掩码。                             |
| STATUS       | 地位         | 32     | 只读着色器状态位。                                  |
| MODE         | 模式         | 32     | 可写着色器模式位。                                  |
| M0           | 内存寄存器      | 32     | 具有多种用途的临时寄存器，包括 GPR 索引和边界检查。               |
| HW_ID        | 硬件 ID      | 32     | 具有各种波形 ID 状态的只读状态寄存器。                      |
| XCC_ID       | 计算 ID      | 32     | 包含计算设备 ID 的只读状态寄存器。                        |
| TRAPSTS      | 陷阱状态       | 32     | 保存有关异常和待处理陷阱的信息。                           |
| TBA          | 陷阱基地址      | 64     | 保存指向当前陷阱处理程序的指针。                           |
| TMA          | 陷阱内存地址     | 64     | 用于着色器操作的临时寄存器。例如，可以保存指向陷阱处理程序使用的内存的指针。     |
| TTMP0-TTMP15 | 临时陷阱 SGPRs | 32     | 16 SGPRs 仅供陷阱处理程序临时存储。                     |
| VMCNT        | 向量内存指令数    | 6      | 计算已发出但尚未完成的 VMEM 指令的数量。                    |
| EXPCNT       | 出口数量       | 3      | 计算已发出但尚未完成的 GDS 指令的数量。                     |

| 缩写      | 名称              | 大小 (位) | 说明                                           |
|---------|-----------------|--------|----------------------------------------------|
| LGKMCNT | LDS、GDS、常数和消息计数 | 4      | 统计已经发出但尚未完成的 LDS、GDS、常量读取 (标量内存读取) 及消息指令的数量。 |

### 3.2. 程序计数器 (PC)

程序计数器 (PC) 保存下一条待执行指令的字节地址。创建波前时, PC 初始化为程序中第一条指令的地址。

S\_GET\_PC、S\_SET\_PC 和 S\_SWAP\_PC 三条指令用于在 PC 与一对按偶数编号对齐的 SGPR 之间传输数据。

分支目标地址为 (PC of the instruction after the branch + offset)。着色器程序不能直接读写 PC。分支指令、GET\_PC 和 SWAP\_PC 以其下一条指令的 PC 为相对地址基准, 而非当前指令的 PC。S\_TRAP 保存 S\_TRAP 指令自身的 PC。

### 3.3. EXEC 执行掩码

执行掩码 (64 位) 确定执行向量中的哪些线程: 1 = 执行, 0 = 不执行。

标量指令可以读写 EXEC, 向量 ALU 比较指令也可以将比较结果写入 EXEC。该掩码控制向量 ALU、向量内存和 LDS 指令, 不影响标量执行或分支。

当 EXEC 为零时, 辅助位 (EXECZ) 可用作分支跳过代码的条件。



EXEC = 0 时, 该加速器不会自动优化: 着色器硬件仍执行每条指令, 消耗指令发射带宽。当 EXEC 掩码可能为零时, 应使用 CBRANCH 或 VSKIP 快速跳过代码。

### 3.4. 状态寄存器

着色器可以读取状态寄存器字段, 但不能写入。这些位在波前创建时初始化。下表列出并简要描述了状态寄存器字段。当 PRIV=1 时可以写入状态寄存器字段。某些字段是根据着色器指令设置的。

表 3. 状态寄存器字段

| 字段  | 位位置 | 说明                                                      |
|-----|-----|---------------------------------------------------------|
| SCC | 1   | 标量条件码, 可用作进位输出位。对于比较指令, 该位表示比较是否成立; 对于逻辑运算, 结果非零时该位为 1。 |

| 字段            | 位位置 | 说明                                                                                          |
|---------------|-----|---------------------------------------------------------------------------------------------|
| SPI_PRIO      | 2:1 | 创建波前时，由着色器处理器插值器 (SPI) 设置的波前优先级。详情请参见 S_SETPRIO 指令（第 12-49 页）。0 是最低优先级，3 是最高优先级。            |
| WAVE_PRIO     | 4:3 | 波前优先级由着色器程序设置。详情请参见 S_SETPRIO 指令（第 12-49 页）。                                                |
| PRIV          | 5   | 特权模式。仅当处于陷阱处理程序中时才能激活。提供对 TTMP、TMA 和 TBA 寄存器的写访问。                                           |
| TRAP_EN       | 6   | 指示存在陷阱处理程序。当设置为零时，不会捕获陷阱。                                                                   |
| EXECZ         | 9   | 执行掩码为零。                                                                                     |
| VCCZ          | 10  | 向量条件码为零。                                                                                    |
| IN_TG         | 11  | 该波前属于一个包含多个波前的工作组。                                                                          |
| IN_BARRIER    | 12  | 波前正在栅栏处等待。                                                                                  |
| HALT          | 13  | 波前已停止或计划停止。HALT 可由主机通过波前控制消息或着色器设置。在陷阱处理程序中该位被忽略（PRIV = 1）；如果接收到主机启动的陷阱（请求进入陷阱处理程序），它也会被忽略。 |
| TRAP          | 14  | Wavefront 被标记为尽快进入陷阱处理器。                                                                    |
| VALID         | 16  | Wavefront 活动( 已创建但尚未结束) 。                                                                   |
| ECC_ERR       | 17  | 发生了 ECC 错误 。                                                                                |
| PERF_EN       | 19  | 为这个波浪前线启用了性能计数器。                                                                            |
| COND_DBG_USER | 20  | 用户模式的有条件调试指标                                                                                |
| COND_DBG_SYS  | 21  | 系统模式的有条件调试指标 。                                                                              |
| ALLOW_REPLAY  | 22  | 表示 ATC 已启用 。                                                                                |
| FATAL_HALT    | 23  | 显示已发生致命停机 。                                                                                 |
| SCRATCH_EN    | 28  | 1=波前有分配的抓取空间;0=没有。                                                                          |
| IDLE          | 31  | 表示波浪闲置 - 没有未完成的指示 。                                                                         |

### 3.5. 模式寄存器

模式寄存字段可以通过缩略语指令从阴影中读取并写入。下面的表格列出并简要描述了模式寄存字段。

表 4. 模式登记字段

| 字段         | 位位置 | 说明                                                                                                                            |
|------------|-----|-------------------------------------------------------------------------------------------------------------------------------|
| FP_ROUND   | 3:0 | [1:0] 指定单精度舍入模式；[3:2] 指定双精度和半精度舍入模式。0 = 舍入到最近值，中点取偶数；1 = 朝正无穷舍入；2 = 朝负无穷舍入；3 = 朝零舍入。                                          |
| FP_DENORM  | 7:4 | [1:0] 指定单精度非规格化数处理模式；[3:2] 指定双精度和半精度非规格化数处理模式。0 = 输入和输出的非规格化数均置零；1 = 允许非规格化输入，非规格化输出置零；2 = 非规格化输入置零，允许非规格化输出；3 = 允许非规格化输入和输出。 |
| DX10_CLAMP | 8   | 由向量 ALU 强制对 NaNs 进行 DX10 式处理:设定时,夹 NaN 为零;否则,通过 NaN.                                                                          |
| IEEE       | 9   | 支持收集异常标志的浮点操作码按照 IEEE 754-2008，将输入的信号 NaN 转为安静 NaN 并传播。Min_dx10 和 max_dx10 因支持信号 NaN 的传播与安静化而符合 IEEE 754-2008。                |

| 字段           | 位位置   | 说明                                                                                                                                                                                                                                                              |
|--------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DEBUG        | 11    | 强制波前跳到每个指令执行之后的例外处理器(但不是在 ENDPGM 之后)。只有 TRAP_EN = 1。                                                                                                                                                                                                            |
| EXCP_EN      | 18:12 | 异常使能掩码。使能后，若发生相应异常且 TRAP_EN==1，就进入陷阱处理。[12]: invalid, 无效操作; [13]: inputDenormal, 非规格化输入; [14]: float_div0, 浮点除零; [15]: overflow, 上溢; [16]: underflow, 下溢; [17]: inexact, 不精确结果; [18]: int_div0, 整数除零; [19]: 地址监视; [20]: 内存访问违规; [20]: 波前结束时触发陷阱。原文最后两项均标为 [20]。 |
| FP16_OVFL    | 23    | 如果设置，溢出的 FP16 结果将被钳位到 +/- MAX_FP16，无论舍入模式如何，同时仍保留真实的 INF 值。                                                                                                                                                                                                     |
| DISABLE_PERF | 26    | 1 = 禁用此波形的性能计数                                                                                                                                                                                                                                                  |
| GPR_IDX_EN   | 27    | GPR 索引使能。                                                                                                                                                                                                                                                       |
| VSKIP        | 28    | 0 = 正常运行。1 = 跳过（不执行）任何向量指令：value、vmem、lds、gds。“跳过”指令发生在高速状态（每个时钟周期 10 个波前可以跳过一条指令）。这比发出和丢弃指令要快得多。                                                                                                                                                               |
| CSP          | 31:29 | 条件分支堆栈指针。                                                                                                                                                                                                                                                       |

## 3.6. GPRs 与 LDS

本节介绍如何为波前分配 GPR 和 LDS 空间，以及如何处理超出范围和未对齐的访问。

### 3.6.1. 越界行为

本节定义当源或目标 GPR 或内存地址超出波前的合法范围时的行为。

GPR 索引或错误编程可能会导致超出范围。从一种寄存器类型索引到另一种寄存器类型是非法的（例如：SGPRs 到陷阱寄存器或内联常量）。在内联常量中建立索引也是非法的。

下面描述了各种存储类型的超出范围行为。

- SGPRs
  - 源或目标超出范围 = (sgpr < 0 || (sgpr >= sgpr\_size))。源超出范围：返回 SGPR0 的值（不是值 0）。目标超出范围：指令未写入 SGPR 结果。
- VGPRs
  - 与 SGPRs 类似。从 SGPRs 索引到 VGPRs 是非法的，反之亦然。超出范围 = (vgpr < 0 || (vgpr >= vgpr\_size))

- 如果源 VGPR 超出范围，则使用 VGPR0。如果目标 VGPR 超出范围，则该指令将被忽略
- (视为 NOP)。
- LDS
  - 如果 LDS-ADDRESS 超出范围 ( $\text{addr} < 0$  或  $\geq (\text{MIN}(\text{lds\_size}, \text{m0}))$ ):
    - 超出范围的写入将被丢弃；如果 SIZE 不是写入数据大小的倍数，则它是未定义的。读取返回值为零。
  - 如果任何源 VGPR 超出范围，则使用 VGPR0 值。如果 dest-VGPR 超出范围，则使该指令无效 (使用  $\text{exec}=0$  发出)
- 内存、LDS 和 GDS：带返回的读取和原子操作。
  - 如果任何源 VGPR 或 SGPR 超出范围，则数据值未定义。如果任何目标 VGPR 超出范围，则通过发出指令使操作无效，就像 EXEC 掩码被清除为 0 一样。
  - 这个超出范围的检查必须检查所有可以返回的 VGPRs (例如：VDST 到 VDST+3) 对于 BUFFER\_LOAD\_DWORDx4)。目标 VGPRs 超出范围的原子操作无效：已发出，但执行掩码为零。

具有多个目的地的指令 (例如：V\_ADDC)：如果任何目的地超出范围，则不写入任何结果。

### 3.6.2. SGPR 分配与存储

一个波前可以分配 16 到 102 个 SGPRs，以 16 个 GPRs (双字) 为单位。这些在逻辑上被视为 SGPRs 0-101。VCC 作为波前的 SGPRs 的一部分物理存储在编号最高的两个 SGPRs 中 (SGPR 106 和 107；源/目标 VCC 是这两个 SGPRs 的别名)。当存在陷阱处理程序时，在 VCC 之后保留 16 个额外的 SGPRs 以保存陷阱地址以及保存的 PC 和陷阱处理程序温度。这些都是有特权的 (除非设置了特权，否则无法写入)。请注意，如果波前分配 16 个 SGPRs，则 2 个 SGPRs 通常用作 VCC，其余 14 个可供着色器使用。着色器硬件并不妨碍使用所有 16 个 SGPRs。

### 3.6.3. SGPR 对齐

以下情况需要偶对齐 SGPRs。

- 当使用 64 位数据时。这是向/自 64 位寄存器 (包括 PC) 移动所必需的。
- 当标量内存读取地址基数来自 SGPR 对齐 (在 SGPR 中)。

当标量内存读取返回四个或更多双字时，数据 GPR 需要四对齐。当 SGPRs 中存储 64 位数量时，LSB 位于 SGPR[n]，MSB 位于 SGPR[n+1]。

### 3.6.4. VGPR 分配与对齐

VGPRs 以八个双字为一组进行分配。

VGPRs 从两个池中分配：常规 VGPRs 和累积 VGPRs。累加 VGPRs 与矩阵 VALU 指令一起使用，也可以直接从内存加载。一波总共最多可以有 512 个

VGPRs, 每种类型 256 个。当一波总数少于 512 个 VGPRs 时, 每种类型的数量是灵活的 - 不需要两种类型的数量相等。

对 64 位数据进行操作指令必须使用对齐 (即偶数) VGPRs。这适用于 ALU 和内存指令。GWS 指令也必须偶对齐。

### 3.6.5. LDS 分配与边界限制

当工作组未使用时, LDS 按工作组或按波前分配。LDS 空间按 512 字节对齐方式分配给工作组或波前的连续 512 字节块。LDS 分配不会环绕 LDS 存储。对 LDS 的所有访问都仅限于分配给该波前/工作组的空间。

LDS 读取和写入的钳位由两个大小寄存器控制, 其中包含 SPI 分配给该波前或工作组的 LDS 空间大小的值, 以及 LDS 指令中指定的可能较小的值 (大小保存在 M0 中)。LDS 操作使用这两个大小中较小的一个来确定如何钳位读/写地址。

## 3.7. M0 内存描述符

每个波前有 1 个 32 位 M0 寄存器, 可用于:

- 本地数据共享 (LDS)
  - LDS 内存/Vfetch 寻址 → LDS: {16' h0, lds\_offset[15:0]} // 以字节为单位
- 全局波前同步 (GWS)
  - { 基地[5:0], 16' h0 }
- 向量和标量指令的间接 GPR 寻址。M0 是无符号索引。

## 3.8. SCC: 标量条件码

大多数标量 ALU 指令设置标量条件码 (SCC) 位, 指示操作结果。

比较运算: 1 = true 算术运算: 1 = 执行位/逻辑运算: 1 = 结果不为零  
移动: 不改变 SCC

SCC 可用作扩展精度整数算术的进位, 以及条件移动和分支的选择器。

## 3.9. 向量比较: VCC 与 VCCZ

向量 ALU 比较设置向量条件码 (VCC) 寄存器 (1=通过, 0=失败)。此外, 向量比较可以选择将 EXEC 设置为 VCC 值。

还有一个 VCC 汇总位 (vccz)，当 VCC 结果为零时，该位设置为 1。这对于提前退出分支测试很有用。VCC 也被设置用于选定的整数 ALU 操作（进位）。

向量比较可以选择将结果写入 VCC（32 位指令编码）或任何 SGPR（64 位指令编码）。每次更新 VCC 时，VCCZ 都会更新：向量比较和标量写入 VCC。

EXEC 掩码确定哪些线程执行指令。VCC 指示哪些执行线程通过了条件测试，或者哪些线程从整数加法或减法生成了进位。

$$V\_CMP\_* \square VCC[n] = EXEC[n] \& (\text{线程}[n]\text{测试通过})$$

VCC 已全部写入；没有部分掩码更新。



VCC 物理上驻留在 SGPR 寄存器文件中，因此当指令源为 VCC 时，该指令将计入给定指令可源的 SGPRs 总数的限制。VCC 物理上驻留在最高的两个用户 SGPRs 中。

用户/编译器必须防止标量-ALU 写给持有 VCC 的 SGPR，

紧接着是使用 VCCZ 的有条件分支。硬件无法检测到此，并插入所需的等待状态（当 SALU 写到 VCC 时，硬件确实检测到此状态，只有当 SALU 指令引用了恰好持有 VCC 的 SGPRs 时，才未能实现此目标）。

## 3.10. 陷阱与异常寄存器

每种例外都可以通过设置或清除 TRAPSTS 注册的 EXCP\_EN 字段中的位数而独立启用或禁用。本节介绍控制和报告核函数例外的注册。

所有陷阱临时 SGPRs(TTMP\*)都享有写作特权——只有在陷阱处理器(status.priv = 1)中才能写作,当不享有特权时,写作会被忽略并读回零. TMA 和 TBA 只读;它们可以通过 S\_GETREG\_B32 访问.

当设定陷阱(用户启动、例外或主机启动)时,阴影硬件会生成 S\_TRAP 指令。这将陷阱信息加载到一对 SGPRs 中：

$$\{TTMP1, TTMP0\} = \{3'h0, pc\_rewind[3:0], HT[0], trapID[7:0], PC[47:0]\}.$$

HT 被设定为主机启动的陷阱一个,而用户陷阱(s\_trap)或例外为零. TRAP\_ID 是例外的零,或者这些陷阱的用户/主机陷阱 ID. 输入陷阱处理器时,断层指令的 PC 是: (PC - PC\_rewind\*4).

STATUS.TRAP\_EN：向着色器指示是否已安装陷阱处理程序。

未安装陷阱处理程序时，浮点异常、用户发起或主机发起的陷阱均不会进入陷阱处理。安装了陷阱处理程序后，波前额外使用 16 个 SGPRs 进行陷阱处理。若 trap\_en == 0，则忽略所有陷阱与异常，硬件把 s\_trap 转换为 NOP。

MODE. EXCP\_EN[8:0] - 浮点例外可以实现,定义哪些例外和事件会导致陷阱.

| 位 | 例外                              |
|---|---------------------------------|
| 0 | 无效                              |
| 1 | 输入异常                            |
| 2 | 除以零                             |
| 3 | 过量流动                            |
| 4 | 下流量                             |
| 5 | 不准确                             |
| 6 | 整数除以零                           |
| 7 | 地址监视 - TC (L1) 见证了对“感兴趣地址”的线程访问 |

### 3.10.1. 陷阱状态寄存器

陷阱状态寄存器记录以前看到的陷阱或异常。核函数可以对其进行读写。

表 5. 异常字段位

| 字段            | 位     | 说明                                                                                                                                                                                                                 |
|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EXCP          | 8:0   | 发生异常的状态位。这些位是粘性的并会累积结果，直到着色器程序清除它们。无论 EXCP_EN 的设置如何，这些位都会累积。这些可以在没有着色器权限的情况下读取或写入。位异常 0 无效 1 输入非规格化数 2 除以零 3 上溢 4 下溢 5 不精确 6 整数除以零 7 地址监视 8 内存违规                                                                   |
| SAVECTX       | 10    | 由主机命令设置的位，指示该波形必须跳转到其陷阱处理程序并保存其上下文。该位必须由陷阱处理程序使用 S_SETREG 清除。注意 - 着色器可以将此位设置为 1 以引发保存上下文陷阱，并且由于硬件延迟，着色器在捕获陷阱之前可能会执行最多 2 条附加指令。                                                                                     |
| ILLEGAL_INST  | 11    | 检测到非法指令。                                                                                                                                                                                                           |
| ADDR_WATCH1-3 | 14:12 | 表示地址监视 1、2 或 3 已命中。位 12 是地址监视 1；位 13 为 2；位 14 是 3。                                                                                                                                                                 |
| EXCP_CYCLE    | 21:16 | 当发生浮点异常时，这会告诉陷阱处理程序异常发生在哪个周期。0-3 用于正常浮点运算，0-7 用于双浮点加法，0-15 用于双浮点 muladd 或超越运算。该寄存器记录第一次发生启用（未屏蔽）异常的周期数。EXCP_CYCLE[1:0] 阶段：线程 0-15 处于阶段 0，线程 48-63 处于阶段 3。EXCP_CYCLE[3:2] 多时隙通过。EXCP_CYCLE[5:4] 混合通行证：用于以较低速率运行的机器。 |
| DP_RATE       | 31:29 | 确定着色器如何解释 TRAP_STS.cycle。不同的向量着色器处理器 (VSP) 以不同的速率处理指令。                                                                                                                                                             |

## 3.11. 内存访问违规

内存违规报告自：



- LDS 对齐错误。
- 内存读/写/原子对齐错误。
- 地址无效的平面访问（不落在任何孔径中）。
- 写入只读内存地址。
- GWS 操作中止（信号量或屏障未执行）。

不会报告指令或标量数据访问的内存违规。

如果 LDS 地址超出范围，LDS 的内存缓冲区不会返回内存违规，但会屏蔽超出范围的线程的 EXEC 位。

当内存访问违规时，相应的内存（LDS 或 TC）将 MEM\_VIOL 返回到波形。这存储在波形的 TRAPSTS.mem\_viol 位中。该位是粘性的，因此一旦设置为 1，它将保持为 1，直到用户将其清除。

有一个相应的异常使能位（EXCP\_EN.mem\_viol）。如果当内存返回违规时设置该位，则波形跳转到陷阱处理程序。

内存违规并不精确。LDS 或 TC 处理该地址时报告违规；在此期间，wave 可能处理了更多指令。当报告 mem\_viol 时，保存的程序计数器是下一条要执行的指令的程序计数器；它与错误指令无关。

### 3.12. 硬件 ID 寄存器

下面的值表示当前执行 wave 的位置。依赖这些值并不安全，因为它们可能会在波形的生命周期内发生变化。

表 6. 硬件 ID (HW\_ID)

| 字段       | 位     | 说明                     |
|----------|-------|------------------------|
| WAVE_ID  | 3:0   | 波形缓冲器槽位数               |
| SIMD_ID  | 5:4   | CU 内波分配到的 SIMD         |
| PIPE_ID  | 7:6   | 发出波浪的管道                |
| CU_ID    | 11:8  | 波分配给的计算单元              |
| SH_ID    | 12    | 波分配到的着色器阵列（SE 内）。设置为零。 |
| SE_ID    | 15:13 | 波分配给的着色器引擎             |
| TG_ID    | 19:16 | 线程组 ID                 |
| VM_ID    | 23:20 | 虚拟内存 ID                |
| QUEUE_ID | 26:24 | 发送此波的队列                |
| STATE_ID | 29:27 | 州 ID（未使用）              |
| ME_ID    | 31:30 | 微引擎 ID                 |

表 7. XCC ID (XCC\_ID)

| 字段     | 位   | 说明         |
|--------|-----|------------|
| XCC_ID | 3:0 | 该 XCC 的 ID |

### 3.13. GPR 初始化

当计算着色器波启动时，VGPR0 和多个 SGPRs 被初始化。

计算着色器使用工作组中的 X、Y 和 Z 索引初始化 VGPR0：{ 2' b00, Z[9:0], Y[9:0], X[9:0] }。

表 8. CS SGPR 负载

| SGPR 订购    | 说明                                                                                      | 使能够                          |
|------------|-----------------------------------------------------------------------------------------|------------------------------|
| 第 0.. 16 个 | 用户数据寄存器                                                                                 | COMPUTE_PGM_RSRC2.user_sgpr  |
| 然后         | work_group_id0[31:0]                                                                    | COMPUTE_PGM_RSRC2.tgid_x_en  |
| 然后         | work_group_id1[31:0]                                                                    | COMPUTE_PGM_RSRC2.tgid_y_en  |
| 然后         | work_group_id2[31:0]                                                                    | COMPUTE_PGM_RSRC2.tgid_z_en  |
| 然后         | {first_wave, 6' h00, wave_id_in_group[4:0], 2' h0, 14' h0, 工作-group_size_in_waves[5:0]} | COMPUTE_PGM_RSRC2.tg_size_en |
| TTMP4,5    | 0                                                                                       |                              |
| TTMP6      | 调度数据包地址 lo                                                                              |                              |
| TTMP7      | 发送数据包地址 hi                                                                              |                              |
| TTMP8      | 调度网格 X[31:0]                                                                            |                              |
| TTMP9      | 调度网格 Y[31:0]                                                                            |                              |
| TTMP10     | 调度网格 Z[31:0]                                                                            |                              |
| TTMP11     | { 26'b0, wave_id_in_workgroup[5:0] }                                                    |                              |

其他 TTMP 未初始化。

# 第 4 章 程序流程控制

所有程序流程控制均使用标量 ALU 指令进行编程。这包括循环、分支、子例程调用和陷阱。程序使用 SGPRs 来存储分支条件和循环计数器。常量可以从标量常量缓存中直接提取到 SGPRs 中。

## 4.1. 程序控制

下表中的指令控制着色器程序的优先级和终止，并为陷阱处理程序提供支持。

表 9. 控制指令

| 指令              | 说明                                                                |
|-----------------|-------------------------------------------------------------------|
| S_ENDPGM        | 终止波前。它可以出现在核函数中的任何位置，并且可以出现多次。                                    |
| S_ENDPGM_SAVE D | 由于上下文保存而终止波前。它可以出现在核函数中的任何位置，并且可以出现多次。                            |
| S_NOP           | 什么也不做；它可以在硬件中重复最多 16 次。                                           |
| S_TRAP          | 跳转到陷阱处理程序。                                                        |
| S_RFE           | 从陷阱处理程序返回                                                         |
| S_SETPRIO       | 修改此波前的优先级：0=最低，3=最高。                                              |
| S_SLEEP         | 导致波前休眠 64 - 8128 个时钟周期。                                           |
| S_SENDMSG       | 向主机 CPU 发送消息（通常是中断）。                                              |
| S_WAKEUP        | 使工作组中的一个波向同一工作组中的所有其他波发出信号，使其提前从 S_SLEEP 唤醒。如果波浪没有休眠，则它们不受此指令的影响。 |

## 4.2. 分支

分支是使用以下标量 ALU 指令之一完成的。

表 10. 分支指令

| 指令                                    | 说明                                                                     |
|---------------------------------------|------------------------------------------------------------------------|
| S_BRANCH                              | 无条件分支。                                                                 |
| S_CBRANCH_<测试>                        | 有条件的分支。仅当 <test> 为 true 时才分支。测试为 VCCZ、VCCNZ、EXECZ、EXECNZ、SCCZ 和 SCCNZ。 |
| S_CBRANCH_CDBGSYS                     | 如果 COND_DBG_SYS 状态位被置位，则执行条件分支。                                        |
| S_CBRANCH_CDBGUSER                    | 如果 COND_DBG_USER 状态位被置位，则执行条件分支。                                       |
| S_CBRANCH_CDBGSYS_AND_USER            | 条件分支，仅当 COND_DBG_SYS 和 COND_DBG_USER 都设置时才进行。                          |
| S_SETPC                               | 直接从 SGPR 对中设置 PC。                                                      |
| S_SWAPPC                              | 将当前的 PC 与 SGPR 对中的地址交换。                                                |
| S_GETPC                               | 检索当前 PC 值（不会导致分支）。                                                     |
| S_CBRANCH_{G,I}_FORK 和 S_CBRANCH_JOIN | 复杂分支的条件分支。                                                             |
| S_SETVSKIP                            | 设置一个导致所有向量指令被忽略的位。分支的有用替代方案。                                           |

| 指令         | 说明                                                   |
|------------|------------------------------------------------------|
| S_CALL_B64 | 跳转到子程序，并保存返回地址。SGPR_pair = PC+4; PC = PC+4+SIMM16*4。 |

对于条件分支，分支条件可以通过标量或向量运算来确定。标量比较操作设置标量条件码 (SCC)，然后可以将其用作条件分支条件。向量比较操作设置 VCC 掩码，然后 VCCZ 或 VCCNZ 可用于确定分支。

### 4.3. 工作组

工作组是在同一计算单元上运行的波前的集合，可以同步和共享数据。最多可将 16 个波前 (1024 工作项) 组合成一个工作组。当多个波前在一个工作组中时，可以使用 S\_BARRIER 指令强制每个波前等待，直到所有其他波前到达相同的指令；然后，一切波前继续。任何波前都可以使用 S\_ENDPGM 提前终止，并且当剩余的活动波达到其屏障指令时，认为屏障已满足。

### 4.4. 数据依赖处理

着色器硬件解决了大多数数据依赖性，但少数情况必须由着色器程序显式处理。在这些情况下，程序必须插入 S\_WAITCNT 指令以确保先前的操作在继续之前已完成。

着色器具有三个计数器，用于跟踪发出指令的进度。S\_WAITCNT 等待这些计数器的值等于或低于指定值，然后再继续。

这些允许着色器编写器安排长延迟指令，执行不相关的工作，并指定何时需要长延迟操作的结果。

给定类型的指令按顺序返回，但不同类型的指令可以乱序完成。例如，GWS 和 LDS 指令都使用 LGKM\_cnt，但它们可能会乱序返回。

- VM\_CNT：向量内存计数。确定内存读取何时将数据返回到 VGPRs，或者内存写入何时完成。
  - 每次发出向量内存读取或写入 (MUBUF、MTBUF 或 FLAT 格式) 指令时递增。当数据已写回 VGPRs 时，读取递减；当数据写入 L2 缓存时，写入递减。排序：内存读取和写入按发出的顺序返回，包括混合读取和写入。
- LGKM\_CNT：(LDS、GWS、(K)constant、(M)essage) 确定这些低延迟指令之一何时完成。
  - 每发出一个 LDS 或 GWS 指令就加 1，标量内存读取的双字计数也加一 (1 表示 1 个双字加载，2 表示 2 个双字或更大加载)。S\_memtime 与 s\_load\_dwordx2 的计数相同。每发出一个 FLAT 指令就加 1。当数据返回到 VGPRs 时，对于 LDS/GWS 读取或原子返回减 1。
  - 
  -

- 每发行一个 S\_SENDMSG 就加 1。消息发送出去时减 1。当数据写入 LDS/GWS 时，LDS/GWS 写入减 1。从数据缓存 (SMEM) 返回的每个 Dword 减 1。

◦

订购：

- 不同类型的指令返回时是乱序的。相同类型的指令按照发出的顺序返回，标量内存除外

■

读取，这可能会无序返回（在这种情况下，只有 S\_WAITCNT 0 是唯一合法值）。

- EXP\_CNT：VGPR-导出计数。确定数据何时从 VGPR 中读出并发送到 GWS，此时可以安全地覆盖 VGPR 的内容。

- 当从波前缓冲区发出 GWS 指令时递增。当 GWS 指令的最后一个周期被授予并执行（VGPRs 读出）时，GWS 递减。

## 4.5. 手动插入等待状态 (NOP)

硬件不检查以下依赖关系；它们必须通过插入 NOP 或独立指令来解决。

表 11. 所需的软件插入等待状态

| 第一条指令                                                                                                                                                                                                             | 第二条指令                              | 等待 | 说明                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|----|-------------------------------------------------|
| S_SETREG <*>                                                                                                                                                                                                      | S_GETREG <相同的寄存器>                  | 2  |                                                 |
| S_SETREG <*>                                                                                                                                                                                                      | S_SETREG <相同的寄存器>                  | 2  |                                                 |
| SET_VSKIP                                                                                                                                                                                                         | S_GETREG MODE                      | 2  | 从 MODE 读取 VSKIP。                                |
| S_SETREG MODE.vskip                                                                                                                                                                                               | 任意向量运算                             | 2  | 需要两个 nop 或非向量指令。                                |
| 设置 VCC 或 EXEC 的 VALU                                                                                                                                                                                              | 使用 EXECZ 或 VCCZ 作为数据源的 VALU        | 5  |                                                 |
| VALU 写入 SGPR/VCC (readlane, cmp, add/sub, div_scale)                                                                                                                                                              | V_{READ,WRITE}通道使用 SGPR/VCC 作为通道选择 | 4  |                                                 |
| VALU 写入 VCC (包括 v_div_scale)                                                                                                                                                                                      | V_DIV_FMAS                         | 4  |                                                 |
| FLAT_STORE_X3 FLAT_STORE_X4<br>FLAT_ATOMIC_{F}CMP_SWAP_X2 (以及全局和临时存储/原子)<br>BUFFER_STORE_DWORD_X3<br>BUFFER_STORE_DWORD_X4<br>BUFFER_STORE_FORMAT_XYZ<br>BUFFER_STORE_FORMAT_XYZW<br>BUFFER_ATOMIC_{F}CMP_SWAP_X2 | 写入 VGPRs, 保存来自这些指令的写入数据。           | 1  | BUFFER_STORE_* 操作若使用 SGPR 提供 offset, 则无需任何等待状态。 |
| FLAT_STORE_X3 FLAT_STORE_X4 (以及全局和临时存储/原子)<br>FLAT_ATOMIC_{F}CMP_SWAP_X2<br>BUFFER_STORE_DWORD_X3<br>BUFFER_STORE_DWORD_X4<br>BUFFER_STORE_FORMAT_XYZ<br>BUFFER_STORE_FORMAT_XYZW<br>BUFFER_ATOMIC_{F}CMP_SWAP_X2 | VALU 写入 VGPRs, 保存来自这些指令的写入数据。      | 2  | 使用 SGPR 作为“偏移”的 BUFFER_STORE_* 操作不需要任何等待状态。     |

| 第一条指令                                                                                                 | 第二条指令                                                    | 等待 | 说明                                                              |
|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------|----|-----------------------------------------------------------------|
| VALU 写入 SGPR                                                                                          | VMEM 读作 SGPR                                             | 5  | 硬件假设这里不存在依赖性。如果 VALU 写入 VMEM 使用的 SGPR, 则用户必须添加五个等待状态。           |
| SALU 写入 M0                                                                                            | GDS, S_SENDSMSG                                          | 1  |                                                                 |
| VALU 写入 VGPR                                                                                          | VALU DPP 读作 VGPR                                         | 2  |                                                                 |
| VALU 写入 EXEC                                                                                          | VALU DPP 操作                                              | 5  | ALU 不会将 EXEC 转发到 DP P。                                          |
| VCC 的混合使用: alias vs SGPR# v_readlane、v_readfirstlane v_cmp v_add*i/u v_sub* i/u v_div_scale* (写入 vcc) | VALU 将 VCC 读取为常量 (而不是作为 0 等待状态的进位) 。                     | 1  | VCC 可以通过名称或通过保存 VCC 的逻辑 SGPR 来访问。数据依赖性检查逻辑不知道这些是相同的寄存器并且不会防止竞争。 |
| S_SETREG TRAPSTS                                                                                      | RFE, RFE_restore                                         | 1  |                                                                 |
| SALU 写入 M0                                                                                            | LDS "add-TID" 指令, buffer_store LDS_dword, 临时或全局, LDS = 1 | 1  |                                                                 |
| SALU 写入 M0                                                                                            | S_MOVEREL                                                | 1  |                                                                 |
| VALU 写入 SGPR/VCC: v_readlane、v_readfirstlane、v_cmp、v_add*_i/u、v_sub*_i/u、v_div_scale*                 | VALU 读取 SGPR 作为常量                                        | 2  |                                                                 |
|                                                                                                       | VALU 将 SGPR 读取为进位                                        | 0  |                                                                 |
|                                                                                                       | v_readlane、v_writelane 将 SGPR 读为通道-select                | 4  |                                                                 |
| v_cmpx                                                                                                | VALU 读取 EXEC 作为常量                                        | 2  |                                                                 |
|                                                                                                       | V_readlane, v_readfirstlane, v_writelane                 | 4  |                                                                 |
|                                                                                                       | 其他 VALU                                                  | 0  |                                                                 |
| VALU 写入 VGPRn                                                                                         | v_readlane vsrc0 读取 VGPRn                                | 1  |                                                                 |
| VALU op 使用 OPSEL 或 SDWA 更改结果的位位置                                                                      | VALU 运算消耗该运算的结果                                          | 1  |                                                                 |
| VALU 传输操作                                                                                             | 非反式 VALU 操作消耗该操作的结果                                      | 1  |                                                                 |

表 12. 传输操作

|            |           |                  |                  |
|------------|-----------|------------------|------------------|
| V_EXP_F32  | V_LOG_F32 | V_RCP_F32        | V_RCP_IFLAG_F32  |
| V_RSQ_F32  | V_RCP_F64 | V_RSQ_F64        | V_SQRT_F32       |
| V_SQRT_F64 | V_SIN_F32 | V_COS_F32        | V_RCP_F16        |
| V_SQRT_F16 | V_RSQ_F16 | V_LOG_F16        | V_EXP_F16        |
| V_SIN_F16  | V_COS_F16 | V_EXP_LEGACY_F32 | V_LOG_LEGACY_F32 |

4.6. 任意分歧控制流

在 CDNA 架构中，条件分支通过以下方式之一进行处理。

- 1. S\_CBRANCH 这种情况用于简单的控制流，其中采用分支的决定基于先前的比较操作。这是条件分支最常见的方法。
- 2. S\_CBRANCH\_I/G\_FORK 和 S\_CBRANCH\_JOIN 该方法适用于复杂的、不可简化的控制

流程图，将在本节的其余部分进行描述。该方法在简单流量控制上的性能低于 S\_CBRANCH；仅在必要时使用它。

条件分支 (CBR) 图被分组为独立的代码块，在入口点用 FORK 表示，在出口点用 JOIN 表示。着色器编译器必须将这些指令添加到代码中。此方法使用六深堆栈，每个 fork/join 块需要三个 SGPRs。Fork/Join 块可以分层嵌套到任意深度（符合 SGPR 要求）；它们还可以与其他条件流控制或计算跳转共存。

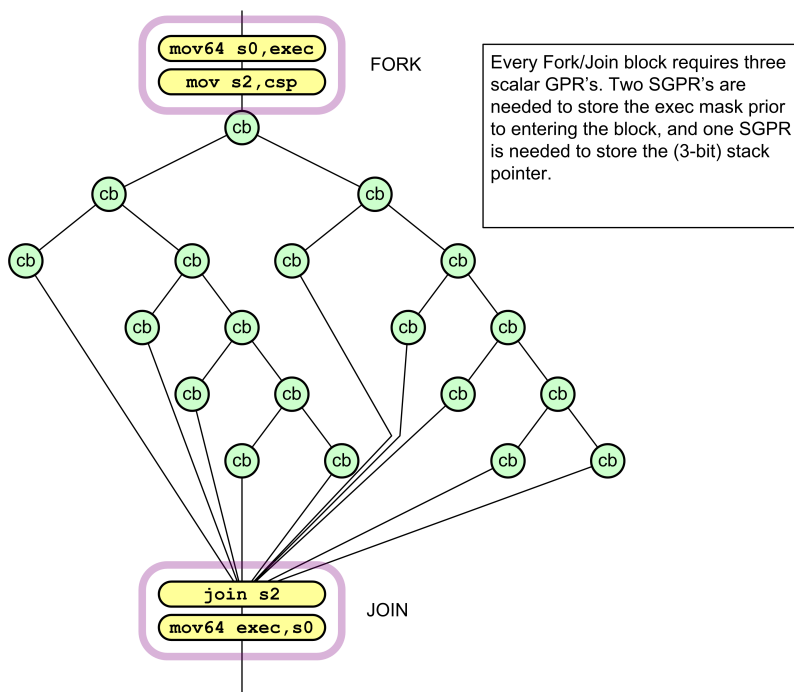


图 3. 复杂控制流图示例

每个波前的寄存器要求是：

- CSP[2:0]——控制堆栈指针。
- 六个堆栈条目，每个 128 位，存储在 SGPRs 中：{ exec[63:0], PC[47:2] }

该方法比较 64 个线程中有多少线程走的是 PASS 路径而不是 FAIL 路径；然后，它首先选择线程数较少的路径。这意味着最多 50% 的线程处于活动状态，这将必要的堆栈深度限制为  $\text{Log}264 = 6$ 。

以下伪代码显示了 CBRANCH Fork 和 Join 操作的详细信息。

```
S_CBRANCH_G_FORK arg0, arg1
    // arg1 是一个 sgpr 对，保存 64 位 (48 位) 目标地址

S_CBRANCH_I_FORK arg0, #target_addr_offset[17:2]
    // target_addr_offset: 16b 有符号立即偏移量

// PC: 在此伪代码中指向 cbranch_*_fork 指令
mask_pass = SGPR[arg0] & exec
mask_fail = ~SGPR[arg0] & exec

if (mask_pass == exec)
```

```

    I_FORK : PC += 4 + target_addr_offset
    G_FORK: PC = SGPR[arg1]
else if (mask_fail == exec)
    PC += 4
else if (bitcount(mask_fail) < bitcount(mask_pass))
    exec = mask_fail
    I_FORK : SGPR[CSP*4] = { (pc + 4 + target_addr_offset), mask_pass }
    G_FORK: SGPR[CSP*4] = { SGPR[arg1], mask_pass }
    CSP++
    PC += 4
else
    exec = mask_pass
    SGPR[CSP*4] = { (pc+4), mask_fail }
    CSP++
    I_FORK : PC += 4 + target_addr_offset
    G_FORK: PC = SGPR[arg1]

```

#### **S\_CBRANCH\_JOIN arg0**

```

if (CSP == SGPR[arg0]) // SGPR[arg0]保存 FORK 启动时的 CSP 值
    PC += 4 // 这是第二次加入: 继续 pgm
else
    CSP -- // 这是第一次 JOIN: 跳转到其他 FORK 路径
    {PC, EXEC} = SGPR[CSP*4] // 从 4 个连续的 SGPRs 读取 128 位

```

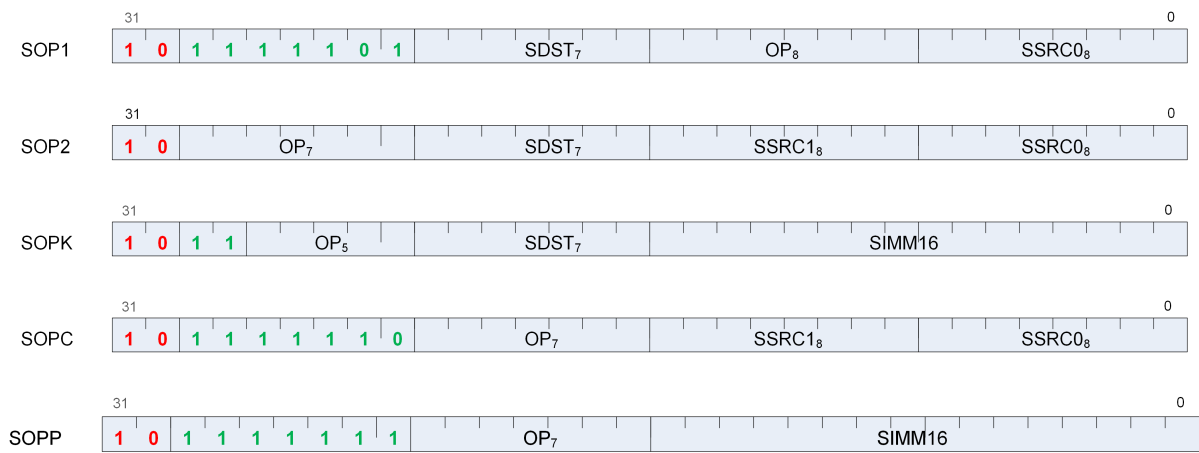


## 第 5 章 标量 ALU 操作

标量 ALU (SALU) 指令对每个波前的单个值进行操作。这些运算由 32 位整数算术和 32 或 64 位按位运算组成。SALU 还可以直接在程序计数器上执行操作，允许程序在 SGPRs 中创建调用堆栈。许多操作还设置标量条件码位 (SCC) 以指示比较结果、进位或指令结果是否为零。

### 5.1. SALU 指令格式

SALU 指令以五种微码格式之一进行编码，如下所示：



这些指令格式中的每一种都使用以下一些字段：

| 字段     | 说明               |
|--------|------------------|
| OP     | 操作码：要执行的指令。      |
| SDST   | 目的地 SGPR。        |
| SSRC0  | 第一个源操作数。         |
| SSRC1  | 第二个源操作数。         |
| SIMM16 | 有符号立即数 16 位整数常量。 |

类似指令的列表有时使用大括号 { } 的压缩形式来表示可能名称的列表。例如，S\_AND\_{B32,B64}定义了两条合法指令：S\_AND\_B32 和 S\_AND\_B64。

### 5.2. 标量 ALU 操作数

SALU 指令的有效操作数为：

- SGPRs，包括临时陷阱 SGPRs。
- 模式寄存器
- 状态寄存器（只读）。
- M0 寄存器。
- TrapSts 寄存器。

- EXEC 面罩。
- VCC 面罩。
- SCC。
- PC。
- 内联常量：从-16 到 64 的整数，以及一些浮点值。
- VCCZ、EXECZ 和 SCC。
- 硬件寄存器。
- 32 位字面量常量。

在下表中，0-127 可以用作标量源或目标； 128-255 只能用作源。

表 13. 标量操作数

|                   | 代码      | 意义              | 说明                       |
|-------------------|---------|-----------------|--------------------------|
| 标量目<br>标 (7<br>位) | 0 - 101 | SGPR 0 至 101    | 标量 GPRs                  |
|                   | 102     | FLAT_SCRATCH_LO | 保存 flat-暂存内存描述符的低位 Dword |
|                   | 103     | FLAT_SCRATCH_HI | 保存 flat-暂存内存描述符的高位 Dword |
|                   | 104     | XNACK_MASK_LO   | 保存 XNACK 掩码的低位 Dword。    |
|                   | 105     | XNACK_MASK_HI   | 保存 XNACK 掩码的高位 Dword。    |
|                   | 106     | VCC_LO          | 保存向量条件码的低位 Dword         |
|                   | 107     | VCC_HI          | 保存向量条件码的高位 Dword         |
|                   | 108-123 | TTMP0 至 TTMP15  | 陷阱温度（特权）                 |
|                   | 124     | M0              | 保存 flat-暂存内存描述符的低位 Dword |
|                   | 125     | 预订的             | 预订的                      |
|                   | 126     | EXEC_LO         | 执行掩码，低双字                 |
|                   | 127     | EXEC_HI         | 执行掩码，高位双字                |
|                   | 128     | 0               | 零                        |
|                   | 129-192 | int 1..64       | 正整数值。                    |
|                   | 193-208 | int -1..-16     | 负整数值。                    |
|                   | 209-234 | 预订的             | 未使用。                     |
|                   | 235     | SHARED_BASE     | 记忆孔径定义。                  |
|                   | 236     | SHARED_LIMIT    |                          |
|                   | 237     | PRIVATE_BASE    |                          |
|                   | 238     | PRIVATE_LIMIT   |                          |
|                   | 239     | 保留              | 保留                       |
|                   | 240     | 0.5             | 单浮子或双浮子                  |
|                   | 241     | -0.5            |                          |
|                   | 242     | 1.0             |                          |
|                   | 243     | -1.0            |                          |
|                   | 244     | 2.0             |                          |
|                   | 245     | -2.0            |                          |
|                   | 246     | 4.0             |                          |
|                   | 247     | -4.0            |                          |
|                   | 248     | 1.0 / (2 * PI)  |                          |
|                   | 249-250 | 预订的             | 未使用                      |
|                   | 251     | VCCZ            | { 零, VCCZ }              |

|  | 代码  | 意义    | 说明              |
|--|-----|-------|-----------------|
|  | 252 | EXECZ | { 零, EXECZ }    |
|  | 253 | SCC   | { 零, SCC }      |
|  | 254 | 预订的   | 未使用             |
|  | 255 | 文字    | 指令流中的常量 32 位常量。 |

SALU 不能使用 VGPRs 或 LDS。SALU 指令可以使用 32 位字面量常量。该常量是指令流的一部分，可用于除 SOPP 和 SOPK 之外的所有 SALU 微码格式。字面量常量通过将源指令字段设置为“literal”（255）来使用，然后使用以下指令双字作为源值。

如果任何源 SGPR 超出范围，则使用 SGPR0 的值。

如果目标 SGPR 超出范围，则不会将结果写入 SGPR。但是，写入了 SCC 和 EXEC（用于 saveexec）。

如果指令使用 SGPRs 中的 64 位数据，则□□ SGPR 对必须与偶数边界对齐。例如，使用 SGPRs 2 和 3 或 8 和 9（但不是 11 和 12）来表示 64 位数据是合法的。

### 5.3. 标量条件码（SCC）

标量条件码 (SCC) 是作为执行大多数 SALU 指令的结果而写入的。

SCC 由许多指令设置：

- 比较操作：1 = true。
- 算术运算：1 = 执行。
  - SCC = 有符号加减运算溢出。对于加法，溢出=两个操作数符号相同，结果的 MSB（符号位）与操作数的符号不同。对于减法 (AB)，溢出 = A 和 B 具有相反的符号，并且结果符号与 A 的符号不同。
- 位/逻辑运算：1 = 结果不为零。

### 5.4. 整数算术指令

本节介绍 SALU 提供的算术运算。下表显示了标量整数算术指令：

表 14. 整数算术指令

| 指令            | 编码   | 设置 SCC? | 操作                                        |
|---------------|------|---------|-------------------------------------------|
| S_ADD_I32     | SOP2 | y       | D = S0 + S1, SCC = overflow.              |
| S_ADD_U32     | SOP2 | y       | D = S0 + S1, SCC = carry out.             |
| S_ADDC_U32    | SOP2 | y       | D = S0 + S1 + SCC = overflow.             |
| S_SUB_I32     | SOP2 | y       | D = S0 - S1, SCC = overflow.              |
| S_SUB_U32     | SOP2 | y       | D = S0 - S1, SCC = carry out.             |
| S_SUBB_U32    | SOP2 | y       | D = S0 - S1 - SCC = carry out.            |
| S_ABSDIFF_I32 | SOP2 | y       | D = abs (s1 - s2), SCC = result not zero. |

| 指令                     | 编码   | 设置 SCC? | 操作                                                                  |
|------------------------|------|---------|---------------------------------------------------------------------|
| S_MIN_I32<br>S_MIN_U32 | SOP2 | y       | D = (S0 < S1) ? S0 : S1. SCC = 1 if S0 was min.                     |
| S_MAX_I32<br>S_MAX_U32 | SOP2 | y       | D = (S0 > S1) ? S0 : S1. SCC = 1 if S0 was max.                     |
| S_MUL_I32              | SOP2 | n       | D = S0 * S1. Low 32 bits of result.                                 |
| S_ADDK_I32             | SOPK | y       | D = D + simm16, SCC = overflow. Sign extended version of simm16.    |
| S_MULK_I32             | SOPK | n       | D = D * simm16. Return low 32bits. Sign extended version of simm16. |
| S_ABS_I32              | SOP1 | y       | D.i = 绝对值 (S0.i)。 SCC=结果不为零。                                        |
| S_SEXT_I32_I8          | SOP1 | n       | D = { 24{S0[7]}, S0[7:0] }.                                         |
| S_SEXT_I32_I16         | SOP1 | n       | D = { 16{S0[15]}, S0[15:0] }.                                       |

5.5. 条件指令

条件指令使用 SCC 标志来确定是否执行操作，或（对于 CSELECT）要使用哪个源操作数。

表 15. 条件指令

| 指令                   | 编码   | 设置 SCC? | 操作                            |
|----------------------|------|---------|-------------------------------|
| S_CSELECT_{B32, B64} | SOP2 | n       | D = SCC ? S0 : S1.            |
| S_CMOVK_I32          | SOPK | n       | if (SCC) D = signext(simm16). |
| S_CMOV_{B32,B64}     | SOP1 | n       | if (SCC) D = S0, else NOP.    |

5.6. 比较指令

这些指令比较两个值，如果比较产生 TRUE 结果，则将 SCC 设置为 1。

表 16. 条件指令

| 指令                                   | 编码   | 设置 SCC? | 操作                                                                         |
|--------------------------------------|------|---------|----------------------------------------------------------------------------|
| S_CMP_EQ_U64, S_CMP_NE_U64           | SOPC | y       | 比较两个 64 位源值。 SCC = S0 <条件> S1。                                             |
| S_CMP_{EQ,NE,GT,GE,LE,LT}_{I32,U32}  | SOPC | y       | 比较两个源值。 SCC = S0 <条件> S1。                                                  |
| S_CMPK_{EQ,NE,GT,GE,LE,LT}_{I32,U32} | SOPK | y       | 将 Dest SGPR 与常量进行比较。 SCC = DST <条件> simm16。 simm16 是零扩展 (U32) 或符号扩展 (I32)。 |
| S_BITCMP0_{B32,B64}                  | SOPC | y       | 测试 “有点为零” 。 SCC = !S0[S1]。                                                 |
| S_BITCMP1_{B32,B64}                  | SOPC | y       | 测试 “有点一” 。 SCC = S0[S1]。                                                   |

5.7. 按位运算指令

按位指令对 32 位或 64 位数据进行操作，而不解释其具有类型。对于下表中注明的按位运算，如果结果非零，则设置 SCC。

表 17. 按位指令

| 指令                                                   | 编码   | 设置 SCC? | 操作                                                                                                                                                                                     |
|------------------------------------------------------|------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S_MOV_{B32,B64}                                      | SOP1 | n       | D = S0                                                                                                                                                                                 |
| S_MOVK_I32                                           | SOPK | n       | D = signext(simm16)                                                                                                                                                                    |
| {S_AND,S_OR,S_XOR}_{B32,B64}                         | SOP2 | y       | D = S0 & S1, S0 OR S1, S0 XOR S1                                                                                                                                                       |
| {S_ANDN2,S_ORN2}_{B32,B64}                           | SOP2 | y       | D = S0 & ~S1, S0 OR ~S1, S0 XOR ~S1,                                                                                                                                                   |
| {S_NAND,S_NOR,S_XNOR}_{B32,B64}                      | SOP2 | y       | D = ~(S0 & S1), ~(S0 OR S1), ~(S0 XOR S1)                                                                                                                                              |
| S_LSHL_{B32,B64}                                     | SOP2 | y       | D = S0 << S1[4:0], [5:0] for B64.                                                                                                                                                      |
| S_LSHR_{B32,B64}                                     | SOP2 | y       | D = S0 >> S1[4:0], [5:0] for B64.                                                                                                                                                      |
| S_ASHR_{I32,I64}                                     | SOP2 | y       | D = sext(S0 >> S1[4:0]) ([5:0] for I64).                                                                                                                                               |
| S_BFM_{B32,B64}                                      | SOP2 | n       | 位域掩码。 D = ((1 << S0[4:0]) - 1) << S1[4:0]。                                                                                                                                             |
| S_BFE_U32、S_BFE_U64<br>S_BFE_I32、S_BFE_I64 (有符号/无符号) | SOP2 | y       | 位字段提取，然后对 I32/64 指令的结果进行符号扩展。 S0 = 数据, S1[5:0] = 偏移量, S1[22:16] = 宽度。                                                                                                                  |
| S_NOT_{B32,B64}                                      | SOP1 | y       | D = ~S0.                                                                                                                                                                               |
| S_WQM_{B32,B64}                                      | SOP1 | y       | D = wholeQuadMode(S0). If any bit in a group of four is set to 1, set the resulting group of four bits all to 1.                                                                       |
| S_QUADMASK_{B32,B64}                                 | SOP1 | y       | D[0] = OR(S0[3:0])、D[1]=OR(S0[7:4]) 等                                                                                                                                                  |
| S_BREV_{B32,B64}                                     | SOP1 | n       | D = S0[0:31] are reverse bits.                                                                                                                                                         |
| S_BCNT0_I32_{B32,B64}                                | SOP1 | y       | D = CountZeroBits(S0).                                                                                                                                                                 |
| S_BCNT1_I32_{B32,B64}                                | SOP1 | y       | D = CountOneBits(S0).                                                                                                                                                                  |
| S_FF0_I32_{B32,B64}                                  | SOP1 | n       | D = Bit position of first zero in S0 starting from LSB. -1 if not found.                                                                                                               |
| S_FF1_I32_{B32,B64}                                  | SOP1 | n       | D = Bit position of first one in S0 starting from LSB. -1 if not found.                                                                                                                |
| S_FLBIT_I32_{B32,B64}                                | SOP1 | n       | 找到最后一点。 D = 从 MSB 开始的第一个 0 之前的零数。如果没有则返回-1。                                                                                                                                            |
| S_FLBIT_I32<br>S_FLBIT_I32_I64                       | SOP1 | n       | 计算一行中（从 MSB 到 LSB）有多少位与符号位相同。如果输入为零或全 1 (-1)，则返回 -1。 32 位伪代码： if (S0 == 0    S0 == -1) D = -1 else D = 0 for (l = 31 .. 0) if (S0[l] == S0[31]) D++ else break 此操作码的行为与 V_FFBH_I32 相同。 |
| S_BITSET0_{B32,B64}                                  | SOP1 | n       | D[S0[4:0], [5:0] 对于 B64] = 0                                                                                                                                                           |
| S_BITSET1_{B32,B64}                                  | SOP1 | n       | D[S0[4:0], [5:0] 对于 B64] = 1                                                                                                                                                           |
| S_{and,or,xor,andn2,orn2,nand,nor,xnor}_SAVEEXEC_B64 | SOP1 | y       | 保存 EXEC 掩码，然后对其应用按位运算。 D = EXEC EXEC = S0 <操作> EXEC SCC = (执行! = 0)                                                                                                                    |
| S_{ANDN{1,2}_WREXEC_B64                              | SOP1 | y       | N1: EXEC, D = ~S0 & EXEC N2: EXEC, D = S0 & ~EXEC D 和 EXEC 得到相同的结果。 SCC = (结果! = 0) 。                                                                                                  |

| 指令                                           | 编码   | 设置 SCC? | 操作                                                                                                  |
|----------------------------------------------|------|---------|-----------------------------------------------------------------------------------------------------|
| S_MOVERELS_{B32,B64}<br>S_MOVERELD_{B32,B64} | SOP1 | n       | 将相对于 M0 中的值移动到 SGPR 中。MOVERELS: D = SGPR[S0+M0] MOVERELD: SGPR[D+M0] = S0 对于 64, 索引必须为偶数。M0 是无符号索引。 |

## 5.8. 访问指令

这些指令访问硬件内部寄存器。

表 18. 硬件内部寄存器

| 指令                 | 编码    | 补2 | 操作                                                                      |
|--------------------|-------|----|-------------------------------------------------------------------------|
| S_GETREG_B32       | SOPK* | n  | 将硬件寄存器读入 D 的 LSB。                                                       |
| S_SETREG_B32       | SOPK* | n  | 将 D 的 LSB 写入硬件寄存器。（注意，D 是源 SGPR。）必须在两个连续的 S_SETREG 之间添加一个 S_NOP 到同一寄存器。 |
| S_SETREG_IMM32_B32 | SOPK* | n  | S_SETREG，其中 32 位数据来自字面量常量（因此这是 64 位指令格式）。                               |

使用上表中的值在指令的 DEST 字段中指定硬件寄存器。DEST 的某些位指定要读/写的寄存器，但其他位指定要读/写寄存器中的哪些位：

`SIMM16 = {size[4:0], offset[4:0], hwRegId[5:0]}`; offset is 0..31, size is 1..32.

表 19. 硬件寄存器值

| 代码     | 登记                  | 说明                                            |
|--------|---------------------|-----------------------------------------------|
| 0      | 预订的                 |                                               |
| 1      | MODE                | R/W.                                          |
| 2      | STATUS              | 只读。                                           |
| 3      | TRAPSTS             | R/W.                                          |
| 4      | HW_ID               | 只读。仅调试。                                       |
| 5      | GPR_ALLOC           | 只读。{sgpr_size、sgpr_base、vgpr_size、vgpr_base}。 |
| 6      | LDS_ALLOC           | 只读。{lds_size, lds_base}。                      |
| 7      | IB_STS              | 只读。{lgkm_cnt、exp_cnt、vm_cnt}。                 |
| 8 - 15 |                     | 预订的。                                          |
| 16     | TBA_LO              | 陷阱基址寄存器[31:0]。                                |
| 17     | TBA_HI              | 陷阱基址寄存器[47:32]。                               |
| 18     | TMA_LO              | 陷阱内存地址寄存器[31:0]。                              |
| 19     | TMA_HI              | 陷阱内存地址寄存器[47:32]。                             |
| 20     | XCC_ID              | 此 wave 正在运行的 XCC 的 ID                         |
| 21     | PERF_SNAPSHOT_DATA  | 随机性能采样数据                                      |
| 22     | PERF_SNAPSHOT_DATA1 | 随机性能采样数据 1                                    |
| 23     | PERF_SNAPSHOT_PC_LO | 随机性能采样程序计数器                                   |
| 24     | PERF_SNAPSHOT_PC_HI | 随机性能采样程序计数器                                   |

表 20. IB\_STS

| 代码       | 登记         | 说明                           |
|----------|------------|------------------------------|
| VM_CNT   | 23:22, 3:0 | 已发出但尚未返回的 VMEM 指令数。          |
| EXP_CNT  | 6:4        | 已发出但尚未从 VGPRs 读取数据的 GDS 数量。  |
| LGKM_CNT | 11:8       | LDS、GDS，已发出但未完成的常量内存和消息指令计数。 |

表 21. GPR\_ALLOC

| 代码        | 登记    | 说明                                               |
|-----------|-------|--------------------------------------------------|
| VGPR_BASE | 5:0   | 第一个 VGPR 分配给这个波前的物理地址，如[7:2]                     |
| VGPR_SIZE | 11:6  | 分配给该波前的 VGPRs 的编号，如[7:2]。 0=4 VGPRs、1=8 VGPRs 等  |
| ACCV_OFF  | 17:12 | 累加 VGPR 相对于 VGPR_BASE 的偏移量，以 4 VGPRs 为单位。        |
| SGPR_BASE | 23:18 | 第一个 SGPR 的物理地址分配给这个波前，如[8:3]。                    |
| SGPR_SIZE | 27:24 | 分配给该波的 SGPRs 的编号，如[7:4]。 0=16 SGPRs、1=32 SGPRs 等 |

表 22. LDS\_ALLOC

| 代码       | 登记    | 说明                                  |
|----------|-------|-------------------------------------|
| LDS_BASE | 7:0   | 分配给该波前的第一个 LDS 位置的物理地址，以 64 个双字为单位。 |
| LDS_SIZE | 20:12 | 分配给此波前的 LDS 空间量，以 64 个 Dword 为单位。   |

## 第 6 章 向量 ALU 操作

向量 ALU 指令 (VALU) 对 64 个线程中的每一个线程的数据执行算术或逻辑运算，并将结果写回 VGPRs、SGPRs 或 EXEC 掩码。

### 6.1. 微码编码

大多数 VALU 指令有两种编码：VOP3 使用 64 位指令并具有全部功能，以及提供一组受限功能的三种 32 位编码之一。少数指令仅在 VOP3 编码中可用。

当一条指令有两种微码格式时，由用户决定使用哪一种。建议尽可能使用 32 位编码。

微码编码如下所示。

VOP2 适用于具有两个输入和一个向量目标的指令。具有进位的指令隐式地将进位写入 VCC 寄存器。



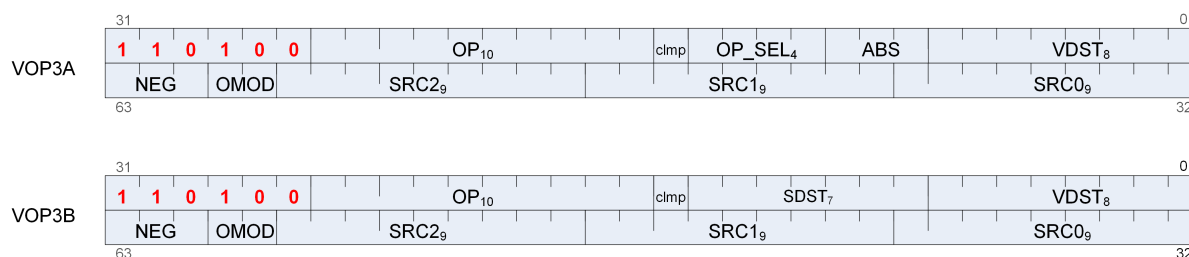
VOP1 适用于没有输入或单个输入和一个目标的指令。



VOPC 为比较指令。



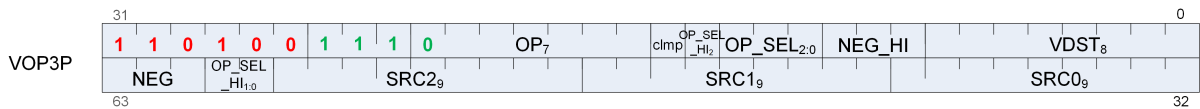
VOP3 适用于具有最多三个输入、输入修饰符（负值和绝对值）和输出修饰符的指令。VOP3 有两种形式：一种使用标量目标字段（仅用于 div\_scale，整数加减）；另一种使用标量目标字段（仅用于 div\_scale，整数加减）；这被指定为 VOP3b。所有其他指令均使用通用形式，指定为 VOP3a。



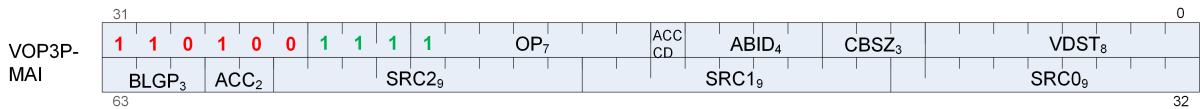
任何 32 位微码格式都可以使用 32 位字面量常量，但不能使用 VOP3。

VOP3P 适用于使用“打包数学”的指令：它们对打包到每个操作数的高 16 位和低 16 位中的一对输入值执行运算；两个 16 位结果作为两个打包值写入单个 VGPR。





VOP3P-MAI 是 VOP3P 格式的变体，用于与矩阵算术指令 (MAI) 一起使用。



## 6.2. 操作数

所有 VALU 指令至少接受一个输入操作数 (V\_NOP 和 V\_CLREXCP 除外)。操作数的数据大小在指令名称中明确定义。例如，V\_MUL\_F32 对 32 位浮点数据进行操作。

### 6.2.1. 指令输入

VALU 指令可以使用以下任意输入源，但须遵守下列限制：

- VGPRs.
- SGPRs.
- 内联常量 - 由特定 VSRC 值选择的常量。
- 字面量常量 - 指令流中的 32 位值。当字面量常量与 64 位指令一起使用时，文字会通过以下方式扩展为 64 位：对于浮点数用零填充 LSB，对于无符号整数用零填充 MSB，以及通过符号扩展有符号整数。
- M0.
- EXEC 面罩。

#### 局限性

- 每条指令最多可读取一个 SGPR，但该值可用于多个操作数。
- 最多可以使用一个字面量常量，并且仅当 SGPR 或 M0 不用作源时。

#### 常量的限制

VALU “ADDC”、“SUBB”和 CNDMASK 都隐式使用 SGPR 值 (VCC)，因此这些指令不能使用额外的 SGPR 或字面量常量。

使用 VOP3 形式并使用浮点输入的指令可以选择对任何输入操作数应用绝对值 (ABS 字段) 或求反 (NEG 字段)。

#### SDWA 和 OPSEL 的限制

DOT 指令不得使用 SDWA 或 OPSEL。使用 SDWA 或 OPSEL 的 VALU 操作不得在下一个 VALU 指令中消耗该指令的结果 - 它们之间必须至少有一个独立指令或 V\_NOP。

6.2.1.1. 字面扩展至 64 位

字面量常量是 32 位，但它们可以用作通常需要 64 位数据的源：

- 64 位浮点数：低 32 位用零填充。
- 64 位无符号整数：零扩展到 64 位
- 64 位有符号整数：符号扩展至 64 位

6.2.2. 指令输出

VALU 指令通常将其结果写入微码字的 VDST 字段中指定的 VGPRs。仅当 EXEC 掩码中的相关位设置为 1 时，线程才会写入结果。

所有 V\_CMPX 指令将其比较结果（每个线程一位）写入 SGPR（或 VCC）和 EXEC 掩码。

当用于 VOP2 形式时，产生进位（整数加法和减法）的指令将其结果写入 VCC，当用于 VOP3 形式时，将其结果写入任意 SGPR 对。

当使用 VOP3 形式时，具有浮点结果的指令可以应用将结果乘以 0.5、1.0、2.0 或 4.0 的输出修饰符（OMOD 字段）。或者，可以将结果钳位（CLAMP 字段）到范围 [0.0, +1.0]。

输出修饰符仅适用于浮点结果，对于整数或位结果将被忽略。输出修饰符与输出非规格化数不兼容：如果启用输出非规格化数，则输出修饰符将被忽略。如果输出非规格化数被禁用，则应用输出修改器并将非规格化数刷新为零。输出修饰符与 IEEE 不兼容：-0 刷新为 +0。如果 IEEE 模式位设置为 1，则忽略输出修饰符。

下表中，向量源为 9 位时可以使用所有代码；如果是八位，则代码 0 到 255 可以作为标量源；如果是七位，则代码 0 到 127 可以作为标量源；代码 256 至 511 可以是向量源或目的地。

表 23. 指令操作数

| 值       | 名称              | 说明            |
|---------|-----------------|---------------|
| 0-101   | SGPR            | 0 .. 101      |
| 102     | FLAT_SCRATCH_LO | 平划痕[31:0]。    |
| 103     | FLAT_SCRATCH_HI | 平划痕[63:32]。   |
| 104     | XNACK_MASK_LO   |               |
| 105     | XNACK_MASK_HI   |               |
| 106     | VCC_LO          | vcc[31:0]。    |
| 107     | VCC_HI          | vcc[63:32]。   |
| 108-123 | TTMP0 至 TTMP 15 | 陷阱处理程序临时（特权）。 |
| 124     | M0              |               |
| 125     | 预订的             |               |
| 126     | EXEC_LO         | 执行[31:0]。     |
| 127     | EXEC_HI         | 执行[63:32]。    |
| 128     | 0               |               |

| 值       | 名称            | 说明                                                                                                       |
|---------|---------------|----------------------------------------------------------------------------------------------------------|
| 129-192 | 整数 1..64      | 整数内联常量。                                                                                                  |
| 193-208 | int -1..-16   |                                                                                                          |
| 209-234 | 预订的           | 未使用。                                                                                                     |
| 235     | SHARED_BASE   | 记忆孔径定义。                                                                                                  |
| 236     | SHARED_LIMIT  |                                                                                                          |
| 237     | PRIVATE_BASE  |                                                                                                          |
| 238     | PRIVATE_LIMIT |                                                                                                          |
| 239     | 保留            | 保留                                                                                                       |
| 240     | 0.5           | 单精度、双精度或半精度内联浮点数。<br><br>1/(2*PI) 为 0.15915494。使用的精确值是： 半： 0x3118<br>单： 0x3e22f983 双： 0x3fc45f306dc9c882 |
| 241     | -0.5          |                                                                                                          |
| 242     | 1.0           |                                                                                                          |
| 243     | -1.0          |                                                                                                          |
| 244     | 2.0           |                                                                                                          |
| 245     | -2.0          |                                                                                                          |
| 246     | 4.0           |                                                                                                          |
| 247     | -4.0          |                                                                                                          |
| 248     | 1/(2*PI)      |                                                                                                          |
| 249     | SDWA          | 子双字地址（仅作为 Source-0 有效）                                                                                   |
| 250     | DPP           | DPP 超过 16 个通道（仅作为 Source-0 有效）                                                                           |
| 251     | VCCZ          | { 零, VCCZ }                                                                                              |
| 252     | EXECZ         | { 零, EXECZ }                                                                                             |
| 253     | SCC           | { 零, SCC }                                                                                               |
| 254     | 保留            | 保留                                                                                                       |
| 255     | 文字            | 指令流中的常量 32 位常量。                                                                                          |
| 256-511 | VGPR          | 0 .. 255                                                                                                 |

6.2.3. 越界 GPR

当源 VGPR 超出范围时，指令使用 VGPR0 的值作为输入。

当目标 GPR 超出范围时，指令执行但不写入结果。

6.3. 指令

下表按微码编码列出了完整的 VALU 指令集，但后面部分列出的 VOP3P 指令除外。

表 24. VALU 指令集

| VOP3            | VOP3 - 2 个操作数        | VOP2          | VOP1              |
|-----------------|----------------------|---------------|-------------------|
| V_ADD3_U32      | V_ADD_F64            | V_ADDC_C0_U32 | V_ACCVGPR_MOV_B32 |
| V_ADD_LSHL_U32  | V_ADD_I16            | V_ADD_C0_U32  | V_BFREV_B32       |
| V_ALIGNBIT_B32  | V_ADD_I32            | V_ADD_F16     | V_CEIL_F16        |
| V_ALIGNBYTE_B32 | V_ASHRREV_I64        | V_ADD_F32     | V_CEIL_F32        |
| V_AND_OR_B32    | V_BCNT_U32_B32       | V_ADD_U16     | V_CEIL_F64        |
| V_BFE_I32       | V_BFM_B32            | V_ADD_U32     | V_CLREXCP         |
| V_BFE_U32       | V_CVT_PKACCUM_U8_F32 | V_AND_B32     | V_COS_F16         |

| VOP3                   | VOP3 - 2 个操作数        | VOP2             | VOP1                |
|------------------------|----------------------|------------------|---------------------|
| V_BFI_B32              | V_CVT_PKNORM_I16_F16 | V_ASHRREV_I16    | V_C0S_F32           |
| V_CUBEID_F32           | V_CVT_PKNORM_I16_F32 | V_ASHRREV_I32    | V_CVT_F16_F32       |
| V_CUBEMA_F32           | V_CVT_PKNORM_U16_F16 | V_CNDMASK_B32    | V_CVT_F16_I16       |
| V_CUBESC_F32           | V_CVT_PKNORM_U16_F32 | V_DOT2C_F32_F16  | V_CVT_F16_U16       |
| V_CUBETC_F32           | V_CVT_PKRTZ_F16_F32  | V_DOT2C_I32_I16  | V_CVT_F32_BF8       |
| V_CVT_PK_U8_F32        | V_CVT_PK_BF8_F32     | V_DOT4C_I32_I8   | V_CVT_F32_F16       |
| V_DIV_FIXUP_F16        | V_CVT_PK_FP8_F32     | V_DOT8C_I32_I4   | V_CVT_F32_F64       |
| V_DIV_FIXUP_F32        | V_CVT_PK_I16_I32     | V_FMAAK_F32      | V_CVT_F32_FP8       |
| V_DIV_FIXUP_F64        | V_CVT_PK_U16_U32     | V_FMAC_F32       | V_CVT_F32_I32       |
| V_DIV_FIXUP_LEGACY_F16 | V_CVT_SR_BF8_F32     | V_FMAC_F64       | V_CVT_F32_U32       |
| V_DIV_FMAS_F32         | V_CVT_SR_FP8_F32     | V_FMAMK_F32      | V_CVT_F32_UBYTE0    |
| V_DIV_FMAS_F64         | V_LDEXP_F32          | V_LDEXP_F16      | V_CVT_F32_UBYTE1    |
| V_DIV_SCALE_F32        | V_LDEXP_F64          | V_LSHLREV_B16    | V_CVT_F32_UBYTE2    |
| V_DIV_SCALE_F64        | V_LSHLREV_B64        | V_LSHLREV_B32    | V_CVT_F32_UBYTE3    |
| V_FMA_F16              | V_LSHRREV_B64        | V_LSHRREV_B16    | V_CVT_F64_F32       |
| V_FMA_F32              | V_MAX_F64            | V_LSHRREV_B32    | V_CVT_F64_I32       |
| V_FMA_F64              | V_MBCNT_HI_U32_B32   | V_MAC_F16        | V_CVT_F64_U32       |
| V_FMA_LEGACY_F16       | V_MBCNT_LO_U32_B32   | V_MADAK_F16      | V_CVT_FLR_I32_F32   |
| V_LERP_U8              | V_MIN_F64            | V_MADMK_F16      | V_CVT_I16_F16       |
| V_LSHL_ADD_U32         | V_MUL_F64            | V_MAX_F16        | V_CVT_I32_F32       |
| V_LSHL_ADD_U64         | V_MUL_HI_I32         | V_MAX_F32        | V_CVT_I32_F64       |
| V_LSHL_OR_B32          | V_MUL_HI_U32         | V_MAX_I16        | V_CVT_NORM_I16_F16  |
| V_MAD_F16              | V_MUL_LEGACY_F32     | V_MAX_I32        | V_CVT_NORM_U16_F16  |
| V_MAD_I16              | V_MUL_LO_U32         | V_MAX_U16        | V_CVT_OFF_F32_I4    |
| V_MAD_I32_I16          | V_PACK_B32_F16       | V_MAX_U32        | V_CVT_PK_F32_BF8    |
| V_MAD_I32_I24          | V_READLANE_B32       | V_MIN_F16        | V_CVT_PK_F32_FP8    |
| V_MAD_I64_I32          | V_SUB_I16            | V_MIN_F32        | V_CVT_RPI_I32_F32   |
| V_MAD_LEGACY_F16       | V_SUB_I32            | V_MIN_I16        | V_CVT_U16_F16       |
| V_MAD_LEGACY_I16       | V_TRIG_PREOP_F64     | V_MIN_I32        | V_CVT_U32_F32       |
| V_MAD_LEGACY_U16       | V_WRITELANE_B32      | V_MIN_U16        | V_CVT_U32_F64       |
| V_MAD_U16              |                      | V_MIN_U32        | V_EXP_F16           |
| V_MAD_U32_U16          |                      | V_MUL_F16        | V_EXP_F32           |
| V_MAD_U32_U24          |                      | V_MUL_F32        | V_FFBH_I32          |
| V_MAD_U64_U32          |                      | V_MUL_HI_I32_I24 | V_FFBH_U32          |
| V_MAX3_F16             |                      | V_MUL_HI_U32_U24 | V_FFBL_B32          |
| V_MAX3_F32             |                      | V_MUL_I32_I24    | V_FLOOR_F16         |
| V_MAX3_I16             |                      | V_MUL_LO_U16     | V_FLOOR_F32         |
| V_MAX3_I32             |                      | V_MUL_U32_U24    | V_FLOOR_F64         |
| V_MAX3_U16             |                      | V_OR_B32         | V_FRACT_F16         |
| V_MAX3_U32             |                      | V_PK_FMAC_F16    | V_FRACT_F32         |
| V_MED3_F16             |                      | V_SUBBREV_CO_U32 | V_FRACT_F64         |
| V_MED3_F32             |                      | V_SUBB_CO_U32    | V_FREXP_EXP_I16_F16 |
| V_MED3_I16             |                      | V_SUBREV_CO_U32  | V_FREXP_EXP_I32_F32 |
| V_MED3_I32             |                      | V_SUBREV_F16     | V_FREXP_EXP_I32_F64 |
| V_MED3_U16             |                      | V_SUBREV_F32     | V_FREXP_MANT_F16    |
| V_MED3_U32             |                      | V_SUBREV_U16     | V_FREXP_MANT_F32    |
| V_MIN3_F16             |                      | V_SUBREV_U32     | V_FREXP_MANT_F64    |
| V_MIN3_F32             |                      | V_SUB_CO_U32     | V_LOG_F16           |
| V_MIN3_I16             |                      | V_SUB_F16        | V_LOG_F32           |
| V_MIN3_I32             |                      | V_SUB_F32        | V_MOV_B32           |
| V_MIN3_U16             |                      | V_SUB_U16        | V_MOV_B64           |
| V_MIN3_U32             |                      | V_SUB_U32        | V_NOP               |
| V_MQSD_PK_U16_U8       |                      | V_XNOR_B32       | V_NOT_B32           |

| VOP3             | VOP3 - 2 个操作数 | VOP2      | VOP1                |
|------------------|---------------|-----------|---------------------|
| V_MQSAD_U32_U8   |               | V_XOR_B32 | V_RCP_F16           |
| V_MSAD_U8        |               |           | V_RCP_F32           |
| V_OR3_B32        |               |           | V_RCP_F64           |
| V_PERM_B32       |               |           | V_RCP_IFLAG_F32     |
| V_QSAD_PK_U16_U8 |               |           | V_READFIRSTLANE_B32 |
| V_SAD_HI_U8      |               |           | V_RNDNE_F16         |
| V_SAD_U16        |               |           | V_RNDNE_F32         |
| V_SAD_U32        |               |           | V_RNDNE_F64         |
| V_SAD_U8         |               |           | V_RSQ_F16           |
| V_XAD_U32        |               |           | V_RSQ_F32           |
|                  |               |           | V_RSQ_F64           |
|                  |               |           | V_SAT_PK_U8_I16     |
|                  |               |           | V_SIN_F16           |
|                  |               |           | V_SIN_F32           |
|                  |               |           | V_SQRT_F16          |
|                  |               |           | V_SQRT_F32          |
|                  |               |           | V_SQRT_F64          |
|                  |               |           | V_SWAP_B32          |
|                  |               |           | V_TRUNC_F16         |
|                  |               |           | V_TRUNC_F32         |
|                  |               |           | V_TRUNC_F64         |

下表列出了比较指令。

表 25. VALU 指令集

| Op           | 格式                           | 功能                                                                                 | 结果          |
|--------------|------------------------------|------------------------------------------------------------------------------------|-------------|
| V_CMP        | I16, I32, I64, U16, U32, U64 | F, LT, EQ, LE, GT, LG, GE, T                                                       | 写 VCC..     |
| V_CMPX       |                              |                                                                                    | 写入 VCC 并执行。 |
| V_CMP        | F16, F32, F64                | F、LT、EQ、LE、GT、LG、GE、T、O、U、NGE、NLG、NGT、NLE、NE Q、NLT (o = 全序, u = 无序, N = NaN 或正常比较) | 写 VCC。      |
| V_CMPX       |                              |                                                                                    | 写入 VCC 并执行。 |
| V_CMP_CLASS  | F16, F32, F64                | 测试以下之一：信令 -NaN、安静 -NaN、正或负：无穷大、正常、次正规数、零。                                          | 写 VCC。      |
| V_CMPX_CLASS |                              |                                                                                    | 写入 VCC 并执行。 |

## 6.4. 非规格化数与舍入模式

着色器程序可以明确控制所应用的舍入模式以及非规格化输入和结果的处理。MODE 寄存器使用 S\_SETREG 指令设置；它有单独的位来控制单精度和双精度浮点数的行为。

注意：对浮点数据进行操作的 V\_DOT2 指令不支持非规格化数和舍入模式。它们刷新输入和输出范数。

表 26. 圆形和非规格化数模式

| 字段        | 位位置 | 说明                                                                                                                  |
|-----------|-----|---------------------------------------------------------------------------------------------------------------------|
| FP_ROUND  | 3:0 | [1:0] 单精度舍入模式。 [3:2] 双精度/半精度舍入模式。 舍入模式： 0=最接近的偶数； 1=+无穷大； 2= -无穷大， 3= 趋于零。                                          |
| FP_DENORM | 7:4 | [5:4] 单精度非正常模式。 [7:6] 双精度/半精度非正常模式。 脱正常模式:0 = Flush 输入和输出解规范。 1 = 允许输入解规范。 2 = Flush 输入解规范,允许输出解规范。 3 = 允许输入和输出解规范。 |

## 6.5. ALU 限幅位的使用

当使用 V\_CMP 指令时,将夹位设置为 1 表示如果出现浮点例外时的比较信号。对于整数操作,它会将结果夹到最大和最小的可代表值。对于浮点操作,它会将结果夹到范围:[0.0,1.0] 。

## 6.6. VGPR 索引寻址

VGPR 索引允许存储在 M0 寄存器中的值作为 VGPRs 索引,用于 VALU 指令中的来源或目的地寄存器。

### 6.6.1. 索引寻址指令

下表描述了启用、禁用和控制 VGPR 索引的指令。

表 27. VGPR 索引指示

| 指令                 | 编码   | 设置 SCC? | 操作                                                                                          |
|--------------------|------|---------|---------------------------------------------------------------------------------------------|
| S_SET_GPR_IDX_OFF  | SOPP | N       | 禁用 VGPR 索引模式。 设置: 模式. gpr_idx_en = 0 。                                                      |
| S_SET_GPR_IDX_ON   | SOPC | N       | 启用 VGPR 索引,并从 SGPR. mode.gpr_idx_en = 1 M0 [7:0] = S0. u [7:0] M0 [15:12] = SIMM4 设置索引值和模式。 |
| S_SET_GPR_IDX_IDX  | SOP1 | N       | 设置 VGPR 指数值: M0[7:0] = S0.u[7:0]                                                            |
| S_SET_GPR_IDX_MODE | SOPP | N       | 更改 VGPR 索引模式,该模式存储于 M0[15:12]. M0[15:12] = SIMM4                                            |

索引的启用和禁用在 MODE 寄存器中 : gpr\_idx\_en 。 当启用时, M0 的两个字段用于确定索引值及其适用于 :

- M0[7:0]持有无符号的索引值,添加到选定的来源或目的地 VGPR 地址.
- M0[15:12]持有一个四位面罩,显示该索引应用到哪个来源或目的地.
  - M0[15] = dest\_enable.
  - M0[14] = src2\_enable.

- M0[13] = src1\_enable.
- M0[12] = src0\_enable.

索引只用于 VGPR 源头和目的地,而不适用于内置常数或 SGPRs. 索引试图处理超出范围的 VGPRs 是非法的.

6.6.2. VGPR 索引寻址细节

本节介绍如何将 VGPR 索引应用于以不同寻常的方式使用来源和目的地寄存器的指令。下表显示 M0 位点控制这些特定指令的来源和目的地寄存器的索引。

| 指令                 | 微码编码                    | VALU 接收器                  | M0[15]<br>(dst) | M0[15]<br>(s2) | M0[15]<br>(s1) | M0[12]<br>(s0) |
|--------------------|-------------------------|---------------------------|-----------------|----------------|----------------|----------------|
| v_readlane         | sdst = src0, SS1        |                           | x               | x              | x              | src0           |
| v_readfirstlane    | sdst = 函数(src0)         |                           | x               | x              | x              | src0           |
| v_writelane        | dst = func(ss0, ss1)    |                           | dst             | x              | x              | x              |
| v_mac_*            | dst = src0 * src1 + dst | 疯狂: dst, src0, src1, src2 | dst, s2         | x              | src1           | src0           |
| v_madak            | dst = src0 * src1 + imm | 疯狂: dst, src0, src1, src2 | dst             | x              | src1           | src0           |
| v_madmkn           | dst = S0 * imm + src1   | 疯狂: dst, src0, src1, src2 | dst             | src2           | x              | src0           |
| v_*sh*_rev         | dst = S1 << S0          | <移位> (src1, src0)         | dst             | x              | src1           | src0           |
| v_cvt_pkaccum      | 使用 dst 作为 src2          |                           | dst, s2         | x              | src1           | src0           |
| SDWA (目标保留, 子双字掩码) | 使用 dst 作为 src2 进行读-模-写  |                           |                 | dst, s2        |                |                |

其中: src= 向量源 SS =  
标量源 dst = 向量目标  
sdst = 标量目标

6.7. 打包数据运算

CDNA 支持压缩数学, 它对 Dword 中的两个 16 位值执行运算, 就好像它们是单独的元素一样。例如, V0=V1+V2 的压缩加法实际上是两个单独的加法: 将每个 Dword 的低 16 位相加并将结果存储在 V0 的低 16 位中, 然后将高半部分相加。

打包数学使用以下指令和微码格式 “VOP3P”。此格式为低操作数和高操作数添加了 op\_sel 和负字段, 并删除了 ABS 和 OMOD。

打包的数学操作码:

|                  |                  |                  |              |
|------------------|------------------|------------------|--------------|
| V_PK_MAD_I16     | V_PK_MUL_LO_U16  | V_PK_ADD_I16     | V_PK_SUB_I16 |
| V_PK_LSHLREV_B16 | V_PK_LSHRREV_B16 | V_PK_ASHRREV_I16 | V_PK_MAX_I16 |
| V_PK_MIN_I16     | V_PK_MAD_U16     | V_PK_ADD_U16     | V_PK_SUB_U16 |
| V_PK_MAX_U16     | V_PK_MIN_U16     | V_PK_FMA_F16     | V_PK_ADD_F16 |
| V_PK_MUL_F16     | V_PK_MIN_F16     | V_PK_MAX_F16     |              |
| V_MAD_MIX_F32    | V_MAD_MIXLO_F16  | V_MAD_MIXHI_F16  |              |

|              |              |              |              |
|--------------|--------------|--------------|--------------|
| V_PK_FMA_F32 | V_PK_MUL_F32 | V_PK_ADD_F32 | V_PK_MOV_B32 |
|--------------|--------------|--------------|--------------|



V\_MAD\_MIX\_\* 不是压缩数学，而是对 16 位和 32 位输入的混合执行单个乘加运算。乘加作为 FMA 融合乘加执行。此处列出它们是因为它们使用 VOP3P 编码。



打包 32 位指令一次对 2 个双字进行操作，并且这些操作数必须是双字对齐的（即偶数 VGPR 地址）。这些指令不支持输出修饰符。OPSEL 和 OPSEL\_HI 用于为每个源选择第一个或第二个 DWORD。



## 第 7 章 矩阵算术指令

Matrix 核心是 CDNA 架构着色器指令集的扩展，支持机器智能 SIMD。矩阵核心有自己的 VGPR 文件：累加（“Acc”）GPRs。这与原始 SIMD 中的正常（建筑或“拱形”）VGPRs 是分开的。着色器 I/O 只能使用两种类型的 VGPRs。指令有一个 ACC 位来指示数据是否传输至架构或累加 VGPRs 或从 VGPRs 传输。数据可以通过 V\_ACCVGPR\_READ 和 V\_ACCVGPR\_WRITE 指令在 ACC 和 ARCH VGPRs 之间移动。

矩阵核心内部实现的核心运算是  $4 \times 1$  乘以  $1 \times 4$  外部矩阵乘积，产生 16 个输出值。外积既可以在密集输入上执行，也可以在 2,4 稀疏输入上执行（其中每组四个值中有两个为零）。矩阵核心单元使用这些操作的并行和串行组合来实现分段矩阵融合乘加 (MFMA) 中描述的稠密矩阵融合乘加 (MFMA) 指令及其稀疏矩阵分段中描述的 2,4 稀疏变体。

由于这些矩阵指令不会在单个周期内产生输出，并且由于它们的部分写入结果可能是可观察到的，因此有时必须在发出矩阵核心指令和访问其结果或修改保存其输入的寄存器之间存在一定数量的独立指令，如“依赖项解析：所需的独立指令”小节中所述。

其他信息可以在 GPUOpen 博客上找到：<https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-README/> 这篇博客文章与 CDNA2 相关，但可能有助于理解 CDNA3。

AMD 矩阵指令计算器 ([https://github.com/RadeonOpenCompute/amd\\_matrix\\_instruction\\_calculator](https://github.com/RadeonOpenCompute/amd_matrix_instruction_calculator)) 包含一个帮助工具，允许开发人员查看有关 CDNA 架构中的 MFMA 指令的详细信息。它允许用户查询指令级信息，例如计算吞吐量和寄存器使用情况。它还允许用户为每个 MFMA 指令及其修饰符生成矩阵元素和硬件寄存器之间的映射。

### 7.1. 矩阵融合乘加 (MFMA)

矩阵融合乘加 (MFMA) 指令使用矩阵核心来执行一个或多个矩阵乘法。请注意，执行这些指令的矩阵核心单元将  $4 \times 1 \times 1 \times 4$  外积作为其基本计算原语，因此 MFMA 指令实现类似外积的操作。

这些指令的名称均采用 V\_MFMA\_ [输出类型]\_ [M]X[N]X[K]\_ [B]B\_ [输入类型] 形式，其中 B（如果未指定则为 1）是相乘的矩阵（或块）的数量，M、N 和 K 是每个块的乘法维度。例如，指令 V\_MFMA\_F32\_32x32x1\_2B\_F32 执行以下操作

```
D[0, :, :] = C[0, :, :] + A[0, :, :] * B[0, :, :]
D[1, :, :] = C[1, :, :] + A[1, :, :] * B[1, :, :]
```

其中 D[b, :, :] 和 C[b, :, :] 是 32 位浮点的  $32 \times 32$  矩阵，A[b, :, :] 是浮点的  $32 \times 1$  矩阵，

B[b,:,:) 是 1 × 32 浮点数矩阵。

MFMA 的输入和输出值可以存储在标准架构向量寄存器 (VGPRs) 或累加 VGPRs (AccVGPR) 中，这是矩阵核心单元专用的附加寄存器。保存矩阵 A 和 B 的寄存器的寄存器文件分别由 MFMA 指令的 ACC 字段的低位和高位控制 (0 表示 VGPRs, 1 表示 AccVGPR)，而 ACC\_CD 位确定 C 和 D 矩阵是否存储在 VGPRs (0) 或 AccVGPR (1) 中。可以使用 V\_ACCVGPR\_\* 指令将数据移入或移出 AGPRs。

请注意，保存 MFMA 指令的输入或输出数据的寄存器必须是连续的，并且第一个寄存器必须与输入或输出所需的寄存器数量对齐。例如，如果指令需要矩阵 A 的四个输入寄存器，则可以使用寄存器 4 到 7（通过将 SRC0 设置为 4），但不能使用寄存器 5 到 8。

7.1.1. 符号约定

当索引具有多个块的值时，M[b,i,j] 是值 M 的块 b、行 i 和列 j 中的值，其中矩阵是零索引的。

在描述 MFMA 操作的输入和输出时，它们被写为矩阵，每列代表波前中的不同通道，每行代表通道具有的不同寄存器（或逻辑项）。

例如，考虑以下一对矩阵 A

|           |           |
|-----------|-----------|
| A[0,0,0]  | A[0,0,1]  |
| A[0,1,0]  | A[0,1,1]  |
| ...       | ...       |
| A[0,31,0] | A[0,31,1] |
| ----      | ----      |
| A[1,0,0]  | A[0,0,1]  |
| ...       | ...       |
| A[1,31,0] | A[1,31,1] |

当该值写为：

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32    | ... | 通道 63     |
|------|----------|----------|-----|-----------|----------|-----|-----------|
| 注册 0 | A[0,0,0] | A[0,1,0] | ... | A[0,31,0] | A[1,0,0] | ... | A[1,31,0] |
| 注册 1 | A[0,0,1] | A[0,1,1] | ... | A[0,31,1] | A[1,0,1] | ... | A[1,31,1] |

这意味着波前的每个通道在两个连续的寄存器中保存两个值，它们是 A 的一个块的一行的两个值，前 32 个通道保存与块 0 不同的行，第二个 32 通道保存块 1 的连续行。

该规范在其存储布局中写入矩阵。

当显示寄存器布局时，本规范假设第一个寄存器为 0。

除非另有说明，否则除法运算符向下舍入（占用发言权）。

7.1.2. 稠密 MFMA 指令列表

表 28. MFMA VALU 操作码：

| 指令                     | 变体         | 块数 | 周期 | 说明                           |
|------------------------|------------|----|----|------------------------------|
| V_MFMA_F32_{*}_F32     | 32x32x1_2B | 2  | 64 | 矩阵乘法，使用 FMA 与 F32 A 和 B 矩阵。  |
|                        | 16x16x1_4B | 4  | 32 |                              |
|                        | 4x4x1_16B  | 16 | 8  |                              |
|                        | 32x32x2    | 1  | 64 |                              |
|                        | 16x16x4    | 1  | 32 |                              |
| V_MFMA_F32_{*}_F16     | 32x32x4_2B | 2  | 64 | 矩阵乘法，使用 FMA 与 F16 A 和 B 矩阵。  |
|                        | 16x16x4_4B | 4  | 32 |                              |
|                        | 4x4x4_16B  | 16 | 8  |                              |
|                        | 32x32x8    | 1  | 32 |                              |
|                        | 16x16x16   | 1  | 16 |                              |
| V_MFMA_F32_{*}_BF16    | 32x32x4_2B | 2  | 64 | 矩阵乘法，使用 FMA 与 BF16 A 和 B 矩阵。 |
|                        | 16x16x4_4B | 4  | 32 |                              |
|                        | 4x4x4_16B  | 16 | 8  |                              |
|                        | 32x32x8    | 1  | 32 |                              |
|                        | 16x16x16   | 1  | 16 |                              |
| V_MFMA_I32_{*}_I8      | 32x32x4_2B | 2  | 64 | 矩阵乘法，使用 FMA 和 I8 A & B 矩阵    |
|                        | 16x16x4_4B | 4  | 32 |                              |
|                        | 4x4x4_16B  | 16 | 8  |                              |
|                        | 32x32x16   | 1  | 32 |                              |
|                        | 16x16x32   | 1  | 16 |                              |
| V_MFMA_F32_{*}_XF32    | 16x16x8    | 1  | 16 | 对 F32 数据进行矩阵乘法，降低乘法精度。       |
|                        | 32x32x4    | 1  | 32 |                              |
| V_MFMA_F64_{*}_F64     | 16x16x4    | 1  | 32 | 对 F64 数据进行矩阵乘法。              |
|                        | 4x4x4_4B   | 4  | 16 |                              |
| V_MFMA_F32_{*}_BF8_BF8 | 16x16x32   | 1  | 16 | 对 FP8 或 BF8 数据进行矩阵乘法。        |
| V_MFMA_F32_{*}_BF8_FP8 |            |    |    |                              |
| V_MFMA_F32_{*}_FP8_BF8 | 32x32x16   | 1  | 32 |                              |
| V_MFMA_F32_{*}_FP8_FP8 |            |    |    |                              |

| 控制   | 行为                                                                                                                              |
|------|---------------------------------------------------------------------------------------------------------------------------------|
| 范数控制 | 忽略来自 MODE 的 Denorm 控制并保持输入/输出 Denorm。                                                                                           |
| 夹钳   | 支持。使用 MODE 中的 FP16_OVFL 位<br><br>如果设置，F32 溢出结果将被钳制为 +/- MAX，否则溢出结果将标准化为 +/-INF。<br><br>如果设置，I32 结果在上溢/下溢时被钳位至 +/-MAX，否则进位位将被丢弃。 |
| 回合模式 | 忽略 MODE 的回合模式并强制其为 RNE。                                                                                                         |
| 例外情况 | 不支持                                                                                                                             |
| 执行掩码 | 忽略 MODE 的执行掩码并强制所有线程将其设置为 1                                                                                                     |
| 来源   | 如果 VGPR 需要偶对齐则为 Src0/1/VDST。<br><br>src0/1 只能是 VGPR，SRC2 可以是内联/常量                                                               |

**XF32**

XF32 指令采用 32 位浮点数，但将尾数舍入为 10 位，以便执行降低精度的乘法

**7.1.3. 使用示例**

**7.1.3.1. V\_MFMA\_F32\_32X32X1\_2B\_F32**

第一个示例显示了 MFMA 的用法，以便建立对这些指令的一般语义的直觉。

假设用户想要将  $32 \times 1$  矩阵  $A[b, :, :]$  与  $1 \times 32$  矩阵  $B[b, :, :]$  进行两次矩阵乘法，将结果累加到  $32 \times 32$  矩阵  $D[b, :, :]$ 。

A 的输入寄存器存储 A 跨连续通道的列（即，第 i 坐标移动最快），并具有以下形式

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32    | ... | 通道 63     |
|------|----------|----------|-----|-----------|----------|-----|-----------|
| 注册 0 | A[0,0,0] | A[0,1,0] | ... | A[0,31,0] | A[1,0,0] | ... | A[1,31,0] |

即，通道 l 保存该值

`A[1 / 32, 1 % 32, 0]`

B 的布局以与 A 布局存储其列相同的方式保存 B 的行。也就是说，B 是用通道 l 保存的

`B[1 / 32, 0, 1 % 32]`

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32    | ... | 通道 63     |
|------|----------|----------|-----|-----------|----------|-----|-----------|
| 注册 0 | B[0,0,0] | B[0,0,1] | ... | B[0,0,31] | B[1,0,0] | ... | B[1,0,31] |

输出布局的核心组件是  $4 \times N$ （此处 N 为 32）值块。（使用  $4 \times N$  块，而不是更简单的布局，是矩阵核心内部结构的结果）。尽可能多的这些块（这里是其中 2 个）被打包到每组寄存器的通道中，先按行，然后按块。

即 D 的布局（以及 C 对应的布局）为：

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32     | ... | 通道 63      |
|------|----------|----------|-----|-----------|-----------|-----|------------|
| 注册 0 | D[0,0,0] | D[0,0,1] | ... | D[0,0,31] | D[0,4,0]  | ... | D[0,4,31]  |
| 注册 1 | D[0,1,0] | D[0,1,1] | ... | D[0,1,31] | D[0,5,0]  | ... | D[0,5,31]  |
| ...  | ...      | ...      | ... | ...       | ...       | ... | ...        |
| 注册 3 | D[0,3,0] | D[0,3,1] | ... | D[0,3,31] | D[0,7,0]  | ... | D[0,7,31]  |
| 注册 4 | D[0,8,0] | D[0,8,1] | ... | D[0,8,31] | D[0,12,0] | ... | D[0,12,31] |
| ...  | ...      | ...      | ... | ...       | ...       | ... | ...        |

|       | 通道 0      | 通道 1      | ... | 通道 31      | 通道 32     | ... | 通道 63      |
|-------|-----------|-----------|-----|------------|-----------|-----|------------|
| 注册 15 | D[0,27,0] | D[0,27,1] | ... | D[0,27,31] | D[0,31,0] | ... | D[0,31,31] |
| 注册 16 | D[1,0,0]  | D[1,0,1]  | ... | D[1,0,31]  | D[1,4,0]  | ... | D[1,4,31]  |
| ...   | ...       | ...       | ... | ...        | ...       | ... | ...        |
| 注册 31 | D[1,27,0] | D[1,27,1] | ... | D[1,27,31] | D[1,31,0] | ... | D[1,31,31] |

换句话说，输出值 D[b, i, j] 位于通道

$$l = j + 32 * ((i/4) \% 2)$$

输出寄存器的

$$r = 16b + 4(i / 8) + (i \% 4)$$

为了产生这些结果，广播字段（CBSZ、ABID 和 BLGP）必须全部设置为 0。这些字段的用法如广播值小节所示。

### 7.1.3.2. V\_MFMA\_F32\_32X32X2\_F32

作为另一个示例，请考虑指令 V\_MFMA\_F32\_32X32X2\_F32。对于该指令，只有一个块被相乘，矩阵 A[0,::] 和 B[0,::] 分别为 32 × 2 和 2 × 32。

该指令为 A 和 B 各取一个输入寄存器，其格式与上述相同，只是通道 32-63 包含 A 的第二列（B 的行）而不是第二块。输出布局与上面相同，只是只有 1 个块，因此只有 16 个输出寄存器。更具体地说，V\_MFMA\_F32\_32X32X2\_F32 的输入和输出布局是

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32    | ... | 通道 63     |
|------|----------|----------|-----|-----------|----------|-----|-----------|
| 注册 0 | A[0,0,0] | A[0,1,0] | ... | A[0,31,0] | A[0,0,1] | ... | A[0,31,1] |

|      | 通道 0     | 通道 1     | ... | 通道 31     | 通道 32    | ... | 通道 63     |
|------|----------|----------|-----|-----------|----------|-----|-----------|
| 注册 0 | B[0,0,0] | B[0,0,1] | ... | B[0,0,31] | B[0,1,0] | ... | B[0,1,31] |

|       | 通道 0      | 通道 1      | ... | 通道 31      | 通道 32     | ... | 通道 63      |
|-------|-----------|-----------|-----|------------|-----------|-----|------------|
| 注册 0  | D[0,0,0]  | D[0,0,1]  | ... | D[0,0,31]  | D[0,4,0]  | ... | D[0,4,31]  |
| 注册 1  | D[0,1,0]  | D[0,1,1]  | ... | D[0,1,31]  | D[0,5,0]  | ... | D[0,5,31]  |
| ...   | ...       | ...       | ... | ...        | ...       | ... | ...        |
| 注册 3  | D[0,3,0]  | D[0,3,1]  | ... | D[0,3,31]  | D[0,7,0]  | ... | D[0,7,31]  |
| 注册 4  | D[0,8,0]  | D[0,8,1]  | ... | D[0,8,31]  | D[0,12,0] | ... | D[0,12,31] |
| ...   | ...       | ...       | ... | ...        | ...       | ... | ...        |
| 注册 15 | D[0,27,0] | D[0,27,1] | ... | D[0,27,31] | D[0,31,0] | ... | D[0,31,31] |

### 7.1.3.3. V\_MFMA\_F32\_4X4X4\_16B\_F16

此示例演示了如何将非 32 位长的值打包到寄存器中，以及在整个矩阵无法填充输出寄存器组中的所有通道的情况下输出格式如何变化。

V\_MFMA\_F32\_4X4X4\_16B\_F16 指令执行 16 个块乘法，其形式为

$$D[b, :, :] = C[b, :, :] + A[b, :, :] * B[b, :, :]$$

其中 A 和 B 的每个块都是  $4 \times 4$  半精度浮点值块，C 和 D 的每个块保存  $4 \times 4$  单精度浮点数。

该指令使用 2 个寄存器来保存 A 和 B，尽管按照上一节的输入格式原则，每个通道需要保存四个值。这是因为每个输入寄存器保存两个半精度值，其中第二个值位于寄存器的高位 (16-31) 中。

即 A 的输入布局为

|              | 通道 0     | 通道 1     | ... | 通道 3     | 通道 4     | ... | 通道 63     |
|--------------|----------|----------|-----|----------|----------|-----|-----------|
| 寄存器 0[15:0]  | A[0,0,0] | A[0,1,0] | ... | A[0,3,0] | A[1,0,0] | ... | A[15,3,0] |
| 寄存器 0[31:16] | A[0,0,1] | A[0,1,1] | ... | A[0,3,1] | A[1,0,1] | ... | A[15,3,1] |
| 寄存器 1[15:0]  | A[0,0,2] | A[0,1,2] | ... | A[0,3,2] | A[1,0,2] | ... | A[15,3,2] |
| 寄存器 1[31:16] | A[0,0,3] | A[0,1,3] | ... | A[0,3,3] | A[1,0,3] | ... | A[15,3,3] |

对于 B 来说是

|              | 通道 0     | 通道 1     | ... | 通道 3     | 通道 4     | ... | 通道 63     |
|--------------|----------|----------|-----|----------|----------|-----|-----------|
| 寄存器 0[15:0]  | B[0,0,0] | B[0,0,1] | ... | B[0,0,3] | B[1,0,0] | ... | B[15,0,3] |
| 寄存器 0[31:16] | B[0,1,0] | B[0,1,1] | ... | B[0,1,3] | B[1,1,0] | ... | B[15,1,3] |
| 寄存器 1[15:0]  | B[0,2,0] | B[0,2,1] | ... | B[0,2,3] | B[1,2,0] | ... | B[15,2,3] |
| 寄存器 1[31:16] | B[0,3,0] | B[0,3,1] | ... | B[0,3,3] | B[1,3,0] | ... | B[15,3,3] |

该指令的 16 个  $4 \times 4$  输出块被排列到四个输出寄存器中，如下所示。

|      | 通道 0     | 通道 1     | ... | 通道 3     | 通道 4     | ... | 通道 63     |
|------|----------|----------|-----|----------|----------|-----|-----------|
| 注册 0 | D[0,0,0] | D[0,0,1] | ... | D[0,0,3] | D[1,0,0] | ... | D[15,0,3] |
| ...  | ...      | ...      | ... | ...      | ...      | ... | ...       |
| 注册 3 | D[0,3,0] | D[0,3,1] | ... | D[0,3,3] | D[1,3,0] | ... | D[15,3,3] |

也就是说，由于一个块中没有足够的 4 行组来填充每个寄存器中的 64 通道输出，因此使用连续的块来代替。请注意，这些输出是 32 位浮点数，因此不会打包到寄存器中。

### 7.1.3.4. V\_MFMA\_F64\_16X16X4\_F64

这以 V\_MFMA\_F64\_16X16X4\_F64 为例演示了如何处理双精度值。该指令遵循与前面的示例相同的输入布局模式，并且操作与 V\_MFMA\_F32\_32X32X2\_F32 最相似。然而，每个输入分布在多个寄存器中，以便容纳完整的 64 位值。

该指令以及其他双精度 MFMA 指令的输出不遵循其他 MFMA 指令的  $4 \times N$  块布局。相反，输出行在每个波前的通道上连续打包，然后打包成寄存器对（以考虑存储输出所需的 64 位），如下所示。

V\_MFMA\_F64\_16X16X4\_F64 的输入和输出格式为

|     | 通道 0             | 通道 1             | ... | 通道 15             | 通道 16            | ... | 通道 63             |
|-----|------------------|------------------|-----|-------------------|------------------|-----|-------------------|
| 补3  | A[0,0,0][31:0]   | A[0,1,0][31:0]   | ... | A[0,15,0][31:0]   | A[0,0,1][31:0]   | ... | A[0,15,3][31:0]   |
| 补4  | A[0,0,0][63:32]  | A[0,1,0][63:32]  | ... | A[0,15,0][63:32]  | A[0,0,1][63:32]  | ... | A[0,15,3][63:32]  |
|     | 通道 0             | 通道 1             | ... | 通道 15             | 通道 16            | ... | 通道 63             |
| 补5  | B[0,0,0][31:0]   | B[0,0,1][31:0]   | ... | B[0,0,15][31:0]   | B[0,1,0][31:0]   | ... | B[0,3,15][31:0]   |
| 补6  | B[0,0,0][63:32]  | B[0,0,1][63:32]  | ... | B[0,0,15][63:32]  | B[0,1,0][63:32]  | ... | B[0,3,15][63:32]  |
|     | 通道 0             | 通道 1             | ... | 通道 15             | 通道 16            | ... | 通道 63             |
| 补7  | D[0,0,0][31:0]   | D[0,0,1][31:0]   | ... | D[0,0,15][31:0]   | D[0,1,0][31:0]   | ... | D[0,3,15][31:0]   |
| 补8  | D[0,0,0][63:32]  | D[0,0,1][63:32]  | ... | D[0,0,15][63:32]  | D[0,1,0][63:32]  | ... | D[0,3,15][63:32]  |
| 补9  | D[0,4,0][31:0]   | D[0,4,1][31:0]   | ... | D[0,4,15][31:0]   | D[0,5,0][31:0]   | ... | D[0,7,15][31:0]   |
| ... | ...              | ...              | ... | ...               | ...              | ... | ...               |
| 补10 | D[0,12,0][63:32] | D[0,12,1][63:32] | ... | D[0,12,15][63:32] | D[0,13,0][63:32] | ... | D[0,15,15][63:32] |

## 7.1.4. 通用输入输出布局

一般来说，MFMA 指令通过其输入和输出数据类型、每个矩阵块的大小 M、N 和 K 以及它在 B 上操作的块的数量来参数化。

从语义上讲，对于每个  $0 \leq b < B$ 、 $0 \leq i < M$  和  $0 \leq j < N$ ，它计算

$$D[b, i, j] = C[b, i, j] + \sum_{\{0 \leq k < K\}} A[b, i, k] * B[b, k, j]$$

其中每个  $A[b, :, :]$  为  $M \times K$ ，每个  $B[b, :, :]$  为  $K \times N$ ，每个  $D[b, :, :]$  和相应的  $C[b, :, :]$  为  $M \times N$ 。

输入和输出的值根据固定布局放入指令的参数中。为简单起见，此布局是根据波前的通道以及每个通道的项目序列来定义的：这些项目被排列到 32 位寄存器中，这些寄存器是小端形式的 MFMA 指令的真正参数。

更具体地说，

- 对于 64 位数量，每一项对应一对寄存器，数量的低位在第一个寄存器中，高位在第二个寄存器中
- 对于 32 位数量，每个项目对应一个不同的寄存器
- 对于 16 位数量，一个项目是寄存器的一半，奇数项目占用位 31-16，偶数项目占用位 0-15
- 对于 8 位数量，四个项目被打包到一个寄存器中，类似于 16 位情况

### 7.1.4.1. 输入布局

要定义矩阵 A 的输入布局，首先定义辅助常量

$$K\_L = K / (64 / (M * B))$$

这是每个通道在其寄存器中保存的 K 的连续值的数量。

例如，对于指令 V\_MFMA\_F32\_32X32X1\_2B\_F32:

$$K\_L = 1 / (64 / (32 * 2)) = 1 / 1 = 1$$

对于 V\_MFMA\_F32\_32X32X2\_F32:

$$K\_L = 2 / (64 / (32 * 1)) = 2 / 2 = 1$$

这些都表明一个输入寄存器在 K 维中保存一个值，但对于 V\_MFMA\_F32\_4X4X4\_16B\_F16:

$$K\_L = 4 / (64 / (4 * 16)) = 4 / 1 = 4$$

表示对于每个通道，两个输入寄存器在 K 维中保存四个值。这些输入值以小端方式打包。例如，每行中的第三个值（其中 k = 2 零索引）位于所有通道中 A 和 B 的第二个输入寄存器的位 15:0 中。该 16 位区域是下面为 V\_MFMA\_F32\_4X4X4\_16B\_F16 计算的布局中的“第 2 项”。

请注意，在所有 MFMA 指令中，乘积 M \* B 和 N \* B 小于 64，即，考虑到所有块，A 的单列或 B 的单行的值适合单个输入项。

定义此布局后，给定的输入值 A[b,i,k] 被放置在项目中

$$k \% K\_L$$

通道的

$$i + M * (b + B * (k / K\_L))$$

B 的布局是将 B[b,k,j] 放置在 item 中的类似函数

$$k \% K\_L$$

通道的

$$j + N * (b + B * (k / K\_L))$$



### 7.1.4.2. 输出布局

MFMA 指令的输出值  $D[b,i,j]$  以及要添加到结果的矩阵的相应值  $C[b,i,j]$  存储在固定布局中，该布局是所使用的 MFMA 指令的函数。

要定义此布局，首先定义以下常量：

- $H$ ，组高，表示每个行组中放置了多少个连续的输出行，  
因此，它们存储在单个通道上的连续项目中。对于  $f64$  指令， $H = 1$ ，但对于所有指令其他 MFMA 指令， $H = 4$ 。
- $B\_I = \text{ceil}(64 / (N * M / H))$ ，每个输出项（存储内的一项）中存储的块数  
对于  $D$  或  $C$ ），适用于所有通道
- $M\_I = (64 / B\_I) / N$ ，所有通道中每个输出项中存储的  $D$  行数
- $G = M / (H * M\_I)$ ，存储  $B\_I$  输出块所需的行组数。

例如，使用指令 `V_MFMA_F32_32X32X1_2B_F32` 给出

```
H = 4
B_I = ceil(64 / (32 * 32 / H)) = ceil(64 / 256) = 1
M_I = (64 / 1) / 32 = 2
G = 32 / (4 * 2) = 4
```

而指令 `V_MFMA_F32_4X4X4_16B_F16` 产生值

```
H = 4
B_I = ceil(64 / (H * 4 / 4)) = ceil(64 / 4) = 16
M_I = (64 / 16) / 4 = 4 / 4 = 1
G = 4 / (H * 1) = 4 / 4 = 1
```

定义这些常量后，矩阵  $D$  的值  $D[b,i,j]$  位于项中

```
(i % H) + H * (i / (H * M_I) + G * (b / B_I))
```

关于通道

```
j + N * ((i / H) % M_I + M_I * (b % B_I))
```

### 7.1.5. 数值广播

虽然  $32 \times 1$  浮点矩阵  $A$  乘以  $1 \times 64$  矩阵  $B$  的操作本身不可用，但可以使用广播控件控制广播大小 (CBSZ)、 $A$  块 ID (ABID) 和  $B$  通道组排列 (BLGP) 来模拟此乘法。

这些控制影响从通道中检索值：计算  $A$  的特定元素所在的通道  $I_a$  后，该值将按照 CBSZ 和 ABID 字段的定义进行排列，以确定

计算期间访问的通道。类似地， $l_b$ （检索 B 的任何特定值时要使用的通道）以 BLGP 字段指定的方式进行排列。

### 7.1.5.1. CBSZ 和 ABID

CBSZ 和 ABID 字段一起控制矩阵 A 块的广播。

当 3 位 CBSZ 字段非零时，通道的一个块将其为矩阵 A 保存的值广播到通道的其他块，取代其他通道为 A 矩阵保存的值。设置 CBSZ 使得  $(1 < CBSZ)$  超过 MFMA 指令处理的块数是未定义的。

广播块大小为

$$S = 64 / (1 < CBSZ)$$

例如，如果 CBSZ 为 1，则 32 个通道的一个块为波前中的两组 32 个通道提供输入，而 CBSZ 为 3 意味着复制来自 8 个通道的块的值。

CBSZ 的最大合法值为 4。

4 位 ABID 字段控制 S 通道的哪个块用作广播源。可能的块按顺序编号，ABID=0 对应选择通道  $S-1:0$ ，ABID=1 对应选择  $2S-1:S$ ，依此类推。例如，如果 CBSZ=2，则 ABID=1 表示通道 16 至 31 的值被广播到通道的其他三个块，而 ABID=3 表示通道 48 至 63 作为其输入源。

设置 ABID 使得  $ABID \geq (1 < CBSZ)$  是不合法的，因为此类值不引用潜在的源块。

换句话说，CBSZ 和 ABID 位导致通道  $l_a$  从由排列给出的通道中读取其输入

$$p_a(l_a) = (l_a \% S) + (S * ABID)$$

作为一个完整的示例，如果在使用指令 `V_MFMA_F32_32X32X1_2B_F32` 时 CBSZ=1 且 ABID=1，则 B 的两个  $1 \times 32$  块都将乘以 A 的第二个  $32 \times 1$  块中的值，该值由前 32 个通道存储。也就是说，操作变成：

$$D[b,i,j] = C[b,i,j] + A[1,i,0] * B[b,0,j]$$

如果将 B 的两个块视为一个 64 行矩阵，则这是  $32 \times 1 \times 1 \times 64$  矩阵乘法。

### 7.1.5.2. BLGP

3 位 BLGP 字段选择读取矩阵 B 中值的路径。一旦确定一些值  $B[b, k, j]$  在  $l_b$  字段的  $r$  中，使用定义的输入布局，访问  $l_b$  的路径将根据 BLGP 字段的 BLGP 显示的路径。

数值。

表 29. 与 BLGP 值相对应的周期

| 值 | 说明          | 表达式                 |
|---|-------------|---------------------|
| 0 | 无广播         | $l\_b$              |
| 1 | 广播头 32 道    | $l\_b \% 32$        |
| 2 | 广播第二 32 道   | $l\_b \% 32 + 32$   |
| 3 | 向左旋转 16 道   | $(l\_b + 16) \% 64$ |
| 4 | 广播头 16 道    | $l\_b \% 16$        |
| 5 | 广播第二台 16 通道 | $l\_b \% 16 + 16$   |
| 6 | 广播第三道 16 道  | $l\_b \% 16 + 32$   |
| 7 | 广播第四道 16 道  | $l\_b \% 16 + 48$   |

7.1.5.3. F64 指令广播场的替代含义

在双精度浮点(f64)上操作的 MFMA 指令不支持上述广播方法.

这些指示忽略了 CBSZ 和 ABID.

BLGP 字段重新用于表示对矩阵 A,B,C 的否定.

- BLGP[0]使矩阵 A 的值在设定时被暗中否定
- BLGP[1] 导致矩阵 B 的值在设定时被暗中否定
- 如果设置，BLGP[2] 会导致矩阵 C 中的值被隐式求反

7.2. BF8 与 FP8 格式及转换

表 30. 小浮点数据格式

| FMt  | 符号表达式曼特 | 偏见 | +0<br>-0         | INF,<br>-INF     | NaN,<br>-NaN | 最大    | 最小值 (标准)               | 最小值 (分母)                |
|------|---------|----|------------------|------------------|--------------|-------|------------------------|-------------------------|
| FP16 | E5M10   | 15 | 0x0000<br>0x8000 | 0x7C00<br>0xFC00 | 补11          | 65504 | 6.10352E-05            | 5.96046E-08             |
| FP8  | E4M3    | 8  | +/-: 0x00        | N/A              | 0x80         | 240   | +/-2.0 <sup>(-7)</sup> | +/-2.0 <sup>(-10)</sup> |
| BF8  | E5M2    | 16 | +/-: 0x00        | 0x80             | 0x80         | 57344 | +/-2 <sup>(-15)</sup>  | 2.0 <sup>(-17)</sup>    |

表 31. 小浮点数据格式转换操作

| 指令             | Dst | Src0 | Src1 | 编码   | 控制                                            | 说明   |
|----------------|-----|------|------|------|-----------------------------------------------|------|
| CVT_PK_FP8_F32 | FP8 | FP32 | FP32 | VOP3 | Op_Sel[3] 忽略:<br>clamp, omod 支持:<br>neg、abs   | RNE  |
| CVT_PK_BF8_F32 | BF8 | FP32 | FP32 | VOP3 | Op_Sel[3] 忽略:<br>clamp, omod 支持:<br>neg、abs   | RNE  |
| CVT_SR_FP8_F32 | FP8 | FP32 | U32  | VOP3 | Op_Sel[3:2] 忽略:<br>clamp, omod 支持:<br>neg、abs | 随机舍入 |

| 指令             | Dst | Src0 | Src1 | 编码   | 控制                                                | 说明        |
|----------------|-----|------|------|------|---------------------------------------------------|-----------|
| CVT_SR_BF8_F32 | BF8 | FP32 | U32  | VOP3 | Op_Sel[3:2] 忽略:<br>clamp, omod 支持:<br>neg、abs     | 随机舍入      |
| CVT_PK_F32_FP8 | F32 | FP8  | -    | VOP1 | SDWA、Op_Sel[0]、<br>dst、dst+1 忽略: ab<br>s、neg、sext | dst 必须是偶数 |
| CVT_PK_F32_BF8 | F32 | BF8  | -    | VOP1 | SDWA、Op_Sel[0]、<br>dst、dst+1 忽略: ab<br>s、neg、sext | dst 必须是偶数 |
| CVT_F32_FP8    | F32 | FP8  | -    | VOP1 | SDWA、Op_Sel 忽<br>略: abs、neg、sext                  | -         |
| CVT_F32_BF8    | F32 | BF8  | -    | VOP1 | SDWA、Op_Sel 忽<br>略: abs、neg、sext                  | -         |

注：VOP3 指令不能使用 SDWA。

在上表中，CVT \* F32 指令不支持这些操作的 4 周期转发。用户必须在写入同一目标寄存器的低/高半或字节的转换之间插入 NOP 或写入其他目标 VREG 的指令。

转换指令有两种类型：

- 打包 – 每条指令将两个 8 位值转换为 32 位值
- 随机舍入 - 一个源具有要转换的数字，另一个源具有用于舍入的随机数
  - 这些操作从指定的 VGPR 添加随机值，然后截断为较小的结果数据类型

从 8 位格式和 SDWA 转换为 VOP1：

- SRC0 必须设置为 “SDWA” ，并且 SRC0 VGPR 在 SDWA 字中指定，SRC0\_SELECT 指定要转换的字节。其他 SDWA 字段将被忽略。

CVT\_SR\_FP8\_F32 和 CVT\_SR\_BF8\_F32 OP\_SEL 用法：

Op\_sel[3:2] == 00: dest\_vgpr[31:0] = {prev\_dst\_vgpr[31:8], 结果[7:0]} Op\_sel[3:2] == 01:  
dest\_vgpr[31:0] = {prev\_dst\_vgpr[31:16],结果[7:0], prev\_dst\_vgpr[7:0]} Op\_sel[3:2] == 10:  
dest\_vgpr[31:0] = {prev\_dst\_vgpr[31:24], 结果[7:0], prev\_dst\_vgpr[15:0]} Op\_sel[3:2] ==  
11: dest\_vgpr[31:0] = {结果[7:0], prev\_dst\_vgpr[23:0]}

CVT\_SR\_FP8\_F32 OP\_SEL 用途：

使用 Src0 和 Src1 作为 VOP3 编码提供的输入，它添加两个操作数，注意不要使用基于操作码的 src1 尾数的 MSB，并在转换为 F8 类型之前依赖于随机轮的 F8 数据类型。然后，OP\_SEL 位 3 和 2 被重新用于此 8b 写操作，并用于将生成的 8 位引导到 32b 输出的正确字节通道，保留目标的剩余 24b。

CVT\_\*FP8\_F32 和 CVT\*\_BF8\_F32 FP16\_OVFL 规则

FP16\_OVFL 标志应用于从 F32 到 FP8/BF8 格式的数据转换。下表指定了溢出行为：

CVT\_SR\_\* 和 CVT\_PK\_\* 仅支持 VGPRs 作为输入，而不支持 SGPRs、文字或内联常量。

| 来源价值        | 目的地价值       |             |             |             |
|-------------|-------------|-------------|-------------|-------------|
|             | FP8         |             | BF8         |             |
|             | FP16_OVFL=1 | FP16_OVFL=0 | FP16_OVFL=1 | FP16_OVFL=0 |
| NaN         | NaN         | NaN         | NaN         | NaN         |
| ±Inf        | ±max_E4M3   | NaN         | ±max_E5M2   | ±Inf        |
| 大于最大 FP8 幅度 | ±max_E4M3   | NaN         | ±max_E5M2   | ±Inf        |

寄存器 SH\_MEM\_CONFIG 的位[8]必须设置为 1，才能为 BF8 和 FP8 操作产生正确的结果。

### 7.3. 浮点处理细节与格式

非规格化数数字的处理根据指令采用的数据类型以及在某些情况下 MODE 标志的不同而有所不同。

- V\_MFMA\_F32\_\*\_F32 指令采用 32 位输入，遵循 MODE 中的非规格化数处理标志
- 但是，V\_MFMA\_F32\_\*\_XF32 指令也采用 32 位输入，但仅使用 10 位尾数，忽略 MODE.denorm 并且不刷新非规格化数
- Matrix-C 输入和结果矩阵输出忽略 MODE.denorm 并且不刷新非规格化数
- 所有采用大小小于 32 位浮点数的指令（F16、BF16、BF8、FP8）都会忽略 MODE.denorm 并且不会刷新非规格化数
- V\_MFMA\_F64\_\*\_F64 指令采用 64 位输入和输出，忽略 MODE 并四舍五入到最近的偶数，并允许输入和输出中存在反范数
- V\_MFMA\_I32\_\*\_I8 执行整数乘加，因此不考虑 MODE 位。I8 输入值相乘的 16 位结果在乘法之前被符号扩展为 32 位，并且乘法的 16 位结果在与 32 位结果相加之前被符号扩展为 32 位

矩阵核心单元不支持算术异常，但 DGEMM 矩阵运算支持异常。

### 7.4. 稀疏矩阵

V\_SMFMAC 指令系列对 4:2 结构稀疏矩阵 A 和稠密矩阵 B、C 和 D 执行矩阵乘法累加运算： $D = C + A \times B$ 。

A 矩阵使用 4:2 结构化稀疏度表示，这意味着矩阵 K 维度上每 4 个元素中有两个为零。这些零不直接存储，而是在单独的 VGPR 中描述，该 VGPR 保存 2 位索引值对。索引值指示每组 4 个值中的哪两个值非零，并用于重建完整的 A 矩阵。非零样本紧密堆积，导致 2:1 压缩。只有 A 矩阵可能是稀疏的。

这些 SMFMAC 指令都是“累加”操作，其中 C 和 D 矩阵相同，由指令的 VDST 字段（D 矩阵）引用。C 操作数输入被重新用于保存索引数据偏移量。

表 32. SMFMA VALU 操作码：

| 指令                       | 变体       | 块数 | 周期 | 说明                  |
|--------------------------|----------|----|----|---------------------|
| V_SMFMAC_F32_{*}_F16     | 16x16x32 | 1  | 16 | F16 数据的稀疏矩阵乘法       |
|                          | 32x32x16 | 1  | 32 |                     |
| V_SMFMAC_F32_{*}_BF16    | 16x16x32 | 1  | 16 | BF16 数据的稀疏矩阵乘法      |
|                          | 32x32x16 | 1  | 32 |                     |
| V_SMFMAC_I32_{*}_I8      | 16x16x64 | 1  | 16 | I8 数据的稀疏矩阵乘法        |
|                          | 32x32x32 | 1  | 32 |                     |
| V_SMFMAC_F32_{*}_BF8_BF8 | 16x16x64 | 1  | 16 | BF8 或 FP8 数据的稀疏矩阵乘法 |
| V_SMFMAC_F32_{*}_BF8_FP8 |          |    |    |                     |
| V_SMFMAC_F32_{*}_FP8_BF8 | 32x32x32 | 1  | 32 |                     |
| V_SMFMAC_F32_{*}_FP8_FP8 |          |    |    |                     |

矩阵 A 在结构上是稀疏的，并且在 srcA 偏移处每个通道占用两个 VGPRs。矩阵 B 是稠密矩阵，在 srcB 偏移处每个通道占用 4 个 VGPRs。矩阵 C 与目标参数共享 VGPR 偏移量并占用 16 个 VGPRs。

16 位源数据

如果 CBSZ[1:0] = 0，ABID[1:0]选择 VGPR 内的四个 8 位稀疏索引组之一，从 srcC 开始，包含通道的 8 位索引信息。如果 CBSZ[1:0] != 0;选择第一个 (VGPR[srcC][7..0])。

8 位源数据

如果 CBSZ[1:0] = 0，则 ABID[0] 选择 VGPR 内的两个 16 位稀疏索引集之一，从 srcC 开始，包含通道的 16 位索引信息。如果 CBSZ[1:0] != 0;选择第一个 (VGPR[srcC][15..0])。

所有 SMFMAC 指令必须遵循以下限制：

- 1. 矩阵 A 是稀疏矩阵，矩阵 B 是稠密矩阵。与矩阵 A 相比，ALU 从 VGPR 为矩阵 B 加载的数据多了一倍。
- 2. 矩阵 C 与结果矩阵相同。ALU 使用 VDST VGPR 加载矩阵 C。所有指令均编码为累加操作码。
- 3. src2 有索引编码（所有索引都在一个 VGPR 中），并且只能是 VGPR。索引数据提供有关 4 个 SRCA 中哪 2 个非零的信息。对于该索引对，索引 0 < 索引 1 & 索引 0 != 索引 1。
- 4. Src0、src1 和 VDST 的 VGPR 地址必须偶数对齐。
- 5. CBSZ 和 ABID 控件仅用于从 VGPR 读取中选择索引，不会影响 SRCA 矩阵广播等。如为使用 CBSZ 和 ABID 控件的其他 MFMA 操作码所定义的。

SMFMAC 指令对 ACC\_CD 的解释与其他指令不同：对于 SMFMAC，ACC\_CD 位仅控制 DEST vgpr (arch 与 Accum)，而不控制 SRC2 位置。SRC2 参数提供由 SRC0 参数提供的稀疏数据的索引数据，该数据必须与 A 和 B 矩阵数据一起驻留在 ARCH-vgprs 中。换句话说，SRC2 的行为就好像 ACC\_CD==0 一样。

7.4.1. 稀疏结构细节

矩阵 B 选择的每个索引都是 2 位数字，其标识 K=4 中的一个被选择。根据矩阵 B 布局，SRC2 可以保存多组索引。

### 7.4.1.1. 16 位 A/B 矩阵

当 A 和 B 矩阵由 16 位数据（FP16、BF16）组成时，适用以下规则。

表 33. 矩阵 B 布局

| 16x16x32 | Row0  | Row1    | Row2    | Row3    |
|----------|-------|---------|---------|---------|
| v0       | k=0,1 | k=8,9   | k=16,17 | k=24,25 |
| v1       | k=2,3 | k=10,11 | k=18,19 | k=26,27 |
| v2       | k=4,5 | k=12,13 | k=20,21 | k=28,29 |
| v3       | k=6,7 | k=14,15 | k=22,23 | k=30,31 |
| 32x32x16 | Row0  | Row1    | Row2    | Row3    |
| v0       | k=0,1 | k=0,1   | k=8,9   | k=8,9   |
| v1       | k=2,3 | k=2,3   | k=10,11 | k=10,11 |
| v2       | k=4,5 | k=4,5   | k=12,13 | k=12,13 |
| v3       | k=6,7 | k=6,7   | k=14,15 | k=14,15 |

每个通道有 K=8 个值，每个通道需要 4 个索引（8 位），因此每个 SRC2 VGPR 保存 4 组索引。

表 34. 索引布局

| 通道 ID     | 0 | 1 | ... 3 | 4               | ... 31 | 32 | ... 63 |
|-----------|---|---|-------|-----------------|--------|----|--------|
| Vn[31:30] |   |   |       | set3, V1[31:16] |        |    |        |
| Vn[29:28] |   |   |       | set3, V1[15:0]  |        |    |        |
| Vn[27:26] |   |   |       | set3, V0[31:16] |        |    |        |
| Vn[25:24] |   |   |       | set3, V0[15:0]  |        |    |        |
| ...       |   |   |       | ...             |        |    |        |
| Vn[9:8]   |   |   |       | set1, V0[15:0]  |        |    |        |
| Vn[7:6]   |   |   |       | set0, V1[31:16] |        |    |        |
| Vn[5:4]   |   |   |       | set0, V1[15:0]  |        |    |        |
| Vn[3:2]   |   |   |       | set0, V0[31:16] |        |    |        |
| Vn[1:0]   |   |   |       | set0, V0[15:0]  |        |    |        |

“Vn” 是保存索引值的 SRC-C VGPR。

### 7.4.1.2. 8 位 A/B 矩阵

当 A 和 B 矩阵由 8 位数据（I8、FP8、BF8）组成时，适用以下规则。

表 35. 矩阵 B 布局

| 16x16x64 | Row0          | Row1          | Row2          | Row3          |
|----------|---------------|---------------|---------------|---------------|
| v0       | k=0,1,2,3     | k=16,17,18,19 | k=32,33,34,35 | k=48,49,50,51 |
| v1       | k=4,5,6,7     | k=20,21,22,23 | k=36,37,38,39 | k=52,53,54,55 |
| v2       | k=8,9,10,11   | k=24,25,26,27 | k=40,41,42,43 | k=56,57,58,59 |
| v3       | k=12,13,14,15 | k=28,29,30,31 | k=44,45,46,47 | k=60,61,62,63 |
| 32x32x32 | Row0          | Row1          | Row2          | Row3          |
| v0       | k=0,1,2,3     | k=0,1,2,3     | k=16,17,18,19 | k=16,17,18,19 |
| v1       | k=4,5,6,7     | k=4,5,6,7     | k=20,21,23,23 | k=20,21,23,23 |
| v2       | k=8,9,10,11   | k=8,9,10,11   | k=24,25,26,27 | k=24,25,26,27 |

| 16x16x64 | Row0          | Row1          | Row2          | Row3          |
|----------|---------------|---------------|---------------|---------------|
| v3       | k=12,13,14,15 | k=12,13,14,15 | k=28,29,30,31 | k=28,29,30,31 |

每个通道有 K=16 个值，每个通道需要 8 个索引（16 位），因此每个 SRC2 VGPR 保存 2 组索引。

表 36. 索引布局

| 通道 ID     | 0 | 1 | ... 3 | 4               | ... 31 | 32 | ... 63 |
|-----------|---|---|-------|-----------------|--------|----|--------|
| Vn[31:30] |   |   |       | set1, V1[31:24] |        |    |        |
| Vn[29:28] |   |   |       | set1, V1[23:16] |        |    |        |
| Vn[27:26] |   |   |       | set1, V1[15:8]  |        |    |        |
| Vn[25:24] |   |   |       | set1, V1[7:0]   |        |    |        |
| Vn[23:22] |   |   |       | set1, V0[31:24] |        |    |        |
| Vn[21:20] |   |   |       | set1, V0[23:16] |        |    |        |
| Vn[19:19] |   |   |       | set1, V0[15:8]  |        |    |        |
| Vn[17:16] |   |   |       | set1, V0[7:0]   |        |    |        |
| Vn[15:14] |   |   |       | set0, V1[31:24] |        |    |        |
| ...       |   |   |       | ...             |        |    |        |
| Vn[3:2]   |   |   |       | set0, V0[15:8]  |        |    |        |
| Vn[1:0]   |   |   |       | set0, V0[7:0]   |        |    |        |

7.5. 依赖处理：所需的独立指令

下表列出了需要用户插入 NOP（或独立的 VALU 指令）的时序条件。

|              |                        |
|--------------|------------------------|
| <b>DLop</b>  | 点积                     |
| <b>XDLOP</b> | {I8, F16, BF16} 上的矩阵数学 |
| <b>DGEMM</b> | V_MFMA...F64           |
| <b>PASS</b>  | 4 个时钟周期                |

表 37. VOP3P 矩阵操作码所需的 NOP

| 第一条指令                | 第二条指令                                         | 需要等待 | 评论                                       |
|----------------------|-----------------------------------------------|------|------------------------------------------|
| 非 DLops VALU 写入 VGPR | V_MFMA* 读取 VGPR 或 V_SMFMA* 读取 VGPR            | 2    | 无内部 4 和 8 周期转发路径。                        |
| DL 操作写入 VGPR         | DLops 将 VGPR 读取为 SrcC, 操作码与第 1 个 DLops 完全相同   | 0    | 支持 DLops 背靠背 SrcC 转发的相同操作码，用于累加。         |
|                      | DLops 将 VGPR 读取为 SrcA/B, 操作码与第 1 个 DLops 完全相同 | 3    | 不支持 DLops 中的 SrcA/B 转发                   |
|                      | 任何与第一个 DLops 操作码不同的操作码读/写 VGPR (RAW + WAW)    | 3    | 禁用从 DL 操作到正常 VALU/VM/ LDS/FLAT 操作的所有转发路径 |



| 第一条指令                          | 第二条指令                                                                                    | 需要等待                       | 评论                                                                                                                                                       |
|--------------------------------|------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| XDL 写入 VGPR 或 V_SMFMA* 写入 VGPR | XDL 将 VGPR 读取为源 C, 与第一个 vDst 完全相同<br><br>OR<br><br>V_SMFMA* 读取矩阵 C 的 VGPR 与第一个 vDst 完全相同 | 2 如果第一个 V_MFMA 是 2 次通过     | 两个 V_MFMA 必须具有相同的通道数, 并且 vDst 和 vSrc 从相同的偏移量和相同的 VGPR 大小开始。<br><br>V_MFMA 和 V_SMFMA 必须是相同的通道数, 并且两个 vDst 从相同的偏移量和相同的 VGPR 大小开始。注: V_SMFMA 读取矩阵 C 的 vdst。 |
|                                |                                                                                          | 如果第一个 V_MFMA 为 4 次, 则为 0   |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 是 8 次则为 0     |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 为 16 次, 则为 0  |                                                                                                                                                          |
| XDL 写入 VGPR 或 V_SMFMA* 写入 VGPR | XDL 将 VGPR 读取为源 C 与第一个 vDst 重叠<br><br>OR<br><br>V_SMFMA* 读取与第一个 vDst 重叠的矩阵 C 的 VGPR      | 3 如果第一个 V_MFMA 是 2 次通过     | 与 XDL 重叠<br><br>注: V_SMFMA 读取矩阵 C 的 vdst。                                                                                                                |
|                                |                                                                                          | 如果第一个 V_MFMA 是 4 次通过, 则为 5 |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 是 8 次通过, 则为 9 |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 为 16 次, 则为 17 |                                                                                                                                                          |
| XDL 写入 VGPR 或 V_SMFMA* 写入 VGPR | S/DGEMM 将 VGPR 读取为源 C                                                                    | 3 如果第一个 V_MFMA 是 2 次通过     | 与 S/DGEMM 重叠                                                                                                                                             |
|                                |                                                                                          | 如果第一个 V_MFMA 是 4 次通过, 则为 5 |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 是 8 次通过, 则为 9 |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 为 16 次, 则为 17 |                                                                                                                                                          |
| XDL 写入 VGPR 或 V_SMFMA* 写入 VGPR | V_MFMA 将 VGPR 读取为 SrcA 或 SrcB<br><br>OR<br><br>V_SMFMA* 将 VGPR 读取为 SrcA 或 SrcB 或索引 SrcC  | 5 如果第一个 V_MFMA 是 2 次通过     | 没有内部转发路径等待先前的 V_MFMA/V_SMFMA* 提交结果到 VGPR<br><br>V_SMFMA 使用 srcC 地址进行额外的索引 C 读取                                                                           |
|                                |                                                                                          | 7 如果第一个 V_MFMA 是 4 次通过     |                                                                                                                                                          |
|                                |                                                                                          | 11 如果第一个 V_MFMA 是 8 次通过    |                                                                                                                                                          |
|                                |                                                                                          | 如果第一个 V_MFMA 是 16 次, 则为 19 |                                                                                                                                                          |
| XDL 写入 VGPR 或 V_SMFMA* 写入 VGPR | 1) VM、L/GDS、FLAT、导出读取 VGPR 与第一个 vDst 重叠<br><br>2) VALU 读/写 VGPR (RAW + WAW)              | 5 如果第一个 V_MFMA 是 2 次通过     | V_MFMA_F32_4X4X4F16                                                                                                                                      |
|                                |                                                                                          | 7 如果第一个 V_MFMA 是 4 次通过     | V_MFMA_F32_16X16X16F16                                                                                                                                   |
|                                |                                                                                          | 11 如果第一个 V_MFMA 是 8 次通过    | V_MFMA_F32_32X32X8F16                                                                                                                                    |
|                                |                                                                                          | 如果第一个 V_MFMA 是 16 次, 则为 19 | V_MFMA_F32_32X32X4F16                                                                                                                                    |
| SGEMM 写入 VGPR                  | XDL 将 VGPR 读取为源 C, 与第一个 vDst 完全相同<br><br>OR<br><br>V_SMFMA* 读取矩阵 C 的 VGPR 与第一个 vDst 完全相同 | 0                          | 两个 V_MFMA 必须具有相同的通道数, 并且 vDst 和 vSrc 从相同的偏移量和相同的 VGPR 大小开始。<br><br>V_MFMA 和 V_SMFMA 必须是相同的通道数, 并且两个 vDst 从相同的偏移量和相同的 VGPR 大小开始。注: V_SMFMA 读取矩阵 C 的 vdst。 |

| 第一条指令         | 第二条指令                                              | 需要等待                       | 评论                                                                    |
|---------------|----------------------------------------------------|----------------------------|-----------------------------------------------------------------------|
| SGEMM 写入 VGPR | XDL 将 VGPR 读取为源 C 与第一个 vDst 重叠                     | 2 如果第一个 V_MFMA 是 2 次通过     | 与 XDL 重叠<br><br>注: V_SMFMA 读取矩阵 C 的 vdst。                             |
|               | OR<br><br>V_SMFMA* 读取与第一个 vDst 重叠的矩阵 C 的 VGPR      | 4 如果第一个 V_MFMA 是 4 次通过     |                                                                       |
|               |                                                    | 如果第一个 V_MFMA 是 8 次通过, 则为 8 |                                                                       |
|               |                                                    | 16 如果第一个 V_MFMA 是 16 次     |                                                                       |
| SGEMM 写入 VGPR | S/DGEMM 将 VGPR 读取为源 C                              | 2 如果第一个 V_MFMA 是 2 次通过     | 与 S/DGEMM 重叠                                                          |
|               |                                                    | 4 如果第一个 V_MFMA 是 4 次通过     |                                                                       |
|               |                                                    | 如果第一个 V_MFMA 是 8 次通过, 则为 8 |                                                                       |
|               |                                                    | 16 如果第一个 V_MFMA 是 16 次     |                                                                       |
| SGEMM 写入 VGPR | V_MFMA 将 VGPR 读取为 SrcA 或 SrcB                      | 4 如果第一个 V_MFMA 是 2 次通过     | 无内部转发路径, 需要等待之前的 V_MFMA 提交结果到 VGPR。<br><br>注: V_SMFMA 使用 SrcC 进行索引读取。 |
|               | OR<br><br>V_SMFMA* 将 VGPR 读取为 SrcA 或 SrcB 或索引 SrcC | 6 如果第一个 V_MFMA 是 4 次通过     |                                                                       |
|               |                                                    | 如果第一个 V_MFMA 为 8 次, 则为 10  |                                                                       |
|               |                                                    | 如果第一个 V_MFMA 是 16 次, 则为 18 |                                                                       |
| SGEMM 写入 VGPR | 1) VM、L/GDS、FLAT、导出读取 VGPR 与第一个 vDst 重叠            | 4 如果第一个 V_MFMA 是 2 次通过     | V_MFMA_F32_4X4X4F16                                                   |
|               |                                                    | 6 如果第一个 V_MFMA 是 4 次通过     | V_MFMA_F32_16X16X16F16                                                |
|               | 2) VALU 读/写 VGPR (RAW + WAW)                       | 如果第一个 V_MFMA 为 8 次, 则为 10  | V_MFMA_F32_32X32X8F16                                                 |
|               |                                                    | 如果第一个 V_MFMA 是 16 次, 则为 18 | V_MFMA_F32_32X32X4F16                                                 |

| 第一条指令                      | 第二条指令                                            | 需要等待 | 评论                                               |
|----------------------------|--------------------------------------------------|------|--------------------------------------------------|
| V_MFMA_16x16x4_F64 写入 VGPR | V_MFMA_16x16x4_F64 将 VGPR 读取为源 C, 与第一个 vDst 完全相同 | 0    | 两个 V_MFMA 必须具有相同的通道数, 并且 vDst 和 vSrc 从相同的偏移量开始。  |
|                            | S/DGEMM 读取 VGPR 作为源 C 与第一个 vDst 重叠               | 9    | 重叠, 不同的 VGPR 访问顺序                                |
|                            | XDL 将 VGPR 读取为源 C 与第一个 vDst 重叠                   | 0    |                                                  |
|                            | V_SMFMA* 读取与第一个 vDst 重叠的矩阵 C 的 VGPR              | 0    | V_SMFMA 读取矩阵 C 的 vdst。                           |
|                            | S/DGEMM 将 VGPR 读取为 SrcA 或 SrcB                   | 11   | 无内部转发路径, 需要等待之前的 V_MFMA 提交结果到 VGPR               |
|                            | XDL 将 VGPR 读取为 SrcA 或 SrcB                       | 11   |                                                  |
|                            | V_SMFMA* 将 VGPR 读取为 SrcA 或 SrcB 或索引 SrcC         | 11   | V_SMFMA 使用 srcC 地址进行额外的索引 C 读取                   |
|                            | VALU 读/写 VGPR<br>(RAW + WAW)                     | 11   |                                                  |
|                            | VM、L/GDS、FLAT 和导出读取 VGPR 与第一个 vDst 重叠            | 18   | 无内部转发路径, 需要等待之前的 V_MFMA 提交结果到 VGPR               |
| V_MFMA_4x4x4_F64 写入 VGPR   | V_MFMA_4x4x4_F64, 将 VGPR 读作源 C, 与第一个 vDst 完全相同   | 4    | 4x4x4 需要做累加, 所以写入 VGPR 比普通 XDL 4x4x4 晚, 所以需要额外等待 |
|                            | S/DGEMM 读取 VGPR 作为源 C 与第一个 vDst 重叠               | 4    | 重叠, 不同的 VGPR 访问顺序                                |
|                            | XDL 将 VGPR 读取为源 C 与第一个 vDst 重叠                   | 0    |                                                  |
|                            | V_SMFMA 读取与第一个 vDst 重叠的矩阵 C 的 VGPR               | 0    | V_SMFMA 读取矩阵 C 的 vdst。                           |
|                            | S/DGEMM 将 VGPR 读取为 SrcA 或 SrcB                   | 6    | 无内部转发路径, 需要等待之前的 V_MFMA 提交结果到 VGPR               |
|                            | XDL 将 VGPR 读取为 SrcA 或 SrcB                       | 6    |                                                  |
|                            | V_SMFMA* 将 VGPR 读取为 SrcA 或 SrcB 或索引 SrcC         | 6    | V_SMFMA 使用 srcC 地址进行额外的索引 C 读取                   |
|                            | VALU 读/写 VGPR<br>(RAW + WAW)                     | 6    |                                                  |
|                            | VM、L/GDS、FLAT 和导出读取 VGPR 与第一个 vDst 重叠            | 9    | 无内部转发路径, 需要等待之前的 V_MFMA 提交结果到 VGPR               |
| V_CMPX* 写入 EXEC 掩码         | V_MFMA*                                          | 4    | CDNA 不支持使用 XDL/DGEMM 转发执行掩码                      |
| XDL/SMFMA 读取 VGPR SrcC     | VALU 写入 VGPR (WAR), 与第一个 SrcC 重叠的协同执行反依赖         | 1    | 首条 MFMA 为 2 趟                                    |
|                            |                                                  | 3    | 首条 MFMA 为 4 趟                                    |
|                            |                                                  | 7    | 首条 MFMA 为 8 趟                                    |
|                            |                                                  | 15   | 首条 MFMA 为 16 趟                                   |

# 第 8 章 标量内存操作

标量内存读取 (SMEM) 指令允许着色器程序通过标量数据缓存将数据从内存加载到 SGPRs，或通过标量数据缓存将数据从 SGPRs 写入内存。指令一次可以读取 1 到 16 个双字，或写入 1 到 4 个双字。数据直接读入 SGPRs，无需任何格式转换。

标量单元在内存和 SGPRs 之间读取和写入连续的双字。这主要用于加载 ALU 常量。不支持数据格式，也不支持字节或短数据。

## 8.1. 微码编码

标量内存读取、写入和原子指令使用 SMEM 微码格式进行编码。



各字段的说明如下表所示：

表 38. SMEM 编码字段说明

| 字段     | 大小 | 说明                                                                                                                                              |
|--------|----|-------------------------------------------------------------------------------------------------------------------------------------------------|
| OP     | 8  | 操作码。                                                                                                                                            |
| IMM    | 1  | 确定如何解释 OFFSET 字段。 IMM=1：偏移量是地址的 21 位无符号字节偏移量。 IMM=0：Offset[6:0] 指定 SGPR 或 M0，它提供无符号字节偏移（对于存储，必须是 M0）。 STORE 和 ATOMIC 指令不能使用 SGPR：只能使用 imm 或 M0。 |
| GLC    | 1  | 全球一致。对于加载，控制 L1 缓存策略：0=hit_lru，1=miss_evict。对于存储，控制 L1 缓存旁路：0=写组合，1=写直通。对于原子，“1”表示原子返回预操作值。                                                     |
| SDATA  | 7  | SGPRs 返回读取数据或写入数据源。两个双字的读取必须具有偶数 SDST-sgpr。读取四个或更多双字时，其 DST-gpr 必须与 4 的倍数对齐。SDATA 必须是：SGPR 或 VCC。不是：exec 或 m0。                                  |
| SBASE  | 6  | SGPR 对 (SBASE 隐含的 LSB 为零) 提供基地址，或者对于 BUFFER 指令，提供一组 4 个 SGPRs (4-sgpr 对齐)，用于保持资源常量。对于 BUFFER 指令，唯一使用的资源字段是：base、stride、num_records。             |
| OFFSET | 21 | 无符号字节偏移量，或保存偏移量的 SGPR 的地址。写入和原子：M0 或仅立即，而不是 SGPR。                                                                                               |
| NV     | 1  | 非挥发性。                                                                                                                                           |
| SOE    | 1  | 标量偏移启用。                                                                                                                                         |

有关 GLC 位的更多信息，请参阅内存范围和时间控制。

## 8.2. 操作

### 8.2.1. S\_LOAD\_DWORD, S\_STORE\_DWORD

这些指令在 SGPRs 和内存之间加载 1-16 个双字或存储 1-4 个双字。SGPRs 中的数据在 SDATA 中指定，地址由 SBASE、OFFSET、SOFFSET 字段组成。

#### 8.2.1.1. 标量内存寻址

S\_LOAD / S\_STORE / S\_DCACHE\_DISCARD:

ADDR = SGPR[基址] + inst\_offset + { M0 或 SGPR[偏移] 或零 }

S\_SCRATCH\_LOAD / S\_SCRATCH\_STORE:

ADDR = SGPR[基址] + inst\_offset + { M0 或 SGPR[偏移] 或零 } \* 64

偏移量字段的使用:

| IMM | SOFFSET_EN (SOE) | 地址                                       |
|-----|------------------|------------------------------------------|
| 0   | 0                | SGPR[基础] + (SGPR[偏移] 或 M0)               |
| 0   | 1                | SGPR[基础] + (SGPR[偏移] 或 M0)               |
| 1   | 0                | SGPR[底座] + inst_offset                   |
| 1   | 1                | SGPR[基础] + inst_offset + (SGPR[偏移] 或 M0) |

地址的所有组成部分（基址、偏移量、inst\_offset、M0）均以字节为单位，但两个 LSB 被忽略并被视为零。S\_DCACHE\_DISCARD 忽略 6 个 LSB 以使地址 64 字节对齐。

如果 inst\_offset 为负且结果 (inst\_offset + (M0 或 SGPR[offset])) 为负，则这是非法且未定义的。

对私有空间的标量访问必须使用缓冲区常量或手动转换地址:

地址 = 地址 - private\_base + private\_base\_addr + scratch\_baseOffset\_for\_this\_wave

着色器无法通过硬件使用“隐藏的私有基础”：它必须预加载到 SGPR 中或通过常量缓冲区提供。这相当于驱动程序必须执行的操作才能从头开始计算缓冲区常量的基地址。

标量指令不得覆盖其自己的源寄存器，因为 ATC XNACK 可能会重放该指令。同样，标量内存子句中的指令不得覆盖该子句中任何指令的源。子句被定义为相同类型的内存指令串。任何非内存指令都会破坏子句。此规则的一个例外是子句中的单个 SMEM 指令本身加载单个 DWORD 可以合法地覆盖其自己的源 SGPRs。（这

指令要么完全成功执行并继续，要么完全失败；它不会仅覆盖一个 DWORD 的一部分）。

原子很不寻常，因为它们自然对齐并且必须位于单个指令子句中。根据定义，返回预操作值的原子会覆盖其数据源，这是可以接受的。

## 使用缓冲区常量读/写/原子

使用的缓冲区常量字段：base\_address、stride、num\_records、NV。其他字段将被忽略。

标量内存读/写不支持“swizzled”缓冲区。Stride 仅用于内存地址边界检查，而不用于计算要访问的地址。

SMEM 仅提供 SBASE 地址（字节）和偏移量（字节或双字）。任何“索引 \* 步幅”都必须着色器代码中手动计算，并添加到 SMEM 之前的偏移量中。

V#.base 和最终地址的两个 LSB 将被忽略以强制双字对齐。

```
"m_#" components come from the buffer constant (V#):
offset      = IMM ? OFFSET : SGPR[OFFSET]
m_base      = { SGPR[SBASE * 2 + 1][15:0], SGPR[SBASE] }
m_stride    = SGPR[SBASE * 2 + 1][31:16]
m_num_records = SGPR[SBASE * 2 + 2]
m_size      = (m_stride == 0) ? 1 : m_num_records
m_addr      = (SGPR[SBASE * 2] + offset) & ~0x3
SGPR[SDST] = read_dword_from_dcache(m_base, offset, m_size)
```

If more than 1 dword is being read, it is returned to SDST+1, SDST+2, etc,  
and the offset is incremented by 4 bytes per DWORD.

## 8.2.2. 标量原子操作

标量内存单元支持与向量内存单元相同的内存原子集。寻址与标量内存加载和存储相同。与向量内存原子一样，标量原子操作可以将“操作前值”返回到 SDATA SGPRs。这是通过将微码 GLC 位设置为 1 来启用的。

## 8.2.3. S\_DCACHE\_INV, S\_DCACHE\_WB

该指令使整个标量数据缓存无效，或者“写回”脏数据。它不会向 SDST 返回任何内容。

## 8.2.4. S\_MEMTIME

该指令将 64 位时钟计数器读入一对 SGPRs：SDST 和 SDST+1。

### 8.2.5. S\_MEMREALTIME

该指令读取 64 位“实时计数器”并将值返回到一对 SGPRs: SDST 和 SDST+1。时间值来自恒定的 100MHz 时钟（不受电源模式或核心时钟频率变化的影响）。

## 8.3. 依赖检查

标量内存读取和写入返回的数据可能与发出时的顺序无序；当读取跨越两个缓存行时，它们可以在不同时间返回部分结果。着色器程序使用 LGKM\_CNT 计数器来确定数据何时返回到 SDST SGPRs。这是按如下方式完成的。

- 每次读取单个 Dword 时，LGKM\_CNT 都会递增 1。
- 每次读取两个或多个双字时，LGKM\_CNT 都会增加 2。
- 每条指令完成时，LGKM\_CNT 都会递减等量。

由于指令可能会乱序返回，因此使用该计数器的唯一合理方法是实现 S\_WAITCNT 0；这会强制等待所有数据从之前的 SMEM 返回，然后再继续。

## 8.4. 对齐与边界检查

### SDST

对于两个双字（包括 S\_MEMTIME）的读取，SDST 的值必须是偶数，对于更大的读取，SDST 的值必须是四的倍数。如果不遵守此规则，可能会导致无效数据。如果 SDST 超出范围，则不执行该指令。

### SBASE

对于 S\_BUFFER\_LOAD，SBASE 的值必须是偶数（指定 SGPR 的地址，它是四的倍数）。如果 SBASE 超出范围，则使用 SGPR0 中的值。

### OFFSET

OFFSET 的值没有对齐限制。

内存地址：如果内存地址超出范围（钳位），则不会对任何内存执行操作超出范围的双字。

## 第 9 章 向量内存操作

向量内存 (VMEM) 指令将波前中的每个工作项的一条数据分别读取或写入 VGPRs。这与标量内存指令形成对比，标量内存指令移动由波前中的所有线程共享的单个数据。所有向量内存 (VM) 操作均由纹理缓存系统 (1 级和 2 级缓存) 处理。

软件通过以下类型的 VMEM 指令之一通过纹理缓存启动加载、存储或原子操作：

- MTBUF：内存类型缓冲区操作。
- MUBUF：内存无类型缓冲区操作。

该指令定义哪些 VGPR 为操作提供地址，哪些 VGPRs 为操作提供或接收数据，以及一系列包含内存缓冲区描述符 (V#) 的 SGPRs。

### 9.1. 向量内存缓冲区指令

向量内存 (VM) 操作通过纹理缓存 (TC) 在 VGPRs 和内存中的缓冲区对象之间传输数据。向量意味着波前中的每个线程都会唯一地传输一个或多个数据，这与标量内存读取相反，标量内存读取仅传输由波前中的所有线程共享的一个值。

缓冲区读取可以选择将数据返回到 VGPRs 或直接返回到 LDS。

缓冲区对象的示例有顶点缓冲区、原始缓冲区、流输出缓冲区和结构化缓冲区。

缓冲区对象支持同质和异构数据，但不过滤读取数据。缓冲区指令分为两组：

- MUBUF：无类型缓冲区对象。
  - 数据格式在资源常量中指定。加载、存储、原子操作，带或不带数据格式转换。
- MTBUF：类型化缓冲区对象。
  - 数据格式在指令中指定。唯一的操作是加载和存储，两者都带有数据格式转换。

原子操作从 VGPRs 获取数据，并将它们与内存中已有的数据进行算术组合。或者，可以将操作发生之前内存中的值返回给着色器。

所有 VM 操作都使用缓冲区资源常量 (V#)，该常量是 SGPRs 中的 128 位值。当指令执行时，该常量被发送到纹理缓存。该常量定义了内存中缓冲区的地址和特征。通常，这些常量是在执行 VM 指令之前使用标量内存读取从内存中获取的，但这些常量也可以在着色器内生成。



9.1.1. 简化缓冲区寻址

下面的等式显示了硬件如何计算缓冲区访问的内存地址。

ADDR =   Base   +   baseOffset   +   Inst\_offset   +   Voffset   +   Stride \* (Vindex + TID)

V#           SGPR           Instr           VGPR           V#    VGPR    0..63

Voffset is ignored when instruction bit “OFFEN” == 0

Vindex is ignored when instructino bit “IDXEN” == 0

TID is a constant value (0..63) unique to each thread in the wave. It is ignored when resource bit ADD\_TID\_ENABLE == 0

9.1.2. 缓冲区指令

缓冲区指令（MTBUF 和 MUBUF）允许着色器程序读取和写入内存中的线性缓冲区。这些操作可以对小至一个字节的  
的数据进行操作，每个工作项最多可以操作四个双字。提供了原子算术运算，可以对内存中的数据值进行操作，并且  
可以选择返回执行算术运算之前内存中的值。

D16 指令变体将结果转换为打包的 16 位值。例如，  
BUFFER\_LOAD\_FORMAT\_D16\_XYZW 写入两个 VGPRs。

表 39. 缓冲区指令

| 指令                                                                                                              | 说明                                                                         |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| MTBUF 使用说明                                                                                                      |                                                                            |
| TBUFFER_LOAD_FORMAT_{x,xy,xyz,xyzw}<br>TBUFFER_STORE_FORMAT_{x,xy,xyz,xyzw}                                     | 从类型化缓冲区对象中读取或写入。也用于顶点获取。                                                   |
| MUBUF 使用说明                                                                                                      |                                                                            |
| BUFFER_LOAD_FORMAT_{x,xy,xyz,xyzw}<br>BUFFER_STORE_FORMAT_{x,xy,xyz,xyzw}<br>BUFFER_LOAD_<大小> BUFFER_STORE_<大小> | 读取或写入非类型化缓冲区对象。 <大小> = 字节、ubyte、short、ushort、Dword、Dwordx2、Dwordx3、Dwordx4 |
| BUFFER_ATOMIC_<操作><br>BUFFER_ATOMIC_<操作>_x2                                                                     |                                                                            |

表 40. 微码格式

| 字段      | 位大小    | 说明                                                                                      |
|---------|--------|-----------------------------------------------------------------------------------------|
| OP      | 4<br>7 | MTBUF：类型化缓冲区指令的操作码。 MUBUF：<br>无类型缓冲区指令的操作码。                                             |
| VADDR   | 8      | VGPR 的地址，用于提供地址的第一个组成部分（偏移量或索引）。当同时使用 index 和 offset 时，index 在<br>第一个 VGPR，offset 在第二个。 |
| VDATA   | 8      | VGPR 的地址，用于提供写数据的第一部分或接收读数据的第一部分。                                                       |
| SOFFSET | 8      | SGPR 提供无符号字节偏移量。必须是 SGPR、M0 或内联常量。                                                      |
| SRSRC   | 5      | 指定哪个 SGPR 在四个或八个连续的 SGPRs 中提供 T#（资源常量）。该字段缺少 SGPR 地址的两个 LS<br>B，因为该地址必须与 4 SGPRs 的倍数对齐。 |

| 字段     | 位大小 | 说明                                                                                                                                                                           |
|--------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DFMT   | 4   | 数据内存缓冲区数据格式：0 无效 1 8 2<br>16 3 8 8 4 32 5 16 16 6 10 11 11 7<br>11 11 10 8 10 10 10 2 9 2 10 10 10<br>10 8 8 8 8 11 32 32 12 16 16 16 16<br>13 32 32 32 14 32 32 32 32 15 保留 |
| NFMT   | 3   | 内存中数据的数值格式：0 = unorm, 1 = snorm, 2 = uscaled, 3 = sscaled, 4 = uint, 5 = sint, 6 = 保留, 7 = float。                                                                            |
| OFFSET | 12  | 无符号字节偏移量。                                                                                                                                                                    |
| OFFEN  | 1   | 1 = 提供来自 VGPR (VADDR) 的偏移。0 = 不 (偏移 = 0) 。                                                                                                                                   |
| IDXEN  | 1   | 1 = 从 VGPR (VADDR) 提供索引。0 = 不 (索引 = 0) 。                                                                                                                                     |
| SC0    | 1   | 范围位 0                                                                                                                                                                        |
| NT     | 1   | 非时间性                                                                                                                                                                         |
| ACC    | 1   | VDATA 是累加 VGPR                                                                                                                                                               |
| SC1    | 1   | 范围位 1                                                                                                                                                                        |
| LDS    | 1   | 仅 MUBUF: 0 = 将读取数据返回到 VGPRs。1 = 将读取数据返回到 LDS 而不是 VGPRs。                                                                                                                      |

### 9.1.3. VGPR 的使用

VGPRs 提供地址和写入数据；此外，它们可以作为返回数据的目的地（另一个选项是 LDS）。

#### 地址

根据指令字中的偏移使能（OFFEN）和索引使能（IDXEN），使用 0 个、1 个或 2 个 VGPRs，如下表所示：

表 41. 地址 VGPRs

| IDXEN | OFFEN | VGPRn | VGPRn+1 |
|-------|-------|-------|---------|
| 0     | 0     | 没有什么  |         |
| 0     | 1     | 单位偏移量 |         |
| 1     | 0     | 单位索引  |         |
| 1     | 1     | 单位索引  | 单位偏移量   |

写入数据：N 个连续的 VGPRs，从 VDATA 开始。指令字中指定的数据格式（NFMT、MTBUF 的 DFMT 或 MUBUF 的操作码字段中编码）确定要写入的 Dword 数量。

读取数据：与写入相同。数据返回到连续的 GPRs。

读取数据格式：读取数据为 32 位，基于指令或资源中的数据格式。浮动或标准化数据以浮点数形式返回；整数格式以整数形式返回（有符号或无符号，与内存存储格式的类型相同）。内存中 32 位或 64 位数据的内存读取不会进行任何格式转换。

带返回的原子：从 VDATA 开始从 VGPR 中读取数据以提供给原子操作。如果原子返回一个值到 VGPRs，该数据返回到从 VDATA 开始的相同 VGPRs。

9.1.4. 缓冲区数据

读取或写入的数据量和类型由以下各项控制：数据格式 (dfmt)、数字格式 (nfmt)、目标组件选择 (dst\_sel) 和操作码。Dfmt 和 nfmt 可以来自资源、指令字段或操作码本身。Dst\_sel 来自资源，但在许多操作中被忽略。

表 42. 缓冲区指令

| 指令                     | 数据格式 | 数字格式 | DST SEL |
|------------------------|------|------|---------|
| TBUFFER_LOAD_FORMAT_*  | 操作说明 | 操作说明 | 身份      |
| TBUFFER_STORE_FORMAT_* | 操作说明 | 操作说明 | 身份      |
| BUFFER_LOAD_<类型>       | 衍生的  | 衍生的  | 身份      |
| BUFFER_STORE_<类型>      | 衍生的  | 衍生的  | 身份      |
| BUFFER_LOAD_FORMAT_*   | 资源   | 资源   | 资源      |
| BUFFER_STORE_FORMAT_*  | 资源   | 资源   | 资源      |
| BUFFER_ATOMIC_*        | 衍生的  | 衍生的  | 身份      |

指令：使用指令的 dfmt 和 nfmt 字段代替资源的字段。

数据格式派生：数据格式从操作码派生，忽略资源定义。为了例如，buffer\_load\_ubyte 将 data-format 设置为 8，将 number-format 设置为 uint。



资源的数据格式不能为 INVALID；该格式具有特定含义（未绑定资源），并且在这种情况下，数据格式不会被指令的隐含数据格式替换。

DST\_SEL 标识：根据数据格式中的组件数量，这是：X000、XY00、XYZ0 或 XYZW。

MTBUF 从指令中导出数据格式。MUBUF、BUFFER\_LOAD\_FORMAT 和 BUFFER\_STORE\_FORMAT 指令使用资源中的 dst\_sel；其他 MUBUF 指令从指令本身导出数据格式。

D16 指令：加载格式和存储格式指令也有“d16”变体。对于商店，每 32-位 VGPR 保存传递到纹理单元的两个 16 位数据元素。该纹理单元在写入内存之前将它们转换为纹理格式。对于加载，从纹理单元返回的数据被转换为 16

位和一对数据存储在每个 32 位 VGPR 中（首先是 LSB，然后是 MSB）。对 int 与 float 的控制由 NFMT 控制。

### 9.1.5. 缓冲区寻址

缓冲区是内存中的数据结构，通过索引和偏移量进行寻址。索引指向大小为 stride 字节的特定记录，偏移量是记录内的字节偏移量。步长来自资源、来自 VGPR（或零）的索引、来自 SGPR 或 VGPR 的偏移量以及来自指令本身的偏移量。

表 43. 用于寻址的 BUFFER 指令字段

| 字段          | 大小 | 说明                                                                          |
|-------------|----|-----------------------------------------------------------------------------|
| inst_offset | 12 | 指令的文字字节偏移量。                                                                 |
| inst_idxen  | 1  | Boolean: true 时从 VGPR 获取索引, false 时不获取索引。                                   |
| inst_offen  | 1  | Boolean: true 时获取相对于 VGPR 的偏移量, false 时不获取偏移量。请注意, 无论该位如何, inst_offset 都存在。 |

缓冲区指令的“元素大小”是指令传输的数据量。对于 MTBUF 指令, 它由 DFMT 字段确定; 对于 MUBUF 指令, 它由操作码确定。它可以是 1、2、4、8 或 16 字节。

表 44. 用于寻址的 V# 缓冲区资源常量字段

| 字段                   | 大小      | 说明                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| const_base           | 48      | 缓冲区资源的基地址（以字节为单位）。                                                                                                                                                                                                                                                                                                                                                                                             |
| const_stride         | 14 或 18 | 记录的跨度（以字节为单位）（0 到 16,383 字节, 或 0 到 262,143 字节）。通常为 14 位, 但在以下情况下扩展为 18 位: const_add_tid_enable = true 与非格式类型（或缓存无效/WB）的 MUBUF 指令一起使用。这是旨在与暂存（私有）缓冲区一起使用的扩展。 <div> <pre> If (const_add_tid_enable &amp;&amp; MUBUF-non-format instr.)     const_stride [17:0] = { V#.DFMT[3:0],                           V#.const_stride[13:0] } else     const_stride is 14 bits: {4'b0, V#.const_stride[13:0]} </pre> </div> |
| const_num_records    | 32      | 缓冲区中的记录数。原始缓冲区以字节为单位, 结构化缓冲区以步幅为单位, 私有（临时）缓冲区则忽略。单位为: (inst_idxen == 1) ? 字节: 跨步                                                                                                                                                                                                                                                                                                                               |
| const_add_tid_enable | 1       | 布尔值。如果为 true, 则将波前内的 thread_ID 添加到索引。                                                                                                                                                                                                                                                                                                                                                                          |
| const_swizzle_enable | 1       | 布尔值。为 true 时表示表面已混合。                                                                                                                                                                                                                                                                                                                                                                                           |
| const_element_size   | 2       | 仅当 const_swizzle_en = true 时使用。给定索引的记录的连续字节数（2、4、8 或 16 字节）。必须 >= 结构中的最大元素大小。const_stride 必须是 const_element_size 的整数倍。                                                                                                                                                                                                                                                                                         |
| const_index_stride   | 2       | 仅当 const_swizzle_en = true 时使用。切换到下一个元素之前单个元素（const_element_size）的连续索引数。有 8、16、32 或 64 个索引。                                                                                                                                                                                                                                                                                                                    |

表 45. GPRs 的地址组件

| 字段          | 大小 | 说明                          |
|-------------|----|-----------------------------|
| SGPR_offset | 32 | 地址的无符号字节偏移量。来自 SGPR 或 M0。   |
| VGPR_offset | 32 | 可选的无符号字节偏移量。它是每线程的，来自 VGPR。 |
| VGPR_index  | 32 | 可选的索引值。它是按线程的，来自 VGPR。      |

最终的缓冲内存地址由三部分组成：

- 缓冲区资源的基地址 (V#)，
- 与 SGPR 的偏移量，以及
- 缓冲区偏移量的计算方式不同，具体取决于缓冲区是线性寻址（简单的结构数组计算）还是混合。

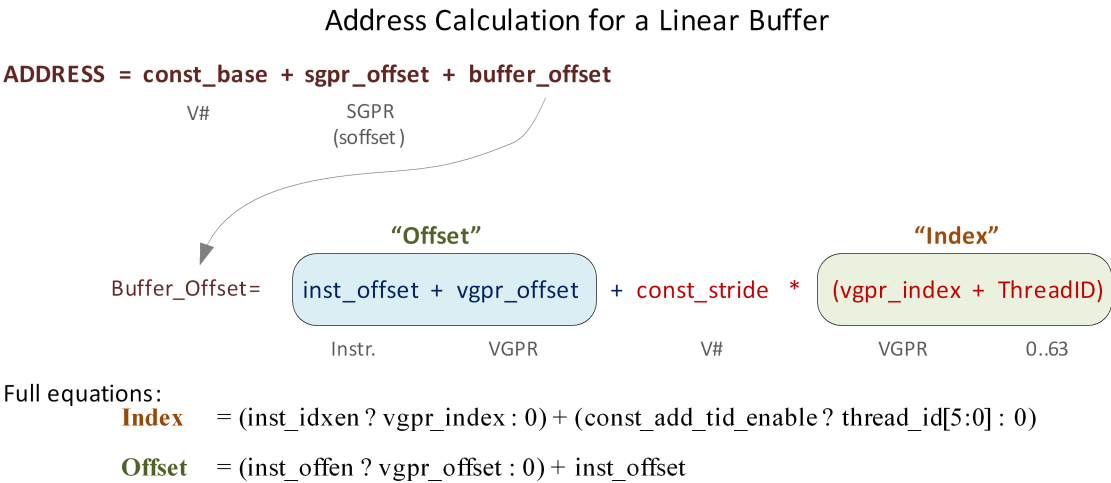


图 4. 线性缓冲区的地址计算

9.1.5.1.范围检查

地址可以被检查以查看其是否在范围之内。当地址超出范围时, 读回零, 并删除写入和算术。地址范围检查方法取决于缓冲类型。

私人( 刮伤) 缓冲

适用条件: AddTID==1 && IdxEn==0。此  
类缓冲区不执行范围检查。

原始缓冲

适用条件: AddTID==0 && SWizzleEn==0 && IdxEn==0。越界条  
件: (InstOffset + (OffEN ? vgpr\_offset : 0)) >= NumRecords。

结构缓冲

适用条件: AddTID==0 && Stride!=0 && IdxEn=  
=1。越界条件: Index(vgpr) >= NumRecords。

注释:

1. 读取离程返回零( V#.dst\_sel = SEL\_1 = 返回 1 的组件除外)。

2. 异域写作不写作.
3. 装入/存储格式-\* 指令和线条被检查范围为“全部或无”——要么完全输入或退出。
4. 装入/存储-Dword-x{2,3,4}和范围检查每个组件.

### 9.1.5.2. 重排缓冲区寻址

Swizzled 地址重排缓冲器中的数据,以帮助为数组结构提供更好的缓存位置。Swizzled 地址也需要 Dword-access 访问。单一的获取指令不能试图获取大于 Const- element- 大小的单位。缓冲器的 STRIDE 必须是 element\_size 的倍数。

```
Index = (inst_idxen ? vgpr_index : 0) +
        (const_add_tid_enable ? thread_id[5:0] : 0)

Offset = (inst_offen ? vgpr_offset : 0) + inst_offset

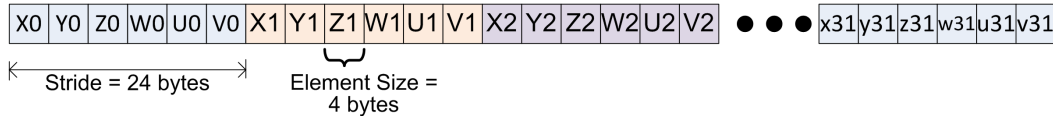
index_msb = index / const_index_stride
index_lsb = index % const_index_stride
offset_msb = offset / const_element_size
offset_lsb = offset % const_element_size

buffer_offset = (index_msb * const_stride + offset_msb *
                 const_element_size) * const_index_stride + index_lsb *
                 const_element_size + offset_lsb

Final Address = const_base + sgpr_offset + buffer_offset
```

记住"sgpr\_offset"不是上述方程式中"offset"术语的一部分.

## Original Buffer



## Swizzled Buffer

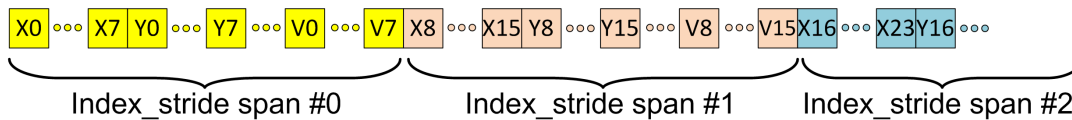
`const_index_stride = 8` // how many consecutive indices to group together  
`const_element_size = 4 bytes` // the size of a single element, in bytes

`index_msb = index / const_index_stride`  
`index_lsb = index % const_index_stride`  
`offset_msb = offset / const_element_size`  
`offset_lsb = offset % const_element_size`

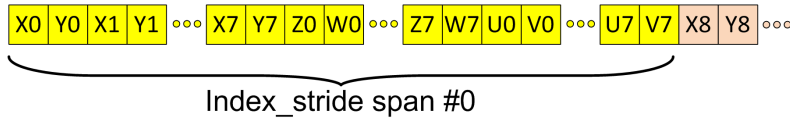
`Buffer_offset = (index_msb * const_stride + offset_msb * const_element_size) * const_index_stride + index_lsb * const_element_size + offset_lsb`

which simplifies to...

`Buffer_offset = (index/8 * const_stride + (offset/4)*4) * 8 + index%8 * 4 + offset%4`  
*Note that because we are dealing with dwords, offset%4 is always == 0.*



**If the const\_element\_size had been 8 :**



## An alternate way to visualize Swizzled Buffers

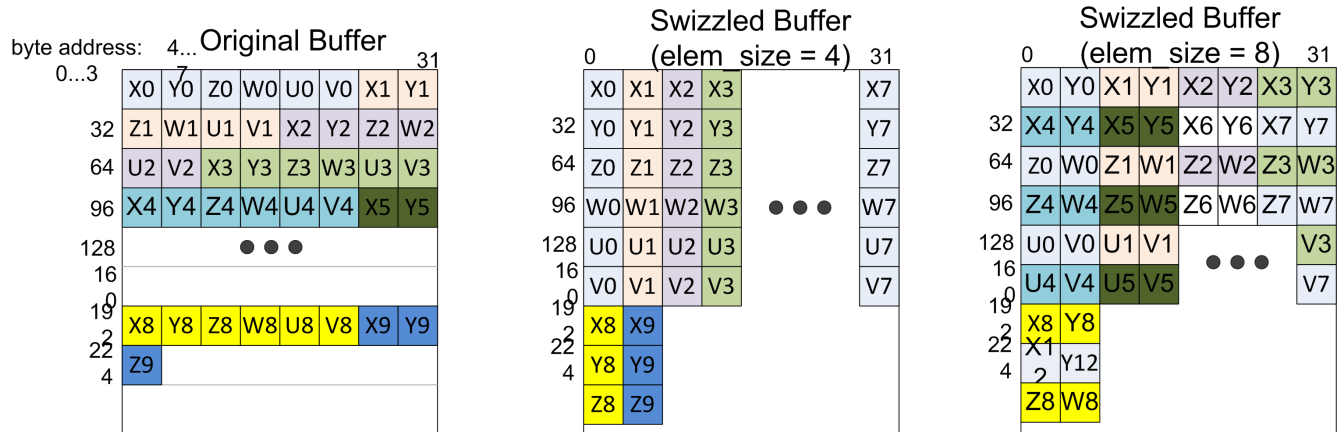


图 5. 缓冲冲击的实例

9.1.5.3. 拟用案例

这里很少提出在常见图形缓冲器中使用 swizzled 地址。

表 46. 破损的缓冲器使用案例

|                          | DX11 Raw<br>Uav OpenCL<br>缓冲对象 | Dx11 结构化缓冲<br>区（字面量偏移） | Dx11 结构化(克/<br>四) | 暂存     | 环/流出 | 压缩缓冲 |
|--------------------------|--------------------------------|------------------------|-------------------|--------|------|------|
| inst_vgpr_offset_e<br>n  | T                              | F                      | T                 | T      | T    | T    |
| inst_vgpr_index_en       | F                              | T                      | T                 | F      | F    | F    |
| const_stride             | 纳                              | <api>                  | <api>             | 抓痕大小   | 纳    | 纳    |
| const_add_tid_ena<br>ble | F                              | F                      | F                 | T      | T    | F    |
| const_buffer_swizz<br>le | F                              | T                      | T                 | T      | F    | F    |
| const_elem_size          | 纳                              | 4                      | 4                 | 4 或 16 | 纳    | 4    |
| const_index_stride       | 纳                              | 16                     | 16                | 64     |      |      |

9.1.6. 16 位内存操作

D16 缓冲区指令允许核函数在 VGPRs 与内存之间为每个工作项仅加载或存储 16 位数据。这些指令有两种变体：

- D16 将数据加载或存储到 VGPR 的下 16 位数据.
- D16\_HI 将数据加载或存储到 VGPR 上 16 位的数据.

例如,BUFFER\_LOAD\_UBYTE\_D16 从内存中读取每个工作项的字节,将其转换成 16 位整数,然后加载到数据 VGPR 的下 16 位.

当启用 ECC 时, 16 位内存负载会写入完整的 32 位 VGPR 。 未使用的位数被设定为 0 。

9.1.7. 对齐

对于 Dword 或更大的读或写,字节地址的两个 LSB 被忽略,从而迫使 Dword 对齐.

9.1.8. 缓冲区资源

缓冲资源描述缓冲器在内存中的位置和缓冲器中的数据格式,在连续的 4 个 SGPRs(4 个对齐的 SGPRs)中指定,并发送到每个缓冲指令的纹理缓冲处.

下表详细列出构成缓冲资源描述符的字段.

表 47. 缓冲资源描述符



| 位       | 大小 | 名称          | 说明                                                                          |
|---------|----|-------------|-----------------------------------------------------------------------------|
| 47:0    | 48 | 基地地址        | 字节地址.                                                                       |
| 61:48   | 14 | 弦乐          | 字节 0 到 16383                                                                |
| 62      | 1  | 缓存摇摆        | 缓冲访问,可选,Swizzle 纹理缓存 TC L1 缓存银行.                                            |
| 63      | 1  | Swizzle 启用  | Swizzle AOS 根据脚步,index_stride,和 element_size,其他线性(脚步 *指数+偏移).               |
| 95:64   | 32 | Num_records | 以步数或字节表示。                                                                   |
| 98:96   | 3  | Dst_sel_x   | 目标频道选择:0=0,1=1,4=R,5=G,6=B,7=A                                              |
| 101:99  | 3  | Dst_sel_y   |                                                                             |
| 104:102 | 3  | Dst_sel_z   |                                                                             |
| 107:105 | 3  | Dst_sel_w   |                                                                             |
| 110:108 | 3  | 数字格式        | 数字数据类型( float, int,...) 。 参见值的指令编码 。                                        |
| 114:111 | 4  | 数据格式        | 每个字段的字段数和大小。请参见值的指令编码。对于带有 ADD_TID_EN = 1. 此字段持有 Stride [17:14] 的 MUBUF 指令。 |
| 115     | 1  | 用户 VM 启用    | 资源通过瓦片池/堆积来映射 。                                                             |
| 116     | 1  | 用户 VM 模式    | 未实现的行为 : 0: 无效( 返回 0/ 丢弃写入); 1: 无效( 错误结果)                                   |
| 118:117 | 2  | 索引脚步        | 8, 16, 32 或 64. 用于 swizzle 缓冲地址.                                            |
| 119     | 1  | 添加 tid 启用   | 在索引中添加线程 ID 以计算地址 。                                                         |
| 122:120 | 3  | RSVD        | 预备,必须定在 0                                                                   |
| 123     | 1  | NV          | 非挥发性(0=挥发性)                                                                 |
| 125:124 | 2  | RSVD        | 预备,必须定在 0                                                                   |
| 127:126 | 2  | 类型          | 缓冲区的该值必须为 0。该字段与 128 位 T# 资源中四位 TYPE 字段的高两位重叠。                              |

所有零的资源集都起到无约束的纹理或缓冲作用(返回 0,0,0).

### 9.1.9. 从内存缓冲区加载到 LDS

MUBUF 指令格式允许从内存缓冲器直接读入 LDS,而不通过 VGPRs. 这为 MUBUF 指令的以下子集所支持.

- BUFFER\_LOAD\_{ubyte, sbyte, ushort, sshort, dword, format\_x}.

LDS\_offset = 16 位未署名字节从 M0[15:0]. Mem\_offset = 32 位未署名字节从 SGPR ( SOFFSET SGPR). idx\_vgpr = 指数值从 VGPR (位于 VADDR). (Zero if idxen=0.) off\_vgpr = 抵消值从 VGPR (位于 VADDR 或 VADDR+1). (Zero if offen=0.

下图显示了 LDS 的组件和内存地址的计算:

$$\text{LDS\_ADDR} = \text{LDSbase} + \text{LDS\_offset} + \text{inst\_offset} + (\text{TIDinWave} * 4)$$

HW-Alloc    M0[15:0]    Instr.    0..63    bytes-per-dword

$$\text{MEM\_ADDR} = \text{Base} + \text{mem\_offset} + \text{inst\_offset} + \text{off\_vgpr} + \text{stride} * (\text{idx\_vgpr} + \text{TIDinWave})$$

T#    SGPR (soffset)    Instr.    VGPR    T#    VGPR    0..63

Zero – no vgpr    4 bytes    Zero

对于内存地址，只有资源 (T#) 的 ADD\_TID\_ENABLE 字段为 1 时才加入 TIDinWave；LDS 地址则会加入该值。MEM\_ADDR 中的 M0 由 VDATA 字段指定。

### 9.1.9.1. 边界限制规则

内存地址夹与其它缓冲获取遵循相同的规则。LDS 地址夹:返回数据不得写入分配到此波的 LDS 空间之外。

- 设定活动任务以限制缓冲读取那些将数据返回到合法 LDS 位置的线程。
- LDSbase (alloc) 以 32 个双字为单位，LDSsize 也使用相同单位。

## 9.1.10. 内存作用域与时间局部性控制

### 9.1.10.1. 斯卡尔记忆

标量 Memory 指令有一个单一的内存控制位点: GLC(Global Coherent). GLC 比特表示负载,存储,和原子操作的不同事物。

#### READ

GLC = 0 读数可以击中 L1,并横跨波头持续着 GLC = 1 读数错过 L1 和 L2,并强制获取到数据结构. No L1 横跨波头持续着.

#### WRITE

GLC = 0 Writes miss the L1 写到 L2 并持续在 L1 横跨波头. GLC = 1 Writes miss the L1 写到 L2 横跨波头没有坚持.

#### ATOMIC

GLC = 0 先前的数据值不返回. 无 L1 跨越波头的持久性. GLC = 1 先前的数据值返回. L1 跨越波头的持久性. 注: GLC 表示对动漫的"返回前置值".

### 9.1.10.2. 病媒记忆

向量内存指令(Flat,Global,Scratch,和 Buffer)有 3 位来控制范围和缓存性:

- SC[1:0]系统缓存级别:0=波,1=组,2=设备,3=系统
- NT 非时间:0=预计时间再利用;1=不期望时间再利用

装入

表 48. 负载控制

| 范围   | SC1 | SC0 | NT | CU 缓存行为 | L2 缓存行为                                     | 最后一级缓存行为 |
|------|-----|-----|----|---------|---------------------------------------------|----------|
| 挥手   | 0   | 0   | 0  | 命中 LRU  | 命中 LRU                                      | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | 点击流                                         | 点击 Evict |
| 组    | 0   | 1   | 0  | 命中 LRU  | 命中 LRU                                      | 点击 Evict |
|      |     |     | 1  | 埃维特小姐   | 点击流                                         | 点击 Evict |
| 设备   | 1   | 0   | 0  | 埃维特小姐   | (1 L2 缓存):按 LRU;( > 1 L2 缓存):一致的缓存旁通        | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | (1 L2 缓存): Hit Stream;( > 1 L2 缓存): 和谐的缓存旁通 | 点击 Evict |
| 丝丝微米 | 1   | 1   | 0  | 埃维特小姐   | 同步缓存副路口                                     | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | 同步缓存副路口                                     | 点击 Evict |

注意,如果 TG\_SPLIT 模式是活跃的,则 SC1==0,SC0==1,NT==0 案例变为:LRU 小姐,Hit LRU,Hit LRU.

存储

表 49. 库存控制

| 范围   | SC1 | SC0 | NT | CU 缓存行为 | L2 缓存行为                                     | 最后一级缓存行为 |
|------|-----|-----|----|---------|---------------------------------------------|----------|
| 挥手   | 0   | 0   | 0  | 吕小姐     | 命中 LRU                                      | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | 点击流                                         | 点击 Evict |
| 组    | 0   | 1   | 0  | 吕小姐     | 命中 LRU                                      | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | 点击流                                         | 点击 Evict |
| 设备   | 1   | 0   | 0  | 埃维特小姐   | (1 L2 缓存):按 LRU;( > 1 L2 缓存):一致的缓存旁通        | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | (1 L2 缓存): Hit Stream;( > 1 L2 缓存): 和谐的缓存旁通 | 点击 Evict |
| 丝丝微米 | 1   | 1   | 0  | 埃维特小姐   | 同步缓存副路口                                     | 命中 LRU   |
|      |     |     | 1  | 埃维特小姐   | 同步缓存副路口                                     | 点击 Evict |

原子操作

对于 atomic,SC0 表示是否返回"pre-op"内存值(原子操作之前的内存值). 0 = 不返回, 1 = 返回 pre-op 值.

SC1:0 = 装置范围原子,1 = 系统范围原子

NT : 0 = 最后一关缓存"分配"策略; 1 = "不分配"策略

无效并重写

表 50. BUFFER\_WBL2

| SC1 | SC0 | L2 缓存行为                         |
|-----|-----|---------------------------------|
| 0   | 任何  | NOP                             |
| 1   | 0   | (1 L2 缓存):NOP, (>1 L2 缓存) 回写脏数据 |
| 1   | 1   | 写回肮脏的数据                         |

表 51. BUFFER\_INV

| SC1 | SC0 | CU 缓存行为                             | L2 缓存行为                               |
|-----|-----|-------------------------------------|---------------------------------------|
| 0   | 0   | NOP                                 | NOP                                   |
| 0   | 1   | (如果 TG 拆分): 无效缓存, (如果不是 TG 拆分): NOP | NOP                                   |
| 1   | 0   | 无效缓存                                | (1 L2 缓存):NOP, ( > 1 L2 缓存) 无效的非一致缓存行 |
| 1   | 1   | 无效缓存                                | 无效的非一致缓存行                             |

9.1.11. 数据格式

下表显示了缓冲数据格式:

| DATA_FORMAT    |          | NUM_FORMAT |         |  | DATA_FORMAT    |             | NUM_FORMAT |         |
|----------------|----------|------------|---------|--|----------------|-------------|------------|---------|
| value          | enum     | value      | enum    |  | value          | enum        | value      | enum    |
| Buffer formats |          |            |         |  | Buffer formats |             |            |         |
| 0              | INVALID  | 0          | Any     |  | 8              | 10_10_10_2  | 0          | UNORM   |
| 1              | 8        | 0          | UNORM   |  | 8              | 10_10_10_2  | 1          | SNORM   |
| 1              | 8        | 1          | SNORM   |  | 8              | 10_10_10_2  | 2          | USCALED |
| 1              | 8        | 2          | USCALED |  | 8              | 10_10_10_2  | 3          | SSCALED |
| 1              | 8        | 3          | SSCALED |  | 8              | 10_10_10_2  | 4          | UINT    |
| 1              | 8        | 4          | UINT    |  | 8              | 10_10_10_2  | 5          | SINT    |
| 1              | 8        | 5          | SINT    |  | 9              | 2_10_10_10  | 0          | UNORM   |
| 1              | 8        | 9          | SRGB    |  | 9              | 2_10_10_10  | 1          | SNORM   |
| 2              | 16       | 0          | UNORM   |  | 9              | 2_10_10_10  | 2          | USCALED |
| 2              | 16       | 1          | SNORM   |  | 9              | 2_10_10_10  | 3          | SSCALED |
| 2              | 16       | 2          | USCALED |  | 9              | 2_10_10_10  | 4          | UINT    |
| 2              | 16       | 3          | SSCALED |  | 9              | 2_10_10_10  | 5          | SINT    |
| 2              | 16       | 4          | UINT    |  | 10             | 8_8_8_8     | 0          | UNORM   |
| 2              | 16       | 5          | SINT    |  | 10             | 8_8_8_8     | 1          | SNORM   |
| 2              | 16       | 7          | FLOAT   |  | 10             | 8_8_8_8     | 2          | USCALED |
| 3              | 8_8      | 0          | UNORM   |  | 10             | 8_8_8_8     | 3          | SSCALED |
| 3              | 8_8      | 1          | SNORM   |  | 10             | 8_8_8_8     | 4          | UINT    |
| 3              | 8_8      | 2          | USCALED |  | 10             | 8_8_8_8     | 5          | SINT    |
| 3              | 8_8      | 3          | SSCALED |  | 10             | 8_8_8_8     | 9          | SRGB    |
| 3              | 8_8      | 4          | UINT    |  | 11             | 32_32       | 4          | UINT    |
| 3              | 8_8      | 5          | SINT    |  | 11             | 32_32       | 5          | SINT    |
| 3              | 8_8      | 9          | SRGB    |  | 11             | 32_32       | 7          | FLOAT   |
| 4              | 32       | 4          | UINT    |  | 12             | 16_16_16_16 | 0          | UNORM   |
| 4              | 32       | 5          | SINT    |  | 12             | 16_16_16_16 | 1          | SNORM   |
| 4              | 32       | 7          | FLOAT   |  | 12             | 16_16_16_16 | 2          | USCALED |
| 5              | 16_16    | 0          | UNORM   |  | 12             | 16_16_16_16 | 3          | SSCALED |
| 5              | 16_16    | 1          | SNORM   |  | 12             | 16_16_16_16 | 4          | UINT    |
| 5              | 16_16    | 2          | USCALED |  | 12             | 16_16_16_16 | 5          | SINT    |
| 5              | 16_16    | 3          | SSCALED |  | 12             | 16_16_16_16 | 7          | FLOAT   |
| 5              | 16_16    | 4          | UINT    |  | 13             | 32_32_32    | 4          | UINT    |
| 5              | 16_16    | 5          | SINT    |  | 13             | 32_32_32    | 5          | SINT    |
| 5              | 16_16    | 7          | FLOAT   |  | 13             | 32_32_32    | 7          | FLOAT   |
| 6              | 10_11_11 | 7          | FLOAT   |  | 14             | 32_32_32_32 | 4          | UINT    |
| 7              | 11_11_10 | 7          | FLOAT   |  | 14             | 32_32_32_32 | 5          | SINT    |
|                |          |            |         |  | 14             | 32_32_32_32 | 7          | FLOAT   |

## 9.2. 浮点内存原子操作

浮点内存存在 LDS 和 L2 缓存中执行,它们可以作为 LDS , 缓冲器, 平面, 环球, 和刷新指令发射。

本章解释了四舍五入,非常态和 NaN 对浮点在动漫中的规则.

### 9.2.1. 浮点原子操作的舍入

所有浮原子的 ADD opcode 都使用“近端偶数”的圆形。

### 9.2.2. 非规格化数（次正规数）处理

当原子学在浮点数据上运行时,有可能存在包含异常数的数据,或者产生异常数的操作.

异常现象:浮点原子指令可以选择将异常值传递到零,或者将其冲到零。这可以控制 MODE.denorm 比特,该比特也控制 VALU 异常行为。和 VALU 的操作一样,“denorm\_single”会影响 F32 的操作,“denorm\_double”会影响 F64 和 F16。有些动漫已经固定了非正常处理行为。

LDS 指令允许以 MODE.deural wave-state records 为基础通过或冲到零.

- Float 16 和 32 位 Adder 使用 MODE 的输入和输出调色器冲洗控制
- 浮点 64 位添加器不冲洗调色器
- Float CMP、MIN 和 MAX 只使用“输入异常”冲洗控制
  - 比较时每次输入都会在比较前将两个操作的 mantissa 冲到 0, 如果该函数为 0, 且该函数的去诺姆控制是活动的。对 Min 和 Max 来说, 返回的实际结果就是选定的非流化输入。比较 Store(“比较互换”)在输入非正常冲洗时将结果冲走。
  -

表 52. 内存操作处理规则

| 原子类型              | LDS 处理       | L2 缓存处理      |
|-------------------|--------------|--------------|
| PK_ADD_F16 / BF16 | 模式           | 切勿 Flush 调色器 |
| ADD_F32           | 模式           | 毛发变色体        |
| 民/MAX_F32         | 模式           | N/A          |
| CMPST_F32         | 模式           | N/A          |
| MIN/MAX_F64       | 模式           | 切勿 Flush 调色器 |
| CMPST_F64         | 模式           | N/A          |
| ADD_F64           | 切勿 Flush 调色器 | 切勿 Flush 调色器 |

- “Flush Denorms” = 冲洗所有输入调值
- “不要花” = 不冲洗输入值
- “模式” = 由遮蔽器的 “MODE. fp\_denorm”寄存器的位点控制的异常冲浪
- “模式+正则” = 上方的“模式”,但是否存在用于冲洗输出的覆盘。

请注意, MIN 和 MAX 冲洗异常时仅用于比较,但结果是一个源的未修改副本。比较 Store (compare switch) 当输入异常的冲洗发生时, 冲洗结果。

### 9.2.3. NaN 处理

A 数字(“NaN”)是不能计算的 IEEE-754 数值。

有两种 NaN:安静和信号

- 安静 NaN 指数=0xFF 曼提萨 MSB = 1
- 信号 NaN 指数=0xFF,曼提萨 MSB=0,至少还有一个曼提萨比特==1

LDS 没有因信号 NaN 而产生任何例外或"信号".

DS\_ADD\_F32 可以创建静态的 NaN,也可以从输入中传播 NaN:如果输入中任何一个 NaN,输出是 NaN,如果两个输入是 NaN,则从第一个输入中选择 NaN 作为输出. Signaling NaN 转换为静态 NaN.

浮点算术( CMPSWAP, MIN, MAX) 冲洗输入异常值, 只有当 MODE (allow\_input\_denorm) = 0 时, 否则值会通过而不修改。 当冲洗时, 冲洗在操作前( 即比较前)。

FP 最大选择规则：

如果(src0 == SNaN)结果 = QNaN (src0) 如果(src1 == SNaN)结果 = QNaN (src1) 其它结果 = 大于(src0,src1) “拉尔夫” 顺序从最小到最大:QNaN, -inf, -f, -float, -denorm, -0,+0,+denorm,+float,+inf

FP 民选规则:

若 src0 == SNaN, 则 result = QNaN(src0); 否则, 若 src1 == SNaN, 则 result = QNaN(src1); 否则, 结果取 src0 与 src1 中较小者。 “较小” 的排序从小到大为: -inf、-float、-denorm、-0、+0、+denorm、+float、+inf、QNaN。

FP 比较交换:只有在比较条件( ==) 属实时才进行交换, 将+0 和 -0 等同

doSwap=(src0!=NaN) &&(src1!=NaN) &&(src0==)/(src0==) /// 允许+0== -0.

浮动添加规则：

1. -INF + INF = QNaN(曼提萨是除 MSB 以外的全部零)
2. +/-INF + NAN = QNaN(NAN 输入被复制到输出中,但使静态 NAN)
3. -0 + 0 = +0
4. INF +(浮点数,+0, -0) = INF,保留了无限标志
5. NaN + NaN = SRC0 的 NaN ,转换为 QNaN

# 第 10 章 平坦内存指令

平坦内存指令读取或写入 VGPRs 的一块数据; 平坦内存指令在波浪前端的每个工作项上单独进行。平坦内存指令与缓冲或图像指令不同, 不使用资源常数来定义表面的基址。相反, 平坦内存指令使用 VGPR 的单一平面地址; 将内存作为单一平坦内存空间处理。这个内存空间包括视频内存、系统内存、LDS 内存和抓取(私用)内存。平坦内存空间的部分可能不会映射到任何真实内存, 访问这些区域会产生内存- 侵犯错误。确定地址地图的内存空间时, 由一组“模具孔径”基和大小寄存器控制。

## 10.1. 平坦内存指令

平坦内存指令允许核函数在内存中读取或写入数据,或在已存在内存中的数据上执行原子操作。这些操作通过纹理 L2 缓存进行。该指令宣布 VGPR 持有地址(视内存配置而定,为 32 位或 64 位),发送的 VGPR 和接收数据的 VGPR。平面指令还使用 M0,如下表所示:

表 53. 平面、全局和刷新微码格式

| 字段     | 位大小 | 说明                                                                                                                                           |
|--------|-----|----------------------------------------------------------------------------------------------------------------------------------------------|
| OP     | 7   | Opcode。可以是 Flat、Scratch 或 Global 指令。请见下一张表格。                                                                                                 |
| ADDR   | 8   | VGPR 持有地址。对于 64 位地址, ADDR 拥有 LSB , ADDR+1 拥有 MSB 。                                                                                           |
| DATA   | 8   | VGPR,它持有数据的第一个 Dword. 指令可以使用 0-4 Dwords.                                                                                                     |
| VDST   | 8   | 数据返回核函数的 VGPR 目的地,从 LOADs 或原子返回,数据为 SC[0]=1(返回前输出值).                                                                                         |
| SC     | 2   | 内存范围                                                                                                                                         |
| NT     | 1   | 非时间性                                                                                                                                         |
| ACC    | 1   | DATA 是累积 VGPR                                                                                                                                |
| SEG    | 2   | 内存段:0=FLAT,1=SCRATCH,2=GLOBAL,3=保留.                                                                                                          |
| SVE    | 1   | Scratch VGPR 启用 - 表示 VGPR 是否有助于计算抓取内存地址。                                                                                                     |
| NV     | 1   | 非挥发性。设定时, 读/ 写正在非挥发性内存上操作。                                                                                                                   |
| OFFSET | 13  | 地址偏移. Scratch, Global: 13 位签名字节偏移. Flat: 12 位无符号的偏移(MSB 被忽略).                                                                                |
| SADDR  | 7   | 提供偏移地址的标量 SGPR. 要禁用此字段, 请设置为 0x7F. 此字段的含义与 Scratch 和 Global: Flat: 未使用. scratch: 使用 SGPR(代替 VGPR) 地址. Global: 使用 SGPR 提供基址; VGPR 提供 32 位的偏移。 |
| M0     | 16  | SCRATCH 和 GLOBAL 在 LDS=1.提供 LDS 的地址抵销时,不慎使用 M0.                                                                                              |

表 54. 平面、全球和切片 Opcode

| 平面 Opcode 码            | 全球 Opcode 编码             | 切换 Opcode 码               |
|------------------------|--------------------------|---------------------------|
| FLAT                   | GLOBAL                   | SCRATCH                   |
| FLAT_LOAD_UBYTE        | GLOBAL_LOAD_UBYTE        | SCRATCH_LOAD_UBYTE        |
| FLAT_LOAD_UBYTE_D16    | GLOBAL_LOAD_UBYTE_D16    | SCRATCH_LOAD_UBYTE_D16    |
| FLAT_LOAD_UBYTE_D16_HI | GLOBAL_LOAD_UBYTE_D16_HI | SCRATCH_LOAD_UBYTE_D16_HI |
| FLAT_LOAD_SBYTE        | GLOBAL_LOAD_SBYTE        | SCRATCH_LOAD_SBYTE        |
| FLAT_LOAD_SBYTE_D16    | GLOBAL_LOAD_SBYTE_D16    | SCRATCH_LOAD_SBYTE_D16    |





| 平面 Opcode 码                 | 全球 Opcode 编码              | 切换 Opcode 码                |
|-----------------------------|---------------------------|----------------------------|
| FLAT_LOAD_SBYTE_D16_HI      | GLOBAL_LOAD_SBYTE_D16_HI  | SCRATCH_LOAD_SBYTE_D16_HI  |
| FLAT_LOAD_USHORT            | GLOBAL_LOAD_USHORT        | SCRATCH_LOAD_USHORT        |
| FLAT_LOAD_SSHORT            | GLOBAL_LOAD_SSHORT        | SCRATCH_LOAD_SSHORT        |
| FLAT_LOAD_SHORT_D16         | GLOBAL_LOAD_SHORT_D16     | SCRATCH_LOAD_SHORT_D16     |
| FLAT_LOAD_SHORT_D16_HI      | GLOBAL_LOAD_SHORT_D16_HI  | SCRATCH_LOAD_SHORT_D16_HI  |
| FLAT_LOAD_DWORD             | GLOBAL_LOAD_DWORD         | SCRATCH_LOAD_DWORD         |
| FLAT_LOAD_DWORDX2           | GLOBAL_LOAD_DWORDX2       | SCRATCH_LOAD_DWORDX2       |
| FLAT_LOAD_DWORDX3           | GLOBAL_LOAD_DWORDX3       | SCRATCH_LOAD_DWORDX3       |
| FLAT_LOAD_DWORDX4           | GLOBAL_LOAD_DWORDX4       | SCRATCH_LOAD_DWORDX4       |
| FLAT_STORE_BYTE             | GLOBAL_STORE_BYTE         | SCRATCH_STORE_BYTE         |
| FLAT_STORE_BYTE_D16_HI      | GLOBAL_STORE_BYTE_D16_HI  | SCRATCH_STORE_BYTE_D16_HI  |
| FLAT_STORE_SHORT            | GLOBAL_STORE_SHORT        | SCRATCH_STORE_SHORT        |
| FLAT_STORE_SHORT_D16_HI     | GLOBAL_STORE_SHORT_D16_HI | SCRATCH_STORE_SHORT_D16_HI |
| FLAT_STORE_DWORD            | GLOBAL_STORE_DWORD        | SCRATCH_STORE_DWORD        |
| FLAT_STORE_DWORDX2          | GLOBAL_STORE_DWORDX2      | SCRATCH_STORE_DWORDX2      |
| FLAT_STORE_DWORDX3          | GLOBAL_STORE_DWORDX3      | SCRATCH_STORE_DWORDX3      |
| FLAT_STORE_DWORDX4          | GLOBAL_STORE_DWORDX4      | SCRATCH_STORE_DWORDX4      |
| FLAT_ATOMIC_SWAP            | GLOBAL_ATOMIC_SWAP        | 无                          |
| FLAT_ATOMIC_CMPSWAP         | GLOBAL_ATOMIC_CMPSWAP     | 无                          |
| FLAT_ATOMIC_ADD             | GLOBAL_ATOMIC_ADD         | 无                          |
| FLAT_ATOMIC_SUB             | GLOBAL_ATOMIC_SUB         | 无                          |
| FLAT_ATOMIC_SMIN            | GLOBAL_ATOMIC_SMIN        | 无                          |
| FLAT_ATOMIC_UMIN            | GLOBAL_ATOMIC_UMIN        | 无                          |
| FLAT_ATOMIC_SMAX            | GLOBAL_ATOMIC_SMAX        | 无                          |
| FLAT_ATOMIC_UMAX            | GLOBAL_ATOMIC_UMAX        | 无                          |
| FLAT_ATOMIC_AND             | GLOBAL_ATOMIC_AND         | 无                          |
| FLAT_ATOMIC_OR              | GLOBAL_ATOMIC_OR          | 无                          |
| FLAT_ATOMIC_XOR             | GLOBAL_ATOMIC_XOR         | 无                          |
| FLAT_ATOMIC_INC             | GLOBAL_ATOMIC_INC         | 无                          |
| FLAT_ATOMIC_DEC             | GLOBAL_ATOMIC_DEC         | 无                          |
| FLAT_ATOMIC_ADD_F32         | GLOBAL_ATOMIC_ADD_F32     | 无                          |
| FLAT_ATOMIC_PK_ADD_F16      | GLOBAL_ATOMIC_PK_ADD_F16  | 无                          |
| FLAT_ATOMIC_PK_ADD_BF16     | GLOBAL_ATOMIC_PK_ADD_BF16 | 无                          |
| FLAT_ATOMIC_ADD_F64         | GLOBAL_ATOMIC_ADD_F64     | 无                          |
| FLAT_ATOMIC_MIN_F64         | GLOBAL_ATOMIC_MIN_F64     | 无                          |
| FLAT_ATOMIC_MAX_F64         | GLOBAL_ATOMIC_MAX_F64     | 无                          |
| 上面的非浮原子指示也以"QQ2"版本(64 位)提供. |                           |                            |

表 55. SVE 位点

| SADDR     | SVE 模式 |                                                                                     |
|-----------|--------|-------------------------------------------------------------------------------------|
| ==EXEC_HI | 0      | ST : 添加器 = flat_scratch + swizzle(inst.offset, 线程 ID)                               |
| !=EXEC_HI | 0      | SS: 添加器 = flat_scratch + swizzle(sgpr_offset + inst.offset, 线程 ID)                  |
| ==EXEC_HI | 1      | SV : 添加器 = flat_scratch + swizzle(vgpr_offset + inst.offset, 线程 ID)                 |
| !=EXEC_HI | 1      | SVS: addr = flat_scratch + swizzle(sgpr_offset + vgpr_offset + inst.offset, 线程 ID). |

## 10.2. 指令

FLAT 指令集与缓冲指令集几乎完全相同,但没有 FORMAT 的读写. FLAT 指令与缓冲指令不同,不能直接返回数据到 LDS,只能返回 VGPRs.

FLAT 指令不使用资源常数(V#),但是它们确实需要特定的 SGPR-pair 来保存抓取空间信息,以防任何线程的地址决心抓取空间. 详情请参见 Scratch 一节.

在内部,FLAT 指令既作为 LDS 指令,又作为缓冲指令执行;因此,VM\_CNT 和 LGKM\_CNT 均递增,在两者均被消减之前不视为完成,无法事先确定 FLAT 指令是只使用 LDS 还是 TA 内存空间.

### 10.2.1. 顺序约束

平面指令可以互相完成。 如果一个平面指令在 Texture 缓存中找到它的全部数据,而下一个指令在 LDS 中找到它的全部数据,第二个指令可能首先完成。 如果两个指令将数据返回到相同的 VGPR,则结果未知。

### 10.2.2. 重要时序要求

由于 FLAT 载荷的数据可以来自 LDS 或纹理缓存,并且由于这些单位有不同的后缀,VM\_CNT 和 LGKM\_CNT 计数器有潜在的赛跑条件,因此 FLAT 指令后唯一明智的 S\_WAITCNT 值为零。

## 10.3. 寻址

FLAT 指令支持 64 位和 32 位地址,地址大小使用模式寄存器(PTR32)设置,每波存储一个值的本地副本。

孔径检查的地址在 32 位和 64 位模式上有所不同;然而,这里没有覆盖。

64 位地址和 LSB 一起存放在 APDR 的 VGPR,而 MSB 则存放在 ADR+1 的 VGPR.

对于刮痕空间,纹理单元从 VGPR 中获取地址,并进行以下操作。

```
Address = VGPR[addr] + TID_in_wave * Size
          - private aperture base (in SH_MEM_BASES)
          + offset (from flat_scratch)
```

返回数据到 LDS 的指令地址是：

```
DWORDX1: LDS_ADDR = LDSbase(hw_alloc) + LDSoffset(M0[17:2] * 4) + INST.OFFSET + ThreadID*4
DWORDX4: LDS_ADDR = LDSbase(hw_alloc) + LDSoffset(M0[17:2] * 4) + INST.OFFSET + ThreadID*16
```

### 10.3.1. 原子操作

Float atomics 必须设置 SC[0]=0(无返回值).

内存数据结构中进行,因此已知它们是具有主机内存访问权限的原子.

FP32 原子操作将异常态冲为零,FP64 和 FP16 原子均不冲向异常态。圆环模式是固定的,“圆形到最近的偶数”。

### 10.4. Global 全局内存

全局指令与 Flat 指令类似,但程序员必须确保没有线程访问 LDS 空间;因此,全局指令没有使用 LDS 带宽.

全局指示提供两种类型的地址:

- $\text{Memory\_addr} = \text{VGPR 地址} + \text{指令偏移}.$
- $\text{Memory\_addr} = \text{SGPR 地址} + \text{VGPR 偏移} + \text{指令偏移}.$

地址部分的宽度由 ADDRESS\_MODE 决定,可使用 32 位或 64 位指针; VGPR 偏移为 32 位.

这些指令也支持在 LDS 与内存之间直接传输数据,无需经过 VGPRs.

由于这些指令不访问 LDS,所以只使用 VM\_CNT,而不是 LGKM\_CNT. 如果一个全局指令确实试图访问 LDS,则该指令返回 MEM\_VIOL.

### 10.5. Scratch 暂存内存

擦拭指令与 Flat 类似,但程序员必须确保没有线程访问 LDS 空间,内存空间被擦拭,因此,抓取指令不会使用 LDS 带宽.

scratch 指令也支持多 Dword 访问和错误的对齐访问(尽管错误的对齐比较慢).

切除指令使用下列地址: 1.

- $\text{Memory\_addr} = \text{flat\_scratch.addr} + \text{swizzle}(\text{V/SGPR\_offset} + \text{inst\_offset}, \text{线程 ID}).$
- 抵消可以来自 SGPR 或 VGPR,是一个 32 位未署名的字节.

地址部分的宽度由 ADDRESS\_MODE 决定,可使用 32 位或 64 位指针; VGPR 偏移为 32 位.

这些指令也支持在 LDS 与内存之间直接传输数据,无需经过 VGPRs.

由于这些指令不访问 LDS,因此只使用 VM\_CNT,而不使用 LGKM\_CNT. 切片指令无法访问 LDS;因此没有做任何错误或孔径检查.

## 10.6. 数据

FLAT 指令可以在 VGPRs 和/或内存中连续使用 0 到 4 个 Dwords 的数据. DATA 字段决定哪个 VGPR(s)提供源数据(如果有的话),而 VDST VGPR 持有返回数据(如果有的话). 没有进行数据格式的转换.

## 10.7. 暂存空间 (私有)

刷新(thread-private memory)是孔径计数器定义的内存区域。当地址掉入抓取空间时,额外地址的计算会自动由硬件进行。核函数必须为这种计算提供附加信息,其形式为 FLAT\_SCRATCH 计数器。

FLAT\_SCRATCH 地址随 FLAT 请求自动发送.

FLAT\_SCRATCH 是一个 64 位,字节地址. 遮蔽器通过将两个单独的值加在一起构成值:基址,可以通过初始化的 SGPR 传递,或可能通过恒定缓冲器传递,以及每波分配抵消(也初始化为 SGPR).

# 第 11 章 数据共享操作

本地数据共享(LDS)是一个非常低纬度的 RAM 抓图,用于临时数据,其有效带宽至少比直接的,无纹的全球内存高一个数量级。它允许在一个工作组中工作项之间共享数据。与只读缓存不同,LDS 允许对内存空间进行高速的写到读重用(完全收集/读/载和散/写/存储操作)。

## 11.1. 概览

下图显示 LDS 的概念框架是用 OpenCL 集成到 AMD 加速器的内存中。

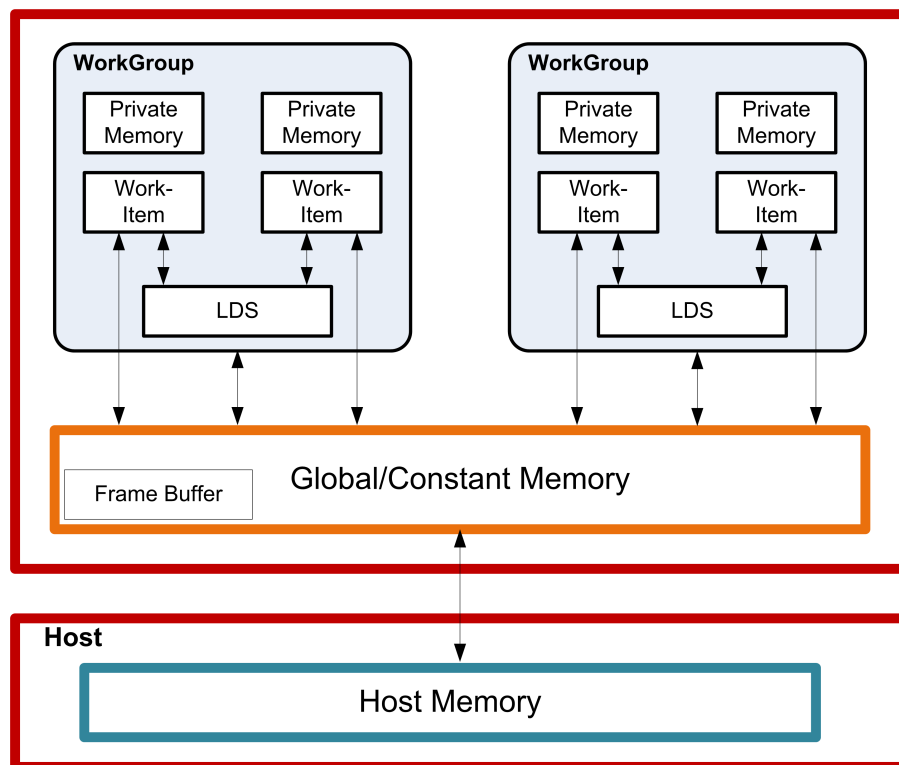


图 6. 高级别记忆配置

实际定位在芯片上,直接靠近 ALU,LDS 大约可以比全球内存快一个数量级(假设没有银行冲突)。

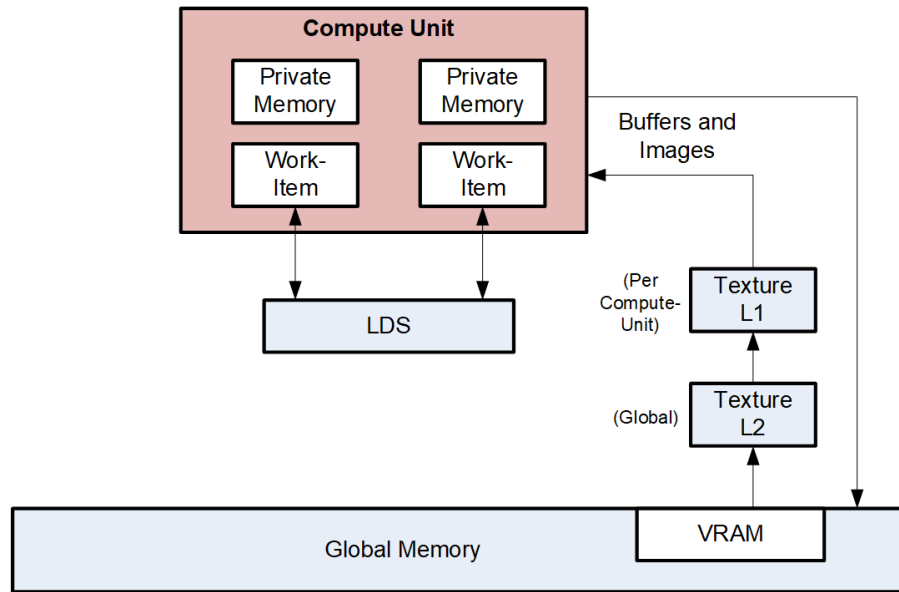
每个计算单元有 64 kB 内存, 分割为 32 个 512 Dwords 的银行, 每个银行有 32 位宽. Dwords 被串行放置在银行中, 但所有银行可以同时执行一个存储或加载。一个工作组可以请求最多 64 kB 内存。跨波前的读数在瀑布的四个循环中发送。

LDS 内存的高带宽不仅通过接近 ALU 实现,而且还通过同时访问其内存库实现。因此,可以同时执行 32 个书写或读取指令,每个名义上是 32 位;扩展指令,读作 2/write2,每个可以是 64 位。但是,如果同时对同一银行进行不止一个访问尝试,就会发生银行冲突。在这种情况下,对于索引操作和原子操作,硬件可以阻止试图同时访问同一银行的行为。

将它们变成串行访问。这会减少 LDS 的有效带宽。因此, 为了实现最佳的吞吐量(最佳效率), 必须避免银行冲突。了解请求调度和地址映射是帮助实现这一点的关键。

## 11.2. 内存层次中的数据流

下图是内存结构内数据流的概念图。



要从全球内存中将数据加载到 LDS 中,它要从全球内存中读取,并放入工作项寄存器中;然后,储存到 LDS 中。类似地,要将数据从 LDS 中读取并放入工作项寄存器中,然后放入全球内存中。为了有效地使用 LDS,一个核函数必须在全球内存和 LDS 之间转移的东西上执行许多操作。也可以直接从内存缓冲器中装入 LDS 中的数据,绕过 VGPRs。

LDS 在 LDS 硬件中进行。(因此,虽然 ALU 不是直接用于这些操作,但执行此功能的 LDS 会发生延迟。)

## 11.3. LDS 访问

LDS 是通过索引或原子指令访问的。

### 11.3.1. 数据共享存储的索引访问与原子访问

索引化和原子化操作提供 VGPRs 到 LDS 每个工作项的独特地址,并将每个工作项的独特数据提供或返回到 VGPRs。由于 LDS 的内部银行结构,根据银行冲突的数量(将这张地图映射到同一个内存库),操作最多只能完成两个周期,或者需要多达 64 个周期。

索引操作是简单的 LDS 加载和存储操作,从 VGPRs 读取数据并返回到 VGPRs。

原子操作是将 VGPRs 的数据和 LDS 的数据合并起来的算术操作,并将结果写回 LDS. 原子操作可以选择将 LDS"预 op"值还给 VGPRs.

下表列出并简要说明了 LDS 的教学领域.

表 56. LDS 教学场

| 字段                           | 大小        | 说明                                                                                                          |
|------------------------------|-----------|-------------------------------------------------------------------------------------------------------------|
| OP                           | 7         | LDS 操作码.                                                                                                    |
| GDS                          | 1         | 0 = LDS, 1 = GWS.                                                                                           |
| OFFSET0                      | 8         | 直接擦除, 以字节表示。带有一个地址的指令将擦除字段合并成一个单一的 16 位无符号的擦除 : {offset1, xif0}。带有两个地址的指令(例如: READ2) 将擦除作为两个 8 位无符号的擦除单独使用。 |
| OFFSET1                      | 8         |                                                                                                             |
|                              | VDS<br>T  | 8                                                                                                           |
| VGPR 是写着结果的:来自 LDS 负载或原子返回值. | ADD<br>R  | 8                                                                                                           |
| 提供字节地址的 VGPR 偏移.             | DAT<br>A0 | 8                                                                                                           |
| 提供第一数据源的 VGPR.               | DAT<br>A1 | 8                                                                                                           |
| 提供第二个数据源的 VGPR.              | ( M0<br>) | 32 默认使用 M0. M0[16:0] 包含 LDS 段的字节大小,用于夹住最终地址.                                                                |

LDS 所有操作都要求 M0 在使用前初始化. M0 包含一个大小值,可用于限制对分配的 LDS 范围的子集的访问,如果不需要夹击,则将 M0 设置为 0xFFFFFFFF.

表 57. LDS 指数载荷/仓库

| 装入/ 存储                                   | 说明                                                                                               |
|------------------------------------------|--------------------------------------------------------------------------------------------------|
| DS_READ_{B32,B64,B96,B128,U8,I8,U16,I16} | 每个线程读取一个值;如果签名,符号扩展为 Dword.                                                                      |
| DS_READ2_{B32,B64}                       | 在唯一地址读取两个值。                                                                                      |
| DS_READ2ST64_{B32,B64}                   | 在独有地址读取 2 个值; 请将 QQ 64.                                                                          |
| DS_WRITE_{B32,B64,B96,B128,B8,B16}       | 写一个值。                                                                                            |
| DS_WRITE2_{B32,B64}                      | 写两个值.                                                                                            |
| DS_WRITE2ST64_{B32,B64}                  | 写两个数值,乘以 QQ 64.                                                                                  |
| DS_WRXCHG2_RTN_{B32,B64}                 | 交换 GPR 与 LDS-memory.                                                                             |
| DS_WRXCHG2ST64_RTN_{B32,B64}             | 与 LDS-memory 交换 GPR;冲抵 QQ 64.                                                                    |
| DS_PERMUTE_B32                           | 正向置换, 实际不写入 LDS 内存。其置换语义可表示为: LDS[dst] = src0; returnVal = LDS[thread_id], 其中 thread_id 为 0..63。 |
| DS_BPERMUTE_B32                          | 反向置换, 实际不写入 LDS 内存。其置换语义可表示为: LDS[thread_id] = src0, 其中 thread_id 为 0..63; returnVal = LDS[dst]。 |

单一地址指令

```
LDS_Addr = LDS_BASE + VGPR[ADDR] + {InstrOffset1, InstrOffset0}
```

双位地址指令

```
LDS_Addr0 = LDS_BASE + VGPR[ADDR] + InstrOffset0*ADJ +
LDS_Addr1 = LDS_BASE + VGPR[ADDR] + InstrOffset1*ADJ
Where ADJ = 4 for 8, 16 and 32-bit data types; and ADJ = 8 for 64-bit.
```

注意 LDS\_ADDR1 只用于 READ2\*,WRITE2\*和 WREXCHG2\*.

地址来自 VGPR,ADDR 和 InstrOffset 都是字节地址.

在波峰创建时,LDS\_BASE 被分配到这个波峰或工作组拥有的物理 LDS 区域.

只指定一个地址, 设置两个偏移到相同值。 这只导致一个读或写, 只使用第一个 DATA0 。

LDS 原子行动

DS\_<原子 Op> OP, GDS=0, OFFSET0, OFFSET1, VDST, ADDR, Data0, Data1

数据大小在原子 Op 中编码:字节,单词,Dword,或双词.

```
LDS_Addr0 = LDS_BASE + VGPR[ADDR] + {InstrOffset1,InstrOffset0}
```

ADDR 是一个 Dword 地址. VGPRs 0,1 和 dst 是双人数据双 GPRs.

VGPR 数据来源只能是 VGPRs 或恒值,不能是 SGPRs.

通过 MODE 寄存器的 FP\_DENORM 字段来控制浮点的异常行为。  
四舍五入模式固定在"圆到最近的偶数".

## 11.4. GWS 编程限制

所有 GWS 指示必须立即执行:

```
s_waitcnt 0
```

任何 GWS 指令使用的 VGPRs 必须均匀.



# 第 12 章 指令

本章列出并提供 CDNA 生成环境中的所有指令的描述。指令按其格式分类。

指令后缀有以下定义: 1.

- B32 Bitfield (未类型数据) 32 位
- B64 Bitfield (无类型数据) 64 位
- F32 浮点 32 位( IEEE 754 单精度浮点)
- F64 浮点 64 位( IEEE 754 双精度浮点)
- BF16 浮点 16 位( Bfloat16 格式)
- I8 签名 8 位整数
- I16 签名 16 位整数
- I32 签名 32 位整数
- I64 签名 64 位整数
- U32 无符号的 32 位整数
- U64 无符号的 64 位整数

如果一个指令有两个后缀(例如\_I32\_F32),第一个后缀表示目的地类型,第二个是源类型.

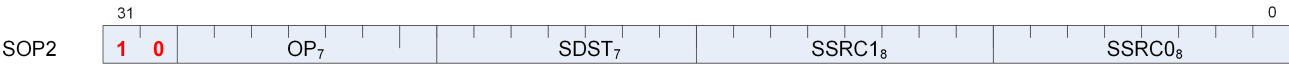
指示定义中使用的简称如下:

- D = destination
- U = 无符号整数
- S=来源
- SCC = 标量条件码
- I = 签名整数
- B = 位字段

注:.u 或.i 指定将参数解释为无符号或署名的浮点.

注意:四舍五入和异常模式适用于所有浮点操作,除非在指令描述中另有说明.

## 12.1. SOP2 指令



这种格式的指令可能使用一个 32 位的字元常数,该常数在指令后立即出现.

**S\_ADD\_U32** 0

添加两个未署名的 32 位整数输入,将结果存储到标量寄存器中,并将运出位存储到 SCC.

```
tmp = 64'U(S0.u32) + 64'U(S1.u32);
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;
// 无符号的 S_ADDC_U32 的溢出或执行。
D0.u32 = tmp.u32
```

**S\_SUB\_U32****1**

从第一个输入中减去第二个无符号的 32 位整数输入,将结果存储到一个标量寄存器中,并将运出位存储到 SCC.

```
tmp = S0.u32 - S1.u32;
SCC = S1.u32 > S0.u32 ? 1'1U : 1'0U;
// 未署名的 S_SUBB_U32 溢出或运出。
D0.u32 = tmp.u32
```

**S\_ADD\_I32****2**

加入两个签名的 32 位整数输入,将结果存储到一个标量寄存器中,并将运出位存储到 SCC.

```
tmp = S0.i32 + S1.i32;
SCC = ((S0.u32[31] == S1.u32[31]) && (S0.u32[31] != tmp.u32[31]));
// 签名的溢出。
D0.i32 = tmp.i32
```

说明

这个 opcode 不适合与 S\_ADDC\_U32 一起用于执行 64 位操作.

**S\_SUB\_I32****3**

从第一个输入中减掉第二个签名的 32 位整数输入,将结果存储到一个标量寄存器中,并将执行的比特存储到 SCC.

```
tmp = S0.i32 - S1.i32;
SCC = ((S0.u32[31] != S1.u32[31]) && (S0.u32[31] != tmp.u32[31]));
// 签名的溢出。
D0.i32 = tmp.i32
```

说明

这个 opcode 不适合与 S\_SUBB\_U32 一起用于执行 64 位操作.

**S\_ADDC\_U32**

4

增加两个未署名的 32 位整数输入和一个来自 SCC 的承载位,将结果存储到一个标量寄存器中,并将承载位存储到 SCC。

```
tmp = 64'U(S0.u32) + 64'U(S1.u32) + SCC.u64;  
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// 无符号的 S_ADDC_U32 的溢出或执行。  
D0.u32 = tmp.u32
```

**S\_SUBB\_U32**

5

从第一个输入中减去第二个无符号的 32 位整数输入,减去承载位,将结果存储到一个标量寄存器中,并将承载位存储到 SCC。

```
tmp = S0.u32 - S1.u32 - SCC.u32;  
SCC = 64'U(S1.u32) + SCC.u64 > 64'U(S0.u32) ? 1'1U : 1'0U;  
// 未署名的 S_SUBB_U32 溢出或运出。  
D0.u32 = tmp.u32
```

**S\_MIN\_I32**

6

选择最少两个签名的 32 位整数输入,将所选值存储到一个标量寄存器中,并设置第一个值被选中的 SCC。

```
SCC = S0.i32 < S1.i32;  
D0.i32 = SCC ? S0.i32 : S1.i32
```

**S\_MIN\_U32**

7

选择两个无符号的 32 位整数输入的最小值,将所选值存储到一个标量寄存器中,并设置 SCC 如果选中第一个值。

```
SCC = S0.u32 < S1.u32;  
D0.u32 = SCC ? S0.u32 : S1.u32
```

**S\_MAX\_I32**

8

选择最多两个签名的 32 位整数输入,将所选值存储到一个标量寄存器中,并设置第一个值被选中的 SCC。

```
SCC = S0.i32 >= S1.i32;  
D0.i32 = SCC ? S0.i32 : S1.i32
```

## S\_MAX\_U32

9

选择两个无符号的 32 位整数输入的最大值,将所选值存储到一个标量寄存器中,并设置 SCC 如果选中第一个值。

```
SCC = S0.u32 >= S1.u32;  
D0.u32 = SCC ? S0.u32 : S1.u32
```

## S\_CSELECT\_B32

10

如果 SCC 是真的, 请选择第一个输入, 否则选择第二个输入, 然后将选中的输入存储到标量寄存器中。

```
D0.u32 = SCC ? S0.u32 : S1.u32
```

## S\_CSELECT\_B64

11

如果 SCC 是真的, 请选择第一个输入, 否则选择第二个输入, 然后将选中的输入存储到标量寄存器中。

```
D0.u64 = SCC ? S0.u64 : S1.u64
```

## S\_AND\_B32

12

以比特方式计算和在两个刻度输入上,将结果存储到一个刻度寄存器中,如果结果为非零,则设置 SCC。

```
D0.u32 = (S0.u32 & S1.u32);  
SCC = D0.u32 != 0U
```

**S\_AND\_B64****13**

以比特方式计算和在两个刻度输入上,将结果存储到一个刻度寄存器中,如果结果为非零,则设置 SCC。

```
D0.u64 = (S0.u64 & S1.u64);  
SCC = D0.u64 != 0ULL
```

**S\_OR\_B32****14**

在两个标量输入上计算位法 OR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u32 = (S0.u32 | S1.u32);  
SCC = D0.u32 != 0U
```

**S\_OR\_B64****15**

在两个标量输入上计算位法 OR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u64 = (S0.u64 | S1.u64);  
SCC = D0.u64 != 0ULL
```

**S\_XOR\_B32****16**

在两个标量输入上计算比特 XOR,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u32 = (S0.u32 ^ S1.u32);  
SCC = D0.u32 != 0U
```

**S\_XOR\_B64****17**

在两个标量输入上计算比特 XOR,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u64 = (S0.u64 ^ S1.u64);
```

```
SCC = D0.u64 != 0ULL
```

### S\_ANDN2\_B32

18

将第二个输入按位取反，再与第一个输入执行按位与运算。结果写入标量寄存器；结果非零时，将 SCC 置位。

```
D0.u32 = (S0.u32 & ~S1.u32);  
SCC = D0.u32 != 0U
```

### S\_ANDN2\_B64

19

将第二个输入按位取反，再与第一个输入执行按位与运算。结果写入标量寄存器；结果非零时，将 SCC 置位。

```
D0.u64 = (S0.u64 & ~S1.u64);  
SCC = D0.u64 != 0ULL
```

### S\_ORN2\_B32

20

将第二个输入按位取反，再与第一个输入执行按位或运算。结果写入标量寄存器；结果非零时，将 SCC 置位。

```
D0.u32 = (S0.u32 | ~S1.u32);  
SCC = D0.u32 != 0U
```

### S\_ORN2\_B64

21

将第二个输入按位取反，再与第一个输入执行按位或运算。结果写入标量寄存器；结果非零时，将 SCC 置位。

```
D0.u64 = (S0.u64 | ~S1.u64);  
SCC = D0.u64 != 0ULL
```

### S\_NAND\_B32

22

在两个刻度输入上计算位元 NAND, 将结果存储到刻度寄存器中, 如果结果是 SCC

非零级。

```
D0.u32 = ~(S0.u32 & S1.u32);  
SCC = D0.u32 != 0U
```

## S\_NAND\_B64

23

在两个标量输入上计算位元 NAND,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u64 = ~(S0.u64 & S1.u64);  
SCC = D0.u64 != 0ULL
```

## S\_NOR\_B32

24

在两个标量输入上计算比特 NOR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u32 = ~(S0.u32 | S1.u32);  
SCC = D0.u32 != 0U
```

## S\_NOR\_B64

25

在两个标量输入上计算比特 NOR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u64 = ~(S0.u64 | S1.u64);  
SCC = D0.u64 != 0ULL
```

## S\_XNOR\_B32

26

在两个标量输入上计算位元 XNOR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u32 = ~(S0.u32 ^ S1.u32);  
SCC = D0.u32 != 0U
```

**S\_XNOR\_B64****27**

在两个标量输入上计算位元 XNOR,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u64 = ~(S0.u64 ^ S1.u64);  
SCC = D0.u64 != 0ULL
```

**S\_LSHL\_B32****28**

鉴于在第二个标量输入中的位移计数,计算第一个标量输入中左侧的逻辑位移,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u32 = (S0.u32 << S1[4 : 0].u32);  
SCC = D0.u32 != 0U
```

**S\_LSHL\_B64****29**

鉴于在第二个标量输入中的位移计数,计算第一个标量输入中左侧的逻辑位移,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u64 = (S0.u64 << S1[5 : 0].u32);  
SCC = D0.u64 != 0ULL
```

**S\_LSHR\_B32****30**

鉴于第二个标量输入中的移位计数,计算第一个标量输入的逻辑移位右侧,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u32 = (S0.u32 >> S1[4 : 0].u32);  
SCC = D0.u32 != 0U
```

**S\_LSHR\_B64****31**

鉴于第二个标量输入中的移位计数,计算第一个标量输入的逻辑移位右侧,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC.

```
D0.u64 = (S0.u64 >> S1[5 : 0].u32);
```



```
SCC = D0.u64 != 0ULL
```

## S\_ASHR\_I32

32

鉴于第二个标量输入中的值为转数,计算第一个标量输入中的算术右移(保留符号位),将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.i32 = 32'I(signext(S0.i32) >> S1[4 : 0].u32);  
SCC = D0.i32 != 0
```

## S\_ASHR\_I64

33

鉴于第二个标量输入中的值为转数,计算第一个标量输入中的算术右移(保留符号位),将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.i64 = (signext(S0.i64) >> S1[5 : 0].u32);  
SCC = D0.i64 != 0LL
```

## S\_BFM\_B32

34

计算一个字段偏移和大小的比特菲尔德掩码,并将结果存储在标量寄存器中。

```
D0.u32 = (((1U << S0[4 : 0].u32) - 1U) << S1[4 : 0].u32)
```

## S\_BFM\_B64

35

计算一个字段偏移和大小的比特菲尔德掩码,并将结果存储在标量寄存器中。

```
D0.u64 = (((1ULL << S0[5 : 0].u32) - 1ULL) << S1[5 : 0].u32)
```

## S\_MUL\_I32

36

乘以两个签名的 32 位整数输入,并将结果存储在一个标量寄存器中。

```
D0.i32 = S0.i32 * S1.i32
```

**S\_BFE\_U32****37**

从第一个输入中提取一个无符号的比特字段,使用字段偏移,并在第二个输入中编码大小,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u32 = ((S0.u32 >> S1[4 : 0].u32) & ((1U << S1[22 : 16].u32) - 1U));
SCC = D0.u32 != 0U
```

**S\_BFE\_I32****38**

从第一个输入中提取一个签名的比特字段,使用字段偏移并在第二个输入中编码大小,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
tmp.i32 = ((S0.i32 >> S1[4 : 0].u32) & ((1 << S1[22 : 16].u32) - 1));
D0.i32 = signext_from_bit(tmp.i32, S1[22 : 16].u32);
SCC = D0.i32 != 0
```

**S\_BFE\_U64****39**

从第一个输入中提取一个无符号的比特字段,使用字段偏移,并在第二个输入中编码大小,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.u64 = ((S0.u64 >> S1[5 : 0].u32) & ((1ULL << S1[22 : 16].u32) - 1ULL));
SCC = D0.u64 != 0ULL
```

**S\_BFE\_I64****40**

从第一个输入中提取一个签名的比特字段,使用字段偏移并在第二个输入中编码大小,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
tmp.i64 = ((S0.i64 >> S1[5 : 0].u32) & ((1LL << S1[22 : 16].u32) - 1LL));
D0.i64 = signext_from_bit(tmp.i64, S1[22 : 16].u32);
SCC = D0.i64 != 0LL
```

**S\_CBRANCH\_G\_FORK****41**

有条件的分支使用分支-sack.

S0 = 比较面罩(VCC 或任何 SGPR)和 S1 = 目标指令的 64 位字节地址. 另见 S\_CBRANCH\_JOIN.

```
mask_pass = (S0.u64 & EXEC.u64);
mask_fail = (~S0.u64 & EXEC.u64);
if mask_pass == EXEC.u64 then
    PC = 64'I(S1.u64)
elsif mask_fail == EXEC.u64 then
    PC += 4LL
elsif bitCount(mask_fail.b64) < bitCount(mask_pass.b64) then
    EXEC = mask_fail.b64;
    SGPR[WAVE_MODE.CSP.u32 * 4U].b128 = { S1.u64, mask_pass };
    WAVE_MODE.CSP += 3'1U;
    PC += 4LL
else
    EXEC = mask_pass.b64;
    SGPR[WAVE_MODE.CSP.u32 * 4U].b128 = { (PC + 4LL), mask_fail };
    WAVE_MODE.CSP += 3'1U;
    PC = 64'I(S1.u64)
endif
```

## S\_ABSDIFF\_I32

42

计算两个标量输入值的绝对值,将结果存储到标量寄存器中,并设置 SCC 如果结果为非零。

```
D0.i32 = S0.i32 - S1.i32;
if D0.i32 < 0 then
    D0.i32 = -D0.i32
endif;
SCC = D0.i32 != 0
```

说明

功能实例:

```
S_ABSDIFF_I32(0x00000002, 0x00000005) => 0x00000003
S_ABSDIFF_I32(0xffffffff, 0x00000000) => 0x00000001
S_ABSDIFF_I32(0x80000000, 0x00000000) => 0x80000000 // 注:结果为负数!
S_ABSDIFF_I32(0x80000000, 0x00000001) => 0x7fffffff
S_ABSDIFF_I32(0x80000000, 0xffffffff) => 0x7fffffff
S_ABSDIFF_I32(0x80000000, 0xfffffffef) => 0x7fffffefe
```

## S\_MUL\_HI\_U32

44

将两个无符号的整数乘以,并将结果的高 32 位存储到一个标量寄存器中.

```
D0.u32 = 32'U((64'U(S0.u32) * 64'U(S1.u32)) >> 32U)
```

## S\_MUL\_HI\_I32

45

乘以两个签名的整数,并将结果的高 32 位存储到一个标量寄存器中。

```
D0.i32 = 32'I((64'I(S0.i32) * 64'I(S1.i32)) >> 32U)
```

## S\_LSHL1\_ADD\_U32

46

计算第一个输入的逻辑左移为 1, 然后添加第二个输入, 将结果存储到一个标量寄存器中, 并设置 SCC 如果总和结果为无符号的溢出。

```
tmp = (64'U(S0.u32) << 1U) + 64'U(S1.u32);  
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// 无符号的溢出。  
D0.u32 = tmp.u32
```

## S\_LSHL2\_ADD\_U32

47

计算第一个输入的逻辑左移为 2, 然后添加第二个输入, 将结果存储到一个标量寄存器中, 如果总和结果为无符号的溢出, 则设置 SCC。

```
tmp = (64'U(S0.u32) << 2U) + 64'U(S1.u32);  
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// 无符号的溢出。  
D0.u32 = tmp.u32
```

## S\_LSHL3\_ADD\_U32

48

计算第一个输入左侧的逻辑移位为 3, 然后添加第二个输入, 将结果存储到标量寄存器中, 并设置 SCC 如果总和结果为无符号的溢出。

```
tmp = (64'U(S0.u32) << 3U) + 64'U(S1.u32);  
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// 无符号的溢出。  
D0.u32 = tmp.u32
```

**S\_LSHL4\_ADD\_U32****49**

计算第一个输入左侧的逻辑移到 4, 然后添加第二个输入, 将结果存储到标量寄存器中, 并设置 SCC 如果总和结果为无符号的溢出。

```
tmp = (64'U(S0.u32) << 4U) + 64'U(S1.u32);  
SCC = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// 无符号的溢出。  
D0.u32 = tmp.u32
```

**S\_PACK\_LL\_B32\_B16****50**

把两个 16 位的标量值打入标量寄存器.

```
D0 = { S1[15 : 0].u16, S0[15 : 0].u16 }
```

**S\_PACK\_LH\_B32\_B16****51**

把两个 16 位的标量值打入标量寄存器.

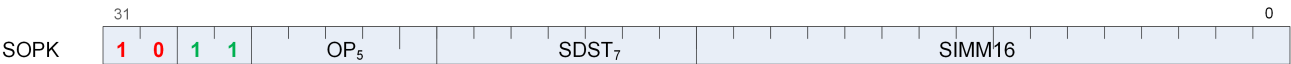
```
D0 = { S1[31 : 16].u16, S0[15 : 0].u16 }
```

**S\_PACK\_HH\_B32\_B16****52**

把两个 16 位的标量值打入标量寄存器.

```
D0 = { S1[31 : 16].u16, S0[31 : 16].u16 }
```

## 12.2. SOPK 指令



这种格式的指令可能使用一个 32 位的字元常数,该常数在指令后立即出现.

S\_MOVK\_I32

0

签名扩展字元 16 位常数,并将结果存储到标量寄存器中.

```
D0.i32 = 32'I(signext(S0.i16))
```

S\_CMOVK\_I32

1

如果 SCC 是非零,则将字面 16 位常数的符号扩展移动到标量寄存器中。

```
if SCC then
    D0.i32 = 32'I(signext(S0.i16))
endif
```

S\_CMPK\_EQ\_I32

2

将 SCC 设置为 1 iff 标量输入等于 16 位元常数的符号扩展。

```
SCC = S0.i32 == 32'I(signext(S1.i16))
```

S\_CMPK\_LG\_I32

3

将 SCC 设置为 1 iff 标量输入小于或大于 16 位元常数的符号扩展。

```
SCC = S0.i32 != 32'I(signext(S1.i16))
```

S\_CMPK\_GT\_I32

4

将 SCC 设置为 1 iff 标量输入大于字形 16 位常数的符号扩展。

```
SCC = S0.i32 > 32'I(signext(S1.i16))
```

### S\_CMPK\_GE\_I32

5

将 SCC 设置为 1 iff 标量输入大于或等于一个字形 16 位常数的符号扩展。

```
SCC = S0.i32 >= 32'I(signext(S1.i16))
```

### S\_CMPK\_LT\_I32

6

将 SCC 设置为 1 iff 标量输入小于 16 位元常数的符号扩展。

```
SCC = S0.i32 < 32'I(signext(S1.i16))
```

### S\_CMPK\_LE\_I32

7

将 SCC 设置为 1 iff 标量输入小于或等于 16 位元常数的符号扩展。

```
SCC = S0.i32 <= 32'I(signext(S1.i16))
```

### S\_CMPK\_EQ\_U32

8

将 SCC 设置为 1 iff 标量输入等于字元 16 位常数的零扩展。

```
SCC = S0.u32 == 32'U(S1.u16)
```

### S\_CMPK\_LG\_U32

9

将 SCC 设置为 1 iff 标量输入小于或大于字数 16 位常数的零扩展。

```
SCC = S0.u32 != 32'U(S1.u16)
```

**S\_CMPK\_GT\_U32****10**

将 SCC 设置为 1 iff 标量输入大于字元 16 位常数的零扩展。

```
SCC = S0.u32 > 32'U(S1.u16)
```

**S\_CMPK\_GE\_U32****11**

将 SCC 设置为 1 iff 标量输入值大于或等于字元 16 位常数的零扩展值。

```
SCC = S0.u32 >= 32'U(S1.u16)
```

**S\_CMPK\_LT\_U32****12**

将 SCC 设置为 1 iff 标量输入小于 16 位元常数的零扩展。

```
SCC = S0.u32 < 32'U(S1.u16)
```

**S\_CMPK\_LE\_U32****13**

将 SCC 设置为 1 iff 标量输入小于或等于字元 16 位常数的零扩展。

```
SCC = S0.u32 <= 32'U(S1.u16)
```

**S\_ADDK\_I32****14**

添加一个标量输入和一个字面 16 位常数的符号扩展,将结果存储到标量寄存器中,并将运出位存储到 SCC 中。

```
tmp = D0.i32;  
// 保存值以检查符号位以备日后溢出。  
D0.i32 = D0.i32 + 32'I(signext(S0.i16));  
SCC = ((tmp[31] == S0.i16[15]) && (tmp[31] != D0.i32[31]));  
// 签名的溢出。
```

**S\_MULK\_I32****15**



用字形 16 位常数的符号扩展来乘以标量输入,并将结果存储到标量寄存器中。

```
D0.i32 = D0.i32 * 32'I(signext(S0.i16))
```

## S\_CBRANCH\_I\_FORK

16

有条件的分支使用分支-sack。

S0 = 比较面罩(VCC 或任何 SGPR),SIMM16 = 与下一个指令相对的签名 DWORD 分支抵消。另见 S\_CBRANCH\_JOIN。

```
// 初设。
mask_pass = (S0.u64 & EXEC.u64);
mask_fail = (~S0.u64 & EXEC.u64);
target_addr = PC + signext(SIMM16.i32 * 4) + 4LL;
// 决定向甚么处跳。
if mask_pass == EXEC.u64 then
    PC = target_addr
elseif mask_fail == EXEC.u64 then
    PC += 4LL
elseif bitCount(mask_fail.b64) < bitCount(mask_pass.b64) then
    EXEC = mask_fail.b64;
    SGPR[WAVE_MODE.CSP.u32 * 4U].b128 = { target_addr, mask_pass };
    WAVE_MODE.CSP += 3'1U;
    PC += 4LL
else
    EXEC = mask_pass.b64;
    SGPR[WAVE_MODE.CSP.u32 * 4U].b128 = { (PC + 4LL), mask_fail };
    WAVE_MODE.CSP += 3'1U;
    PC = target_addr
endif
```

## S\_GETREG\_B32

17

向目的地的 LSB 读取部分或全部硬件寄存器。

```
hwRegId = SIMM16.u16[5 : 0];
offset = SIMM16.u16[10 : 6];
size = SIMM16.u16[15 : 11].u32 + 1U;
// 逻辑大小在 1:32 之间
value = HW_REGISTERS[hwRegId];
D0.u32 = 32'U(32'I(value >> offset.u32) & ((1 << size) - 1))
```

## S\_SETREG\_B32

18

将源参数的一些或全部 LSB 写入硬件寄存器中。

```
hwRegId = SIMM16.u16[5 : 0];
offset = SIMM16.u16[10 : 6];
size = SIMM16.u16[15 : 11].u32 + 1U;
// 逻辑大小在 1:32 之间
mask = (1 << size) - 1;
mask = (mask << offset.u32);
mask = (mask & HwRegWriteMask(hwRegId, WAVE_STATUS.PRIV));
// 可修改的比特的遮盖
value = ((S0.u32 << offset.u32) & mask.u32);
value = (value | 32'U(HW_REGISTERS[hwRegId].i32 & ~mask));
HW_REGISTERS[hwRegId] = value.b32;
// 如果修改了某些位点, 副作用可能在此触发
```

## S\_SETREG\_IMM32\_B32

20

将 32 位字元常数的部分或全部 LSB 写入硬件寄存器;此指令需要 32 位字元常数。

```
hwRegId = SIMM16.u16[5 : 0];
offset = SIMM16.u16[10 : 6];
size = SIMM16.u16[15 : 11].u32 + 1U;
// 逻辑大小在 1:32 之间
mask = (1 << size) - 1;
mask = (mask << offset.u32);
mask = (mask & HwRegWriteMask(hwRegId, WAVE_STATUS.PRIV));
// 可修改的比特的遮盖
value = ((SIMM32.u32 << offset.u32) & mask.u32);
value = (value | 32'U(HW_REGISTERS[hwRegId].i32 & ~mask));
HW_REGISTERS[hwRegId] = value.b32;
// 如果修改了某些位点, 副作用可能在此触发
```

## S\_CALL\_B64

21

将下一个指令的地址存储到一个标量寄存器中,然后跳转到相对于当前 PC 的恒定偏移中。

字面参数是一个签名的 DWORD 相对下一个指令的 PC 的偏移。此指令之后的指令的字节地址将保存到目的地。

```
D0.i64 = PC + 4LL;
PC = PC + signnext(SIMM16.i16 * 16'4) + 4LL
```

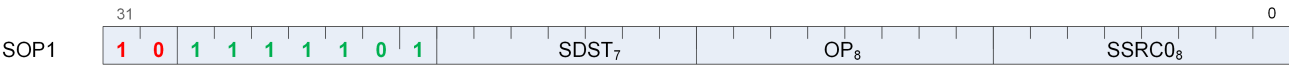
说明

此执行一个简短的子例程调用, 返回地址( S\_CALL\_ B64 之后的下一个指令) 保存到 D 。 长调时应考虑 S\_SWAPPC\_ B64 。

此指令必须是 4 字节。

---

### 12.3. SOP1 指令



这种格式的指令可能使用一个 32 位的字元常数,该常数在指令后立即出现。

#### S\_MOV\_B32 0

将标量输入移动到标量寄存器中。

```
D0.b32 = S0.b32
```

#### S\_MOV\_B64 1

将标量输入移动到标量寄存器中。

```
D0.b64 = S0.b64
```

#### S\_CMOV\_B32 2

将标量输入移动到标量寄存器 iff SCC 是非零。

```
if SCC then
    D0.b32 = S0.b32
endif
```

#### S\_CMOV\_B64 3

将标量输入移动到标量寄存器 iff SCC 是非零。

```
if SCC then
    D0.b64 = S0.b64
endif
```

#### S\_NOT\_B32 4

在刻度输入上计算比特按位取反值, 将结果存储到刻度寄存器中, 如果结果为非零则设置 SCC。

```
D0.u32 = ~S0.u32;
SCC = D0.u32 != 0U
```

## S\_NOT\_B64

5

在刻度输入上计算比特按位取反值, 将结果存储到刻度寄存器中, 如果结果为非零则设置 SCC。

```
D0.u64 = ~S0.u64;
SCC = D0.u64 != 0ULL
```

## S\_WQM\_B32

6

考虑到在标量输入中有一个活动像素遮罩, 为该输入计算整个四面体模式遮罩, 将结果存储到标量寄存器中, 如果结果为非零, 则设置 SCC。

在整个四重奏模式中, 如果一个四重奏中的任何像素活动, 那么所有四重奏的像素都会被标记为活动。

```
tmp = 0U;
declare i : 6'U;
for i in 6'0U : 6'31U do
    tmp[i] = S0.u32[i & 6'60U +: 6'4U] != 0U
endfor;
D0.u32 = tmp;
SCC = D0.u32 != 0U
```

## S\_WQM\_B64

7

考虑到在标量输入中有一个活动像素遮罩, 为该输入计算整个四面体模式遮罩, 将结果存储到标量寄存器中, 如果结果为非零, 则设置 SCC。

在整个四重奏模式中, 如果一个四重奏中的任何像素活动, 那么所有四重奏的像素都会被标记为活动。

```
tmp = 0ULL;
declare i : 6'U;
for i in 6'0U : 6'63U do
    tmp[i] = S0.u64[i & 6'60U +: 6'4U] != 0ULL
endfor;
D0.u64 = tmp;
SCC = D0.u64 != 0ULL
```

**S\_BREV\_B32****8**

反转一个标量输入中位数的顺序,并将结果存储到一个标量寄存器中.

```
D0.u32[31 : 0] = S0.u32[0 : 31]
```

**S\_BREV\_B64****9**

反转一个标量输入中位数的顺序,并将结果存储到一个标量寄存器中.

```
D0.u64[63 : 0] = S0.u64[0 : 63]
```

**S\_BCNT0\_I32\_B32****10**

计算在标量输入中的"0"位数,将结果存储到标量寄存器中,如果结果为非零则设置 SCC.

```
tmp = 0;
for i in 0 : 31 do
    tmp += S0.u32[i] == 1'0U ? 1 : 0
endfor;
D0.i32 = tmp;
SCC = D0.u32 != 0U
```

说明

功能实例:

```
S_BCNT0_I32_B32(0x00000000) => 32
S_BCNT0_I32_B32(0xffffffff) => 16
S_BCNT0_I32_B32(0xffffffff) => 0
```

**S\_BCNT0\_I32\_B64****11**

计算在标量输入中的"0"位数,将结果存储到标量寄存器中,如果结果为非零则设置 SCC.

```
tmp = 0;
for i in 0 : 63 do
    tmp += S0.u64[i] == 1'0U ? 1 : 0
endfor;
```

```
D0.i32 = tmp;
SCC = D0.u64 != 0ULL
```

## S\_BCNT1\_I32\_B32

12

计算在标量输入中的"1"位数,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
tmp = 0;
for i in 0 : 31 do
    tmp += S0.u32[i] == 1'1U ? 1 : 0
endfor;
D0.i32 = tmp;
SCC = D0.u32 != 0U
```

说明

功能实例:

```
S_BCNT1_I32_B32(0x00000000) => 0
S_BCNT1_I32_B32(0xffffffff) => 32
S_BCNT1_I32_B32(0x00000001) => 1
```

## S\_BCNT1\_I32\_B64

13

计算在标量输入中的"1"位数,将结果存储到标量寄存器中,如果结果为非零,则设置 SCC.

```
tmp = 0;
for i in 0 : 63 do
    tmp += S0.u64[i] == 1'1U ? 1 : 0
endfor;
D0.i32 = tmp;
SCC = D0.u64 != 0ULL
```

## S\_FF0\_I32\_B32

14

从标量输入的最低位开始,统计遇到第一个 0 之前连续的 1 的数量,并将结果写入标量寄存器。若输入中没有 0,则写入 -1。

```
tmp = -1;
// 如果找不到零则设定
```

```

for i in 0 : 31 do
    // 从 LSB 搜索
    if S0.u32[i] == 1'0U then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp

```

说明

功能实例:

```

S_FF0_I32_B32(0xaaaaaaaa) => 0
S_FF0_I32_B32(0x55555555) => 1
S_FF0_I32_B32(0x00000000) => 0
S_FF0_I32_B32(0xffffffff) => 0xffffffff
S_FF0_I32_B32(0xfffeffff) => 16

```

## S\_FF0\_I32\_B64

15

从标量输入的最低位开始，统计遇到第一个 0 之前连续的 1 的数量，并将结果写入标量寄存器。若输入中没有 0，则写入 -1。

```

tmp = -1;
// 如果找不到零则设定
for i in 0 : 63 do
    // 从 LSB 搜索
    if S0.u64[i] == 1'0U then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp

```

## S\_FF1\_I32\_B32

16

统计标量输入的尾随 0 的数量，即从最低位起到第一个 1 之前连续的 0，并将结果写入标量寄存器。若输入中没有 1，则写入 -1。

```

tmp = -1;
// 如果找不到, 则设定
for i in 0 : 31 do
    // 从 LSB 搜索
    if S0.u32[i] == 1'1U then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp

```



```

    endif
endfor;
D0.i32 = tmp

```

说明

功能实例:

```

S_FF1_I32_B32(0xaaaaaaaa) => 1
S_FF1_I32_B32(0x55555555) => 0
S_FF1_I32_B32(0x00000000) => 0xffffffff
S_FF1_I32_B32(0xffffffff) => 0
S_FF1_I32_B32(0x00010000) => 16

```

## S\_FF1\_I32\_B64

17

统计标量输入的尾随 0 的数量，即从最低位起到第一个 1 之前连续的 0，并将结果写入标量寄存器。若输入中没有 1，则写入 -1。

```

tmp = -1;
// 如果找不到, 则设定
for i in 0 : 63 do
    // 从 LSB 搜索
    if S0.u64[i] == 1'1U then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp

```

## S\_FLBIT\_I32\_B32

18

统计标量输入的前导 0 的数量，即从最高位起到第一个 1 之前连续的 0，并将结果写入标量寄存器。若没有 1，则写入 -1。

```

tmp = -1;
// 如果找不到, 则设定
for i in 0 : 31 do
    // 从 MSB 搜索
    if S0.u32[31 - i] == 1'1U then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp

```

说明

功能实例:

```
S_FLBIT_I32_B32(0x00000000) => 0xffffffff
S_FLBIT_I32_B32(0x0000cccc) => 16
S_FLBIT_I32_B32(0xffff3333) => 0
S_FLBIT_I32_B32(0x7fffffff) => 1
S_FLBIT_I32_B32(0x80000000) => 0
S_FLBIT_I32_B32(0xffffffff) => 0
```

## S\_FLBIT\_I32\_B64

19

统计标量输入的前导 0 的数量，即从最高位起到第一个 1 之前连续的 0，并将结果写入标量寄存器。若没有 1，则写入 -1。

```
tmp = -1;
// 如果找不到, 则设定
for i in 0 : 63 do
    // 从 MSB 搜索
    if S0.u64[63 - i] == 1'u then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp
```

## S\_FLBIT\_I32

20

统计标量输入中从最高位起、连续与符号位相同的位数，并将结果写入标量寄存器。若所有输入位都相同，则写入 -1。

```
tmp = -1;
// 如果所有位码都是相同的, 设定
for i in 1 : 31 do
    // 从 MSB 搜索
    if S0.u32[31 - i] != S0.u32[31] then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp
```

说明

功能实例:

```
S_FLBIT_I32(0x00000000) => 0xffffffff
S_FLBIT_I32(0x0000cccc) => 16
S_FLBIT_I32(0xffff3333) => 16
S_FLBIT_I32(0x7fffffff) => 1
S_FLBIT_I32(0x80000000) => 1
S_FLBIT_I32(0xffffffff) => 0xffffffff
```

**S\_FLBIT\_I32\_I64****21**

统计标量输入中从最高位起、连续与符号位相同的位数，并将结果写入标量寄存器。若所有输入位都相同，则写入 -1。

```
tmp = -1;
// 如果所有位码都是相同的, 设定
for i in 1 : 63 do
    // 从 MSB 搜索
    if S0.u64[63 - i] != S0.u64[63] then
        tmp = i;
        break
    endif
endfor;
D0.i32 = tmp
```

**S\_SEXT\_I32\_I8****22**

签名将签名的 8 比特标量输入扩展至 32 比特,并将结果存储到标量寄存器中。

```
D0.i32 = 32'I(signext(S0.i8))
```

**S\_SEXT\_I32\_I16****23**

签名将一个签名的 16 比特标量输入扩展至 32 比特,并将结果存储到标量寄存器中。

```
D0.i32 = 32'I(signext(S0.i16))
```

**S\_BITSET0\_B32****24**

由于在标量输入中略微偏移,将目的地标量寄存器中显示的比特设置为 0。

```
D0.u32[S0.u32[4 : 0]] = 1'0U
```

### S\_BITSET0\_B64

25

由于在标量输入中略微偏移,将目的地标量寄存器中显示的比特设置为 0。

```
D0.u64[S0.u32[5 : 0]] = 1'0U
```

### S\_BITSET1\_B32

26

由于在标量输入中略微抵消,将目的地标量寄存器中显示的位数设置为 1。

```
D0.u32[S0.u32[4 : 0]] = 1'1U
```

### S\_BITSET1\_B64

27

由于在标量输入中略微抵消,将目的地标量寄存器中显示的位数设置为 1。

```
D0.u64[S0.u32[5 : 0]] = 1'1U
```

### S\_GETPC\_B64

28

将下一个指令的地址存储到标量寄存器中。

在此指示之后, 指令的字节地址将被保存到目的地。

```
D0.i64 = PC + 4LL
```

说明

此指令必须是 4 字节。

### S\_SETPC\_B64

29

跳转到标量寄存器中指定的地址。

参数是一个要跳转到的指令的字节地址。

```
PC = S0.i64
```

---

### S\_SWAPPC\_B64

30

将下一个指令的地址存储到标量寄存器,然后跳转到标量输入中指定的地址。

参数是一个要跳转到的指令的字节地址。 此指令之后的指令的字节地址将被保存到目的地。

```
jump_addr = S0.i64;  
D0.i64 = PC + 4LL;  
PC = jump_addr.i64
```

说明

此指令必须是 4 字节。

---

### S\_RFE\_B64

31

从例外处理器返回。 清除波段的 PRIV 位, 然后跳转到由标量输入指定的地址。

参数是指示跳转到的字节地址;这个地址可能是从传递到陷阱处理器的状态中衍生出来的。

此指令只能在陷阱处理器内使用。

```
WAVE_STATUS.PRIV = 1'0U;  
PC = S0.i64
```

---

### S\_AND\_SAVEEXEC\_B64

32

以位法计算,并在标量输入和 EXEC 掩码上,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
```

```
EXEC.u64 = (S0.u64 & EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

### S\_OR\_SAVEEXEC\_B64

33

在标量输入和 EXEC 口罩上计算位元 OR,将计算结果存储到 EXEC 口罩中,将 SCC 设定为非零计算结果,并将 EXEC 口罩的原始值存储到标量目的寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
EXEC.u64 = (S0.u64 | EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

### S\_XOR\_SAVEEXEC\_B64

34

在标量输入和 EXEC 口罩上计算位元 XOR,将计算结果存储到 EXEC 口罩中,将 SCC 设定为非零计算结果,并将 EXEC 口罩的原始值存储到标量目的寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
EXEC.u64 = (S0.u64 ^ EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

### S\_ANDN2\_SAVEEXEC\_B64

35

以位法计算和在 EXEC 掩码的刻度输入和按位取反时,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的地寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
EXEC.u64 = (S0.u64 & ~EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

**S\_ORN2\_SAVEEXEC\_B64****36**

在标量输入和 EXEC 掩码的按位取反上计算位元 OR,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的地寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;  
EXEC.u64 = (S0.u64 | ~EXEC.u64);  
D0.u64 = saveexec.u64;  
SCC = EXEC.u64 != 0ULL
```

**S\_NAND\_SAVEEXEC\_B64****37**

在标量输入和 EXEC 掩码上计算位元 NAND,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的寄存器中。

```
saveexec = EXEC.u64;  
EXEC.u64 = ~(S0.u64 & EXEC.u64);  
D0.u64 = saveexec.u64;  
SCC = EXEC.u64 != 0ULL
```

**S\_NOR\_SAVEEXEC\_B64****38**

在标量输入和 EXEC 口罩上计算位元 NOR, 将计算结果存储到 EXEC 口罩中, 设定 SCC 如果计算结果为非零, 并将 EXEC 口罩的原始值存储到标量目的地寄存器中。

```
saveexec = EXEC.u64;  
EXEC.u64 = ~(S0.u64 | EXEC.u64);  
D0.u64 = saveexec.u64;  
SCC = EXEC.u64 != 0ULL
```

**S\_XNOR\_SAVEEXEC\_B64****39**

在标量输入和 EXEC 口罩上计算位元 XNOR, 将计算结果存储到 EXEC 口罩中, 设定 SCC 如果计算结果为非零, 并将 EXEC 口罩的原始值存储到标量目的地寄存器中。

```

saveexec = EXEC.u64;
EXEC.u64 = ~(S0.u64 ^ EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL

```

## S\_QUADMASK\_B32

40

将像素遮罩从标量输入到四面罩中,将结果存储在标量寄存器中,如果结果为非零,则设置 SCC.

```

tmp = 0U;
for i in 0 : 7 do
    tmp[i] = S0.u32[i * 4 +: 4] != 0U
endfor;
D0.u32 = tmp;
SCC = D0.u32 != 0U

```

说明

要进行反向操作,请参见 S\_BITREPLICATE\_B64\_B32.

## S\_QUADMASK\_B64

41

将像素遮罩从标量输入到四面罩中,将结果存储在标量寄存器中,如果结果为非零,则设置 SCC.

```

tmp = 0ULL;
for i in 0 : 15 do
    tmp[i] = S0.u64[i * 4 +: 4] != 0ULL
endfor;
D0.u64 = tmp;
SCC = D0.u64 != 0ULL

```

说明

要进行反向操作,请参见 S\_BITREPLICATE\_B64\_B32.

## S\_MOVRELS\_B32

42

将数据从一个相对索引的标量寄存器移动到另一个标量寄存器中.

```

addr = SRC0.u32;
// 指令的原始值

```



```
addr += M0.u32[31 : 0];  
D0.b32 = SGPR[addr].b32
```

说明

示例:以下指令序列执行移动 s5 QQ s17 :

```
s_mov_b32 m0, 10  
s_movrels_b32 s5, s7
```

---

## S\_MOVRELS\_B64

43

将数据从一个相对索引的标量寄存器移动到另一个标量寄存器中.

M0.u 的索引和 SRC0.u 的操作地址对于这个操作来说必须是均匀的.

```
addr = SRC0.u32;  
// 指令的原始值  
addr += M0.u32[31 : 0];  
D0.b64 = SGPR[addr].b64
```

---

## S\_MOVRELD\_B32

44

将数据从一个标量输入器移动到一个相对索引的标量寄存器中.

```
addr = DST.u32;  
// 指令的原始值  
addr += M0.u32[31 : 0];  
SGPR[addr].b32 = S0.b32
```

说明

示例: 以下指令序列执行移动 s15 QQ s7 :

```
s_mov_b32 m0, 10  
s_movreld_b32 s5, s7
```

---

## S\_MOVRELD\_B64

45

将数据从一个标量输入器移动到一个相对索引的标量寄存器中.

M0.u 的索引和 DST.u 的操作地址必须对于这个操作来说是均匀的。

```
addr = DST.u32;
// 指令的原始值
addr += M0.u32[31 : 0];
SGPR[addr].b64 = S0.b64
```

## S\_CBRANCH\_JOIN

46

有条件的分支结合点(条件分支块的结束)。

S0 是保存的 CSP 值。 相关指令参见 S\_CBRANCH\_G\_FORK 和 S\_CBRANCH\_I\_FORK。

```
saved_csp = S0.u32;
if WAVE_MODE.CSP.u32 == saved_csp then
    PC += 4LL;
    // 第二次到 JOIN:继续节目。
else
    WAVE_MODE.CSP -= 3'1U;
    // 第一次到 JOIN;跳到其他 FORK 路径。
    { PC, EXEC } = SGPR[WAVE_MODE.CSP.u32 * 4U].b128;
    // 从 4 个连续的 SGPRs 中读取 128 位。
endif
```

## S\_ABS\_I32

48

计算一个标量输入的绝对值,将结果存储到一个标量寄存器中,如果结果为非零,则设置 SCC。

```
D0.i32 = S0.i32 < 0 ? -S0.i32 : S0.i32;
SCC = D0.i32 != 0
```

说明

功能实例:

```
S_ABS_I32(0x00000001) => 0x00000001
S_ABS_I32(0x7fffffff) => 0x7fffffff
S_ABS_I32(0x80000000) => 0x80000000 // 注意,这是负的!
S_ABS_I32(0x80000001) => 0x7fffffff
S_ABS_I32(0x80000002) => 0x7ffffffe
S_ABS_I32(0xffffffff) => 0x00000001
```

**S\_SET\_GPR\_IDX\_IDX****50**

设置向量 GPR 索引中使用的索引。

S\_SET\_GPR\_IDX\_ON, S\_SET\_GPR\_IDX\_OFF, S\_SET\_GPR\_IDX\_MODE 和 S\_SET\_GPR\_IDX\_IDX 是相关的指令。

```
M0[7 : 0] = S0.u32[7 : 0].b8
```

**S\_ANDN1\_SAVEEXEC\_B64****51**

在 EXEC 掩码和按位取反标量输入时用位元计算,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的地寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
EXEC.u64 = (~S0.u64 & EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

**S\_ORN1\_SAVEEXEC\_B64****52**

在 EXEC 掩码和按位取反标量输入上计算位元 OR,将计算结果存储到 EXEC 掩码中,如果计算结果为非零,则设置 SCC,并将 EXEC 掩码的原始值存储到标量目的地寄存器中。

最初的 EXEC 口罩在进行位点操作前保存到目的地 SGPRs。

```
saveexec = EXEC.u64;
EXEC.u64 = (~S0.u64 | EXEC.u64);
D0.u64 = saveexec.u64;
SCC = EXEC.u64 != 0ULL
```

**S\_ANDN1\_WREXEC\_B64****53**

在 EXEC 掩码和按位取反标量输入时按位法计算,将计算结果存储在 EXEC 掩码和标量目的地寄存器中,如果计算结果为非零,则设置 SCC。

与 SAVEEXEC 系列的 opcode 不同,写给目标 SGPRs 的值是位码结果的结果。EXEC 和目标 SGPRs 在本指令结尾处具有相同的值,本指令旨在帮助加速瀑布。

```
EXEC.u64 = (~S0.u64 & EXEC.u64);
D0.u64 = EXEC.u64;
SCC = EXEC.u64 != 0ULL
```

## S\_ANDN2\_WREXEC\_B64

54

按位数计算,并用 EXEC 掩码的刻度输入和按位取反来计算,将计算结果存储到 EXEC 掩码中,也存储在刻度目标寄存器中,如果计算结果为非零,则设置 SCC。

与 SAVEEXEC 系列的 opcode 不同, 写给目标 SGPRs 的值是位码结果的结果. EXEC 和目标 SGPRs 在本指令结尾处具有相同的值, 本指令旨在帮助加速瀑布。

```
EXEC.u64 = (S0.u64 & ~EXEC.u64);
D0.u64 = EXEC.u64;
SCC = EXEC.u64 != 0ULL
```

### 说明

特别是使用 WREXEC 来优化以下瀑布代码序列,而不是两个独立的标量 ops:

```
// V0 持有每个通道的指数值
// 保存执行掩码以在结尾恢复
s_mov_b64 s2, exec
// 剩余(未处理)线程的执行掩码
s_mov_b64 s4, exec
loop:
// 获取第一个活动道的索引值
v_readfirstlane_b32 s0, v0
// 找到指数值相同的所有其他通道
v_cmpx_eq s0, v0
<0P> // 使用当前 EXEC 掩码进行操作。S0 持有索引。
// 刚刚执行的线程外罩
// s_andn2_b64 s4, s4, 执行
// s_mov_b64 执行,s4
s_andn2_wrexec_b64 s4, s4 // 替换以上 2 项行动
// 重复执行,直到 EXEC==0。
s_cbranch_scc1 loop
s_mov_b64 exec, s2
```

## S\_BITREPLICATE\_B64\_B32

55

用两个实例代替每点 32 点的标量输入,并将结果存储到 64 点的标量寄存器中。

```
tmp = S0.u32;  
for i in 0 : 31 do  
    D0.u64[i * 2] = tmp[i];  
    D0.u64[i * 2 + 1] = tmp[i]  
endfor
```

#### 说明

此 opcode 可用于将四面罩转换成像素遮罩;在 s0 中给定四面罩,以下序列在 s2 中生成像素遮罩:

```
s_bitreplicate_b64 s2, s0  
s_bitreplicate_b64 s2, s2
```

要进行反向操作,请参见 S\_QUADMASK\_B64.

---

## 12.4. SOPC 指令



这种格式的指令可能使用一个 32 位的字元常数,该常数在指令后立即出现.

S\_CMP\_EQ\_I32

0

将 SCC 设置为 1 if,第一个标量输入等于第二个标量输入。

```
SCC = S0.i32 == S1.i32
```

说明

请注意,S\_CMP\_EQ\_I32 和 S\_CMP\_EQ\_U32 是相同的 opcode,但两者都规定了对称性.

S\_CMP\_LG\_I32

1

将 SCC 设置为 1 if 第一个标量输入小于或大于第二个标量输入。

```
SCC = S0.i32 <> S1.i32
```

说明

请注意,S\_CMP\_LG\_I32 和 S\_CMP\_LG\_U32 是相同的 opcode,但两者都规定了对称性.

S\_CMP\_GT\_I32

2

将 SCC 设置为 1 if 的第一个标量输入大于第二个标量输入。

```
SCC = S0.i32 > S1.i32
```

S\_CMP\_GE\_I32

3

将 SCC 设置为 1 iff 的第一个标量输入大于或等于第二个标量输入。

```
SCC = S0.i32 >= S1.i32
```

**S\_CMP\_LT\_I32****4**

将 SCC 设置为 1 if 的第一个标量输入小于第二个标量输入。

```
SCC = S0.i32 < S1.i32
```

**S\_CMP\_LE\_I32****5**

将 SCC 设置为 1 if 的第一个标量输入小于或等于第二个标量输入。

```
SCC = S0.i32 <= S1.i32
```

**S\_CMP\_EQ\_U32****6**

将 SCC 设置为 1 if 第一个标量输入等于第二个标量输入。

```
SCC = S0.u32 == S1.u32
```

说明

请注意,S\_CMP\_EQ\_I32 和 S\_CMP\_EQ\_U32 是相同的 opcode,但两者都规定了对称性.

**S\_CMP\_LG\_U32****7**

将 SCC 设置为 1 if 第一个标量输入小于或大于第二个标量输入。

```
SCC = S0.u32 <> S1.u32
```

说明

请注意,S\_CMP\_LG\_I32 和 S\_CMP\_LG\_U32 是相同的 opcode,但两者都规定了对称性.

**S\_CMP\_GT\_U32****8**

将 SCC 设置为 1 if 的第一个标量输入大于第二个标量输入。

```
SCC = S0.u32 > S1.u32
```

**S\_CMP\_GE\_U32****9**

将 SCC 设置为 1 iff 的第一个标量输入大于或等于第二个标量输入。

```
SCC = S0.u32 >= S1.u32
```

**S\_CMP\_LT\_U32****10**

将 SCC 设置为 1 if 的第一个标量输入小于第二个标量输入。

```
SCC = S0.u32 < S1.u32
```

**S\_CMP\_LE\_U32****11**

将 SCC 设置为 1 if 的第一个标量输入小于或等于第二个标量输入。

```
SCC = S0.u32 <= S1.u32
```

**S\_BITCMP0\_B32****12**

从第一个标量输入中根据第二个标量输入中的索引抽取一点,然后将 SCC 设置为 1 if, 提取的比特等于 0。

```
SCC = S0.u32[S1.u32[4 : 0]] == 1'0U
```

**S\_BITCMP1\_B32****13**

根据第二个标量输入中的索引从第一个标量输入中抽出一一点,然后将 SCC 设置为 1 ,如果提取的比特等于 1。

```
SCC = S0.u32[S1.u32[4 : 0]] == 1'1U
```

**S\_BITCMP0\_B64****14**



从第一个标量输入中根据第二个标量输入中的索引抽取一点,然后将 SCC 设置为 1 if, 提取的比特等于 0。

```
SCC = S0.u64[S1.u32[5 : 0]] == 1'0U
```

## S\_BITCMP1\_B64

15

根据第二个标量输入中的索引从第一个标量输入中抽出一块,然后将 SCC 设置为 1,如果提取的比特等于 1。

```
SCC = S0.u64[S1.u32[5 : 0]] == 1'1U
```

## S\_SETVSKIP

16

启用或禁用 VSKIP 模式。

VSKIP 启用后,没有发布 VOP\*/M\*BUF/MIMG/DS/FLAT 指令. 注意 VSKIPed 内存指令不会操纵等待计数器;因此,如果存在未完成的内存请求,阴影器可能希望在启用 VSKIP 前发布 S\_WAITCNT 0,否则阴影器必须注意在等待计数中不计算 VSKIPed 指令。

```
VSKIP = S0.u32[S1.u32[4 : 0]]
```

说明

功能实例:

```
s_setvskip 1, 0    // 启用 vskip 模式。
s_setvskip 0, 0    // 禁用 vskip 模式。
```

## S\_SET\_GPR\_IDX\_ON

17

启用 GPR 索引模式。

在此之后的向量操作根据 M0 的内容执行相对的 GPR 地址. 该索引在 SRC0 操作中指定。SRC1 字段的原始位值被读取并用于设置启用位值. S1[0] = VSRC0\_REL, S1[1] = VSRC1\_REL, S1[2] = VSRC2\_REL 和 S1[3] = VDST\_REL。

S\_SET\_GPR\_IDX\_ON, S\_SET\_GPR\_IDX\_OFF, S\_SET\_GPR\_IDX\_MODE 和 S\_SET\_GPR\_IDX\_IDX 是相关的指令。

```
WAVE_MODE.GPR_IDX_EN = 1'1U;  
M0[7 : 0] = S0.u32[7 : 0].b8;  
M0[15 : 12] = SRC1.u32[3 : 0].b4;  
// 这是原始 S1 字段的直接内容  
// M0 的剩余位元未修改。
```

## S\_CMP\_EQ\_U64

18

将 SCC 设置为 1 if 第一个标量输入等于第二个标量输入。

```
SCC = S0.u64 == S1.u64
```

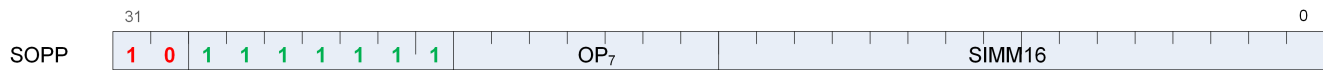
## S\_CMP\_LG\_U64

19

将 SCC 设置为 1 if 第一个标量输入小于或大于第二个标量输入。

```
SCC = S0.u64 <> S1.u64
```

## 12.5. SOPP 指令



S\_NOP

0

什么都不做。下一份指令的延迟发布由少量固定的金额。

根据 SIMM16[3:0] 插入 0..15 个等待状态。0x0 表示下一条指令可在下一个时钟周期发射；0xf 表示下一条指令可在 16 个时钟周期之后发射。

```
for i in 0U : SIMM16.u16[3 : 0].u32 do
    nop()
endfor
```

说明

实例:

```
s_nop 0          // 等待 1 个周期。
s_nop 0xf        // 等待 16 个周期。
```

S\_ENDPGM

1

程序结束; 终止波前端。

执行本指令前，硬件隐式执行 S\_WAITCNT 0。本指令用于上下文切换的版本见 S\_ENDPGM\_SAVED。

S\_BRANCH

2

跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
PC = PC + signext(SIMM16.i16 * 16'4) + 4LL;
// 短暂跳跃。
```

说明

长跳或间接跳用 S\_SETPC\_B64.

实例:

```

s_branch label    // 设置 SIMM16 = +4 = 0x0004。
s_nop 0          // 4 bytes
label:
s_nop 0          // 4 bytes
s_branch label    // 设定 SIMM16 = -8 = 0xffff8

```

## S\_WAKEUP

3

允许波浪在它的线组中“正在”所有其他波浪,以迫使它们从 S\_SLEEP 指令中提前醒来。

如果波没有入睡,便忽略了 ping。这样就可以在记忆位置上有效投票。投票的波可以在记忆读数之间长长的 S\_SLEEP 中坐着,但是,写值的波可以让他们在数据可用后提早醒来。这种方法也可以避免种族冲突,因为任何在完成 S\_SLEEP 时错过 ping 恢复的波。

如果执行 S\_WAKEUP 的波处于线组(in\_tg set)中,那么它就会唤醒所有与同一线组 ID 相关的波。否则, S\_WAKEUP 会被作为 S\_NOP 处理。

## S\_CBRANCH\_SCC0

4

如果 SCC 为 0,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```

if SCC == 1'0U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif

```

## S\_CBRANCH\_SCC1

5

如果 SCC 为 1,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```

if SCC == 1'1U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif

```

**S\_CBRANCH\_VCCZ**

6

如果 VCCZ 为 1,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if VCCZ.u1 == 1'1U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_CBRANCH\_VCCNZ**

7

如果 VCCZ 为 0,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if VCCZ.u1 == 1'0U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_CBRANCH\_EXECZ**

8

如果 EXECZ 为 1,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if EXECZ.u1 == 1'1U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_CBRANCH\_EXECNZ**

9

如果 EXECZ 为 0,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if EXECZ.u1 == 1'0U then
```

```
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

## S\_BARRIER

10

在一个线组内同步波浪。

如果线索组的全部波尚未创建, 请在启动前等待整个组。如果线索组中的某些波已经终止, 则仅等待幸存的波。屏障是合法的陷阱处理器。

屏障指令不会等待任何计数器在发布前到达零。如果屏障用于保护未完成的内存操作, 则在屏障之前使用适当的 S\_WAITCNT 指令。

## S\_SETKILL

11

如果瞬间常数中最不重要的一点是 1。

主要用于调试杀死波宿主命令行为。

## S\_WAITCNT

12

等待未完成的本地数据共享、向量内存和导出指令的计数达到或低于指定水平。

```
SIMM16[3:0] = vmcount (vector memory operations) lower bits [3:0],
```

```
SIMM16[6:4] = export/mem-write-data count,
```

```
SIMM16[11:8] = LGKMcnt (scalar-mem/GDS/LDS count),
```

```
SIMM16[15:14] = vmcount (vector memory operations) upper bits [5:4].
```

## S\_SETHALT

13

设置或清除 HALT 状态位。

将 HALT 位值设定为 SIMM16[0]; 1 = stop, 0 = clear HALT bit. 停止旗在 Priv = 1 (内置陷阱处理器) 时被忽略,但如果 HALT 此时仍被设置, 处理器返回后阴影就会停止。

S\_SLEEP

14

造成波浪沉睡至~8000 钟.

该波睡眠为(64\*(SIMM16[6:0]-1). 64\*SIMM16[6:0])时钟,确切的延迟量约为 S\_NOP. 当 SIMM16[6:0]为零时,不发生睡眠.

说明

实例:

```
s_sleep 0      // 等待 0 时钟。
s_sleep 1      // 等待 1-64 个时钟周期。
s_sleep 2      // 等 65-128 钟。
```

S\_SETPRIO

15

更改波用户优先级.

用户设置的波优先级设定为 SIMM16[1:0]. 0=最低,3=最高,总波优先级为{SPIPrio[1:0],UserPrio[1:0],WaveAge[3:0]}.

S\_SENDMSG

16

向图形控制硬件发送上游消息。

SIMM16[9:0]包含消息类型.

说明

S\_SENDMSGHALT

17

向上游控制硬件发送消息,然后 HALT 的波前端;详情请参见 S\_SENDMSG.

S\_TRAP

18

输入陷阱处理器。

此指令可能是内部生成的,也可能是对主机陷阱( HT = 1) 或例外的回应. TrapID

0 被保留用于硬件用途,不应在阴影生成的陷阱中使用.

```
TrapID = SIMM16.u16[7 : 0];
"Wait for all instructions to complete";
// PC 传入陷阱处理器指 S_TRAP 本身,
// *not* to the next instruction.
{ TTMP[1], TTMP[0] } = { 3'0, PCRewind[3 : 0], HT[0], TrapID[7 : 0], PC[47 : 0] };
PC = TBA.i64;
// 陷阱基础地址
WAVE_STATUS.PRIV = 1'1U
```

S\_ICACHE\_INV

19

无效整个一级指令缓存。

在此指示之后,必须有 16 个单独的 S\_NOP 指令或跳动/分支指令,以确保内部指令缓冲器也无效.

S\_INCPERFLEVEL

20

SIMM16[3:0]中指定的增量性能计数器 1.

S\_DECPERFLEVEL

21

SIMM16[3:0]指定的减值性能计数器乘 1.

S\_TTRACEDATA

22

将 M0 作为用户数据发送到线程跟踪流.

S\_CBRANCH\_CDBGSYS

23

如果设定了系统调试旗,则跳转到相对于当前 PC 的恒定偏移.

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if WAVE_STATUS.COND_DBG_SYS.u32 != 0U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```



**S\_CBRANCH\_CDBGUSER****24**

如果设置了用户调试旗,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if WAVE_STATUS.COND_DBG_USER.u32 != 0U then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_CBRANCH\_CDBGSYS\_OR\_USER****25**

如果系统调试旗或用户调试旗被设定,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if (WAVE_STATUS.COND_DBG_SYS || WAVE_STATUS.COND_DBG_USER) then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_CBRANCH\_CDBGSYS\_AND\_USER****26**

如果同时设置系统调试旗和用户调试旗,则跳转到相对于当前 PC 的恒定偏移。

字面上的论点是一个签名的 DWORD 相对下一个指令的 PC 的抵消。

```
if (WAVE_STATUS.COND_DBG_SYS && WAVE_STATUS.COND_DBG_USER) then
    PC = PC + signext(SIMM16.i16 * 16'4) + 4LL
else
    PC = PC + 4LL
endif
```

**S\_ENDPGM\_SAVED****27**

程序的结束; 表示一个波已被上下文切换陷阱处理器保存并终止波前端。

执行本指令前，硬件隐式执行 S\_WAITCNT 0。

补充变种见 S\_ENDPGM。

## S\_SET\_GPR\_IDX\_OFF

28

清除 GPR 索引模式。

之后的向量操作不会执行相对的 GPR 地址,无论 M0 的内容如何。此指令不会修改 M0。

S\_SET\_GPR\_IDX\_ON, S\_SET\_GPR\_IDX\_OFF, S\_SET\_GPR\_IDX\_MODE 和 S\_SET\_GPR\_IDX\_IDX 是相关的指令。

```
WAVE_MODE.GPR_IDX_EN = 1'0U
```

## S\_SET\_GPR\_IDX\_MODE

29

修改向量 GPR 索引所用的模式。

源字段的原始内容被读取并用于设置启用位. SIMM16[0] = VSRC0\_REL, SIMM16[1] = VSRC1\_REL, SIMM16[2] = VSRC2\_REL 和 SIMM16[3] = VDST\_REL.

S\_SET\_GPR\_IDX\_ON, S\_SET\_GPR\_IDX\_OFF, S\_SET\_GPR\_IDX\_MODE 和 S\_SET\_GPR\_IDX\_IDX 是相关的指令。

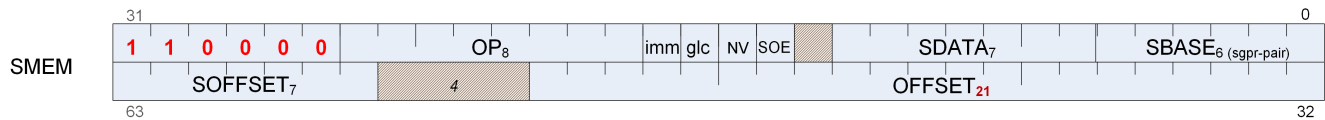
```
M0[15 : 12] = SIMM16.u16[3 : 0].b4
```

12.5.1. 发送消息

S\_SENDMSG 指令将消息类型编码为 M0,也可以从 SIMM16 字段发送数据,有时也可以从 EXEC 发送数据.

| 消息     | SIMM16[3:0] | SIMM16[6:4] | 载荷                                |
|--------|-------------|-------------|-----------------------------------|
| 无      | 0           | -           | 非法                                |
| 中断     | 1           | -           | M0[23:0]携带数据载荷                    |
| 保存波浪   | 4           | -           | 用于上下文切换                           |
| 悬浮波源   | 5           | -           | 停止新浪生成                            |
| 停止波涛   | 6           | -           | 停止这个 Vmid 的所有运行中的波                |
| 获取门铃标识 | 10          | -           | 将门铃返回到 EXEC 中,门铃物理地址以比特为单位[12:3]. |

12.6. SMEM 指令



S\_LOAD\_DWORD 0

从标量内存中装入 32 位数据到标量寄存器.

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略.

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消.

S\_LOAD\_DWORDX2 1

从标量内存中装入 64 位数据到标量寄存器.

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略.

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_LOAD\_DWORDX4

2

从标量内存中装入 128 位数据到标量寄存器。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_LOAD\_DWORDX8

3

从标量内存中装入 256 位数据到标量寄存器。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32;
SDATA[159 : 128] = MEM[addr + 16U].b32;
SDATA[191 : 160] = MEM[addr + 20U].b32;
SDATA[223 : 192] = MEM[addr + 24U].b32;
SDATA[255 : 224] = MEM[addr + 28U].b32
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_LOAD\_DWORDX16

4

从标量内存中装入 512 位数据到一个标量寄存器。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
```

```

SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32;
SDATA[159 : 128] = MEM[addr + 16U].b32;
SDATA[191 : 160] = MEM[addr + 20U].b32;
SDATA[223 : 192] = MEM[addr + 24U].b32;
SDATA[255 : 224] = MEM[addr + 28U].b32;
SDATA[287 : 256] = MEM[addr + 32U].b32;
SDATA[319 : 288] = MEM[addr + 36U].b32;
SDATA[351 : 320] = MEM[addr + 40U].b32;
SDATA[383 : 352] = MEM[addr + 44U].b32;
SDATA[415 : 384] = MEM[addr + 48U].b32;
SDATA[447 : 416] = MEM[addr + 52U].b32;
SDATA[479 : 448] = MEM[addr + 56U].b32;
SDATA[511 : 480] = MEM[addr + 60U].b32

```

#### 说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数, 则该常数为 SIGNED BYTE 抵消。

## S\_SCRATCH\_LOAD\_DWORD

5

从标量抓痕孔径装入 32 位数据到标量寄存器。

```

addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32

```

#### 说明

如果将冲抵定为 SGPR, 则 SGPR 包含一个 UNSIGNED 64 字节冲抵, 与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数, 则该常数为 SIGNED BYTE 抵消。

## S\_SCRATCH\_LOAD\_DWORDX2

6

从标量抓痕孔径装入 64 位数据到标量寄存器。

```

addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32

```

#### 说明

如果将冲抵定为 SGPR, 则 SGPR 包含一个 UNSIGNED 64 字节冲抵, 与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

---

## S\_SCRATCH\_LOAD\_DWORDX4

7

从标量刮伤孔径装入 128 位数据到标量寄存器。

```
addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32
```

说明

如果将冲抵定为 SGPR,则 SGPR 包含一个 UNSIGNED 64 字节冲抵,与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

---

## S\_BUFFER\_LOAD\_DWORD

8

从标量缓冲表面装入 32 位数据到标量寄存器。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

---

## S\_BUFFER\_LOAD\_DWORDX2

9

从标量缓冲表面装入 64 位数据到标量寄存器。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_BUFFER\_LOAD\_DWORDX4

10

从标量缓冲表面装入 128 位数据到标量寄存器。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_BUFFER\_LOAD\_DWORDX8

11

从标量缓冲表面装入 256 位数据到标量寄存器。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32;
SDATA[159 : 128] = MEM[addr + 16U].b32;
SDATA[191 : 160] = MEM[addr + 20U].b32;
SDATA[223 : 192] = MEM[addr + 24U].b32;
SDATA[255 : 224] = MEM[addr + 28U].b32
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_BUFFER\_LOAD\_DWORDX16

12

从标量缓冲表面装入 512 位数据到标量寄存器。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
SDATA[31 : 0] = MEM[addr].b32;
SDATA[63 : 32] = MEM[addr + 4U].b32;
```

```

SDATA[95 : 64] = MEM[addr + 8U].b32;
SDATA[127 : 96] = MEM[addr + 12U].b32;
SDATA[159 : 128] = MEM[addr + 16U].b32;
SDATA[191 : 160] = MEM[addr + 20U].b32;
SDATA[223 : 192] = MEM[addr + 24U].b32;
SDATA[255 : 224] = MEM[addr + 28U].b32;
SDATA[287 : 256] = MEM[addr + 32U].b32;
SDATA[319 : 288] = MEM[addr + 36U].b32;
SDATA[351 : 320] = MEM[addr + 40U].b32;
SDATA[383 : 352] = MEM[addr + 44U].b32;
SDATA[415 : 384] = MEM[addr + 48U].b32;
SDATA[447 : 416] = MEM[addr + 52U].b32;
SDATA[479 : 448] = MEM[addr + 56U].b32;
SDATA[511 : 480] = MEM[addr + 60U].b32

```

#### 说明

若偏移由 SGPR 指定，该 SGPR 保存无符号字节偏移，最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数，则该常数为 SIGNED BYTE 抵消。

## S\_STORE\_DWORD

16

将 32 位数据从标量寄存器存储到标量内存中。

```

addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0]

```

#### 说明

若偏移由 SGPR 指定，该 SGPR 保存无符号字节偏移，最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数，则该常数为 SIGNED BYTE 抵消。

## S\_STORE\_DWORDX2

17

将 64 位数据从一个标量寄存器存储到标量内存中。

```

addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32]

```

#### 说明

若偏移由 SGPR 指定，该 SGPR 保存无符号字节偏移，最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数，则该常数为 SIGNED BYTE 抵消。



## S\_STORE\_DWORDX4

18

将 128 位数据从一个标量寄存器存储到标量内存中。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32];
MEM[addr + 8U].b32 = SDATA[95 : 64];
MEM[addr + 12U].b32 = SDATA[127 : 96]
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数, 则该常数为 SIGNED BYTE 抵消。

## S\_SCRATCH\_STORE\_DWORD

21

将 32 位数据从标量寄存器存储到标量刮伤孔径中。

```
addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0]
```

说明

如果将冲抵定为 SGPR, 则 SGPR 包含一个 UNSIGNED 64 字节冲抵, 与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数, 则该常数为 SIGNED BYTE 抵消。

## S\_SCRATCH\_STORE\_DWORDX2

22

将 64 比特的数据从一个标量寄存器存储到标量刮痕孔径中。

```
addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32]
```

说明

如果将冲抵定为 SGPR, 则 SGPR 包含一个 UNSIGNED 64 字节冲抵, 与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_SCRATCH\_STORE\_DWORDX4

23

将 128 比特的数据从一个标量寄存器存储到标量刮痕孔径中。

```
addr = CalcScalarScratchAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32];
MEM[addr + 8U].b32 = SDATA[95 : 64];
MEM[addr + 12U].b32 = SDATA[127 : 96]
```

说明

如果将冲抵定为 SGPR,则 SGPR 包含一个 UNSIGNED 64 字节冲抵,与其他刮伤操作一致。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_BUFFER\_STORE\_DWORD

24

将 32 位数据从一个标量寄存器存储到一个标量缓冲表面。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0]
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消。

## S\_BUFFER\_STORE\_DWORDX2

25

将 64 位数据从标量寄存器存储到标量缓冲表面。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32]
```

说明

若偏移由 SGPR 指定,该 SGPR 保存无符号字节偏移,最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消.

S\_BUFFER\_STORE\_DWORDX4

26

将 128 位数据从标量寄存器存储到标量缓冲表面.

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
MEM[addr].b32 = SDATA[31 : 0];
MEM[addr + 4U].b32 = SDATA[63 : 32];
MEM[addr + 8U].b32 = SDATA[95 : 64];
MEM[addr + 12U].b32 = SDATA[127 : 96]
```

说明

若偏移由 SGPR 指定, 该 SGPR 保存无符号字节偏移, 最低 2 位会被忽略。

如果将抵消指定为直接的 21 位常数,则该常数为 SIGNED BYTE 抵消.

S\_DCACHE\_INV

32

无效的标量 (L0) 数据缓存。

S\_DCACHE\_WB

33

在标量 (L0) 数据缓存中写回脏数据。

S\_DCACHE\_INV\_VOL

34

无效标量 (L0) 数据缓存挥发线条。

S\_DCACHE\_WB\_VOL

35

在标量(L0)数据缓存挥发行中写入脏数据.

S\_MEMTIME

36

返回当前 64 位时间戳。

**S\_MEMREALTIME****37**

返回当前 64 位 RTC。

**S\_DCACHE\_DISCARD****40**

丢弃一条肮脏的标量( L0) 数据缓存行。缓存行是 64 字节。

通常情况下, 脏缓存线( 由遮阳器写成) 会被写回内存, 但此指令允许遮阳器无效, 而不是写回它以前写的缓存线。这是当遮阳器知道它不再需要该数据时使用的性能优化。

地址的计算与 S\_STORE\_DWORD 相同, 但忽略了 6 个 LSB 以获得 64 字节对齐的地址. LGKM 计数为本 opcode 递增 1 个。

**S\_DCACHE\_DISCARD\_X2****41**

丢弃连续两条肮脏的标量( L0) 数据缓存行。缓存行是 64 字节。

通常情况下, 脏缓存线( 由遮阳器写成) 会被写回内存, 但此指令允许遮阳器无效, 而不是写回它以前写的缓存线。这是当遮阳器知道它不再需要该数据时使用的性能优化。

地址的计算与 S\_STORE\_DWORD 相同, 但忽略了 6 个 LSB, 以获得 64 字节对齐的地址. LGKM 计数在此 opcode 中递增 2 个。

**S\_BUFFER\_ATOMIC\_SWAP****64**

在数据寄存器中交换一个无符号的 32 位整数值, 其位置位于一个标量缓冲面。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

**S\_BUFFER\_ATOMIC\_CMPSWAP****65**

比较两个未署名的 32 位整数值存储在数据比较寄存器和一个空间缓冲表面的位置。如果比较是等值, 则用数据源寄存器中的一个值修改内存位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA[31 : 0].u32;
```

```
cmp = DATA[63 : 32].u32;  
MEM[addr].u32 = tmp == cmp ? src : tmp;  
RETURN_DATA.u32 = tmp
```

## S\_BUFFER\_ATOMIC\_ADD

66

添加两个未署名的 32 位整数存储在数据寄存器中,和一个位于标量缓冲表面的位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 += DATA.u32;  
RETURN_DATA.u32 = tmp
```

## S\_BUFFER\_ATOMIC\_SUB

67

从存储在某一位置的标量缓冲表面的数值中减去一个未署名的 32 位整数数值存储在数据寄存器中。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 -= DATA.u32;  
RETURN_DATA.u32 = tmp
```

## S\_BUFFER\_ATOMIC\_SMIN

68

选择两个签名的 32 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置在标量缓冲表。用选定的值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);  
tmp = MEM[addr].i32;  
src = DATA.i32;  
MEM[addr].i32 = src < tmp ? src : tmp;  
RETURN_DATA.i32 = tmp
```

## S\_BUFFER\_ATOMIC\_UMIN

69

选择两个无符号的 32 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于一个标量缓冲表面。用选定的值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
```

```
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## S\_BUFFER\_ATOMIC\_SMAX

70

选择最多两个签名的 32 位整数输入, 前提是有两个数值存储在数据寄存器中, 以及一个位置位于标量缓冲表面。用选定的值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

## S\_BUFFER\_ATOMIC\_UMAX

71

选择两个无符号的 32 位整数输入的最大值, 因为有两个数值存储在数据寄存器中, 一个位置位于一个标量缓冲表面。用选定的数值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## S\_BUFFER\_ATOMIC\_AND

72

以位法计算并给出两个无符号的 32 位整数值存储在数据寄存器中, 以及一个位于标量缓冲表层中的位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
RETURN_DATA.b32 = tmp
```

## S\_BUFFER\_ATOMIC\_OR

73

计算位元 OR 给定了两个无符号的 32 位整数值存储在数据寄存器中, 并在一个标量缓冲面上设置一个位置。

```

addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp

```

## S\_BUFFER\_ATOMIC\_XOR

74

计算比特 XOR 给定了两个无符号的 32 位整数值存储在数据寄存器和一个位置在一个标量缓冲表面。

```

addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp

```

## S\_BUFFER\_ATOMIC\_INC

75

将一个无符号的 32 位整数值从一个位于标量缓冲面中的位置加宽到 0,如果值超过数据寄存器中的值。

```

addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp

```

## S\_BUFFER\_ATOMIC\_DEC

76

将一个无符号的 32 位整数值从一个位于一个平面缓冲面上,并有环绕的位点降为数据寄存器中的一个值,如果该减值产生负值的话。

```

addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
RETURN_DATA.u32 = tmp

```

## S\_BUFFER\_ATOMIC\_SWAP\_X2

96

在数据寄存器中交换一个无符号的 64 位整数值,其位置位于一个标量缓冲面。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = DATA.b64;
RETURN_DATA.b64 = tmp
```

## S\_BUFFER\_ATOMIC\_CMPSWAP\_X2

97

比较两个无符号的 64 位整数值, 存储在数据比较寄存器中, 和一个位于标量缓冲表层中的位置。使用数据源寄存器中的值来修改内存位置, 如果比较是等值的。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA[63 : 0].u64;
cmp = DATA[127 : 64].u64;
MEM[addr].u64 = tmp == cmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

## S\_BUFFER\_ATOMIC\_ADD\_X2

98

添加两个无符号的 64 位整数存储在数据寄存器中, 和一个位于标量缓冲表面的位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
MEM[addr].u64 += DATA.u64;
RETURN_DATA.u64 = tmp
```

## S\_BUFFER\_ATOMIC\_SUB\_X2

99

从存储在一个位置上的一个位于标量缓冲表面的数值中减去一个无符号的 64 位整数值, 存储在数据寄存器中。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
MEM[addr].u64 -= DATA.u64;
RETURN_DATA.u64 = tmp
```

## S\_BUFFER\_ATOMIC\_SMIN\_X2

100

选择两个签名的 64 位整数输入的最小值, 因为有两个数值存储在数据寄存器中, 一个位置位于一个标量缓冲表面。用选定的数值更新标量缓冲。



```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_UMIN\_X2

101

选择两个无符号的 64 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于一个标量缓冲表。用选定的数值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_SMAX\_X2

102

选择最多两个签名的 64 位整数输入,并给出两个存储在数据寄存器中的值和位于一个标量缓冲表面中的位置。用选定的值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_UMAX\_X2

103

选择两个无符号的 64 位整数输入的最大值,因为有两个数值存储在数据寄存器中,一个位置位于一个标量缓冲表面。用选定的数值更新标量缓冲。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src >= tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_AND\_X2

104

以位法计算并给出两个无符号的 64 位整数,存储在数据寄存器中,并在一个标量缓冲面上设置一个位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp & DATA.b64);
RETURN_DATA.b64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_OR\_X2

105

计算位元 OR 给定了两个无符号的 64 位整数,存储在数据寄存器和一个位于标量缓冲表面的位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp | DATA.b64);
RETURN_DATA.b64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_XOR\_X2

106

计算比特 XOR 给两个无符号的 64 位整数,存储在数据寄存器和一个位于标量缓冲表面的位置。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp ^ DATA.b64);
RETURN_DATA.b64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_INC\_X2

107

将一个无符号的 64 位整数,从一个位于标量缓冲面上的位置加宽到 0,如果值超过数据寄存器中的值。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

---

## S\_BUFFER\_ATOMIC\_DEC\_X2

108

将一个无符号的 64 位整数值从一个带有环绕的阶梯缓冲面上的位置降为数据寄存器中的一个值,如果减值产生负值的话。

```
addr = CalcScalarBufferAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```

## S\_ATOMIC\_SWAP

128

在数据寄存器中交换一个无符号的 32 位整数值,其位置位于标量内存中。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

## S\_ATOMIC\_CMPSWAP

129

比较两个无符号的 32 位整数值存储在数据比较寄存器中和一个位于标量内存中的位置。如果比较结果相等,则用数据源寄存器中的一个值来修改内存位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA[31 : 0].u32;
cmp = DATA[63 : 32].u32;
MEM[addr].u32 = tmp == cmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## S\_ATOMIC\_ADD

130

添加两个无符号的 32 位整数存储在数据寄存器中和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
MEM[addr].u32 += DATA.u32;
RETURN_DATA.u32 = tmp
```

## S\_ATOMIC\_SUB

131

从存储在标量内存位置的数值中减去一个无符号的 32 位整数值,存储在数据寄存器中。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
MEM[addr].u32 -= DATA.u32;
RETURN_DATA.u32 = tmp
```

## S\_ATOMIC\_SMIN

132

选择最少两个签名的 32 位整数输入, 给定两个存储在数据寄存器和标量内存中的位置。用所选值更新标量内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src < tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

## S\_ATOMIC\_UMIN

133

选择两个无符号的 32 位整数输入的最小值, 给两个存储在数据寄存器和标度内存中的位置。用所选值更新标度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## S\_ATOMIC\_SMAX

134

选择最多两个签名的 32 位整数输入, 给定两个存储在数据寄存器和标度内存中的位置。用所选值更新标度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

**S\_ATOMIC\_UMAX****135**

选择两个无符号的 32 位整数输入的最大值,因为有两个数值存储在数据寄存器中,一个位置位于刻度内存中。用选定的数值更新刻度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

**S\_ATOMIC\_AND****136**

计算比特并给出了两个无符号的 32 位整数值存储在数据寄存器中和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
RETURN_DATA.b32 = tmp
```

**S\_ATOMIC\_OR****137**

计算位元 OR 给定了两个无符号的 32 位整数值存储在数据寄存器中,和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp
```

**S\_ATOMIC\_XOR****138**

计算比特 XOR 给两个无符号的 32 位整数值,存储在数据寄存器中,并在标量内存中有一个位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp
```

**S\_ATOMIC\_INC****139**

将一个无符号的 32 位整数从标量内存中的位置加宽到 0,如果值超过数据寄存器中的值。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp
```

**S\_ATOMIC\_DEC****140**

将一个无符号的 32 位整数从标量内存中一个位置并加包,降为数据寄存器中的一个值,如果减值产生负值的话。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
RETURN_DATA.u32 = tmp
```

**S\_ATOMIC\_SWAP\_X2****160**

在数据寄存器中交换一个无符号的 64 位整数,其位置位于标量内存中。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = DATA.b64;
RETURN_DATA.b64 = tmp
```

**S\_ATOMIC\_CMPSWAP\_X2****161**

比较数据比较寄存器和标量内存中两个无符号的 64 位整数。如果比较,则用数据源寄存器中的一个值修改内存位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA[63 : 0].u64;
cmp = DATA[127 : 64].u64;
MEM[addr].u64 = tmp == cmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**S\_ATOMIC\_ADD\_X2****162**

添加两个无符号的 64 位整数存储在数据寄存器中,和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
MEM[addr].u64 += DATA.u64;
RETURN_DATA.u64 = tmp
```

**S\_ATOMIC\_SUB\_X2****163**

从储存在标量内存中的位置的数值中减去一个无符号的 64 位整数值。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
MEM[addr].u64 -= DATA.u64;
RETURN_DATA.u64 = tmp
```

**S\_ATOMIC\_SMIN\_X2****164**

选择两个签名的 64 位整数输入的最小值, 给两个存储在数据寄存器和刻度内存中的位置。用所选值更新刻度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**S\_ATOMIC\_UMIN\_X2****165**

选择两个无符号的 64 位整数输入的最小值, 给定两个存储在数据寄存器和刻度内存中的位置。用所选值更新刻度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**S\_ATOMIC\_SMAX\_X2****166**

选择最多两个签名的 64 位整数输入, 并给出两个存储在数据寄存器和刻度内存中的位置。用所选值更新刻度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**S\_ATOMIC\_UMAX\_X2****167**

选择两个无符号的 64 位整数输入的最大值,因为有两个数值存储在数据寄存器中,一个位置位于刻度内存中。用选定的数值更新刻度内存。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src >= tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**S\_ATOMIC\_AND\_X2****168**

以位法计算并给出两个无符号的 64 位整数值存储在数据寄存器和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp & DATA.b64);
RETURN_DATA.b64 = tmp
```

**S\_ATOMIC\_OR\_X2****169**

计算 bitwise OR 给定了两个无符号的 64 位整数值存储在数据寄存器中,和一个位于标量内存中的位置。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp | DATA.b64);
```



```
RETURN_DATA.b64 = tmp
```

## S\_ATOMIC\_XOR\_X2

170

计算位元 XOR 给定了两个无符号的 64 位整数值存储在数据寄存器中,和一个位于标量内存中的位置.

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp ^ DATA.b64);
RETURN_DATA.b64 = tmp
```

## S\_ATOMIC\_INC\_X2

171

将一个无符号的 64 位整数值从刻度内存中的位置加宽到 0,如果值超过数据寄存器中的值。

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

## S\_ATOMIC\_DEC\_X2

172

将一个无符号的 64 位整数值从刻度内存中一个位置(用包包)降为数据寄存器中的一个值,如果减值产生负值的话.

```
addr = CalcScalarGlobalAddr(SBASE.b32, SOFFSET.b32, OFFSET.i32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```



**V\_SUBREV\_F32****3**

从第二个输入中减掉第一个浮点输入,并将结果存储到向量寄存器中.

```
D0.f32 = S1.f32 - S0.f32
```

说明

0.5ULP 精度,异常态得到支持.

**V\_FMAC\_F64****4**

乘以两个浮点输入,利用融合乘法将结果累积到目的地寄存器中.

```
D0.f64 = fma(S0.f64, S1.f64, D0.f64)
```

**V\_MUL\_F32****5**

乘以两个浮点输入并将结果存储到向量寄存器中.

```
D0.f32 = S0.f32 * S1.f32
```

说明

0.5ULP 精度,异常态得到支持.

**V\_MUL\_I32\_I24****6**

乘以两个签名的 24 位整数输入,并将结果作为签名的 32 位整数存储到向量寄存器中.

```
D0.i32 = 32'I(S0.i24) * 32'I(S1.i24)
```

说明

这个 opcode 预计会和基本的单精度的 opcode 一样高效,因为它使用了单精度的浮点乘数. 另见 V\_MUL\_HI\_I32\_I24.

**V\_MUL\_HI\_I32\_I24****7**

乘以两个签名的 24 位整数输入,并将结果的高 32 位作为签名的 32 位整数存储到向量寄存器中。

```
D0.i32 = 32'I((64'I(S0.i24) * 64'I(S1.i24)) >> 32U)
```

说明

另见 V\_MUL\_I32\_I24.

## V\_MUL\_U32\_U24

8

将两个无符号的 24 位整数输入数乘以,并将结果作为无符号的 32 位整数存储到向量寄存器中。

```
D0.u32 = 32'U(S0.u24) * 32'U(S1.u24)
```

说明

这个 opcode 预计会和基本的单精度的 opcode 一样高效,因为它使用了单精度的浮点乘数. 另见 V\_MUL\_HI\_U32\_U24.

## V\_MUL\_HI\_U32\_U24

9

乘以两个无符号的 24 位整数输入,并将结果的高 32 位作为无符号的 32 位整数存储到向量寄存器中。

```
D0.u32 = 32'U((64'U(S0.u24) * 64'U(S1.u24)) >> 32U)
```

说明

另见 V\_MUL\_U32\_U24.

## V\_MIN\_F32

10

选择至少两个单精度浮点输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S0.f32)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S1.f32)))
elsif isNaN(64'F(S0.f32)) then
    D0.f32 = S1.f32
```

```

elseif isNAN(64'F(S1.f32)) then
    D0.f32 = S0.f32
elseif ((64'F(S0.f32) == +0.0) && (64'F(S1.f32) == -0.0)) then
    D0.f32 = S1.f32
elseif ((64'F(S0.f32) == -0.0) && (64'F(S1.f32) == +0.0)) then
    D0.f32 = S0.f32
else
    // 注意:这里没有 IEEE 案件,像 V_MAX_F32 一样.
    D0.f32 = S0.f32 < S1.f32 ? S0.f32 : S1.f32
endif

```

## V\_MAX\_F32

11

选择两个单精度浮点输入的最大值,并将结果存储到向量寄存器中。

```

if (WAVE_MODE.IEEE && isSignalNAN(64'F(S0.f32))) then
    D0.f32 = 32'F(cvtToQuietNAN(64'F(S0.f32)))
elseif (WAVE_MODE.IEEE && isSignalNAN(64'F(S1.f32))) then
    D0.f32 = 32'F(cvtToQuietNAN(64'F(S1.f32)))
elseif isNAN(64'F(S0.f32)) then
    D0.f32 = S1.f32
elseif isNAN(64'F(S1.f32)) then
    D0.f32 = S0.f32
elseif ((64'F(S0.f32) == +0.0) && (64'F(S1.f32) == -0.0)) then
    D0.f32 = S0.f32
elseif ((64'F(S0.f32) == -0.0) && (64'F(S1.f32) == +0.0)) then
    D0.f32 = S1.f32
elseif WAVE_MODE.IEEE then
    D0.f32 = S0.f32 >= S1.f32 ? S0.f32 : S1.f32
else
    D0.f32 = S0.f32 > S1.f32 ? S0.f32 : S1.f32
endif

```

## V\_MIN\_I32

12

选择最少两个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```

D0.i32 = S0.i32 < S1.i32 ? S0.i32 : S1.i32

```

## V\_MAX\_I32

13

选择最多两个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```

D0.i32 = S0.i32 >= S1.i32 ? S0.i32 : S1.i32

```

**V\_MIN\_U32****14**

选择两个无符号的 32 位整数输入的最小值,并将所选值存储到向量寄存器中。

```
D0.u32 = S0.u32 < S1.u32 ? S0.u32 : S1.u32
```

**V\_MAX\_U32****15**

选择两个无符号的 32 位整数输入的最大值,并将选定的数值存储到向量寄存器中。

```
D0.u32 = S0.u32 >= S1.u32 ? S0.u32 : S1.u32
```

**V\_LSHRREV\_B32****16**

鉴于第一个向量输入中的位移计数,计算第二个向量输入的逻辑位移右侧,并将结果存储到一个向量寄存器中。

```
D0.u32 = (S1.u32 >> S0[4 : 0].u32)
```

**V\_ASHRREV\_I32****17**

鉴于第一个向量输入中的位移计数,计算第二个向量输入的算术右移(保留符号位),并将结果存储到向量寄存器中。

```
D0.i32 = (S1.i32 >> S0[4 : 0].u32)
```

**V\_LSHLREV\_B32****18**

鉴于第一个向量输入中的位移计数,计算第二个向量输入中左侧的逻辑位移,并将结果存储到一个向量寄存器中。

```
D0.u32 = (S1.u32 << S0[4 : 0].u32)
```

**V\_AND\_B32****19**

在两个向量输入上以位法计算,并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 & S1.u32)
```

说明

不支持输入和输出修改符。

**V\_OR\_B32****20**

在两个向量输入上按位计算 OR 并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 | S1.u32)
```

说明

不支持输入和输出修改符。

**V\_XOR\_B32****21**

在两个向量输入上计算比特 XOR,并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 ^ S1.u32)
```

说明

不支持输入和输出修改符。

**V\_FMAMK\_F32****23**

用字形常数乘以单精度浮点输入,并使用融合乘加添加第二精度浮点输入,并将结果存储到向量寄存器中。

```
D0.f32 = fma(S0.f32, SIMM32.f32, S1.f32)
```

说明

这个 opcode 不能使用 VOP3 编码,也不能使用输入/输出修改器。

**V\_FMAAK\_F32****24**

乘以两个单精度浮点数输入,并用融合乘法添加一个字形常数,然后将结果存储到向量寄存器中.

```
D0.f32 = fma(S0.f32, S1.f32, SIMM32.f32)
```

说明

这个 opcode 不能使用 VOP3 编码,也不能使用输入/输出修改器.

**V\_ADD\_CO\_U32****25**

增加两个未署名的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中.

```
tmp = 64'U(S0.u32) + 64'U(S1.u32);  
VCC.u64[laneId] = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// VCC 是 V_ADDCO_U32 的 UNSSIGED 外溢/运出.  
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

**V\_SUB\_CO\_U32****26**

从第一个输入中减掉第二个无符号的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中.

```
tmp = S0.u32 - S1.u32;  
VCC.u64[laneId] = S1.u32 > S0.u32 ? 1'1U : 1'0U;  
// VCC 是 V_SUBCO_U32 的 UNSSIGED 外溢/运出.  
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).



**V\_SUBREV\_CO\_U32****27**

从第二次输入中减掉第一个无符号的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中。

```
tmp = S1.u32 - S0.u32;
VCC.u64[laneId] = S0.u32 > S1.u32 ? 1'1U : 1'0U;
// VCC 是 V_SUBB_CO_U32 的 UNSSIGED 外溢/运出。
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

**V\_ADDC\_CO\_U32****28**

增加两个未署名的 32 位整数输入和一个随身携带的掩码的一点,将结果存储到向量寄存器中,并将随身携带的掩码存储到标量寄存器中。

```
tmp = 64'U(S0.u32) + 64'U(S1.u32) + VCC.u64[laneId].u64;
VCC.u64[laneId] = tmp >= 0x100000000ULL ? 1'1U : 1'0U;
// VCC 是 V_ADDC_CO_U32 的 UNSSIGED 外溢/运出。
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

**V\_SUBB\_CO\_U32****29**

从第一次输入中减掉第二个无符号的 32 位整数输入,从携带的掩码中减去一点,将结果存储到向量寄存器中,并将携带的掩码存储到标量寄存器中。

```
tmp = S0.u32 - S1.u32 - VCC.u64[laneId].u32;
VCC.u64[laneId] = 64'U(S1.u32) + VCC.u64[laneId].u64 > 64'U(S0.u32) ? 1'1U : 1'0U;
// VCC 是 V_SUBB_CO_U32 的 UNSSIGED 外溢/运出。
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

### V\_SUBBREV\_CO\_U32

30

从第二次输入中减去第一个无符号的 32 位整数输入,从携带的掩码中减去一点,将结果存储到向量寄存器中,并将携带的掩码存储到标量寄存器中.

```
tmp = S1.u32 - S0.u32 - VCC.u64[laneId].u32;
VCC.u64[laneId] = 64'U(S0.u32) + VCC.u64[laneId].u64 > 64'U(S1.u32) ? 1'1U : 1'0U;
// VCC 是 V_SUBB_CO_U32 的 UNSSIGED 外溢/运出.
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

### V\_ADD\_F16

31

添加两个浮点输入并将结果存储到向量寄存器中.

```
D0.f16 = S0.f16 + S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

### V\_SUB\_F16

32

从第一个输入中减掉第二个浮点输入,并将结果存储到向量寄存器中.

```
D0.f16 = S0.f16 - S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

**V\_SUBREV\_F16****33**

从第二个输入中减掉第一个浮点输入,并将结果存储到向量寄存器中。

```
D0.f16 = S1.f16 - S0.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

**V\_MUL\_F16****34**

乘以两个浮点输入并将结果存储到向量寄存器中。

```
D0.f16 = S0.f16 * S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

**V\_MAC\_F16****35**

乘以两个浮点输入并累积结果到目的寄存器中. 为 OPSEL 执行 IEEE 规则和非标准规则。

```
tmp = S0.f16 * S1.f16 + D0.f16;  
if OPSEL.u4[3] then  
    D0 = { tmp.f16, D0[15 : 0] }  
else  
    D0 = { 16'0, tmp.f16 }  
endif
```

说明

支持舍入模式,例外旗帜,饱和度.

**V\_MADMK\_F16****36**

用字面常数乘以浮点输入,并添加第二个浮点输入,然后将结果存储到向量寄存器中。执行 IEEE 规则。

```
tmp = S0.f16 * SIMM16.f16 + S1.f16;
```

```
D0 = { 16'0, tmp.f16 }
```

#### 说明

此 opcode 无法使用 VOP3 编码, 也无法使用输入/输出修改器。支持回合模式、例外旗帜、饱和度。

### V\_MADAK\_F16

37

乘以两个浮点输入并添加一个字形常数,然后将结果存储到向量寄存器中。执行 IEEE 规则。

```
tmp = S0.f16 * S1.f16 + SIMM16.f16;  
D0 = { 16'0, tmp.f16 }
```

#### 说明

此 opcode 无法使用 VOP3 编码, 也无法使用输入/输出修改器。支持回合模式、例外旗帜、饱和度。

### V\_ADD\_U16

38

添加两个无符号的 16 位整数输入, 并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
D0.u16 = S0.u16 + S1.u16
```

#### 说明

支持饱和(无符号的 16 位整数域)。

### V\_SUB\_U16

39

从第一个输入中减去第二个无符号的 16 位整数输入, 并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
D0.u16 = S0.u16 - S1.u16
```

#### 说明

支持饱和(无符号的 16 位整数域)。

**V\_SUBREV\_U16****40**

从第二次输入中减去第一个无符号的 16 位整数输入,并将结果存储到向量寄存器中。没有随身携带或执行支持。

$$D0.u16 = S1.u16 - S0.u16$$

说明

支持饱和(无符号的 16 位整数域)。

**V\_MUL\_LO\_U16****41**

将两个无符号的 16 位整数输入数乘以,并将结果的低位数存储到向量寄存器中。

$$D0.u16 = S0.u16 * S1.u16$$

说明

支持饱和(无符号的 16 位整数域)。

**V\_LSHLREV\_B16****42**

鉴于第一个向量输入中的位移计数,计算第二个向量输入中左侧的逻辑位移,并将结果存储到一个向量寄存器中。

$$D0.u16 = (S1.u16 << S0[3 : 0].u32)$$
**V\_LSHRREV\_B16****43**

鉴于第一个向量输入中的位移计数,计算第二个向量输入的逻辑位移右侧,并将结果存储到一个向量寄存器中。

$$D0.u16 = (S1.u16 >> S0[3 : 0].u32)$$
**V\_ASHRREV\_I16****44**

鉴于第一个向量输入中的位数, 计算第二个向量输入的算术右移(保留符号位数)

向量输入并将结果存储到向量寄存器中。

```
D0.i16 = (S1.i16 >> S0[3 : 0].u32)
```

## V\_MAX\_F16

45

选择最多两个半精度浮点数输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S0.f16)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S1.f16)))
elsif isNaN(64'F(S0.f16)) then
    D0.f16 = S1.f16
elsif isNaN(64'F(S1.f16)) then
    D0.f16 = S0.f16
elsif ((64'F(S0.f16) == +0.0) && (64'F(S1.f16) == -0.0)) then
    D0.f16 = S0.f16
elsif ((64'F(S0.f16) == -0.0) && (64'F(S1.f16) == +0.0)) then
    D0.f16 = S1.f16
elsif WAVE_MODE.IEEE then
    D0.f16 = S0.f16 >= S1.f16 ? S0.f16 : S1.f16
else
    D0.f16 = S0.f16 > S1.f16 ? S0.f16 : S1.f16
endif
```

说明

IEEE 符合. 支持异常,舍入模式,例外旗帜,饱和度.

## V\_MIN\_F16

46

选择至少两个半精度浮点输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S0.f16)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S1.f16)))
elsif isNaN(64'F(S0.f16)) then
    D0.f16 = S1.f16
elsif isNaN(64'F(S1.f16)) then
    D0.f16 = S0.f16
elsif ((64'F(S0.f16) == +0.0) && (64'F(S1.f16) == -0.0)) then
    D0.f16 = S1.f16
elsif ((64'F(S0.f16) == -0.0) && (64'F(S1.f16) == +0.0)) then
    D0.f16 = S0.f16
else
    // 注意:这里没有 IEEE 案件,像 V_MAX_F16 一样.
```

```
D0.f16 = S0.f16 < S1.f16 ? S0.f16 : S1.f16
endif
```

说明

IEEE 符合. 支持异常,舍入模式,例外旗帜,饱和度.

## V\_MAX\_U16

47

选择两个无符号的 16 位整数输入的最大值,并将所选值存储到向量寄存器中。

```
D0.u16 = S0.u16 >= S1.u16 ? S0.u16 : S1.u16
```

## V\_MAX\_I16

48

选择最多两个签名的 16 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i16 = S0.i16 >= S1.i16 ? S0.i16 : S1.i16
```

## V\_MIN\_U16

49

选择两个无符号的 16 位整数输入的最小值,并将所选值存储到向量寄存器中。

```
D0.u16 = S0.u16 < S1.u16 ? S0.u16 : S1.u16
```

## V\_MIN\_I16

50

选择至少两个签名的 16 位整数输入,并将选定的数值存储到向量寄存器中。

```
D0.i16 = S0.i16 < S1.i16 ? S0.i16 : S1.i16
```

## V\_LDEXP\_F16

51

乘以第一个输入,一个浮点值,通过第二个输入中指定的 2 个整体功率,一个签名的整数值,并将浮点结果存储到向量寄存器中。

```
D0.f16 = S0.f16 * 16'F(2.0F ** 32'I(S1.i16))
```

说明

与 C 中的 `ldexp()` 函数相比。注意 S1 的格式为 f16, 因为浮点字元常数被解释为 16 比特值。

---

## V\_ADD\_U32

52

添加两个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有随身携带或随身携带支持。

```
D0.u32 = S0.u32 + S1.u32
```

说明

支持饱和(无符号的 32 位整数域)。

---

## V\_SUB\_U32

53

从第一个输入中减去第二个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有载入或载出支持。

```
D0.u32 = S0.u32 - S1.u32
```

说明

支持饱和(无符号的 32 位整数域)。

---

## V\_SUBREV\_U32

54

从第二个输入中减去第一个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
D0.u32 = S1.u32 - S0.u32
```

说明

支持饱和(无符号的 32 位整数域)。



**V\_DOT2C\_F32\_F16****55**

在单精度浮点域中，计算两个打包的二维半精度浮点输入的点积，再与目标寄存器中的单精度浮点值累加。

```
tmp = D0.f32;
tmp += f16_to_f32(S0[15 : 0].f16) * f16_to_f32(S1[15 : 0].f16);
tmp += f16_to_f32(S0[31 : 16].f16) * f16_to_f32(S1[31 : 16].f16);
D0.f32 = tmp
```

**V\_DOT2C\_I32\_I16****56**

计算两个打包的 2-D 的点产品在签名的 32 位整数域中签名的 16 位整数输入,并与签名的 32 位整数值在目的地寄存器中积累。

```
tmp = D0.i32;
tmp += i16_to_i32(S0[15 : 0].i16) * i16_to_i32(S1[15 : 0].i16);
tmp += i16_to_i32(S0[31 : 16].i16) * i16_to_i32(S1[31 : 16].i16);
D0.i32 = tmp
```

**V\_DOT4C\_I32\_I8****57**

计算两个包装的 4-D 在签名的 32 位整数域中签名的 8 位整数输入的点积,并在目的地寄存器中与签名的 32 位整数值一起累积。

```
tmp = D0.i32;
tmp += i8_to_i32(S0[7 : 0].i8) * i8_to_i32(S1[7 : 0].i8);
tmp += i8_to_i32(S0[15 : 8].i8) * i8_to_i32(S1[15 : 8].i8);
tmp += i8_to_i32(S0[23 : 16].i8) * i8_to_i32(S1[23 : 16].i8);
tmp += i8_to_i32(S0[31 : 24].i8) * i8_to_i32(S1[31 : 24].i8);
D0.i32 = tmp
```

**V\_DOT8C\_I32\_I4****58**

计算两个打包的 8-D 的点产品在签名的 32 位整数域中签名的 4 位整数输入,并与签名的 32 位整数值在目的地寄存器中积累。

```
tmp = D0.i32;
tmp += i4_to_i32(S0[3 : 0].i4) * i4_to_i32(S1[3 : 0].i4);
tmp += i4_to_i32(S0[7 : 4].i4) * i4_to_i32(S1[7 : 4].i4);
tmp += i4_to_i32(S0[11 : 8].i4) * i4_to_i32(S1[11 : 8].i4);
tmp += i4_to_i32(S0[15 : 12].i4) * i4_to_i32(S1[15 : 12].i4);
tmp += i4_to_i32(S0[19 : 16].i4) * i4_to_i32(S1[19 : 16].i4);
```

```
tmp += i4_to_i32(S0[23 : 20].i4) * i4_to_i32(S1[23 : 20].i4);
tmp += i4_to_i32(S0[27 : 24].i4) * i4_to_i32(S1[27 : 24].i4);
tmp += i4_to_i32(S0[31 : 28].i4) * i4_to_i32(S1[31 : 28].i4);
D0.i32 = tmp
```

**V\_FMAC\_F32****59**

乘以两个浮点输入,利用融合乘法将结果累积到目的地寄存器中。

```
D0.f32 = fma(S0.f32, S1.f32, D0.f32)
```

**V\_PK\_FMAC\_F16****60**

将两个包装的半精度浮点输入组件相乘,并使用融合乘法将结果累积到目的地寄存器中。

```
D0[15 : 0].f16 = fma(S0[15 : 0].f16, S1[15 : 0].f16, D0[15 : 0].f16);
D0[31 : 16].f16 = fma(S0[31 : 16].f16, S1[31 : 16].f16, D0[31 : 16].f16)
```

**V\_XNOR\_B32****61**

在两个向量输入上计算比特 XNOR,并将结果存储到向量寄存器中。

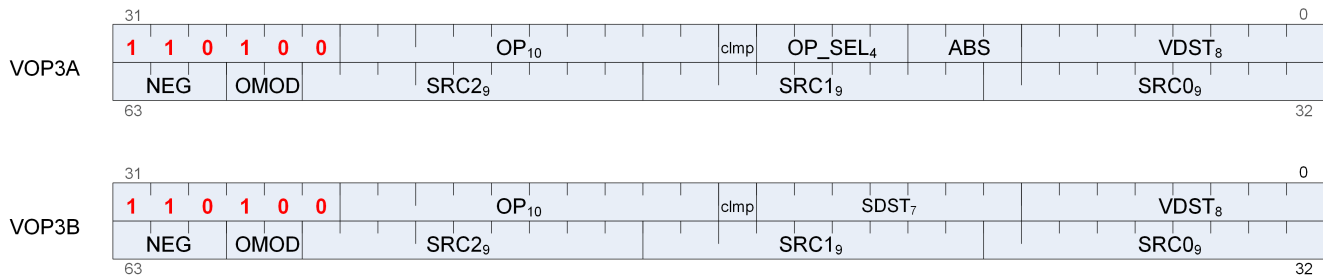
```
D0.u32 = ~(S0.u32 ^ S1.u32)
```

说明

不支持输入和输出修改符。

### 12.7.1. 采用 VOP3 编码的 VOP2 指令

此格式的指令也可能被编码为 VOP3. 这允许访问额外的控制位(例如 ABS,OMOD),以换取无法使用字元常数. VOP3 opcode 是: VOP2 opcode + 0x100.



## 12.8. VOP1 指令



该格式的指令可能使用 32 位的字元常数,DPP 或 SDWA,该常数在指令后立即出现.

V\_NOP

0

什么都别做。

说明

此指令可用于在向量 ALU 管道中插入单个循环气泡。 对于多个循环, 重复此操作码 。

V\_MOV\_B32

1

将向量输入的 32 位数据移动到向量寄存器中。

```
D0.b32 = S0.b32
```

说明

如果 S0 是 32 位浮点值,则浮点修改符对本指令有效。 本指令适用于否定或占用浮点值的绝对值。

功能实例:

```
v_mov_b32 v0, v1    // 从 v1 移动到 v0
v_mov_b32 v0, -v1    // 设定 v0 到否定 v1
```

```
v_mov_b32 v0, abs(v1)    // 将 v0 设定为 v1 的绝对值
```

## V\_READFIRSTLANE\_B32

2

读取输入向量寄存器最低活动道的标量值,将其存储到标量寄存器中。

```
declare lane : 32'I;  
if EXEC == 0x0LL then  
    lane = 0;  
    // 如果全部通道禁用, 则强制通道 0  
else  
    lane = s_ff1_i32_b64(EXEC);  
    // 最低有效通道  
endif;  
D0.b32 = VGPR[lane][SRC0.u32]
```

说明

VGPR 的遮罩被替换为 EXEC 。 不支持输入和输出修改器; 这是一个未键入的操作 。

## V\_CVT\_I32\_F64

3

从双精度浮点数输入转换为签名的 32 位整数值,并将结果存储到向量寄存器中。

```
D0.i32 = f64_to_i32(S0.f64)
```

说明

0.5ULP 精度,外程浮点值(包括无限)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

## V\_CVT\_F64\_I32

4

从一个签名的 32 位整数输入转换为双精度浮点值,并将结果存储到向量寄存器中。

```
D0.f64 = i32_to_f64(S0.i32)
```

说明

0ULP 精度.

---

### V\_CVT\_F32\_I32

5

从签名的 32 位整数输入转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = i32_to_f32(S0.i32)
```

说明

0.5ULP 精度.

---

### V\_CVT\_F32\_U32

6

从无符号的 32 位整数输入转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0.u32)
```

说明

0.5ULP 精度.

---

### V\_CVT\_U32\_F32

7

从单精度浮点输入转换为无符号的 32 位整数值,并将结果存储到向量寄存器中.

```
D0.u32 = f32_to_u32(S0.f32)
```

说明

1ULP 精度,外程浮点值(包括无穷)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

---

**V\_CVT\_I32\_F32****8**

从单精度浮点输入转换为签名的 32 位整数值,并将结果存储到向量寄存器中。

```
D0.i32 = f32_to_i32(S0.f32)
```

说明

1ULP 精度,外程浮点值(包括无穷)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

**V\_CVT\_F16\_F32****10**

从单精度浮点输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = f32_to_f16(S0.f32)
```

说明

0.5ULP 精度,支持输入修饰器,并在适当的时候创建 FP16 非正常性. Flush denorms 如果根据 DP denorm 模式指定,则在输出上进行 Flush denorms. 输出四舍五入基于 DP 四舍五入模式.

**V\_CVT\_F32\_F16****11**

从半精度浮点输入转换为单精度浮点值,并将结果存储为向量寄存器。

```
D0.f32 = f16_to_f32(S0.f16)
```

说明

0ULP 精度,FP16 非正常输入被接受. Flush denorm 如果根据 DP denorm 模式指定,则在输入上进行 flush denorm.

**V\_CVT\_RPI\_I32\_F32****12**

从单精度浮点输入转换为签名的 32 位整数值,使用圆形到最近的整数语义(忽略默认的四舍五入模式),并将结果存储到向量寄存器中。

```
D0.i32 = f32_to_i32(floor(S0.f32 + 0.5F))
```

说明

0.5ULP 精度,异常值得到支持.

## V\_CVT\_FLR\_I32\_F32

13

使用圆形的语义(忽略默认的四舍五入模式)从单精度浮点输入转换为签名的 32 位整数值,并将结果存储到向量寄存器中.

```
D0.i32 = f32_to_i32(floor(S0.f32))
```

说明

1ULP 精确度,异常值得到支持.

## V\_CVT\_OFF\_F32\_I4

14

使用一个偏移表,从一个签名的 4 位整数输入转换成一个单精度浮点值,并将结果存储到向量寄存器中.

在阴影中用于插值。S0 [3: 0] 上的查询表：

S0 二进制值与结果：

```
1000 -
0.5000f 1001 -
0.4375f 1010 -
0.3750f 1011 -
0.3125f 1100 -
0.2500f 1101 -
0.1875f 1110 -
0.1250f 1111 -
0.0625f 0000 +
0.0000f 0001 +
0.0625f 0010 +
0.1250f 0011 +
0.1875f 0100 +
0.2500f 0101 +
0.3125f 0110 +
0.3750f 0111 +
0.4375f
```

```
declare CVT_OFF_TABLE : 32'F[16];
D0.f32 = CVT_OFF_TABLE[S0.u32[3 : 0]]
```

**V\_CVT\_F32\_F64****15**

从双精度浮点输入转换为单精度浮点值,并将结果存储为向量寄存器。

```
D0.f32 = f64_to_f32(S0.f64)
```

说明

0.5ULP 精度,异常值得到支持。

**V\_CVT\_F64\_F32****16**

从单精度浮点输入转换为双精度浮点值,并将结果存储到向量寄存器中。

```
D0.f64 = f32_to_f64(S0.f32)
```

说明

0ULP 精确度,异常值得到支持。

**V\_CVT\_F32\_UBYTE0****17**

将输入的字节 0 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中。

```
D0.f32 = u32_to_f32(S0[7 : 0].u32)
```

**V\_CVT\_F32\_UBYTE1****18**

将输入的字节 1 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中。

```
D0.f32 = u32_to_f32(S0[15 : 8].u32)
```

**V\_CVT\_F32\_UBYTE2****19**



将输入的字节 2 中的无符号字节转换为单精度浮点值,并将结果存储为向量寄存器.

```
D0.f32 = u32_to_f32(S0[23 : 16].u32)
```

---

### V\_CVT\_F32\_UBYTE3

20

将输入的字节 3 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0[31 : 24].u32)
```

---

### V\_CVT\_U32\_F64

21

从双精度浮点数输入转换为无符号的 32 位整数值,并将结果存储到向量寄存器中.

```
D0.u32 = f64_to_u32(S0.f64)
```

说明

0.5ULP 精度,外程浮点值(包括无限)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

---

### V\_CVT\_F64\_U32

22

从无符号的 32 位整数输入转换成双精度浮点值,并将结果存储到向量寄存器中.

```
D0.f64 = u32_to_f64(S0.u32)
```

说明

0ULP 精度.

---

### V\_TRUNC\_F64

23

使用朝零舍入语义计算双精度浮点输入的整数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f64 = trunc(S0.f64)
```

## V\_CEIL\_F64

24

将双精度浮点输入绕到下一个整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f64 = trunc(S0.f64);  
if ((S0.f64 > 0.0) && (S0.f64 != D0.f64)) then  
    D0.f64 += 1.0  
endif
```

## V\_RNDNE\_F64

25

将双精度浮点输入舍入到最近整数, 恰好位于两个整数中间时取偶数; 以浮点格式将结果写入向量寄存器。

```
D0.f64 = floor(S0.f64 + 0.5);  
if (isEven(floor(S0.f64)) && (fract(S0.f64) == 0.5)) then  
    D0.f64 -= 1.0  
endif
```

## V\_FLOOR\_F64

26

将双精度浮点输入向下绕到之前的整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f64 = trunc(S0.f64);  
if ((S0.f64 < 0.0) && (S0.f64 != D0.f64)) then  
    D0.f64 += -1.0  
endif
```

## V\_FRACT\_F32

27

计算单精度浮点输入的小数部分, 并以浮点格式将结果写入向量寄存器。

```
D0.f32 = S0.f32 + -floor(S0.f32)
```

说明

精度为 0.5 ULP，允许非规格化输入。

本指令旨在符合 DX 对 `fract` 的定义，其行为相当于整数取模的扩展。其他领域对 `fract()` 的定义可能不同。例如，DX 中 `fract(-1.2) = 0.8`。

遵循舍入模式，结果限制在 0x3f7ffff 以内。

---

### V\_TRUNC\_F32

28

按朝零舍入的语义计算单精度浮点输入的整数部分，并以浮点格式将结果写入向量寄存器。

```
D0.f32 = trunc(S0.f32)
```

---

### V\_CEIL\_F32

29

将单精度浮点输入向上舍入到整数，并以浮点格式将结果写入向量寄存器。

```
D0.f32 = trunc(S0.f32);  
if ((S0.f32 > 0.0F) && (S0.f32 != D0.f32)) then  
    D0.f32 += 1.0F  
endif
```

---

### V\_RNDNE\_F32

30

将单精度浮点输入舍入到最近整数，恰好位于两个整数中间时取偶数；以浮点格式将结果写入向量寄存器。

```
D0.f32 = floor(S0.f32 + 0.5F);  
if (isEven(64 * F(floor(S0.f32))) && (fract(S0.f32) == 0.5F)) then  
    D0.f32 -= 1.0F  
endif
```

**V\_FLOOR\_F32****31**

将单精度浮点输入向下绕到以前的整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f32 = trunc(S0.f32);
if ((S0.f32 < 0.0F) && (S0.f32 != D0.f32)) then
    D0.f32 += -1.0F
endif
```

**V\_EXP\_F32****32**

计算 2 升至单精度浮点输入的功率,并将结果存储到向量寄存器中。

```
D0.f32 = pow(2.0F, S0.f32)
```

说明

1ULP 精度,异常值被冲洗。

功能实例:

```
V_EXP_F32(0xff800000) => 0x00000000    // exp(-INF) = 0
V_EXP_F32(0x80000000) => 0x3f800000    // exp(-0.0) = 1
V_EXP_F32(0x7f800000) => 0x7f800000    // exp(+INF) = +INF
```

**V\_LOG\_F32****33**

计算单精度浮点输入的基数 2 对数,并将结果存储到向量寄存器中。

```
D0.f32 = log2(S0.f32)
```

说明

1ULP 精度,异常值被冲洗。

功能实例:

```
V_LOG_F32(0xff800000) => 0xffc00000    // log(-INF) = NAN
V_LOG_F32(0xbf800000) => 0xffc00000    // log(-1.0) = NAN
V_LOG_F32(0x80000000) => 0xff800000    // log(-0.0) = -INF
V_LOG_F32(0x00000000) => 0xff800000    // log(+0.0) = -INF
V_LOG_F32(0x3f800000) => 0x00000000    // log(+1.0) = 0
```

```
V_LOG_F32(0x7f800000) => 0x7f800000    // log(+INF) = +INF
```

## V\_RCP\_F32

34

使用 IEEE 规则计算单精度浮点输入的对等值,并将结果存储到向量寄存器中。

```
D0.f32 = 1.0F / S0.f32
```

说明

1ULP 精度. 使用牛顿-拉弗森法和 2 个 FMA 操作时,精度与 < 0.5ULP 汇合。异常值被冲掉。

功能实例:

```
V_RCP_F32(0xff800000) => 0x80000000    // rcp(-INF) = -0
V_RCP_F32(0xc0000000) => 0xbf000000    // rcp(-2.0) = -0.5
V_RCP_F32(0x80000000) => 0xff800000    // rcp(-0.0) = -INF
V_RCP_F32(0x00000000) => 0x7f800000    // rcp(+0.0) = +INF
V_RCP_F32(0x7f800000) => 0x00000000    // rcp(+INF) = +0
```

## V\_RCP\_IFLAG\_F32

35

以适合整数分割的方式计算向量浮点输入的对等值,并将结果存储到向量寄存器中。此 opcode 意在作为整数分割宏的一部分使用。

```
D0.f32 = 1.0F / S0.f32;
// 只能提高整数 DIV_BY_ZERO 例外
```

说明

可以提出整数 DIV\_BY\_ZERO 例外,但不能提出浮点例外。由编译器在整数对等宏中使用,其序列如下(取决于签名或无符号操作)。

无符号数的用法:

```
CVT_F32_U32
RCP_IFLAG_F32
MUL_F32 (2**32 -
1) CVT_U32_F32
```

签字用法:

```
CVT_F32_I32
```

RCP\_IFLAG\_F32  
 MUL\_F32 ( $2^{31} - 1$ )  
 CVT\_I32\_F32

## V\_RSQ\_F32

36

使用 IEEE 规则计算单精度浮点输入的平方根对等,并将结果存储到向量寄存器中.

```
D0.f32 = 1.0F / sqrt(S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_RSQ_F32(0xff800000) => 0xffc00000    // rsq(-INF) = NAN
V_RSQ_F32(0x80000000) => 0xff800000    // rsq(-0.0) = -INF
V_RSQ_F32(0x00000000) => 0x7f800000    // rsq(+0.0) = +INF
V_RSQ_F32(0x40800000) => 0x3f000000    // rsq(+4.0) = +0.5
V_RSQ_F32(0x7f800000) => 0x00000000    // rsq(+INF) = +0
```

## V\_RCP\_F64

37

使用 IEEE 规则计算双精度浮点输入的对等值,并将结果存储到向量寄存器中.

```
D0.f64 = 1.0 / S0.f64
```

说明

这个 opcode 具有( $2^{29}$ )ULP 的精度,并支持异常.

## V\_RSQ\_F64

38

使用 IEEE 规则计算双精度浮点输入的平方根的对等值,并将结果存储到向量寄存器中.

```
D0.f64 = 1.0 / sqrt(S0.f64)
```

说明

这个 opcode 具有 $(2^{29})$ ULP 的精度,并支持异常.

### V\_SQRT\_F32

39

使用 IEEE 规则计算单精度浮点输入的平方根,并将结果存储到向量寄存器中.

```
D0.f32 = sqrt(S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_SQRT_F32(0xff800000) => 0xffc00000    // sqrt(-INF) = NAN
V_SQRT_F32(0x80000000) => 0x80000000    // sqrt(-0.0) = -0
V_SQRT_F32(0x00000000) => 0x00000000    // sqrt(+0.0) = +0
V_SQRT_F32(0x40800000) => 0x40000000    // sqrt(+4.0) = +2.0
V_SQRT_F32(0x7f800000) => 0x7f800000    // sqrt(+INF) = +INF
```

### V\_SQRT\_F64

40

使用 IEEE 规则计算双精度浮点输入的平方根,并将结果存储到向量寄存器中.

```
D0.f64 = sqrt(S0.f64)
```

说明

这个 opcode 具有 $(2^{29})$ ULP 的精度,并支持异常.

### V\_SIN\_F32

41

使用 IEEE 规则计算单精度浮点值的三角正弦, 并将结果存储到向量寄存器中。 操作通过将向量输入缩放 2 PI 来计算。

```
D0.f32 = sin(S0.f32 * 32'F(PI * 2.0))
```

## 说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_SIN_F32(0xff800000) => 0xffc00000    // sin(-INF) = NAN
V_SIN_F32(0xff7fffff) => 0x00000000    // -MaxFloat, finite
V_SIN_F32(0x80000000) => 0x80000000    // sin(-0.0) = -0
V_SIN_F32(0x3e800000) => 0x3f800000    // sin(0.25) = 1
V_SIN_F32(0x7f800000) => 0xffc00000    // sin(+INF) = NAN
```

**V\_COS\_F32****42**

使用 IEEE 规则计算单精度浮点值的三角余弦, 并将结果存储到向量寄存器中。 操作通过将向量输入缩放 2 PI 来计算。

```
D0.f32 = cos(S0.f32 * 32'F(PI * 2.0))
```

## 说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_COS_F32(0xff800000) => 0xffc00000    // cos(-INF) = NAN
V_COS_F32(0xff7fffff) => 0x3f800000    // -MaxFloat, finite
V_COS_F32(0x80000000) => 0x3f800000    // cos(-0.0) = 1
V_COS_F32(0x3e800000) => 0x00000000    // cos(0.25) = 0
V_COS_F32(0x7f800000) => 0xffc00000    // cos(+INF) = NAN
```

**V\_NOT\_B32****43**

计算向量输入时的比特按位取反, 并将结果存储到向量寄存器中。

```
D0.u32 = ~S0.u32
```

## 说明

不支持输入和输出修改符。

**V\_BFREV\_B32****44**



反转向量输入中的位数顺序,并将结果存储到向量寄存器中。

```
D0.u32[31 : 0] = S0.u32[0 : 31]
```

说明

不支持输入和输出修改符。

## V\_FFBH\_U32

45

统计向量输入的前导 0 的数量,即从最高位起到第一个 1 之前连续的 0,并将结果写入向量寄存器。若没有 1,则写入 -1。

```
D0.i32 = -1;
// 如果找不到,则设定
for i in 0 : 31 do
    // 从 MSB 搜索
    if S0.u32[31 - i] == 1'1U then
        D0.i32 = i;
        break
    endif
endfor
```

说明

功能实例:

```
V_FFBH_U32(0x00000000) => 0xffffffff
V_FFBH_U32(0x800000ff) => 0
V_FFBH_U32(0x100000ff) => 3
V_FFBH_U32(0x0000ffff) => 16
V_FFBH_U32(0x00000001) => 31
```

## V\_FFBL\_B32

46

统计向量输入的尾随 0 的数量,即从最低位起到第一个 1 之前连续的 0,并将结果写入向量寄存器。若输入中没有 1,则写入 -1。

```
D0.i32 = -1;
// 如果找不到,则设定
for i in 0 : 31 do
    // 从 LSB 搜索
    if S0.u32[i] == 1'1U then
        D0.i32 = i;
        break
    endif
endif
```

```
endfor
```

### 说明

#### 功能实例:

```
V_FFBL_B32(0x00000000) => 0xffffffff
V_FFBL_B32(0xff000001) => 0
V_FFBL_B32(0xff000008) => 3
V_FFBL_B32(0xffff0000) => 16
V_FFBL_B32(0x80000000) => 31
```

## V\_FFBH\_I32

47

统计向量输入中从最高位起、连续与符号位相同的位数，并将结果写入向量寄存器。若所有输入位都相同，则写入 -1。

```
D0.i32 = -1;
// 如果所有位码都是相同的, 设定
for i in 1 : 31 do
    // 从 MSB 搜索
    if S0.i32[31 - i] != S0.i32[31] then
        D0.i32 = i;
        break
    endif
endfor
```

### 说明

#### 功能实例:

```
V_FFBH_I32(0x00000000) => 0xffffffff
V_FFBH_I32(0x40000000) => 1
V_FFBH_I32(0x80000000) => 1
V_FFBH_I32(0x0fffffff) => 4
V_FFBH_I32(0xffffffff) => 16
V_FFBH_I32(0xffffffffe) => 31
V_FFBH_I32(0xfffffffff) => 0xffffffff
```

## V\_FREXP\_EXP\_I32\_F64

48

提取双精度浮点输入的表示器,并将结果作为签名的 32 位整数存储到向量寄存器中。

```
if ((S0.f64 == +INF) || (S0.f64 == -INF) || isNaN(S0.f64)) then
```

```
D0.i32 = 0
else
    D0.i32 = exponent(S0.f64) - 1023 + 1
endif
```

#### 说明

此操作满足了不变量的  $S0.f64 = \text{significand} * (2^{**} \text{exponent})$ 。另参见 `V_FREXP_MANT_F64`，它返回了 significand。更多信息请参见 C 库函数 `frexp()`。

---

### V\_FREXP\_MANT\_F64

49

提取双精度浮点数输入的二进制符号或 mantissa,并将结果作为双精度浮点数存储到向量寄存器中。

```
if ((S0.f64 == +INF) || (S0.f64 == -INF) || isNaN(S0.f64)) then
    D0.f64 = S0.f64
else
    D0.f64 = mantissa(S0.f64)
endif
```

#### 说明

此操作满足了变化无常的  $S0.f64 = \text{significand} * (2^{**} \text{exponent})$ 。结果范围在正常情况下为  $(-1.0, -0.5) [0.5, 1.0)$ 。另见 `V_FREXP_EXP_I32_F64`, 返回整数符号。请参见 C 库函数。更多信息请考虑()。

---

### V\_FRACT\_F64

50

计算双精度浮点输入的小数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f64 = S0.f64 + -floor(S0.f64)
```

#### 说明

精度为 0.5 ULP, 允许非规格化输入。

本指令旨在符合 DX 对 `fract` 的定义, 其行为相当于整数取模的扩展。其他领域对 `fract()` 的定义可能不同。例如, DX 中  $\text{fract}(-1.2) = 0.8$ 。

遵循舍入模式, 结果限制在 `0x3fefffffffffffff` 以内。

**V\_FREXP\_EXP\_I32\_F32****51**

提取单精度浮点输入的缩写,并将结果作为签名的 32 位整数存储到向量寄存器中。

```
if ((64'F(S0.f32) == +INF) || (64'F(S0.f32) == -INF) || isNaN(64'F(S0.f32))) then
    D0.i32 = 0
else
    D0.i32 = exponent(S0.f32) - 127 + 1
endif
```

说明

此操作满足了不变量的  $S0.f32 = \text{significand} * (2^{**} \text{exponent})$ 。另参见 V\_FREXP\_MANT\_F32, 它返回了 significand。更多信息请参见 C 库函数 frexp()。

**V\_FREXP\_MANT\_F32****52**

提取单精度浮点输入的 32 位二进制符号或 mantissa,并将结果作为单精度浮点数存储到向量寄存器中。

```
if ((64'F(S0.f32) == +INF) || (64'F(S0.f32) == -INF) || isNaN(64'F(S0.f32))) then
    D0.f32 = S0.f32
else
    D0.f32 = mantissa(S0.f32)
endif
```

说明

此操作满足了变化无常的  $S0.f32 = \text{significand} * (2^{**} \text{exponent})$ 。结果范围在正常情况下为 (-1.0,-0.5) [0.5,1.0)。另见 V\_FREXP\_EXP\_I32\_F32, 返回整数符。请参见 C 库函数。更多信息请考虑()。

**V\_CLREXCP****53**

在向量 ALU 中清除此波的例外状态。

**V\_MOV\_B64****56**

将数据从 64 位向量输入移到向量寄存器中。

```
D0.b64 = S0.b64
```

## 说明

如果  $S0.u64$  是一个 64 位浮点值,则浮点修改器对本指令有效。本指令适用于否定或取浮点值的绝对值。

**V\_CVT\_F16\_U16****57**

从无符号的 16 位整数输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = u16_to_f16(S0.u16)
```

## 说明

0.5ULP 精度,支持非正常,四舍五入,例外旗和饱和。

**V\_CVT\_F16\_I16****58**

从签名的 16 位整数输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = i16_to_f16(S0.i16)
```

## 说明

0.5ULP 精度,支持非正常,四舍五入,例外旗和饱和。

**V\_CVT\_U16\_F16****59**

从半精度浮点输入转换为无符号的 16 位整数值,并将结果存储到向量寄存器中。

```
D0.u16 = f16_to_u16(S0.f16)
```

## 说明

1ULP 精度,支持四舍五入,例外旗帜和饱和度。FP16 异常值被接受,转换时采用截断方式。

INEXACT 例外的生成由 CLAMP 位控制。INEXACT 例外被启用用于此转换 if  $CLAMP == 1$ 。

**V\_CVT\_I16\_F16**

60

从半精度的浮点输入转换为签名的 16 位整数,并将结果存储到向量寄存器中。

```
D0.i16 = f16_to_i16(S0.f16)
```

说明

1ULP 精度,支持四舍五入,例外旗帜和饱和度. FP16 异常值被接受,转换时采用截断方式。

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

**V\_RCP\_F16**

61

使用 IEEE 规则计算半精度浮点输入的对等值,并将结果存储到向量寄存器中。

```
D0.f16 = 16'1.0 / S0.f16
```

说明

1ULP 精度。

功能实例:

```
V_RCP_F16(0xfc00) => 0x8000    // rcp(-INF) = -0
V_RCP_F16(0xc000) => 0xb800    // rcp(-2.0) = -0.5
V_RCP_F16(0x8000) => 0xfc00    // rcp(-0.0) = -INF
V_RCP_F16(0x0000) => 0x7c00    // rcp(+0.0) = +INF
V_RCP_F16(0x7c00) => 0x0000    // rcp(+INF) = +0
```

**V\_SQRT\_F16**

62

使用 IEEE 规则计算半精度浮点输入的方根,并将结果存储到向量寄存器中。

```
D0.f16 = sqrt(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_SQRT_F16(0xfc00) => 0xfe00    // sqrt(-INF) = NAN
V_SQRT_F16(0x8000) => 0x8000    // sqrt(-0.0) = -0
V_SQRT_F16(0x0000) => 0x0000    // sqrt(+0.0) = +0
V_SQRT_F16(0x4400) => 0x4000    // sqrt(+4.0) = +2.0
V_SQRT_F16(0x7c00) => 0x7c00    // sqrt(+INF) = +INF
```

## V\_RSQ\_F16

63

使用 IEEE 规则计算半精度浮点输入的平方根对等值,并将结果存储到向量寄存器中.

```
D0.f16 = 16'1.0 / sqrt(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_RSQ_F16(0xfc00) => 0xfe00    // rsq(-INF) = NAN
V_RSQ_F16(0x8000) => 0xfc00    // rsq(-0.0) = -INF
V_RSQ_F16(0x0000) => 0x7c00    // rsq(+0.0) = +INF
V_RSQ_F16(0x4400) => 0x3800    // rsq(+4.0) = +0.5
V_RSQ_F16(0x7c00) => 0x0000    // rsq(+INF) = +0
```

## V\_LOG\_F16

64

计算半精度浮点输入的基数 2 对数,并将结果存储到向量寄存器中.

```
D0.f16 = log2(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_LOG_F16(0xfc00) => 0xfe00    // log(-INF) = NAN
V_LOG_F16(0xbc00) => 0xfe00    // log(-1.0) = NAN
```

```
V_LOG_F16(0x8000) => 0xfc00    // log(-0.0) = -INF
V_LOG_F16(0x0000) => 0xfc00    // log(+0.0) = -INF
V_LOG_F16(0x3c00) => 0x0000    // log(+1.0) = 0
V_LOG_F16(0x7c00) => 0x7c00    // log(+INF) = +INF
```

## V\_EXP\_F16

65

计算 2 升至半精度浮点输入的功率,并将结果存储到向量寄存器中。

```
D0.f16 = pow(16'2.0, S0.f16)
```

说明

1ULP 精确度,异常值得到支持。

功能实例:

```
V_EXP_F16(0xfc00) => 0x0000    // exp(-INF) = 0
V_EXP_F16(0x8000) => 0x3c00    // exp(-0.0) = 1
V_EXP_F16(0x7c00) => 0x7c00    // exp(+INF) = +INF
```

## V\_FREXP\_MANT\_F16

66

提取半精度浮点输入的二进制符号或 mantissa,并将半精度浮点数结果存储到向量寄存器中。

```
if ((64'F(S0.f16) == +INF) || (64'F(S0.f16) == -INF) || isNaN(64'F(S0.f16))) then
    D0.f16 = S0.f16
else
    D0.f16 = mantissa(S0.f16)
endif
```

说明

此操作满足了变化无常的  $S0.f16 = \text{significand} * (2 ** \text{exponent})$ 。结果范围在正常情况下为  $(-1.0, -0.5) [0.5, 1.0)$ 。另见 V\_FREXP\_EXP\_I16\_F16, 返回整数符号。请参见 C 库函数。

更多信息请考虑()。

## V\_FREXP\_EXP\_I16\_F16

67

解析半精度浮点输入的缩写,并将结果作为签名的 16 位整数存储到向量寄存器中。



```
if ((64'F(S0.f16) == +INF) || (64'F(S0.f16) == -INF) || isNaN(64'F(S0.f16))) then
    D0.i16 = 16'0
else
    D0.i16 = 16'I(exponent(S0.f16) - 15 + 1)
endif
```

#### 说明

此操作满足了不变量的  $S0.f16 = \text{significand} * (2^{**} \text{exponent})$ 。另参见 `V_FREXP_MANT_F16`，它返回了 significand。更多信息请参见 C 库函数 `frexp()`。

---

### V\_FLOOR\_F16

68

将半精度浮点输入向下绕到以前的整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16);
if ((S0.f16 < 16'0.0) && (S0.f16 != D0.f16)) then
    D0.f16 += -16'1.0
endif
```

---

### V\_CEIL\_F16

69

将半精度浮点输入绕到下一个整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16);
if ((S0.f16 > 16'0.0) && (S0.f16 != D0.f16)) then
    D0.f16 += 16'1.0
endif
```

---

### V\_TRUNC\_F16

70

用朝零舍入语义计算半精度浮点输入的整数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16)
```

---

### V\_RNDNE\_F16

71

将半精度浮点输入舍入到最近整数，恰好位于两个整数中间时取偶数；以浮点格式将结果写入向量寄存器。

```
D0.f16 = floor(S0.f16 + 16'0.5);
if (isEven(64'F(floor(S0.f16))) && (fract(S0.f16) == 16'0.5)) then
    D0.f16 -= 16'1.0
endif
```

## V\_FRACT\_F16

72

计算半精度浮点输入的小数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = S0.f16 + -floor(S0.f16)
```

说明

精度为 0.5 ULP，允许非规格化输入。

本指令旨在符合 DX 对 fract 的定义，其行为相当于整数取模的扩展。其他领域对 fract() 的定义可能不同。例如，DX 中  $\text{fract}(-1.2) = 0.8$ 。

## V\_SIN\_F16

73

使用 IEEE 规则计算半精度浮点值的三角正弦值, 并将结果存储到向量寄存器中。操作通过将向量输入缩放 2 PI 来计算。

```
D0.f16 = sin(S0.f16 * 16'F(PI * 2.0))
```

说明

异常值被支持。全程输入被支持。

功能实例:

```
V_SIN_F16(0xfc00) => 0xfe00 // sin(-INF) = NAN
V_SIN_F16(0xfbff) => 0x0000 // 最负限 FP16
V_SIN_F16(0x8000) => 0x8000 // sin(-0.0) = -0
V_SIN_F16(0x3400) => 0x3c00 // sin(0.25) = 1
V_SIN_F16(0x7bff) => 0x0000 // 最积极的有限 FP16
V_SIN_F16(0x7c00) => 0xfe00 // sin(+INF) = NAN
```

## V\_COS\_F16

74

使用 IEEE 规则计算半精度浮点值的三角余弦, 并将结果存储到向量寄存器中。操作通过将向量输入缩放 2 PI 来计算。

```
D0.f16 = cos(S0.f16 * 16'F(PI * 2.0))
```

说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_COS_F16(0xfc00) => 0xfe00    // cos(-INF) = NAN  
V_COS_F16(0xfbff) => 0x3c00    // 最负限 FP16  
V_COS_F16(0x8000) => 0x3c00    // cos(-0.0) = 1  
V_COS_F16(0x3400) => 0x0000    // cos(0.25) = 0  
V_COS_F16(0x7bff) => 0x3c00    // 最积极的有限 FP16  
V_COS_F16(0x7c00) => 0xfe00    // cos(+INF) = NAN
```

## V\_CVT\_NORM\_I16\_F16

77

从半精度浮点输入转换成签名的正态短句,并将结果存储到向量寄存器中。

```
D0.i16 = f16_to_snorm(S0.f16)
```

说明

0.5ULP 精度,支持四舍五入,例外旗帜和饱和度,异常态得到支持。

## V\_CVT\_NORM\_U16\_F16

78

从半精度浮点输入转换为无符号的普通短数,并将结果存储到向量寄存器中。

```
D0.u16 = f16_to_unorm(S0.f16)
```

说明

0.5ULP 精度,支持四舍五入,例外旗帜和饱和度,异常态得到支持。

**V\_SAT\_PK\_U8\_I16**

79

在 2 个签名的 16 位整数输入中,将每个输入在一个无符号的 8 位整数范围内饱和,将所产生的值打包成一个包的 16 位值,并将结果存储到向量寄存器中。

```
SAT8 = lambda(n) (  
    if n <= 16'0 then  
        return 8'0U  
    elsif n >= 16'255 then  
        return 8'255U  
    else  
        return n[7 : 0].u8  
    endif);  
tmp = 16'0;  
tmp[7 : 0].u8 = SAT8(S0[15 : 0].i16);  
tmp[15 : 8].u8 = SAT8(S0[31 : 16].i16);  
D0.b16 = tmp.b16
```

说明

用于 4x16bit 数据包装为 4x8bit 数据。

**V\_SWAP\_B32**

81

在两个向量登记簿中交换值。

```
tmp = D0.b32;  
D0.b32 = S0.b32;  
S0.b32 = tmp
```

说明

不支持输入和输出修改器;这是一个未打字的操作。

**V\_ACCVGPR\_MOV\_B32**

82

将数据从一个累积器寄存器移动到另一个累积器寄存器。

**V\_CVT\_F32\_FP8**

84

从 FP8 浮动输入转换为单精度浮点值,并将结果存储到向量寄存器中。

```
if SDWA_SRC0_SEL == BYTE1.b3 then  
    D0.f32 = fp8_to_f32(S0[15 : 8].fp8)  
elsif SDWA_SRC0_SEL == BYTE2.b3 then
```

```

    D0.f32 = fp8_to_f32(S0[23 : 16].fp8)
elseif SDWA_SRC0_SEL == BYTE3.b3 then
    D0.f32 = fp8_to_f32(S0[31 : 24].fp8)
else
    // BYTE0 隐含
    D0.f32 = fp8_to_f32(S0[7 : 0].fp8)
endif

```

#### 说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个字节被转换,只有 BYTE 选择的 SRC0\_SEL 是合法的,如果此指令没有在 SDWA 编码,那么 BYTE0 就是隐含的。

### V\_CVT\_F32\_BF8

85

从 BF8 浮点数转换为单精度浮点数,并将结果存储到向量寄存器中。

```

if SDWA_SRC0_SEL == BYTE1.b3 then
    D0.f32 = bf8_to_f32(S0[15 : 8].bf8)
elseif SDWA_SRC0_SEL == BYTE2.b3 then
    D0.f32 = bf8_to_f32(S0[23 : 16].bf8)
elseif SDWA_SRC0_SEL == BYTE3.b3 then
    D0.f32 = bf8_to_f32(S0[31 : 24].bf8)
else
    // BYTE0 隐含
    D0.f32 = bf8_to_f32(S0[7 : 0].bf8)
endif

```

#### 说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个字节被转换,只有 BYTE 选择的 SRC0\_SEL 是合法的,如果此指令没有在 SDWA 编码,那么 BYTE0 就是隐含的。

### V\_CVT\_PK\_F32\_FP8

86

从包装的 2 组件 FP8 浮点数转换成包装的单精度浮点数,并将结果存储到向量寄存器中。

```

tmp = SDWA_SRC0_SEL[1 : 0] == WORD1.b2 ? S0[31 : 16] : S0[15 : 0];
D0[31 : 0].f32 = fp8_to_f32(tmp[7 : 0].fp8);
D0[63 : 32].f32 = fp8_to_f32(tmp[15 : 8].fp8)

```

#### 说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个单词被转换,只有 WORD 选择的 SRC0\_SEL 是合法的,如果这个指令没有在 SDWA 编码,那么 WORD0 就是隐含的。

**V\_CVT\_PK\_F32\_BF8****87**

从包装的 2 组件 BF8 浮点输入转换为包装的单精度浮点值,并将结果存储到向量寄存器中.

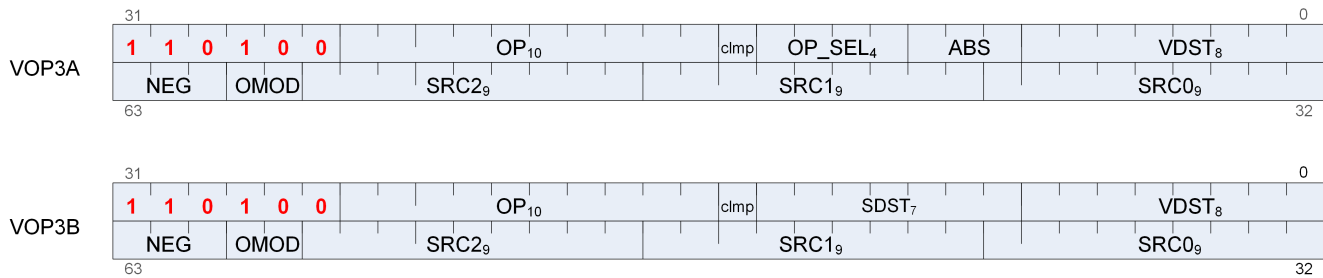
```
tmp = SDWA_SRC0_SEL[1 : 0] == WORD1.b2 ? S0[31 : 16] : S0[15 : 0];  
D0[31 : 0].f32 = bf8_to_f32(tmp[7 : 0].bf8);  
D0[63 : 32].f32 = bf8_to_f32(tmp[15 : 8].bf8)
```

**说明**

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个单词被转换,只有 WORD 选择的 SRC0\_SEL 是合法的,如果这个指令没有在 SDWA 编码,那么 WORD0 就是隐含的.

### 12.8.1. 采用 VOP3 编码的 VOP1 指令

此格式的指令也可能被编码为 VOP3. 这允许访问额外的控制位(例如 ABS,OMOD),以换取无法使用字元常数. VOP3 opcode 是: VOP2 opcode + 0x140.



| 比较操作 | Opcode 偏移 | 说明                               |
|------|-----------|----------------------------------|
| GE   | 6         | D.u = (S0 >= S1)                 |
| O    | 7         | D.u = (!isNaN(S0) && !isNaN(S1)) |
| U    | 8         | D.u = (!isNaN(S0)    !isNaN(S1)) |
| NGE  | 9         | D.u = !(S0 >= S1)                |
| NLG  | 10        | D.u = !(S0 <> S1)                |
| NGT  | 11        | D.u = !(S0 > S1)                 |
| NLE  | 12        | D.u = !(S0 <= S1)                |
| NEQ  | 13        | D.u = !(S0 == S1)                |
| NLT  | 14        | D.u = !(S0 < S1)                 |
| TRU  | 15        | D.u = 1                          |

表 59. 16 项比较业务的指令

| 指令                  | 说明                  | 十六进制范围      |
|---------------------|---------------------|-------------|
| V_CMP_{COMPF}_F16   | 16 位浮点比较.           | 0x20 至 0x2F |
| V_CMPX_{COMPF}_F16  | 16 位浮点比较。同时写入 EXEC。 | 0x30 至 0x3F |
| V_CMP_{COMPF}_F32   | 32 位浮点比较.           | 0x40 至 0x4F |
| V_CMPX_{COMPF}_F32  | 32 位浮点数比较,还写着 EXEC. | 0x50 至 0x5F |
| V_CMPS_{COMPF}_F64  | 64 位浮点比较.           | 0x60 至 0x6F |
| V_CMPSX_{COMPF}_F64 | 64 位浮点数比较,还写着 EXEC. | 0x70 至 0x7F |

表 60. 整数比较操作

| 比较操作 | Opcode 偏移 | 说明               |
|------|-----------|------------------|
| F    | 0         | D.u = 0          |
| LT   | 1         | D.u = (S0 < S1)  |
| EQ   | 2         | D.u = (S0 == S1) |
| LE   | 3         | D.u = (S0 <= S1) |
| GT   | 4         | D.u = (S0 > S1)  |
| LG   | 5         | D.u = (S0 <> S1) |
| GE   | 6         | D.u = (S0 >= S1) |
| TRU  | 7         | D.u = 1          |

表 61. 有 8 项比较业务的指令

| 指令                 | 说明                     | 十六进制范围      |
|--------------------|------------------------|-------------|
| V_CMP_{COMPI}_I16  | 16 位签名整数比较.            | 0xA0 - 0xA7 |
| V_CMP_{COMPI}_U16  | 16 位签名整数比较.            | 0xA8 - 0xAF |
| V_CMPX_{COMPI}_I16 | 16 位无符号的整数比较。同时写 EXEC。 | 0xB0 - 0xB7 |
| V_CMPX_{COMPI}_U16 | 16 位无符号的整数比较。同时写 EXEC。 | 0xB8 - 0xBF |
| V_CMP_{COMPI}_I32  | 32 位签名整数比较.            | 0xC0 - 0xC7 |
| V_CMP_{COMPI}_U32  | 32 位签名整数比较.            | 0xC8 - 0xCF |
| V_CMPX_{COMPI}_I32 | 32 位无符号整数比较。同时写入 EXEC。 | 0xD0 - 0xD7 |
| V_CMPX_{COMPI}_U32 | 32 位无符号整数比较。同时写入 EXEC。 | 0xD8 - 0xDF |
| V_CMP_{COMPI}_I64  | 64 位签名整数比较.            | 0xE0 - 0xE7 |
| V_CMP_{COMPI}_U64  | 64 位签名整数比较.            | 0xE8 - 0xEF |
| V_CMPX_{COMPI}_I64 | 64 位无符号的整数比较。同时写 EXEC。 | 0xF0 - 0xF7 |
| V_CMPX_{COMPI}_U64 | 64 位无符号的整数比较。同时写 EXEC。 | 0xF8 - 0xFF |

表 62. VOPC 比较 Opcode



**V\_CMP\_CLASS\_F32****16**

评估第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数, 一个单精度浮点, 并将每行条件码设置到结果中。 将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```
declare result : 1'U;
if isSignalNAN(64'F(S0.f32)) then
    result = S1.u32[0]
elsif isQuietNAN(64'F(S0.f32)) then
    result = S1.u32[1]
elsif exponent(S0.f32) == 255 then
    // +-INF
    result = S1.u32[sign(S0.f32) ? 2 : 9]
elsif exponent(S0.f32) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f32) ? 3 : 8]
elsif 64'F(abs(S0.f32)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f32) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f32) ? 5 : 6]
endif;
D0.u64[laneId] = result;
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_CLASS\_F32****17**

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数, 一个单精度浮点, 并将每行条件码设置到结果中。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```
declare result : 1'U;
if isSignalNAN(64'F(S0.f32)) then
    result = S1.u32[0]
elseif isQuietNAN(64'F(S0.f32)) then
    result = S1.u32[1]
elseif exponent(S0.f32) == 255 then
    // +-INF
    result = S1.u32[sign(S0.f32) ? 2 : 9]
elseif exponent(S0.f32) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f32) ? 3 : 8]
elseif 64'F(abs(S0.f32)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f32) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f32) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result
```

## V\_CMP\_CLASS\_F64

18

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数,双精度浮点,并将每行条件码设置在结果中。将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```

declare result : 1'U;
if isSignalNaN(S0.f64) then
    result = S1.u32[0]
elsif isQuietNaN(S0.f64) then
    result = S1.u32[1]
elsif exponent(S0.f64) == 2047 then
    // +-INF
    result = S1.u32[sign(S0.f64) ? 2 : 9]
elsif exponent(S0.f64) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f64) ? 3 : 8]
elsif abs(S0.f64) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f64) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f64) ? 5 : 6]
endif;
D0.u64[laneId] = result;
// D0 = VCC 在 VOPC 编码中。

```

## V\_CMPX\_CLASS\_F64

19

评价在第一个输入的第二个输入中指定为 10 bit 掩码的 IEEE 数字类函数, 双精度浮点数, 并设定每行条件码为结果。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型, 则函数报告为真:

S1.u[0]值是一个信号 NaN. S1.u[1]值是一个安静的 NaN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```

declare result : 1'U;
if isSignalNaN(S0.f64) then
    result = S1.u32[0]
elsif isQuietNaN(S0.f64) then
    result = S1.u32[1]
elsif exponent(S0.f64) == 2047 then
    // +-INF
    result = S1.u32[sign(S0.f64) ? 2 : 9]
elsif exponent(S0.f64) > 0 then
    // +-normal value

```

```

    result = S1.u32[sign(S0.f64) ? 3 : 8]
elseif abs(S0.f64) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f64) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f64) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result

```

## V\_CMP\_CLASS\_F16

20

评价第一个输入的第二个输入中指定的 IEEE 数字类函数为 10 bit 掩码、半精度浮点数, 并将每行条件码设置到结果中。将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型, 则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```

declare result : 1'U;
if isSignalNAN(64'F(S0.f16)) then
    result = S1.u32[0]
elseif isQuietNAN(64'F(S0.f16)) then
    result = S1.u32[1]
elseif exponent(S0.f16) == 31 then
    // +-INF
    result = S1.u32[sign(S0.f16) ? 2 : 9]
elseif exponent(S0.f16) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f16) ? 3 : 8]
elseif 64'F(abs(S0.f16)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f16) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f16) ? 5 : 6]
endif;
D0.u64[laneId] = result;
// D0 = VCC 在 VOPC 编码中.

```

## 说明

注意 S1 的格式为 f16,因为浮点字面常数被解释为这个 opcode 的 16 位值.

**V\_CMPX\_CLASS\_F16****21**

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数值类函数, 半精度浮点, 并设置每行条件码到结果中。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```
declare result : 1'U;
if isSignalNAN(64'F(S0.f16)) then
    result = S1.u32[0]
elseif isQuietNAN(64'F(S0.f16)) then
    result = S1.u32[1]
elseif exponent(S0.f16) == 31 then
    // +-INF
    result = S1.u32[sign(S0.f16) ? 2 : 9]
elseif exponent(S0.f16) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f16) ? 3 : 8]
elseif 64'F(abs(S0.f16)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f16) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f16) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result
```

## 说明

注意 S1 的格式为 f16,因为浮点字面常数被解释为这个 opcode 的 16 位值.

**V\_CMP\_F\_F16****32**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LT\_F16****33**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 < S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_EQ\_F16****34**

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 == S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LE\_F16****35**

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 <= S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_GT\_F16****36**

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 > S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LG\_F16****37**

将每段条件码设定为 1 iff ,第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 <> S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_GE\_F16****38**

将每段条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 >= S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_O\_F16****39**

将每条线路的条件码设定为 1 iff, 第一个输入可命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(64'F(S0.f16)) && !isNAN(64'F(S1.f16)));  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_U\_F16****40**

将每行区间条件码设定为 1 iff , 第一个输入不能命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(64'F(S0.f16)) || isNAN(64'F(S1.f16)));  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NGE\_F16****41**

将每行条件码设定为 1 iff ,第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 >= S1.f16);  
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMP\_NLG\_F16

42

将每段条件码设定为 1 iff ,第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 <> S1.f16);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMP\_NGT\_F16

43

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 > S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMP\_NLE\_F16

44

将每行条件码设定为 1 iff ,第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 <= S1.f16);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMP\_NEQ\_F16

45

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 == S1.f16);  
// NAN 输入与操作不同!=
```



```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLT\_F16

46

将每段条件码设定为 1 iff 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 < S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_TRU\_F16

47

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_F\_F16

48

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_LT\_F16

49

将每道条件码设定为 1 iff 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 < S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_EQ\_F16

50

将每条线路的条件码设定为 1 iff 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 == S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LE\_F16

51

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 <= S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_GT\_F16

52

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 > S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LG\_F16

53

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 <> S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_GE\_F16

54

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 >= S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_O\_F16

55

将每道条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNAN(64'F(S0.f16)) && !isNAN(64'F(S1.f16)));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_U\_F16

56

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNAN(64'F(S0.f16)) || isNAN(64'F(S1.f16)));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGE\_F16

57

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 >= S1.f16);
// 用 NAN 输入此操作与 <
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLG\_F16

58

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 <> S1.f16);
// 有了 NAN 的输入,这不是相同的操作与
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGT\_F16

59

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 > S1.f16);
// 用 NAN 输入此操作与 QQ 不同
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLE\_F16

60

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 <= S1.f16);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NEQ\_F16

61

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 == S1.f16);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLT\_F16

62

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 < S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_TRU\_F16

63

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_F\_F32

64

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'00;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_F32

65

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 < S1.f32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_F32

66

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 == S1.f32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_F32

67

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 <= S1.f32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_GT\_F32

68

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 > S1.f32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LG\_F32****69**

将每段条件码设定为 1 iff ,第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 <> S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_F32****70**

将每段条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 >= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_O\_F32****71**

将每条线路的条件码设定为 1 iff , 第一个输入可命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(64'F(S0.f32)) && !isNAN(64'F(S1.f32)));  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_U\_F32****72**

将每行区间条件码设定为 1 iff , 第一个输入不能命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(64'F(S0.f32)) || isNAN(64'F(S1.f32)));  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NGE\_F32****73**

将每行条件码设定为 1 iff ,第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 >= S1.f32);
```

```
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLG\_F32

74

将每段条件码设定为 1 iff, 第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 <> S1.f32);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NGT\_F32

75

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 > S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLE\_F32

76

将每行条件码设定为 1 iff, 第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 <= S1.f32);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NEQ\_F32

77

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 == S1.f32);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLT\_F32****78**

将每段条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 < S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_TRU\_F32****79**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_F\_F32****80**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_F32****81**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 < S1.f32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_F32****82**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 == S1.f32;
```



```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LE\_F32

83

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 <= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GT\_F32

84

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 > S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LG\_F32

85

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 <> S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GE\_F32

86

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 >= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_O\_F32

87

将每条线路的条件码设定为 1 iff, 第一个输入可以命令第二个输入。将结果存储到

EXEC 口罩,并到 VCC 或一个缩写寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNAN(64'F(S0.f32)) && !isNAN(64'F(S1.f32)));
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_U\_F32

88

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNAN(64'F(S0.f32)) || isNAN(64'F(S1.f32)));
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_NGE\_F32

89

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 >= S1.f32);
// 用 NAN 输入此操作与 <
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_NLG\_F32

90

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 <> S1.f32);
// 有了 NAN 的输入,这不是相同的操作与
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_NGT\_F32

91

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 > S1.f32);
// 用 NAN 输入此操作与 QQ 不同
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLE\_F32

92

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 <= S1.f32);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NEQ\_F32

93

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 == S1.f32);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLT\_F32

94

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 < S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_TRU\_F32

95

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_F\_F64

96

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'00;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_F64

97

将每行条件码设定为 1 iff 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 < S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_F64

98

将每段条件码设定为 1 iff 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 == S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_F64

99

将每条线路的条件码设定为 1 iff 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 <= S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_GT\_F64

100

将每行条件码设定为 1 iff 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 > S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LG\_F64****101**

将每段条件码设定为 1 iff ,第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 <> S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_F64****102**

将每段条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 >= S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_O\_F64****103**

将每条线路的条件码设定为 1 iff , 第一个输入可命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(S0.f64) && !isNAN(S1.f64));  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_U\_F64****104**

将每行区间条件码设定为 1 iff , 第一个输入不能命令第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(S0.f64) || isNAN(S1.f64));  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NGE\_F64****105**

将每行条件码设定为 1 iff ,第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 >= S1.f64);
```

```
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLG\_F64

106

将每段条件码设定为 1 iff, 第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 <> S1.f64);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NGT\_F64

107

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 > S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLE\_F64

108

将每行条件码设定为 1 iff, 第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 <= S1.f64);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NEQ\_F64

109

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 == S1.f64);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLT\_F64****110**

将每段条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 < S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_TRU\_F64****111**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_F\_F64****112**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_F64****113**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 < S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_F64****114**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 == S1.f64;
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LE\_F64

115

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 <= S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GT\_F64

116

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 > S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LG\_F64

117

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 <> S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GE\_F64

118

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 >= S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_O\_F64

119

将每条线路的条件码设定为 1 iff, 第一个输入可以命令第二个输入。将结果存储到



EXEC 口罩,并到 VCC 或一个缩写寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNAN(S0.f64) && !isNAN(S1.f64));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_U\_F64

120

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNAN(S0.f64) || isNAN(S1.f64));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGE\_F64

121

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 >= S1.f64);
// 用 NAN 输入此操作与 <
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLG\_F64

122

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 <> S1.f64);
// 有了 NAN 的输入,这不是相同的操作与
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGT\_F64

123

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 > S1.f64);
// 用 NAN 输入此操作与 QQ 不同
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLE\_F64

124

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 <= S1.f64);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NEQ\_F64

125

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 == S1.f64);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLT\_F64

126

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 < S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_TRU\_F64

127

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_F\_I16

160

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'00U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_I16

161

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 < S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_I16

162

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 == S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_I16

163

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 <= S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_GT\_I16

164

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 > S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NE\_I16****165**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 <> S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_I16****166**

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 >= S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_T\_I16****167**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_U16****168**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_U16****169**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 < S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_EQ\_U16****170**

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 == S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LE\_U16****171**

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 <= S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GT\_U16****172**

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 > S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NE\_U16****173**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 <> S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_U16****174**

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 >= S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMP\_T\_U16

175

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_F\_I16

176

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LT\_I16

177

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 < S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_EQ\_I16

178

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 == S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LE\_I16

179

将每行条件码设定为 1 iff, 第一次输入小于或等于第二次输入。存储结果

输入 EXEC 口罩,并输入 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 <= S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GT\_I16

**180**

将每段条件码设置为 1 iff , 第一个输入大于第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 > S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NE\_I16

**181**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 <> S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GE\_I16

**182**

将每条线路的条件码设定为 1 iff ,第一个输入大于或等于第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 >= S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_T\_I16

**183**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_U16****184**

将每道条件码设置为 0。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_U16****185**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 < S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_U16****186**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 == S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LE\_U16****187**

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 <= S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GT\_U16****188**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 > S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```



**V\_CMPX\_NE\_U16****189**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 <> S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GE\_U16****190**

将每条线路的条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 >= S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_T\_U16****191**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_I32****192**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_I32****193**

将每行条件码设定为 1 iff ,第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 < S1.i32;
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_EQ\_I32

194

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 == S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_LE\_I32

195

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 <= S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_GT\_I32

196

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 > S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NE\_I32

197

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 <> S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_GE\_I32

198

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.i32 >= S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_T\_I32

199

将每道条件码设置为 1。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_F\_U32

200

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'00U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_U32

201

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 < S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_U32

202

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 == S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_U32

203

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 <= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GT\_U32

204

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 > S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_NE\_U32

205

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 <> S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GE\_U32

206

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 >= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_T\_U32

207

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_I32****208**

将每道条件码设置为 0。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_I32****209**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 < S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_I32****210**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 == S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LE\_I32****211**

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 <= S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GT\_I32****212**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 > S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_NE\_I32****213**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 <> S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GE\_I32****214**

将每条线路的条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 >= S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_T\_I32****215**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_U32****216**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_LT\_U32****217**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 < S1.u32;
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_EQ\_U32

218

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 == S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_LE\_U32

219

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 <= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GT\_U32

220

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 > S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NE\_U32

221

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 <> S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GE\_U32

222

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

进入 EXEC 口罩和 VCC 或缩写表。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 >= S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_T\_U32

223

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_F\_I64

224

将每条线路的条件码设置为 0。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_I64

225

将每行条件码设定为 1 iff ,第一个输入小于第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 < S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_I64

226

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 == S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_I64

227



将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 <= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GT\_I64

228

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 > S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_NE\_I64

229

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 <> S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GE\_I64

230

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 >= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_T\_I64

231

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_U64****232**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LT\_U64****233**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 < S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_EQ\_U64****234**

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 == S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LE\_U64****235**

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 <= S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_GT\_U64****236**

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 > S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NE\_U64****237**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 <> S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_U64****238**

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 >= S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_T\_U64****239**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_I64****240**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_LT\_I64****241**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 < S1.i64;
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_EQ\_I64

242

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 == S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LE\_I64

243

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 <= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GT\_I64

244

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 > S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NE\_I64

245

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 <> S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GE\_I64

246

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

进入 EXEC 口罩和 VCC 或缩写表。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 >= S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_T\_I64

247

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_F\_U64

248

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LT\_U64

249

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 < S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_EQ\_U64

250

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中,并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 == S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_LE\_U64

251

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 <= S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_GT\_U64

252

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 > S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_NE\_U64

253

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 <> S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_GE\_U64

254

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 >= S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_T\_U64

255

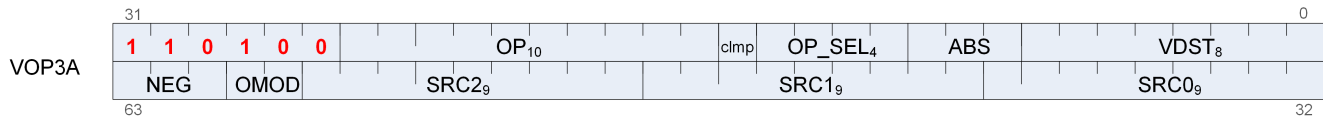
将每通道条件码设置为 1. 将结果存储到 EXEC 面具中, 并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

12.9.1. 采用 VOP3A 编码的 VOPC 指令

此格式的指令也可能被编码为 VOP3A. 这允许访问额外的控制位(例如 ABS,OMOD),以换取无法使用字元常数. VOP3 opcode 是: VOP2 opcode + 0x000.

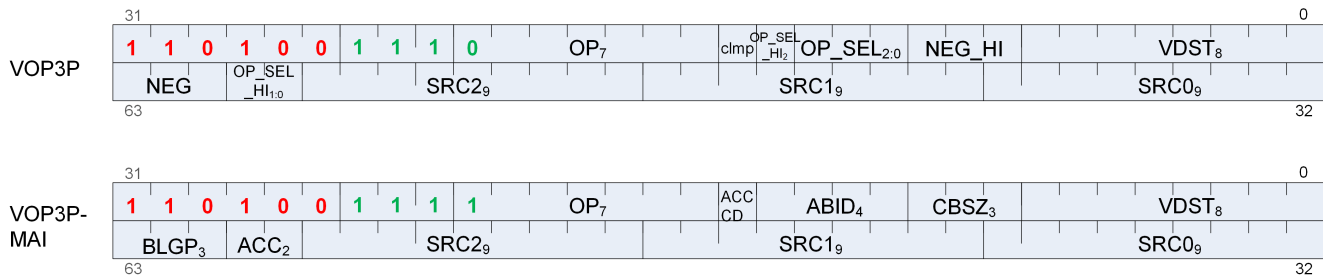
当 CLAMP 微码位被设置为 1 时, 这些比较指令表示一个例外, 当其中任一输入是 NaN 。 当 CLAMP 被设定为 0 时, NaN 不表示一个例外 。 后面的 8 个 VOPC 指令已经嵌入其中 {OP8} 。 这指的是下面列出的每个比较操作 。



where:

VDST = Destination for instruction in the VGPR.  
ABS = Floating-point absolute value.  
CLMP = Clamp output.  
OP = Instructions.  
SRC0 = First operand for instruction.  
SRC1 = Second operand for instruction.  
SRC2 = Third operand for instruction. Unused in VOPC instructions.  
OMOD = Output modifier for instruction. Unused in VOPC instructions.  
NEG = Floating-point negation.

12.10. VOP3P 指令



V\_PK\_MAD\_I16 0

将 2 个被包包的 16 位整数输入元件依次签名,从第三个输入元件依次添加一个被包包的 16 位整数值,并将结果存储到向量寄存器中.

```
declare tmp : 32'B;  
tmp[15 : 0].i16 = S0[15 : 0].i16 * S1[15 : 0].i16 + S2[15 : 0].i16;  
tmp[31 : 16].i16 = S0[31 : 16].i16 * S1[31 : 16].i16 + S2[31 : 16].i16;  
D0.b32 = tmp
```

**V\_PK\_MUL\_LO\_U16****1**

将两个无符号的 16 位整数输入元件乘以 2,并将每个生成的元件的低位存储到向量寄存器中。

```
tmp[31 : 16].u16 = S0[31 : 16].u16 * S1[31 : 16].u16;  
tmp[15 : 0].u16 = S0[15 : 0].u16 * S1[15 : 0].u16;  
D0.b32 = tmp.b32
```

**V\_PK\_ADD\_I16****2**

按 16 位整数输入组件添加 2 个已装入的 16 位整数输入, 并将结果存储到向量寄存器中。没有承载或承载支持。

```
declare tmp : 32'B;  
tmp[15 : 0].i16 = S0[15 : 0].i16 + S1[15 : 0].i16;  
tmp[31 : 16].i16 = S0[31 : 16].i16 + S1[31 : 16].i16;  
D0.b32 = tmp
```

**V\_PK\_SUB\_I16****3**

按分量从第一个输入减去第二个打包的有符号 16 位整数输入, 结果写入向量寄存器。不支持进位输入或进位输出。

```
declare tmp : 32'B;  
tmp[15 : 0].i16 = S0[15 : 0].i16 - S1[15 : 0].i16;  
tmp[31 : 16].i16 = S0[31 : 16].i16 - S1[31 : 16].i16;  
D0.b32 = tmp
```

**V\_PK\_LSHLREV\_B16****4**

鉴于第一个向量输入中一个打包的移位计数,计算第二个打包的向量输入中左侧的组件逻辑移位,并将结果存储到一个向量寄存器中。

```
tmp[31 : 16].u16 = (S1[31 : 16].u16 << S0.u32[19 : 16].u32);  
tmp[15 : 0].u16 = (S1[15 : 0].u16 << S0.u32[3 : 0].u32);  
D0.b32 = tmp.b32
```

**V\_PK\_LSHRREV\_B16****5**



鉴于第一个向量输入时的组合式移位计数,计算第二个组合式向量输入时的构件逻辑移位右侧,并将结果存储到向量寄存器中。

```
tmp[31 : 16].u16 = (S1[31 : 16].u16 >> S0.u32[19 : 16].u32);
tmp[15 : 0].u16 = (S1[15 : 0].u16 >> S0.u32[3 : 0].u32);
D0.b32 = tmp.b32
```

## V\_PK\_ASHRREV\_I16

6

鉴于第一个向量输入时的组合移位计数,计算第二个被组合的向量输入时的构件偏右算术移位(保留符号位),并将结果存储到向量寄存器中。

```
tmp[31 : 16].i16 = (S1[31 : 16].i16 >> S0.u32[19 : 16].u32);
tmp[15 : 0].i16 = (S1[15 : 0].i16 >> S0.u32[3 : 0].u32);
D0.b32 = tmp.b32
```

## V\_PK\_MAX\_I16

7

选择两个有符号的 16 位整数输入的组件最大值,并将所选值存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].i16 = S0[15 : 0].i16 >= S1[15 : 0].i16 ? S0[15 : 0].i16 : S1[15 : 0].i16;
tmp[31 : 16].i16 = S0[31 : 16].i16 >= S1[31 : 16].i16 ? S0[31 : 16].i16 : S1[31 : 16].i16;
D0.b32 = tmp
```

## V\_PK\_MIN\_I16

8

选择至少两个有符号的 16 位整数输入的组件,并将所选值存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].i16 = S0[15 : 0].i16 < S1[15 : 0].i16 ? S0[15 : 0].i16 : S1[15 : 0].i16;
tmp[31 : 16].i16 = S0[31 : 16].i16 < S1[31 : 16].i16 ? S0[31 : 16].i16 : S1[31 : 16].i16;
D0.b32 = tmp
```

## V\_PK\_MAD\_U16

9

将两个无符号的 16 位整数输入元件乘以 2,从第三个输入元件中添加一个无符号的 16 位整数值,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = S0[15 : 0].u16 * S1[15 : 0].u16 + S2[15 : 0].u16;
tmp[31 : 16].u16 = S0[31 : 16].u16 * S1[31 : 16].u16 + S2[31 : 16].u16;
D0.b32 = tmp
```

**V\_PK\_ADD\_U16****10**

依次添加两个无符号的 16 位整数输入符, 并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = S0[15 : 0].u16 + S1[15 : 0].u16;
tmp[31 : 16].u16 = S0[31 : 16].u16 + S1[31 : 16].u16;
D0.b32 = tmp
```

**V\_PK\_SUB\_U16****11**

从第一个输入组件中减去第二个无符号的 16 位整数输入, 并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = S0[15 : 0].u16 - S1[15 : 0].u16;
tmp[31 : 16].u16 = S0[31 : 16].u16 - S1[31 : 16].u16;
D0.b32 = tmp
```

**V\_PK\_MAX\_U16****12**

选择两个无符号的 16 位整数输入的组件最大值, 并将选定的值存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = S0[15 : 0].u16 >= S1[15 : 0].u16 ? S0[15 : 0].u16 : S1[15 : 0].u16;
tmp[31 : 16].u16 = S0[31 : 16].u16 >= S1[31 : 16].u16 ? S0[31 : 16].u16 : S1[31 : 16].u16;
D0.b32 = tmp
```

**V\_PK\_MIN\_U16****13**

选择两个无符号的 16 位整数输入的组件最小值, 并将选定的数值存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = S0[15 : 0].u16 < S1[15 : 0].u16 ? S0[15 : 0].u16 : S1[15 : 0].u16;
tmp[31 : 16].u16 = S0[31 : 16].u16 < S1[31 : 16].u16 ? S0[31 : 16].u16 : S1[31 : 16].u16;
D0.b32 = tmp
```

**V\_PK\_FMA\_F16****14**

对两个打包的半精度浮点输入逐分量相乘，再通过融合乘加逐分量加上第三个输入，结果写入向量寄存器。

```
declare tmp : 32'B;
tmp[15 : 0].f16 = fma(S0[15 : 0].f16, S1[15 : 0].f16, S2[15 : 0].f16);
tmp[31 : 16].f16 = fma(S0[31 : 16].f16, S1[31 : 16].f16, S2[31 : 16].f16);
D0.b32 = tmp
```

**V\_PK\_ADD\_F16****15**

按组件添加两个包装的半精度浮点数输入，并将结果存储到向量寄存器中。没有随身携带或随身携带支持。

```
declare tmp : 32'B;
tmp[15 : 0].f16 = S0[15 : 0].f16 + S1[15 : 0].f16;
tmp[31 : 16].f16 = S0[31 : 16].f16 + S1[31 : 16].f16;
D0.b32 = tmp
```

**V\_PK\_MUL\_F16****16**

将两个装箱的半精度浮点数输入组件相乘，并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].f16 = S0[15 : 0].f16 * S1[15 : 0].f16;
tmp[31 : 16].f16 = S0[31 : 16].f16 * S1[31 : 16].f16;
D0.b32 = tmp
```

**V\_PK\_MIN\_F16****17**

选择两组半精度半精度浮点输入的组件最小值，并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].f16 = v_min_f16(S0[15 : 0].f16, S1[15 : 0].f16);
tmp[31 : 16].f16 = v_min_f16(S0[31 : 16].f16, S1[31 : 16].f16);
D0.b32 = tmp
```

**V\_PK\_MAX\_F16****18**

选择两个包装的半精度浮点数的组件最大值,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].f16 = v_max_f16(S0[15 : 0].f16, S1[15 : 0].f16);
tmp[31 : 16].f16 = v_max_f16(S0[31 : 16].f16, S1[31 : 16].f16);
D0.b32 = tmp
```

**V\_MAD\_MIX\_F32****32**

乘以两个输入并添加第三个输入,其中输入是半精度浮点和单精度浮点值的组合。将结果存储到向量寄存器中。

3 个输入的大小和位置由{OPSEL\_HI[i],OPSEL[i]}控制:0=src[31:0],1=src[31:0],  
2=src[15:0], 3=src[31:16]. 对于 MIX 操作码,NEG\_HI 指令字段作为绝对值修饰符。

3 个输入。

```
declare in : 32'F[3];
declare S : 32'B[3];
for i in 0 : 2 do
    if !OPSEL_HI.u3[i] then
        in[i] = S[i].f32
    elsif OPSEL.u3[i] then
        in[i] = f16_to_f32(S[i][31 : 16].f16)
    else
        in[i] = f16_to_f32(S[i][15 : 0].f16)
    endif
endfor;
D0[31 : 0].f32 = in[0] * in[1] + in[2]
```

说明

**V\_MAD\_MIXLO\_F16****33**

乘以 2 个输入并添加第三个输入,其中输入是半精度浮点和单精度浮点值的组合。将结果转换为半精度浮点。将结果存储到向量寄存器的低位值中。

3 个输入的大小和位置由{OPSEL\_HI[i],OPSEL[i]}控制:0=src[31:0],1=src[31:0],  
2=src[15:0], 3=src[31:16]. 对于 MIX 操作码,NEG\_HI 指令字段作为绝对值修饰符.

3 个输入。

```
declare in : 32'F[3];
declare S : 32'B[3];
for i in 0 : 2 do
    if !OPSEL_HI.u3[i] then
        in[i] = S[i].f32
    elsif OPSEL.u3[i] then
        in[i] = f16_to_f32(S[i][31 : 16].f16)
    else
        in[i] = f16_to_f32(S[i][15 : 0].f16)
    endif
endfor;
D0[15 : 0].f16 = f32_to_f16(in[0] * in[1] + in[2])
```

说明

## V\_MAD\_MIXHI\_F16

34

乘以 2 个输入并添加第三个输入,其中输入是半精度浮点和单精度浮点值的组合。将结果转换为半精度浮点。将结果存储到向量寄存器的高位值中。

3 个输入的大小和位置由{OPSEL\_HI[i],OPSEL[i]}控制:0=src[31:0],1=src[31:0],  
2=src[15:0], 3=src[31:16]. 对于 MIX 操作码,NEG\_HI 指令字段作为绝对值修饰符.

3 个输入。

```
declare in : 32'F[3];
declare S : 32'B[3];
for i in 0 : 2 do
    if !OPSEL_HI.u3[i] then
        in[i] = S[i].f32
    elsif OPSEL.u3[i] then
        in[i] = f16_to_f32(S[i][31 : 16].f16)
    else
        in[i] = f16_to_f32(S[i][15 : 0].f16)
    endif
endfor;
D0[31 : 16].f16 = f32_to_f16(in[0] * in[1] + in[2])
```

说明

## V\_DOT2\_F32\_F16

35

在单精度浮点域中, 计算两个打包的二维半精度浮点输入的点积, 加上第三个输入的单精度浮点值, 结果写入向量寄存器。

```
tmp = S2.f32;
tmp += f16_to_f32(S0[15 : 0].f16) * f16_to_f32(S1[15 : 0].f16);
tmp += f16_to_f32(S0[31 : 16].f16) * f16_to_f32(S1[31 : 16].f16);
D0.f32 = tmp
```

**V\_DOT2\_I32\_I16****38**

计算两个打包的 2-D 在签名的 32 位整数域中签名的 16 位整数输入的点产品,从第三位输入中添加一个签名的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.i32;
tmp += i16_to_i32(S0[15 : 0].i16) * i16_to_i32(S1[15 : 0].i16);
tmp += i16_to_i32(S0[31 : 16].i16) * i16_to_i32(S1[31 : 16].i16);
D0.i32 = tmp
```

**V\_DOT2\_U32\_U16****39**

在无符号的 32 位整数域中计算两个打包的 2-D 无符号的 16 位整数输入的点产品,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.u32;
tmp += u16_to_u32(S0[15 : 0].u16) * u16_to_u32(S1[15 : 0].u16);
tmp += u16_to_u32(S0[31 : 16].u16) * u16_to_u32(S1[31 : 16].u16);
D0.u32 = tmp
```

**V\_DOT4\_I32\_I8****40**

在签名的 32 位整数域中计算两个包 4-D 签名的 8 位整数输入的点产品,从第三位输入中添加一个签名的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.i32;
tmp += i8_to_i32(S0[7 : 0].i8) * i8_to_i32(S1[7 : 0].i8);
tmp += i8_to_i32(S0[15 : 8].i8) * i8_to_i32(S1[15 : 8].i8);
tmp += i8_to_i32(S0[23 : 16].i8) * i8_to_i32(S1[23 : 16].i8);
tmp += i8_to_i32(S0[31 : 24].i8) * i8_to_i32(S1[31 : 24].i8);
D0.i32 = tmp
```

**V\_DOT4\_U32\_U8****41**

在无符号的 32 位整数中计算两个已打包的 4-D 无符号的 8 位整数输入的点产品

域中,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.u32;
tmp += u8_to_u32(S0[7 : 0].u8) * u8_to_u32(S1[7 : 0].u8);
tmp += u8_to_u32(S0[15 : 8].u8) * u8_to_u32(S1[15 : 8].u8);
tmp += u8_to_u32(S0[23 : 16].u8) * u8_to_u32(S1[23 : 16].u8);
tmp += u8_to_u32(S0[31 : 24].u8) * u8_to_u32(S1[31 : 24].u8);
D0.u32 = tmp
```

## V\_DOT8\_I32\_I4

42

在签名的 32 位整数域中计算两个打包的 8-D 签名的 4 位整数输入的点产品,从第三个输入中添加一个签名的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.i32;
tmp += i4_to_i32(S0[3 : 0].i4) * i4_to_i32(S1[3 : 0].i4);
tmp += i4_to_i32(S0[7 : 4].i4) * i4_to_i32(S1[7 : 4].i4);
tmp += i4_to_i32(S0[11 : 8].i4) * i4_to_i32(S1[11 : 8].i4);
tmp += i4_to_i32(S0[15 : 12].i4) * i4_to_i32(S1[15 : 12].i4);
tmp += i4_to_i32(S0[19 : 16].i4) * i4_to_i32(S1[19 : 16].i4);
tmp += i4_to_i32(S0[23 : 20].i4) * i4_to_i32(S1[23 : 20].i4);
tmp += i4_to_i32(S0[27 : 24].i4) * i4_to_i32(S1[27 : 24].i4);
tmp += i4_to_i32(S0[31 : 28].i4) * i4_to_i32(S1[31 : 28].i4);
D0.i32 = tmp
```

## V\_DOT8\_U32\_U4

43

在无符号的 32 位整数域中计算两个打包的 8-D 无符号的 4 位整数输入的点积,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
tmp = S2.u32;
tmp += u4_to_u32(S0[3 : 0].u4) * u4_to_u32(S1[3 : 0].u4);
tmp += u4_to_u32(S0[7 : 4].u4) * u4_to_u32(S1[7 : 4].u4);
tmp += u4_to_u32(S0[11 : 8].u4) * u4_to_u32(S1[11 : 8].u4);
tmp += u4_to_u32(S0[15 : 12].u4) * u4_to_u32(S1[15 : 12].u4);
tmp += u4_to_u32(S0[19 : 16].u4) * u4_to_u32(S1[19 : 16].u4);
tmp += u4_to_u32(S0[23 : 20].u4) * u4_to_u32(S1[23 : 20].u4);
tmp += u4_to_u32(S0[27 : 24].u4) * u4_to_u32(S1[27 : 24].u4);
tmp += u4_to_u32(S0[31 : 28].u4) * u4_to_u32(S1[31 : 28].u4);
D0.u32 = tmp
```

## V\_PK\_FMA\_F32

48

将两个装箱的单精度浮点形输入组件进行乘法,并使用第三个输入组件进行乘法

融合乘加,并将结果存储到向量寄存器中。

```
declare tmp : 64'B;
tmp[31 : 0].f32 = fma(S0[31 : 0].f32, S1[31 : 0].f32, S2[31 : 0].f32);
tmp[63 : 32].f32 = fma(S0[63 : 32].f32, S1[63 : 32].f32, S2[63 : 32].f32);
D0.b64 = tmp
```

## V\_PK\_MUL\_F32

49

将两个装箱的单精度浮点输入组件相乘,并将结果存储到向量寄存器中。

```
declare tmp : 64'B;
tmp[31 : 0].f32 = S0[31 : 0].f32 * S1[31 : 0].f32;
tmp[63 : 32].f32 = S0[63 : 32].f32 * S1[63 : 32].f32;
D0.b64 = tmp
```

## V\_PK\_ADD\_F32

50

依次添加两个组合的单精度浮点输入组件,并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
declare tmp : 64'B;
tmp[31 : 0].f32 = S0[31 : 0].f32 + S1[31 : 0].f32;
tmp[63 : 32].f32 = S0[63 : 32].f32 + S1[63 : 32].f32;
D0.b64 = tmp
```

## V\_PK\_MOV\_B32

51

将数据从两个向量输入器移到两个向量记录器中。

```
tmp0.u32 = S0.u32[OPSEL[0].i32 * 32 + 31 : OPSEL[0].i32 * 32];
tmp1.u32 = S1.u32[OPSEL[1].i32 * 32 + 31 : OPSEL[1].i32 * 32];
D0.u32[31 : 0] = tmp0.u32;
D0.u32[63 : 32] = tmp1.u32
```

### 说明

源操作被作为 64 位处理,并受到 SGPR 和 VGPR 的对齐限制。

对于两个 VGPR 输入,这个 opcode 可以使用 OP\_SEL 任意的集合来选择指定的偶数 VGPR 或下一个奇数 VGPR。



```
v_pk_mov_b32 v0, v2, v4 op_sel:[0,1] // 评估 v0 <-v2 和 v1 <-v5。
```

由于标量广播限制,如果将两个 SGPRs 指定为操作,它们必须是相同的 SGPR。

```
v_pk_mov_b32 v0, s6, s6 op_sel:[0,1] // 64-bit move from scalar s[6:7].
```

## V\_MFMA\_F32\_16X16X8\_XF32

62

将第一个输入的 16x8 矩阵与第二个输入的 8x16 矩阵相乘, 通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 8) * B (8 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

Matrices A 和 B 是单精度浮点格式. Matrices C 和 D 是单精度浮点格式. XF32 是使用 FP32 输入和输出的 FP32 操作,但执行时中间精度降低,其中曼提萨切为 10 位(不包括非零值的领先 1 位),结果累积为 FP32 值,为 23 位曼提萨。

说明

本指令执行 4 次通行证。

## V\_MFMA\_F32\_32X32X4\_XF32

63

将第一个输入的 32x4 矩阵与第二个输入的 4x32 矩阵相乘, 通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 4) * B (4 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

Matrices A 和 B 是单精度浮点格式. Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 8 次通行证。

**V\_MFMA\_F32\_32X32X1\_2B\_F32****64**

将第一个输入的 32x1 矩阵与第二个输入的 1x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 1) * B (1 \times 32) + C (32 \times 32)$$

此指令执行 2 矩阵乘法。每个操作包含两个矩阵，每个矩阵包含分布在波段所有通道的元素。每个矩阵多被计算，并且行列点产品分布在向量 ALU 之间，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是单精度浮点格式。Matrices C 和 D 是单精度浮点格式。

说明

本指示执行 16 次通行证。

**V\_MFMA\_F32\_16X16X1\_4B\_F32****65**

将第一个输入的 16x1 矩阵与第二个输入的 1x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 1) * B (1 \times 16) + C (16 \times 16)$$

此指令执行 4 个矩阵乘数。每个操作包含 4 个矩阵，每个矩阵都有分布在波段所有通道的元素。每个矩阵多重被计算出来，并且行柱点产品被分布在向量 ALU 上，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是单精度浮点格式。Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 8 次通行证。

**V\_MFMA\_F32\_4X4X1\_16B\_F32****66**

将第一个输入的 4x1 矩阵与第二个输入的 1x4 矩阵相乘，通过融合乘加累加第三个输入的 4x4 矩阵。将结果矩阵写入向量寄存器。

$$D = A (4 \times 1) * B (1 \times 4) + C (4 \times 4)$$

本指令执行 16 个矩阵乘数。每个运行符包含 16 个矩阵, 每个矩阵包含分布在波浪所有通道的元素。每个矩阵将计算出多个矩阵, 并且将列柱点产品分布在向量 ALU 之间, 用于更高的性能。结果矩阵将背对背存储在目的地向量寄存器中。

Matrices A 和 B 是单精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指示执行 2 次通行证。

**V\_MFMA\_F32\_32X32X2\_F32**

**68**

将第一个输入的 32x2 矩阵与第二个输入的 2x32 矩阵相乘, 通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A \text{ (32x2)} * B \text{ (2x32)} + C \text{ (32x32)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能.

Matrices A 和 B 是单精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指示执行 16 次通行证。

**V\_MFMA\_F32\_16X16X4\_F32**

**69**

将第一个输入的 16x4 矩阵与第二个输入的 4x16 矩阵相乘, 通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A \text{ (16x4)} * B \text{ (4x16)} + C \text{ (16x16)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能.

Matrices A 和 B 是单精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 8 次通行证。

**V\_MFMA\_F32\_32X32X4\_2B\_F16**

72

将第一个输入的 32x4 矩阵与第二个输入的 4x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 4) * B (4 \times 32) + C (32 \times 32)$$

此指令执行 2 矩阵乘法。每个操作包含两个矩阵，每个矩阵包含分布在波段所有通道的元素。每个矩阵多被计算，并且行列点产品分布在向量 ALU 之间，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是半精度浮点格式。Matrices C 和 D 是单精度浮点格式。

说明

本指示执行 16 次通行证。

**V\_MFMA\_F32\_16X16X4\_4B\_F16**

73

将第一个输入的 16x4 矩阵与第二个输入的 4x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 4) * B (4 \times 16) + C (16 \times 16)$$

此指令执行 4 个矩阵乘数。每个操作包含 4 个矩阵，每个矩阵都有分布在波段所有通道的元素。每个矩阵多重被计算出来，并且行柱点产品被分布在向量 ALU 上，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是半精度浮点格式。Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 8 次通行证。

**V\_MFMA\_F32\_4X4X4\_16B\_F16**

74

将第一个输入的 4x4 矩阵与第二个输入的 4x4 矩阵相乘，通过融合乘加累加第三个输入的 4x4 矩阵。将结果矩阵写入向量寄存器。

$$D = A (4 \times 4) * B (4 \times 4) + C (4 \times 4)$$

此指令执行 16 个矩阵乘法。每个操作包含 16 个矩阵。

矩阵有分布在波浪所有行的元素。每个矩阵多重被计算出来,行柱点产品被分布在向量 ALU 之间,以达到更高的性能。结果矩阵被背靠背地存储在目的地向量寄存器中。

Matrices A 和 B 是半精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指示执行 2 次通行证。

V\_MFMA\_F32\_32X32X8\_F16

76

将第一个输入的 32x8 矩阵与第二个输入的 8x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

D = A (32x8) \* B (8x32) + C (32x32)

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能.

Matrices A 和 B 是半精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 8 次通行证。

V\_MFMA\_F32\_16X16X16\_F16

77

将第一个输入的 16x16 矩阵与第二个输入的 16x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

D = A (16x16) \* B (16x16) + C (16x16)

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能.

Matrices A 和 B 是半精度浮点格式. Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 4 次通行证。

**V\_MFMA\_I32\_32X32X4\_2B\_I8****80**

将第一个输入的 32x4 矩阵与第二个输入的 4x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 4) * B (4 \times 32) + C (32 \times 32)$$

此指令执行 2 矩阵乘法。每个操作包含两个矩阵，每个矩阵包含分布在波段所有通道的元素。每个矩阵多被计算，并且行列点产品分布在向量 ALU 之间，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是签名的 8 位整数格式, Matrices C 和 D 是签名的 32 位整数格式。

说明

本指示执行 16 次通行证。

**V\_MFMA\_I32\_16X16X4\_4B\_I8****81**

将第一个输入的 16x4 矩阵与第二个输入的 4x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 4) * B (4 \times 16) + C (16 \times 16)$$

此指令执行 4 个矩阵乘数。每个操作包含 4 个矩阵，每个矩阵都有分布在波段所有通道的元素。每个矩阵多重被计算出来，并且行柱点产品被分布在向量 ALU 上，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是签名的 8 位整数格式, Matrices C 和 D 是签名的 32 位整数格式。

说明

本指令执行 8 次通行证。

**V\_MFMA\_I32\_4X4X4\_16B\_I8****82**

将第一个输入的 4x4 矩阵与第二个输入的 4x4 矩阵相乘，通过融合乘加累加第三个输入的 4x4 矩阵。将结果矩阵写入向量寄存器。

$$D = A (4 \times 4) * B (4 \times 4) + C (4 \times 4)$$

此指令执行 16 个矩阵乘法。每个操作包含 16 个矩阵。

矩阵有分布在波浪所有行的元素。每个矩阵多重被计算出来,行柱点产品被分布在向量 ALU 之间,以达到更高的性能。结果矩阵被背靠背地存储在目的地向量寄存器中。

Matrices A 和 B 是签名的 8 位整数格式,Matrices C 和 D 是签名的 32 位整数格式。

说明

本指示执行 2 次通行证。

---

### V\_MFMA\_I32\_32X32X16\_I8

86

将第一个输入的 32x16 矩阵与第二个输入的 16x32 矩阵相乘,通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 16) * B (16 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

Matrices A 和 B 是签名的 8 位整数格式,Matrices C 和 D 是签名的 32 位整数格式。

说明

本指令执行 8 次通行证。

---

### V\_MFMA\_I32\_16X16X32\_I8

87

将第一个输入的 16x32 矩阵与第二个输入的 32x16 矩阵相乘,通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 32) * B (32 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

Matrices A 和 B 是签名的 8 位整数格式,Matrices C 和 D 是签名的 32 位整数格式。

说明

本指令执行 4 次通行证。

**V\_ACCVGPR\_READ****88**

将 32 位数据从累积器向量寄存器移到建筑向量寄存器中。

**V\_ACCVGPR\_WRITE****89**

将 32 位数据从建筑向量寄存器中移动到蓄积器向量寄存器中。

**V\_MFMA\_F32\_32X32X4\_2B\_BF16****93**

将第一个输入的 32x4 矩阵与第二个输入的 4x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 4) * B (4 \times 32) + C (32 \times 32)$$

此指令执行 2 矩阵乘法。每个操作包含两个矩阵，每个矩阵包含分布在波段所有通道的元素。每个矩阵多被计算，并且行列点产品分布在向量 ALU 之间，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 为 BF16 浮点格式。Matrices C 和 D 为单精度浮点格式。

说明

本指示执行 16 次通行证。

**V\_MFMA\_F32\_16X16X4\_4B\_BF16****94**

将第一个输入的 16x4 矩阵与第二个输入的 4x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 4) * B (4 \times 16) + C (16 \times 16)$$

此指令执行 4 个矩阵乘数。每个操作包含 4 个矩阵，每个矩阵都有分布在波段所有通道的元素。每个矩阵多重被计算出来，并且行柱点产品被分布在向量 ALU 上，以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 为 BF16 浮点格式。Matrices C 和 D 为单精度浮点格式。

说明

本指令执行 8 次通行证。



**V\_MFMA\_F32\_4X4X4\_16B\_BF16**

95

将第一个输入的 4x4 矩阵与第二个输入的 4x4 矩阵相乘，通过融合乘加累加第三个输入的 4x4 矩阵。将结果矩阵写入向量寄存器。

$$D = A (4 \times 4) * B (4 \times 4) + C (4 \times 4)$$

本指令执行 16 个矩阵乘数。每个运行符包含 16 个矩阵，每个矩阵包含分布在波浪所有通道的元素。每个矩阵将计算出多个矩阵，并且将列柱点产品分布在向量 ALU 之间，用于更高的性能。结果矩阵将背对背存储在目的地向量寄存器中。

Matrices A 和 B 为 BF16 浮点格式。Matrices C 和 D 为单精度浮点格式。

说明

本指示执行 2 次通行证。

**V\_MFMA\_F32\_32X32X8\_BF16**

96

将第一个输入的 32x8 矩阵与第二个输入的 8x32 矩阵相乘，通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 8) * B (8 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵，其元素分布在波浪的所有航道上，计算出一个单一的矩阵乘数，并在向量 ALU 之间分配行柱点产品，以达到更高的性能。

Matrices A 和 B 为 BF16 浮点格式。Matrices C 和 D 为单精度浮点格式。

说明

本指令执行 8 次通行证。

**V\_MFMA\_F32\_16X16X16\_BF16**

97

将第一个输入的 16x16 矩阵与第二个输入的 16x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 16) * B (16 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一矩阵，其元素分布在波浪的所有通道上。

计算矩阵乘数,并将行柱点产品分布在向量 ALU 之间,以便提高性能。

Matrices A 和 B 为 BF16 浮点格式. Matrices C 和 D 为单精度浮点格式.

说明

本指令执行 4 次通行证。

**V\_SMFMAC\_F32\_16X16X32\_F16**

98

将第一个输入的 16x32 稀疏矩阵与第二个输入的 32x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A \text{ (sparse 16x32)} * B \text{ (32x16)} + D \text{ (16x16)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是一种半精度浮点格式的稀疏矩阵,消耗了同样尺寸的稠密矩阵的物理存储量的一半. 矩阵 B 是一个半精度浮点格式的稠密矩阵. 矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。 稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

**V\_SMFMAC\_F32\_32X32X16\_F16**

100

将第一个输入的 32x16 稀疏矩阵与第二个输入的 16x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A \text{ (sparse 32x16)} * B \text{ (16x32)} + D \text{ (32x32)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是一个半精度浮点格式的稀疏矩阵,消耗了同样尺寸的稠密矩阵的物理存储量的一半. 矩阵 B 是一个半精度浮点格式的稠密矩阵. 矩阵 D 是单-。

精密浮点格式,是输出和累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

### V\_SMFMAC\_F32\_16X16X32\_BF16

102

将第一个输入的 16x32 稀疏矩阵与第二个输入的 32x16 矩阵相乘,通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 16 \times 32) * B (32 \times 16) + D (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF16 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 BF16 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_F32\_32X32X16\_BF16

104

将第一个输入的 32x16 稀疏矩阵与第二个输入的 16x32 矩阵相乘,通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 32 \times 16) * B (16 \times 32) + D (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF16 浮点格式的稀疏矩阵,消耗了相同尺寸的稠密矩阵的一半物理存储量。矩阵 B 是 BF16 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式和

是输出和累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

### V\_SMFMAC\_I32\_16X16X64\_I8

106

将第一个输入的 16x64 稀疏矩阵与第二个输入的 64x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A \text{ (sparse 16x64)} * B \text{ (64x16)} + D \text{ (16x16)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是一种签名的 8 位整数格式的稀疏矩阵,消耗了相同维度的稠密矩阵的物理存储的一半。矩阵 B 是一个签名的 8 位整数格式的稠密矩阵。矩阵 D 是签名的 32 位整数格式,既是输出,也是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_I32\_32X32X32\_I8

108

将第一个输入的 32x32 稀疏矩阵与第二个输入的 32x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A \text{ (sparse 32x32)} * B \text{ (32x32)} + D \text{ (32x32)}$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是签字 8 位整数格式的稀疏矩阵,消耗了相同尺寸的稠密矩阵实际存储的一半。矩阵 B 是签字 8 位整数格式的稠密矩阵。矩阵 D 是签字 32-

位整数格式,是输出和累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

### V\_MFMA\_F64\_16X16X4\_F64

110

将第一个输入的 16x4 矩阵与第二个输入的 4x16 矩阵相乘,通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 4) * B (4 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

Matrices A 和 B 是双精度浮点格式。Matrices C 和 D 是双精度浮点格式。

说明

本指令执行 8 次通行证。

### V\_MFMA\_F64\_4X4X4\_4B\_F64

111

将第一个输入的 4x4 矩阵与第二个输入的 4x4 矩阵相乘,通过融合乘加累加第三个输入的 4x4 矩阵。将结果矩阵写入向量寄存器。

$$D = A (4 \times 4) * B (4 \times 4) + C (4 \times 4)$$

此指令执行 4 个矩阵乘数。每个操作包含 4 个矩阵,每个矩阵都有分布在波段所有通道的元素。每个矩阵多重被计算出来,并且行柱点产品被分布在向量 ALU 上,以获取更高的性能。结果矩阵被背向存储在目标向量寄存器中。

Matrices A 和 B 是双精度浮点格式。Matrices C 和 D 是双精度浮点格式。

说明

本指令执行 4 次通行证。

**V\_MFMA\_F32\_16X16X32\_BF8\_BF8****112**

将第一个输入的 16x32 矩阵与第二个输入的 32x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 32) * B (32 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式,矩阵 B 是 BF8 浮点格式,Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 4 次通行证。

**V\_MFMA\_F32\_16X16X32\_BF8\_FP8****113**

将第一个输入的 16x32 矩阵与第二个输入的 32x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 32) * B (32 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式,矩阵 B 是 FP8 浮点格式,Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 4 次通行证。

**V\_MFMA\_F32\_16X16X32\_FP8\_BF8****114**

将第一个输入的 16x32 矩阵与第二个输入的 32x16 矩阵相乘，通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 32) * B (32 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式,矩阵 B 是 BF8 浮点格式,Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 4 次通行证。

### V\_MFMA\_F32\_16X16X32\_FP8\_FP8

115

将第一个输入的 16x32 矩阵与第二个输入的 32x16 矩阵相乘, 通过融合乘加累加第三个输入的 16x16 矩阵。将结果矩阵写入向量寄存器。

$$D = A (16 \times 32) * B (32 \times 16) + C (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式,矩阵 B 是 FP8 浮点格式,Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 4 次通行证。

### V\_MFMA\_F32\_32X32X16\_BF8\_BF8

116

将第一个输入的 32x16 矩阵与第二个输入的 16x32 矩阵相乘, 通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 16) * B (16 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式,矩阵 B 是 BF8 浮点格式,Matrices C 和 D 是单精度浮点格式.

说明

本指令执行 8 次通行证。

### V\_MFMA\_F32\_32X32X16\_BF8\_FP8

117

将第一个输入的 32x16 矩阵与第二个输入的 16x32 矩阵相乘, 通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 16) * B (16 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式,矩阵 B 是 FP8 浮点格式,Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 8 次通行证。

### V\_MFMA\_F32\_32X32X16\_FP8\_BF8

118

将第一个输入的 32x16 矩阵与第二个输入的 16x32 矩阵相乘,通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 16) * B (16 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式,矩阵 B 是 BF8 浮点格式,Matrices C 和 D 是单精度浮点格式。

说明

本指令执行 8 次通行证。

### V\_MFMA\_F32\_32X32X16\_FP8\_FP8

119

将第一个输入的 32x16 矩阵与第二个输入的 16x32 矩阵相乘,通过融合乘加累加第三个输入的 32x32 矩阵。将结果矩阵写入向量寄存器。

$$D = A (32 \times 16) * B (16 \times 32) + C (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式,矩阵 B 是 FP8 浮点格式,Matrices C 和 D 是单精度浮点格式。

说明



本指令执行 8 次通行证。

### V\_SMFMAC\_F32\_16X16X64\_BF8\_BF8

120

将第一个输入的 16x64 稀疏矩阵与第二个输入的 64x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 16 \times 64) * B (64 \times 16) + D (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 BF8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_F32\_16X16X64\_BF8\_FP8

121

将第一个输入的 16x64 稀疏矩阵与第二个输入的 64x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 16 \times 64) * B (64 \times 16) + D (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 FP8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_F32\_16X16X64\_FP8\_BF8

122

将第一个输入的 16x64 稀疏矩阵与第二个输入的 64x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 16 \times 64) * B (64 \times 16) + D (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 BF8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_F32\_16X16X64\_FP8\_FP8

123

将第一个输入的 16x64 稀疏矩阵与第二个输入的 64x16 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 16x16 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 16 \times 64) * B (64 \times 16) + D (16 \times 16)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 FP8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 4 次通行证。

### V\_SMFMAC\_F32\_32X32X32\_BF8\_BF8

124

将第一个输入的 32x32 稀疏矩阵与第二个输入的 32x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 32 \times 32) * B (32 \times 32) + D (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 BF8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

### V\_SMFMAC\_F32\_32X32X32\_BF8\_FP8

125

将第一个输入的 32x32 稀疏矩阵与第二个输入的 32x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

$$D = A (\text{sparse } 32 \times 32) * B (32 \times 32) + D (32 \times 32)$$

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 BF8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 FP8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

V\_SMFMAC\_F32\_32X32X32\_FP8\_BF8

126

将第一个输入的 32x32 稀疏矩阵与第二个输入的 32x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

D = A (sparse 32x32) \* B (32x32) + D (32x32)

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 BF8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

说明

本指令执行 8 次通行证。

V\_SMFMAC\_F32\_32X32X32\_FP8\_FP8

127

将第一个输入的 32x32 稀疏矩阵与第二个输入的 32x32 矩阵相乘，通过融合乘加把结果累加到目标寄存器保存的 32x32 矩阵中。第三个输入提供第一个矩阵的稀疏索引。

D = A (sparse 32x32) \* B (32x32) + D (32x32)

每个操作都包含一个单一的矩阵,其元素分布在波浪的所有航道上,计算出一个单一的矩阵乘数,并在向量 ALU 之间分配行柱点产品,以达到更高的性能。

矩阵 A 是 FP8 浮点格式的稀疏矩阵,消耗了相同维度的稠密矩阵的一半物理存储。矩阵 B 是 FP8 浮点格式的稠密矩阵。矩阵 D 是单精度浮点格式,既是输出,又是累积输入。

K 矩阵 A 轴上每 4 个元素中有 2 个是零。稀疏的指数用于确定哪些元素是零。

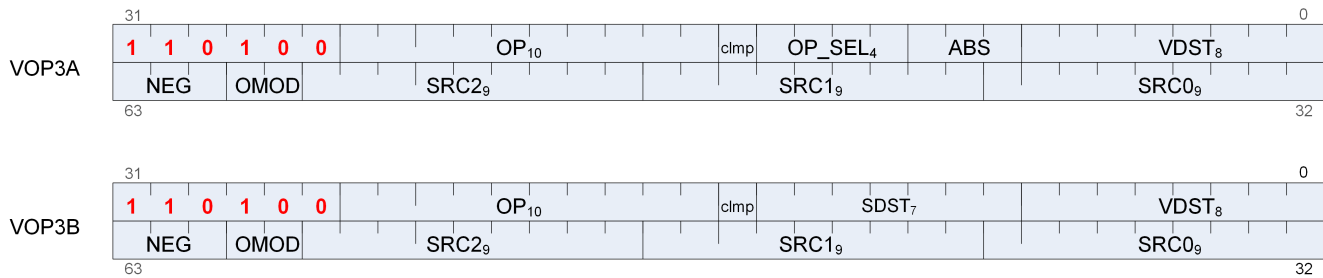
说明

本指令执行 8 次通行证。

---

## 12.11. VOP3A & VOP3B 指令

VOP3 指令使用两个编码中的一个:



**VOP3B** 此编码允许单独指定一个标量目标操作数，仅用于以下指令：V\_ADD\_CO\_U32、V\_SUB\_CO\_U32、V\_SUBREV\_CO\_U32、V\_ADDC\_CO\_U32、V\_SU<sub>BB</sub>\_CO\_U32、V\_SUBBREV\_CO\_U32、V\_DIV\_SCALE\_F32、V\_DIV\_SCALE\_F64、V\_MAD\_U64\_U32、V\_MAD\_I64\_I32。

**VOP3A** 所有其他 VALU 指令使用此编码

V\_NOP

384

什么都别做。

说明

此指令可用于在向量 ALU 管道中插入单个循环气泡。对于多个循环, 重复此操作码。

V\_MOV\_B32

385

将向量输入的 32 位数据移动到向量寄存器中。

D0.b32 = S0.b32

说明

如果 S0 是 32 位浮点值,则浮点修改符对本指令有效。本指令适用于否定或占用浮点值的绝对值。

功能实例:

```
v_mov_b32 v0, v1    // 从 v1 移动到 v0
v_mov_b32 v0, -v1   // 设定 v0 到否定 v1
v_mov_b32 v0, abs(v1) // 将 v0 设定为 v1 的绝对值
```

## V\_READFIRSTLANE\_B32

386

读取输入向量寄存器最低活动道的标量值,将其存储到标量寄存器中。

```
declare lane : 32'I;
if EXEC == 0x0LL then
    lane = 0;
    // 如果全部通道禁用,则强制通道 0
else
    lane = s_ff1_i32_b64(EXEC);
    // 最低有效通道
endif;
D0.b32 = VGPR[lane][SRC0.u32]
```

说明

VGPR 的遮罩被替换为 EXEC 。 不支持输入和输出修改器; 这是一个未键入的操作 。

## V\_CVT\_I32\_F64

387

从双精度浮点数输入转换为签名的 32 位整数,并将结果存储到向量寄存器中。

```
D0.i32 = f64_to_i32(S0.f64)
```

说明

0.5ULP 精度,外程浮点值(包括无限)饱和。 NAN 转换为 0。

INEXACT 例外的生成由 CLAMP 位控制。 INEXACT 例外被启用用于此转换 if CLAMP == 1。

## V\_CVT\_F64\_I32

388

从一个签名的 32 位整数输入转换为双精度浮点值,并将结果存储到向量寄存器中。

```
D0.f64 = i32_to_f64(S0.i32)
```

说明

0ULP 精度.

---

### V\_CVT\_F32\_I32

**389**

从签名的 32 位整数输入转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = i32_to_f32(S0.i32)
```

说明

0.5ULP 精度.

---

### V\_CVT\_F32\_U32

**390**

从无符号的 32 位整数输入转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0.u32)
```

说明

0.5ULP 精度.

---

### V\_CVT\_U32\_F32

**391**

从单精度浮点输入转换为无符号的 32 位整数值,并将结果存储到向量寄存器中.

```
D0.u32 = f32_to_u32(S0.f32)
```

说明

1ULP 精度,外程浮点值(包括无穷)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.



**V\_CVT\_I32\_F32****392**

从单精度浮点输入转换为签名的 32 位整数值,并将结果存储到向量寄存器中。

```
D0.i32 = f32_to_i32(S0.f32)
```

说明

1ULP 精度,外程浮点值(包括无穷)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

**V\_CVT\_F16\_F32****394**

从单精度浮点输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = f32_to_f16(S0.f32)
```

说明

0.5ULP 精度,支持输入修饰器,并在适当的时候创建 FP16 非正常性. Flush denorms 如果根据 DP denorm 模式指定,则在输出上进行 Flush denorms. 输出四舍五入基于 DP 四舍五入模式.

**V\_CVT\_F32\_F16****395**

从半精度浮点输入转换为单精度浮点值,并将结果存储为向量寄存器。

```
D0.f32 = f16_to_f32(S0.f16)
```

说明

0ULP 精度,FP16 非正常输入被接受. Flush denorm 如果根据 DP denorm 模式指定,则在输入上进行 flush denorm.

**V\_CVT\_RPI\_I32\_F32****396**

从单精度浮点输入转换为签名的 32 位整数值,使用圆形到最近的整数语义(忽略默认的四舍五入模式),并将结果存储到向量寄存器中。

```
D0.i32 = f32_to_i32(floor(S0.f32 + 0.5F))
```

说明

0.5ULP 精度,异常值得到支持。

## V\_CVT\_FLR\_I32\_F32

397

使用圆形的语义(忽略默认的四舍五入模式)从单精度浮点输入转换为签名的 32 位整数值,并将结果存储到向量寄存器中。

```
D0.i32 = f32_to_i32(floor(S0.f32))
```

说明

1ULP 精确度,异常值得到支持。

## V\_CVT\_OFF\_F32\_I4

398

使用一个偏移表,从一个签名的 4 位整数输入转换成一个单精度浮点值,并将结果存储到向量寄存器中。

在阴影中用于插值。S0 [3: 0] 上的查询表：

S0 二进制值与结

果: 1000 -

0.5000f 1001 -

0.4375f 1010 -

0.3750f 1011 -

0.3125f 1100 -

0.2500f 1101 -

0.1875f 1110 -

0.1250f 1111 -

0.0625f 0000 +

0.0000f 0001 +

0.0625f 0010 +

0.1250f 0011 +

0.1875f 0100 +

0.2500f 0101 +

0.3125f 0110 +

0.3750f 0111 +

0.4375f

```
declare CVT_OFF_TABLE : 32'F[16];  
D0.f32 = CVT_OFF_TABLE[S0.u32[3 : 0]]
```

### V\_CVT\_F32\_F64

399

从双精度浮点输入转换为单精度浮点值,并将结果存储为向量寄存器.

```
D0.f32 = f64_to_f32(S0.f64)
```

说明

0.5ULP 精度,异常值得到支持.

### V\_CVT\_F64\_F32

400

从单精度浮点输入转换为双精度浮点值,并将结果存储到向量寄存器中.

```
D0.f64 = f32_to_f64(S0.f32)
```

说明

0ULP 精确度,异常值得到支持.

### V\_CVT\_F32\_UBYTE0

401

将输入的字节 0 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0[7 : 0].u32)
```

### V\_CVT\_F32\_UBYTE1

402

将输入的字节 1 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0[15 : 8].u32)
```

### V\_CVT\_F32\_UBYTE2

403

将输入的字节 2 中的无符号字节转换为单精度浮点值,并将结果存储为向量寄存器.

```
D0.f32 = u32_to_f32(S0[23 : 16].u32)
```

### V\_CVT\_F32\_UBYTE3

404

将输入的字节 3 中的无符号字节转换为单精度浮点值,并将结果存储到向量寄存器中.

```
D0.f32 = u32_to_f32(S0[31 : 24].u32)
```

### V\_CVT\_U32\_F64

405

从双精度浮点数输入转换为无符号的 32 位整数值,并将结果存储到向量寄存器中.

```
D0.u32 = f64_to_u32(S0.f64)
```

说明

0.5ULP 精度,外程浮点值(包括无限)饱和度. NAN 转换为 0.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

### V\_CVT\_F64\_U32

406

从无符号的 32 位整数输入转换成双精度浮点值,并将结果存储到向量寄存器中.

```
D0.f64 = u32_to_f64(S0.u32)
```

说明

OULP 精度.

---

### V\_TRUNC\_F64

407

使用朝零舍入语义计算双精度浮点输入的整数部分,并将结果以浮点格式存储到向量寄存器中.

```
D0.f64 = trunc(S0.f64)
```

---

### V\_CEIL\_F64

408

将双精度浮点输入绕到下一个整数,并将结果以浮点格式存储到向量寄存器中.

```
D0.f64 = trunc(S0.f64);  
if ((S0.f64 > 0.0) && (S0.f64 != D0.f64)) then  
    D0.f64 += 1.0  
endif
```

---

### V\_RNDNE\_F64

409

将双精度浮点输入舍入到最近整数, 恰好位于两个整数中间时取偶数; 以浮点格式将结果写入向量寄存器.

```
D0.f64 = floor(S0.f64 + 0.5);  
if (isEven(floor(S0.f64)) && (fract(S0.f64) == 0.5)) then  
    D0.f64 -= 1.0  
endif
```

---

### V\_FLOOR\_F64

410

将双精度浮点输入向下绕到之前的整数,并将结果以浮点格式存储到向量寄存器中.

```
D0.f64 = trunc(S0.f64);  
if ((S0.f64 < 0.0) && (S0.f64 != D0.f64)) then  
    D0.f64 += -1.0  
endif
```

**V\_FRACT\_F32****411**

计算单精度浮点输入的小数部分，并以浮点格式将结果写入向量寄存器。

```
D0.f32 = S0.f32 + -floor(S0.f32)
```

说明

精度为 0.5 ULP，允许非规格化输入。

本指令旨在符合 DX 对 fract 的定义，其行为相当于整数取模的扩展。其他领域对 fract() 的定义可能不同。例如，DX 中  $\text{fract}(-1.2) = 0.8$ 。

遵循舍入模式，结果限制在 0x3f7ffff 以内。

**V\_TRUNC\_F32****412**

按朝零舍入的语义计算单精度浮点输入的整数部分，并以浮点格式将结果写入向量寄存器。

```
D0.f32 = trunc(S0.f32)
```

**V\_CEIL\_F32****413**

将单精度浮点输入向上舍入到整数，并以浮点格式将结果写入向量寄存器。

```
D0.f32 = trunc(S0.f32);  
if ((S0.f32 > 0.0F) && (S0.f32 != D0.f32)) then  
    D0.f32 += 1.0F  
endif
```

**V\_RNDNE\_F32****414**

将单精度浮点输入舍入到最近整数，恰好位于两个整数中间时取偶数；以浮点格式将结果写入向量寄存器。

```
D0.f32 = floor(S0.f32 + 0.5F);
```

```
if (isEven(64'F(floor(S0.f32))) && (fract(S0.f32) == 0.5F)) then
    D0.f32 -= 1.0F
endif
```

**V\_FLOOR\_F32****415**

将单精度浮点输入向下绕到以前的整数,并将结果以浮点格式存储到向量寄存器中.

```
D0.f32 = trunc(S0.f32);
if ((S0.f32 < 0.0F) && (S0.f32 != D0.f32)) then
    D0.f32 += -1.0F
endif
```

**V\_EXP\_F32****416**

计算 2 升至单精度浮点输入的功率,并将结果存储到向量寄存器中.

```
D0.f32 = pow(2.0F, S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_EXP_F32(0xff800000) => 0x00000000    // exp(-INF) = 0
V_EXP_F32(0x80000000) => 0x3f800000    // exp(-0.0) = 1
V_EXP_F32(0x7f800000) => 0x7f800000    // exp(+INF) = +INF
```

**V\_LOG\_F32****417**

计算单精度浮点输入的基数 2 对数,并将结果存储到向量寄存器中.

```
D0.f32 = log2(S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_LOG_F32(0xff800000) => 0xffc00000    // log(-INF) = NAN
V_LOG_F32(0xbf800000) => 0xffc00000    // log(-1.0) = NAN
V_LOG_F32(0x80000000) => 0xff800000    // log(-0.0) = -INF
V_LOG_F32(0x00000000) => 0xff800000    // log(+0.0) = -INF
V_LOG_F32(0x3f800000) => 0x00000000    // log(+1.0) = 0
V_LOG_F32(0x7f800000) => 0x7f800000    // log(+INF) = +INF
```

**V\_RCP\_F32****418**

使用 IEEE 规则计算单精度浮点输入的对等值,并将结果存储到向量寄存器中。

```
D0.f32 = 1.0F / S0.f32
```

## 说明

1ULP 精度. 使用牛顿-拉弗森法和 2 个 FMA 操作时,精度与 < 0.5ULP 汇合。异常值被冲掉。

## 功能实例:

```
V_RCP_F32(0xff800000) => 0x80000000    // rcp(-INF) = -0
V_RCP_F32(0xc0000000) => 0xbf000000    // rcp(-2.0) = -0.5
V_RCP_F32(0x80000000) => 0xff800000    // rcp(-0.0) = -INF
V_RCP_F32(0x00000000) => 0x7f800000    // rcp(+0.0) = +INF
V_RCP_F32(0x7f800000) => 0x00000000    // rcp(+INF) = +0
```

**V\_RCP\_IFLAG\_F32****419**

以适合整数分割的方式计算向量浮点输入的对等值,并将结果存储到向量寄存器中。此 opcode 意在作为整数分割宏的一部分使用。

```
D0.f32 = 1.0F / S0.f32;
// 只能提高整数 DIV_BY_ZERO 例外
```

## 说明

可以提出整数 DIV\_BY\_ZERO 例外,但不能提出浮点例外。由编译器在整数对等宏中使用,其序列如下(取决于签名或无符号操作)。

无符号用途:

CVT\_F32\_U32  
RCP\_IFLAG\_F32



MUL\_F32 (2\*\*32 - 1)  
CVT\_U32\_F32

有符号数的用法:  
CVT\_F32\_I32  
RCP\_IFLAG\_F32  
MUL\_F32 (2\*\*31 -  
1) CVT\_I32\_F32

## V\_RSQ\_F32

420

使用 IEEE 规则计算单精度浮点输入的平方根对等,并将结果存储到向量寄存器中.

```
D0.f32 = 1.0F / sqrt(S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_RSQ_F32(0xff800000) => 0xffc00000    // rsq(-INF) = NAN
V_RSQ_F32(0x80000000) => 0xff800000    // rsq(-0.0) = -INF
V_RSQ_F32(0x00000000) => 0x7f800000    // rsq(+0.0) = +INF
V_RSQ_F32(0x40800000) => 0x3f000000    // rsq(+4.0) = +0.5
V_RSQ_F32(0x7f800000) => 0x00000000    // rsq(+INF) = +0
```

## V\_RCP\_F64

421

使用 IEEE 规则计算双精度浮点输入的对等值,并将结果存储到向量寄存器中.

```
D0.f64 = 1.0 / S0.f64
```

说明

这个 opcode 具有(2\*\*29)ULP 的精度,并支持异常.

## V\_RSQ\_F64

422

使用 IEEE 规则计算双精度浮点输入的平方根的对等值,并将结果存储到向量寄存器中.

```
D0.f64 = 1.0 / sqrt(S0.f64)
```

说明

这个 opcode 具有(2\*\*29)ULP 的精度,并支持异常.

## V\_SQRT\_F32

423

使用 IEEE 规则计算单精度浮点输入的平方根,并将结果存储到向量寄存器中.

```
D0.f32 = sqrt(S0.f32)
```

说明

1ULP 精度,异常值被冲洗.

功能实例:

```
V_SQRT_F32(0xff800000) => 0xffc00000    // sqrt(-INF) = NAN
V_SQRT_F32(0x80000000) => 0x80000000    // sqrt(-0.0) = -0
V_SQRT_F32(0x00000000) => 0x00000000    // sqrt(+0.0) = +0
V_SQRT_F32(0x40800000) => 0x40000000    // sqrt(+4.0) = +2.0
V_SQRT_F32(0x7f800000) => 0x7f800000    // sqrt(+INF) = +INF
```

## V\_SQRT\_F64

424

使用 IEEE 规则计算双精度浮点输入的平方根,并将结果存储到向量寄存器中.

```
D0.f64 = sqrt(S0.f64)
```

说明

这个 opcode 具有(2\*\*29)ULP 的精度,并支持异常.

## V\_SIN\_F32

425

使用 IEEE 规则计算单精度浮点值的三角正弦, 并将结果存储到向量寄存器中。操作通过将向量输入缩放 2 PI 来计算。

```
D0.f32 = sin(S0.f32 * 32'F(PI * 2.0))
```

#### 说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_SIN_F32(0xff800000) => 0xffc00000    // sin(-INF) = NAN
V_SIN_F32(0xff7fffff) => 0x00000000    // -MaxFloat, finite
V_SIN_F32(0x80000000) => 0x80000000    // sin(-0.0) = -0
V_SIN_F32(0x3e800000) => 0x3f800000    // sin(0.25) = 1
V_SIN_F32(0x7f800000) => 0xffc00000    // sin(+INF) = NAN
```

## V\_COS\_F32

426

使用 IEEE 规则计算单精度浮点值的三角余弦, 并将结果存储到向量寄存器中。 操作通过将向量输入缩放 2 PI 来计算。

```
D0.f32 = cos(S0.f32 * 32'F(PI * 2.0))
```

#### 说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_COS_F32(0xff800000) => 0xffc00000    // cos(-INF) = NAN
V_COS_F32(0xff7fffff) => 0x3f800000    // -MaxFloat, finite
V_COS_F32(0x80000000) => 0x3f800000    // cos(-0.0) = 1
V_COS_F32(0x3e800000) => 0x00000000    // cos(0.25) = 0
V_COS_F32(0x7f800000) => 0xffc00000    // cos(+INF) = NAN
```

## V\_NOT\_B32

427

计算向量输入时的比特按位取反, 并将结果存储到向量寄存器中。

```
D0.u32 = ~S0.u32
```

#### 说明

不支持输入和输出修改符。

**V\_BFREV\_B32****428**

反转向量输入中的位数顺序,并将结果存储到向量寄存器中。

```
D0.u32[31 : 0] = S0.u32[0 : 31]
```

说明

不支持输入和输出修改符。

**V\_FFBH\_U32****429**

统计向量输入的前导 0 的数量, 即从最高位起到第一个 1 之前连续的 0, 并将结果写入向量寄存器。若没有 1, 则写入 -1。

```
D0.i32 = -1;
// 如果找不到, 则设定
for i in 0 : 31 do
    // 从 MSB 搜索
    if S0.u32[31 - i] == 1'u then
        D0.i32 = i;
        break
    endif
endfor
```

说明

功能实例:

```
V_FFBH_U32(0x00000000) => 0xffffffff
V_FFBH_U32(0x800000ff) => 0
V_FFBH_U32(0x100000ff) => 3
V_FFBH_U32(0x0000ffff) => 16
V_FFBH_U32(0x00000001) => 31
```

**V\_FFBL\_B32****430**

统计向量输入的尾随 0 的数量, 即从最低位起到第一个 1 之前连续的 0, 并将结果写入向量寄存器。若输入中没有 1, 则写入 -1。

```
D0.i32 = -1;
// 如果找不到, 则设定
for i in 0 : 31 do
```

```
// 从 LSB 搜索
if S0.u32[i] == 1'1U then
    D0.i32 = i;
    break
endif
endfor
```

说明

功能实例:

```
V_FFBL_B32(0x00000000) => 0xffffffff
V_FFBL_B32(0xff000001) => 0
V_FFBL_B32(0xff000008) => 3
V_FFBL_B32(0xffff0000) => 16
V_FFBL_B32(0x80000000) => 31
```

## V\_FFBH\_I32

431

统计向量输入中从最高位起、连续与符号位相同的位数，并将结果写入向量寄存器。若所有输入位都相同，则写入 -1。

```
D0.i32 = -1;
// 如果所有位码都是相同的, 设定
for i in 1 : 31 do
    // 从 MSB 搜索
    if S0.i32[31 - i] != S0.i32[31] then
        D0.i32 = i;
        break
    endif
endfor
```

说明

功能实例:

```
V_FFBH_I32(0x00000000) => 0xffffffff
V_FFBH_I32(0x40000000) => 1
V_FFBH_I32(0x80000000) => 1
V_FFBH_I32(0x0fffffff) => 4
V_FFBH_I32(0xffff0000) => 16
V_FFBH_I32(0xffffffffe) => 31
V_FFBH_I32(0xfffffffff) => 0xffffffff
```

## V\_FREXP\_EXP\_I32\_F64

432

提取双精度浮点输入的表示器,并将结果作为签名的 32 位整数存储到向量寄存器中。

```
if ((S0.f64 == +INF) || (S0.f64 == -INF) || isNaN(S0.f64)) then
    D0.i32 = 0
else
    D0.i32 = exponent(S0.f64) - 1023 + 1
endif
```

#### 说明

此操作满足不变量的  $S0.f64 = \text{significand} * (2^{**} \text{exponent})$ 。另参见 `V_FREXP_MANT_F64`, 它返回了 `significand`。更多信息请参见 C 库函数 `frexp()`。

---

### V\_FREXP\_MANT\_F64

433

提取双精度浮点数输入的 32 位二进制符号或 mantissa,并将结果作为双精度浮点数存储到向量寄存器中。

```
if ((S0.f64 == +INF) || (S0.f64 == -INF) || isNaN(S0.f64)) then
    D0.f64 = S0.f64
else
    D0.f64 = mantissa(S0.f64)
endif
```

#### 说明

此操作满足变化无常的  $S0.f64 = \text{significand} * (2^{**} \text{exponent})$ 。结果范围在正常情况下为  $(-1.0, -0.5) [0.5, 1.0)$ 。另见 `V_FREXP_EXP_I32_F64`, 返回整数符号。请参见 C 库函数。

更多信息请考虑 `frexp()`。

---

### V\_FRACT\_F64

434

计算双精度浮点输入的小数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f64 = S0.f64 + -floor(S0.f64)
```

#### 说明

精度为 0.5 ULP, 允许非规格化输入。

本指令旨在符合 DX 对 `fract` 的定义, 其行为相当于整数取模的扩展。其他领域对 `fract()` 的定义可能不同。例如, DX 中  $\text{fract}(-1.2) = 0.8$ 。

遵循舍入模式，结果限制在 0x3fefffffffffffff 以内。

---

### V\_FREXP\_EXP\_I32\_F32

435

提取单精度浮点输入的缩写,并将结果作为签名的 32 位整数存储到向量寄存器中。

```
if ((64'F(S0.f32) == +INF) || (64'F(S0.f32) == -INF) || isNaN(64'F(S0.f32))) then
    D0.i32 = 0
else
    D0.i32 = exponent(S0.f32) - 127 + 1
endif
```

说明

此操作满足了不变量的  $S0.f32 = \text{significand} * (2^{**} \text{exponent})$ 。另参见 V\_FREXP\_MANT\_F32，它返回了 significand。更多信息请参见 C 库函数 frexp()。

---

### V\_FREXP\_MANT\_F32

436

提取单精度浮点数输入的二进制符号或 mantissa,并将结果作为单精度浮点数存储到向量寄存器中。

```
if ((64'F(S0.f32) == +INF) || (64'F(S0.f32) == -INF) || isNaN(64'F(S0.f32))) then
    D0.f32 = S0.f32
else
    D0.f32 = mantissa(S0.f32)
endif
```

说明

此操作满足变化无常的  $S0.f32 = \text{significand} * (2^{**} \text{exponent})$ 。结果范围在正常情况下为 (-1.0,-0.5) [0.5,1.0)。另见 V\_FREXP\_EXP\_I32\_F32, 返回整数符。请参见 C 库函数。更多信息请考虑()。

---

### V\_CLREXCP

437

在向量 ALU 中清除此波的例外状态。

---

### V\_MOV\_B64

440

将数据从 64 位向量输入移到向量寄存器中。

```
D0.b64 = S0.b64
```

#### 说明

如果 S0.u64 是一个 64 位浮点值,则浮点修改器对本指令有效。本指令适用于否定或取浮点值的绝对值。

---

### V\_CVT\_F16\_U16

441

从无符号的 16 位整数输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = u16_to_f16(S0.u16)
```

#### 说明

0.5ULP 精度,支持非正常,四舍五入,例外旗和饱和。

---

### V\_CVT\_F16\_I16

442

从签名的 16 位整数输入转换为半精度浮点值,并将结果存储到向量寄存器中。

```
D0.f16 = i16_to_f16(S0.i16)
```

#### 说明

0.5ULP 精度,支持非正常,四舍五入,例外旗和饱和。

---

### V\_CVT\_U16\_F16

443

从半精度浮点输入转换为无符号的 16 位整数值,并将结果存储到向量寄存器中。

```
D0.u16 = f16_to_u16(S0.f16)
```

#### 说明

1ULP 精度,支持四舍五入,例外旗帜和饱和度。FP16 异常值被接受,转换时采用截断方式。



INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

## V\_CVT\_I16\_F16

444

从半精度的浮点输入转换为签名的 16 位整数,并将结果存储到向量寄存器中.

```
D0.i16 = f16_to_i16(S0.f16)
```

说明

1ULP 精度,支持四舍五入,例外旗帜和饱和度. FP16 异常值被接受,转换时采用截断方式.

INEXACT 例外的生成由 CLAMP 位控制. INEXACT 例外被启用用于此转换 if CLAMP == 1.

## V\_RCP\_F16

445

使用 IEEE 规则计算半精度浮点输入的对等值,并将结果存储到向量寄存器中。

```
D0.f16 = 16'1.0 / S0.f16
```

说明

1ULP 精度.

功能实例:

```
V_RCP_F16(0xfc00) => 0x8000    // rcp(-INF) = -0
V_RCP_F16(0xc000) => 0xb800    // rcp(-2.0) = -0.5
V_RCP_F16(0x8000) => 0xfc00    // rcp(-0.0) = -INF
V_RCP_F16(0x0000) => 0x7c00    // rcp(+0.0) = +INF
V_RCP_F16(0x7c00) => 0x0000    // rcp(+INF) = +0
```

## V\_SQRT\_F16

446

使用 IEEE 规则计算半精度浮点输入的方根,并将结果存储到向量寄存器中.

```
D0.f16 = sqrt(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_SQRT_F16(0xfc00) => 0xfe00    // sqrt(-INF) = NAN
V_SQRT_F16(0x8000) => 0x8000    // sqrt(-0.0) = -0
V_SQRT_F16(0x0000) => 0x0000    // sqrt(+0.0) = +0
V_SQRT_F16(0x4400) => 0x4000    // sqrt(+4.0) = +2.0
V_SQRT_F16(0x7c00) => 0x7c00    // sqrt(+INF) = +INF
```

## V\_RSQ\_F16

447

使用 IEEE 规则计算半精度浮点输入的平方根对等值,并将结果存储到向量寄存器中.

```
D0.f16 = 16'1.0 / sqrt(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_RSQ_F16(0xfc00) => 0xfe00    // rsq(-INF) = NAN
V_RSQ_F16(0x8000) => 0xfc00    // rsq(-0.0) = -INF
V_RSQ_F16(0x0000) => 0x7c00    // rsq(+0.0) = +INF
V_RSQ_F16(0x4400) => 0x3800    // rsq(+4.0) = +0.5
V_RSQ_F16(0x7c00) => 0x0000    // rsq(+INF) = +0
```

## V\_LOG\_F16

448

计算半精度浮点输入的基数 2 对数,并将结果存储到向量寄存器中.

```
D0.f16 = log2(S0.f16)
```

说明

1ULP 精确度,异常值得到支持.

功能实例:

```
V_LOG_F16(0xfc00) => 0xfe00    // log(-INF) = NAN
V_LOG_F16(0xbc00) => 0xfe00    // log(-1.0) = NAN
V_LOG_F16(0x8000) => 0xfc00    // log(-0.0) = -INF
V_LOG_F16(0x0000) => 0xfc00    // log(+0.0) = -INF
V_LOG_F16(0x3c00) => 0x0000    // log(+1.0) = 0
V_LOG_F16(0x7c00) => 0x7c00    // log(+INF) = +INF
```

## V\_EXP\_F16

449

计算 2 升至半精度浮点输入的功率,并将结果存储到向量寄存器中。

```
D0.f16 = pow(16'2.0, S0.f16)
```

说明

1ULP 精确度,异常值得到支持。

功能实例:

```
V_EXP_F16(0xfc00) => 0x0000    // exp(-INF) = 0
V_EXP_F16(0x8000) => 0x3c00    // exp(-0.0) = 1
V_EXP_F16(0x7c00) => 0x7c00    // exp(+INF) = +INF
```

## V\_FREXP\_MANT\_F16

450

提取半精度浮点数输入的二进制符号或 mantissa,并将半精度浮点数结果存储到向量寄存器中。

```
if ((64'F(S0.f16) == +INF) || (64'F(S0.f16) == -INF) || isNaN(64'F(S0.f16))) then
    D0.f16 = S0.f16
else
    D0.f16 = mantissa(S0.f16)
endif
```

说明

此操作满足了变化无常的  $S0.f16 = \text{significand} * (2 ** \text{exponent})$ 。结果范围在正常情况下为  $(-1.0, -0.5) [0.5, 1.0)$ 。另见 `V_FREXP_EXP_I16_F16`, 返回整数符。请参见 C 库函数。

更多信息请考虑()。

**V\_FREXP\_EXP\_I16\_F16****451**

解析半精度浮点输入的缩写,并将结果作为签名的 16 位整数存储到向量寄存器中。

```
if ((64'F(S0.f16) == +INF) || (64'F(S0.f16) == -INF) || isNaN(64'F(S0.f16))) then
    D0.i16 = 16'0
else
    D0.i16 = 16'I(exponent(S0.f16) - 15 + 1)
endif
```

说明

此操作满足了不变量的  $S0.f16 = \text{significand} * (2 ** \text{exponent})$ 。另参见 V\_FREXP\_MANT\_F16, 它返回了 significand。更多信息请参见 C 库函数 frexp()。

**V\_FLOOR\_F16****452**

将半精度浮点输入向下绕到以前的整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16);
if ((S0.f16 < 16'0.0) && (S0.f16 != D0.f16)) then
    D0.f16 += -16'1.0
endif
```

**V\_CEIL\_F16****453**

将半精度浮点输入绕到下一个整数,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16);
if ((S0.f16 > 16'0.0) && (S0.f16 != D0.f16)) then
    D0.f16 += 16'1.0
endif
```

**V\_TRUNC\_F16****454**

用朝零舍入语义计算半精度浮点输入的整数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = trunc(S0.f16)
```

**V\_RNDNE\_F16****455**

将半精度浮点输入舍入到最近整数，恰好位于两个整数中间时取偶数；以浮点格式将结果写入向量寄存器。

```
D0.f16 = floor(S0.f16 + 16'0.5);
if (isEven(64'F(floor(S0.f16))) && (fract(S0.f16) == 16'0.5)) then
    D0.f16 -= 16'1.0
endif
```

**V\_FRACT\_F16****456**

计算半精度浮点输入的小数部分,并将结果以浮点格式存储到向量寄存器中。

```
D0.f16 = S0.f16 + -floor(S0.f16)
```

说明

精度为 0.5 ULP，允许非规格化输入。

本指令旨在符合 DX 对 fract 的定义，其行为相当于整数取模的扩展。其他领域对 fract() 的定义可能不同。例如，DX 中  $\text{fract}(-1.2) = 0.8$ 。

**V\_SIN\_F16****457**

使用 IEEE 规则计算半精度浮点值的三角正弦值, 并将结果存储到向量寄存器中。操作通过将向量输入缩放 2 PI 来计算。

```
D0.f16 = sin(S0.f16 * 16'F(PI * 2.0))
```

说明

异常值被支持。全程输入被支持。

功能实例:

```
V_SIN_F16(0xfc00) => 0xfe00    // sin(-INF) = NAN
V_SIN_F16(0xfbff)  => 0x0000    // 最负限 FP16
V_SIN_F16(0x8000)  => 0x8000    // sin(-0.0) = -0
V_SIN_F16(0x3400)  => 0x3c00    // sin(0.25) = 1
```

```
V_SIN_F16(0x7bff) => 0x0000    // 最积极的有限 FP16
V_SIN_F16(0x7c00) => 0xfe00    // sin(+INF) = NAN
```

## V\_COS\_F16

458

使用 IEEE 规则计算半精度浮点值的三角余弦, 并将结果存储到向量寄存器中。操作通过将向量输入缩放 2 PI 来计算。

```
D0.f16 = cos(S0.f16 * 16'F(PI * 2.0))
```

说明

异常值被支持。 全程输入被支持。

功能实例:

```
V_COS_F16(0xfc00) => 0xfe00    // cos(-INF) = NAN
V_COS_F16(0xfbff) => 0x3c00    // 最负限 FP16
V_COS_F16(0x8000) => 0x3c00    // cos(-0.0) = 1
V_COS_F16(0x3400) => 0x0000    // cos(0.25) = 0
V_COS_F16(0x7bff) => 0x3c00    // 最积极的有限 FP16
V_COS_F16(0x7c00) => 0xfe00    // cos(+INF) = NAN
```

## V\_CVT\_NORM\_I16\_F16

461

从半精度浮点输入转换成签名的正态短句, 并将结果存储到向量寄存器中。

```
D0.i16 = f16_to_snorm(S0.f16)
```

说明

0.5ULP 精度, 支持四舍五入, 例外旗帜和饱和度, 异常态得到支持。

## V\_CVT\_NORM\_U16\_F16

462

从半精度浮点输入转换为无符号的普通短数, 并将结果存储到向量寄存器中。

```
D0.u16 = f16_to_unorm(S0.f16)
```

## 说明

0.5ULP 精度,支持四舍五入,例外旗帜和饱和度,异常态得到支持.

**V\_SAT\_PK\_U8\_I16****463**

在 2 个签名的 16 位整数输入中,将每个输入在一个无符号的 8 位整数范围内饱和,将所产生的值打包成一个包的 16 位值,并将结果存储到向量寄存器中.

```
SAT8 = lambda(n) (
    if n <= 16'0 then
        return 8'0U
    elsif n >= 16'255 then
        return 8'255U
    else
        return n[7 : 0].u8
    endif);
tmp = 16'0;
tmp[7 : 0].u8 = SAT8(S0[15 : 0].i16);
tmp[15 : 8].u8 = SAT8(S0[31 : 16].i16);
D0.b16 = tmp.b16
```

## 说明

用于 4x16bit 数据包装为 4x8bit 数据.

**V\_SWAP\_B32****465**

在两个向量登记簿中交换值。

```
tmp = D0.b32;
D0.b32 = S0.b32;
S0.b32 = tmp
```

## 说明

不支持输入和输出修改器;这是一个未打字的操作.

**V\_ACCVGPR\_MOV\_B32****466**

将数据从一个累积器寄存器移动到另一个累积器寄存器。

**V\_CVT\_F32\_FP8****468**

从 FP8 浮动输入转换为单精度浮点值,并将结果存储到向量寄存器中.

```
if SDWA_SRC0_SEL == BYTE1.b3 then
    D0.f32 = fp8_to_f32(S0[15 : 8].fp8)
elsif SDWA_SRC0_SEL == BYTE2.b3 then
    D0.f32 = fp8_to_f32(S0[23 : 16].fp8)
elsif SDWA_SRC0_SEL == BYTE3.b3 then
    D0.f32 = fp8_to_f32(S0[31 : 24].fp8)
else
    // BYTE0 隐含
    D0.f32 = fp8_to_f32(S0[7 : 0].fp8)
endif
```

说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个字节被转换,只有 BYTE 选择的 SRC0\_SEL 是合法的,如果此指令没有在 SDWA 编码,那么 BYTE0 就是隐含的.

## V\_CVT\_F32\_BF8

469

从 BF8 浮点数转换为单精度浮点数,并将结果存储到向量寄存器中.

```
if SDWA_SRC0_SEL == BYTE1.b3 then
    D0.f32 = bf8_to_f32(S0[15 : 8].bf8)
elsif SDWA_SRC0_SEL == BYTE2.b3 then
    D0.f32 = bf8_to_f32(S0[23 : 16].bf8)
elsif SDWA_SRC0_SEL == BYTE3.b3 then
    D0.f32 = bf8_to_f32(S0[31 : 24].bf8)
else
    // BYTE0 隐含
    D0.f32 = bf8_to_f32(S0[7 : 0].bf8)
endif
```

说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个字节被转换,只有 BYTE 选择的 SRC0\_SEL 是合法的,如果此指令没有在 SDWA 编码,那么 BYTE0 就是隐含的.

## V\_CVT\_PK\_F32\_FP8

470

从包装的 2 组件 FP8 浮点数转换成包装的单精度浮点数,并将结果存储到向量寄存器中.

```
tmp = SDWA_SRC0_SEL[1 : 0] == WORD1.b2 ? S0[31 : 16] : S0[15 : 0];
D0[31 : 0].f32 = fp8_to_f32(tmp[7 : 0].fp8);
D0[63 : 32].f32 = fp8_to_f32(tmp[15 : 8].fp8)
```



## 说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个单词被转换,只有 WORD 选择的 SRC0\_SEL 是合法的,如果这个指令没有在 SDWA 编码,那么 WORD0 就是隐含的.

**V\_CVT\_PK\_F32\_BF8****471**

从包装的 2 组件 BF8 浮点输入转换为包装的单精度浮点值,并将结果存储到向量寄存器中.

```
tmp = SDWA_SRC0_SEL[1 : 0] == WORD1.b2 ? S0[31 : 16] : S0[15 : 0];
D0[31 : 0].f32 = bf8_to_f32(tmp[7 : 0].bf8);
D0[63 : 32].f32 = bf8_to_f32(tmp[15 : 8].bf8)
```

## 说明

SDWA 编码允许 SRC0\_SEL 控制 S0 的哪个单词被转换,只有 WORD 选择的 SRC0\_SEL 是合法的,如果这个指令没有在 SDWA 编码,那么 WORD0 就是隐含的.

**V\_CNDMASK\_B32****256**

根据每行条件码复制两个输入中的一个数据,并将结果存储到向量寄存器中.

```
D0.u32 = VCC.u64[laneId] ? S1.u32 : S0.u32
```

## 说明

在 VOP3 中,VCC 源可能是 S2 中指定的标量 GPR.

如果 S0 和 S1 是 32 位浮点值,则浮点修改符对本指令有效。本指令适用于否定或占用浮点值的绝对值。

**V\_ADD\_F32****257**

添加两个浮点输入并将结果存储到向量寄存器中.

```
D0.f32 = S0.f32 + S1.f32
```

## 说明

0.5ULP 精度,异常态得到支持.

**V\_SUB\_F32****258**

从第一个输入中减掉第二个浮点输入,并将结果存储到向量寄存器中.

$$D0.f32 = S0.f32 - S1.f32$$

说明

0.5ULP 精度,异常态得到支持.

**V\_SUBREV\_F32****259**

从第二个输入中减掉第一个浮点输入,并将结果存储到向量寄存器中.

$$D0.f32 = S1.f32 - S0.f32$$

说明

0.5ULP 精度,异常态得到支持.

**V\_FMAC\_F64****260**

乘以两个浮点输入,利用融合乘法将结果累积到目的地寄存器中.

$$D0.f64 = \text{fma}(S0.f64, S1.f64, D0.f64)$$
**V\_MUL\_F32****261**

乘以两个浮点输入并将结果存储到向量寄存器中.

$$D0.f32 = S0.f32 * S1.f32$$

说明

0.5ULP 精度,异常态得到支持.

**V\_MUL\_I32\_I24****262**

乘以两个签名的 24 位整数输入,并将结果作为签名的 32 位整数存储到向量寄存器中。

```
D0.i32 = 32'I(S0.i24) * 32'I(S1.i24)
```

说明

这个 opcode 预计会和基本的单精度的 opcode 一样高效,因为它使用了单精度的浮点乘数。另见 V\_MUL\_HI\_I32\_I24。

---

### V\_MUL\_HI\_I32\_I24

**263**

乘以两个签名的 24 位整数输入,并将结果的高 32 位作为签名的 32 位整数存储到向量寄存器中。

```
D0.i32 = 32'I((64'I(S0.i24) * 64'I(S1.i24)) >> 32U)
```

说明

另见 V\_MUL\_I32\_I24。

---

### V\_MUL\_U32\_U24

**264**

将两个无符号的 24 位整数输入数乘以,并将结果作为无符号的 32 位整数存储到向量寄存器中。

```
D0.u32 = 32'U(S0.u24) * 32'U(S1.u24)
```

说明

这个 opcode 预计会和基本的单精度的 opcode 一样高效,因为它使用了单精度的浮点乘数。另见 V\_MUL\_HI\_U32\_U24。

---

### V\_MUL\_HI\_U32\_U24

**265**

乘以两个无符号的 24 位整数输入,并将结果的高 32 位作为无符号的 32 位整数存储到向量寄存器中。

```
D0.u32 = 32'U((64'U(S0.u24) * 64'U(S1.u24)) >> 32U)
```

说明

另见 V\_MUL\_U32\_U24.

## V\_MIN\_F32

266

选择至少两个单精度浮点输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S0.f32)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S1.f32)))
elsif isNaN(64'F(S0.f32)) then
    D0.f32 = S1.f32
elsif isNaN(64'F(S1.f32)) then
    D0.f32 = S0.f32
elsif ((64'F(S0.f32) == +0.0) && (64'F(S1.f32) == -0.0)) then
    D0.f32 = S1.f32
elsif ((64'F(S0.f32) == -0.0) && (64'F(S1.f32) == +0.0)) then
    D0.f32 = S0.f32
else
    // 注意:这里没有 IEEE 案件,像 V_MAX_F32 一样.
    D0.f32 = S0.f32 < S1.f32 ? S0.f32 : S1.f32
endif
```

## V\_MAX\_F32

267

选择两个单精度浮点输入的最大值,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S0.f32)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f32))) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S1.f32)))
elsif isNaN(64'F(S0.f32)) then
    D0.f32 = S1.f32
elsif isNaN(64'F(S1.f32)) then
    D0.f32 = S0.f32
elsif ((64'F(S0.f32) == +0.0) && (64'F(S1.f32) == -0.0)) then
    D0.f32 = S0.f32
elsif ((64'F(S0.f32) == -0.0) && (64'F(S1.f32) == +0.0)) then
    D0.f32 = S1.f32
elsif WAVE_MODE.IEEE then
    D0.f32 = S0.f32 >= S1.f32 ? S0.f32 : S1.f32
else
    D0.f32 = S0.f32 > S1.f32 ? S0.f32 : S1.f32
endif
```

## V\_MIN\_I32

268

选择最少两个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i32 = S0.i32 < S1.i32 ? S0.i32 : S1.i32
```

### V\_MAX\_I32

269

选择最多两个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i32 = S0.i32 >= S1.i32 ? S0.i32 : S1.i32
```

### V\_MIN\_U32

270

选择两个无符号的 32 位整数输入的最小值,并将所选值存储到向量寄存器中。

```
D0.u32 = S0.u32 < S1.u32 ? S0.u32 : S1.u32
```

### V\_MAX\_U32

271

选择两个无符号的 32 位整数输入的最大值,并将选定的数值存储到向量寄存器中。

```
D0.u32 = S0.u32 >= S1.u32 ? S0.u32 : S1.u32
```

### V\_LSHRREV\_B32

272

鉴于第一个向量输入中的位移计数,计算第二个向量输入的逻辑位移右侧,并将结果存储到一个向量寄存器中。

```
D0.u32 = (S1.u32 >> S0[4 : 0].u32)
```

### V\_ASHRREV\_I32

273

鉴于第一个向量输入中的位移计数,计算第二个向量输入的算术右移(保留符号位),并将结果存储到向量寄存器中。

```
D0.i32 = (S1.i32 >> S0[4 : 0].u32)
```

### V\_LSHLREV\_B32

274

鉴于第一个向量输入中的位移计数,计算第二个向量输入中左侧的逻辑位移,并将结果存储到一个向量寄存器中。

```
D0.u32 = (S1.u32 << S0[4 : 0].u32)
```

### V\_AND\_B32

275

在两个向量输入上以位法计算,并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 & S1.u32)
```

说明

不支持输入和输出修改符。

### V\_OR\_B32

276

在两个向量输入上按位计算 OR 并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 | S1.u32)
```

说明

不支持输入和输出修改符。

### V\_XOR\_B32

277

在两个向量输入上计算比特 XOR,并将结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 ^ S1.u32)
```

说明

不支持输入和输出修改符。

**V\_ADD\_CO\_U32****281**

增加两个未署名的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中.

```
tmp = 64'U(S0.u32) + 64'U(S1.u32);
VCC.u64[laneId] = tmp >= 0x100000000ULL ? 1'1U : 1'0U;
// VCC 是 V_ADDCO_U32 的 UNSSIGED 外溢/运出.
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

**V\_SUB\_CO\_U32****282**

从第一个输入中减掉第二个无符号的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中.

```
tmp = S0.u32 - S1.u32;
VCC.u64[laneId] = S1.u32 > S0.u32 ? 1'1U : 1'0U;
// VCC 是 V_SUBCO_U32 的 UNSSIGED 外溢/运出.
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

**V\_SUBREV\_CO\_U32****283**

从第二次输入中减掉第一个无符号的 32 位整数输入,将结果存储到向量寄存器中,并将运出掩码存储到标量寄存器中.

```
tmp = S1.u32 - S0.u32;
VCC.u64[laneId] = S0.u32 > S1.u32 ? 1'1U : 1'0U;
// VCC 是 V_SUBREV_CO_U32 的 UNSSIGED 外溢/运出.
D0.u32 = tmp.u32
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

### V\_ADDC\_CO\_U32

284

增加两个未署名的 32 位整数输入和一个随身携带的掩码的一点,将结果存储到向量寄存器中,并将随身携带的掩码存储到标量寄存器中.

```
tmp = 64'U(S0.u32) + 64'U(S1.u32) + VCC.u64[laneId].u64;  
VCC.u64[laneId] = tmp >= 0x100000000ULL ? 1'1U : 1'0U;  
// VCC 是 V_ADDC_CO_U32 的 UNSSIGED 外溢/运出.  
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

### V\_SUBB\_CO\_U32

285

从第一次输入中减掉第二个无符号的 32 位整数输入,从携带的掩码中减去一点,将结果存储到向量寄存器中,并将携带的掩码存储到标量寄存器中.

```
tmp = S0.u32 - S1.u32 - VCC.u64[laneId].u32;  
VCC.u64[laneId] = 64'U(S1.u32) + VCC.u64[laneId].u64 > 64'U(S0.u32) ? 1'1U : 1'0U;  
// VCC 是 V_SUBB_CO_U32 的 UNSSIGED 外溢/运出.  
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

### V\_SUBBREV\_CO\_U32

286

从第二次输入中减去第一个无符号的 32 位整数输入,从携带的掩码中减去一点,将结果存储到向量寄存器中,并将携带的掩码存储到标量寄存器中.

```
tmp = S1.u32 - S0.u32 - VCC.u64[laneId].u32;
```



```
VCC.u64[laneId] = 64'U(S0.u32) + VCC.u64[laneId].u64 > 64'U(S1.u32) ? 1'1U : 1'0U;
// VCC 是 V_SUBB_CO_U32 的 UNSSIGED 外溢/运出.
D0.u32 = tmp.u32
```

说明

在 VOP3 中,VCC 目的地可能是任意的 SGPR-pair,VCC 来源于 S2.u 的 SGPR-pair.

支持饱和(无符号的 32 位整数域).

## V\_ADD\_F16

287

添加两个浮点输入并将结果存储到向量寄存器中.

```
D0.f16 = S0.f16 + S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

## V\_SUB\_F16

288

从第一个输入中减掉第二个浮点输入,并将结果存储到向量寄存器中.

```
D0.f16 = S0.f16 - S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

## V\_SUBREV\_F16

289

从第二个输入中减掉第一个浮点输入,并将结果存储到向量寄存器中.

```
D0.f16 = S1.f16 - S0.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

**V\_MUL\_F16****290**

乘以两个浮点输入并将结果存储到向量寄存器中。

```
D0.f16 = S0.f16 * S1.f16
```

说明

0.5ULP 精度. 支持异常,舍入模式,例外旗帜和饱和度.

**V\_MAC\_F16****291**

乘以两个浮点输入并累积结果到目的寄存器中. 为 OPSEL 执行 IEEE 规则和非标准规则.

```
tmp = S0.f16 * S1.f16 + D0.f16;  
if OPSEL.u4[3] then  
    D0 = { tmp.f16, D0[15 : 0] }  
else  
    D0 = { 16'0, tmp.f16 }  
endif
```

说明

支持舍入模式,例外旗帜,饱和度.

**V\_ADD\_U16****294**

添加两个无符号的 16 位整数输入, 并将结果存储到向量寄存器中。 没有随身携带或执行支持。

```
D0.u16 = S0.u16 + S1.u16
```

说明

支持饱和(无符号的 16 位整数域).

**V\_SUB\_U16****295**

从第一个输入中减去第二个无符号的 16 位整数输入, 并将结果存储到向量寄存器中。 没有随身携带或执行支持。

$$D0.u16 = S0.u16 - S1.u16$$

说明

支持饱和(无符号的 16 位整数域).

### V\_SUBREV\_U16

**296**

从第二次输入中减去第一个无符号的 16 位整数输入,并将结果存储到向量寄存器中。没有随身携带或执行支持。

$$D0.u16 = S1.u16 - S0.u16$$

说明

支持饱和(无符号的 16 位整数域).

### V\_MUL\_LO\_U16

**297**

将两个无符号的 16 位整数输入数乘以,并将结果的低位数存储到向量寄存器中。

$$D0.u16 = S0.u16 * S1.u16$$

说明

支持饱和(无符号的 16 位整数域).

### V\_LSHLREV\_B16

**298**

鉴于第一个向量输入中的位移计数,计算第二个向量输入中左侧的逻辑位移,并将结果存储到一个向量寄存器中。

$$D0.u16 = (S1.u16 << S0[3 : 0].u32)$$

### V\_LSHRREV\_B16

**299**

鉴于第一个向量输入中的位移计数,计算第二个向量输入的逻辑位移右侧,并将结果存储到一个向量寄存器中。

```
D0.u16 = (S1.u16 >> S0[3 : 0].u32)
```

## V\_ASHRREV\_I16

300

鉴于第一个向量输入中的位移计数,计算第二个向量输入的算术右移(保留符号位),并将结果存储到向量寄存器中。

```
D0.i16 = (S1.i16 >> S0[3 : 0].u32)
```

## V\_MAX\_F16

301

选择最多两个半精度浮点数输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S0.f16)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S1.f16)))
elsif isNaN(64'F(S0.f16)) then
    D0.f16 = S1.f16
elsif isNaN(64'F(S1.f16)) then
    D0.f16 = S0.f16
elsif ((64'F(S0.f16) == +0.0) && (64'F(S1.f16) == -0.0)) then
    D0.f16 = S0.f16
elsif ((64'F(S0.f16) == -0.0) && (64'F(S1.f16) == +0.0)) then
    D0.f16 = S1.f16
elsif WAVE_MODE.IEEE then
    D0.f16 = S0.f16 >= S1.f16 ? S0.f16 : S1.f16
else
    D0.f16 = S0.f16 > S1.f16 ? S0.f16 : S1.f16
endif
```

说明

IEEE 符合. 支持异常,舍入模式,例外旗帜,饱和度.

## V\_MIN\_F16

302

选择至少两个半精度浮点输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(64'F(S0.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S0.f16)))
elsif (WAVE_MODE.IEEE && isSignalNaN(64'F(S1.f16))) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S1.f16)))
elsif isNaN(64'F(S0.f16)) then
```

```

    D0.f16 = S1.f16
elseif isNaN(64'F(S1.f16)) then
    D0.f16 = S0.f16
elseif ((64'F(S0.f16) == +0.0) && (64'F(S1.f16) == -0.0)) then
    D0.f16 = S1.f16
elseif ((64'F(S0.f16) == -0.0) && (64'F(S1.f16) == +0.0)) then
    D0.f16 = S0.f16
else
    // 注意:这里没有 IEEE 案件,像 V_MAX_F16 一样.
    D0.f16 = S0.f16 < S1.f16 ? S0.f16 : S1.f16
endif

```

说明

IEEE 符合. 支持异常,舍入模式,例外旗帜,饱和度.

## V\_MAX\_U16

303

选择两个无符号的 16 位整数输入的最大值,并将所选值存储到向量寄存器中。

```
D0.u16 = S0.u16 >= S1.u16 ? S0.u16 : S1.u16
```

## V\_MAX\_I16

304

选择最多两个签名的 16 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i16 = S0.i16 >= S1.i16 ? S0.i16 : S1.i16
```

## V\_MIN\_U16

305

选择两个无符号的 16 位整数输入的最小值,并将所选值存储到向量寄存器中。

```
D0.u16 = S0.u16 < S1.u16 ? S0.u16 : S1.u16
```

## V\_MIN\_I16

306

选择至少两个签名的 16 位整数输入,并将选定的数值存储到向量寄存器中。

```
D0.i16 = S0.i16 < S1.i16 ? S0.i16 : S1.i16
```

**V\_LDEXP\_F16****307**

乘以第一个输入,一个浮点值,通过第二个输入中指定的 2 个整体功率,一个签名的整数值,并将浮点结果存储到向量寄存器中。

```
D0.f16 = S0.f16 * 16'F(2.0F ** 32'I(S1.i16))
```

说明

与 C 中的 `ldexp()` 函数相比。注意 S1 的格式为 f16, 因为浮点字元常数被解释为 16 比特值。

**V\_ADD\_U32****308**

添加两个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有随身携带或随身携带支持。

```
D0.u32 = S0.u32 + S1.u32
```

说明

支持饱和(无符号的 32 位整数域)。

**V\_SUB\_U32****309**

从第一个输入中减去第二个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有载入或载出支持。

```
D0.u32 = S0.u32 - S1.u32
```

说明

支持饱和(无符号的 32 位整数域)。

**V\_SUBREV\_U32****310**

从第二个输入中减去第一个无符号的 32 位整数输入,并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
D0.u32 = S1.u32 - S0.u32
```

## 说明

支持饱和(无符号的 32 位整数域).

## V\_DOT2C\_F32\_F16

311

在单精度浮点域中，计算两个打包的二维半精度浮点输入的点积，再与目标寄存器中的单精度浮点值累加。

```
tmp = D0.f32;
tmp += f16_to_f32(S0[15 : 0].f16) * f16_to_f32(S1[15 : 0].f16);
tmp += f16_to_f32(S0[31 : 16].f16) * f16_to_f32(S1[31 : 16].f16);
D0.f32 = tmp
```

## V\_DOT2C\_I32\_I16

312

计算两个打包的 2-D 的点产品在签名的 32 位整数域中签名的 16 位整数输入,并与签名的 32 位整数值在目的地寄存器中积累。

```
tmp = D0.i32;
tmp += i16_to_i32(S0[15 : 0].i16) * i16_to_i32(S1[15 : 0].i16);
tmp += i16_to_i32(S0[31 : 16].i16) * i16_to_i32(S1[31 : 16].i16);
D0.i32 = tmp
```

## V\_DOT4C\_I32\_I8

313

计算两个包装的 4-D 在签名的 32 位整数域中签名的 8 位整数输入的点积,并在目的地寄存器中与签名的 32 位整数值一起累积。

```
tmp = D0.i32;
tmp += i8_to_i32(S0[7 : 0].i8) * i8_to_i32(S1[7 : 0].i8);
tmp += i8_to_i32(S0[15 : 8].i8) * i8_to_i32(S1[15 : 8].i8);
tmp += i8_to_i32(S0[23 : 16].i8) * i8_to_i32(S1[23 : 16].i8);
tmp += i8_to_i32(S0[31 : 24].i8) * i8_to_i32(S1[31 : 24].i8);
D0.i32 = tmp
```

## V\_DOT8C\_I32\_I4

314

计算两个打包的 8-D 的点产品在签名的 32 位整数域中签名的 4 位整数输入,并与签名的 32 位整数值在目的地寄存器中积累。

```
tmp = D0.i32;
tmp += i4_to_i32(S0[3 : 0].i4) * i4_to_i32(S1[3 : 0].i4);
tmp += i4_to_i32(S0[7 : 4].i4) * i4_to_i32(S1[7 : 4].i4);
tmp += i4_to_i32(S0[11 : 8].i4) * i4_to_i32(S1[11 : 8].i4);
tmp += i4_to_i32(S0[15 : 12].i4) * i4_to_i32(S1[15 : 12].i4);
tmp += i4_to_i32(S0[19 : 16].i4) * i4_to_i32(S1[19 : 16].i4);
tmp += i4_to_i32(S0[23 : 20].i4) * i4_to_i32(S1[23 : 20].i4);
tmp += i4_to_i32(S0[27 : 24].i4) * i4_to_i32(S1[27 : 24].i4);
tmp += i4_to_i32(S0[31 : 28].i4) * i4_to_i32(S1[31 : 28].i4);
D0.i32 = tmp
```

## V\_FMAC\_F32

315

乘以两个浮点输入,利用融合乘法将结果累积到目的地寄存器中。

```
D0.f32 = fma(S0.f32, S1.f32, D0.f32)
```

## V\_PK\_FMAC\_F16

316

将两个包装的半精度浮点输入组件相乘,并使用融合乘法将结果累积到目的地寄存器中。

```
D0[15 : 0].f16 = fma(S0[15 : 0].f16, S1[15 : 0].f16, D0[15 : 0].f16);
D0[31 : 16].f16 = fma(S0[31 : 16].f16, S1[31 : 16].f16, D0[31 : 16].f16)
```

## V\_XNOR\_B32

317

在两个向量输入上计算比特 XNOR,并将结果存储到向量寄存器中。

```
D0.u32 = ~(S0.u32 ^ S1.u32)
```

说明

不支持输入和输出修改符。

## V\_MAD\_I32\_I24

450



在有符号的 32 位整数域中乘以 2 个有符号的 24 位整数输入,从第三个输入中添加一个有符号的 32 位整数值,并将结果作为有符号的 32 位整数存储到向量寄存器中。

```
D0.i32 = 32'I(S0.i24) * 32'I(S1.i24) + S2.i32
```

## V\_MAD\_U32\_U24

451

在无符号的 32 位整数域中乘以两个无符号的 24 位整数输入,从第三个输入中添加一个无符号的 32 位整数值,并将结果作为无符号的 32 位整数存储到向量寄存器中。

```
D0.u32 = 32'U(S0.u24) * 32'U(S1.u24) + S2.u32
```

## V\_CUBEID\_F32

452

计算指定为三个单精度浮点数输入的 3D 坐标的立方体面代号。将结果存储为单精度浮点数格式。

```
// 设置 D0.f = 立方体面代号 ({0.0, 1.0,..., 5.0}).
// XYZ 坐标以(S0.f,S1.f,S2.f)表示.
// S0.f = x
// S1.f = y
// S2.f = z
if ((abs(S2.f32) >= abs(S0.f32)) && (abs(S2.f32) >= abs(S1.f32))) then
    if S2.f32 < 0.0F then
        D0.f32 = 5.0F
    else
        D0.f32 = 4.0F
    endif
elseif abs(S1.f32) >= abs(S0.f32) then
    if S1.f32 < 0.0F then
        D0.f32 = 3.0F
    else
        D0.f32 = 2.0F
    endif
else
    if S0.f32 < 0.0F then
        D0.f32 = 1.0F
    else
        D0.f32 = 0.0F
    endif
endif
```

## V\_CUBESC\_F32

453

计算 3D 坐标的立方体 S 坐标, 指定为三个单精度浮点数输入。将结果存储为单精度浮点数格式, 保存到向量寄存器中。

```
// D0.f=立方体 S 坐标.
// XYZ 坐标以(S0.f,S1.f,S2.f)表示.
// S0.f = x
// S1.f = y
// S2.f = z
if ((abs(S2.f32) >= abs(S0.f32)) && (abs(S2.f32) >= abs(S1.f32))) then
    if S2.f32 < 0.0F then
        D0.f32 = -S0.f32
    else
        D0.f32 = S0.f32
    endif
elseif abs(S1.f32) >= abs(S0.f32) then
    D0.f32 = S0.f32
else
    if S0.f32 < 0.0F then
        D0.f32 = S2.f32
    else
        D0.f32 = -S2.f32
    endif
endif
```

## V\_CUBETC\_F32

454

计算三维坐标的立方体 T 坐标, 指定为三个单精度浮点数输入。将结果保存为单精度浮点数格式, 保存到向量寄存器中。

```
// D0.f=立方体 T 坐标.
// XYZ 坐标以(S0.f,S1.f,S2.f)表示.
// S0.f = x
// S1.f = y
// S2.f = z
if ((abs(S2.f32) >= abs(S0.f32)) && (abs(S2.f32) >= abs(S1.f32))) then
    D0.f32 = -S1.f32
elseif abs(S1.f32) >= abs(S0.f32) then
    if S1.f32 < 0.0F then
        D0.f32 = -S2.f32
    else
        D0.f32 = S2.f32
    endif
else
    D0.f32 = -S1.f32
endif
```

## V\_CUBEMA\_F32

455

计算 3D 坐标的立方体主轴, 指定为三个单精度浮点数输入。

生成一个向量寄存器中的单精度浮点格式。

```
// D0.f = 2.0 *立方体主轴.
// XYZ 坐标以(S0.f,S1.f,S2.f)表示.
// S0.f = x
// S1.f = y
// S2.f = z
if ((abs(S2.f32) >= abs(S0.f32)) && (abs(S2.f32) >= abs(S1.f32))) then
    D0.f32 = S2.f32 * 2.0F
elseif abs(S1.f32) >= abs(S0.f32) then
    D0.f32 = S1.f32 * 2.0F
else
    D0.f32 = S0.f32 * 2.0F
endif
```

### V\_BFE\_U32

456

从第一个输入中提取一个无符号的比特字段,使用第二个输入和第三个输入的大小抵消,然后将结果存储到向量寄存器中。

```
D0.u32 = ((S0.u32 >> S1[4 : 0].u32) & ((1U << S2[4 : 0].u32) - 1U))
```

### V\_BFE\_I32

457

从第一个输入中提取一个签名的比特字段,使用第二个输入和第三个输入的大小相抵消,然后将结果存储到向量寄存器中。

```
tmp.i32 = ((S0.i32 >> S1[4 : 0].u32) & ((1 << S2[4 : 0].u32) - 1));
D0.i32 = signext_from_bit(tmp.i32, S2[4 : 0].u32)
```

### V\_BFI\_B32

458

在第三个输入中用第二个输入中的一个比特字段覆盖一个比特字段,使用第一个输入中的一个掩码,然后将结果存储到向量寄存器中。

```
D0.u32 = ((S0.u32 & S1.u32) | (~S0.u32 & S2.u32))
```

### V\_FMA\_F32

459

乘以两个单精度浮点数输入,并使用融合乘法输入第三个输入,并存储结果

输入向量寄存器。

```
D0.f32 = fma(S0.f32, S1.f32, S2.f32)
```

说明

0.5ULP 精度,异常值得到支持.

## V\_FMA\_F64

460

乘以两个双精度浮点数输入,再用融合乘法输入第三个输入,然后将结果存储到向量寄存器中.

```
D0.f64 = fma(S0.f64, S1.f64, S2.f64)
```

说明

0.5ULP 精度,异常值得到支持.

## V\_LERP\_U8

461

前两个输入各以打包字节形式保存一个四维向量。计算两个向量的平均值，由第三个输入控制舍入，再把结果写入向量寄存器。第三个输入的每个字节指定对应元素的舍入方式：LSB 为 1 时，0.5 向上舍入；否则截去 0.5。

```
tmp = ((S0.u32[31 : 24] + S1.u32[31 : 24] + S2.u32[24].u8) >> 1U << 24U);
tmp += ((S0.u32[23 : 16] + S1.u32[23 : 16] + S2.u32[16].u8) >> 1U << 16U);
tmp += ((S0.u32[15 : 8] + S1.u32[15 : 8] + S2.u32[8].u8) >> 1U << 8U);
tmp += ((S0.u32[7 : 0] + S1.u32[7 : 0] + S2.u32[0].u8) >> 1U);
D0.u32 = tmp.u32
```

## V\_ALIGNBIT\_B32

462

将前两个输入中编码的 64 位值对齐到第三个输入中指定的位位置,然后将结果存储到一个 32 位向量寄存器中.

```
D0.u32 = 32'U(({ S0.u32, S1.u32 } >> S2.u32[4 : 0]) & 0xffffffffLL)
```

说明



S0 搭载 MSB,S1 搭载正对齐值的 LSB.

**V\_ALIGNBYTE\_B32****463**

将前两个输入中编码的 64 位值对齐到第三个输入中指定的字节位置,然后将结果存储到 32 位向量寄存器中。

```
D0.u32 = 32'U(({ S0.u32, S1.u32 } >> (S2.u32[1 : 0] * 8U)) & 0xffffffffLL)
```

说明



S0 搭载 MSB,S1 搭载正对齐值的 LSB.

**V\_MIN3\_F32****464**

选择最小的 3 个单精度浮点输入,并将所选值存储到向量寄存器中。

```
D0.f32 = v_min_f32(v_min_f32(S0.f32, S1.f32), S2.f32)
```

**V\_MIN3\_I32****465**

选择至少三个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i32 = v_min_i32(v_min_i32(S0.i32, S1.i32), S2.i32)
```

**V\_MIN3\_U32****466**

选择三个无符号的 32 位整数输入的最小值,并将选定的值存储到向量寄存器中。

```
D0.u32 = v_min_u32(v_min_u32(S0.u32, S1.u32), S2.u32)
```

**V\_MAX3\_F32****467**

选择最多三个单精度浮点数输入,并将所选值存储到向量寄存器中。

```
D0.f32 = v_max_f32(v_max_f32(S0.f32, S1.f32), S2.f32)
```

**V\_MAX3\_I32****468**

选择最多三个签名的 32 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i32 = v_max_i32(v_max_i32(S0.i32, S1.i32), S2.i32)
```

**V\_MAX3\_U32****469**

选择最多 3 个无符号的 32 位整数输入,并将选定的数值存储到向量寄存器中。

```
D0.u32 = v_max_u32(v_max_u32(S0.u32, S1.u32), S2.u32)
```

**V\_MED3\_F32****470**

选择三个单精度浮点值的中间值,并将所选值存储到向量寄存器中。

```
if (isNAN(64'F(S0.f32)) || isNAN(64'F(S1.f32)) || isNAN(64'F(S2.f32))) then
    D0.f32 = v_min3_f32(S0.f32, S1.f32, S2.f32)
elseif v_max3_f32(S0.f32, S1.f32, S2.f32) == S0.f32 then
    D0.f32 = v_max_f32(S1.f32, S2.f32)
elseif v_max3_f32(S0.f32, S1.f32, S2.f32) == S1.f32 then
    D0.f32 = v_max_f32(S0.f32, S2.f32)
else
    D0.f32 = v_max_f32(S0.f32, S1.f32)
endif
```

**V\_MED3\_I32****471**

选择三个签名的 32 位整数的中位数,并将选定的数值存储到向量寄存器中。

```
if v_max3_i32(S0.i32, S1.i32, S2.i32) == S0.i32 then
    D0.i32 = v_max_i32(S1.i32, S2.i32)
elseif v_max3_i32(S0.i32, S1.i32, S2.i32) == S1.i32 then
    D0.i32 = v_max_i32(S0.i32, S2.i32)
else
    D0.i32 = v_max_i32(S0.i32, S1.i32)
endif
```

**V\_MED3\_U32****472**

选择 3 个无符号的 32 位整数的中位数,并将选定的数值存储到向量寄存器中。

```
if v_max3_u32(S0.u32, S1.u32, S2.u32) == S0.u32 then
    D0.u32 = v_max_u32(S1.u32, S2.u32)
elsif v_max3_u32(S0.u32, S1.u32, S2.u32) == S1.u32 then
    D0.u32 = v_max_u32(S0.u32, S2.u32)
else
    D0.u32 = v_max_u32(S0.u32, S1.u32)
endif
```

## V\_SAD\_U8

473

计算两个组装的 4 个组件无符号的 8 位整数输入中元素的绝对差异总和,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
ABSDIFF = lambda(x, y) (
    x > y ? x - y : y - x);
// UNSICED 比较
tmp = S2.u32;
tmp += 32'U(ABSDIFF(S0.u32[7 : 0], S1.u32[7 : 0]));
tmp += 32'U(ABSDIFF(S0.u32[15 : 8], S1.u32[15 : 8]));
tmp += 32'U(ABSDIFF(S0.u32[23 : 16], S1.u32[23 : 16]));
tmp += 32'U(ABSDIFF(S0.u32[31 : 24], S1.u32[31 : 24]));
D0.u32 = tmp
```

说明

允许向上位流。

## V\_SAD\_HI\_U8

474

计算两个组装的 4 个组件无符号的 8 位整数输入中元素的绝对差异总和,将剩下的总和移换 16 位,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到一个向量寄存器中。

```
D0.u32 = (32'U(v_sad_u8(S0, S1, 0U)) << 16U) + S2.u32
```

说明

允许向上位流。

## V\_SAD\_U16

475

计算两个组装的 2 个组件的绝对差数总和, 无符号的 16 位整数

输入,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
ABSDIFF = lambda(x, y) (
    x > y ? x - y : y - x);
// UNSICED 比较
tmp = S2.u32;
tmp += ABSDIFF(S0[15 : 0].u16, S1[15 : 0].u16);
tmp += ABSDIFF(S0[31 : 16].u16, S1[31 : 16].u16);
D0.u32 = tmp
```

## V\_SAD\_U32

476

计算两个无符号的 32 位整数输入的绝对差值,从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
ABSDIFF = lambda(x, y) (
    x > y ? x - y : y - x);
// UNSICED 比较
D0.u32 = ABSDIFF(S0.u32, S1.u32) + S2.u32
```

## V\_CVT\_PK\_U8\_F32

477

将第一个输入的单精度浮点值转换为无符号的 8 位整数值,并使用第二个输入作为字节选择将结果打包成第三个输入的一个字节。将结果存储到向量寄存器中。

```
tmp = (S2.u32 & 32'U(~(0xff << (S1.u32[1 : 0].u32 * 8U))));
tmp = (tmp | ((32'U(f32_to_u8(S0.f32)) & 255U) << (S1.u32[1 : 0].u32 * 8U)));
D0.u32 = tmp
```

## V\_DIV\_FIXUP\_F32

478

鉴于第一个输入中有一个单精度浮商,第二个输入中有一个分母,第三个输入中有一个数字,检测和应用与除法相关的角大小写,包括除零,NaN 输入和溢出,并相应修改商数。生成任何无效的,非正常的和除零的例外,这些例外是除法的结果。将修改过的商数存储到向量寄存器中。

此操作在除以 0 和 NaN 输入等分区宏中处理角大小写。当商数大约等于分母除以数字时,此操作被定义良好。其他输入会产生可预测的结果,但可能不会在数学上有用。

```
sign_out = (sign(S1.f32) ^ sign(S2.f32));
if isNaN(64'F(S2.f32)) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S2.f32)))
```



```

elseif isNaN(64'F(S1.f32)) then
    D0.f32 = 32'F(cvtToQuietNaN(64'F(S1.f32)))
elseif ((64'F(S1.f32) == 0.0) && (64'F(S2.f32) == 0.0)) then
    // 0/0
    D0.f32 = 32'F(0xffc00000)
elseif ((64'F(abs(S1.f32)) == +INF) && (64'F(abs(S2.f32)) == +INF)) then
    // inf/inf
    D0.f32 = 32'F(0xffc00000)
elseif ((64'F(S1.f32) == 0.0) || (64'F(abs(S2.f32)) == +INF)) then
    // x/0, 或 inf/y (单位)
    D0.f32 = sign_out ? -INF.f32 : +INF.f32
elseif ((64'F(abs(S1.f32)) == +INF) || (64'F(S2.f32) == 0.0)) then
    // x/inf, 0/y
    D0.f32 = sign_out ? -0.0F : 0.0F
elseif exponent(S2.f32) - exponent(S1.f32) < -150 then
    D0.f32 = sign_out ? -UNDERFLOW_F32 : UNDERFLOW_F32
elseif exponent(S1.f32) == 255 then
    D0.f32 = sign_out ? -OVERFLOW_F32 : OVERFLOW_F32
else
    D0.f32 = sign_out ? -abs(S0.f32) : abs(S0.f32)
endif

```

## 说明

这一行动是高精度师宏的最后一步,处理所有特殊师级情况.

## V\_DIV\_FIXUP\_F64

479

鉴于第一个输入时的双精度浮商,第二个输入时的分母和第三个输入时的分母,检测和应用与除法相关的角大小写,包括除零,NaN 输入和溢出,并相应修改商号。生成任何无效的,非正常的和除零的例外,这些例外是除法的结果。将修改过的商号存储到向量寄存器中。

此操作在除以 0 和 NaN 输入等分区宏中处理角大小写。当商数大约等于分母除以数字时,此操作被定义良好。其他输入会产生可预测的结果,但可能不会在数学上有用。

```

sign_out = (sign(S1.f64) ^ sign(S2.f64));
if isNaN(S2.f64) then
    D0.f64 = cvtToQuietNaN(S2.f64)
elseif isNaN(S1.f64) then
    D0.f64 = cvtToQuietNaN(S1.f64)
elseif ((S1.f64 == 0.0) && (S2.f64 == 0.0)) then
    // 0/0
    D0.f64 = 64'F(0xffff800000000000LL)
elseif ((abs(S1.f64) == +INF) && (abs(S2.f64) == +INF)) then
    // inf/inf
    D0.f64 = 64'F(0xffff800000000000LL)
elseif ((S1.f64 == 0.0) || (abs(S2.f64) == +INF)) then
    // x/0, 或 inf/y (单位)
    D0.f64 = sign_out ? -INF : +INF
elseif ((abs(S1.f64) == +INF) || (S2.f64 == 0.0)) then
    // x/inf, 0/y

```

```

    D0.f64 = sign_out ? -0.0 : 0.0
elseif exponent(S2.f64) - exponent(S1.f64) < -1075 then
    D0.f64 = sign_out ? -UNDERFLOW_F64 : UNDERFLOW_F64
elseif exponent(S1.f64) == 2047 then
    D0.f64 = sign_out ? -OVERFLOW_F64 : OVERFLOW_F64
else
    D0.f64 = sign_out ? -abs(S0.f64) : abs(S0.f64)
endif

```

## 说明

这一行动是高精度师宏的最后一步,处理所有特殊师级情况.

## V\_DIV\_SCALE\_F32

480

鉴于在第一次输入中要缩放一个单精度浮点值,第二次输入中的分母和第三次输入中的数字,如果为了避免在应用牛顿-拉夫森校正方法时出现非正常名词需要,则要缩放第一个输入。如果需要缩放后需要,则将缩放结果存储到向量寄存器中,并设置向量条件码。

此操作是为在高精度分割宏中使用而设计的。第一个输入值应与第二个输入值或第三个输入值相同;其他缩放值会产生可预测的结果,但可能无法在数学上有用。V\_DIV\_FMAS\_F32 使用向量条件码来确定商数是否需要后缩放。

```

VCC = 0x0LL;
if ((64'F(S2.f32) == 0.0) || (64'F(S1.f32) == 0.0)) then
    D0.f32 = NAN.f32
elseif exponent(S2.f32) - exponent(S1.f32) >= 96 then
    // MAX_FLOAT_F32 附近无编号/D
    VCC = 0x1LL;
    if S0.f32 == S1.f32 then
        // 只缩放分母
        D0.f32 = ldexp(S0.f32, 64)
    endif
elseif S1.f32 == DENORM.f32 then
    D0.f32 = ldexp(S0.f32, 64)
elseif ((1.0 / 64'F(S1.f32) == DENORM.f64) && (S2.f32 / S1.f32 == DENORM.f32)) then
    VCC = 0x1LL;
    if S0.f32 == S1.f32 then
        // 只缩放分母
        D0.f32 = ldexp(S0.f32, 64)
    endif
elseif 1.0 / 64'F(S1.f32) == DENORM.f64 then
    D0.f32 = ldexp(S0.f32, -64)
elseif S2.f32 / S1.f32 == DENORM.f32 then
    VCC = 0x1LL;
    if S0.f32 == S2.f32 then
        // 只缩放数字
        D0.f32 = ldexp(S0.f32, 64)
    endif
elseif exponent(S2.f32) <= 23 then
    // 算术很小

```

```
D0.f32 = ldexp(S0.f32, 64)
endif
```

## 说明

V\_DIV\_SCALE\_F32、V\_DIV\_FMAS\_F32 和 V\_DIV\_FIXUP\_F32 均被设计用于高精度的分区宏中,利用 V\_RCP\_F32 和 V\_MUL\_F32 来计算大致结果,然后应用牛顿-拉弗森方法的两个步骤来趋同到商数。如果在计算中出现次常数,则发生精度损失。这种精度损失可以通过缩小输入量,然后在牛顿-拉弗森应用后对商数进行测算来避免。

## V\_DIV\_SCALE\_F64

481

鉴于在第一个输入中具有双精度浮点值,在第二个输入中具有分母值,在第三个输入中具有一个数字值,如果为了避免在应用牛顿-拉夫森校正方法时出现非正常名词而需要,则将第一个输入分母缩放。如果需要后缩放,则将缩放结果存储到向量寄存器中并设定向量条件码。

此操作是为在高精度分割宏中使用而设计的。第一个输入值应与第二个或第三个输入值相同;其他比例值会产生可预测的结果,但可能无法在数学上有用。V\_DIV\_FMAS\_F64 使用向量条件码来确定商号是否要求进行后缩放。

```
VCC = 0x0LL;
if ((S2.f64 == 0.0) || (S1.f64 == 0.0)) then
    D0.f64 = NAN.f64
elseif exponent(S2.f64) - exponent(S1.f64) >= 768 then
    // MAX_FLOAT_F64 附近无编号/D
    VCC = 0x1LL;
    if S0.f64 == S1.f64 then
        // 只缩放分母
        D0.f64 = ldexp(S0.f64, 128)
    endif
elseif S1.f64 == DENORM.f64 then
    D0.f64 = ldexp(S0.f64, 128)
elseif ((1.0 / S1.f64 == DENORM.f64) && (S2.f64 / S1.f64 == DENORM.f64)) then
    VCC = 0x1LL;
    if S0.f64 == S1.f64 then
        // 只缩放分母
        D0.f64 = ldexp(S0.f64, 128)
    endif
elseif 1.0 / S1.f64 == DENORM.f64 then
    D0.f64 = ldexp(S0.f64, -128)
elseif S2.f64 / S1.f64 == DENORM.f64 then
    VCC = 0x1LL;
    if S0.f64 == S2.f64 then
        // 只缩放数字
        D0.f64 = ldexp(S0.f64, 128)
    endif
elseif exponent(S2.f64) <= 53 then
    // 算术很小
    D0.f64 = ldexp(S0.f64, 128)
```

```
endif
```

### 说明

V\_DIV\_SCALE\_F64、V\_DIV\_FMAS\_F64 和 V\_DIV\_FIXUP\_F64 均被设计用于高精度的分区宏中,利用 V\_RCP\_F64 和 V\_MUL\_F64 来计算大致结果,然后应用牛顿-拉弗森方法的两个步骤来趋同到商数。如果在计算中出现次常数,则发生精度损失。这种精度损失可以通过缩小输入量,然后在牛顿-拉弗森应用后对商数进行测算来避免。

## V\_DIV\_FMAS\_F32

482

乘以两个单精度浮点输入,并使用融合乘法添加第三个输入,然后如果设定了向量条件码,则将结果的缩放为固定因子。将结果存储到向量寄存器中。

此操作是为用于浮点划分宏而设计的,并依靠 V\_DIV\_SCALE\_F32 设定向量条件码,如果商号需要后缩放。

```
if VCC.u64[laneId] then
    D0.f32 = 2.0F ** 32 * fma(S0.f32, S1.f32, S2.f32)
else
    D0.f32 = fma(S0.f32, S1.f32, S2.f32)
endif
```

### 说明

输入非正常不冲,但允许输出冲洗。

V\_DIV\_SCALE\_F32、V\_DIV\_FMAS\_F32 和 V\_DIV\_FIXUP\_F32 均被设计用于高精度的分区宏中,利用 V\_RCP\_F32 和 V\_MUL\_F32 来计算大致结果,然后应用牛顿-拉弗森方法的两个步骤来趋同到商数。如果在计算中出现次常数,则发生精度损失。这种精度损失可以通过缩小输入量,然后在牛顿-拉弗森应用后对商数进行测算来避免。

## V\_DIV\_FMAS\_F64

483

乘以两个双精度浮点输入,并使用融合乘法添加第三个输入,然后如果设定了向量条件码,则将结果的表示系数缩放为固定系数。将结果存储到向量寄存器中。

此操作是为用于浮点划分宏而设计的,并依靠 V\_DIV\_SCALE\_F64 设定向量条件码,如果商号需要后缩放。

```
if VCC.u64[laneId] then
    D0.f64 = 2.0 ** 64 * fma(S0.f64, S1.f64, S2.f64)
else
    D0.f64 = fma(S0.f64, S1.f64, S2.f64)
```

```
endif
```

## 说明

输入非正常不冲,但允许输出冲洗。

V\_DIV\_SCALE\_F64, V\_DIV\_FMAS\_F64 和 V\_DIV\_FIXUP\_F64 均被设计用于高精度的分区宏中,利用 V\_RCP\_F64 和 V\_MUL\_F64 来计算大致结果,然后应用牛顿-拉弗森方法的两个步骤来趋同到商数。如果在计算中出现次常数,则发生精度损失。这种精度损失可以通过缩小输入量,然后在牛顿-拉弗森应用后对商数进行测算来避免。

## V\_MSAD\_U8

484

计算两个组合的 4 组件无符号的 8 位整数输入中元素的绝对差异总和,但不包括第二个输入为 0 的元素(称为参考输入)。从第三个输入中添加一个无符号的 32 位整数值,并将结果存储到向量寄存器中。

```
ABSDIFF = lambda(x, y) (
    x > y ? x - y : y - x);
// UNSICED 比较
tmp = S2.u32;
tmp += S1.u32[7 : 0] == 8'0U ? 0U : 32'U(ABSDIFF(S0.u32[7 : 0], S1.u32[7 : 0]));
tmp += S1.u32[15 : 8] == 8'0U ? 0U : 32'U(ABSDIFF(S0.u32[15 : 8], S1.u32[15 : 8]));
tmp += S1.u32[23 : 16] == 8'0U ? 0U : 32'U(ABSDIFF(S0.u32[23 : 16], S1.u32[23 : 16]));
tmp += S1.u32[31 : 24] == 8'0U ? 0U : 32'U(ABSDIFF(S0.u32[31 : 24], S1.u32[31 : 24]));
D0.u32 = tmp
```

## 说明

允许向上位流。

## V\_QSAD\_PK\_U16\_U8

485

使用第一个阵列的不同切片,第二个阵列的所有条目和第三个阵列的每个条目,四次执行 V\_SAD\_U8 操作。将每个阵列的值排成 16 位,将数值装入 4 位阵列,并将数组存储到向量寄存器中。第一个输入是一个 8 位无符号的 8 位整数的 8 位数阵列,第二个输入是一个 4 位无符号的 8 位整数的 4 位数,第三个输入是一个 4 位无符号的 16 位整数的 4 位数。

```
tmp[63 : 48] = 16'B(v_sad_u8(S0[55 : 24], S1[31 : 0], S2[63 : 48].u32));
tmp[47 : 32] = 16'B(v_sad_u8(S0[47 : 16], S1[31 : 0], S2[47 : 32].u32));
tmp[31 : 16] = 16'B(v_sad_u8(S0[39 : 8], S1[31 : 0], S2[31 : 16].u32));
tmp[15 : 0] = 16'B(v_sad_u8(S0[31 : 0], S1[31 : 0], S2[15 : 0].u32));
D0.b64 = tmp.b64
```

**V\_MQSAD\_PK\_U16\_U8****486**

V\_MSAD\_U8 四次操作,使用第一个阵列的不同切片,第二个阵列的所有条目和第三个阵列的每个条目。将每个位数排成 16 位,将数值装入一个 4 位数阵列,并将该阵列存储到向量寄存器中。第一个输入是 8 位数无符号的 8 位整数阵列,第二个输入是 4 位数无符号的 8 位整数阵列,第三个输入是 4 位数无符号的 16 位整数阵列。

```
tmp[63 : 48] = 16'B(v_msad_u8(S0[55 : 24], S1[31 : 0], S2[63 : 48].u32));
tmp[47 : 32] = 16'B(v_msad_u8(S0[47 : 16], S1[31 : 0], S2[47 : 32].u32));
tmp[31 : 16] = 16'B(v_msad_u8(S0[39 : 8], S1[31 : 0], S2[31 : 16].u32));
tmp[15 : 0] = 16'B(v_msad_u8(S0[31 : 0], S1[31 : 0], S2[15 : 0].u32));
D0.b64 = tmp.b64
```

**V\_MQSAD\_U32\_U8****487**

V\_MSAD\_U8 操作四次,使用第一个阵列的不同切片,第二个阵列的所有条目和第三个阵列的每个条目。将每个 32 位值打入一个 4 位数阵列,并将该数阵列存储到一个向量寄存器中。第一次输入是一个 8 位数无符号的 8 位数整数阵列,第二次输入是一个 4 位数无符号的 8 位数整数阵列,第三次输入是一个 4 位数无符号的 32 位数整数阵列。

```
tmp[127 : 96] = 32'B(v_msad_u8(S0[55 : 24], S1[31 : 0], S2[127 : 96].u32));
tmp[95 : 64] = 32'B(v_msad_u8(S0[47 : 16], S1[31 : 0], S2[95 : 64].u32));
tmp[63 : 32] = 32'B(v_msad_u8(S0[39 : 8], S1[31 : 0], S2[63 : 32].u32));
tmp[31 : 0] = 32'B(v_msad_u8(S0[31 : 0], S1[31 : 0], S2[31 : 0].u32));
D0.b128 = tmp.b128
```

**V\_MAD\_U64\_U32****488**

乘以两个无符号的整数输入,加上第三个无符号的整数输入,将结果存储到 64 位向量寄存器中,并将溢出/递出存储到标量 mask 寄存器中。

```
{ D1.u1, D0.u64 } = 65'B(65'U(S0.u32) * 65'U(S1.u32) + 65'U(S2.u64))
```

**说明**

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

**V\_MAD\_I64\_I32****489**

乘以两个签名的整数输入,加上第三个签名的整数输入,将结果存储到 64 位向量寄存器中,并将溢出/运出存储到标量掩码寄存器中。

```
{ D1.i1, D0.i64 } = 65'B(65'I(S0.i32) * 65'I(S1.i32) + 65'I(S2.i64))
```

说明

在 VOP3,VCC 目的地可能是任意的 SGPR-pair.

## V\_MAD\_LEGACY\_F16

490

乘以 FP16 值。为 OPSEL 执行 IEEE 规则和非标准规则。

```
tmp = S0.f16 * S1.f16 + S2.f16;
if OPSEL.u4[3] then
    D0 = { tmp.f16, D0[15 : 0] }
else
    D0 = { 16'0, tmp.f16 }
endif
```

说明

支持舍入模式,例外旗帜,饱和度.

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特写为 0(这与 V\_MAD\_F16 不同).

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来.

## V\_MAD\_LEGACY\_U16

491

乘以无符号的短值。对 OPSEL 有非标准规则。

```
tmp = S0.u16 * S1.u16 + S2.u16;
if OPSEL.u4[3] then
    D0 = { tmp.u16, D0[15 : 0] }
else
    D0 = { 16'0, tmp.u16 }
endif
```

说明

支持饱和(无符号的 16 位整数域).

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特写为 0(这与 V\_MAD\_U16 不同).

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来.

**V\_MAD\_LEGACY\_I16****492**

乘以签名的短值。对 OPSEL 有非标准规则。

```
tmp = S0.i16 * S1.i16 + S2.i16;
if OPSEL.u4[3] then
    D0 = { tmp.i16, D0[15 : 0] }
else
    D0 = { 16'0, tmp.i16 }
endif
```

说明

支持饱和(签名的 16 位整数域)。

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特写为 0(这与 V\_MAD\_I16 不同)。

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来。

**V\_PERM\_B32****493**

Permute 从两个向量输入构建一个 64 位值(最显著的位来自第一个输入),使用第三个输入的每行选择器。通道选择器允许结果的每个字节从 8 个输入字节中选择一个字节,执行 0/1 位的符号扩展或垫。将结果存储到一个向量寄存器中。

```
BYTE_PERMUTE = lambda(data, sel) (
    declare in : 8'B[8];
    for i in 0 : 7 do
        in[i] = data[i * 8 + 7 : i * 8].b8
    endfor;
    if sel.u32 >= 13U then
        return 8'0xff
    elsif sel.u32 == 12U then
        return 8'0x0
    elsif sel.u32 == 11U then
        return in[7][7].b8 * 8'0xff
    elsif sel.u32 == 10U then
        return in[5][7].b8 * 8'0xff
    elsif sel.u32 == 9U then
        return in[3][7].b8 * 8'0xff
    elsif sel.u32 == 8U then
        return in[1][7].b8 * 8'0xff
    else
        return in[sel]
    endif);
D0[31 : 24] = BYTE_PERMUTE({ S0.u32, S1.u32 }, S2.u32[31 : 24]);
D0[23 : 16] = BYTE_PERMUTE({ S0.u32, S1.u32 }, S2.u32[23 : 16]);
D0[15 : 8] = BYTE_PERMUTE({ S0.u32, S1.u32 }, S2.u32[15 : 8]);
```



```
D0[7 : 0] = BYTE_PERMUTE({ S0.u32, S1.u32 }, S2.u32[7 : 0])
```

### 说明

选择 0 到 7 选择 64 位输入值的相应字节。

选择 8 至 11 通过复制选定字节的前置位点,可以将一个较小精度的签名整数的符号扩展为更大的精度结果。

选择 12 和 13 个返回补丁值,分别为 0 位和 1 位。

注意所选择的 64 位值的 MSB 被存储在 S0 中。对于一个小 endian 架构来说,这是反直观的。

## V\_FMA\_LEGACY\_F16

494

Fused 半精度乘法添加。为 OPSEL 执行 IEEE 规则和非标准规则。

```
tmp = fma(S0.f16, S1.f16, S2.f16);
if OPSEL.u4[3] then
    D0 = { tmp.f16, D0[15 : 0] }
else
    D0 = { 16'0, tmp.f16 }
endif
```

## V\_DIV\_FIXUP\_LEGACY\_F16

495

半精度分裂修复,对 OPSEL 有非标准规则.

S0 = 引文使用过时参数 coauthors (帮助) S1 = 解码器 S2 = 数字器.

鉴于一个数字、分母和从一个分数算出的商号,这个 opcode 检测并应用了特定的大小写数字,必要时可以触摸到商号。这个 opcode 还生成无效的、解的和由除法引起的零例外的除法。

```
sign_out = (sign(S1.f16) ^ sign(S2.f16));
if isNaN(64'F(S2.f16)) then
    tmp = cvtToQuietNaN(64'F(S2.f16))
elseif isNaN(64'F(S1.f16)) then
    tmp = cvtToQuietNaN(64'F(S1.f16))
elseif ((64'F(S1.f16) == 0.0) && (64'F(S2.f16) == 0.0)) then
    // 0/0
    tmp = 16'F(0xfe00)
elseif ((64'F(abs(S1.f16)) == +INF) && (64'F(abs(S2.f16)) == +INF)) then
    // inf/inf
    tmp = 16'F(0xfe00)
elseif ((64'F(S1.f16) == 0.0) || (64'F(abs(S2.f16)) == +INF)) then
    // x/0, 或 inf/y (单位)
```

```

    tmp = sign_out ? -INF : +INF
elseif ((64'F(abs(S1.f16)) == +INF) || (64'F(S2.f16) == 0.0)) then
    // x/inf, 0/y
    tmp = sign_out ? -0.0 : 0.0
else
    tmp = sign_out ? -abs(S0.f16) : abs(S0.f16)
endif;
if OPSEL.u4[3] then
    D0 = { tmp.f16, D0[15 : 0] }
else
    D0 = { 16'0, tmp.f16 }
endif

```

**V\_CVT\_PKACCUM\_U8\_F32****496**

将第一个输入中的单精度浮点值转换为无符号的 8 位整数值,并将结果存储为目标寄存器的一个字节,使用第二个输入作为字节选择。

```

byte = S1.u32[1 : 0];
bit = byte.u32 * 8U;
D0.u32[bit + 7U : bit] = 32'U(f32_to_u8(S0.f32))

```

**说明**

此 opcode 使用 src\_c 作为源代码通过目的地。

**V\_MAD\_U32\_U16****497**

在无符号的 32 位整数域中乘以两个无符号的 16 位整数输入,从第三个输入中添加一个无符号的 32 位整数值,并将结果作为无符号的 32 位整数存储到向量寄存器中。

```

D0.u32 = 32'U(S0.u16) * 32'U(S1.u16) + S2.u32

```

**V\_MAD\_I32\_I16****498**

在签名的 32 位整数域中,乘以 2 个签名的 16 位整数输入,从第三个输入中添加一个签名的 32 位整数值,并将结果作为签名的 32 位整数存储到向量寄存器中。

```

D0.i32 = 32'I(S0.i16) * 32'I(S1.i16) + S2.i32

```

**V\_XAD\_U32****499**

计算前两个向量输入的位元 XOR,然后将第三个向量输入添加到中间结果,然后将最终结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 ^ S1.u32) + S2.u32
```

说明

没有载运/载运和饱和。此操作码旨在帮助加速 SHA256 散列算法。

---

### V\_MIN3\_F16

**500**

选择最小的 3 个半精度浮点输入,并将所选值存储到向量寄存器中。

```
D0.f16 = v_min_f16(v_min_f16(S0.f16, S1.f16), S2.f16)
```

---

### V\_MIN3\_I16

**501**

选择至少三个签名的 16 位整数输入,并将所选值存储到向量寄存器中。

```
D0.i16 = v_min_i16(v_min_i16(S0.i16, S1.i16), S2.i16)
```

---

### V\_MIN3\_U16

**502**

选择三个无符号的 16 位整数输入的最小值,并将所选值存储到向量寄存器中。

```
D0.u16 = v_min_u16(v_min_u16(S0.u16, S1.u16), S2.u16)
```

---

### V\_MAX3\_F16

**503**

选择最多三个半精度浮点数输入,并将所选值存储到向量寄存器中。

```
D0.f16 = v_max_f16(v_max_f16(S0.f16, S1.f16), S2.f16)
```

---

### V\_MAX3\_I16

**504**

选择最多三个签名的 16 位整数输入,并将选定的数值存储到向量寄存器中。

```
D0.i16 = v_max_i16(v_max_i16(S0.i16, S1.i16), S2.i16)
```

## V\_MAX3\_U16

505

选择三个无符号的 16 位整数输入的最大值,并将所选值存储到向量寄存器中。

```
D0.u16 = v_max_u16(v_max_u16(S0.u16, S1.u16), S2.u16)
```

## V\_MED3\_F16

506

选择三个半精度浮点值的中位数,并将所选值存储到向量寄存器中。

```
if (isNAN(64'F(S0.f16)) || isNAN(64'F(S1.f16)) || isNAN(64'F(S2.f16))) then
    D0.f16 = v_min3_f16(S0.f16, S1.f16, S2.f16)
elseif v_max3_f16(S0.f16, S1.f16, S2.f16) == S0.f16 then
    D0.f16 = v_max_f16(S1.f16, S2.f16)
elseif v_max3_f16(S0.f16, S1.f16, S2.f16) == S1.f16 then
    D0.f16 = v_max_f16(S0.f16, S2.f16)
else
    D0.f16 = v_max_f16(S0.f16, S1.f16)
endif
```

## V\_MED3\_I16

507

选择三个签名的 16 位整数的中位值,并将选定的数值存储到向量寄存器中。

```
if v_max3_i16(S0.i16, S1.i16, S2.i16) == S0.i16 then
    D0.i16 = v_max_i16(S1.i16, S2.i16)
elseif v_max3_i16(S0.i16, S1.i16, S2.i16) == S1.i16 then
    D0.i16 = v_max_i16(S0.i16, S2.i16)
else
    D0.i16 = v_max_i16(S0.i16, S1.i16)
endif
```

## V\_MED3\_U16

508

选择 3 个无符号的 16 位整数的中位数,并将选定的数值存储到向量寄存器中。

```
if v_max3_u16(S0.u16, S1.u16, S2.u16) == S0.u16 then
    D0.u16 = v_max_u16(S1.u16, S2.u16)
```

```
elseif v_max3_u16(S0.u16, S1.u16, S2.u16) == S1.u16 then
    D0.u16 = v_max_u16(S0.u16, S2.u16)
else
    D0.u16 = v_max_u16(S0.u16, S1.u16)
endif
```

## V\_LSHL\_ADD\_U32

509

考虑到第二个输入中的移位计数,计算第一个输入左侧的逻辑移位,然后将第三个输入加入中间结果,然后将最终结果存储到向量寄存器中。

```
D0.u32 = (S0.u32 << S1.u32[4 : 0].u32) + S2.u32
```

## V\_ADD\_LSHL\_U32

510

添加前两个整数输入,然后在第三个输入中给出一个转数,计算中间结果左侧的逻辑转数,然后将最终结果存储到向量寄存器中。

```
D0.u32 = ((S0.u32 + S1.u32) << S2.u32[4 : 0].u32)
```

## V\_ADD3\_U32

511

添加三个无符号输入,并将结果存储到向量寄存器中。没有随身携带或随身携带支持。

```
D0.u32 = S0.u32 + S1.u32 + S2.u32
```

## V\_LSHL\_OR\_B32

512

鉴于第二个输入中的移位计数,计算第一个输入左侧的逻辑移位,然后计算中间结果的位元 OR 和第三个输入,然后将最终结果存储到向量寄存器中。

```
D0.u32 = ((S0.u32 << S1.u32[4 : 0].u32) | S2.u32)
```

## V\_AND\_OR\_B32

513

在前两个向量输入上按位计算,然后计算中间结果的位元 OR 和第三个向量输入,然后将最终结果存储到向量寄存器中。

$$D0.u32 = ((S0.u32 \& S1.u32) | S2.u32)$$

说明

不支持输入和输出修改符。

### V\_OR3\_B32

514

计算三次向量输入的位元 OR,并将结果存储到向量寄存器中.

$$D0.u32 = (S0.u32 | S1.u32 | S2.u32)$$

说明

不支持输入和输出修改符。

### V\_MAD\_F16

515

将两个半精度的浮点输入乘以第三个输入,并将结果存储到向量寄存器中.

$$D0.f16 = S0.f16 * S1.f16 + S2.f16$$

说明

支持舍入模式,例外旗帜,饱和度. 1ULP 精度,异常值被冲洗.

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特被保存下来.

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来.

### V\_MAD\_U16

516

乘以两个无符号的 16 位整数输入,从第三个输入中添加一个无符号的 16 位整数值,并将结果存储到向量寄存器中.

$$D0.u16 = S0.u16 * S1.u16 + S2.u16$$

说明

支持饱和(无符号的 16 位整数域).

如果 OPSEL[3]为 0,结果会写给目的地的 16 个 LSB VGPR,高的 16 位被保留下来.

如果 OPSEL [3] 是 1,则结果会写入目的地的 16 个 MSB VGPR,低 16 位被保存下来.

---

### V\_MAD\_I16

517

乘以两个签名的 16 位整数输入,从第三个输入中添加一个签名的 16 位整数值,并将结果存储到向量寄存器中.

```
D0.i16 = S0.i16 * S1.i16 + S2.i16
```

说明

支持饱和(签名的 16 位整数域).

如果 OPSEL[3]为 0,结果会写给目的地的 16 个 LSB VGPR,高的 16 位被保留下来.

如果 OPSEL [3] 是 1,则结果会写入目的地的 16 个 MSB VGPR,低 16 位被保存下来.

---

### V\_FMA\_F16

518

乘以两个半精度浮点数输入,再用融合乘法输入第三个输入,然后将结果存储到向量寄存器中.

```
D0.f16 = fma(S0.f16, S1.f16, S2.f16)
```

说明

0.5ULP 精度,异常值得到支持.

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特被保存下来.

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来.

---

### V\_DIV\_FIXUP\_F16

519

鉴于第一个输入的半精度浮商、第二个输入的一个分母和第三个输入的一个数字,检测和应用与除法有关的角大小写,包括除零、NaN 输入和溢出,并相应修改商号。生成任何无效的、不正常的和除零的例外,这些例外是除法的结果。将已修改的商号存储到向量寄存器中。

此操作在除以 0 和 NaN 输入等分区宏中处理角大小写。当商数大约等于分母除以数字时,此操作被定义良好。其他输入会产生可预测的结果,但可能不会在数学上有用。

```

sign_out = (sign(S1.f16) ^ sign(S2.f16));
if isNaN(64'F(S2.f16)) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S2.f16)))
elseif isNaN(64'F(S1.f16)) then
    D0.f16 = 16'F(cvtToQuietNaN(64'F(S1.f16)))
elseif ((64'F(S1.f16) == 0.0) && (64'F(S2.f16) == 0.0)) then
    // 0/0
    D0.f16 = 16'F(0xfe00)
elseif ((64'F(abs(S1.f16)) == +INF) && (64'F(abs(S2.f16)) == +INF)) then
    // inf/inf
    D0.f16 = 16'F(0xfe00)
elseif ((64'F(S1.f16) == 0.0) || (64'F(abs(S2.f16)) == +INF)) then
    // x/0, 或 inf/y (单位)
    D0.f16 = sign_out ? -INF.f16 : +INF.f16
elseif ((64'F(abs(S1.f16)) == +INF) || (64'F(S2.f16) == 0.0)) then
    // x/inf, 0/y
    D0.f16 = sign_out ? -16'0.0 : 16'0.0
else
    D0.f16 = sign_out ? -abs(S0.f16) : abs(S0.f16)
endif

```

#### 说明

这一行动是高精度师宏的最后一步,处理所有特殊师级情况.

如果 OPSEL[3]是 0 结果写给目的地的 16 个 LSB VGPR,hi 16 比特被保存下来.

如果 OPSEL[3]是 1 结果写给目的地的 16 个 MSB VGPR,并且 lo 16 位被保存下来.

## V\_LSHL\_ADD\_U64

520

考虑到第二个输入中的移位计数,计算第一个输入左侧的逻辑移位,然后将第三个输入加入中间结果,然后将最终结果存储到向量寄存器中.

对于此 opcode , 值必须介于 0 到 4 之间。 值高的值是不支持的。

```
D0.u64 = (S0.u64 << S1.u32[2 : 0].u32) + S2.u64
```

#### 说明

设计将无支持的移位计数视为零的移位.

## V\_ADD\_F64

640

添加两个浮点输入并将结果存储到向量寄存器中.



```
D0.f64 = S0.f64 + S1.f64
```

说明

0.5ULP 精度,异常态得到支持.

## V\_MUL\_F64

641

乘以两个浮点输入并将结果存储到向量寄存器中。

```
D0.f64 = S0.f64 * S1.f64
```

说明

0.5ULP 精度,异常态得到支持.

## V\_MIN\_F64

642

选择至少两个双精度浮点输入,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(S0.f64)) then
    D0.f64 = cvtToQuietNaN(S0.f64)
elseif (WAVE_MODE.IEEE && isSignalNaN(S1.f64)) then
    D0.f64 = cvtToQuietNaN(S1.f64)
elseif isNaN(S0.f64) then
    D0.f64 = S1.f64
elseif isNaN(S1.f64) then
    D0.f64 = S0.f64
elseif ((S0.f64 == +0.0) && (S1.f64 == -0.0)) then
    D0.f64 = S1.f64
elseif ((S0.f64 == -0.0) && (S1.f64 == +0.0)) then
    D0.f64 = S0.f64
else
    // 注意:这里没有 IEEE 案件,像 V_MAX_F64 一样.
    D0.f64 = S0.f64 < S1.f64 ? S0.f64 : S1.f64
endif
```

## V\_MAX\_F64

643

选择两个双精度浮点数的最大值,并将结果存储到向量寄存器中。

```
if (WAVE_MODE.IEEE && isSignalNaN(S0.f64)) then
    D0.f64 = cvtToQuietNaN(S0.f64)
```

```

elseif (WAVE_MODE.IEEE && isSignalNaN(S1.f64)) then
    D0.f64 = cvtToQuietNaN(S1.f64)
elseif isNaN(S0.f64) then
    D0.f64 = S1.f64
elseif isNaN(S1.f64) then
    D0.f64 = S0.f64
elseif ((S0.f64 == +0.0) && (S1.f64 == -0.0)) then
    D0.f64 = S0.f64
elseif ((S0.f64 == -0.0) && (S1.f64 == +0.0)) then
    D0.f64 = S1.f64
elseif WAVE_MODE.IEEE then
    D0.f64 = S0.f64 >= S1.f64 ? S0.f64 : S1.f64
else
    D0.f64 = S0.f64 > S1.f64 ? S0.f64 : S1.f64
endif

```

**V\_LDEXP\_F64****644**

乘以第一个输入,一个浮点值,通过第二个输入中指定的 2 个整体功率,一个签名的整数值,并将浮点结果存储到向量寄存器中。

```
D0.f64 = S0.f64 * 2.0 ** S1.i32
```

说明

与 C 中的 `ldexp()` 函数比较。

**V\_MUL\_LO\_U32****645**

乘以两个无符号的 32 位整数输入,并将结果存储到向量寄存器中。

```
D0.u32 = S0.u32 * S1.u32
```

说明

以小尺寸的整数乘以 `V_MUL_U32_U24`,意在更高效地执行。

**V\_MUL\_HI\_U32****646**

乘以两个无符号的 32 位整数输入,并将结果的高 32 位存储到向量寄存器中。

```
D0.u32 = 32'U((64'U(S0.u32) * 64'U(S1.u32)) >> 32U)
```

## 说明

以小尺寸的整数乘以 V\_MUL\_HI\_U32\_U24,意在更高效地执行。

**V\_MUL\_HI\_I32****647**

乘以两个签名的 32 位整数输入,并将结果的高 32 位存储到向量寄存器中。

```
D0.i32 = 32'I((64'I(S0.i32) * 64'I(S1.i32)) >> 32U)
```

## 说明

以小尺寸的整数乘以 V\_MUL\_HI\_I32\_I24,意在更高效地执行。

**V\_LDEXP\_F32****648**

乘以第一个输入,一个浮点值,通过第二个输入中指定的 2 个整体功率,一个签名的整数值,并将浮点结果存储到向量寄存器中。

```
D0.f32 = S0.f32 * 2.0F ** S1.i32
```

## 说明

与 C 中的 ldexp () 函数比较。

**V\_READLANE\_B32****649**

读取在第二个输入中选择的首个输入的指定道中的标量值。将结果存储到一个标量寄存器中。

```
lane = S1.u32[5 : 0];  
// 巷子选择  
D0.b32 = VGPR[lane][SRC0.u32]
```

## 说明

VGPR 的遮罩被替换为 EXEC。不支持输入和输出修改器; 这是一个未键入的操作。

**V\_WRITELANE\_B32****650**

将第一个输入中的标量值写入一个向量寄存器的指定通道,该通道选择在第二个输入中。

```
lane = S1.u32[5 : 0];  
// 巷子选择  
VGPR[lane][VDST.u32] = S0.b32
```

说明

覆盖 VGPR 写入的 EXEC 掩码。 不支持输入和输出修改器; 这是一个未键入的操作。

**V\_BCNT\_U32\_B32****651**

计算向量输入中的"1"位数,并将结果存储到向量寄存器中。

```
tmp = S1.u32;  
for i in 0 : 31 do  
    tmp += S0[i].u32;  
    // 计数我咬  
endfor;  
D0.u32 = tmp
```

**V\_MBCNT\_LO\_U32\_B32****652**

每一个道 0 QQ N < 32 检查第一个输入中最不显著的位数, 并计算这些位数中有多少是"1"。 对于每个道 32 QQ N < 64, 第一个输入中的所有"1" 位数都计算在内。 在第二个输入中的值中加上这个数, 并将结果存储到向量寄存器中。

结合 V\_MBCNT\_HI\_U32\_B32,并以向量条件码作为输入,这计算了已设置其向量条件码位的当前行通道号或以下的通道数。

```
ThreadMask = (1LL << laneId.u32) - 1LL;  
MaskedValue = (S0.u32 & ThreadMask[31 : 0].u32);  
tmp = S1.u32;  
for i in 0 : 31 do  
    tmp += MaskedValue[i] == 1'1U ? 1U : 0U  
endfor;  
D0.u32 = tmp
```

说明

另见 V\_MBCNT\_HI\_U32\_B32.

**V\_MBCNT\_HI\_U32\_B32****653**

对于每 32  $Q \leq N < 64$  通道,请检查第一次输入中最不重要的位数,并计算这些位数中有多少是"1"。对于 0  $Q \leq N < 32$  通道位置,不检查位数,计数为零。在第二次输入中将这个数加到值中,并将结果存储到向量寄存器中。

结合 V\_MBCNT\_LO\_U32\_B32,并以向量条件码作为输入,这计算了已设置其向量条件码位的当前行通道号或以下的通道数。

```
ThreadMask = (1LL << laneId.u32) - 1LL;
MaskedValue = (S0.u32 & ThreadMask[63 : 32].u32);
tmp = S1.u32;
for i in 0 : 31 do
    tmp += MaskedValue[i] == 1 ? 1U : 0U
endfor;
D0.u32 = tmp
```

说明

计算各通道在 0..63 范围内位置的示例:

```
v_mbcnt_lo_u32_b32 v0, -1, 0
v_mbcnt_hi_u32_b32 v0, -1, v0
// v0 现在包含通道 ld
```

示例计算每条通道位置在 VCC 位设置的所有通道列表中,其中首条通道与 VCC 位被分配为 1 号通道,第二条通道与 VCC 位被分配为 2 号通道等:

```
v_mbcnt_lo_u32_b32 v0, vcc_lo, 0
v_mbcnt_hi_u32_b32 v0, vcc_hi, v0 // 说明 vcc_hi 传入第二指令
// v0 现在包含通道之间的位置, VCC=1
```

另见 V\_MBCNT\_LO\_U32\_B32.

**V\_LSHLREV\_B64****655**

鉴于第一个向量输入中的位移计数,计算第二个向量输入中左侧的逻辑位移,并将结果存储到一个向量寄存器中。

```
D0.u64 = (S1.u64 << S0[5 : 0].u32)
```

**V\_LSHRREV\_B64****656**

鉴于第一个向量输入中的位移计数,计算第二个向量输入的逻辑位移右侧,并将结果存储到一个向量寄存器中。

```
D0.u64 = (S1.u64 >> S0[5 : 0].u32)
```

## V\_ASHRREV\_I64

657

鉴于第一个向量输入中的位移计数,计算第二个向量输入的算术右移(保留符号位),并将结果存储到向量寄存器中。

```
D0.i64 = (S1.i64 >> S0[5 : 0].u32)
```

## V\_TRIG\_PREOP\_F64

658

使用第二个输入中选择的整数段查找一个  $2/\pi$  的 53 位元段。从第一个双精度浮点输入中缩放中间结果,并将双精度浮点输入结果存储到向量寄存器中。

此操作返回一个对齐的双精度段  $2/\pi$ , 用于在浮点输入上进行三角参数还原。可以使用第一个输入访问多个段。四舍五入是零。大型浮点输入(带有  $\text{exponent} > 1968$ ) 被缩放, 以避免因非正常化而失去精度。

```
shift = 32'I(S1[4 : 0].u32) * 53;
if exponent(S0.f64) > 1077 then
    shift += exponent(S0.f64) - 1077
endif;
// (2.0/PI) == 0.{b_1200, b_1199, b_1198, ..., b_1, b_0}
// b_1200 是 2.0/PI 的小数部分的 MSB
// 左移操作显示带入的位数
// 输入数字的全部部分。
// 只保留了全部结果。
result = 64'F((1201'B(2.0 / PI)[1200 : 0] << shift.u32) & 1201'0xfffffffffffff);
scale = -53 - shift;
if exponent(S0.f64) >= 1968 then
    scale += 128
endif;
D0.f64 = ldexp(result, scale)
```

### 说明

为了更全面地处理三角参数减少问题,可在网上查阅 " 巨大参数减少参数:好到最后一点 " (K.C.Ng et.al,1992 年 3 月)。

**V\_BFM\_B32****659**

计算一个字段偏移和大小的比特菲尔德掩码,并将结果存储到向量寄存器中。

```
D0.u32 = (((1U << S0[4 : 0].u32) - 1U) << S1[4 : 0].u32)
```

**V\_CVT\_PKNORM\_I16\_F32****660**

从两个单精度浮点输入转换成一个有符号的普通短片,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].i16 = f32_to_snorm(S0.f32);
tmp[31 : 16].i16 = f32_to_snorm(S1.f32);
D0 = tmp.b32
```

**V\_CVT\_PKNORM\_U16\_F32****661**

从两个单精度浮点输入转换成一个无符号的普通短片,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = f32_to_unorm(S0.f32);
tmp[31 : 16].u16 = f32_to_unorm(S1.f32);
D0 = tmp.b32
```

**V\_CVT\_PKRTZ\_F16\_F32****662**

将两个单精度浮点数输入转换成一个用朝零舍入语义的半精度浮点数(忽略当前四舍五入模式),并将结果存储到向量寄存器中。

```
prev_mode = ROUND_MODE;
ROUND_MODE = ROUND_TOWARD_ZERO;
tmp[15 : 0].f16 = f32_to_f16(S0.f32);
tmp[31 : 16].f16 = f32_to_f16(S1.f32);
D0 = tmp.b32;
ROUND_MODE = prev_mode;
// 不管硬件中目前的圆形模式设置如何,圆形向零方向移动。
```

**说明**

此操作码用于 16 位压缩导出。 尊重的版本请参见 V\_CVT\_F16\_F32

当前四舍五入模式。

---

**V\_CVT\_PK\_U16\_U32****663**

从两个无符号的 32 位整数输入转换成一个无符号的 16 位整数值,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = u32_to_u16(S0.u32);
tmp[31 : 16].u16 = u32_to_u16(S1.u32);
D0 = tmp.b32
```

---

**V\_CVT\_PK\_I16\_I32****664**

从两个签名的 32 位整数输入转换成一个有符号的 16 位整数值,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].i16 = i32_to_i16(S0.i32);
tmp[31 : 16].i16 = i32_to_i16(S1.i32);
D0 = tmp.b32
```

---

**V\_CVT\_PKNORM\_I16\_F16****665**

从两个半精度浮点输入转换成一个有符号的普通短片,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].i16 = f16_to_snorm(S0.f16);
tmp[31 : 16].i16 = f16_to_snorm(S1.f16);
D0 = tmp.b32
```

---

**V\_CVT\_PKNORM\_U16\_F16****666**

从两个半精度浮点输入转换成一个无符号的普通短片,并将结果存储到向量寄存器中。

```
declare tmp : 32'B;
tmp[15 : 0].u16 = f16_to_unorm(S0.f16);
tmp[31 : 16].u16 = f16_to_unorm(S1.f16);
```



```
D0 = tmp.b32
```

## V\_ADD\_I32

668

添加两个签名的 32 位整数输入,并将结果存储到向量寄存器中。没有随身携带或随身携带支持。

```
D0.i32 = S0.i32 + S1.i32
```

说明

支持饱和(签名的 32 位整数域)。

## V\_SUB\_I32

669

将第二个签名的 32 位整数输入从第一次输入中减掉,并将结果存储到向量寄存器中。没有随身携带或执行支持。

```
D0.i32 = S0.i32 - S1.i32
```

说明

支持饱和(签名的 32 位整数域)。

## V\_ADD\_I16

670

添加两个签名的 16 位整数输入, 并将结果存储到向量寄存器中。 没有随身携带或随身携带支持。

```
D0.i16 = S0.i16 + S1.i16
```

说明

支持饱和(签名的 16 位整数域)。

## V\_SUB\_I16

671

从第一个输入中减去第二个签名的 16 位整数输入,并将结果存储到向量寄存器中。没有载入或载出支持。

```
D0.i16 = S0.i16 - S1.i16
```

说明

支持饱和(签名的 16 位整数域).

## V\_PACK\_B32\_F16

672

将两个半精度浮点值包成单个的 32 位值,并将结果存储到向量寄存器中.

```
D0[31 : 16].f16 = S1.f16;
D0[15 : 0].f16 = S0.f16
```

## V\_MUL\_LEGACY\_F32

673

乘以两个浮点输入,并将结果存储到向量寄存器中。遵循 DX9 规则,当任何物质的 0.0 倍生成 0.0(这与其他 API 不同,当其他输入是无限或 NaN)。

```
if ((64'F(S0.f32) == 0.0) || (64'F(S1.f32) == 0.0)) then
    // DX9 规则, 0.0 * x = 0.0
    D0.f32 = 0.0F
else
    D0.f32 = S0.f32 * S1.f32
endif
```

## V\_CVT\_PK\_FP8\_F32

674

从两个单精度浮点数输入转换成一个装有圆形的 FP8 浮点数到最接近的甚至语义值,并使用 OPSEL 将结果存储为 16 位向量寄存器.

```
prev_mode = ROUND_MODE;
ROUND_MODE = ROUND_NEAREST_EVEN;
if OPSEL[3].u32 == 0U then
    VGPR[laneId][VDST.u32][15 : 0].b16 = { f32_to_fp8(S1.f32), f32_to_fp8(S0.f32) };
    // D0[31:16] 保存完好
else
    VGPR[laneId][VDST.u32][31 : 16].b16 = { f32_to_fp8(S1.f32), f32_to_fp8(S0.f32) };
    // D0[15:0] 保存完好
endif;
ROUND_MODE = prev_mode
```

说明

转到最近的地方忽略了 OMOD 和夹子

## V\_CVT\_PK\_BF8\_F32

675

从两个单精度浮点数输入转换成一个装有圆形的 BF8 浮点数到最近的甚至语义值,并使用 OPSEL 存储结果为 16 比特的向量寄存器。

```
prev_mode = ROUND_MODE;
ROUND_MODE = ROUND_NEAREST_EVEN;
if OPSEL[3].u32 == 0U then
    VGPR[laneId][VDST.u32][15 : 0].b16 = { f32_to_bf8(S1.f32), f32_to_bf8(S0.f32) };
    // D0[31:16] 保存完好
else
    VGPR[laneId][VDST.u32][31 : 16].b16 = { f32_to_bf8(S1.f32), f32_to_bf8(S0.f32) };
    // D0[15:0] 保存完好
endif;
ROUND_MODE = prev_mode
```

说明

转到最近的地方忽略了 OMOD 和夹子

## V\_CVT\_SR\_FP8\_F32

676

从一个单精度浮点输入转换成一个 FP8 值,并使用来自第二个输入的种子数据进行切换。使用 OPSEL 将结果存储为 8 位的向量寄存器,以确定要覆盖的目的地的字节。

```
prev_mode = ROUND_MODE;
ROUND_MODE = ROUND_NEAREST_EVEN;
s = sign(S0.f32);
e = exponent(S0.f32);
m = 23'U(32'U(23'B(mantissa(S0.f32))) + S1[31 : 12].u32);
tmp = float32(s, e, m);
// 给 Mantissa 添加随机值, 在溢出处包裹
if OPSEL[3 : 2].u2 == 2'0U then
    VGPR[laneId][VDST.u32][7 : 0].fp8 = f32_to_fp8(tmp.f32)
elsif OPSEL[3 : 2].u2 == 2'1U then
    VGPR[laneId][VDST.u32][15 : 8].fp8 = f32_to_fp8(tmp.f32)
elsif OPSEL[3 : 2].u2 == 2'2U then
    VGPR[laneId][VDST.u32][23 : 16].fp8 = f32_to_fp8(tmp.f32)
else
    VGPR[laneId][VDST.u32][31 : 24].fp8 = f32_to_fp8(tmp.f32)
endif;
// D 的未写字节保存下来.
ROUND_MODE = prev_mode
```

说明

圆形圆形忽略了 OMOD 和夹子

## V\_CVT\_SR\_BF8\_F32

677

使用二次输入的种子数据将单精度浮点输入转换为 BF8 值, 并进行轮廓切换。使用 OPSEL 将结果存储为 8 比特的向量寄存器, 以确定要覆盖的目的地的字节。

```
prev_mode = ROUND_MODE;
ROUND_MODE = ROUND_NEAREST_EVEN;
s = sign(S0.f32);
e = exponent(S0.f32);
m = 23'U(32'U(23'B(mantissa(S0.f32))) + S1[31 : 11].u32);
tmp = float32(s, e, m);
// 给 Mantissa 添加随机值, 在溢出处包裹
if OPSEL[3 : 2].u2 == 2'0U then
    VGPR[laneId][VDST.u32][7 : 0].bf8 = f32_to_bf8(tmp.f32)
elsif OPSEL[3 : 2].u2 == 2'1U then
    VGPR[laneId][VDST.u32][15 : 8].bf8 = f32_to_bf8(tmp.f32)
elsif OPSEL[3 : 2].u2 == 2'2U then
    VGPR[laneId][VDST.u32][23 : 16].bf8 = f32_to_bf8(tmp.f32)
else
    VGPR[laneId][VDST.u32][31 : 24].bf8 = f32_to_bf8(tmp.f32)
endif;
// D 的未写字节保存下来。
ROUND_MODE = prev_mode
```

说明

圆形圆形忽略了 OMOD 和夹子

## V\_CMP\_CLASS\_F32

16

评估第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数, 一个单精度浮点, 并将每行条件码设置到结果中。将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型, 则函数报告为真:

S1.u[0]值是信号 NAN. S1.u[1]值是静态  
NAN. S1.u[2]值是负无穷值 S1.u[3]值是负正  
常值 S1.u[4]值是负非正常值 S1.u[5]值是负  
零值 S1.u[6]值是正零值 S1.u[7]值是正非正  
常值 S1.u[8]值是正正常值。

S1.u[9]值为正无穷.

```
declare result : 1'U;
if isSignalNaN(64'F(S0.f32)) then
    result = S1.u32[0]
elsif isQuietNaN(64'F(S0.f32)) then
    result = S1.u32[1]
elsif exponent(S0.f32) == 255 then
    // +-INF
    result = S1.u32[sign(S0.f32) ? 2 : 9]
elsif exponent(S0.f32) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f32) ? 3 : 8]
elsif 64'F(abs(S0.f32)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f32) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f32) ? 5 : 6]
endif;
D0.u64[laneId] = result;
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_CLASS\_F32

17

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数, 一个单精度浮点, 并将每行条件码设置到结果中。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型, 则函数报告为真:

S1.u[0]值是一个信号 NaN. S1.u[1]值是一个安静的 NaN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```
declare result : 1'U;
if isSignalNaN(64'F(S0.f32)) then
    result = S1.u32[0]
elsif isQuietNaN(64'F(S0.f32)) then
    result = S1.u32[1]
elsif exponent(S0.f32) == 255 then
    // +-INF
    result = S1.u32[sign(S0.f32) ? 2 : 9]
```

```

elseif exponent(S0.f32) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f32) ? 3 : 8]
elseif 64'F(abs(S0.f32)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f32) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f32) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result

```

## V\_CMP\_CLASS\_F64

18

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数字类函数,双精度浮点,并将每行条件码设置在结果中。将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```

declare result : 1'U;
if isSignalNAN(S0.f64) then
    result = S1.u32[0]
elseif isQuietNAN(S0.f64) then
    result = S1.u32[1]
elseif exponent(S0.f64) == 2047 then
    // +-INF
    result = S1.u32[sign(S0.f64) ? 2 : 9]
elseif exponent(S0.f64) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f64) ? 3 : 8]
elseif abs(S0.f64) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f64) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f64) ? 5 : 6]
endif;
D0.u64[laneId] = result;

```

```
// D0 = VCC 在 VOPC 编码中。
```

## V\_CMPX\_CLASS\_F64

19

评价在第一个输入的第二个输入中指定为 10 bit 掩码的 IEEE 数字类函数, 双精度浮点数, 并设定每行条件码为结果。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型, 则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值。

```
declare result : 1'U;
if isSignalNAN(S0.f64) then
    result = S1.u32[0]
elseif isQuietNAN(S0.f64) then
    result = S1.u32[1]
elseif exponent(S0.f64) == 2047 then
    // +-INF
    result = S1.u32[sign(S0.f64) ? 2 : 9]
elseif exponent(S0.f64) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f64) ? 3 : 8]
elseif abs(S0.f64) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f64) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f64) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result
```

## V\_CMP\_CLASS\_F16

20

评价第一个输入的第二个输入中指定的 IEEE 数字类函数为 10 bit 掩码、半精度浮点数, 并将每行条件码设置到结果中。将结果存储到 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN. S1.u[2]值是一个负无穷值 S1.u[3]值是一个负正常值 S1.u[4]值是一个负非正常值 S1.u[5]值是一个负零值 S1.u[6]值是一个正无穷值 S1.u[7]值是一个正非正常值 S1.u[8]值是一个正正常值 S1.u[9]值是一个正无穷值.

```
declare result : 1'U;
if isSignalNAN(64'F(S0.f16)) then
    result = S1.u32[0]
elseif isQuietNAN(64'F(S0.f16)) then
    result = S1.u32[1]
elseif exponent(S0.f16) == 31 then
    // +-INF
    result = S1.u32[sign(S0.f16) ? 2 : 9]
elseif exponent(S0.f16) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f16) ? 3 : 8]
elseif 64'F(abs(S0.f16)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f16) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f16) ? 5 : 6]
endif;
D0.u64[laneId] = result;
// D0 = VCC 在 VOPC 编码中.
```

说明

注意 S1 的格式为 f16,因为浮点字面常数被解释为这个 opcode 的 16 位值.

## V\_CMPX\_CLASS\_F16

21

评价第一个输入的第二个输入中指定为 10 位掩码的 IEEE 数值类函数, 半精度浮点, 并设置每行条件码到结果中。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

如果浮点值是根据以下列表在 10 位掩码中选择的任何数字类型,则函数报告为真:

S1.u[0]值是一个信号 NAN. S1.u[1]值是一个安静的 NAN.



S1.u[2]值为负无穷值. S1.u[3]值为负正常值.  
 S1.u[4]值为负非正常值. S1.u[5]值为负零值.  
 S1.u[6]值为正零值. S1.u[7]值为正非正常值.  
 S1.u[8]值为正正常值. S1.u[9]值为正无穷值.

```
declare result : 1'U;
if isSignalNaN(64'F(S0.f16)) then
    result = S1.u32[0]
elsif isQuietNaN(64'F(S0.f16)) then
    result = S1.u32[1]
elsif exponent(S0.f16) == 31 then
    // +-INF
    result = S1.u32[sign(S0.f16) ? 2 : 9]
elsif exponent(S0.f16) > 0 then
    // +-normal value
    result = S1.u32[sign(S0.f16) ? 3 : 8]
elsif 64'F(abs(S0.f16)) > 0.0 then
    // +-denormal value
    result = S1.u32[sign(S0.f16) ? 4 : 7]
else
    // +-0.0
    result = S1.u32[sign(S0.f16) ? 5 : 6]
endif;
EXEC.u64[laneId] = D0.u64[laneId] = result
```

#### 说明

注意 S1 的格式为 f16,因为浮点字面常数被解释为这个 opcode 的 16 位值.

## V\_CMP\_F\_F16

32

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_LT\_F16

33

将每行条件码设定为 1 iff ,第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 < S1.f16;
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_EQ\_F16

34

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 == S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LE\_F16

35

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 <= S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GT\_F16

36

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 > S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LG\_F16

37

将每段条件码设定为 1 iff, 第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f16 <> S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GE\_F16

38

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.f16 >= S1.f16;
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_O\_F16

39

将每条线路的条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(64'F(S0.f16)) && !isNAN(64'F(S1.f16)));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_U\_F16

40

将每行区间条件码设定为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(64'F(S0.f16)) || isNAN(64'F(S1.f16)));
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NGE\_F16

41

将每行条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 >= S1.f16);
// 用 NAN 输入此操作与 <
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLG\_F16

42

将每段条件码设定为 1 iff, 第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 <> S1.f16);
// 有了 NAN 的输入,这不是相同的操作与
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NGT\_F16****43**

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 > S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLE\_F16****44**

将每行条件码设定为 1 iff, 第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 <= S1.f16);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NEQ\_F16****45**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 == S1.f16);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLT\_F16****46**

将每段条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f16 < S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_TRU\_F16****47**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_F\_F16

48

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LT\_F16

49

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 < S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_EQ\_F16

50

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 == S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LE\_F16

51

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 <= S1.f16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GT\_F16****52**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 > S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LG\_F16****53**

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 <> S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GE\_F16****54**

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f16 >= S1.f16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_O\_F16****55**

将每道条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNAN(64'F(S0.f16)) && !isNAN(64'F(S1.f16)));  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_U\_F16****56**

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNAN(64'F(S0.f16)) || isNAN(64'F(S1.f16)));
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGE\_F16

57

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 >= S1.f16);
// 用 NAN 输入此操作与 <
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLG\_F16

58

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 <> S1.f16);
// 有了 NAN 的输入,这不是相同的操作与
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NGT\_F16

59

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 > S1.f16);
// 用 NAN 输入此操作与 QQ 不同
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NLE\_F16

60

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 <= S1.f16);
// 使用 NAN 输入此操作与 > 不同。
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NEQ\_F16****61**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 == S1.f16);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NLT\_F16****62**

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f16 < S1.f16);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_TRU\_F16****63**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_F32****64**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_F32****65**

将每行条件码设定为 1 iff,第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 < S1.f32;
```



```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_EQ\_F32

66

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 == S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LE\_F32

67

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 <= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GT\_F32

68

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 > S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LG\_F32

69

将每段条件码设定为 1 iff, 第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f32 <> S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GE\_F32

70

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.f32 >= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_O\_F32

71

将每条线路的条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(64'F(S0.f32)) && !isNAN(64'F(S1.f32)));  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_U\_F32

72

将每行区间条件码设定为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(64'F(S0.f32)) || isNAN(64'F(S1.f32)));  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NGE\_F32

73

将每行条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 >= S1.f32);  
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NLG\_F32

74

将每段条件码设定为 1 iff, 第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 <> S1.f32);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NGT\_F32****75**

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 > S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLE\_F32****76**

将每行条件码设定为 1 iff, 第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 <= S1.f32);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NEQ\_F32****77**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 == S1.f32);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLT\_F32****78**

将每段条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f32 < S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_TRU\_F32****79**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_F\_F32

80

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LT\_F32

81

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 < S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_EQ\_F32

82

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 == S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LE\_F32

83

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 <= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GT\_F32****84**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 > S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_LG\_F32****85**

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 <> S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GE\_F32****86**

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f32 >= S1.f32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_O\_F32****87**

将每道条件码设定为 1 iff, 第一个输入可命令第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNAN(64'F(S0.f32)) && !isNAN(64'F(S1.f32)));  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_U\_F32****88**

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNAN(64'F(S0.f32)) || isNAN(64'F(S1.f32)));
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NGE\_F32

89

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 >= S1.f32);  
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NLG\_F32

90

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 <> S1.f32);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NGT\_F32

91

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 > S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NLE\_F32

92

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 <= S1.f32);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NEQ\_F32****93**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 == S1.f32);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NLT\_F32****94**

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f32 < S1.f32);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_TRU\_F32****95**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_F64****96**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_F64****97**

将每行条件码设定为 1 iff,第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 < S1.f64;
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_EQ\_F64

98

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 == S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LE\_F64

99

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 <= S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GT\_F64

100

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 > S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LG\_F64

101

将每段条件码设定为 1 iff, 第一个输入小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.f64 <> S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GE\_F64

102

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。



生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.f64 >= S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

### V\_CMP\_O\_F64

103

将每条线路的条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (!isNAN(S0.f64) && !isNAN(S1.f64));  
// D0 = VCC 在 VOPC 编码中。
```

### V\_CMP\_U\_F64

104

将每行区间条件码设定为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = (isNAN(S0.f64) || isNAN(S1.f64));  
// D0 = VCC 在 VOPC 编码中。
```

### V\_CMP\_NGE\_F64

105

将每行条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 >= S1.f64);  
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中。
```

### V\_CMP\_NLG\_F64

106

将每段条件码设定为 1 iff, 第一个输入不小于或大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 <> S1.f64);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NGT\_F64****107**

将每条通道条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 > S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLE\_F64****108**

将每行条件码设定为 1 iff, 第一个输入不小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 <= S1.f64);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NEQ\_F64****109**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 == S1.f64);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_NLT\_F64****110**

将每段条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = !(S0.f64 < S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_TRU\_F64****111**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_F\_F64

112

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LT\_F64

113

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 < S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_EQ\_F64

114

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 == S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_LE\_F64

115

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 <= S1.f64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GT\_F64****116**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 > S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LG\_F64****117**

将每条线路的条件码设置为 1 iff, 第一次输入小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 <> S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GE\_F64****118**

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.f64 >= S1.f64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_O\_F64****119**

将每道条件码设定为 1 iff, 第一个输入可命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (!isNaN(S0.f64) && !isNaN(S1.f64));  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_U\_F64****120**

将每条线路的条件码设置为 1 iff, 第一个输入不能命令第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = (isNaN(S0.f64) || isNaN(S1.f64));
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NGE\_F64

121

将每条线路的条件码设定为 1 iff, 第一个输入不大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 >= S1.f64);  
// 用 NAN 输入此操作与 <  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NLG\_F64

122

将每道条件码设定为 1 iff, 第一次输入不小于或大于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 <> S1.f64);  
// 有了 NAN 的输入,这不是相同的操作与  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NGT\_F64

123

将每条线路的条件码设定为 1 iff, 第一个输入不大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 > S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NLE\_F64

124

将每条线路的条件码设定为 1 iff, 第一次输入不小于或等于第二次输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 <= S1.f64);  
// 使用 NAN 输入此操作与 > 不同。  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NEQ\_F64****125**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 == S1.f64);  
// NAN 输入与操作不同!=  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_NLT\_F64****126**

将每条线路的条件码设定为 1 iff, 第一个输入不小于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = !(S0.f64 < S1.f64);  
// 用 NAN 输入此操作与 QQ 不同  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_TRU\_F64****127**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_I16****160**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_I16****161**

将每行条件码设定为 1 iff,第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 < S1.i16;
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_EQ\_I16

162

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 == S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_LE\_I16

163

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 <= S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_GT\_I16

164

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 > S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_NE\_I16

165

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i16 <> S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMP\_GE\_I16

166

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.i16 >= S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_T\_I16

167

将每道条件码设置为 1。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_F\_U16

168

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_U16

169

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 < S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_U16

170

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 == S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_U16

171



将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 <= S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GT\_U16

172

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 > S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_NE\_U16

173

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 <> S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GE\_U16

174

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u16 >= S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_T\_U16

175

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_I16****176**

将每道条件码设置为 0。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_I16****177**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 < S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_I16****178**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 == S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LE\_I16****179**

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 <= S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GT\_I16****180**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 > S1.i16;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_NE\_I16****181**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 <> S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GE\_I16****182**

将每条线路的条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i16 >= S1.i16;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_T\_I16****183**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_U16****184**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_LT\_U16****185**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 < S1.u16;
```

```
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_EQ\_U16

186

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 == S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_LE\_U16

187

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 <= S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GT\_U16

188

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 > S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_NE\_U16

189

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 <> S1.u16;  
// D0 = VCC 在 VOPC 编码中.
```

## V\_CMPX\_GE\_U16

190

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

进入 EXEC 口罩和 VCC 或缩写表。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u16 >= S1.u16;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_T\_U16

191

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_F\_I32

192

将每条线路的条件码设置为 0。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LT\_I32

193

将每行条件码设定为 1 iff ,第一个输入小于第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 < S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_EQ\_I32

194

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 == S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_LE\_I32

195

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 <= S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GT\_I32

196

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 > S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_NE\_I32

197

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 <> S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_GE\_I32

198

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i32 >= S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMP\_T\_I32

199

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_U32****200**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LT\_U32****201**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 < S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_EQ\_U32****202**

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 == S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LE\_U32****203**

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 <= S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_GT\_U32****204**

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 > S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NE\_U32****205**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 <> S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_U32****206**

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u32 >= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_T\_U32****207**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_F\_I32****208**

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_LT\_I32****209**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 < S1.i32;
```



```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_EQ\_I32

210

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 == S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_LE\_I32

211

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 <= S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GT\_I32

212

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 > S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_NE\_I32

213

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 <> S1.i32;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMPX\_GE\_I32

214

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

进入 EXEC 口罩和 VCC 或缩写表。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i32 >= S1.i32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_T\_I32

215

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_F\_U32

216

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_LT\_U32

217

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 < S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_EQ\_U32

218

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中,并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 == S1.u32;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_LE\_U32

219

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 <= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_GT\_U32

220

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 > S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_NE\_U32

221

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 <> S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_GE\_U32

222

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u32 >= S1.u32;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_T\_U32

223

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中, 并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_I64****224**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LT\_I64****225**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 < S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_EQ\_I64****226**

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 == S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_LE\_I64****227**

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 <= S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_GT\_I64****228**

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 > S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMP\_NE\_I64****229**

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 <> S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_GE\_I64****230**

将每段条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.i64 >= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_T\_I64****231**

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_F\_U64****232**

将每条线路的条件码设置为 0。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMP\_LT\_U64****233**

将每行条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 < S1.u64;
```

```
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_EQ\_U64

234

将每段条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 == S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_LE\_U64

235

将每条线路的条件码设定为 1 iff, 第一个输入小于或等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 <= S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GT\_U64

236

将每行条件码设定为 1 iff, 第一个输入大于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 > S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_NE\_U64

237

将每条线路的条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = S0.u64 <> S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

### V\_CMP\_GE\_U64

238

将每行条件码设定为 1 iff, 第一个输入大于或等于第二个输入。

生成 VCC 或标量寄存器。

```
D0.u64[laneId] = S0.u64 >= S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMP\_T\_U64

239

将每道条件码设置为 1. 将结果存储到 VCC 或标量寄存器中。

```
D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_F\_I64

240

将每道条件码设置为 0. 将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_LT\_I64

241

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 < S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_EQ\_I64

242

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 == S1.i64;  
// D0 = VCC 在 VOPC 编码中。
```

---

### V\_CMPX\_LE\_I64

243

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 <= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_GT\_I64

244

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 > S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_NE\_I64

245

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 <> S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_GE\_I64

246

将每条线路的条件码设定为 1 iff, 第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.i64 >= S1.i64;  
// D0 = VCC 在 VOPC 编码中.
```

---

### V\_CMPX\_T\_I64

247

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中, 并保存到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```



**V\_CMPX\_F\_U64****248**

将每道条件码设置为 0。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'0U;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LT\_U64****249**

将每道条件码设定为 1 iff, 第一个输入小于第二个输入。将结果存储到 EXEC 面具和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 < S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_EQ\_U64****250**

将每条线路的条件码设定为 1 iff, 第一个输入等于第二个输入。将结果存储到 EXEC 面具中, 并存储到 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 == S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_LE\_U64****251**

将每段条件码设定为 1 iff, 第一次输入小于或等于第二次输入。将结果存储到 EXEC 口罩中, 并存储到 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 <= S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_GT\_U64****252**

将每段条件码设置为 1 iff, 第一个输入大于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 > S1.u64;  
// D0 = VCC 在 VOPC 编码中。
```

**V\_CMPX\_NE\_U64****253**

将每条通道条件码设定为 1 iff, 第一个输入不等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 <> S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

**V\_CMPX\_GE\_U64****254**

将每条线路的条件码设定为 1 iff ,第一个输入大于或等于第二个输入。将结果存储到 EXEC 掩码和 VCC 或标量寄存器中。

```
EXEC.u64[laneId] = D0.u64[laneId] = S0.u64 >= S1.u64;  
// D0 = VCC 在 VOPC 编码中.
```

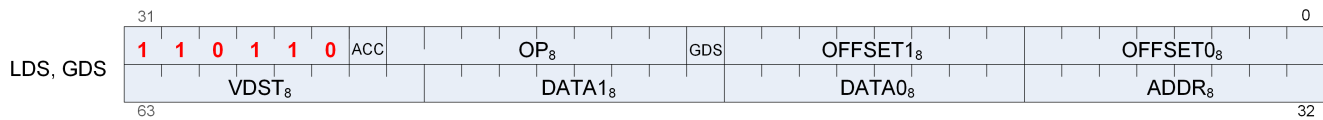
**V\_CMPX\_T\_U64****255**

将每通道条件码设置为 1. 将结果存储到 EXEC 面具中,并保存到 VCC 或标量寄存器中.

```
EXEC.u64[laneId] = D0.u64[laneId] = 1'1U;  
// D0 = VCC 在 VOPC 编码中.
```

## 12.12. LDS & GWS 指令

这组指令操作数据共享内存中的数据，在 VGPRs 与数据共享内存之间传输数据。LDS/GWS 的位字段布局如下：



GWS 指令只在单通道上运行(第一通道在 EXEC 口罩中活动).

where:  
OFFSET0 = Unsigned byte offset added to the address from the ADDR VGPR.  
OFFSET1 = Unsigned byte offset added to the address from the ADDR VGPR.  
GDS = Set if GWS, cleared if LDS.  
OP = DS instructions.  
ADDR = Source LDS address VGPR 0 - 255.  
DATA0 = Source data0 VGPR 0 - 255.  
DATA1 = Source data1 VGPR 0 - 255.  
VDST = Destination VGPR 0- 255.



名称中所有带有 RTN 的指令都会返回操作前的内存值.

DS\_ADD\_U32

0

添加两个无符号的 32 位整数值存储在数据寄存器和数据共享中的位置.

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 += DATA.u32;  
RETURN_DATA.u32 = tmp
```

DS\_SUB\_U32

1

从一个数据共享位置上存储的数值中减去一个无符号的 32 位整数值.

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 -= DATA.u32;  
RETURN_DATA.u32 = tmp
```

**DS\_RSUB\_U32****2**

将一个无符号的 32 位整数数值从存储在数据寄存器中的数值中减去存储在一个数据共享位置上的数值。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
MEM[addr].u32 = DATA.u32 - MEM[addr].u32;
RETURN_DATA.u32 = tmp
```

**DS\_INC\_U32****3**

递增数据共享存储中某位置的无符号 32 位整数数值；如果该值超过数据寄存器中的值，则回绕为 0。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp
```

**DS\_DEC\_U32****4**

将一个未署名的 32 位整数数值从数据共享中一个带环绕的位置降为数据寄存器中的一个值,如果该减值产生负值的话。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
RETURN_DATA.u32 = tmp
```

**DS\_MIN\_I32****5**

选择两个签名的 32 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src < tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

**DS\_MAX\_I32****6**

选择最多两个签名的 32 位整数输入, 给定两个存储在数据寄存器和数据共享中的位置。用选定的值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

**DS\_MIN\_U32****7**

选择两个无符号的 32 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

**DS\_MAX\_U32****8**

选择两个无符号的 32 位整数输入的最大值,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

**DS\_AND\_B32****9**

以位法计算并给出两个无符号的 32 位整数值,存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
```

```
RETURN_DATA.b32 = tmp
```

## DS\_OR\_B32

10

计算位元 OR 给定两个无符号的 32 位整数值存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b32;  
MEM[addr].b32 = (tmp | DATA.b32);  
RETURN_DATA.b32 = tmp
```

## DS\_XOR\_B32

11

在数据寄存器和数据共享位置上计算两个无符号的 32 位整数值。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b32;  
MEM[addr].b32 = (tmp ^ DATA.b32);  
RETURN_DATA.b32 = tmp
```

## DS\_MSKOR\_B32

12

在数据共享中无符号的 32 位整数位置上计算掩码的位元 OR, 给数据寄存器中的掩码值和位元 OR。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b32;  
MEM[addr].b32 = ((tmp & ~DATA.b32) | DATA2.b32);  
RETURN_DATA.b32 = tmp
```

## DS\_WRITE\_B32

13

从向量输入寄存器存储 32 位数据到数据共享。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
MEM[addr + OFFSET.u32].b32 = DATA[31 : 0]
```

**DS\_WRITE2\_B32****14**

从一个向量输入寄存器存储 32 位数据,然后从第二个向量输入寄存器内存存储 32 位数据进入数据共享。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET0.u32 * 4U].b32 = DATA[31 : 0];
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET1.u32 * 4U].b32 = DATA2[31 : 0]
```

**DS\_WRITE2ST64\_B32****15**

从一个向量输入寄存器存储 32 位数据,然后从第二个向量输入寄存器内存存储 32 位数据到数据共享。将每个向量输入寄存器作为索引处理,并乘以 64 个元素(256 字节),为每个 DS 地址生成一个偏移。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET0.u32 * 256U].b32 = DATA[31 : 0];
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET1.u32 * 256U].b32 = DATA2[31 : 0]
```

**DS\_CMPST\_B32****16**

将数据比较寄存器中无符号的 32 位整数值与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
src = DATA2.b32;
cmp = DATA.b32;
MEM[addr].b32 = tmp == cmp ? src : tmp;
RETURN_DATA.b32 = tmp
```

**说明**

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反。

**DS\_CMPST\_F32****17**

将数据比较寄存器中的单精度浮点值与数据共享中的位置进行比较,如果比较是等值的,则修改数据源寄存器位置中具有值的内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA2.f32;
cmp = DATA.f32;
MEM[addr].f32 = tmp == cmp ? src : tmp;
RETURN_DATA.f32 = tmp
```

#### 说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反.

---

### DS\_MIN\_F32

18

选择至少两个单精度浮点数输入, 因为有两个数值存储在数据寄存器中, 一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA.f32;
MEM[addr].f32 = src < tmp ? src : tmp;
RETURN_DATA.f32 = tmp
```

#### 说明

浮点比较手柄 NAN/INF/denorm.

---

### DS\_MAX\_F32

19

选择最多两个单精度浮点数输入,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA.f32;
MEM[addr].f32 = src > tmp ? src : tmp;
RETURN_DATA.f32 = tmp
```

#### 说明

浮点比较手柄 NAN/INF/denorm.

---

### DS\_NOP

20

什么都别做。



**DS\_ADD\_F32****21**

添加两个存储在数据寄存器中的单精度浮点值和一个数据共享位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
MEM[addr].f32 += DATA.f32;
RETURN_DATA.f32 = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**DS\_PK\_ADD\_F16****23**

将数据寄存器中打包的两个半精度浮点分量，分别加到数据共享存储中对应位置的值上。

```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].f16 = tmp[31 : 16].f16 + src[31 : 16].f16;
dst[15 : 0].f16 = tmp[15 : 0].f16 + src[15 : 0].f16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**DS\_PK\_ADD\_BF16****24**

在数据寄存器中添加一个装有 2 个组件的 BF16 浮点值到数据共享中的位置。

```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].bf16 = tmp[31 : 16].bf16 + src[31 : 16].bf16;
dst[15 : 0].bf16 = tmp[15 : 0].bf16 + src[15 : 0].bf16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**DS\_WRITE\_ADDTID\_B32****29**

从向量输入寄存器存储 32 位数据到数据共享。内存基址作为即时值提供,并使用通道 ID 作为抵消。

```
declare OFFSET0 : 8'U;  
declare OFFSET1 : 8'U;  
MEM[32'I({ OFFSET1, OFFSET0 } + M0[15 : 0]) + laneID.i32 * 4].u32 = DATA0.u32
```

**DS\_WRITE\_B8****30**

从向量寄存器存储 8 位数据到数据共享。

```
MEM[ADDR].b8 = DATA[7 : 0]
```

**DS\_WRITE\_B16****31**

从向量寄存器存储 16 位数据到数据共享。

```
MEM[ADDR].b16 = DATA[15 : 0]
```

**DS\_ADD\_RTN\_U32****32**

添加两个无符号的 32 位整数值存储在数据寄存器和数据共享中的位置。将数据共享中的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 += DATA.u32;  
RETURN_DATA.u32 = tmp
```

**DS\_SUB\_RTN\_U32****33**

从存储在数据共享位置的数值中减去无符号的 32 位整数值。将原始值从数据共享存储到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 -= DATA.u32;
```

```
RETURN_DATA.u32 = tmp
```

## DS\_RSUB\_RTN\_U32

34

将一个无符号的 32 位整数数值从数据寄存器中存储的数值中减去存储在一个数据共享位置中的数值。将原始值从数据共享中存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 = DATA.u32 - MEM[addr].u32;  
RETURN_DATA.u32 = tmp
```

## DS\_INC\_RTN\_U32

35

递增数据共享存储中某位置的无符号 32 位整数值；如果该值超过数据寄存器中的值，则回绕为 0。将数据共享存储中的原值写入向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
src = DATA.u32;  
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;  
RETURN_DATA.u32 = tmp
```

## DS\_DEC\_RTN\_U32

36

将一个未署名的 32 位整数值从数据共享中一个带包片的位置降为数据寄存器中的一个值,如果该值降为负值。将数据共享中的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u32;  
src = DATA.u32;  
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;  
RETURN_DATA.u32 = tmp
```

## DS\_MIN\_RTN\_I32

37

选择最少两个签名的 32 位整数输入,前提是有两个值存储在数据寄存器中,一个位置位于数据共享中。用选定的值更新数据共享。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src < tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

## DS\_MAX\_RTN\_I32

38

选择最多两个签名的 32 位整数输入, 因为有两个数值存储在数据寄存器中, 一个位置位于数据共享器中。用选定的值更新数据共享器。将原始值从数据共享器存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

## DS\_MIN\_RTN\_U32

39

选择两个无符号的 32 位整数输入的最小值, 因为在数据寄存器和数据共享位置中存储两个值。用选定的值更新数据共享。将数据共享的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## DS\_MAX\_RTN\_U32

40

选择最多两个无符号的 32 位整数输入, 给定两个存储在数据寄存器和数据共享位置。用选定值更新数据共享。将数据共享的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

**DS\_AND\_RTN\_B32****41**

以位法计算并给出了两个无符号的 32 位整数值, 存储在数据寄存器和数据共享器中的位置。将原始值从数据共享器存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
RETURN_DATA.b32 = tmp
```

**DS\_OR\_RTN\_B32****42**

计算位元 OR 给定了两个无符号的 32 位整数值存储在数据寄存器和数据共享器中的位置。将数据共享器的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp
```

**DS\_XOR\_RTN\_B32****43**

计算位元 XOR 给定了两个无符号的 32 位整数值, 存储在数据寄存器和数据共享器中的位置。将原始值从数据共享器存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp
```

**DS\_MSKOR\_RTN\_B32****44**

在数据共享中无符号的 32 位整数位置上计算掩码的位元 OR, 给数据寄存器中的掩码值和位元 OR。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = ((tmp & ~DATA.b32) | DATA.b32);
RETURN_DATA.b32 = tmp
```

**DS\_WRXCHG\_RTN\_B32**

45

在数据寄存器中以数据共享中的位置交换一个无符号的 32 位整数值。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

**DS\_WRXCHG2\_RTN\_B32**

46

在数据寄存器中用数据共享的两个位置交换两个无符号的 32 位整数值。

```
addr1 = ADDR_BASE.u32 + OFFSET0.u32 * 4U;
addr2 = ADDR_BASE.u32 + OFFSET1.u32 * 4U;
tmp1 = MEM[addr1].b32;
tmp2 = MEM[addr2].b32;
MEM[addr1].b32 = DATA.b32;
MEM[addr2].b32 = DATA2.b32;
// 注意 DATA2 可以是任何其他的登记簿
RETURN_DATA[31 : 0] = tmp1;
RETURN_DATA[63 : 32] = tmp2
```

**DS\_WRXCHG2ST64\_RTN\_B32**

47

在数据寄存器中将两个无符号的 32 位整数值与两个位置在数据共享中交换。将每个位置作为索引处理,并乘以 64 个元素(256 字节)的阶梯,以产生每个 DS 地址的偏移。

```
addr1 = ADDR_BASE.u32 + OFFSET0.u32 * 256U;
addr2 = ADDR_BASE.u32 + OFFSET1.u32 * 256U;
tmp1 = MEM[addr1].b32;
tmp2 = MEM[addr2].b32;
MEM[addr1].b32 = DATA.b32;
MEM[addr2].b32 = DATA2.b32;
// 注意 DATA2 可以是任何其他的登记簿
RETURN_DATA[31 : 0] = tmp1;
RETURN_DATA[63 : 32] = tmp2
```

**DS\_CMPST\_RTN\_B32**

48

将数据比较寄存器中无符号的 32 位整数值与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```

addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b32;
src = DATA2.b32;
cmp = DATA.b32;
MEM[addr].b32 = tmp == cmp ? src : tmp;
RETURN_DATA.b32 = tmp

```

#### 说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反.

### DS\_CMPST\_RTN\_F32

49

将数据比较寄存器中的单精度浮点值与数据共享中的位置进行比较,如果比较是等值的,则修改数据源寄存器位置中具有值的内存位置.

```

addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA2.f32;
cmp = DATA.f32;
MEM[addr].f32 = tmp == cmp ? src : tmp;
RETURN_DATA.f32 = tmp

```

#### 说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反.

### DS\_MIN\_RTN\_F32

50

选择至少两个单精度浮点输入, 给定两个存储在数据寄存器和数据共享位置的值。 用选定的值更新数据共享。 将原始值从数据共享存储到向量寄存器。

```

addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA.f32;
MEM[addr].f32 = src < tmp ? src : tmp;
RETURN_DATA.f32 = tmp

```

#### 说明

浮点比较手柄 NAN/INF/denorm.

### DS\_MAX\_RTN\_F32

51

选择最多两个单精度浮点数输入, 因为有两个数值存储在数据寄存器中, 一个位置位于数据共享中。用所选值更新数据共享。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
src = DATA.f32;
MEM[addr].f32 = src > tmp ? src : tmp;
RETURN_DATA.f32 = tmp
```

说明

浮点比较手柄 NAN/INF/denorm.

## DS\_WRAP\_RTN\_B32

52

考虑到数据共享位置的分数和向量寄存器的子推算,如果结果不负,则减去两个值;否则,从第二个向量寄存器向内存寄存器位置添加一个值。

这种计算为减法提供了灵活的包式语义。

```
tmp = MEM[ADDR].u32;
MEM[ADDR].u32 = tmp >= DATA.u32 ? tmp - DATA.u32 : tmp + DATA2.u32;
RETURN_DATA = tmp
```

说明

本规范旨在用于环缓冲管理。

## DS\_ADD\_RTN\_F32

53

添加两个存储在数据寄存器中的单精度浮点值和一个数据共享位置。将数据共享的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f32;
MEM[addr].f32 += DATA.f32;
RETURN_DATA.f32 = tmp
```

说明

浮点加法处理 NAN/INF/denorm.



**DS\_READ\_B32****54**

从数据共享中装入 32 位数据到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[31 : 0] = MEM[addr + OFFSET.u32].b32
```

**DS\_READ2\_B32****55**

从一个数据共享的一个位置装入 32 位数据,然后从一个数据共享的一个第二个位置装入 32 位数据,并将结果存储到一个 64 位向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[31 : 0] = MEM[addr + OFFSET0.u32 * 4U].b32;  
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[63 : 32] = MEM[addr + OFFSET1.u32 * 4U].b32
```

**DS\_READ2ST64\_B32****56**

在数据共享中从一个位置装入 32 位数据,然后从另一个位置装入 32 位数据,并将结果存储到一个 64 位向量寄存器中。将每个向量寄存器作为索引处理,并乘以 64 个元素(256 字节)的脚步,为每个 DS 地址生成一个抵消。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[31 : 0] = MEM[addr + OFFSET0.u32 * 256U].b32;  
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[63 : 32] = MEM[addr + OFFSET1.u32 * 256U].b32
```

**DS\_READ\_I8****57**

从数据共享中装入 8 位签名数据,签到 32 位,并将结果存储到向量寄存器中。

```
RETURN_DATA.i32 = 32'I(signext(MEM[ADDR].i8))
```

**DS\_READ\_U8****58**

从数据共享中装入 8 位无符号的数据,0 延伸至 32 位并将结果存储到向量寄存器中。

```
RETURN_DATA.u32 = 32'U({ 24'0U, MEM[ADDR].u8 })
```

## DS\_READ\_I16

59

从数据共享中装入 16 位签名数据,签到 32 位,并将结果存储到向量寄存器中.

```
RETURN_DATA.i32 = 32'I(signext(MEM[ADDR].i16))
```

## DS\_READ\_U16

60

从数据共享中装入 16 位无符号的数据,0 扩展至 32 位,并将结果存储到向量寄存器中.

```
RETURN_DATA.u32 = 32'U({ 16'0U, MEM[ADDR].u16 })
```

## DS\_SWIZZLE\_B32

61

Dword swizzle,没有将数据写入 LDS 内存.

Swizzles 基于偏移掩码和返回的输入线程数据;注释不读或写入 DS 内存库.

注意从无效的线程读取结果为 0x0.

这个 opcode 支持两个特定的模式,即 FFT 和旋转,加上两个基本模式,它们以连续的 4 或 32 个线程组合进行曲折.

FFT 模式 (offset >= 0xe000) 根据 offset[4:0] 重排输入, 以支持 FFT 计算。以 {1, 2, ... 20} 为输入的重排示例如下:

```
Offset[4:0]: Swizzle
0x00: {1, 11, 9, 19, 5, 15, d, 1d, 3, 13, b, 1b, 7, 17, f, 1f, 2, 12, a, 1a, 6, 16, e, 1e, 4, 14, c, 1c, 8, 18, 10, 20}
0x10: {1, 9, 5, d, 3, b, 7, f, 2, a, 6, e, 4, c, 8, 10, 11, 19, 15, 1d, 13, 1b, 17, 1f, 12, 1a, 16, 1e, 14, 1c, 18, 20}
0x1f: No swizzle
```

旋转模式 (offset >= 0xc000 且 offset < 0xe000) 按 offset[9:5] 指定的线程数量移动输入: offset[10] == 0 时向左旋转, offset[10] == 1 时向右旋转。该模式还使用掩码修改旋转结果。例如, mask == 1 时, 左旋会将奇数线程跨过间隔的偶数线程进行交换; 右旋则将偶数线程跨过间隔的奇数线程进行交换。

```
Offset[9:5]: Swizzle
0x01, mask=0, rotate left:
{2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 1a, 1b, 1c, 1d, 1e, 1f, 20, 1}
```

```

0x01, mask=0, rotate right:
{20,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f,10,11,12,13,14,15,16,17,18,19,1a,1b,1c,1d,1e,1f}
0x01, mask=1, rotate left:
{1,4,3,6,5,8,7,a,9,c,b,e,d,10,f,12,11,14,13,16,15,18,17,1a,19,1c,1b,1e,1d,20,1f,2}
0x01, mask=1, rotate right:
{1f,2,1,4,3,6,5,8,7,a,9,c,b,e,d,10,f,12,11,14,13,16,15,18,17,1a,19,1c,1b,1e,1d,20}

```

如果偏移 < 0xc000, 则使用基于 offset[15] 的基本 swizzle 模式之一. 如果 offset[15] == 1, 连续的 4 个线条组合在一起, 如果 offset[15] == 0, 所有 32 个线条一起被扭曲。

第一个基本的 swizzle 模式(当 offset[15] == 1)允许一组连续 4 个线程的完整数据共享. 4 组内部的任何线程都可以从对应的偏移位点指定的 4 组内部的其他线程中获取数据 -- [1:0] 对于第一线程,[3:2] 对于第二线程,[5:4] 对于第三线程,[7:6] 对于第四线程,[7:6] 请注意, 偏移位点适用于波前方内所有 4 组;因此如果 offset[1:0] == z == 1, 那么 Three0 抓取线程 1, 线索 4 抓取线程 5 等.

第二个基本的 swizzle 模式(当 offset[15] == 0)允许在 32 个连续线程之间进行有限的数据共享。在此情况下, 偏移用于指定一个 5 位 xor mask, 5 位或 mask, 以及 5 位和 mask 生成线程映射。请注意, 偏移位数适用于波前的每组 32 个。线程映射的细节如下。一些实例使用:

SWAPX16 : xor\_mask = 0x10, or\_mask = 0x00, and\_mask = 0x1f

SWAPX8 : xor\_mask = 0x08, or\_mask = 0x00, and\_mask = 0x1f

SWAPX4 : xor\_mask = 0x04, or\_mask = 0x00, and\_mask = 0x1f

SWAPX2 : xor\_mask = 0x02, or\_mask = 0x00, and\_mask = 0x1f

SWAPX1 : xor\_mask = 0x01, or\_mask = 0x00, and\_mask = 0x1f

REVERSEX32 : xor\_mask = 0x1f, or\_mask = 0x00, and\_mask = 0x1f

REVERSEX16 : xor\_mask = 0x0f, or\_mask = 0x00, and\_mask = 0x1f

REVERSEX8 : xor\_mask = 0x07, or\_mask = 0x00, and\_mask = 0x1f

REVERSEX4 : xor\_mask = 0x03, or\_mask = 0x00, and\_mask = 0x1f

REVERSEX2 : xor\_mask = 0x01 or\_mask = 0x00, and\_mask = 0x1f

BCASTX32:xor\_mask=0x00,or\_mask=线条,and\_mask=0x00.

BCASTX16:xor\_mask = 0x00,or\_mask = 线条,and\_mask = 0x10

BCASTX8:xor\_mask=0x00,or\_mask=线条,and\_mask=0x18.

BCASTX4:xor\_mask = 0x00,or\_mask = 线条,and\_mask = 0x1c

BCASTX2: xor\_mask = 0x00, or\_mask = 线条, and\_mask = 0x1e

伪代码如下:

```

offset = offset1:offset0;

if (offset >= 0xe000) {
    // FFT 分解
    mask = offset[4:0];
    for (i = 0; i < 64; i++) {
        j = reverse_bits(i & 0x1f);
        j = (j >> count_ones(mask));
        j |= (i & mask);
        j |= i & 0x20;
        thread_out[i] = thread_valid[j] ? thread_in[j] : 0;
    }
}

```

```

} elsif (offset >= 0xc000) {
    // 旋转
    rotate = offset[9:5];
    mask = offset[4:0];
    if (offset[10]) {
        rotate = -rotate;
    }
    for (i = 0; i < 64; i++) {
        j = (i & mask) | ((i + rotate) & ~mask);
        j |= i & 0x20;
        thread_out[i] = thread_valid[j] ? thread_in[j] : 0;
    }
}

```

```

} elsif (offset[15]) {
    // 在 4 条连续线程内进行全部数据共享
    for (i = 0; i < 64; i+=4) {
        thread_out[i+0] = thread_valid[i+offset[1:0]]?thread_in[i+offset[1:0]]:0;
        thread_out[i+1] = thread_valid[i+offset[3:2]]?thread_in[i+offset[3:2]]:0;
        thread_out[i+2] = thread_valid[i+offset[5:4]]?thread_in[i+offset[5:4]]:0;
        thread_out[i+3] = thread_valid[i+offset[7:6]]?thread_in[i+offset[7:6]]:0;
    }
}

```

```

} else { // offset[15] == 0
    // 在 32 个连续线程内有限数据共享
    xor_mask = offset[14:10];
    or_mask = offset[9:5];
    and_mask = offset[4:0];
    for (i = 0; i < 64; i++) {
        j = (((i & 0x1f) & and_mask) | or_mask) ^ xor_mask;
        j |= (i & 0x20); // 32 人中的哪一组人
        thread_out[i] = thread_valid[j] ? thread_in[j] : 0;
    }
}
}

```

Forward permute。它不能访问 LDS 内存, 即使没有 LDS 内存被分配到波中, 也可能被调用。它使用 LDS 在波前执行任意的线程旋绕。

注意地址是线形 ID 乘以 4

如果多个来源映射到同一个目的地航道, 标准的 LDS 仲裁规则确定哪个写得胜。

另见 DS\_BPERMUTE\_B32.

```
// VGPR[laneId][index]是 VGPR 内存.
// VDST, ADDR 和 DATA0 来自微码 DS 编码
declare tmp : 32'B[64];
declare OFFSET : 16'U;
declare DATA0 : 32'U;
declare VDST : 32'U;
for i in 0 : 63 do
    tmp[i] = 0x0
endfor;
for i in 0 : 63 do
    // 如果一个源线程被禁用, 它不会传播数据。
    if EXEC[i].u1 then
        // 解甲返乡需要除以 4.
        // 高序位数被忽略.
        dst_lane = (VGPR[i][ADDR].u32 + OFFSET.u32) / 4U % 64U;
        tmp[dst_lane] = VGPR[i][DATA0]
    endif
endfor;
// 将数据复制到目标 VGPRs。若有多个源
// 选择同一目的线程, 最高数字
// 源线条获胜。
for i in 0 : 63 do
    if EXEC[i].u1 then
        VGPR[i][VDST] = tmp[i]
    endif
endfor
```

说明

实例( 简化的 4- 线程波浪面 ):

```
VGPR[Src0] = { A, B, C, D }
VGPR[ADDR] = { 0, 0, 12, 4 }
EXEC = 0xF, OFFSET = 0
VGPR[VDST] = { B, D, 0, C }
```

```
VGPR[Src0] = { A, B, C, D }
VGPR[ADDR] = { 0, 0, 12, 4 }
EXEC = 0xA, OFFSET = 0
VGPR[VDST] = { -, D, -, 0 }
```

**DS\_BPERMUTE\_B32****63**

向后 permute。无法访问 LDS 内存, 即使没有为波分配 LDS 内存, 也可以调用它。它使用 LDS 硬件在波前执行任意的线程环绕。

注意地址是线形 ID 乘以 4

请注意, EXEC 口罩被应用到 VGPR 读写。如果 src\_lane 选择一个禁用线程, 则返回零。

另见 DS\_PERMUTE\_B32.

```
// VGPR[laneld][index]是 VGPR 内存.
// VDST,ADDR 和 DATA0 来自微码 DS 编码
declare tmp : 32'B[64];
declare OFFSET : 16'U;
declare DATA0 : 32'U;
declare VDST : 32'U;
for i in 0 : 63 do
    tmp[i] = 0x0
endfor;
for i in 0 : 63 do
    // 解甲返乡需要除以 4.
    // 高序位数被忽略.
    src_lane = (VGPR[i][ADDR].u32 + OFFSET.u32) / 4U % 64U;
    // EXEC 适用于 VGPR 读本的来源.
    if EXEC[src_lane].u1 then
        tmp[i] = VGPR[src_lane][DATA0]
    endif
endfor;
// 将数据复制到目标 VGPRs。部分源
// 数据可广播到多条通道。
for i in 0 : 63 do
    if EXEC[i].u1 then
        VGPR[i][VDST] = tmp[i]
    endif
endfor
```

说明

实例( 简化的 4- 线程波浪面 ):

```
VGPR[Src0] = { A, B, C, D }
VGPR[Addr] = { 0, 0, 12, 4 }
EXEC = 0xF, OFFSET = 0
VGPR[Vdst] = { A, A, D, B }
```

```
VGPR[Src0] = { A, B, C, D }
VGPR[Addr] = { 0, 0, 12, 4 }
EXEC = 0xA, OFFSET = 0
VGPR[Vdst] = { -, 0, -, B }
```

**DS\_ADD\_U64****64**

添加两个无符号的 64 位整数存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 += DATA.u64;
RETURN_DATA.u64 = tmp
```

**DS\_SUB\_U64****65**

从一个数据共享位置上存储的数值中减去一个无符号的 64 位整数。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 -= DATA.u64;
RETURN_DATA.u64 = tmp
```

**DS\_RSUB\_U64****66**

将一个无符号的 64 位整数数值从数据寄存器中存储的数值中减去存储在一个数据共享位置中的数值。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 = DATA.u64 - MEM[addr].u64;
RETURN_DATA.u64 = tmp
```

**DS\_INC\_U64****67**

将一个无符号的 64 位整数数值从一个数据共享位置加宽加宽加宽到 0,如果值超过数据寄存器中的值。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

**DS\_DEC\_U64****68**

将一个无符号的 64 位整数从数据共享中一个带环绕的位置降为数据寄存器中的一个值,如果该减值产生负值的话。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```

**DS\_MIN\_I64****69**

选择两个签名的 64 位整数输入的最小值,因为有两个值存储在数据寄存器中,一个位置位于数据共享中。用选定的值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**DS\_MAX\_I64****70**

选择最多 2 个签名的 64 位整数输入,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**DS\_MIN\_U64****71**

选择两个无符号的 64 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
```



```
RETURN_DATA.u64 = tmp
```

## DS\_MAX\_U64

72

选择两个无符号的 64 位整数输入的上限,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定的数值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u64;  
src = DATA.u64;  
MEM[addr].u64 = src >= tmp ? src : tmp;  
RETURN_DATA.u64 = tmp
```

## DS\_AND\_B64

73

以位法计算并给出两个无符号的 64 位整数值,存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp & DATA.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_OR\_B64

74

计算位元 OR 给定两个无符号的 64 位整数存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp | DATA.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_XOR\_B64

75

计算比特 XOR 给定了两个无符号的 64 位整数,存储在数据寄存器和数据共享中的位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;
```

```
MEM[addr].b64 = (tmp ^ DATA.b64);
RETURN_DATA.b64 = tmp
```

**DS\_MSKOR\_B64****76**

在数据共享中无符号的 64 位整数位置上,将掩码值和位数计算为 OR。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = ((tmp & ~DATA.b64) | DATA2.b64);
RETURN_DATA.b64 = tmp
```

**DS\_WRITE\_B64****77**

从向量输入寄存器存储 64 位数据到数据共享。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET.u32].b32 = DATA[31 : 0];
MEM[addr + OFFSET.u32 + 4U].b32 = DATA[63 : 32]
```

**DS\_WRITE2\_B64****78**

从一个向量输入寄存器存储 64 位数据,然后从第二个向量输入寄存器内存存储 64 位数据,形成数据共享。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET0.u32 * 8U].b32 = DATA[31 : 0];
MEM[addr + OFFSET0.u32 * 8U + 4U].b32 = DATA[63 : 32];
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET1.u32 * 8U].b32 = DATA2[31 : 0];
MEM[addr + OFFSET1.u32 * 8U + 4U].b32 = DATA2[63 : 32]
```

**DS\_WRITE2ST64\_B64****79**

从一个向量输入寄存器存储了 64 位数据,然后从第二个向量输入寄存器内存存储了 64 位数据,然后将每个向量输入寄存器内存存储到一个数据共享器中。将每个向量输入寄存器作为索引处理,并乘以 64 个元素(256 字节)的阶梯,为每个 DS 地址生成一个向量。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
```

```
MEM[addr + OFFSET0.u32 * 512U].b32 = DATA[31 : 0];
MEM[addr + OFFSET0.u32 * 512U + 4U].b32 = DATA[63 : 32];
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET1.u32 * 512U].b32 = DATA2[31 : 0];
MEM[addr + OFFSET1.u32 * 512U + 4U].b32 = DATA2[63 : 32]
```

**DS\_CMPST\_B64****80**

将数据比较寄存器中无符号的 64 位整数值与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b64;
src = DATA2.b64;
cmp = DATA.b64;
MEM[addr].b64 = tmp == cmp ? src : tmp;
RETURN_DATA.b64 = tmp
```

说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反。

**DS\_CMPST\_F64****81**

将数据比较寄存器中的双精度浮点值与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f64;
src = DATA2.f64;
cmp = DATA.f64;
MEM[addr].f64 = tmp == cmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反。

**DS\_MIN\_F64****82**

选择至少两个双精度浮点数输入, 因为有两个数值存储在数据寄存器和数据共享位置。 用选定的值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
```

```
tmp = MEM[addr].f64;  
src = DATA.f64;  
MEM[addr].f64 = src < tmp ? src : tmp;  
RETURN_DATA.f64 = tmp
```

说明

浮点比较手柄 NAN/INF/denorm.

---

### DS\_MAX\_F64

**83**

选择最多两个双精度浮点数输入, 给两个存储在数据寄存器中的值和一个数据共享位置。用选定的值更新数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].f64;  
src = DATA.f64;  
MEM[addr].f64 = src > tmp ? src : tmp;  
RETURN_DATA.f64 = tmp
```

说明

浮点比较手柄 NAN/INF/denorm.

---

### DS\_WRITE\_B8\_D16\_HI

**84**

将来自向量寄存器高位的 8 位数据存储到数据共享中.

```
MEM[ADDR].b8 = DATA[23 : 16]
```

---

### DS\_WRITE\_B16\_D16\_HI

**85**

从向量寄存器的高位存储 16 位数据到数据共享.

```
MEM[ADDR].b16 = DATA[31 : 16]
```

---

### DS\_READ\_U8\_D16

**86**

从数据共享中装入 8 位无符号的数据,0 扩展至 16 位,并将结果存储到一个向量寄存器的低 16 位.

```
RETURN_DATA[15 : 0].u16 = 16'U({ 8'0U, MEM[ADDR].u8 });  
// RETURN_DATA[31:16]保存完好.
```

---

**DS\_READ\_U8\_D16\_HI****87**

从一个数据共享中装入 8 位无符号的数据,0 扩展至 16 位,并将结果存储到一个向量寄存器的高 16 位.

```
RETURN_DATA[31 : 16].u16 = 16'U({ 8'0U, MEM[ADDR].u8 });  
// RETURN_DATA[15:0]保存完好.
```

---

**DS\_READ\_I8\_D16****88**

从数据共享中装入 8 位签名数据,签名扩展至 16 位,并将结果存储到一个向量寄存器的低 16 位.

```
RETURN_DATA[15 : 0].i16 = 16'I(signext(MEM[ADDR].i8));  
// RETURN_DATA[31:16]保存完好.
```

---

**DS\_READ\_I8\_D16\_HI****89**

从数据共享中装入 8 位签名数据,签名扩展至 16 位,并将结果存储到一个向量寄存器的高 16 位.

```
RETURN_DATA[31 : 16].i16 = 16'I(signext(MEM[ADDR].i8));  
// RETURN_DATA[15:0]保存完好.
```

---

**DS\_READ\_U16\_D16****90**

从数据共享中装入 16 位无符号的数据,并将结果存储到一个向量寄存器的低 16 位.

```
RETURN_DATA[15 : 0].u16 = MEM[ADDR].u16;  
// RETURN_DATA[31:16]保存完好.
```

---

**DS\_READ\_U16\_D16\_HI****91**

从一个数据共享中装入 16 位无符号的数据,并将结果存储到一个向量寄存器的高 16 位.

```
RETURN_DATA[31 : 16].u16 = MEM[ADDR].u16;  
// RETURN_DATA[15:0]保存完好.
```

## DS\_ADD\_F64

92

在数据寄存器中添加双精度浮点值到数据共享中的位置.

```
tmp = MEM[ADDR].f64;  
MEM[ADDR].f64 += DATA.f64;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

## DS\_ADD\_RTN\_U64

96

添加两个无符号的 64 位整数值存储在数据寄存器和数据共享中的位置。将数据共享中的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u64;  
MEM[addr].u64 += DATA.u64;  
RETURN_DATA.u64 = tmp
```

## DS\_SUB\_RTN\_U64

97

将一个无符号的 64 位整数值从一个存储在数据共享位置的数值中减去。将数据共享的原始值存储到一个向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].u64;  
MEM[addr].u64 -= DATA.u64;  
RETURN_DATA.u64 = tmp
```

## DS\_RSUB\_RTN\_U64

98

将无符号的 64 位整数值从存储在数据中的数值中减去存储在数据共享位置中的数值

。将数据共享的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 = DATA.u64 - MEM[addr].u64;
RETURN_DATA.u64 = tmp
```

## DS\_INC\_RTN\_U64

99

将一个无符号的 64 位整数从数据共享中的位置加宽到 0,如果数值超过数据内存中的值。将原始值从数据共享存储到向量内存中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

## DS\_DEC\_RTN\_U64

100

将一个无符号的 64 位整数从数据共享中一个带包片的位置降为数据寄存器中的一个值,如果该值的减少产生负值。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```

## DS\_MIN\_RTN\_I64

101

选择两个签名的 64 位整数输入的最小值,因为有两个数值存储在数据寄存器中,一个位置位于数据共享中。用选定值更新数据共享。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**DS\_MAX\_RTN\_I64****102**

选择最多两个签名的 64 位整数输入, 给两个存储在数据寄存器和数据共享器的位置。用选定的值更新数据共享器。从数据共享器存储原始值到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**DS\_MIN\_RTN\_U64****103**

选择两个无符号的 64 位整数输入的最小值, 因为在数据寄存器和数据共享位置中存储两个值。用选定值更新数据共享。将原始值从数据共享存储到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**DS\_MAX\_RTN\_U64****104**

选择两个无符号的 64 位整数输入的最大值, 因为有两个数值存储在数据寄存器中, 一个位置位于数据共享中。用所选值更新数据共享。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src >= tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**DS\_AND\_RTN\_B64****105**

以位法计算并给出两个无符号的 64 位整数值, 存储在数据寄存器和数据共享器中的位置。将原值从数据共享器存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
```



```
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp & DATA.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_OR\_RTN\_B64

106

计算位元 OR 给定了两个无符号的 64 位整数值存储在数据寄存器和数据共享器中的位置。将数据共享器的原始值存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp | DATA.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_XOR\_RTN\_B64

107

以位法计算 XOR 给定了两个无符号的 64 位整数值, 存储在数据寄存器和数据共享器中的位置。将原始值从数据共享器存储到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp ^ DATA.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_MSKOR\_RTN\_B64

108

在数据共享中无符号的 64 位整数位置上,将掩码值和位数计算为 OR。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = ((tmp & ~DATA.b64) | DATA2.b64);  
RETURN_DATA.b64 = tmp
```

## DS\_WRXCHG\_RTN\_B64

109

在数据寄存器中交换一个无符号的 64 位整数值,其位置为数据共享。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);  
tmp = MEM[addr].b64;
```

```
MEM[addr].b64 = DATA.b64;
RETURN_DATA.b64 = tmp
```

**DS\_WRXCHG2\_RTN\_B64****110**

将数据寄存器中两个无符号的 64 位整数与数据共享的两个位置交换。

```
addr1 = ADDR_BASE.u32 + OFFSET0.u32 * 8U;
addr2 = ADDR_BASE.u32 + OFFSET1.u32 * 8U;
tmp1 = MEM[addr1].b64;
tmp2 = MEM[addr2].b64;
MEM[addr1].b64 = DATA.b64;
MEM[addr2].b64 = DATA2.b64;
// 注意 DATA2 可以是任何其他的登记簿
RETURN_DATA[63 : 0] = tmp1;
RETURN_DATA[127 : 64] = tmp2
```

**DS\_WRXCHG2ST64\_RTN\_B64****111**

将数据寄存器中两个无符号的 64 位整数与两个位置在数据共享中交换。将每个位置作为索引处理,并乘以 64 个元素(256 字节)的阶梯,为每个 DS 地址生成一个偏移。

```
addr1 = ADDR_BASE.u32 + OFFSET0.u32 * 512U;
addr2 = ADDR_BASE.u32 + OFFSET1.u32 * 512U;
tmp1 = MEM[addr1].b64;
tmp2 = MEM[addr2].b64;
MEM[addr1].b64 = DATA.b64;
MEM[addr2].b64 = DATA2.b64;
// 注意 DATA2 可以是任何其他的登记簿
RETURN_DATA[63 : 0] = tmp1;
RETURN_DATA[127 : 64] = tmp2
```

**DS\_CMPST\_RTN\_B64****112**

将数据比较寄存器中无符号的 64 位整数与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].b64;
src = DATA2.b64;
cmp = DATA.b64;
MEM[addr].b64 = tmp == cmp ? src : tmp;
RETURN_DATA.b64 = tmp
```

## 说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反。

**DS\_CMPST\_RTN\_F64****113**

将数据比较寄存器中的双精度浮点值与数据共享中的位置进行比较,如果比较是等值的,则用数据源寄存器寄存器中的一个值修改内存位置。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f64;
src = DATA2.f64;
cmp = DATA.f64;
MEM[addr].f64 = tmp == cmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

## 说明

注意,src 和 cmp 的顺序与 BUFFER\_ATOMIC\_CMPSWAP opcode 相反。

**DS\_MIN\_RTN\_F64****114**

选择至少两个双精度浮点数输入, 因为有两个数值存储在数据寄存器中, 一个位置位于数据共享器中。用所选值更新数据共享器。从数据共享器存储原始值到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src < tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

## 说明

浮点比较手柄 NAN/INF/denorm.

**DS\_MAX\_RTN\_F64****115**

选择最多两个双精度浮点数输入, 设定数据寄存器和数据共享位置的两个值。用所选值更新数据共享。将原始值从数据共享存储到向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, OFFSET0.b32, OFFSET1.b32);
tmp = MEM[addr].f64;
```

```
src = DATA.f64;
MEM[addr].f64 = src > tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

说明

浮点比较手柄 NAN/INF/denorm.

## DS\_READ\_B64

118

从数据共享中装入 64 位数据到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
RETURN_DATA[31 : 0] = MEM[addr + OFFSET.u32].b32;
RETURN_DATA[63 : 32] = MEM[addr + OFFSET.u32 + 4U].b32
```

## DS\_READ2\_B64

119

在一个数据共享中从一个位置装入 64 位数据,然后从一个数据共享中从另一个位置装入 64 位数据,并将结果存储到 128 位向量寄存器中。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
RETURN_DATA[31 : 0] = MEM[addr + OFFSET0.u32 * 8U].b32;
RETURN_DATA[63 : 32] = MEM[addr + OFFSET0.u32 * 8U + 4U].b32;
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
RETURN_DATA[95 : 64] = MEM[addr + OFFSET1.u32 * 8U].b32;
RETURN_DATA[127 : 96] = MEM[addr + OFFSET1.u32 * 8U + 4U].b32
```

## DS\_READ2ST64\_B64

120

从数据共享中一个位置装入 64 位数据,然后从另一个位置装入 64 位数据,并将结果存储到 128 位向量寄存器中。将每个向量寄存器作为索引处理,并乘以 64 个元素(256 字节)的阶梯,为每个 DS 地址生成一个抵消。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
RETURN_DATA[31 : 0] = MEM[addr + OFFSET0.u32 * 512U].b32;
RETURN_DATA[63 : 32] = MEM[addr + OFFSET0.u32 * 512U + 4U].b32;
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
RETURN_DATA[95 : 64] = MEM[addr + OFFSET1.u32 * 512U].b32;
RETURN_DATA[127 : 96] = MEM[addr + OFFSET1.u32 * 512U + 4U].b32
```

**DS\_ADD\_RTN\_F64****124**

在数据寄存器中添加双精度浮点值到数据共享中的位置。将数据共享中的原始值存储到向量寄存器中。

```
tmp = MEM[ADDR].f64;
MEM[ADDR].f64 += DATA.f64;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**DS\_CONDXCHG32\_RTN\_B64****126**

执行 2 个条件书写交换,每个条件书写交换从数据寄存器到数据共享位置的 32 位值,if 是数据值中最显著的位值设置.

```
declare OFFSET0 : 8'U;
declare OFFSET1 : 8'U;
declare RETURN_DATA : 32'U[2];
ADDR = S0.u32;
DATA = S1.u64;
offset = { OFFSET1, OFFSET0 };
ADDR0 = ((ADDR + offset.u32) & 0xfff8U);
ADDR1 = ADDR0 + 4U;
RETURN_DATA[0] = LDS[ADDR0].u32;
if DATA[31] then
    LDS[ADDR0] = { 1'0, DATA[30 : 0] }
endif;
RETURN_DATA[1] = LDS[ADDR1].u32;
if DATA[63] then
    LDS[ADDR1] = { 1'0, DATA[62 : 32] }
endif
```

**DS\_GWS\_SEMA\_RELEASE\_ALL****152**

GDS 只:GWS 资源(rid)通过更新计数器和将指定的资源标注为 semaphore 来处理这个 opcode.

```
// 确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + offset0[5:0];

// 输入资源状态计数器
state.counter[rid] = state.wave_in_queue;
state.type = SEMAPHORE;
```

```
return rd_done; //释放调用波
```

此动作释放所有队列中的波; 如果没有波则没有效果。

## DS\_GWS\_INIT

153

GDS 只:初始化一个屏障或 semaphore 资源.

```
// 确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + offset0[5:0];

// 获取输入中使用的值
index = find_first_valid(vector mask)
value = DATA[thread: index]

// 设置资源状态
state.counter[rid] = lsb(value); //限制 #波
state.flag[rid] = 0;
return rd_done; //释放调用波
```

CAUTION : VGPR 操作要对齐此指令。 仅使用 32 位但硬件将此指示视为 64 位读取.

## DS\_GWS\_SEMA\_V

154

GDS 只:GWS 资源通过更新计数器并标注资源为 semaphore 来处理这个 opcode.

```
//确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + offset0[5:0];

//输入资源状态计数器
state.counter[rid] += 1;
state.type = SEMAPHORE;
return rd_done; //释放调用波
```

如果在此资源中排队, 此动作会释放一波。

## DS\_GWS\_SEMA\_BR

155

GDS 只:GWS 资源表示通过批量释放发送的计数器更新计数器处理这个 opcode,并将资源标注为 semaphore.

```
//确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + offset0[5:0];
```

```

index = find first valid (vector mask)
count = DATA[thread: index];

//在资源状态计数器中添加计数
state.counter[rid] += count;
state.type = SEMAPHORE;
return rd_done; //释放调用波

```

此动作会释放波数, 如果排队, 或当它们从注解的资源到达时, 即刻释放波数。

**CAUTION :** VGPR 操作要对齐此指令。仅使用 32 位但硬件将此指示视为 64 位读取。

## DS\_GWS\_SEMA\_P

156

GDS 只:GWS 资源表示通过排队处理这个 opcode,直到计数器允许放出,然后将资源计数器降为 semaphore.

```

//确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + offset0[5:0];
state.type = SEMAPHORE;
ENQUEUE until(state[rid].counter > 0)
state[rid].counter -= 1;
return rd_done;

```

## DS\_GWS\_BARRIER

157

GDS 只: GWS 资源通过队列处理这个 opcode,直到屏障满足。所需波数作为第一个有效线程的 DATA 传递。

```

//确定 GWS 资源进行工作
rid[5:0] = gds_base[5:0] + OFFSET0[5:0];
index = find first valid (vector mask);
value = DATA[thread: index];

// 输入决定机
state.type[rid] = BARRIER;
if(state[rid].counter <= 0) then
    thread[rid].flag = state[rid].flag;
    ENQUEUE;
    state[rid].flag = !state.flag;
    state[rid].counter = value;
    return rd_done;
else
    state[rid].counter -= 1;
    thread.flag = state[rid].flag;
    ENQUEUE;
endif.

```

由于波浪为下一个屏障提供计数,这个函数每次发生时可以有不同尺寸屏障。

```
// 释放机
if(state.type == BARRIER) then
    if(state.flag != thread.flag) then
        return rd_done;
    endif;
endif.
```

CAUTION : VGPR 操作要对齐此指令。仅使用 32 位但硬件将此指示视为 64 位读取。

## DS\_READ\_ADDTID\_B32

182

从数据共享中装入 32 位数据到向量寄存器。内存基址作为即时值提供,并使用通道 ID 作为抵消。

```
declare OFFSET0 : 8'U;
declare OFFSET1 : 8'U;
RETURN_DATA.u32 = MEM[32'I({ OFFSET1, OFFSET0 } + M0[15 : 0]) + laneID.i32 * 4].u32
```

## DS\_PK\_ADD\_RTN\_F16

183

在数据寄存器中添加一个包装的 2 组件半精度浮点值到数据共享中的位置。将数据共享中的原始值存储到向量寄存器中。

```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].f16 = tmp[31 : 16].f16 + src[31 : 16].f16;
dst[15 : 0].f16 = tmp[15 : 0].f16 + src[15 : 0].f16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

## DS\_PK\_ADD\_RTN\_BF16

184

在数据寄存器中添加一个装有 2 个组件的 BF16 浮点值到数据共享中的位置。将数据共享的原始值存储到向量寄存器中。



```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].bf16 = tmp[31 : 16].bf16 + src[31 : 16].bf16;
dst[15 : 0].bf16 = tmp[15 : 0].bf16 + src[15 : 0].bf16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

## 说明

浮点加法处理 NAN/INF/denorm.

## DS\_CONSUME

189

从数据共享内存中的值减去 count bits(exec\_mask)。GDS 的地址为 M0.base + instr\_offset, LDS 的地址为 instr\_offset。将操作前的值返回到 VGPRs。

## DS\_APPEND

190

添加(count bits(exec\_mask))到 DS 内存(M0.base + instr\_offset),如果 GDS,或者 instr\_offset,如果 LDS。返回操作前值到 VGPRs。

## DS\_WRITE\_B96

222

从向量输入寄存器中存储 96 位数据到数据共享。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET.u32].b32 = DATA[31 : 0];
MEM[addr + OFFSET.u32 + 4U].b32 = DATA[63 : 32];
MEM[addr + OFFSET.u32 + 8U].b32 = DATA[95 : 64]
```

## DS\_WRITE\_B128

223

将 128 位数据从向量输入寄存器存储到数据共享中。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);
MEM[addr + OFFSET.u32].b32 = DATA[31 : 0];
MEM[addr + OFFSET.u32 + 4U].b32 = DATA[63 : 32];
MEM[addr + OFFSET.u32 + 8U].b32 = DATA[95 : 64];
MEM[addr + OFFSET.u32 + 12U].b32 = DATA[127 : 96]
```

**DS\_READ\_B96****254**

从数据共享中装入 96 位数据到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[31 : 0] = MEM[addr + OFFSET.u32].b32;  
RETURN_DATA[63 : 32] = MEM[addr + OFFSET.u32 + 4U].b32;  
RETURN_DATA[95 : 64] = MEM[addr + OFFSET.u32 + 8U].b32
```

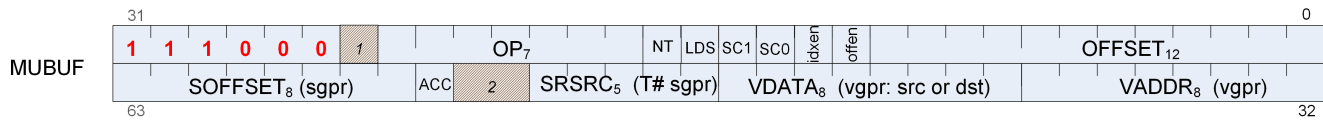
**DS\_READ\_B128****255**

从数据共享中装入 128 位数据到向量寄存器。

```
addr = CalcDsAddr(ADDR.b32, 0x0, 0x0);  
RETURN_DATA[31 : 0] = MEM[addr + OFFSET.u32].b32;  
RETURN_DATA[63 : 32] = MEM[addr + OFFSET.u32 + 4U].b32;  
RETURN_DATA[95 : 64] = MEM[addr + OFFSET.u32 + 8U].b32;  
RETURN_DATA[127 : 96] = MEM[addr + OFFSET.u32 + 12U].b32
```

## 12.13. MUBUF 指令

MUBUF 格式的位域图是:



where:

OFFSET = Unsigned immediate byte offset.  
 OFFEN = Send offset either as VADDR or as zero..  
 IDXEN = Send index either as VADDR or as zero.  
 LDS = Data read from/written to LDS or VGPR.  
 OP = Instruction Opcode.  
 VADDR = VGPR address source.  
 VDATA = Destination vector GPR.  
 SRSRC = Scalar GPR that specifies resource constant.  
 ACC = Return to ACC VGPRs  
 SC = Scope  
 NT = Non-Temporal  
 SOFFSET = Byte offset added to the memory address of an SGPR.

### BUFFER\_LOAD\_FORMAT\_X

0

从缓冲表面装入 1 组件格式化数据, 将数据转换为 32 位集成或浮点格式, 然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式

```

### BUFFER\_LOAD\_FORMAT\_XY

1

从缓冲表面装入 2 个组件格式化的数据, 将数据转换为 32 位集成或浮点格式, 然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()])

```

### BUFFER\_LOAD\_FORMAT\_XYZ

2

从缓冲表面装入 3 个组件格式化的数据,将数据转换为 32 位集成或浮点格式,然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()]);
VDATA[95 : 64].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetZ()]);
```

### BUFFER\_LOAD\_FORMAT\_XYZW

3

从缓冲表面装入 4 组件格式化的数据, 将数据转换为 32 位集成或浮点格式, 然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()]);
VDATA[95 : 64].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetZ()]);
VDATA[127 : 96].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetW()]);
```

### BUFFER\_STORE\_FORMAT\_X

4

将 32 位数据从向量输入寄存器转换为 1 个组件格式化的数据, 并将数据存储到缓冲表面。指令指定了表面的数据格式, 凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
```

### BUFFER\_STORE\_FORMAT\_XY

5

从向量输入寄存器中将 64 位数据转换成 2 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32)
```

**BUFFER\_STORE\_FORMAT\_XYZ****6**

从向量输入寄存器中将 96 位数据转换成 3 个组件格式化的数据,并将数据存储到缓冲面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32);
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(VDATA[95 : 64].b32);
```

**BUFFER\_STORE\_FORMAT\_XYZW****7**

将 128 比特的数据从向量输入寄存器转换成 4 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32);
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(VDATA[95 : 64].b32);
MEM[addr + ChannelOffsetW()] = ConvertToFormat(VDATA[127 : 96].b32);
```

**BUFFER\_LOAD\_FORMAT\_D16\_X****8**

从缓冲面上装入 1 组件格式化数据,将数据转换为 16 位集成或浮点格式,然后将结果存储到 32 位向量寄存器的低 16 位。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
// VDATA[31:16].b16 保存完好.
```

**BUFFER\_LOAD\_FORMAT\_D16\_XY****9**

从缓冲面上装入 2 个组件格式化的数据,将数据转换为装入 16 位元集成或浮点格式,然后将结果存储到向量寄存器中。资源描述器指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
```

```
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()])))
```

## BUFFER\_LOAD\_FORMAT\_D16\_XYZ

10

从缓冲面上装入 3 个组件格式化的数据,将数据转换为装入 16 位集成或浮点格式,然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()]));
VDATA[47 : 32].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetZ()]));
// VDATA[63:48].b16 保存完好。
```

## BUFFER\_LOAD\_FORMAT\_D16\_XYZW

11

从缓冲表面装入 4 组件格式化的数据,将数据转换为 16 位集成或浮点格式,然后将结果存储到向量寄存器中。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()]));
VDATA[47 : 32].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetZ()]));
VDATA[63 : 48].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetW()]));
```

## BUFFER\_STORE\_FORMAT\_D16\_X

12

将 32 位向量输入寄存器的低 16 位数据转换成 1 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
```

## BUFFER\_STORE\_FORMAT\_D16\_XY

13

将 32 位数据从向量输入寄存器转换为 2 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16))
```

## BUFFER\_STORE\_FORMAT\_D16\_XYZ

14

从向量输入寄存器中将 48 位数据转换成 3 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16));
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(32'B(VDATA[47 : 32].b16))
```

## BUFFER\_STORE\_FORMAT\_D16\_XYZW

15

将 64 位数据从向量输入寄存器转换成 4 个组件格式化的数据,并将数据存储到缓冲面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16));
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(32'B(VDATA[47 : 32].b16));
MEM[addr + ChannelOffsetW()] = ConvertToFormat(32'B(VDATA[63 : 48].b16))
```

## BUFFER\_LOAD\_UBYTE

16

从缓冲表面装入 8 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 24'0U, MEM[addr].u8 })
```

## BUFFER\_LOAD\_SBYTE

17

从缓冲表面装入 8 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i8))
```

---

## **BUFFER\_LOAD\_USHORT**

**18**

从缓冲表面装入 16 位无符号的数据,0 扩展至 32 位并将结果存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 16'0U, MEM[addr].u16 })
```

---

## **BUFFER\_LOAD\_SSHORT**

**19**

从缓冲表面装入 16 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i16))
```

---

## **BUFFER\_LOAD\_DWORD**

**20**

从缓冲表面装入 32 位数据到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0] = MEM[addr].b32
```

---

## **BUFFER\_LOAD\_DWORDX2**

**21**

从缓冲表面装入 64 位数据到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0] = MEM[addr].b32;
VDATA[63 : 32] = MEM[addr + 4U].b32
```



**BUFFER\_LOAD\_DWORDX3****22**

将 96 位数据从缓冲表面装入向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0] = MEM[addr].b32;
VDATA[63 : 32] = MEM[addr + 4U].b32;
VDATA[95 : 64] = MEM[addr + 8U].b32
```

**BUFFER\_LOAD\_DWORDX4****23**

从缓冲表面装入 128 位数据到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0] = MEM[addr].b32;
VDATA[63 : 32] = MEM[addr + 4U].b32;
VDATA[95 : 64] = MEM[addr + 8U].b32;
VDATA[127 : 96] = MEM[addr + 12U].b32
```

**BUFFER\_STORE\_BYTE****24**

从向量寄存器存储到缓冲表面的数据有 8 位。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr].b8 = VDATA[7 : 0]
```

**BUFFER\_STORE\_BYTE\_D16\_HI****25**

从 32 位向量寄存器的高 16 位中取出 8 位数据，存入缓冲区。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr].b8 = VDATA[23 : 16]
```

**BUFFER\_STORE\_SHORT****26**

从向量寄存器中存储 16 位数据到缓冲表面。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
```

```
MEM[addr].b16 = VDATA[15 : 0]
```

## **BUFFER\_STORE\_SHORT\_D16\_HI**

**27**

将 32 位向量寄存器的高 16 位的数据存储到缓冲表面。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[31 : 16]
```

## **BUFFER\_STORE\_DWORD**

**28**

将来自向量输入的 32 位数据存储到缓冲表面。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0]
```

## **BUFFER\_STORE\_DWORDX2**

**29**

将来自向量输入的 64 位数据存储到缓冲表面。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32]
```

## **BUFFER\_STORE\_DWORDX3**

**30**

将来自向量输入的 96 位数据存储到缓冲表面。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32];  
MEM[addr + 8U].b32 = VDATA[95 : 64]
```

## **BUFFER\_STORE\_DWORDX4**

**31**

将 128 位来自向量输入的数据存储到缓冲表面。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr].b32 = VDATA[31 : 0];
MEM[addr + 4U].b32 = VDATA[63 : 32];
MEM[addr + 8U].b32 = VDATA[95 : 64];
MEM[addr + 12U].b32 = VDATA[127 : 96]

```

**BUFFER\_LOAD\_UBYTE\_D16****32**

从缓冲表面装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的低 16 位.

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[31:16]保存完好.

```

**BUFFER\_LOAD\_UBYTE\_D16\_HI****33**

从缓冲表面装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 16].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[15:0]保存完好.

```

**BUFFER\_LOAD\_SBYTE\_D16****34**

从缓冲表面装入 8 位签名数据,标志扩展至 16 位,并将结果存储到一个 32 位向量寄存器的低 16 位.

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[31:16]保存完好.

```

**BUFFER\_LOAD\_SBYTE\_D16\_HI****35**

从缓冲表面装入 8 位签名数据,标志扩展至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);

```

```
VDATA[31 : 16].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[15:0]保存完好.
```

## BUFFER\_LOAD\_SHORT\_D16

36

从缓冲表面装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的低 16 位.

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = MEM[addr].b16;
// VDATA[31:16]保存完好.
```

## BUFFER\_LOAD\_SHORT\_D16\_HI

37

从缓冲表面装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 16].b16 = MEM[addr].b16;
// VDATA[15:0]保存完好.
```

## BUFFER\_LOAD\_FORMAT\_D16\_HI\_X

38

从缓冲表面装入 1 组件格式化数据, 将数据转换为 16 位集成或浮点格式, 然后将结果存储到 32 位向量寄存器的高 16 位。资源描述符指定了表面的数据格式。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
// VDATA[15:0].b16 保存完好.
```

## BUFFER\_STORE\_FORMAT\_D16\_HI\_X

39

将 32 位向量输入寄存器的高 16 位数据转换成 1 个元件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[31 : 16].b16));
```

```
// Mem 访问大小取决于格式
```

**BUFFER\_WBL2****40**

写入 L2 缓存。返回 AK 到阴影。

**BUFFER\_INV****41**

根据 sc0 和 sc1 位数, 无效的 CU 和/ 或 L2 缓存。返回 ACK 到阴影。

**BUFFER\_ATOMIC\_SWAP****64**

在数据寄存器中将无符号的 32 位整数值与缓冲面的位置相交换。如果设定了 SC0 位,则将原值从缓冲面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

**BUFFER\_ATOMIC\_CMPSWAP****65**

比较两个无符号的 32 位整数值存储在数据比较寄存器和缓冲表面的位置。如果比较值相等,则修改数据源寄存器的内存位置。如果设定了 SC0 位,则将缓冲表面的原始值存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA[31 : 0].u32;
cmp = DATA[63 : 32].u32;
MEM[addr].u32 = tmp == cmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

**BUFFER\_ATOMIC\_ADD****66**

添加两个无符号的 32 位整数值存储在数据寄存器和缓冲表面的位置。如果设置了 SC0 比特,则将缓冲表面的原始值存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
```

```
tmp = MEM[addr].u32;  
MEM[addr].u32 += DATA.u32;  
RETURN_DATA.u32 = tmp
```

## BUFFER\_ATOMIC\_SUB

67

从存储在缓冲面某个位置的数值中减去一个无符号的 32 位整数值。如果设置了 SC0 比特,则将原值从缓冲面存储到向量表。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 -= DATA.u32;  
RETURN_DATA.u32 = tmp
```

## BUFFER\_ATOMIC\_SMIN

68

选择两个签名的 32 位整数输入的最小值, 因为有两个数值存储在数据寄存器和缓冲表面。用所选值更新缓冲表面。如果设置了 SC0 比特, 则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
tmp = MEM[addr].i32;  
src = DATA.i32;  
MEM[addr].i32 = src < tmp ? src : tmp;  
RETURN_DATA.i32 = tmp
```

## BUFFER\_ATOMIC\_UMIN

69

选择两个无符号的 32 位整数输入的最小值, 因为有两个值存储在数据寄存器和缓冲表的位置。用选定的值更新缓冲表。如果设置了 SC0 位值, 则将原值从缓冲表存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
tmp = MEM[addr].u32;  
src = DATA.u32;  
MEM[addr].u32 = src < tmp ? src : tmp;  
RETURN_DATA.u32 = tmp
```

## BUFFER\_ATOMIC\_SMAX

70

选择最多两个签名的 32 位整数输入, 给两个存储在数据寄存器中的值和一个

。将缓冲表面的原始值从缓冲表面存储到向量寄存器,如果设置了 SC0 位值。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

## BUFFER\_ATOMIC\_UMAX

71

选择两个无符号的 32 位整数输入的最大值, 因为有两个值存储在数据寄存器和缓冲表面。将缓冲表面以所选值更新。将原始值从缓冲表面存储到向量寄存器, iff SC0 位值被设定。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp
```

## BUFFER\_ATOMIC\_AND

72

以位法计算并给出两个无符号的 32 位整数值, 存储在数据寄存器和缓冲表面。如果设定了 SC0 位, 则将原值从缓冲表面存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
RETURN_DATA.b32 = tmp
```

## BUFFER\_ATOMIC\_OR

73

计算 bitwise OR 给定两个无符号的 32 位整数值, 存储在数据寄存器和缓冲表面。如果设置了 SC0 位, 则将原值从缓冲表面存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp
```

**BUFFER\_ATOMIC\_XOR**

74

在数据寄存器和缓冲面的位置上计算了两个无符号的 32 位整数值。如果设置了 SC0 比特,则将原值从缓冲面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp
```

**BUFFER\_ATOMIC\_INC**

75

将一个无符号的 32 位整数值从缓冲面上的位置加宽到 0,如果数值超过数据寄存器中的值。如果设定了 SC0 位,则将原值从缓冲面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp
```

**BUFFER\_ATOMIC\_DEC**

76

将一个无符号的 32 位整数值从缓冲表面的某个位置并包到数据寄存器中的某个值。如果设定了 SC0 位,则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
RETURN_DATA.u32 = tmp
```

**BUFFER\_ATOMIC\_ADD\_F32**

77

在数据寄存器中添加一个单精度浮点值到缓冲表面的某个位置。如果设置了 SC0 比特,则将原值从缓冲表面存储到向量寄存器中。

```
tmp = MEM[ADDR].f32;
MEM[ADDR].f32 += DATA.f32;
RETURN_DATA = tmp
```



## 说明

浮点加法处理 NAN/INF/denorm.

**BUFFER\_ATOMIC\_PK\_ADD\_F16****78**

在数据寄存器中添加一个包装的 2 组件半精度浮点值到缓冲表面的位置。如果设置了 SC0 位值, 则将原值从缓冲表面存储到向量寄存器。

```
tmp = MEM[ADDR];  
src = DATA;  
dst[31 : 16].f16 = tmp[31 : 16].f16 + src[31 : 16].f16;  
dst[15 : 0].f16 = tmp[15 : 0].f16 + src[15 : 0].f16;  
MEM[ADDR] = dst.b32;  
RETURN_DATA = tmp
```

## 说明

浮点加法处理 NAN/INF/denorm.

**BUFFER\_ATOMIC\_ADD\_F64****79**

在数据寄存器中添加一个双精度浮点值到缓冲表面的位置。如果设置了 SC0 比特, 则将原值从缓冲表面存储到向量寄存器中。

```
tmp = MEM[ADDR].f64;  
MEM[ADDR].f64 += DATA.f64;  
RETURN_DATA = tmp
```

## 说明

浮点加法处理 NAN/INF/denorm.

**BUFFER\_ATOMIC\_MIN\_F64****80**

选择至少两个双精度浮点数输入, 前提是有两个数值存储在数据寄存器和缓冲表面。将缓冲表面以选定的数值更新。如果设置了 SC0 位值, 请将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);  
tmp = MEM[addr].f64;  
src = DATA.f64;  
MEM[addr].f64 = src < tmp ? src : tmp;
```

```
RETURN_DATA.f64 = tmp
```

## BUFFER\_ATOMIC\_MAX\_F64

81

选择两个双精度浮点输入的最大值, 以两个数值存储在数据寄存器和缓冲表面。用所选值更新缓冲表面。将原值从缓冲表面存储到向量寄存器 iff SC0 位值已设定。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src > tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

## BUFFER\_ATOMIC\_SWAP\_X2

96

在数据寄存器中将一个无符号的 64 位整数值与缓冲表面的位置相交换。如果设置了 SC0 位,则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = DATA.b64;
RETURN_DATA.b64 = tmp
```

## BUFFER\_ATOMIC\_CMPSWAP\_X2

97

比较两个无符号的 64 位整数值存储在数据比较寄存器和缓冲表面。如果比较是等值的,则修改数据源寄存器的内存位置。如果设置了 SC0 位,则将缓冲表面的原始值存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA[63 : 0].u64;
cmp = DATA[127 : 64].u64;
MEM[addr].u64 = tmp == cmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

## BUFFER\_ATOMIC\_ADD\_X2

98

添加两个无符号的 64 位整数值存储在数据寄存器和缓冲表面的位置。如果设置了 SC0 比特,则将原值从缓冲表面存储到向量寄存器中。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 += DATA.u64;
RETURN_DATA.u64 = tmp

```

**BUFFER\_ATOMIC\_SUB\_X2****99**

从存储在缓冲表面的某个位置的数值中减去一个无符号的 64 位整数。如果设置了 SC0 位,则将原值从缓冲表面存储到向量寄存器中。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 -= DATA.u64;
RETURN_DATA.u64 = tmp

```

**BUFFER\_ATOMIC\_SMIN\_X2****100**

选择两个签名的 64 位整数输入的最小值, 因为有两个值存储在数据寄存器和缓冲表面。用所选值更新缓冲表面。如果设置了 SC0 位值, 请将原值从缓冲表面存储到向量寄存器中。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp

```

**BUFFER\_ATOMIC\_UMIN\_X2****101**

选择两个没有符号的 64 位整数输入的最小值, 因为有两个数值存储在数据寄存器和缓冲表面。用选定的数值更新缓冲表面。如果设置了 SC0 位值, 请将原始值从缓冲表面存储到向量寄存器。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp

```

**BUFFER\_ATOMIC\_SMAX\_X2****102**

选择最多两个签名的 64 位整数输入, 给定两个存储在数据寄存器和缓冲表面的位置。用所选值更新缓冲表面。如果设置了 SC0 位, 则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

## BUFFER\_ATOMIC\_UMAX\_X2

103

选择两个无符号的 64 位整数输入的最大值, 给两个存储在数据寄存器和缓冲表面的位置。用所选值更新缓冲表面。如果设置了 SC0 比特, 则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src >= tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

## BUFFER\_ATOMIC\_AND\_X2

104

计算比特并给出了两个无符号的 64 位整数值, 存储在数据寄存器和缓冲表面。如果设置了 SC0 位, 将原值从缓冲表面存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp & DATA.b64);
RETURN_DATA.b64 = tmp
```

## BUFFER\_ATOMIC\_OR\_X2

105

计算 bitwise OR 给定两个无符号的 64 位整数值, 存储在数据寄存器和缓冲表面。如果设置了 SC0 位, 则将原值从缓冲表面存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp | DATA.b64);
RETURN_DATA.b64 = tmp
```

**BUFFER\_ATOMIC\_XOR\_X2****106**

以比特方式计算 XOR 给两个无符号的 64 位整数值, 存储在数据寄存器和缓冲表面。如果设置了 SC0 位, 则将原值从缓冲表面存储到向量寄存器。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp ^ DATA.b64);
RETURN_DATA.b64 = tmp
```

**BUFFER\_ATOMIC\_INC\_X2****107**

将一个无符号的 64 位整数值从缓冲面上的位置加宽到 0, 如果该值超过数据寄存器中的值。如果设定了 SC0 位, 则将原值从缓冲面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

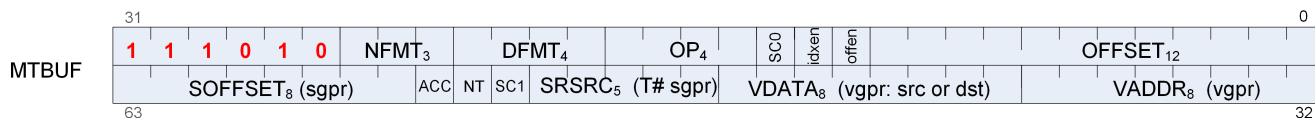
**BUFFER\_ATOMIC\_DEC\_X2****108**

将一个无符号的 64 位整数值从缓冲表面的某个位置, 并加包, 到数据寄存器中的某个值, 如果该递减值产生负值。如果设置了 SC0 位, 则将原值从缓冲表面存储到向量寄存器中。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```

## 12.14. MTBUF 指令

MTBUF 格式的位域图是:



where:

```

OFFSET  = Unsigned immediate byte offset.
OFFEN   = Send offset either as VADDR or as zero.
IDXEN   = Send index either as VADDR or as zero.
LDS     = Data is transferred between LDS and Memory, not VGPRs.
OP      = Instruction Opcode.
DFMT    = Data format for typed buffer.
NFMT    = Number format for typed buffer.
VADDR   = VGPR address source.
VDATA   = Vector GPR for read/write result.
SRSRC   = Scalar GPR that specifies resource constant.
SOFFSET = Unsigned byte offset from an SGPR.
SC      = Scope
NT      = Non-Temporal
ACC     = Return to ACC VGPRs

```

**TBUFFER\_LOAD\_FORMAT\_X**

## O

从缓冲表面装入 1 组件格式化数据, 将数据转换为 32 位集成或浮点格式, 然后将结果存储到向量寄存器中。指令指定了表面的数据格式, 凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
```

**TBUFFER\_LOAD\_FORMAT\_XY**

**1**

从缓冲表面装入 2 个组件格式化的数据,将数据转换为 32 位集成或浮点格式,然后将结果存储到向量寄存器中。指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()])
```

**TBUFFER\_LOAD\_FORMAT\_XYZ****2**

从缓冲表面装入 3 个组件格式化的数据,将数据转换为 32 位集成或浮点格式,然后将结果存储到向量寄存器中。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()]);
VDATA[95 : 64].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetZ()]);
```

**TBUFFER\_LOAD\_FORMAT\_XYZW****3**

从缓冲表面装入 4 组件格式化数据, 将数据转换为 32 位集成或浮点格式, 然后将结果存储到向量寄存器中。指令指定了表面的数据格式, 压倒资源描述符。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[31 : 0].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetX()]);
// Mem 访问大小取决于格式
VDATA[63 : 32].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetY()]);
VDATA[95 : 64].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetZ()]);
VDATA[127 : 96].b32 = ConvertFromFormat(MEM[addr + ChannelOffsetW()]);
```

**TBUFFER\_STORE\_FORMAT\_X****4**

将 32 位数据从向量输入寄存器转换为 1 个组件格式化的数据, 并将数据存储到缓冲表面。指令指定了表面的数据格式, 凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
```

**TBUFFER\_STORE\_FORMAT\_XY****5**

从向量输入寄存器中将 64 位数据转换成 2 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
```

```
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32)
```

## TBUFFER\_STORE\_FORMAT\_XYZ

6

从向量输入寄存器中将 96 位数据转换成 3 个组件格式化的数据,并将数据存储到缓冲面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32);
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(VDATA[95 : 64].b32)
```

## TBUFFER\_STORE\_FORMAT\_XYZW

7

将 128 比特的数据从向量输入寄存器转换成 4 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(VDATA[31 : 0].b32);
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(VDATA[63 : 32].b32);
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(VDATA[95 : 64].b32);
MEM[addr + ChannelOffsetW()] = ConvertToFormat(VDATA[127 : 96].b32)
```

## TBUFFER\_LOAD\_FORMAT\_D16\_X

8

从缓冲面上装入 1 组件格式化数据,将数据转换为 16 位集成或浮点格式,然后将结果存储到向量寄存器中。指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
// VDATA[31:16].b16 保存完好。
```

## TBUFFER\_LOAD\_FORMAT\_D16\_XY

9

从缓冲表面装入 2 个组件格式化的数据,将数据转换为装入 16 位集成或浮点格式,然后将结果存储到向量寄存器中。指令指定了表面的数据格式,凌驾于资源描述符之上。



```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()]));

```

## TBUFFER\_LOAD\_FORMAT\_D16\_XYZ

10

从缓冲面上装入 3 个组件格式化数据,将数据转换为装入 16 位元集成或浮点格式,然后将结果存储到向量寄存器中。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()]));
VDATA[47 : 32].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetZ()]));
// VDATA[63:48].b16 保存完好。

```

## TBUFFER\_LOAD\_FORMAT\_D16\_XYZW

11

从缓冲表面装入 4 组件格式化数据, 将数据转换为 16 位集成或浮点格式, 然后将结果存储到向量寄存器中。指令指定了表面的数据格式, 凌驾于资源描述符之上。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
VDATA[15 : 0].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetX()]));
// Mem 访问大小取决于格式
VDATA[31 : 16].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetY()]));
VDATA[47 : 32].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetZ()]));
VDATA[63 : 48].b16 = 16'B(ConvertFromFormat(MEM[addr + ChannelOffsetW()]));

```

## TBUFFER\_STORE\_FORMAT\_D16\_X

12

从向量输入记录器中将 16 位数据转换成 1 元格式数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```

addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式

```

**TBUFFER\_STORE\_FORMAT\_D16\_XY****13**

将 32 位数据从向量输入寄存器转换为 2 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16))
```

**TBUFFER\_STORE\_FORMAT\_D16\_XYZ****14**

从向量输入寄存器中将 48 位数据转换成 3 个组件格式化的数据,并将数据存储到缓冲表面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16));
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(32'B(VDATA[47 : 32].b16))
```

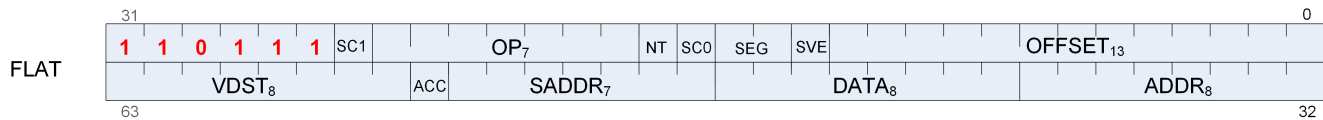
**TBUFFER\_STORE\_FORMAT\_D16\_XYZW****15**

将 64 位数据从向量输入寄存器转换成 4 个组件格式化的数据,并将数据存储到缓冲面。该指令指定了表面的数据格式,凌驾于资源描述符之上。

```
addr = CalcBufferAddr(VADDR.b32, SRSRC.b32, SOFFSET.b32, OFFSET.b32);
MEM[addr + ChannelOffsetX()] = ConvertToFormat(32'B(VDATA[15 : 0].b16));
// Mem 访问大小取决于格式
MEM[addr + ChannelOffsetY()] = ConvertToFormat(32'B(VDATA[31 : 16].b16));
MEM[addr + ChannelOffsetZ()] = ConvertToFormat(32'B(VDATA[47 : 32].b16));
MEM[addr + ChannelOffsetW()] = ConvertToFormat(32'B(VDATA[63 : 48].b16))
```

## 12.15. FLAT、Scratch 与 Global 指令

FLAT 格式的位域图是:



where:

OP = Instruction Opcode.  
ADDR = Source of flat address VGPR.  
DATA = Source data.  
VDST = Destination VGPR.  
NV = Access to non-volatile memory.  
SADDR = SGPR holding address or offset  
SEG = Instruction type: Flat, Scratch, or Global  
LDS = Data is transferred between LDS and Memory, not VGPRs.  
OFFSET = Immediate address byte-offset.  
SC = Scope  
NT = Non-Temporal

### 12.15.1. Flat 指令

平面指令查看每个工作项地址,确定每个工作项的目标内存地址是否为全局,私有或抓取内存.

FLAT\_LOAD\_UBYTE

16

从平面孔径装入 8 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA.u32 = 32'U({ 24'0U, MEM[addr].u8 })
```

FLAT\_LOAD\_SBYTE

17

从平面孔径装入 8 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA.i32 = 32'I(signext(MEM[addr].i8))
```

**FLAT\_LOAD\_USHORT****18**

从平面孔径装入 16 位无符号的数据,0 延伸至 32 位并将结果存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA.u32 = 32'U({ 16'0U, MEM[addr].u16 })
```

**FLAT\_LOAD\_SSHORT****19**

从平面孔径装入 16 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA.i32 = 32'I(signext(MEM[addr].i16))
```

**FLAT\_LOAD\_DWORD****20**

从平面孔径装入 32 位数据到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32
```

**FLAT\_LOAD\_DWORDX2****21**

从平面孔径装入 64 位数据到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32
```

**FLAT\_LOAD\_DWORDX3****22**

将 96 位数据从平面孔径装入向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;
```

```
VDATA[95 : 64] = MEM[addr + 8U].b32
```

## FLAT\_LOAD\_DWORDX4

23

从平面孔径装入 128 位数据到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;  
VDATA[95 : 64] = MEM[addr + 8U].b32;  
VDATA[127 : 96] = MEM[addr + 12U].b32
```

## FLAT\_STORE\_BYTE

24

将向量寄存器中的 8 位数据存储到扁孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[7 : 0]
```

## FLAT\_STORE\_BYTE\_D16\_HI

25

将 32 位向量寄存器的高 16 位的 8 位数据存储到扁孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[23 : 16]
```

## FLAT\_STORE\_SHORT

26

从向量寄存器中存储 16 位数据到平孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[15 : 0]
```

## FLAT\_STORE\_SHORT\_D16\_HI

27

将 32 位向量寄存器的高 16 位数据存储到扁孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[31 : 16]
```

---

**FLAT\_STORE\_DWORD****28**

将来自向量输入的 32 位数据存储到平孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0]
```

---

**FLAT\_STORE\_DWORDX2****29**

将来自向量输入的 64 位数据存储到平孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32]
```

---

**FLAT\_STORE\_DWORDX3****30**

将来自向量输入的 96 位数据存储到平孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32];  
MEM[addr + 8U].b32 = VDATA[95 : 64]
```

---

**FLAT\_STORE\_DWORDX4****31**

将来自向量输入的 128 位数据存储到平孔径中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32];  
MEM[addr + 8U].b32 = VDATA[95 : 64];  
MEM[addr + 12U].b32 = VDATA[127 : 96]
```

**FLAT\_LOAD\_UBYTE\_D16****32**

从平面孔径上装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的低 16 位.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[15 : 0].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[31:16]保存完好.
```

**FLAT\_LOAD\_UBYTE\_D16\_HI****33**

从平面孔径装入 8 位无符号数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[31 : 16].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[15:0]保存完好.
```

**FLAT\_LOAD\_SBYTE\_D16****34**

从平面孔径装入 8 位签名数据,标志扩展至 16 位,并将结果存储到 32 位向量寄存器的低 16 位.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[15 : 0].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[31:16]保存完好.
```

**FLAT\_LOAD\_SBYTE\_D16\_HI****35**

从平面孔径装入 8 位签名数据,标志扩展至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[31 : 16].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[15:0]保存完好.
```

**FLAT\_LOAD\_SHORT\_D16****36**

从平面孔径装入 16 位无符号的数据,并将结果存储到 32 位向量的低 16 位

登记簿。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[15 : 0].b16 = MEM[addr].b16;
// VDATA[31:16]保存完好.
```

## FLAT\_LOAD\_SHORT\_D16\_HI

37

从平面孔径上装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的高 16 位。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
VDATA[31 : 16].b16 = MEM[addr].b16;
// VDATA[15:0]保存完好.
```

## FLAT\_ATOMIC\_SWAP

64

在数据寄存器中将一个无符号的 32 位整数值与平面孔径位置相交换。如果设定了 SC0 比特,则将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

## FLAT\_ATOMIC\_CMPSWAP

65

比较两个无符号的 32 位整数值存储在数据比较寄存器和平面孔径中的位置。如果比较是等值,则修改数据源寄存器的内存位置。如果设定了 SC0 位值,则将原值从平面孔径存储到向量寄存器中。

注:RETURN\_DATA[1]没有修改。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA[31 : 0].u32;
cmp = DATA[63 : 32].u32;
MEM[addr].u32 = tmp == cmp ? src : tmp;
RETURN_DATA.u32 = tmp
```



**FLAT\_ATOMIC\_ADD**

66

添加两个无符号的 32 位整数值存储在数据寄存器和一个位置在平面孔径中。如果设置了 SC0 比特,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
MEM[addr].u32 += DATA.u32;
RETURN_DATA.u32 = tmp
```

**FLAT\_ATOMIC\_SUB**

67

从存储在平面孔径位置的数值中将数据寄存器中一个无符号的 32 位整数值减掉。如果设置了 SC0 位,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
MEM[addr].u32 -= DATA.u32;
RETURN_DATA.u32 = tmp
```

**FLAT\_ATOMIC\_SMIN**

68

选择两个签名的 32 位整数输入的最小值, 因为有两个值存储在数据寄存器中, 和一个位置位于平面孔径中。用所选值更新平面孔径。如果设置了 SC0 比特, 请将原始值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src < tmp ? src : tmp;
RETURN_DATA.i32 = tmp
```

**FLAT\_ATOMIC\_UMIN**

69

选择两个没有符号的 32 位整数输入的最小值, 因为有两个数值存储在数据寄存器和一个位置在平面孔径中。将平面孔径的原始值从平面孔径保存到向量寄存器中, 如果设置了 SC0 比特。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
```

```
RETURN_DATA.u32 = tmp
```

## FLAT\_ATOMIC\_SMAX

70

选择最多两个签名的 32 位整数输入, 给出两个存储在数据寄存器和平面孔径中的位置。用选定的值更新平面孔径。如果设置了 SC0 位, 将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].i32;  
src = DATA.i32;  
MEM[addr].i32 = src >= tmp ? src : tmp;  
RETURN_DATA.i32 = tmp
```

## FLAT\_ATOMIC\_UMAX

71

选择两个无符号的 32 位整数输入的最大值, 给数据寄存器和平面孔径中的位置。将平面孔径的原始值从平面孔径存储到向量寄存器, 如果设置了 SC0 位值。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u32;  
src = DATA.u32;  
MEM[addr].u32 = src >= tmp ? src : tmp;  
RETURN_DATA.u32 = tmp
```

## FLAT\_ATOMIC\_AND

72

以位法计算并给出两个无符号的 32 位整数值存储在数据寄存器和平面孔径中的位置。如果设定了 SC0 位, 则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].b32;  
MEM[addr].b32 = (tmp & DATA.b32);  
RETURN_DATA.b32 = tmp
```

## FLAT\_ATOMIC\_OR

73

计算位元 OR 给定了两个无符号的 32 位整数值, 存储在数据寄存器和平面孔径中的位置。如果设置了 SC0 位元, 将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp
```

## FLAT\_ATOMIC\_XOR

74

计算位元 XOR 给定了两个无符号的 32 位整数, 存储在数据寄存器和平面孔径中的位置。如果设置了 SC0 位, 则将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp
```

## FLAT\_ATOMIC\_INC

75

将一个无符号的 32 位整数从平面孔径中的位置加宽到 0, 如果值超过数据寄存器中的值。如果设置了 SC0 位, 则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp
```

## FLAT\_ATOMIC\_DEC

76

将一个无符号的 32 位整数从平面孔径中的位置, 并加包, 降幅产生负值时, 降为数据寄存器中的值。如果设定了 SC0 位值, 将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
RETURN_DATA.u32 = tmp
```

## FLAT\_ATOMIC\_ADD\_F32

77

在数据寄存器中添加一个单精度浮点值到平面孔径中的位置。如果设置了 SC0 比特,则将原值从平面孔径存储到向量寄存器中。

```
tmp = MEM[ADDR].f32;  
MEM[ADDR].f32 += DATA.f32;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

---

### FLAT\_ATOMIC\_PK\_ADD\_F16

**78**

在数据寄存器中添加一个装有 2 个组件的半精度浮点值到平面孔径中的位置。如果设定了 SC0 比特,则将原值从平面孔径存储到向量寄存器中。

```
tmp = MEM[ADDR];  
src = DATA;  
dst[31 : 16].f16 = tmp[31 : 16].f16 + src[31 : 16].f16;  
dst[15 : 0].f16 = tmp[15 : 0].f16 + src[15 : 0].f16;  
MEM[ADDR] = dst.b32;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

---

### FLAT\_ATOMIC\_ADD\_F64

**79**

在数据寄存器中添加一个双精度浮点值到平面孔径中的位置。如果设置了 SC0 位,则将原值从平面孔径存储到向量寄存器中。

```
tmp = MEM[ADDR].f64;  
MEM[ADDR].f64 += DATA.f64;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

---

### FLAT\_ATOMIC\_MIN\_F64

**80**

选择至少两个双精度浮点数输入, 因为有两个数值存储在数据寄存器和一个

。将平面孔径的原始值从平面孔径存储到向量寄存器,如果设置了 SC0 比特。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src < tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

## FLAT\_ATOMIC\_MAX\_F64

81

选择两个双精度浮点输入的最大值, 给两个数值存储在数据寄存器和一个位置在平面孔径中。将平面孔径的原始值从平面孔径存储到向量寄存器, 如果设置了 SC0 位值。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src > tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

## FLAT\_ATOMIC\_PK\_ADD\_BF16

82

在数据寄存器中添加一个装有 2 个组件的 BF16 浮点值到平面孔径中的位置。如果设定了 SC0 比特,则将原值从平面孔径存储到向量寄存器中。

```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].bf16 = tmp[31 : 16].bf16 + src[31 : 16].bf16;
dst[15 : 0].bf16 = tmp[15 : 0].bf16 + src[15 : 0].bf16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

## FLAT\_ATOMIC\_SWAP\_X2

96

在数据寄存器中将一个无符号的 64 位整数值与平面孔径位置相交换。如果设定了 SC0 位,则将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
```

```
tmp = MEM[addr].b64;  
MEM[addr].b64 = DATA.b64;  
RETURN_DATA.b64 = tmp
```

## FLAT\_ATOMIC\_CMPSWAP\_X2

97

比较两个无符号的 64 位整数值存储在数据比较寄存器和平面孔径中的位置。如果比较是等值,则修改数据源寄存器的内存位置。如果设定了 SC0 位值,则将原值从平面孔径存储到向量寄存器中。

注:RETURN\_DATA[2:3]没有修改。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u64;  
src = DATA[63 : 0].u64;  
cmp = DATA[127 : 64].u64;  
MEM[addr].u64 = tmp == cmp ? src : tmp;  
RETURN_DATA.u64 = tmp
```

## FLAT\_ATOMIC\_ADD\_X2

98

添加两个无符号的 64 位整数值存储在数据寄存器和一个位置在平面孔径中。如果设定了 SC0 比特,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u64;  
MEM[addr].u64 += DATA.u64;  
RETURN_DATA.u64 = tmp
```

## FLAT\_ATOMIC\_SUB\_X2

99

从存储在平面孔径位置的数值中减去一个无符号的 64 位整数值。如果设置了 SC0 位,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u64;  
MEM[addr].u64 -= DATA.u64;  
RETURN_DATA.u64 = tmp
```

## FLAT\_ATOMIC\_SMIN\_X2

100

选择两个签名的 64 位整数输入的最小值, 因为在数据寄存器中有两个值, 在平面孔径中有一个位置。用所选值更新平面孔径。如果设置了 SC0 位, 请将原始值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

## FLAT\_ATOMIC\_UMIN\_X2

101

选择两个无符号的 64 位整数输入的最小值, 因为有两个数值存储在数据寄存器和一个位置在平面孔径中。将平面孔径的原始值从平面孔径存储到向量寄存器, 如果设置了 SC0 位值。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

## FLAT\_ATOMIC\_SMAX\_X2

102

选择最多两个签名的 64 位整数输入, 给定两个存储在数据寄存器和平面孔径中的位置。将平面孔径的原始值从平面孔径存储到向量寄存器, 如果设置了 SC0 位值。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

## FLAT\_ATOMIC\_UMAX\_X2

103

选择两个无符号的 64 位整数输入的最大值, 给两个数值存储在数据寄存器和平面孔径中的位置。用所选值更新平面孔径。如果设置了 SC0 比特, 将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
```

```
MEM[addr].u64 = src >= tmp ? src : tmp;  
RETURN_DATA.u64 = tmp
```

## FLAT\_ATOMIC\_AND\_X2

104

计算比特并给出两个无符号的 64 位整数值存储在数据寄存器和平面孔径中的位置。如果设定了 SC0 位,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp & DATA.b64);  
RETURN_DATA.b64 = tmp
```

## FLAT\_ATOMIC\_OR\_X2

105

计算位元 OR 给定了两个无符号的 64 位整数值, 存储在数据寄存器和平面孔径中的位置。 如果设置了 SC0 位元, 将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp | DATA.b64);  
RETURN_DATA.b64 = tmp
```

## FLAT\_ATOMIC\_XOR\_X2

106

计算位元 XOR 给定了两个无符号的 64 位整数值, 存储在数据寄存器和平面孔径中的位置。 如果设置了 SC0 比特, 将原值从平面孔径存储到向量寄存器。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].b64;  
MEM[addr].b64 = (tmp ^ DATA.b64);  
RETURN_DATA.b64 = tmp
```

## FLAT\_ATOMIC\_INC\_X2

107

将一个无符号的 64 位整数值从平面孔径中的位置加宽到 0,如果数值超过数据寄存器中的值。如果设置了 SC0 位,则将原值从平面孔径存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);
```



```
tmp = MEM[addr].u64;  
src = DATA.u64;  
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;  
RETURN_DATA.u64 = tmp
```

---

**FLAT\_ATOMIC\_DEC\_X2****108**

将一个无符号的 64 位整数值从平面孔径中的某个位置和包线降为数据寄存器中的某个值。如果设定了 SC0 位,则将平面孔径中的原始值存储到向量寄存器中。

```
addr = CalcFlatAddr(ADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u64;  
src = DATA.u64;  
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;  
RETURN_DATA.u64 = tmp
```

## 12.15.2. Scratch 指令

划痕指令类似于 Flat,但假设所有工作项地址都掉落到划痕(私密)空间中。

### SCRATCH\_LOAD\_UBYTE

16

从抓取孔径装入 8 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 24'0U, MEM[addr].u8 })
```

### SCRATCH\_LOAD\_SBYTE

17

从抓取孔径装入 8 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i8))
```

### SCRATCH\_LOAD\_USHORT

18

从抓取孔径装入 16 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 16'0U, MEM[addr].u16 })
```

### SCRATCH\_LOAD\_SSHORT

19

从抓取孔径装入 16 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i16))
```

### SCRATCH\_LOAD\_DWORD

20

从抓取孔径装入 32 位数据到向量寄存器。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32
```

---

**SCRATCH\_LOAD\_DWORDX2****21**

从抓取孔径装入 64 位数据到向量寄存器。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32
```

---

**SCRATCH\_LOAD\_DWORDX3****22**

从抓取孔径上装入 96 位数据到向量寄存器。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;  
VDATA[95 : 64] = MEM[addr + 8U].b32
```

---

**SCRATCH\_LOAD\_DWORDX4****23**

从刮角孔径装入 128 位数据到向量寄存器。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;  
VDATA[95 : 64] = MEM[addr + 8U].b32;  
VDATA[127 : 96] = MEM[addr + 12U].b32
```

---

**SCRATCH\_STORE\_BYTE****24**

将向量寄存器中的 8 位数据存储到抓取孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[7 : 0]
```

**SCRATCH\_STORE\_BYTE\_D16\_HI****25**

将 32 位向量寄存器的高 16 位的 8 位数据存储到抓取孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[23 : 16]
```

**SCRATCH\_STORE\_SHORT****26**

从向量寄存器中存储 16 位数据到抓取孔径。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[15 : 0]
```

**SCRATCH\_STORE\_SHORT\_D16\_HI****27**

将 32 位向量寄存器的高 16 位数据存储到刮角孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[31 : 16]
```

**SCRATCH\_STORE\_DWORD****28**

将来自向量输入的 32 位数据存储到刮角孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0]
```

**SCRATCH\_STORE\_DWORDX2****29**

将来自向量输入的 64 位数据存储到刮角孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32]
```

**SCRATCH\_STORE\_DWORDX3****30**

将来自向量输入的 96 位数据存储到刮痕孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
MEM[addr].b32 = VDATA[31 : 0];
MEM[addr + 4U].b32 = VDATA[63 : 32];
MEM[addr + 8U].b32 = VDATA[95 : 64]
```

**SCRATCH\_STORE\_DWORDX4****31**

将来自向量输入的 128 位数据存储到刮毛孔径中。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
MEM[addr].b32 = VDATA[31 : 0];
MEM[addr + 4U].b32 = VDATA[63 : 32];
MEM[addr + 8U].b32 = VDATA[95 : 64];
MEM[addr + 12U].b32 = VDATA[127 : 96]
```

**SCRATCH\_LOAD\_UBYTE\_D16****32**

从刮角孔径装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的低 16 位。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[31:16]保存完好。
```

**SCRATCH\_LOAD\_UBYTE\_D16\_HI****33**

从抓取孔径装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的高 16 位。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[15:0]保存完好。
```

**SCRATCH\_LOAD\_SBYTE\_D16****34**

从抓取孔径装入 8 位签名数据, 标记扩展至 16 位并将结果存储到低 16 位

32 位向量寄存器的位数。

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[31:16]保存完好.
```

---

### SCRATCH\_LOAD\_SBYTE\_D16\_HI

35

从抓取孔径装入 8 位签名数据,标志扩展至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[15:0]保存完好.
```

---

### SCRATCH\_LOAD\_SHORT\_D16

36

从抓取孔径装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的低 16 位.

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].b16 = MEM[addr].b16;
// VDATA[31:16]保存完好.
```

---

### SCRATCH\_LOAD\_SHORT\_D16\_HI

37

从抓取孔径装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcScratchAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].b16 = MEM[addr].b16;
// VDATA[15:0]保存完好.
```

---

### SCRATCH\_LOAD\_LDS\_UBYTE

38

从刮角孔径装入 8 位未键入的数据,0 延伸至 32 位,并将结果存储为数据共享.

---

**SCRATCH\_LOAD\_LDS\_SBYTE****39**

从抓取孔径装入 8 位未键入的数据,标志扩展至 32 位,并将结果存储为数据共享.

**SCRATCH\_LOAD\_LDS\_USHORT****40**

从抓取孔径装入 16 位未键入数据,0 延伸至 32 位,并将结果存储为数据共享.

**SCRATCH\_LOAD\_LDS\_SSHORT****41**

从刮角孔径装入 16 位未键入的数据,标志扩展至 32 位并将结果存储为数据共享.

**SCRATCH\_LOAD\_LDS\_DWORD****42**

从抓取孔径装入 32 位未键入的数据,并将结果存储到数据共享中.

### 12.15.3. Global 指令

全球指令类似于 Flat,但假设所有工作项地址都落在全球记忆空间中.

#### GLOBAL\_LOAD\_UBYTE

16

从全域孔径装入 8 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 24'0U, MEM[addr].u8 })
```

#### GLOBAL\_LOAD\_SBYTE

17

从全球孔径装入 8 位签名数据,标志扩展至 32 位,并将结果存储到向量寄存器中.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i8))
```

#### GLOBAL\_LOAD\_USHORT

18

从全域孔径装入 16 位无符号的数据,0 延伸至 32 位,并将结果存储到向量寄存器中.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.u32 = 32'U({ 16'0U, MEM[addr].u16 })
```

#### GLOBAL\_LOAD\_SSHORT

19

从全域孔径装入 16 位签名数据,标志扩展至 32 位并将结果存储到向量寄存器中.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA.i32 = 32'I(signext(MEM[addr].i16))
```

#### GLOBAL\_LOAD\_DWORD

20

从全域孔径装入 32 位数据到向量寄存器.



```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32
```

---

**GLOBAL\_LOAD\_DWORDX2****21**

从全域孔径装入 64 位数据到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32
```

---

**GLOBAL\_LOAD\_DWORDX3****22**

将全球孔径的 96 位数据装入向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;  
VDATA[95 : 64] = MEM[addr + 8U].b32
```

---

**GLOBAL\_LOAD\_DWORDX4****23**

从全球孔径装入 128 位数据到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
VDATA[31 : 0] = MEM[addr].b32;  
VDATA[63 : 32] = MEM[addr + 4U].b32;  
VDATA[95 : 64] = MEM[addr + 8U].b32;  
VDATA[127 : 96] = MEM[addr + 12U].b32
```

---

**GLOBAL\_STORE\_BYTE****24**

从向量寄存器中存储 8 位数据到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[7 : 0]
```

**GLOBAL\_STORE\_BYTE\_D16\_HI****25**

将 32 位向量寄存器的高 16 位的 8 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b8 = VDATA[23 : 16]
```

**GLOBAL\_STORE\_SHORT****26**

从向量寄存器存储 16 位数据到全球孔径。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[15 : 0]
```

**GLOBAL\_STORE\_SHORT\_D16\_HI****27**

将 32 位向量寄存器的高 16 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b16 = VDATA[31 : 16]
```

**GLOBAL\_STORE\_DWORD****28**

将来自向量输入的 32 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0]
```

**GLOBAL\_STORE\_DWORDX2****29**

将来自向量输入的 64 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
MEM[addr].b32 = VDATA[31 : 0];  
MEM[addr + 4U].b32 = VDATA[63 : 32]
```

**GLOBAL\_STORE\_DWORDX3****30**

将来自向量输入的 96 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
MEM[addr].b32 = VDATA[31 : 0];
MEM[addr + 4U].b32 = VDATA[63 : 32];
MEM[addr + 8U].b32 = VDATA[95 : 64]
```

**GLOBAL\_STORE\_DWORDX4****31**

将来自向量输入的 128 位数据存储到全球孔径中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
MEM[addr].b32 = VDATA[31 : 0];
MEM[addr + 4U].b32 = VDATA[63 : 32];
MEM[addr + 8U].b32 = VDATA[95 : 64];
MEM[addr + 12U].b32 = VDATA[127 : 96]
```

**GLOBAL\_LOAD\_UBYTE\_D16****32**

从全域孔径装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的低 16 位。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[31:16]保存完好。
```

**GLOBAL\_LOAD\_UBYTE\_D16\_HI****33**

从全域孔径装入 8 位无符号的数据,0 延伸至 16 位,并将结果存储到 32 位向量寄存器的高 16 位。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].u16 = 16'U({ 8'0U, MEM[addr].u8 });
// VDATA[15:0]保存完好。
```

**GLOBAL\_LOAD\_SBYTE\_D16****34**

从全域孔径装入 8 位签名数据,标志扩展至 16 位并将结果存储到低 16 位

32 位向量寄存器的位数。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[31:16]保存完好.
```

---

### GLOBAL\_LOAD\_SBYTE\_D16\_HI

35

从全域孔径装入 8 位签名数据,标志扩展至 16 位,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].i16 = 16'I(signext(MEM[addr].i8));
// VDATA[15:0]保存完好.
```

---

### GLOBAL\_LOAD\_SHORT\_D16

36

从全域孔径装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的低 16 位.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[15 : 0].b16 = MEM[addr].b16;
// VDATA[31:16]保存完好.
```

---

### GLOBAL\_LOAD\_SHORT\_D16\_HI

37

从全域孔径装入 16 位无符号的数据,并将结果存储到 32 位向量寄存器的高 16 位.

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
VDATA[31 : 16].b16 = MEM[addr].b16;
// VDATA[15:0]保存完好.
```

---

### GLOBAL\_LOAD\_LDS\_UBYTE

38

从全域孔径装入 8 位未键入的数据,0 延伸至 32 位,并将结果存储为数据共享.

---

**GLOBAL\_LOAD\_LDS\_SBYTE****39**

从全域孔径装入 8 位未键入的数据,标志扩展至 32 位并将结果存储为数据共享。

**GLOBAL\_LOAD\_LDS\_USHORT****40**

从全球孔径装入 16 比特的非类型数据,0 扩展至 32 比特,并将结果存储为数据共享。

**GLOBAL\_LOAD\_LDS\_SSHORT****41**

从全域孔径装入 16 位未键入的数据,标志扩展至 32 位并将结果存储为数据共享。

**GLOBAL\_LOAD\_LDS\_DWORD****42**

从全域孔径装入 32 位未键入的数据,并将结果存储为数据共享。

**GLOBAL\_ATOMIC\_SWAP****64**

在数据寄存器中将一个无符号的 32 位整数值与位于全域孔径中的位置交换。如果设定了 SC0 位,则将原值从全域孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = DATA.b32;
RETURN_DATA.b32 = tmp
```

**GLOBAL\_ATOMIC\_CMPSWAP****65**

比较数据比较寄存器和全域孔径中两个无符号的 32 位整数值。如果比较值相等,则修改数据源寄存器的内存位置。如果设置了 SC0 位,则将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA[31 : 0].u32;
cmp = DATA[63 : 32].u32;
MEM[addr].u32 = tmp == cmp ? src : tmp;
```

```
RETURN_DATA.u32 = tmp
```

## GLOBAL\_ATOMIC\_ADD

66

添加两个无符号的 32 位整数值, 存储在数据寄存器和全局孔径中的位置。如果设置了 SC0 比特, 则将原值从全局孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 += DATA.u32;  
RETURN_DATA.u32 = tmp
```

## GLOBAL\_ATOMIC\_SUB

67

从存储在全局孔径位置的数值中减去数据寄存器中无符号的 32 位整数值。如果设置了 SC0 位, 则将原值从全局孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
tmp = MEM[addr].u32;  
MEM[addr].u32 -= DATA.u32;  
RETURN_DATA.u32 = tmp
```

## GLOBAL\_ATOMIC\_SMIN

68

选择最少两个签名的 32 位整数输入, 条件是数据寄存在两个值和全局孔径中的位置。用选定的值更新全局孔径。如果设定了 SC0 位值, 请将原值从全局孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);  
tmp = MEM[addr].i32;  
src = DATA.i32;  
MEM[addr].i32 = src < tmp ? src : tmp;  
RETURN_DATA.i32 = tmp
```

## GLOBAL\_ATOMIC\_UMIN

69

选择两个无符号的 32 位整数输入的最小值, 因为在数据寄存器中有两个值, 并在全局孔径中有一个位置。用选定的值更新全局孔径。如果设置了 SC0 比特, 将把原值从全局孔径存储到向量寄存器中。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src < tmp ? src : tmp;
RETURN_DATA.u32 = tmp

```

## GLOBAL\_ATOMIC\_SMAX

70

选择最多两个签名的 32 位整数输入, 前提是有两个数值存储在数据寄存器中, 和一个位置位于全域孔径中。用所选值更新全域孔径。如果设置了 SC0 位, 将把原值存储到向量寄存器中。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].i32;
src = DATA.i32;
MEM[addr].i32 = src >= tmp ? src : tmp;
RETURN_DATA.i32 = tmp

```

## GLOBAL\_ATOMIC\_UMAX

71

选择两个无符号的 32 位整数输入的最大值, 因为在数据寄存器中有两个值, 并在全局孔径中有一个位置。用所选值更新全局孔径。如果设置了 SC0 位, 将把原值从全局孔径存储到向量寄存器中。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = src >= tmp ? src : tmp;
RETURN_DATA.u32 = tmp

```

## GLOBAL\_ATOMIC\_AND

72

以位法计算并给出两个无符号的 32 位整数值, 存储在数据寄存器和全局孔径中。如果设置了 SC0 位, 则将原值从全局孔径存储到向量寄存器。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp & DATA.b32);
RETURN_DATA.b32 = tmp

```

**GLOBAL\_ATOMIC\_OR****73**

计算位元 OR 给定两个无符号的 32 位整数存储在数据寄存器和全球孔径中的位置。如果设定了 SC0 位元,则将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp | DATA.b32);
RETURN_DATA.b32 = tmp
```

**GLOBAL\_ATOMIC\_XOR****74**

在数据寄存器和全域孔径中设置了两个无符号的 32 位整数。如果设置了 SC0 比特,则将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b32;
MEM[addr].b32 = (tmp ^ DATA.b32);
RETURN_DATA.b32 = tmp
```

**GLOBAL\_ATOMIC\_INC****75**

将一个无符号的 32 位整数从全域孔径中的位置加宽到 0, 如果数值超过数据寄存器中的值。如果设置了 SC0 位, 将把原值从全域孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = tmp >= src ? 0U : tmp + 1U;
RETURN_DATA.u32 = tmp
```

**GLOBAL\_ATOMIC\_DEC****76**

将一个无符号的 32 位整数从全域孔径中的位置, 并加包, 降为数据寄存器中值, 如果该寄存器产生负值的话。如果设定了 SC0 位, 将把原值从全域孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u32;
src = DATA.u32;
MEM[addr].u32 = ((tmp == 0U) || (tmp > src)) ? src : tmp - 1U;
```



```
RETURN_DATA.u32 = tmp
```

## GLOBAL\_ATOMIC\_ADD\_F32

77

在数据寄存器中添加一个单精度浮点值到全域孔径中的位置。如果设置了 SC0 比特, 则将原值从全域孔径存储到向量寄存器中。

```
tmp = MEM[ADDR].f32;  
MEM[ADDR].f32 += DATA.f32;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

## GLOBAL\_ATOMIC\_PK\_ADD\_F16

78

在数据寄存器中添加一个装入的 2 组件半精度浮点值到全域孔径中的位置。如果设置了 SC0 比特, 将原值从全域孔径存储到向量寄存器中。

```
tmp = MEM[ADDR];  
src = DATA;  
dst[31 : 16].f16 = tmp[31 : 16].f16 + src[31 : 16].f16;  
dst[15 : 0].f16 = tmp[15 : 0].f16 + src[15 : 0].f16;  
MEM[ADDR] = dst.b32;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

## GLOBAL\_ATOMIC\_ADD\_F64

79

在数据寄存器中添加一个双精度浮点值到全域孔径中的位置。如果设置了 SC0 比特, 则将原值从全域孔径存储到向量寄存器中。

```
tmp = MEM[ADDR].f64;  
MEM[ADDR].f64 += DATA.f64;  
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**GLOBAL\_ATOMIC\_MIN\_F64****80**

选择至少两个双精度浮点输入, 给定两个存储在数据寄存器和全域孔径中的位置。用所选值更新全域孔径。如果设置了 SC0 位值, 请将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src < tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

**GLOBAL\_ATOMIC\_MAX\_F64****81**

选择两个双精度浮点输入的最大值, 给两个存储在数据寄存器和全域孔径中的位置。用所选值更新全域孔径。如果设置了 SC0 位值, 请将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].f64;
src = DATA.f64;
MEM[addr].f64 = src > tmp ? src : tmp;
RETURN_DATA.f64 = tmp
```

**GLOBAL\_ATOMIC\_PK\_ADD\_BF16****82**

在数据寄存器中添加一个装有 2 个组件的 BF16 浮点值到全域孔径中的位置。如果设定了 SC0 比特, 则将原值从全域孔径存储到向量寄存器中。

```
tmp = MEM[ADDR];
src = DATA;
dst[31 : 16].bf16 = tmp[31 : 16].bf16 + src[31 : 16].bf16;
dst[15 : 0].bf16 = tmp[15 : 0].bf16 + src[15 : 0].bf16;
MEM[ADDR] = dst.b32;
RETURN_DATA = tmp
```

说明

浮点加法处理 NAN/INF/denorm.

**GLOBAL\_ATOMIC\_SWAP\_X2****96**

在数据寄存器中将无符号的 64 位整数与位于全域孔径中的位置相交换。如果设置了 SC0 比特,则将原值从全域孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = DATA.b64;
RETURN_DATA.b64 = tmp
```

## GLOBAL\_ATOMIC\_CMPSWAP\_X2

97

比较数据比较寄存器和全域孔径中两个无符号的 64 位整数。如果比较值相等,则修改数据源寄存器的内存位置。如果设定了 SC0 位值,则将原值从全域孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA[63 : 0].u64;
cmp = DATA[127 : 64].u64;
MEM[addr].u64 = tmp == cmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

## GLOBAL\_ATOMIC\_ADD\_X2

98

添加两个无符号的 64 位整数, 存储在数据寄存器和全球孔径中的位置。 如果设置了 SC0 比特, 将把原值从全球孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 += DATA.u64;
RETURN_DATA.u64 = tmp
```

## GLOBAL\_ATOMIC\_SUB\_X2

99

从存储在全域孔径位置的数值中减去一个无符号的 64 位整数。如果设置了 SC0 位,则将原值从全域孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
MEM[addr].u64 -= DATA.u64;
RETURN_DATA.u64 = tmp
```

**GLOBAL\_ATOMIC\_SMIN\_X2****100**

选择两个签名的 64 位整数输入的最小值, 因为有两个值存储在数据寄存器中, 和一个位置位于全局孔径中。用所选值更新全局孔径。如果设置了 SC0 比特, 请将原值从全局孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src < tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**GLOBAL\_ATOMIC\_UMIN\_X2****101**

选择两个没有符号的 64 位整数输入的最小值, 因为在数据寄存器中有两个值, 并在全局孔径中有一个位置。用所选值更新全局孔径。如果设置了 SC0 位, 请将原值从全局孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src < tmp ? src : tmp;
RETURN_DATA.u64 = tmp
```

**GLOBAL\_ATOMIC\_SMAX\_X2****102**

选择最多两个签名的 64 位整数输入, 给两个数值存储在数据寄存器和全局孔径中的位置。用选定的值更新全局孔径。如果设置了 SC0 比特, 请将原值从全局孔径存储到向量寄存器中。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].i64;
src = DATA.i64;
MEM[addr].i64 = src >= tmp ? src : tmp;
RETURN_DATA.i64 = tmp
```

**GLOBAL\_ATOMIC\_UMAX\_X2****103**

选择两个无符号的 64 位整数输入的最大值, 因为有两个数值存储在数据寄存器中, 和一个位置位于全局孔径中。用所选值更新全局孔径。如果设定了 SC0 位值, 请将原值从全局孔径存储到向量寄存器中。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = src >= tmp ? src : tmp;
RETURN_DATA.u64 = tmp

```

**GLOBAL\_ATOMIC\_AND\_X2****104**

以位法计算并给出两个无符号的 64 位整数, 存储在数据寄存器和全局孔径中的位置。如果设置了 SC0 位, 将把原值从全局孔径存储到向量寄存器。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp & DATA.b64);
RETURN_DATA.b64 = tmp

```

**GLOBAL\_ATOMIC\_OR\_X2****105**

计算位元 OR 给定了两个无符号的 64 位整数, 存储在数据寄存器和全局孔径中。如果设定了 SC0 位元, 则将原值从全局孔径存储到向量寄存器。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp | DATA.b64);
RETURN_DATA.b64 = tmp

```

**GLOBAL\_ATOMIC\_XOR\_X2****106**

计算位元 XOR 给定了两个无符号的 64 位整数, 存储在数据寄存器和全局孔径中。如果设置了 SC0 位, 则将原值从全局孔径存储到向量寄存器。

```

addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].b64;
MEM[addr].b64 = (tmp ^ DATA.b64);
RETURN_DATA.b64 = tmp

```

**GLOBAL\_ATOMIC\_INC\_X2****107**

将一个无符号的 64 位整数从全局孔径中的位置加宽到 0, 如果数值超过数据寄存器中的值。如果将原值从全局孔径存储到向量寄存器中。

设定了 SC0 位元。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = tmp >= src ? 0ULL : tmp + 1ULL;
RETURN_DATA.u64 = tmp
```

---

## GLOBAL\_ATOMIC\_DEC\_X2

**108**

将一个无符号的 64 位整数从环绕全球孔径的位置降为数据寄存器的负值。如果设定了 SC0 比特,则将原值从全球孔径存储到向量寄存器。

```
addr = CalcGlobalAddr(ADDR.b32, SADDR.b32, OFFSET.b32);
tmp = MEM[addr].u64;
src = DATA.u64;
MEM[addr].u64 = ((tmp == 0ULL) || (tmp > src)) ? src : tmp - 1ULL;
RETURN_DATA.u64 = tmp
```

## 12.16. 指令限制

### 12.16.1. DPP

下列指令不能使用 DPP：

- V\_MADMK\_F32
- V\_MADAK\_F32
- V\_MADMK\_F16
- V\_MADAK\_F16
- V\_READFIRSTLANE\_B32
- V\_CVT\_I32\_F64
- V\_CVT\_F64\_I32
- V\_CVT\_F32\_F64
- V\_CVT\_F64\_F32
- V\_CVT\_U32\_F64
- V\_CVT\_F64\_U32
- V\_TRUNC\_F64
- V\_CEIL\_F64
- V\_RNDNE\_F64
- V\_FLOOR\_F64
- V\_RCP\_F64
- V\_RSQ\_F64
- V\_SQRT\_F64
- V\_FREXP\_EXP\_I32\_F64
- V\_FREXP\_MANT\_F64
- V\_FRACT\_F64
- V\_CLREXCP
- V\_SWAP\_B32
- V\_CMP\_CLASS\_F64
- V\_CMPX\_CLASS\_F64
- V\_CMP\_\*\_F64
- V\_CMPX\_\*\_F64
- V\_CMP\_\*\_I64
- V\_CMP\_\*\_U64
- V\_CMPX\_\*\_I64
- V\_CMPX\_\*\_U64

### 12.16.2. SDWA

以下指令不能使用 SDWA：

- V\_MAC\_F32
- V\_MADMK\_F32

- V\_MADAK\_F32
- V\_MAC\_F16
- V\_MADMK\_F16
- V\_MADAK\_F16
- V\_FMAC\_F32
- V\_READFIRSTLANE\_B32
- V\_CLREXCP
- V\_SWAP\_B32



# 第 13 章 微码格式

本节指定了微码格式。定义可以通过提供各种指令格式的标准模板和点名来简化汇编。

Endian Order - CDNA 架构用很少的端元字节排序和位元排序处理内存和登记。多位字节值用最小的(低序)字节(LSB)存储在最小的字节地址,它们用它们的 LSB 在右边显示。字节值用最小的(低序)字节(lsb)存储在最小的位地址,用它们的 lsb 在右边显示。

下表汇总了微码格式及其宽度。以下各节提供细节

表 63. 微码格式摘要

| 微码格式           | 参考文献                      | 宽度( 位数) |
|----------------|---------------------------|---------|
| 标量 ALU 与控制指令格式 |                           |         |
| SOP2           | <a href="#">SOP2</a>      | 32      |
| SOP1           | <a href="#">SOP1</a>      |         |
| SOPK           | <a href="#">SOPK</a>      |         |
| SOPP           | <a href="#">SOPP</a>      |         |
| SOPC           | <a href="#">SOPC</a>      |         |
| 标量内存格式         |                           |         |
| SMEM           | <a href="#">SMEM</a>      | 64      |
| 向量 ALU 格式      |                           |         |
| VOP1           | <a href="#">VOP1</a>      | 32      |
| VOP2           | <a href="#">VOP2</a>      | 32      |
| VOPC           | <a href="#">VOPC</a>      | 32      |
| VOP3A          | <a href="#">VOP3A</a>     | 64      |
| VOP3B          | <a href="#">VOP3B</a>     | 64      |
| VOP3P          | <a href="#">VOP3P</a>     | 64      |
| VOP3P-MAI      | <a href="#">VOP3P-MAI</a> | 64      |
| DPP            | <a href="#">DPP</a>       | 32      |
| SDWA           | <a href="#">VOP2</a>      | 32      |
| LDS/GWS 格式     |                           |         |
| DS             | <a href="#">DS</a>        | 64      |
| 向量内存缓冲区格式      |                           |         |
| MTBUF          | <a href="#">MTBUF</a>     | 64      |
| MUBUF          | <a href="#">MUBUF</a>     | 64      |
| Flat 格式        |                           |         |
| FLAT           | <a href="#">FLAT</a>      | 64      |
| GLOBAL         | <a href="#">GLOBAL</a>    | 64      |
| SCRATCH        | <a href="#">SCRATCH</a>   | 64      |

以下各节说明所附的外地定义表格使用下列注解。

- int(2) - 指定无符号整数的双位字段。
- enum(7) - 一个七位字段,指定了一组已列出的值(在这种情况下,最多为 27 个值)。有效值的数量可能小于最大值。

所有字段的默认值为零。任何未识别的比特字段都被假定为保留 。

指令后缀

多数指令包含一个后缀,表示指令手柄的数据类型。这个后缀可能还包括一个数字,表示数据大小。

例如:"F32"表示"32 位浮点数据",或"B16"表示"16 位二进制数据".

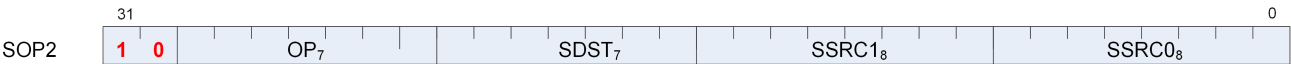
- B = 二进制
- F = 浮点
- U = 无符号整数
- S = 签名整数

当一个指令中出现一个以上的数据类型描述符时,最后一个结果是结果类型和大小,而前一个(s)是/是输入数据类型和大小.

13.1. 标量 ALU 与控制指令格式

13.1.1. SOP2

带有两个输入, 一个输出的标量格式



|    |                                           |
|----|-------------------------------------------|
| 格式 | SOP2                                      |
| 说明 | 这是一个带有两个输入和一个输出的刻度指令。 之后可以有一个 32 位的字元常数 。 |

表 64. SOP2 字段

| 字段名称     | 位         | 格式或说明                                  |
|----------|-----------|----------------------------------------|
| SSRC0    | [7:0]     | 源操作数 0，即本指令的第一个操作数。                    |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。               |
|          | 102       | FLAT_SCRATCH_LO.                       |
|          | 103       | FLAT_SCRATCH_HI.                       |
|          | 104       | XNACK_MASK_LO.                         |
|          | 105       | XNACK_MASK_HI.                         |
|          | 106       | VCC_LO: vcc[31:0]。                     |
|          | 107       | VCC_HI: vcc[63:32]。                    |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。            |
|          | 124       | M0：内存寄存器 0。                            |
|          | 125       | 保留                                     |
|          | 126       | EXEC_LO: exec[31:0]。                   |
|          | 127       | EXEC_HI: exec[63:32]。                  |
|          | 128       | 0.                                     |
|          | 129-192   | 有符号整数 1 至 64。                          |
|          | 193-208   | 有符号整数 -1 至 -16。                        |
|          | 209-234   | 保留。                                    |
|          | 235       | SHARED_BASE（内存窗口定义）。                   |
|          | 236       | SHARED_LIMIT（内存窗口定义）。                  |
|          | 237       | PRIVATE_BASE（内存窗口定义）。                  |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。                 |
|          | 239       | RESERVED .                             |
|          | 240       | 0.5.                                   |
|          | 241       | -0.5.                                  |
|          | 242       | 1.0.                                   |
|          | 243       | -1.0.                                  |
|          | 244       | 2.0.                                   |
|          | 245       | -2.0.                                  |
|          | 246       | 4.0.                                   |
|          | 247       | -4.0.                                  |
|          | 248       | 1/(2*PI).                              |
|          | 249 - 250 | 保留。                                    |
|          | 251       | VCCZ.                                  |
|          | 252       | EXECZ.                                 |
|          | 253       | SCC.                                   |
|          | 254       | 保留。                                    |
|          | 255       | 字面量常量。                                 |
| SSRC1    | [15:8]    | 第二次标量源代码操作,与上面的 SSRC0 相同.              |
| SDST     | [22:16]   | 标量目标操作数。编码与上面的 SSRC0 相同，但仅编码 0-127 有效。 |
| OP       | [29:23]   | 参见以下 Opcode 表.                         |
| ENCODING | [31:30]   | 必须是: 10                                |

表 65. SOP2 Opcode

| 操作码编号 | 名称         | 操作码编号 | 名称         |
|-------|------------|-------|------------|
| 0     | S_ADD_U32  | 26    | S_XNOR_B32 |
| 1     | S_SUB_U32  | 27    | S_XNOR_B64 |
| 2     | S_ADD_I32  | 28    | S_LSHL_B32 |
| 3     | S_SUB_I32  | 29    | S_LSHL_B64 |
| 4     | S_ADDC_U32 | 30    | S_LSHR_B32 |
| 5     | S_SUBB_U32 | 31    | S_LSHR_B64 |
| 6     | S_MIN_I32  | 32    | S_ASHR_I32 |

| 操作码编号 | 名称            | 操作码编号 | 名称                |
|-------|---------------|-------|-------------------|
| 7     | S_MIN_U32     | 33    | S_ASHR_I64        |
| 8     | S_MAX_I32     | 34    | S_BFM_B32         |
| 9     | S_MAX_U32     | 35    | S_BFM_B64         |
| 10    | S_CSELECT_B32 | 36    | S_MUL_I32         |
| 11    | S_CSELECT_B64 | 37    | S_BFE_U32         |
| 12    | S_AND_B32     | 38    | S_BFE_I32         |
| 13    | S_AND_B64     | 39    | S_BFE_U64         |
| 14    | S_OR_B32      | 40    | S_BFE_I64         |
| 15    | S_OR_B64      | 41    | S_CBRANCH_G_FORK  |
| 16    | S_XOR_B32     | 42    | S_ABSDIFF_I32     |
| 17    | S_XOR_B64     | 44    | S_MUL_HI_U32      |
| 18    | S_ANDN2_B32   | 45    | S_MUL_HI_I32      |
| 19    | S_ANDN2_B64   | 46    | S_LSHL1_ADD_U32   |
| 20    | S_ORN2_B32    | 47    | S_LSHL2_ADD_U32   |
| 21    | S_ORN2_B64    | 48    | S_LSHL3_ADD_U32   |
| 22    | S_NAND_B32    | 49    | S_LSHL4_ADD_U32   |
| 23    | S_NAND_B64    | 50    | S_PACK_LL_B32_B16 |
| 24    | S_NOR_B32     | 51    | S_PACK_LH_B32_B16 |
| 25    | S_NOR_B64     | 52    | S_PACK_HH_B32_B16 |

### 13.1.2. SOPK



格式 SOPK

说明 这是一个刻度指令,一个 16 位签名的即时输入(SIMM16)和一个单一的目的地。使用 2 个输入指令将目的地作为第二个输入。

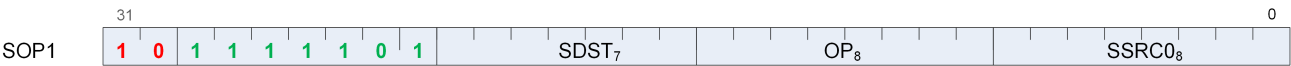
表 66. SOPK 字段

| 字段名称     | 位                                                                                                   | 格式或说明                                                                                                                                                                                                                                                                          |
|----------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SIMM16   | [15:0]                                                                                              | 有符号的 16 位值。                                                                                                                                                                                                                                                                    |
| SDST     | [22:16]<br>0 - 101<br>102<br>103<br>104<br>105<br>106<br>107<br>108-123<br>124<br>125<br>126<br>127 | 标量目标操作数，也可提供第二个源操作数。<br>SGPR0 至 SGPR101：标量通用寄存器。<br>FLAT_SCRATCH_LO.<br>FLAT_SCRATCH_HI.<br>XNACK_MASK_LO.<br>XNACK_MASK_HI.<br>VCC_LO: vcc[31:0]。<br>VCC_HI: vcc[63:32]。<br>TTMP0 至 TTMP15：陷阱处理程序临时寄存器。<br>M0：内存寄存器 0。<br>保留<br>EXEC_LO: exec[31:0]。<br>EXEC_HI: exec[63:32]。 |
| OP       | [27:23]                                                                                             | 参见以下 Opcode 表。                                                                                                                                                                                                                                                                 |
| ENCODING | [31:28]                                                                                             | 必须是: 1011                                                                                                                                                                                                                                                                      |

表 67. SOPK Opcode

| 操作码编号 | 名称            | 操作码编号 | 名称                 |
|-------|---------------|-------|--------------------|
| 0     | S_MOVK_I32    | 11    | S_CMPK_GE_U32      |
| 1     | S_CMOVK_I32   | 12    | S_CMPK_LT_U32      |
| 2     | S_CMPK_EQ_I32 | 13    | S_CMPK_LE_U32      |
| 3     | S_CMPK_LG_I32 | 14    | S_ADDK_I32         |
| 4     | S_CMPK_GT_I32 | 15    | S_MULK_I32         |
| 5     | S_CMPK_GE_I32 | 16    | S_CBRANCH_I_FORK   |
| 6     | S_CMPK_LT_I32 | 17    | S_GETREG_B32       |
| 7     | S_CMPK_LE_I32 | 18    | S_SETREG_B32       |
| 8     | S_CMPK_EQ_U32 | 20    | S_SETREG_IMM32_B32 |
| 9     | S_CMPK_LG_U32 | 21    | S_CALL_B64         |
| 10    | S_CMPK_GT_U32 |       |                    |

13.1.3. SOP1



格式 SOP1

说明 这是一个带有两个输入和一个输出的刻度指令。之后可以有一个 32 位的字元常数。

表 68. SOP1 字段

| 字段名称     | 位         | 格式或说明                                  |
|----------|-----------|----------------------------------------|
| SSRC0    | [7:0]     | 源操作数 0，即本指令的第一个操作数。                    |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。               |
|          | 102       | FLAT_SCRATCH_LO。                       |
|          | 103       | FLAT_SCRATCH_HI。                       |
|          | 104       | XNACK_MASK_LO。                         |
|          | 105       | XNACK_MASK_HI。                         |
|          | 106       | VCC_LO: vcc[31:0]。                     |
|          | 107       | VCC_HI: vcc[63:32]。                    |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。            |
|          | 124       | M0：内存寄存器 0。                            |
|          | 125       | 保留                                     |
|          | 126       | EXEC_LO: exec[31:0]。                   |
|          | 127       | EXEC_HI: exec[63:32]。                  |
|          | 128       | 0。                                     |
|          | 129-192   | 有符号整数 1 至 64。                          |
|          | 193-208   | 有符号整数 -1 至 -16。                        |
|          | 209-234   | 保留。                                    |
|          | 235       | SHARED_BASE（内存窗口定义）。                   |
|          | 236       | SHARED_LIMIT（内存窗口定义）。                  |
|          | 237       | PRIVATE_BASE（内存窗口定义）。                  |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。                 |
|          | 239       | RESERVED。                              |
|          | 240       | 0.5。                                   |
|          | 241       | -0.5。                                  |
|          | 242       | 1.0。                                   |
|          | 243       | -1.0。                                  |
|          | 244       | 2.0。                                   |
|          | 245       | -2.0。                                  |
|          | 246       | 4.0。                                   |
|          | 247       | -4.0。                                  |
|          | 248       | $1/(2*\pi)$ 。                          |
|          | 249 - 250 | 保留。                                    |
|          | 251       | VCCZ。                                  |
|          | 252       | EXECZ。                                 |
|          | 253       | SCC。                                   |
|          | 254       | 保留。                                    |
|          | 255       | 字面量常量。                                 |
| OP       | [15:8]    | 参见以下 Opcode 表。                         |
| SDST     | [22:16]   | 标量目标操作数。编码与上面的 SSRC0 相同，但仅编码 0-127 有效。 |
| ENCODING | [31:23]   | 必须是: 10_1111101                        |

表 69. SOP1 操作码

| 操作码编号 | 名称         | 操作码编号 | 名称                 |
|-------|------------|-------|--------------------|
| 0     | S_MOV_B32  | 27    | S_BITSET1_B64      |
| 1     | S_MOV_B64  | 28    | S_GETPC_B64        |
| 2     | S_CMOV_B32 | 29    | S_SETPC_B64        |
| 3     | S_CMOV_B64 | 30    | S_SWAPPC_B64       |
| 4     | S_NOT_B32  | 31    | S_RFE_B64          |
| 5     | S_NOT_B64  | 32    | S_AND_SAVEEXEC_B64 |
| 6     | S_WQM_B32  | 33    | S_OR_SAVEEXEC_B64  |
| 7     | S_WQM_B64  | 34    | S_XOR_SAVEEXEC_B64 |

| 操作码编号 | 名称              | 操作码编号 | 名称                     |
|-------|-----------------|-------|------------------------|
| 8     | S_BREV_B32      | 35    | S_ANDN2_SAVEEXEC_B64   |
| 9     | S_BREV_B64      | 36    | S_ORN2_SAVEEXEC_B64    |
| 10    | S_BCNT0_I32_B32 | 37    | S_NAND_SAVEEXEC_B64    |
| 11    | S_BCNT0_I32_B64 | 38    | S_NOR_SAVEEXEC_B64     |
| 12    | S_BCNT1_I32_B32 | 39    | S_XNOR_SAVEEXEC_B64    |
| 13    | S_BCNT1_I32_B64 | 40    | S_QUADMASK_B32         |
| 14    | S_FF0_I32_B32   | 41    | S_QUADMASK_B64         |
| 15    | S_FF0_I32_B64   | 42    | S_MOVRELS_B32          |
| 16    | S_FF1_I32_B32   | 43    | S_MOVRELS_B64          |
| 17    | S_FF1_I32_B64   | 44    | S_MOVRELD_B32          |
| 18    | S_FLBIT_I32_B32 | 45    | S_MOVRELD_B64          |
| 19    | S_FLBIT_I32_B64 | 46    | S_CBRANCH_JOIN         |
| 20    | S_FLBIT_I32     | 48    | S_ABS_I32              |
| 21    | S_FLBIT_I32_I64 | 50    | S_SET_GPR_IDX_IDX      |
| 22    | S_SEXT_I32_I8   | 51    | S_ANDN1_SAVEEXEC_B64   |
| 23    | S_SEXT_I32_I16  | 52    | S_ORN1_SAVEEXEC_B64    |
| 24    | S_BITSET0_B32   | 53    | S_ANDN1_WREXEC_B64     |
| 25    | S_BITSET0_B64   | 54    | S_ANDN2_WREXEC_B64     |
| 26    | S_BITSET1_B32   | 55    | S_BITREPLICATE_B64_B32 |

13.1.4. SOPC



格式 SOPC

说明 这是一个有两种输入的刻度指令, 并因此比较并生成 SCC 。 之后可以有一个 32 位的字元常数。

表 70. SOPC 字段

| 字段名称     | 位         | 格式或说明                       |
|----------|-----------|-----------------------------|
| SSRC0    | [7:0]     | 源操作数 0，即本指令的第一个操作数。         |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。    |
|          | 102       | FLAT_SCRATCH_LO.            |
|          | 103       | FLAT_SCRATCH_HI.            |
|          | 104       | XNACK_MASK_LO.              |
|          | 105       | XNACK_MASK_HI.              |
|          | 106       | VCC_LO: vcc[31:0]。          |
|          | 107       | VCC_HI: vcc[63:32]。         |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。 |
|          | 124       | M0：内存寄存器 0。                 |
|          | 125       | 保留                          |
|          | 126       | EXEC_LO: exec[31:0]。        |
|          | 127       | EXEC_HI: exec[63:32]。       |
|          | 128       | 0.                          |
|          | 129-192   | 有符号整数 1 至 64。               |
|          | 193-208   | 有符号整数 -1 至 -16。             |
|          | 209-234   | 保留。                         |
|          | 235       | SHARED_BASE（内存窗口定义）。        |
|          | 236       | SHARED_LIMIT（内存窗口定义）。       |
|          | 237       | PRIVATE_BASE（内存窗口定义）。       |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。      |
|          | 239       | RESERVED .                  |
|          | 240       | 0.5.                        |
|          | 241       | -0.5.                       |
|          | 242       | 1.0.                        |
|          | 243       | -1.0.                       |
|          | 244       | 2.0.                        |
|          | 245       | -2.0.                       |
|          | 246       | 4.0.                        |
|          | 247       | -4.0.                       |
|          | 248       | 1/(2*PI).                   |
|          | 249 - 250 | 保留。                         |
|          | 251       | VCCZ.                       |
|          | 252       | EXECZ.                      |
|          | 253       | SCC.                        |
|          | 254       | 保留。                         |
|          | 255       | 字面量常量。                      |
| SSRC1    | [15:8]    | 第二次标量源代码操作,与上面的SSRC0 相同.    |
| OP       | [22:16]   | 参见以下 Opcode 表.              |
| ENCODING | [31:23]   | 必须是: 10_11110               |

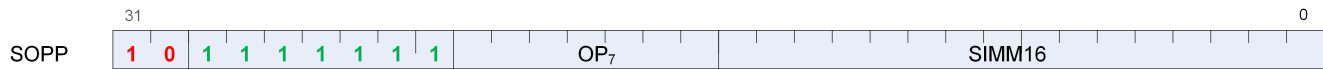
表 71. SOPC Opcodes

| 操作码编号 | 名称           | 操作码编号 | 名称               |
|-------|--------------|-------|------------------|
| 0     | S_CMP_EQ_I32 | 10    | S_CMP_LT_U32     |
| 1     | S_CMP_LG_I32 | 11    | S_CMP_LE_U32     |
| 2     | S_CMP_GT_I32 | 12    | S_BITCMP0_B32    |
| 3     | S_CMP_GE_I32 | 13    | S_BITCMP1_B32    |
| 4     | S_CMP_LT_I32 | 14    | S_BITCMP0_B64    |
| 5     | S_CMP_LE_I32 | 15    | S_BITCMP1_B64    |
| 6     | S_CMP_EQ_U32 | 16    | S_SETVSKIP       |
| 7     | S_CMP_LG_U32 | 17    | S_SET_GPR_IDX_ON |



| 操作码编号 | 名称           | 操作码编号 | 名称           |
|-------|--------------|-------|--------------|
| 8     | S_CMP_GT_U32 | 18    | S_CMP_EQ_U64 |
| 9     | S_CMP_GE_U32 | 19    | S_CMP_LG_U64 |

13.1.5. SOPP



格式 SOPP

说明 这是一个刻度指令,有一个 16 位签名的即时输入(SIMM16).

表 72. SOPP 字段

| 字段名称     | 位       | 格式或说明            |
|----------|---------|------------------|
| SIMM16   | [15:0]  | 有符号的 16 位值。      |
| OP       | [22:16] | 参见以下 Opcode 表.   |
| ENCODING | [31:23] | 必须是: 10_11111111 |

表 73. SOPP Opcode

| 操作码编号 | 名称                | 操作码编号 | 名称                         |
|-------|-------------------|-------|----------------------------|
| 0     | S_NOP             | 15    | S_SETPRIO                  |
| 1     | S_ENDPGM          | 16    | S_SENDMSG                  |
| 2     | S_BRANCH          | 17    | S_SENDMSGHALT              |
| 3     | S_WAKEUP          | 18    | S_TRAP                     |
| 4     | S_CBRANCH_SCC0    | 19    | S_ICACHE_INV               |
| 5     | S_CBRANCH_SCC1    | 20    | S_INCPERFLEVEL             |
| 6     | S_CBRANCH_VCCZ    | 21    | S_DECPERFLEVEL             |
| 7     | S_CBRANCH_VCCNZ   | 22    | S_TTRACEDATA               |
| 8     | S_CBRANCH_EXECZ   | 23    | S_CBRANCH_CDBGSYS          |
| 9     | S_CBRANCH_EXECONZ | 24    | S_CBRANCH_CDBGUSER         |
| 10    | S_BARRIER         | 25    | S_CBRANCH_CDBGSYS_OR_USER  |
| 11    | S_SETKILL         | 26    | S_CBRANCH_CDBGSYS_AND_USER |
| 12    | S_WAITCNT         | 27    | S_ENDPGM_SAVED             |
| 13    | S_SETHALT         | 28    | S_SET_GPR_IDX_OFF          |
| 14    | S_SLEEP           | 29    | S_SET_GPR_IDX_MODE         |

13.2. 标量内存格式

13.2.1. SMEM



|    |             |
|----|-------------|
| 格式 | SMEM        |
| 说明 | 标量内存数据负载/存储 |

表 74. SMEM 字段

| 字段名称     | 位       | 格式或说明                                                     |
|----------|---------|-----------------------------------------------------------|
| SBASE    | [5:0]   | SGPR-pair 提供基址或 SGPR-quad 提供 V#. (LSB of SGPR 地址省略).      |
| SDATA    | [12:6]  | SGPR 提供写入数据或接受返回数据.                                       |
| SOE      | [14]    | 标量偏移使能.                                                   |
| NV       | [15]    | 非挥发性                                                      |
| GLC      | [16]    | 全球内存 Coherent. 强制绕行 L1 和 L2 缓存, 或用于动漫, 导致返回预操作值。          |
| IMM      | [17]    | 即时启动.                                                     |
| OP       | [25:18] | 参见以下 Opcode 表.                                            |
| ENCODING | [31:26] | 0) 琰: 110000                                              |
| OFFSET   | [52:32] | 即时签名的字节对冲, 或 SGPR 持有无符号字节对冲的地址. 有符号的字节只对 S_LOAD/STORE 工作. |
| SOFFSET  | [63:57] | SGPR 偏移. 仅在 SOFFSET_EN = 5 月 1 日只指定 SGPR 或 M0 时使用.        |

表 75. SMEM Opcode

| 操作码编号 | 名称                      | 操作码编号 | 名称                         |
|-------|-------------------------|-------|----------------------------|
| 0     | S_LOAD_DWORD            | 75    | S_BUFFER_ATOMIC_INC        |
| 1     | S_LOAD_DWORDX2          | 76    | S_BUFFER_ATOMIC_DEC        |
| 2     | S_LOAD_DWORDX4          | 96    | S_BUFFER_ATOMIC_SWAP_X2    |
| 3     | S_LOAD_DWORDX8          | 97    | S_BUFFER_ATOMIC_CMPSWAP_X2 |
| 4     | S_LOAD_DWORDX16         | 98    | S_BUFFER_ATOMIC_ADD_X2     |
| 5     | S_SCRATCH_LOAD_DWORD    | 99    | S_BUFFER_ATOMIC_SUB_X2     |
| 6     | S_SCRATCH_LOAD_DWORDX2  | 100   | S_BUFFER_ATOMIC_SMIN_X2    |
| 7     | S_SCRATCH_LOAD_DWORDX4  | 101   | S_BUFFER_ATOMIC_UMIN_X2    |
| 8     | S_BUFFER_LOAD_DWORD     | 102   | S_BUFFER_ATOMIC_SMAX_X2    |
| 9     | S_BUFFER_LOAD_DWORDX2   | 103   | S_BUFFER_ATOMIC_UMAX_X2    |
| 10    | S_BUFFER_LOAD_DWORDX4   | 104   | S_BUFFER_ATOMIC_AND_X2     |
| 11    | S_BUFFER_LOAD_DWORDX8   | 105   | S_BUFFER_ATOMIC_OR_X2      |
| 12    | S_BUFFER_LOAD_DWORDX16  | 106   | S_BUFFER_ATOMIC_XOR_X2     |
| 16    | S_STORE_DWORD           | 107   | S_BUFFER_ATOMIC_INC_X2     |
| 17    | S_STORE_DWORDX2         | 108   | S_BUFFER_ATOMIC_DEC_X2     |
| 18    | S_STORE_DWORDX4         | 128   | S_ATOMIC_SWAP              |
| 21    | S_SCRATCH_STORE_DWORD   | 129   | S_ATOMIC_CMPSWAP           |
| 22    | S_SCRATCH_STORE_DWORDX2 | 130   | S_ATOMIC_ADD               |
| 23    | S_SCRATCH_STORE_DWORDX4 | 131   | S_ATOMIC_SUB               |
| 24    | S_BUFFER_STORE_DWORD    | 132   | S_ATOMIC_SMIN              |
| 25    | S_BUFFER_STORE_DWORDX2  | 133   | S_ATOMIC_UMIN              |
| 26    | S_BUFFER_STORE_DWORDX4  | 134   | S_ATOMIC_SMAX              |
| 32    | S_DCACHE_INV            | 135   | S_ATOMIC_UMAX              |
| 33    | S_DCACHE_WB             | 136   | S_ATOMIC_AND               |
| 34    | S_DCACHE_INV_VOL        | 137   | S_ATOMIC_OR                |
| 35    | S_DCACHE_WB_VOL         | 138   | S_ATOMIC_XOR               |
| 36    | S_MEMTIME               | 139   | S_ATOMIC_INC               |

| 操作码编号 | 名称                      | 操作码编号 | 名称                  |
|-------|-------------------------|-------|---------------------|
| 37    | S_MEMREALTIME           | 140   | S_ATOMIC_DEC        |
| 40    | S_DCACHE_DISCARD        | 160   | S_ATOMIC_SWAP_X2    |
| 41    | S_DCACHE_DISCARD_X2     | 161   | S_ATOMIC_CMPSWAP_X2 |
| 64    | S_BUFFER_ATOMIC_SWAP    | 162   | S_ATOMIC_ADD_X2     |
| 65    | S_BUFFER_ATOMIC_CMPSWAP | 163   | S_ATOMIC_SUB_X2     |
| 66    | S_BUFFER_ATOMIC_ADD     | 164   | S_ATOMIC_SMIN_X2    |
| 67    | S_BUFFER_ATOMIC_SUB     | 165   | S_ATOMIC_UMIN_X2    |
| 68    | S_BUFFER_ATOMIC_SMIN    | 166   | S_ATOMIC_SMAX_X2    |
| 69    | S_BUFFER_ATOMIC_UMIN    | 167   | S_ATOMIC_UMAX_X2    |
| 70    | S_BUFFER_ATOMIC_SMAX    | 168   | S_ATOMIC_AND_X2     |
| 71    | S_BUFFER_ATOMIC_UMAX    | 169   | S_ATOMIC_OR_X2      |
| 72    | S_BUFFER_ATOMIC_AND     | 170   | S_ATOMIC_XOR_X2     |
| 73    | S_BUFFER_ATOMIC_OR      | 171   | S_ATOMIC_INC_X2     |
| 74    | S_BUFFER_ATOMIC_XOR     | 172   | S_ATOMIC_DEC_X2     |

### 13.3. 向量 ALU 格式

#### 13.3.1. VOP2

|      |                                                                                                                                                             |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VOP2 | <div><div>31</div><div>0</div><div>0</div><div>OP<sub>6</sub></div><div>VDST<sub>8</sub></div><div>VSRC1<sub>8</sub></div><div>SRC0<sub>9</sub></div></div> |
| 格式   | VOP2                                                                                                                                                        |
| 说明   | 向量 ALU 格式,有两个操作符                                                                                                                                            |

表 76. VOP2 字段



| 字段名称     | 位         | 格式或说明                       |
|----------|-----------|-----------------------------|
| SRC0     | [8:0]     | 源操作数 0，即本指令的第一个操作数。         |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。    |
|          | 102       | FLAT_SCRATCH_LO.            |
|          | 103       | FLAT_SCRATCH_HI.            |
|          | 104       | XNACK_MASK_LO.              |
|          | 105       | XNACK_MASK_HI.              |
|          | 106       | VCC_LO: vcc[31:0]。          |
|          | 107       | VCC_HI: vcc[63:32]。         |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。 |
|          | 124       | M0：内存寄存器 0。                 |
|          | 125       | 保留                          |
|          | 126       | EXEC_LO: exec[31:0]。        |
|          | 127       | EXEC_HI: exec[63:32]。       |
|          | 128       | 0.                          |
|          | 129-192   | 有符号整数 1 至 64。               |
|          | 193-208   | 有符号整数 -1 至 -16。             |
|          | 209-234   | 保留。                         |
|          | 235       | SHARED_BASE（内存窗口定义）。        |
|          | 236       | SHARED_LIMIT（内存窗口定义）。       |
|          | 237       | PRIVATE_BASE（内存窗口定义）。       |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。      |
|          | 239       | RESERVED .                  |
|          | 240       | 0.5.                        |
|          | 241       | -0.5.                       |
|          | 242       | 1.0.                        |
|          | 243       | -1.0.                       |
|          | 244       | 2.0.                        |
|          | 245       | -2.0.                       |
|          | 246       | 4.0.                        |
|          | 247       | -4.0.                       |
|          | 248       | 1/(2*PI).                   |
|          | 249       | SDWA                        |
|          | 250       | DPP                         |
|          | 251       | VCCZ.                       |
|          | 252       | EXECZ.                      |
|          | 253       | SCC.                        |
|          | 254       | 保留。                         |
|          | 255       | 字面量常量。                      |
|          | 256 - 511 | VGPR 0 - 255                |
| VSRC1    | [16:9]    | VGPR 提供第二个操作。               |
| VDST     | [24:17]   | 目标 VGPR。                    |
| OP       | [30:25]   | 参见以下 Opcode 表。              |
| ENCODING | [31]      | 必须是: 0                      |

表 77. VOP2 操作码

| 操作码编号 | 名称            | 操作码编号 | 名称           |
|-------|---------------|-------|--------------|
| 0     | V_CNDMASK_B32 | 32    | V_SUB_F16    |
| 1     | V_ADD_F32     | 33    | V_SUBREV_F16 |
| 2     | V_SUB_F32     | 34    | V_MUL_F16    |
| 3     | V_SUBREV_F32  | 35    | V_MAC_F16    |
| 4     | V_FMAC_F64    | 36    | V_MADMK_F16  |
| 5     | V_MUL_F32     | 37    | V_MADAK_F16  |
| 6     | V_MUL_I32_I24 | 38    | V_ADD_U16    |

| 操作码编号 | 名称               | 操作码编号 | 名称              |
|-------|------------------|-------|-----------------|
| 7     | V_MUL_HI_I32_I24 | 39    | V_SUB_U16       |
| 8     | V_MUL_U32_U24    | 40    | V_SUBREV_U16    |
| 9     | V_MUL_HI_U32_U24 | 41    | V_MUL_LO_U16    |
| 10    | V_MIN_F32        | 42    | V_LSHLREV_B16   |
| 11    | V_MAX_F32        | 43    | V_LSHRREV_B16   |
| 12    | V_MIN_I32        | 44    | V_ASHRREV_I16   |
| 13    | V_MAX_I32        | 45    | V_MAX_F16       |
| 14    | V_MIN_U32        | 46    | V_MIN_F16       |
| 15    | V_MAX_U32        | 47    | V_MAX_U16       |
| 16    | V_LSHRREV_B32    | 48    | V_MAX_I16       |
| 17    | V_ASHRREV_I32    | 49    | V_MIN_U16       |
| 18    | V_LSHLREV_B32    | 50    | V_MIN_I16       |
| 19    | V_AND_B32        | 51    | V_LDEXP_F16     |
| 20    | V_OR_B32         | 52    | V_ADD_U32       |
| 21    | V_XOR_B32        | 53    | V_SUB_U32       |
| 23    | V_FMAMK_F32      | 54    | V_SUBREV_U32    |
| 24    | V_FMAAK_F32      | 55    | V_DOT2C_F32_F16 |
| 25    | V_ADD_CO_U32     | 56    | V_DOT2C_I32_I16 |
| 26    | V_SUB_CO_U32     | 57    | V_DOT4C_I32_I8  |
| 27    | V_SUBREV_CO_U32  | 58    | V_DOT8C_I32_I4  |
| 28    | V_ADDC_CO_U32    | 59    | V_FMAC_F32      |
| 29    | V_SUBB_CO_U32    | 60    | V_PK_FMAC_F16   |
| 30    | V_SUBBREV_CO_U32 | 61    | V_XNOR_B32      |
| 31    | V_ADD_F16        |       |                 |

13.3.2. VOP1



格式 VOP1

说明 向量 ALU 格式,有一个操作符

表 78. VOP1 字段

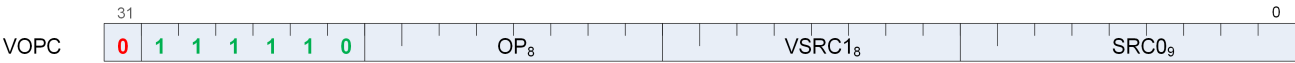
| 字段名称     | 位         | 格式或说明                       |
|----------|-----------|-----------------------------|
| SRC0     | [8:0]     | 源操作数 0，即本指令的第一个操作数。         |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。    |
|          | 102       | FLAT_SCRATCH_LO.            |
|          | 103       | FLAT_SCRATCH_HI.            |
|          | 104       | XNACK_MASK_LO.              |
|          | 105       | XNACK_MASK_HI.              |
|          | 106       | VCC_LO: vcc[31:0]。          |
|          | 107       | VCC_HI: vcc[63:32]。         |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。 |
|          | 124       | M0：内存寄存器 0。                 |
|          | 125       | 保留                          |
|          | 126       | EXEC_LO: exec[31:0]。        |
|          | 127       | EXEC_HI: exec[63:32]。       |
|          | 128       | 0.                          |
|          | 129-192   | 有符号整数 1 至 64。               |
|          | 193-208   | 有符号整数 -1 至 -16。             |
|          | 209-234   | 保留。                         |
|          | 235       | SHARED_BASE（内存窗口定义）。        |
|          | 236       | SHARED_LIMIT（内存窗口定义）。       |
|          | 237       | PRIVATE_BASE（内存窗口定义）。       |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。      |
|          | 239       | RESERVED .                  |
|          | 240       | 0.5.                        |
|          | 241       | -0.5.                       |
|          | 242       | 1.0.                        |
|          | 243       | -1.0.                       |
|          | 244       | 2.0.                        |
|          | 245       | -2.0.                       |
|          | 246       | 4.0.                        |
|          | 247       | -4.0.                       |
|          | 248       | 1/(2*PI).                   |
|          | 249       | SDWA                        |
|          | 250       | DPP                         |
|          | 251       | VCCZ.                       |
|          | 252       | EXECZ.                      |
|          | 253       | SCC.                        |
|          | 254       | 保留。                         |
|          | 255       | 字面量常量。                      |
|          | 256 - 511 | VGPR 0 - 255                |
| OP       | [16:9]    | 参见以下 Opcode 表.              |
| VDST     | [24:17]   | 目标 VGPR。                    |
| ENCODING | [31:25]   | 必须是: 0_111111               |

表 79. VOP1 Opcode

| 操作码编号 | 名称                  | 操作码编号 | 名称                  |
|-------|---------------------|-------|---------------------|
| 0     | V_NOP               | 42    | V_COS_F32           |
| 1     | V_MOV_B32           | 43    | V_NOT_B32           |
| 2     | V_READFIRSTLANE_B32 | 44    | V_BFREV_B32         |
| 3     | V_CVT_I32_F64       | 45    | V_FFBH_U32          |
| 4     | V_CVT_F64_I32       | 46    | V_FFBL_B32          |
| 5     | V_CVT_F32_I32       | 47    | V_FFBH_I32          |
| 6     | V_CVT_F32_U32       | 48    | V_FREXP_EXP_I32_F64 |
| 7     | V_CVT_U32_F32       | 49    | V_FREXP_MANT_F64    |

| 操作码编号 | 名称                | 操作码编号 | 名称                  |
|-------|-------------------|-------|---------------------|
| 8     | V_CVT_I32_F32     | 50    | V_FRACT_F64         |
| 10    | V_CVT_F16_F32     | 51    | V_FREXP_EXP_I32_F32 |
| 11    | V_CVT_F32_F16     | 52    | V_FREXP_MANT_F32    |
| 12    | V_CVT_RPI_I32_F32 | 53    | V_CLREXCP           |
| 13    | V_CVT_FLR_I32_F32 | 56    | V_MOV_B64           |
| 14    | V_CVT_OFF_F32_I4  | 57    | V_CVT_F16_U16       |
| 15    | V_CVT_F32_F64     | 58    | V_CVT_F16_I16       |
| 16    | V_CVT_F64_F32     | 59    | V_CVT_U16_F16       |
| 17    | V_CVT_F32_UBYTE0  | 60    | V_CVT_I16_F16       |
| 18    | V_CVT_F32_UBYTE1  | 61    | V_RCP_F16           |
| 19    | V_CVT_F32_UBYTE2  | 62    | V_SQRT_F16          |
| 20    | V_CVT_F32_UBYTE3  | 63    | V_RSQ_F16           |
| 21    | V_CVT_U32_F64     | 64    | V_LOG_F16           |
| 22    | V_CVT_F64_U32     | 65    | V_EXP_F16           |
| 23    | V_TRUNC_F64       | 66    | V_FREXP_MANT_F16    |
| 24    | V_CEIL_F64        | 67    | V_FREXP_EXP_I16_F16 |
| 25    | V_RNDNE_F64       | 68    | V_FLOOR_F16         |
| 26    | V_FLOOR_F64       | 69    | V_CEIL_F16          |
| 27    | V_FRACT_F32       | 70    | V_TRUNC_F16         |
| 28    | V_TRUNC_F32       | 71    | V_RNDNE_F16         |
| 29    | V_CEIL_F32        | 72    | V_FRACT_F16         |
| 30    | V_RNDNE_F32       | 73    | V_SIN_F16           |
| 31    | V_FLOOR_F32       | 74    | V_COS_F16           |
| 32    | V_EXP_F32         | 77    | V_CVT_NORM_I16_F16  |
| 33    | V_LOG_F32         | 78    | V_CVT_NORM_U16_F16  |
| 34    | V_RCP_F32         | 79    | V_SAT_PK_U8_I16     |
| 35    | V_RCP_IFLAG_F32   | 81    | V_SWAP_B32          |
| 36    | V_RSQ_F32         | 82    | V_ACCVGPR_MOV_B32   |
| 37    | V_RCP_F64         | 84    | V_CVT_F32_FP8       |
| 38    | V_RSQ_F64         | 85    | V_CVT_F32_BF8       |
| 39    | V_SQRT_F32        | 86    | V_CVT_PK_F32_FP8    |
| 40    | V_SQRT_F64        | 87    | V_CVT_PK_F32_BF8    |
| 41    | V_SIN_F32         |       |                     |

13.3.3. VOPC



格式 VOPC

说明 向量指令需要两个输入并产生比较结果。 后面可有一个 32 位字元常数。 向量比较操作分为三组：

- 那些可以使用 16 个比较操作中任何一个操作的,
- 那些可以使用 8 中任何一种的,以及
- 那些有单一的比较操作。

最后的 opcode 编号是通过将 opcode 家族的基数加上比较 op 的偏移来决定的。每个比较指令都会写出 VCC(为 VOPC)或 SGPR(为 VOP3)的结果。此外,比较指令还有变体,也写到 EXEC 的面具。比较结果的目的地是使用 VOPC 格式编码时的 VCC,在 VOP3 格式编码时可以是任意的 SGPR。

比较业务

表 80. 比较业务

| 比较操作          | Opcod<br>e 偏移 | 说明                               |
|---------------|---------------|----------------------------------|
| 十六项比较操作(OP16) |               |                                  |
| F             | 0             | D.u = 0                          |
| LT            | 1             | D.u = (S0 < S1)                  |
| EQ            | 2             | D.u = (S0 == S1)                 |
| LE            | 3             | D.u = (S0 <= S1)                 |
| GT            | 4             | D.u = (S0 > S1)                  |
| LG            | 5             | D.u = (S0 <> S1)                 |
| GE            | 6             | D.u = (S0 >= S1)                 |
| O             | 7             | D.u = (!isNaN(S0) && !isNaN(S1)) |
| U             | 8             | D.u = (!isNaN(S0)    !isNaN(S1)) |
| NGE           | 9             | D.u = !(S0 >= S1)                |
| NLG           | 10            | D.u = !(S0 <> S1)                |
| NGT           | 11            | D.u = !(S0 > S1)                 |
| NLE           | 12            | D.u = !(S0 <= S1)                |
| NEQ           | 13            | D.u = !(S0 == S1)                |
| NLT           | 14            | D.u = !(S0 < S1)                 |
| TRU           | 15            | D.u = 1                          |
| 八个比较操作(OP8)   |               |                                  |
| F             | 0             | D.u = 0                          |
| LT            | 1             | D.u = (S0 < S1)                  |
| EQ            | 2             | D.u = (S0 == S1)                 |
| LE            | 3             | D.u = (S0 <= S1)                 |
| GT            | 4             | D.u = (S0 > S1)                  |
| LG            | 5             | D.u = (S0 <> S1)                 |
| GE            | 6             | D.u = (S0 >= S1)                 |
| TRU           | 7             | D.u = 1                          |

表 81. VOPC 字段



| 字段名称     | 位         | 格式或说明                       |
|----------|-----------|-----------------------------|
| SRC0     | [8:0]     | 源操作数 0，即本指令的第一个操作数。         |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。    |
|          | 102       | FLAT_SCRATCH_LO.            |
|          | 103       | FLAT_SCRATCH_HI.            |
|          | 104       | XNACK_MASK_LO.              |
|          | 105       | XNACK_MASK_HI.              |
|          | 106       | VCC_LO: vcc[31:0]。          |
|          | 107       | VCC_HI: vcc[63:32]。         |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。 |
|          | 124       | M0：内存寄存器 0。                 |
|          | 125       | 保留                          |
|          | 126       | EXEC_LO: exec[31:0]。        |
|          | 127       | EXEC_HI: exec[63:32]。       |
|          | 128       | 0.                          |
|          | 129-192   | 有符号整数 1 至 64。               |
|          | 193-208   | 有符号整数 -1 至 -16。             |
|          | 209-234   | 保留。                         |
|          | 235       | SHARED_BASE（内存窗口定义）。        |
|          | 236       | SHARED_LIMIT（内存窗口定义）。       |
|          | 237       | PRIVATE_BASE（内存窗口定义）。       |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。      |
|          | 239       | RESERVED .                  |
|          | 240       | 0.5.                        |
|          | 241       | -0.5.                       |
|          | 242       | 1.0.                        |
|          | 243       | -1.0.                       |
|          | 244       | 2.0.                        |
|          | 245       | -2.0.                       |
|          | 246       | 4.0.                        |
|          | 247       | -4.0.                       |
|          | 248       | 1/(2*PI).                   |
|          | 249       | SDWA                        |
|          | 250       | DPP                         |
|          | 251       | VCCZ.                       |
|          | 252       | EXECZ.                      |
|          | 253       | SCC.                        |
|          | 254       | 保留。                         |
|          | 255       | 字面量常量。                      |
|          | 256 - 511 | VGPR 0 - 255                |
| VSRC1    | [16:9]    | VGPR 提供第二个操作。               |
| OP       | [24:17]   | 参见以下 Opcode 表。              |
| ENCODING | [31:25]   | 必须是: 0_111110               |

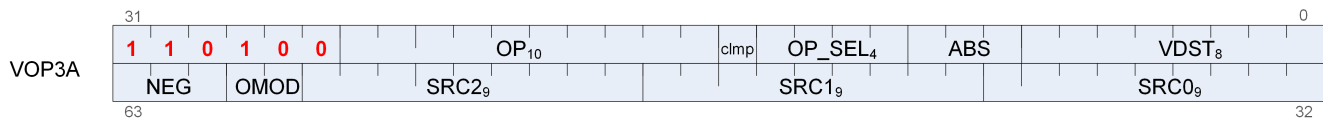
表 82. VOPC Opcode

| 操作码编号 | 名称               | 操作码编号 | 名称             |
|-------|------------------|-------|----------------|
| 16    | V_CMP_CLASS_F32  | 125   | V_CMPX_NEQ_F64 |
| 17    | V_CMPX_CLASS_F32 | 126   | V_CMPX_NLT_F64 |
| 18    | V_CMP_CLASS_F64  | 127   | V_CMPX_TRU_F64 |
| 19    | V_CMPX_CLASS_F64 | 160   | V_CMP_F_I16    |
| 20    | V_CMP_CLASS_F16  | 161   | V_CMP_LT_I16   |
| 21    | V_CMPX_CLASS_F16 | 162   | V_CMP_EQ_I16   |
| 32    | V_CMP_F_F16      | 163   | V_CMP_LE_I16   |
| 33    | V_CMP_LT_F16     | 164   | V_CMP_GT_I16   |

| 操作码编号 | 名称             | 操作码编号 | 名称            |
|-------|----------------|-------|---------------|
| 34    | V_CMP_EQ_F16   | 165   | V_CMP_NE_I16  |
| 35    | V_CMP_LE_F16   | 166   | V_CMP_GE_I16  |
| 36    | V_CMP_GT_F16   | 167   | V_CMP_T_I16   |
| 37    | V_CMP_LG_F16   | 168   | V_CMP_F_U16   |
| 38    | V_CMP_GE_F16   | 169   | V_CMP_LT_U16  |
| 39    | V_CMP_O_F16    | 170   | V_CMP_EQ_U16  |
| 40    | V_CMP_U_F16    | 171   | V_CMP_LE_U16  |
| 41    | V_CMP_NGE_F16  | 172   | V_CMP_GT_U16  |
| 42    | V_CMP_NLG_F16  | 173   | V_CMP_NE_U16  |
| 43    | V_CMP_NGT_F16  | 174   | V_CMP_GE_U16  |
| 44    | V_CMP_NLE_F16  | 175   | V_CMP_T_U16   |
| 45    | V_CMP_NEQ_F16  | 176   | V_CMPX_F_I16  |
| 46    | V_CMP_NLT_F16  | 177   | V_CMPX_LT_I16 |
| 47    | V_CMP_TRU_F16  | 178   | V_CMPX_EQ_I16 |
| 48    | V_CMPX_F_F16   | 179   | V_CMPX_LE_I16 |
| 49    | V_CMPX_LT_F16  | 180   | V_CMPX_GT_I16 |
| 50    | V_CMPX_EQ_F16  | 181   | V_CMPX_NE_I16 |
| 51    | V_CMPX_LE_F16  | 182   | V_CMPX_GE_I16 |
| 52    | V_CMPX_GT_F16  | 183   | V_CMPX_T_I16  |
| 53    | V_CMPX_LG_F16  | 184   | V_CMPX_F_U16  |
| 54    | V_CMPX_GE_F16  | 185   | V_CMPX_LT_U16 |
| 55    | V_CMPX_O_F16   | 186   | V_CMPX_EQ_U16 |
| 56    | V_CMPX_U_F16   | 187   | V_CMPX_LE_U16 |
| 57    | V_CMPX_NGE_F16 | 188   | V_CMPX_GT_U16 |
| 58    | V_CMPX_NLG_F16 | 189   | V_CMPX_NE_U16 |
| 59    | V_CMPX_NGT_F16 | 190   | V_CMPX_GE_U16 |
| 60    | V_CMPX_NLE_F16 | 191   | V_CMPX_T_U16  |
| 61    | V_CMPX_NEQ_F16 | 192   | V_CMP_F_I32   |
| 62    | V_CMPX_NLT_F16 | 193   | V_CMP_LT_I32  |
| 63    | V_CMPX_TRU_F16 | 194   | V_CMP_EQ_I32  |
| 64    | V_CMP_F_F32    | 195   | V_CMP_LE_I32  |
| 65    | V_CMP_LT_F32   | 196   | V_CMP_GT_I32  |
| 66    | V_CMP_EQ_F32   | 197   | V_CMP_NE_I32  |
| 67    | V_CMP_LE_F32   | 198   | V_CMP_GE_I32  |
| 68    | V_CMP_GT_F32   | 199   | V_CMP_T_I32   |
| 69    | V_CMP_LG_F32   | 200   | V_CMP_F_U32   |
| 70    | V_CMP_GE_F32   | 201   | V_CMP_LT_U32  |
| 71    | V_CMP_O_F32    | 202   | V_CMP_EQ_U32  |
| 72    | V_CMP_U_F32    | 203   | V_CMP_LE_U32  |
| 73    | V_CMP_NGE_F32  | 204   | V_CMP_GT_U32  |
| 74    | V_CMP_NLG_F32  | 205   | V_CMP_NE_U32  |
| 75    | V_CMP_NGT_F32  | 206   | V_CMP_GE_U32  |
| 76    | V_CMP_NLE_F32  | 207   | V_CMP_T_U32   |
| 77    | V_CMP_NEQ_F32  | 208   | V_CMPX_F_I32  |
| 78    | V_CMP_NLT_F32  | 209   | V_CMPX_LT_I32 |
| 79    | V_CMP_TRU_F32  | 210   | V_CMPX_EQ_I32 |
| 80    | V_CMPX_F_F32   | 211   | V_CMPX_LE_I32 |

| 操作码编号 | 名称             | 操作码编号 | 名称            |
|-------|----------------|-------|---------------|
| 81    | V_CMPX_LT_F32  | 212   | V_CMPX_GT_I32 |
| 82    | V_CMPX_EQ_F32  | 213   | V_CMPX_NE_I32 |
| 83    | V_CMPX_LE_F32  | 214   | V_CMPX_GE_I32 |
| 84    | V_CMPX_GT_F32  | 215   | V_CMPX_T_I32  |
| 85    | V_CMPX_LG_F32  | 216   | V_CMPX_F_U32  |
| 86    | V_CMPX_GE_F32  | 217   | V_CMPX_LT_U32 |
| 87    | V_CMPX_O_F32   | 218   | V_CMPX_EQ_U32 |
| 88    | V_CMPX_U_F32   | 219   | V_CMPX_LE_U32 |
| 89    | V_CMPX_NGE_F32 | 220   | V_CMPX_GT_U32 |
| 90    | V_CMPX_NLG_F32 | 221   | V_CMPX_NE_U32 |
| 91    | V_CMPX_NGT_F32 | 222   | V_CMPX_GE_U32 |
| 92    | V_CMPX_NLE_F32 | 223   | V_CMPX_T_U32  |
| 93    | V_CMPX_NEQ_F32 | 224   | V_CMP_F_I64   |
| 94    | V_CMPX_NLT_F32 | 225   | V_CMP_LT_I64  |
| 95    | V_CMPX_TRU_F32 | 226   | V_CMP_EQ_I64  |
| 96    | V_CMP_F_F64    | 227   | V_CMP_LE_I64  |
| 97    | V_CMP_LT_F64   | 228   | V_CMP_GT_I64  |
| 98    | V_CMP_EQ_F64   | 229   | V_CMP_NE_I64  |
| 99    | V_CMP_LE_F64   | 230   | V_CMP_GE_I64  |
| 100   | V_CMP_GT_F64   | 231   | V_CMP_T_I64   |
| 101   | V_CMP_LG_F64   | 232   | V_CMP_F_U64   |
| 102   | V_CMP_GE_F64   | 233   | V_CMP_LT_U64  |
| 103   | V_CMP_O_F64    | 234   | V_CMP_EQ_U64  |
| 104   | V_CMP_U_F64    | 235   | V_CMP_LE_U64  |
| 105   | V_CMP_NGE_F64  | 236   | V_CMP_GT_U64  |
| 106   | V_CMP_NLG_F64  | 237   | V_CMP_NE_U64  |
| 107   | V_CMP_NGT_F64  | 238   | V_CMP_GE_U64  |
| 108   | V_CMP_NLE_F64  | 239   | V_CMP_T_U64   |
| 109   | V_CMP_NEQ_F64  | 240   | V_CMPX_F_I64  |
| 110   | V_CMP_NLT_F64  | 241   | V_CMPX_LT_I64 |
| 111   | V_CMP_TRU_F64  | 242   | V_CMPX_EQ_I64 |
| 112   | V_CMPX_F_F64   | 243   | V_CMPX_LE_I64 |
| 113   | V_CMPX_LT_F64  | 244   | V_CMPX_GT_I64 |
| 114   | V_CMPX_EQ_F64  | 245   | V_CMPX_NE_I64 |
| 115   | V_CMPX_LE_F64  | 246   | V_CMPX_GE_I64 |
| 116   | V_CMPX_GT_F64  | 247   | V_CMPX_T_I64  |
| 117   | V_CMPX_LG_F64  | 248   | V_CMPX_F_U64  |
| 118   | V_CMPX_GE_F64  | 249   | V_CMPX_LT_U64 |
| 119   | V_CMPX_O_F64   | 250   | V_CMPX_EQ_U64 |
| 120   | V_CMPX_U_F64   | 251   | V_CMPX_LE_U64 |
| 121   | V_CMPX_NGE_F64 | 252   | V_CMPX_GT_U64 |
| 122   | V_CMPX_NLG_F64 | 253   | V_CMPX_NE_U64 |
| 123   | V_CMPX_NGT_F64 | 254   | V_CMPX_GE_U64 |
| 124   | V_CMPX_NLE_F64 | 255   | V_CMPX_T_U64  |

13.3.4. VOP3A



格式 VOP3A

说明 向量 ALU 格式,包含三个操作符

表 83. VOP3A 字段

| 字段名称     | 位       | 格式或说明                                                                                   |
|----------|---------|-----------------------------------------------------------------------------------------|
| VDST     | [7:0]   | 目标 VGPR                                                                                 |
| ABS      | [10:8]  | 输入绝对值. [8] = src0, [9] = src1, [10] = src2                                              |
| OPSEL    | [14:11] | 操作为 16 位数据选择. 0 = 选择低半音, 1 = 选择高半音. [11] = src0, [12] = src1, [13] = src2, [14] = dest. |
| CLMP     | [15]    | 粘合输出                                                                                    |
| OP       | [25:16] | Opcode,见下一张表格。                                                                          |
| ENCODING | [31:26] | 一定是: 110100 号                                                                           |

| 字段名称 | 位         | 格式或说明                                      |
|------|-----------|--------------------------------------------|
| SRC0 | [40:32]   | 源操作数 0，即本指令的第一个操作数。                        |
|      | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。                   |
|      | 102       | FLAT_SCRATCH_LO.                           |
|      | 103       | FLAT_SCRATCH_HI.                           |
|      | 104       | XNACK_MASK_LO.                             |
|      | 105       | XNACK_MASK_HI.                             |
|      | 106       | VCC_LO: vcc[31:0]。                         |
|      | 107       | VCC_HI: vcc[63:32]。                        |
|      | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。                |
|      | 124       | M0：内存寄存器 0。                                |
|      | 125       | 保留                                         |
|      | 126       | EXEC_LO: exec[31:0]。                       |
|      | 127       | EXEC_HI: exec[63:32]。                      |
|      | 128       | 0.                                         |
|      | 129-192   | 有符号整数 1 至 64。                              |
|      | 193-208   | 有符号整数 -1 至 -16。                            |
|      | 209-234   | 保留。                                        |
|      | 235       | SHARED_BASE（内存窗口定义）。                       |
|      | 236       | SHARED_LIMIT（内存窗口定义）。                      |
|      | 237       | PRIVATE_BASE（内存窗口定义）。                      |
|      | 238       | PRIVATE_LIMIT（内存窗口定义）。                     |
|      | 239       | 保留。                                        |
|      | 240       | 0.5.                                       |
|      | 241       | -0.5.                                      |
|      | 242       | 1.0.                                       |
|      | 243       | -1.0.                                      |
|      | 244       | 2.0.                                       |
|      | 245       | -2.0.                                      |
|      | 246       | 4.0.                                       |
|      | 247       | -4.0.                                      |
|      | 248       | 1/(2*PI).                                  |
|      | 249       | SDWA                                       |
|      | 250       | DPP                                        |
|      | 251       | VCCZ.                                      |
|      | 252       | EXECZ.                                     |
|      | 253       | SCC.                                       |
|      | 254       | 保留。                                        |
|      | 255       | 字面量常量。                                     |
|      | 256 - 511 | VGPR 0 - 255                               |
| SRC1 | [49:41]   | 第二输入操作。选项与 SRC0 相同。                        |
| SRC2 | [58:50]   | 第三个输入操作。选项与 SRC0 相同。                       |
| OMOD | [60:59]   | 输出修饰器:0=无,1=2,2=4,3=div-2                  |
| NEG  | [63:61]   | 忽略输入 [61] = src0, [62] = src1, [63] = src2 |

表 84. VOP3A Opcodes

| 操作码编号 | 名称                  | 操作码编号 | 名称                   |
|-------|---------------------|-------|----------------------|
| 384   | V_NOP               | 659   | V_BFM_B32            |
| 385   | V_MOV_B32           | 660   | V_CVT_PKNORM_I16_F32 |
| 386   | V_READFIRSTLANE_B32 | 661   | V_CVT_PKNORM_U16_F32 |
| 387   | V_CVT_I32_F64       | 662   | V_CVT_PKRTZ_F16_F32  |
| 388   | V_CVT_F64_I32       | 663   | V_CVT_PK_U16_U32     |
| 389   | V_CVT_F32_I32       | 664   | V_CVT_PK_I16_I32     |
| 390   | V_CVT_F32_U32       | 665   | V_CVT_PKNORM_I16_F16 |

| 操作码编号 | 名称                  | 操作码编号 | 名称                   |
|-------|---------------------|-------|----------------------|
| 391   | V_CVT_U32_F32       | 666   | V_CVT_PKNORM_U16_F16 |
| 392   | V_CVT_I32_F32       | 668   | V_ADD_I32            |
| 394   | V_CVT_F16_F32       | 669   | V_SUB_I32            |
| 395   | V_CVT_F32_F16       | 670   | V_ADD_I16            |
| 396   | V_CVT_RPI_I32_F32   | 671   | V_SUB_I16            |
| 397   | V_CVT_FLR_I32_F32   | 672   | V_PACK_B32_F16       |
| 398   | V_CVT_OFF_F32_I4    | 673   | V_MUL_LEGACY_F32     |
| 399   | V_CVT_F32_F64       | 674   | V_CVT_PK_FP8_F32     |
| 400   | V_CVT_F64_F32       | 675   | V_CVT_PK_BF8_F32     |
| 401   | V_CVT_F32_UBYTE0    | 676   | V_CVT_SR_FP8_F32     |
| 402   | V_CVT_F32_UBYTE1    | 677   | V_CVT_SR_BF8_F32     |
| 403   | V_CVT_F32_UBYTE2    | 16    | V_CMP_CLASS_F32      |
| 404   | V_CVT_F32_UBYTE3    | 17    | V_CMPX_CLASS_F32     |
| 405   | V_CVT_U32_F64       | 18    | V_CMP_CLASS_F64      |
| 406   | V_CVT_F64_U32       | 19    | V_CMPX_CLASS_F64     |
| 407   | V_TRUNC_F64         | 20    | V_CMP_CLASS_F16      |
| 408   | V_CEIL_F64          | 21    | V_CMPX_CLASS_F16     |
| 409   | V_RNDNE_F64         | 32    | V_CMP_F_F16          |
| 410   | V_FLOOR_F64         | 33    | V_CMP_LT_F16         |
| 411   | V_FRACT_F32         | 34    | V_CMP_EQ_F16         |
| 412   | V_TRUNC_F32         | 35    | V_CMP_LE_F16         |
| 413   | V_CEIL_F32          | 36    | V_CMP_GT_F16         |
| 414   | V_RNDNE_F32         | 37    | V_CMP_LG_F16         |
| 415   | V_FLOOR_F32         | 38    | V_CMP_GE_F16         |
| 416   | V_EXP_F32           | 39    | V_CMP_O_F16          |
| 417   | V_LOG_F32           | 40    | V_CMP_U_F16          |
| 418   | V_RCP_F32           | 41    | V_CMP_NGE_F16        |
| 419   | V_RCP_IFLAG_F32     | 42    | V_CMP_NLG_F16        |
| 420   | V_RSQ_F32           | 43    | V_CMP_NGT_F16        |
| 421   | V_RCP_F64           | 44    | V_CMP_NLE_F16        |
| 422   | V_RSQ_F64           | 45    | V_CMP_NEQ_F16        |
| 423   | V_SQRT_F32          | 46    | V_CMP_NLT_F16        |
| 424   | V_SQRT_F64          | 47    | V_CMP_TRU_F16        |
| 425   | V_SIN_F32           | 48    | V_CMPX_F_F16         |
| 426   | V_COS_F32           | 49    | V_CMPX_LT_F16        |
| 427   | V_NOT_B32           | 50    | V_CMPX_EQ_F16        |
| 428   | V_BFREV_B32         | 51    | V_CMPX_LE_F16        |
| 429   | V_FFBH_U32          | 52    | V_CMPX_GT_F16        |
| 430   | V_FFBH_B32          | 53    | V_CMPX_LG_F16        |
| 431   | V_FFBH_I32          | 54    | V_CMPX_GE_F16        |
| 432   | V_FREXP_EXP_I32_F64 | 55    | V_CMPX_O_F16         |
| 433   | V_FREXP_MANT_F64    | 56    | V_CMPX_U_F16         |
| 434   | V_FRACT_F64         | 57    | V_CMPX_NGE_F16       |
| 435   | V_FREXP_EXP_I32_F32 | 58    | V_CMPX_NLG_F16       |
| 436   | V_FREXP_MANT_F32    | 59    | V_CMPX_NGT_F16       |
| 437   | V_CLREXCP           | 60    | V_CMPX_NLE_F16       |
| 440   | V_MOV_B64           | 61    | V_CMPX_NEQ_F16       |

| 操作码编号 | 名称                  | 操作码编号 | 名称             |
|-------|---------------------|-------|----------------|
| 441   | V_CVT_F16_U16       | 62    | V_CMPX_NLT_F16 |
| 442   | V_CVT_F16_I16       | 63    | V_CMPX_TRU_F16 |
| 443   | V_CVT_U16_F16       | 64    | V_CMP_F_F32    |
| 444   | V_CVT_I16_F16       | 65    | V_CMP_LT_F32   |
| 445   | V_RCP_F16           | 66    | V_CMP_EQ_F32   |
| 446   | V_SQRT_F16          | 67    | V_CMP_LE_F32   |
| 447   | V_RSQ_F16           | 68    | V_CMP_GT_F32   |
| 448   | V_LOG_F16           | 69    | V_CMP_LG_F32   |
| 449   | V_EXP_F16           | 70    | V_CMP_GE_F32   |
| 450   | V_FREXP_MANT_F16    | 71    | V_CMP_O_F32    |
| 451   | V_FREXP_EXP_I16_F16 | 72    | V_CMP_U_F32    |
| 452   | V_FLOOR_F16         | 73    | V_CMP_NGE_F32  |
| 453   | V_CEIL_F16          | 74    | V_CMP_NLG_F32  |
| 454   | V_TRUNC_F16         | 75    | V_CMP_NGT_F32  |
| 455   | V_RNDNE_F16         | 76    | V_CMP_NLE_F32  |
| 456   | V_FRACT_F16         | 77    | V_CMP_NEQ_F32  |
| 457   | V_SIN_F16           | 78    | V_CMP_NLT_F32  |
| 458   | V_COS_F16           | 79    | V_CMP_TRU_F32  |
| 461   | V_CVT_NORM_I16_F16  | 80    | V_CMPX_F_F32   |
| 462   | V_CVT_NORM_U16_F16  | 81    | V_CMPX_LT_F32  |
| 463   | V_SAT_PK_U8_I16     | 82    | V_CMPX_EQ_F32  |
| 465   | V_SWAP_B32          | 83    | V_CMPX_LE_F32  |
| 466   | V_ACCVGPR_MOV_B32   | 84    | V_CMPX_GT_F32  |
| 468   | V_CVT_F32_FP8       | 85    | V_CMPX_LG_F32  |
| 469   | V_CVT_F32_BF8       | 86    | V_CMPX_GE_F32  |
| 470   | V_CVT_PK_F32_FP8    | 87    | V_CMPX_O_F32   |
| 471   | V_CVT_PK_F32_BF8    | 88    | V_CMPX_U_F32   |
| 256   | V_CNDMASK_B32       | 89    | V_CMPX_NGE_F32 |
| 257   | V_ADD_F32           | 90    | V_CMPX_NLG_F32 |
| 258   | V_SUB_F32           | 91    | V_CMPX_NGT_F32 |
| 259   | V_SUBREV_F32        | 92    | V_CMPX_NLE_F32 |
| 260   | V_FMAC_F64          | 93    | V_CMPX_NEQ_F32 |
| 261   | V_MUL_F32           | 94    | V_CMPX_NLT_F32 |
| 262   | V_MUL_I32_I24       | 95    | V_CMPX_TRU_F32 |
| 263   | V_MUL_HI_I32_I24    | 96    | V_CMP_F_F64    |
| 264   | V_MUL_U32_U24       | 97    | V_CMP_LT_F64   |
| 265   | V_MUL_HI_U32_U24    | 98    | V_CMP_EQ_F64   |
| 266   | V_MIN_F32           | 99    | V_CMP_LE_F64   |
| 267   | V_MAX_F32           | 100   | V_CMP_GT_F64   |
| 268   | V_MIN_I32           | 101   | V_CMP_LG_F64   |
| 269   | V_MAX_I32           | 102   | V_CMP_GE_F64   |
| 270   | V_MIN_U32           | 103   | V_CMP_O_F64    |
| 271   | V_MAX_U32           | 104   | V_CMP_U_F64    |
| 272   | V_LSHRREV_B32       | 105   | V_CMP_NGE_F64  |
| 273   | V_ASHRREV_I32       | 106   | V_CMP_NLG_F64  |
| 274   | V_LSHLREV_B32       | 107   | V_CMP_NGT_F64  |
| 275   | V_AND_B32           | 108   | V_CMP_NLE_F64  |

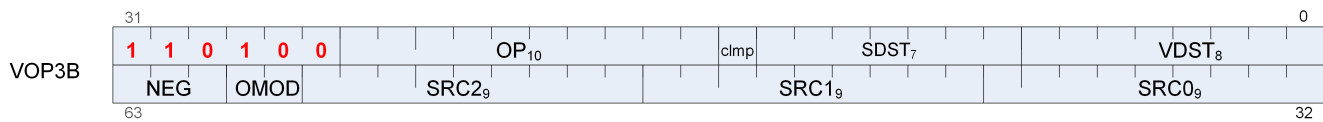
| 操作码编号 | 名称              | 操作码编号 | 名称             |
|-------|-----------------|-------|----------------|
| 276   | V_OR_B32        | 109   | V_CMP_NEQ_F64  |
| 277   | V_XOR_B32       | 110   | V_CMP_NLT_F64  |
| 287   | V_ADD_F16       | 111   | V_CMP_TRU_F64  |
| 288   | V_SUB_F16       | 112   | V_CMPX_F_F64   |
| 289   | V_SUBREV_F16    | 113   | V_CMPX_LT_F64  |
| 290   | V_MUL_F16       | 114   | V_CMPX_EQ_F64  |
| 291   | V_MAC_F16       | 115   | V_CMPX_LE_F64  |
| 294   | V_ADD_U16       | 116   | V_CMPX_GT_F64  |
| 295   | V_SUB_U16       | 117   | V_CMPX_LG_F64  |
| 296   | V_SUBREV_U16    | 118   | V_CMPX_GE_F64  |
| 297   | V_MUL_LO_U16    | 119   | V_CMPX_O_F64   |
| 298   | V_LSHLREV_B16   | 120   | V_CMPX_U_F64   |
| 299   | V_LSHRREV_B16   | 121   | V_CMPX_NGE_F64 |
| 300   | V_ASHRREV_I16   | 122   | V_CMPX_NLG_F64 |
| 301   | V_MAX_F16       | 123   | V_CMPX_NGT_F64 |
| 302   | V_MIN_F16       | 124   | V_CMPX_NLE_F64 |
| 303   | V_MAX_U16       | 125   | V_CMPX_NEQ_F64 |
| 304   | V_MAX_I16       | 126   | V_CMPX_NLT_F64 |
| 305   | V_MIN_U16       | 127   | V_CMPX_TRU_F64 |
| 306   | V_MIN_I16       | 160   | V_CMP_F_I16    |
| 307   | V_LDEXP_F16     | 161   | V_CMP_LT_I16   |
| 308   | V_ADD_U32       | 162   | V_CMP_EQ_I16   |
| 309   | V_SUB_U32       | 163   | V_CMP_LE_I16   |
| 310   | V_SUBREV_U32    | 164   | V_CMP_GT_I16   |
| 311   | V_DOT2C_F32_F16 | 165   | V_CMP_NE_I16   |
| 312   | V_DOT2C_I32_I16 | 166   | V_CMP_GE_I16   |
| 313   | V_DOT4C_I32_I8  | 167   | V_CMP_T_I16    |
| 314   | V_DOT8C_I32_I4  | 168   | V_CMP_F_U16    |
| 315   | V_FMAC_F32      | 169   | V_CMP_LT_U16   |
| 316   | V_PK_FMAC_F16   | 170   | V_CMP_EQ_U16   |
| 317   | V_XNOR_B32      | 171   | V_CMP_LE_U16   |
| 450   | V_MAD_I32_I24   | 172   | V_CMP_GT_U16   |
| 451   | V_MAD_U32_U24   | 173   | V_CMP_NE_U16   |
| 452   | V_CUBEID_F32    | 174   | V_CMP_GE_U16   |
| 453   | V_CUBESC_F32    | 175   | V_CMP_T_U16    |
| 454   | V_CUBETC_F32    | 176   | V_CMPX_F_I16   |
| 455   | V_CUBEMA_F32    | 177   | V_CMPX_LT_I16  |
| 456   | V_BFE_U32       | 178   | V_CMPX_EQ_I16  |
| 457   | V_BFE_I32       | 179   | V_CMPX_LE_I16  |
| 458   | V_BFI_B32       | 180   | V_CMPX_GT_I16  |
| 459   | V_FMA_F32       | 181   | V_CMPX_NE_I16  |
| 460   | V_FMA_F64       | 182   | V_CMPX_GE_I16  |
| 461   | V_LERP_U8       | 183   | V_CMPX_T_I16   |
| 462   | V_ALIGNBIT_B32  | 184   | V_CMPX_F_U16   |
| 463   | V_ALIGNBYTE_B32 | 185   | V_CMPX_LT_U16  |
| 464   | V_MIN3_F32      | 186   | V_CMPX_EQ_U16  |
| 465   | V_MIN3_I32      | 187   | V_CMPX_LE_U16  |



| 操作码编号 | 名称                     | 操作码编号 | 名称            |
|-------|------------------------|-------|---------------|
| 466   | V_MIN3_U32             | 188   | V_CMPX_GT_U16 |
| 467   | V_MAX3_F32             | 189   | V_CMPX_NE_U16 |
| 468   | V_MAX3_I32             | 190   | V_CMPX_GE_U16 |
| 469   | V_MAX3_U32             | 191   | V_CMPX_T_U16  |
| 470   | V_MED3_F32             | 192   | V_CMP_F_I32   |
| 471   | V_MED3_I32             | 193   | V_CMP_LT_I32  |
| 472   | V_MED3_U32             | 194   | V_CMP_EQ_I32  |
| 473   | V_SAD_U8               | 195   | V_CMP_LE_I32  |
| 474   | V_SAD_HI_U8            | 196   | V_CMP_GT_I32  |
| 475   | V_SAD_U16              | 197   | V_CMP_NE_I32  |
| 476   | V_SAD_U32              | 198   | V_CMP_GE_I32  |
| 477   | V_CVT_PK_U8_F32        | 199   | V_CMP_T_I32   |
| 478   | V_DIV_FIXUP_F32        | 200   | V_CMP_F_U32   |
| 479   | V_DIV_FIXUP_F64        | 201   | V_CMP_LT_U32  |
| 482   | V_DIV_FMAS_F32         | 202   | V_CMP_EQ_U32  |
| 483   | V_DIV_FMAS_F64         | 203   | V_CMP_LE_U32  |
| 484   | V_MSAD_U8              | 204   | V_CMP_GT_U32  |
| 485   | V_QSAD_PK_U16_U8       | 205   | V_CMP_NE_U32  |
| 486   | V_MQSAD_PK_U16_U8      | 206   | V_CMP_GE_U32  |
| 487   | V_MQSAD_U32_U8         | 207   | V_CMP_T_U32   |
| 490   | V_MAD_LEGACY_F16       | 208   | V_CMPX_F_I32  |
| 491   | V_MAD_LEGACY_U16       | 209   | V_CMPX_LT_I32 |
| 492   | V_MAD_LEGACY_I16       | 210   | V_CMPX_EQ_I32 |
| 493   | V_PERM_B32             | 211   | V_CMPX_LE_I32 |
| 494   | V_FMA_LEGACY_F16       | 212   | V_CMPX_GT_I32 |
| 495   | V_DIV_FIXUP_LEGACY_F16 | 213   | V_CMPX_NE_I32 |
| 496   | V_CVT_PKACCUM_U8_F32   | 214   | V_CMPX_GE_I32 |
| 497   | V_MAD_U32_U16          | 215   | V_CMPX_T_I32  |
| 498   | V_MAD_I32_I16          | 216   | V_CMPX_F_U32  |
| 499   | V_XAD_U32              | 217   | V_CMPX_LT_U32 |
| 500   | V_MIN3_F16             | 218   | V_CMPX_EQ_U32 |
| 501   | V_MIN3_I16             | 219   | V_CMPX_LE_U32 |
| 502   | V_MIN3_U16             | 220   | V_CMPX_GT_U32 |
| 503   | V_MAX3_F16             | 221   | V_CMPX_NE_U32 |
| 504   | V_MAX3_I16             | 222   | V_CMPX_GE_U32 |
| 505   | V_MAX3_U16             | 223   | V_CMPX_T_U32  |
| 506   | V_MED3_F16             | 224   | V_CMP_F_I64   |
| 507   | V_MED3_I16             | 225   | V_CMP_LT_I64  |
| 508   | V_MED3_U16             | 226   | V_CMP_EQ_I64  |
| 509   | V_LSHL_ADD_U32         | 227   | V_CMP_LE_I64  |
| 510   | V_ADD_LSHL_U32         | 228   | V_CMP_GT_I64  |
| 511   | V_ADD3_U32             | 229   | V_CMP_NE_I64  |
| 512   | V_LSHL_OR_B32          | 230   | V_CMP_GE_I64  |
| 513   | V_AND_OR_B32           | 231   | V_CMP_T_I64   |
| 514   | V_OR3_B32              | 232   | V_CMP_F_U64   |
| 515   | V_MAD_F16              | 233   | V_CMP_LT_U64  |
| 516   | V_MAD_U16              | 234   | V_CMP_EQ_U64  |

| 操作码编号 | 名称                 | 操作码编号 | 名称            |
|-------|--------------------|-------|---------------|
| 517   | V_MAD_I16          | 235   | V_CMP_LE_U64  |
| 518   | V_FMA_F16          | 236   | V_CMP_GT_U64  |
| 519   | V_DIV_FIXUP_F16    | 237   | V_CMP_NE_U64  |
| 520   | V_LSHL_ADD_U64     | 238   | V_CMP_GE_U64  |
| 640   | V_ADD_F64          | 239   | V_CMP_T_U64   |
| 641   | V_MUL_F64          | 240   | V_CMPX_F_I64  |
| 642   | V_MIN_F64          | 241   | V_CMPX_LT_I64 |
| 643   | V_MAX_F64          | 242   | V_CMPX_EQ_I64 |
| 644   | V_LDEXP_F64        | 243   | V_CMPX_LE_I64 |
| 645   | V_MUL_LO_U32       | 244   | V_CMPX_GT_I64 |
| 646   | V_MUL_HI_U32       | 245   | V_CMPX_NE_I64 |
| 647   | V_MUL_HI_I32       | 246   | V_CMPX_GE_I64 |
| 648   | V_LDEXP_F32        | 247   | V_CMPX_T_I64  |
| 649   | V_READLANE_B32     | 248   | V_CMPX_F_U64  |
| 650   | V_WRITELANE_B32    | 249   | V_CMPX_LT_U64 |
| 651   | V_BCNT_U32_B32     | 250   | V_CMPX_EQ_U64 |
| 652   | V_MBCNT_LO_U32_B32 | 251   | V_CMPX_LE_U64 |
| 653   | V_MBCNT_HI_U32_B32 | 252   | V_CMPX_GT_U64 |
| 655   | V_LSHLREV_B64      | 253   | V_CMPX_NE_U64 |
| 656   | V_LSHRREV_B64      | 254   | V_CMPX_GE_U64 |
| 657   | V_ASHRREV_I64      | 255   | V_CMPX_T_U64  |
| 658   | V_TRIG_PREOP_F64   |       |               |

13.3.5. VOP3B



格式 VOP3B

说明 向量 ALU 格式,有 3 个操作符和一个标量结果。这个编码只用于几个 opcode 。

此编码允许指定一个独特的标量目标操作数, 并且只用于下面列出的操作码。 所有其他操作码都使用 VOP3A 。

- V\_ADD\_CO\_U32
- V\_SUB\_CO\_U32
- V\_SUBREV\_CO\_U32
- V\_ADDC\_CO\_U32
- V\_SUBB\_CO\_U32
- V\_SUBBREV\_CO\_U32
- V\_DIV\_SCALE\_F32
- V\_DIV\_SCALE\_F64
- V\_MAD\_U64\_U32

## • V\_MAD\_I64\_I32

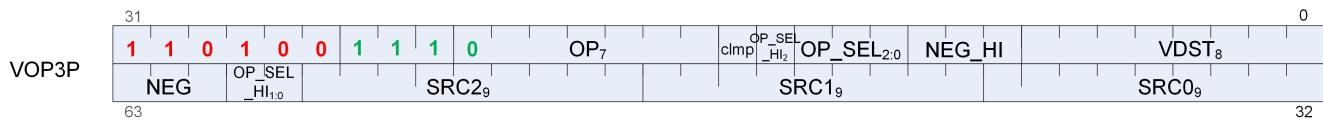
表 85. VOP3B 字段

| 字段名称     | 位                                                                                                                                                                                                                                                                                                           | 格式或说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VDST     | [7:0]                                                                                                                                                                                                                                                                                                       | 目标 VGPR                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| SDST     | [14:8]                                                                                                                                                                                                                                                                                                      | 斯卡拉尔目的地                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| CLMP     | [15]                                                                                                                                                                                                                                                                                                        | 剪切结果                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| OP       | [25:16]                                                                                                                                                                                                                                                                                                     | Opcode,见下表.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| ENCODING | [31:26]                                                                                                                                                                                                                                                                                                     | 一定是: 110100 号                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| SRC0     | [40:32]<br>0 - 101<br>102<br>103<br>104<br>105<br>106<br>107<br>108-123<br>124<br>125<br>126<br>127<br>128<br>129-192<br>193-208<br>209-234<br>235<br>236<br>237<br>238<br>239<br>240<br>241<br>242<br>243<br>244<br>245<br>246<br>247<br>248<br>249<br>250<br>251<br>252<br>253<br>254<br>255<br>256 - 511 | 源操作数 0, 即本指令的第一个操作数。<br>SGPR0 至 SGPR101: 标量通用寄存器。<br>FLAT_SCRATCH_LO.<br>FLAT_SCRATCH_HI.<br>XNACK_MASK_LO.<br>XNACK_MASK_HI.<br>VCC_LO: vcc[31:0].<br>VCC_HI: vcc[63:32].<br>TTMP0 至 TTMP15: 陷阱处理程序临时寄存器。<br>M0: 内存寄存器 0。<br>保留<br>EXEC_LO: exec[31:0].<br>EXEC_HI: exec[63:32].<br>0.<br>有符号整数 1 至 64。<br>有符号整数 -1 至 -16。<br>保留。<br>SHARED_BASE (内存窗口定义)。<br>SHARED_LIMIT (内存窗口定义)。<br>PRIVATE_BASE (内存窗口定义)。<br>PRIVATE_LIMIT (内存窗口定义)。<br>保留。<br>0.5.<br>-0.5.<br>1.0.<br>-1.0.<br>2.0.<br>-2.0.<br>4.0.<br>-4.0.<br>$1/(2*\text{PI})$ .<br>SDWA<br>DPP<br>VCCZ.<br>EXECZ.<br>SCC.<br>保留。<br>字面量常量。<br>VGPR 0 - 255 |
| SRC1     | [49:41]                                                                                                                                                                                                                                                                                                     | 第二输入操作。选项与 SRC0 相同。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| SRC2     | [58:50]                                                                                                                                                                                                                                                                                                     | 第三个输入操作。选项与 SRC0 相同。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| OMOD     | [60:59]                                                                                                                                                                                                                                                                                                     | 输出修饰器:0=无,1=2,2=4,3=div-2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| NEG      | [63:61]                                                                                                                                                                                                                                                                                                     | 忽略输入 [61] = src0, [62] = src1, [63] = src2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

表 86. VOP3B 操作码

| 操作码编号 | 名称              | 操作码编号 | 名称               |
|-------|-----------------|-------|------------------|
| 281   | V_ADD_CO_U32    | 286   | V_SUBBREV_CO_U32 |
| 282   | V_SUB_CO_U32    | 480   | V_DIV_SCALE_F32  |
| 283   | V_SUBREV_CO_U32 | 481   | V_DIV_SCALE_F64  |
| 284   | V_ADDDC_CO_U32  | 488   | V_MAD_U64_U32    |
| 285   | V_SUBB_CO_U32   | 489   | V_MAD_I64_I32    |

13.3.6. VOP3P



- 格式

VOP3P
- 说明

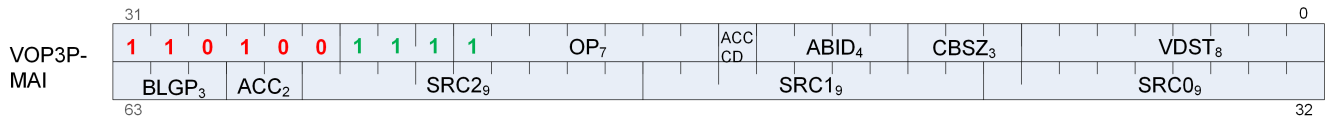
向量 ALU 格式,取一,二,三对 16 位输入,并产生两个 16 位输出(被包装成 1 个词).

表 87. VOP3P 字段

| 字段名称      | 位       | 格式或说明                            |
|-----------|---------|----------------------------------|
| VDST      | [7:0]   | 目标 VGPR                          |
| NEG_HI    | [10:8]  | 忽略了高 16 位中 0,1,2 位的来源.           |
| OPSEL     | [13:11] | 选择低源或高源为 0=[11],1=[12],2=[13].   |
| OPSEL_HI2 | [14]    | 为高源选择低值或高值 0=[14],1=[60],2=[59]. |
| CLMP      | [15]    | 1=夹子结果。                          |
| OP        | [22:16] | Opcode,见下表.                      |
| ENCODING  | [31:24] | 一定是 110100111 号                  |

| 字段名称     | 位         | 格式或说明                                              |
|----------|-----------|----------------------------------------------------|
| SRC0     | [40:32]   | 源操作数 0，即本指令的第一个操作数。                                |
|          | 0 - 101   | SGPR0 至 SGPR101：标量通用寄存器。                           |
|          | 102       | FLAT_SCRATCH_LO.                                   |
|          | 103       | FLAT_SCRATCH_HI.                                   |
|          | 104       | XNACK_MASK_LO.                                     |
|          | 105       | XNACK_MASK_HI.                                     |
|          | 106       | VCC_LO: vcc[31:0]。                                 |
|          | 107       | VCC_HI: vcc[63:32]。                                |
|          | 108-123   | TTMP0 至 TTMP15：陷阱处理程序临时寄存器。                        |
|          | 124       | M0：内存寄存器 0。                                        |
|          | 125       | 保留                                                 |
|          | 126       | EXEC_LO: exec[31:0]。                               |
|          | 127       | EXEC_HI: exec[63:32]。                              |
|          | 128       | 0.                                                 |
|          | 129-192   | 有符号整数 1 至 64。                                      |
|          | 193-208   | 有符号整数 -1 至 -16。                                    |
|          | 209-234   | 保留。                                                |
|          | 235       | SHARED_BASE（内存窗口定义）。                               |
|          | 236       | SHARED_LIMIT（内存窗口定义）。                              |
|          | 237       | PRIVATE_BASE（内存窗口定义）。                              |
|          | 238       | PRIVATE_LIMIT（内存窗口定义）。                             |
|          | 239       | 保留。                                                |
|          | 240       | 0.5.                                               |
|          | 241       | -0.5.                                              |
|          | 242       | 1.0.                                               |
|          | 243       | -1.0.                                              |
|          | 244       | 2.0.                                               |
|          | 245       | -2.0.                                              |
|          | 246       | 4.0.                                               |
|          | 247       | -4.0.                                              |
|          | 248       | 1/(2*PI).                                          |
|          | 249       | SDWA                                               |
|          | 250       | DPP                                                |
|          | 251       | VCCZ.                                              |
|          | 252       | EXECZ.                                             |
|          | 253       | SCC.                                               |
|          | 254       | 保留。                                                |
|          | 255       | 字面量常量。                                             |
|          | 256 - 511 | VGPR 0 - 255                                       |
| SRC1     | [49:41]   | 第二输入操作。选项与 SRC0 相同。                                |
| SRC2     | [58:50]   | 第三个输入操作。选项与 SRC0 相同。                               |
| OPSEL_HI | [60:59]   | 见 OP_SEL_HI2.                                      |
| NEG      | [63:61]   | 忽略低 16 位来源输入。[61] = src0, [62] = src1, [63] = src2 |

13.3.6.1. VOP3P-MAI





| 字段名称     | 位                                                                                                                                                   | 格式或说明                                                                                                                                                                                                                                                                        |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VDST     | [7:0]                                                                                                                                               | 目标 VGPR                                                                                                                                                                                                                                                                      |
| CBSZ     | [10:8]                                                                                                                                              | Control Broadcasting Size: 向 2CBSZ 输入其它矩阵乘法块播放 A 矩阵的一个选定块。任何 MFMA 指令中的法律值 = 0– 4, 但不得大于 log2( blocks )。要播放的块 ID 来自 ABID。 定义一个组内可以进行广播的块数。 法律值 = 0– 4。 这个组的块 ID 来自 ABID 。                                                                                                     |
| ABID     | [14:11]                                                                                                                                             | A-matrix Broadcasting Identifier:当 CBSZ 被设定为非零值时,在每组毗连的 2CBSZ 区域内,此选项选择 A 的哪个区块向其他区的矩阵乘法输入广播。                                                                                                                                                                                |
| ACC_CD   | [15]                                                                                                                                                | 指出 SRC-C 和 VDST 使用 ACC VGPRs. 对于 SMFMAC 的操作,ACC_CD 只影响 D 矩阵,SRC2 所持有的压缩指数来自 Arch VGPRs.                                                                                                                                                                                      |
| OP       | [22:16]                                                                                                                                             | Opcode,见下表.                                                                                                                                                                                                                                                                  |
| ENCODING | [31:24]                                                                                                                                             | 一定是 110100111 号                                                                                                                                                                                                                                                              |
| SRC0     | [40:32]<br>0 - 107<br>128<br>129-192<br>193-208<br>209-239<br>240<br>241<br>242<br>243<br>244<br>245<br>246<br>247<br>248<br>249 - 255<br>256 - 511 | 源操作数 0, 即本指令的第一个操作数。<br>保留。<br>0。<br>有符号整数 1 至 64。<br>有符号整数 -1 至 -16。<br>保留。<br>0.5. (float32)<br>-0.5.(float32)<br>1.0. (float32)<br>-1.0. (float32)<br>2.0. (float32)<br>-2.0. (float32)<br>4.0. (float32)<br>-4.0. (float32)<br>1/(2*PI). (float32)<br>保留<br>VGPR 0 - 255 |
| SRC1     | [49:41]                                                                                                                                             | 第二输入操作。 选项与 SRC0 相同 。                                                                                                                                                                                                                                                        |
| SRC2     | [58:50]                                                                                                                                             | 第三个输入操作。 选项与 SRC0 相同 。                                                                                                                                                                                                                                                       |
| ACC      | [60:59]                                                                                                                                             | ACC[0]:0 = 读 SRC-A 从 Arch VGPR; 1 = 读 SRC-A 从 Acc VGPR. ACC[1]:<br>0 = 读 SRC-B 从 Arch VGPR; 1 = 读 Acc VGPR.                                                                                                                                                                  |
| BLGP     | [63:61]                                                                                                                                             | “ B” – Matrix Lane- Group 模式 。 在通过控制 Swizzle mux 来进行矩阵乘法时, 控制在 VGPRs 的矩阵通道组( LG)。 对于 V_MFMA_F64_4X4X4F64 和 V_MFMA_F64_16X16X4F64 , 此字段指定了 NEG 修饰符, 而不是 BLGP 。                                                                                                                |

表 89. VOP3P Opcodes

| 操作码编号 | 名称               | 操作码编号 | 名称                        |
|-------|------------------|-------|---------------------------|
| 0     | V_PK_MAD_I16     | 73    | V_MFMA_F32_16X16X4_4B_F16 |
| 1     | V_PK_MUL_LO_U16  | 74    | V_MFMA_F32_4X4X4_16B_F16  |
| 2     | V_PK_ADD_I16     | 76    | V_MFMA_F32_32X32X8_F16    |
| 3     | V_PK_SUB_I16     | 77    | V_MFMA_F32_16X16X16_F16   |
| 4     | V_PK_LSHLREV_B16 | 80    | V_MFMA_I32_32X32X4_2B_I8  |
| 5     | V_PK_LSHRREV_B16 | 81    | V_MFMA_I32_16X16X4_4B_I8  |
| 6     | V_PK_ASHRREV_I16 | 82    | V_MFMA_I32_4X4X4_16B_I8   |
| 7     | V_PK_MAX_I16     | 86    | V_MFMA_I32_32X32X16_I8    |
| 8     | V_PK_MIN_I16     | 87    | V_MFMA_I32_16X16X32_I8    |

| 操作码编号 | 名称                        | 操作码编号 | 名称                            |
|-------|---------------------------|-------|-------------------------------|
| 9     | V_PK_MAD_U16              | 88    | V_ACCVGPR_READ                |
| 10    | V_PK_ADD_U16              | 89    | V_ACCVGPR_WRITE               |
| 11    | V_PK_SUB_U16              | 93    | V_MFMA_F32_32X32X4_2B_BF16    |
| 12    | V_PK_MAX_U16              | 94    | V_MFMA_F32_16X16X4_4B_BF16    |
| 13    | V_PK_MIN_U16              | 95    | V_MFMA_F32_4X4X4_16B_BF16     |
| 14    | V_PK_FMA_F16              | 96    | V_MFMA_F32_32X32X8_BF16       |
| 15    | V_PK_ADD_F16              | 97    | V_MFMA_F32_16X16X16_BF16      |
| 16    | V_PK_MUL_F16              | 98    | V_SMFMAC_F32_16X16X32_F16     |
| 17    | V_PK_MIN_F16              | 100   | V_SMFMAC_F32_32X32X16_F16     |
| 18    | V_PK_MAX_F16              | 102   | V_SMFMAC_F32_16X16X32_BF16    |
| 32    | V_MAD_MIX_F32             | 104   | V_SMFMAC_F32_32X32X16_BF16    |
| 33    | V_MAD_MIXLO_F16           | 106   | V_SMFMAC_I32_16X16X64_I8      |
| 34    | V_MAD_MIXHI_F16           | 108   | V_SMFMAC_I32_32X32X32_I8      |
| 35    | V_DOT2_F32_F16            | 110   | V_MFMA_F64_16X16X4_F64        |
| 38    | V_DOT2_I32_I16            | 111   | V_MFMA_F64_4X4X4_4B_F64       |
| 39    | V_DOT2_U32_U16            | 112   | V_MFMA_F32_16X16X32_BF8_BF8   |
| 40    | V_DOT4_I32_I8             | 113   | V_MFMA_F32_16X16X32_BF8_FP8   |
| 41    | V_DOT4_U32_U8             | 114   | V_MFMA_F32_16X16X32_FP8_BF8   |
| 42    | V_DOT8_I32_I4             | 115   | V_MFMA_F32_16X16X32_FP8_FP8   |
| 43    | V_DOT8_U32_U4             | 116   | V_MFMA_F32_32X32X16_BF8_BF8   |
| 48    | V_PK_FMA_F32              | 117   | V_MFMA_F32_32X32X16_BF8_FP8   |
| 49    | V_PK_MUL_F32              | 118   | V_MFMA_F32_32X32X16_FP8_BF8   |
| 50    | V_PK_ADD_F32              | 119   | V_MFMA_F32_32X32X16_FP8_FP8   |
| 51    | V_PK_MOV_B32              | 120   | V_SMFMAC_F32_16X16X64_BF8_BF8 |
| 62    | V_MFMA_F32_16X16X8_XF32   | 121   | V_SMFMAC_F32_16X16X64_BF8_FP8 |
| 63    | V_MFMA_F32_32X32X4_XF32   | 122   | V_SMFMAC_F32_16X16X64_FP8_BF8 |
| 64    | V_MFMA_F32_32X32X1_2B_F32 | 123   | V_SMFMAC_F32_16X16X64_FP8_FP8 |
| 65    | V_MFMA_F32_16X16X1_4B_F32 | 124   | V_SMFMAC_F32_32X32X32_BF8_BF8 |
| 66    | V_MFMA_F32_4X4X1_16B_F32  | 125   | V_SMFMAC_F32_32X32X32_BF8_FP8 |
| 68    | V_MFMA_F32_32X32X2_F32    | 126   | V_SMFMAC_F32_32X32X32_FP8_BF8 |
| 69    | V_MFMA_F32_16X16X4_F32    | 127   | V_SMFMAC_F32_32X32X32_FP8_FP8 |
| 72    | V_MFMA_F32_32X32X4_2B_F16 |       |                               |

### 13.3.7. SDWA

SDWA

格式

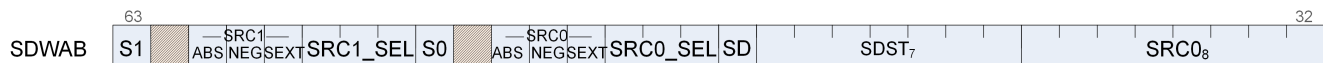
说明

子 Dword 地址。这是第二个 dword, 可以遵循 VOP1 或 VOP2 指令(代替字数常数)来控制 sub dword (16- bit) 操作符的选择。SDWA 的用法是将 SRC0 字段指定为 SDWA 字段, 然后用 SDWA 指令词确定实际的 VGPR 字段为 0。

表 90. SDWA 字段

| 字段名称      | 位       | 格式或说明                                                                                |
|-----------|---------|--------------------------------------------------------------------------------------|
| SRC0      | [39:32] | 实际 SRC0 操作(VGPR).                                                                    |
| DST_SEL   | [42:40] | 选择数据目的地:0-3=保留 4<br>=data[15:0] 5=data[31:16]<br>6=data[31:0] 7=保留.                  |
| DST_U     | [44:43] | 目标格式: 规定如何处理 VGPR 中未被 DST_SEL 选中的位。0 = 填零; 1 = 高位<br>做符号扩展、低位填零; 2 = 保留原值; 3 = 保留编码。 |
| CLMP      | [45]    | 1 = 夹结果                                                                              |
| OMOD      | [47:46] | 输出修饰器(见 VOP3). [46] = 低半径,[47] = 高半径                                                 |
| SRC0_SEL  | [50:48] | 来源 0 选择。 选项与 DST_SEL 相同。                                                             |
| SRC0_SEXT | [51]    | 签署源 0 的扩展修改符。                                                                        |
| SRC0_NEG  | [52]    | 1=否定源 0.                                                                             |
| SRC0_ABS  | [53]    | 1 = 源 0 的绝对值。                                                                        |
| S0        | [55]    | 0=来源 0 为 VGPR,1=为 SGPR.                                                              |
| SRC1_SEL  | [58:56] | 和 SRC0_SEL 一样的选项.                                                                    |
| SRC1_SEXT | [59]    | 签署源 1 的扩展修改符。                                                                        |
| SRC1_NEG  | [60]    | 1=否定源 1.                                                                             |
| SRC1_ABS  | [61]    | 1=源的绝对值 1.                                                                           |
| S1        | [63]    | 0=来源 1 为 VGPR,1=为 SGPR.                                                              |

13.3.8. SDWAB



- 格式SDWAB
- 说明子 Dword 地址。 这是第二个 dword, 可以遵循 VOPC 指令( 代替字数常数) 来控制子 Dword(16- bit) 操作符的选择。 SDWA 的使用方式是将 SRC0 字段指定为 SDWA , 然后将 VGPR 实际用作源零的 SDWA 指令词确定为 SDWA 。 此版本具有一个 scal 目的地。

表 91. SDWAB 领域

| 字段名称      | 位       | 格式或说明                      |
|-----------|---------|----------------------------|
| SRC0      | [39:32] | 实际 SRC0 操作(VGPR).          |
| SDST      | [46:40] | 标量 GPR 目的地.                |
| SD        | [47]    | 斯卡拉尔目的地类型:0=VCC,1=普通 SGPR. |
| SRC0_SEL  | [50:48] | 来源 0 选择。 选项与 DST_SEL 相同。   |
| SRC0_SEXT | [51]    | 签署源 0 的扩展修改符。              |
| SRC0_NEG  | [52]    | 1=否定源 0.                   |
| SRC0_ABS  | [53]    | 1 = 源 0 的绝对值。              |
| S0        | [55]    | 0=来源 0 为 VGPR,1=为 SGPR.    |



| 字段名称      | 位       | 格式或说明                   |
|-----------|---------|-------------------------|
| SRC1_SEL  | [58:56] | 和 SRC0_SEL 一样的选项.       |
| SRC1_SEXT | [59]    | 签署源 1 的扩展修改符 。          |
| SRC1_NEG  | [60]    | 1=否定源 1.                |
| SRC1_ABS  | [61]    | 1=源的绝对值 1.              |
| S1        | [63]    | 0=来源 1 为 VGPR,1=为 SGPR. |

13.3.9. DPP



|    |                                                                   |
|----|-------------------------------------------------------------------|
| 格式 | DPP                                                               |
| 说明 | 数据平行首字母。这是第二个词, 可以遵循 VOP1, VOP2 或 VOPC 指令( 代替字元常数) 来控制其他通道的数据选择 。 |

表 92. DPP 字段

| 字段名称      | 位       | 格式或说明                                                                                                                                                                                                                                                                       |
|-----------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SRC0      | [39:32] | 实际 SRC0 操作(VGPR).                                                                                                                                                                                                                                                           |
| DPP_CTRL  | [48:40] | 参见下表:"DPP_CTRL 编号".                                                                                                                                                                                                                                                         |
| BC        | [51]    | Bondes Control:0 = 来源超出范围时不写,1 = 写.                                                                                                                                                                                                                                         |
| SRC0_NEG  | [52]    | 1=否定源 0.                                                                                                                                                                                                                                                                    |
| SRC0_ABS  | [53]    | 1 = 源 0 的绝对值。                                                                                                                                                                                                                                                               |
| SRC1_NEG  | [54]    | 1=否定源 1.                                                                                                                                                                                                                                                                    |
| SRC1_ABS  | [55]    | 1=源的绝对值 1.                                                                                                                                                                                                                                                                  |
| BANK_MASK | [59:56] | Bank 掩码仅控制目标 VGPR 的写入, 不改变读取源 VGPR 数据时的线程掩码。第 27 位为 0: 禁用 lanes[12:15, 28:31, 44:47, 60:63]; 第 26 位为 0: 禁用 lanes[8:11, 24:27, 40:43, 56:59]; 第 25 位为 0: 禁用 lanes[4:7, 20:23, 36:39, 52:55]; 第 24 位为 0: 禁用 lanes[0:3, 16:19, 32:35, 48:51]。这里的 bank 与 VGPR 寄存器组中的 bank 含义不同。 |
| ROW_MASK  | [63:60] | Row 行掩码仅控制目标 VGPR 的写入, 不改变读取源 VGPR 数据时的线程掩码。第 31 位为 0: 禁用 lanes[63:48], 仅适用于 64 线程波前; 第 30 位为 0: 禁用 lanes[47:32], 仅适用于 64 线程波前; 第 29 位为 0: 禁用 lanes[31:16]; 第 28 位为 0: 禁用 lanes[15:0]。                                                                                      |

表 93. DPP\_CTRL 编号

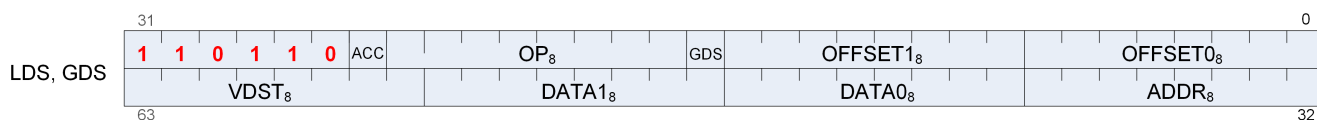
| DPP_Cntl 编号    | 十六进制值   | 函数                                                                                                                                | 说明         |
|----------------|---------|-----------------------------------------------------------------------------------------------------------------------------------|------------|
| DPP_QUAD_PERM* | 000-0FF | $\text{pix}[n].\text{srca} = \text{pix}[(n \& 0x3c) + \text{dpp\_cntl}[n\%4*2+1 : n\%4*2]]$ . srca.                               | 四根线的全长。    |
| DPP_UNUSED     | 100     | 未定义                                                                                                                               | 保留.        |
| DPP_ROW_SL*    | 101-10F | 如果 $((n \& 0xf) < (16 - \text{cntl}[3:0]))$ $\text{pix}[n].\text{srca} = \text{pix}[n + \text{cntl}[3:0]]$ .srca 其它使用 bound_cntl. | 左转 1 -15 线 |
| DPP_ROW_SR*    | 111-11F | if $((n \& 0xf) \geq \text{cntl}[3:0])$ $\text{pix}[n].\text{srca} = \text{pix}[n - \text{cntl}[3:0]]$ .srca else use bound_cntl  | 右转 1 -15 线 |

| DPP_Cntl 编号          | 十六进制值     | 函数                                                                                                             | 说明                  |
|----------------------|-----------|----------------------------------------------------------------------------------------------------------------|---------------------|
| DPP_ROW_RR*          | 121-12F   | if ((n&0xf) >= cnt[3:0]) pix[n].srca = pix[n - cntl[3:0]].srca else pix[n].srca = pix[n + 16 - cntl[3:0]].srca | 在行内向右旋转 1-15 个线程位置。 |
| DPP_WF_SL1*          | 130       | 如果(n<63) pix[n].srca = pix[n+1].srca 其他使用 bound_cntl                                                           | 波前左转 1 线。           |
| DPP_WF_RL1*          | 134       | 如果(n<63)pix[n].srca = pix[n+1].srca 其它 pix[n].srca = pix[0].srca                                               | 左侧波前旋转 1 线。         |
| DPP_WF_SR1*          | 138       | 如果(n>0) pix[n].srca = pix[n-1].srca 其它使用 bound_cntl                                                            | 波浪前右转 1 线。          |
| DPP_WF_RR1*          | 13C       | 如果(n>0) pix[n].srca = pix[n-1].srca 其它 pix[n].srca = pix[63].srca                                              | 将波前向右旋转 1 个线程位置。    |
| DPP_ROW_MIRROR*      | 140       | pix[n].srca = pix [15-(n&f)].srca.                                                                             | 镜线在行中。              |
| DPP_ROW_HALF_MIRROR* | 141       | pix[n].srca = pix [7-(n&7)]. srca.                                                                             | 行内镜线( 8 线)。         |
| DPP_ROW_BCAST 15*    | 142       | 如果(n>15) pix[n].srca = pix[n & 0x30 - 1].srca                                                                  | 每行的第 15 条线广播到下一行。   |
| DPP_ROW_BCAST 31*    | 143       | 若(n>31) pix[n].srca = pix[n & 0x20 - 1].srca                                                                   | 广播线 31 到第 2 行和第 3 行 |
| DPP_ROW*             | 150 - 15F | (一) pix[n].srca = pix[(n & 0xfffffff0)+count].srca;                                                            | 广播线 0 -15 在一行到整行。   |

注意,对于 64 位输入数据,唯一合法的 DPP 类型是"DPP ROW\*".

## 13.4. LDS 与 GWS 格式

### 13.4.1. DS



格式 LDS 与 GDS

说明 本地和全球数据共享指令

表 94. DS 领域

| 字段名称     | 位       | 格式或说明                            |
|----------|---------|----------------------------------|
| OFFSET0  | [7:0]   | 页:1                              |
| OFFSET1  | [15:8]  | 第二个地址偏移。 对于一些操作码, 它与 OFFSET0 连接。 |
| GDS      | [16]    | 1=GWS,0=LDS 业务.                  |
| OP       | [24:17] | 参见以下 Opcode 表.                   |
| ACC      | [25]    | VDST 是积分 VGPR                    |
| ENCODING | [31:26] | □)臻 110110                       |
| ADDR     | [39:32] | VGPR 提供地址.                       |
| DATA0    | [47:40] | 首个数据 VGPR.                       |
| DATA1    | [55:48] | 第二次数据 VGPR.                      |
| VDST     | [63:56] | 目标 VGPR 当结果返回 VGPRs.             |

表 95. DS 操作码

| 操作码编号 | 名称                     | 操作码编号 | 名称                     |
|-------|------------------------|-------|------------------------|
| 0     | DS_ADD_U32             | 69    | DS_MIN_I64             |
| 1     | DS_SUB_U32             | 70    | DS_MAX_I64             |
| 2     | DS_RSUB_U32            | 71    | DS_MIN_U64             |
| 3     | DS_INC_U32             | 72    | DS_MAX_U64             |
| 4     | DS_DEC_U32             | 73    | DS_AND_B64             |
| 5     | DS_MIN_I32             | 74    | DS_OR_B64              |
| 6     | DS_MAX_I32             | 75    | DS_XOR_B64             |
| 7     | DS_MIN_U32             | 76    | DS_MSKOR_B64           |
| 8     | DS_MAX_U32             | 77    | DS_WRITE_B64           |
| 9     | DS_AND_B32             | 78    | DS_WRITE2_B64          |
| 10    | DS_OR_B32              | 79    | DS_WRITE2ST64_B64      |
| 11    | DS_XOR_B32             | 80    | DS_CMPST_B64           |
| 12    | DS_MSKOR_B32           | 81    | DS_CMPST_F64           |
| 13    | DS_WRITE_B32           | 82    | DS_MIN_F64             |
| 14    | DS_WRITE2_B32          | 83    | DS_MAX_F64             |
| 15    | DS_WRITE2ST64_B32      | 84    | DS_WRITE_B8_D16_HI     |
| 16    | DS_CMPST_B32           | 85    | DS_WRITE_B16_D16_HI    |
| 17    | DS_CMPST_F32           | 86    | DS_READ_U8_D16         |
| 18    | DS_MIN_F32             | 87    | DS_READ_U8_D16_HI      |
| 19    | DS_MAX_F32             | 88    | DS_READ_I8_D16         |
| 20    | DS_NOP                 | 89    | DS_READ_I8_D16_HI      |
| 21    | DS_ADD_F32             | 90    | DS_READ_U16_D16        |
| 23    | DS_PK_ADD_F16          | 91    | DS_READ_U16_D16_HI     |
| 24    | DS_PK_ADD_BF16         | 92    | DS_ADD_F64             |
| 29    | DS_WRITE_ADDTID_B32    | 96    | DS_ADD_RTN_U64         |
| 30    | DS_WRITE_B8            | 97    | DS_SUB_RTN_U64         |
| 31    | DS_WRITE_B16           | 98    | DS_RSUB_RTN_U64        |
| 32    | DS_ADD_RTN_U32         | 99    | DS_INC_RTN_U64         |
| 33    | DS_SUB_RTN_U32         | 100   | DS_DEC_RTN_U64         |
| 34    | DS_RSUB_RTN_U32        | 101   | DS_MIN_RTN_I64         |
| 35    | DS_INC_RTN_U32         | 102   | DS_MAX_RTN_I64         |
| 36    | DS_DEC_RTN_U32         | 103   | DS_MIN_RTN_U64         |
| 37    | DS_MIN_RTN_I32         | 104   | DS_MAX_RTN_U64         |
| 38    | DS_MAX_RTN_I32         | 105   | DS_AND_RTN_B64         |
| 39    | DS_MIN_RTN_U32         | 106   | DS_OR_RTN_B64          |
| 40    | DS_MAX_RTN_U32         | 107   | DS_XOR_RTN_B64         |
| 41    | DS_AND_RTN_B32         | 108   | DS_MSKOR_RTN_B64       |
| 42    | DS_OR_RTN_B32          | 109   | DS_WRXCHG_RTN_B64      |
| 43    | DS_XOR_RTN_B32         | 110   | DS_WRXCHG2_RTN_B64     |
| 44    | DS_MSKOR_RTN_B32       | 111   | DS_WRXCHG2ST64_RTN_B64 |
| 45    | DS_WRXCHG_RTN_B32      | 112   | DS_CMPST_RTN_B64       |
| 46    | DS_WRXCHG2_RTN_B32     | 113   | DS_CMPST_RTN_F64       |
| 47    | DS_WRXCHG2ST64_RTN_B32 | 114   | DS_MIN_RTN_F64         |
| 48    | DS_CMPST_RTN_B32       | 115   | DS_MAX_RTN_F64         |
| 49    | DS_CMPST_RTN_F32       | 118   | DS_READ_B64            |
| 50    | DS_MIN_RTN_F32         | 119   | DS_READ2_B64           |

| 操作码编号 | 名称               | 操作码编号 | 名称                      |
|-------|------------------|-------|-------------------------|
| 51    | DS_MAX_RTN_F32   | 120   | DS_READ2ST64_B64        |
| 52    | DS_WRAP_RTN_B32  | 124   | DS_ADD_RTN_F64          |
| 53    | DS_ADD_RTN_F32   | 126   | DS_CONDXCHG32_RTN_B64   |
| 54    | DS_READ_B32      | 152   | DS_GWS_SEMA_RELEASE_ALL |
| 55    | DS_READ2_B32     | 153   | DS_GWS_INIT             |
| 56    | DS_READ2ST64_B32 | 154   | DS_GWS_SEMA_V           |
| 57    | DS_READ_I8       | 155   | DS_GWS_SEMA_BR          |
| 58    | DS_READ_U8       | 156   | DS_GWS_SEMA_P           |
| 59    | DS_READ_I16      | 157   | DS_GWS_BARRIER          |
| 60    | DS_READ_U16      | 182   | DS_READ_ADDTID_B32      |
| 61    | DS_SWIZZLE_B32   | 183   | DS_PK_ADD_RTN_F16       |
| 62    | DS_PERMUTE_B32   | 184   | DS_PK_ADD_RTN_BF16      |
| 63    | DS_BPERMUTE_B32  | 189   | DS_CONSUME              |
| 64    | DS_ADD_U64       | 190   | DS_APPEND               |
| 65    | DS_SUB_U64       | 222   | DS_WRITE_B96            |
| 66    | DS_RSUB_U64      | 223   | DS_WRITE_B128           |
| 67    | DS_INC_U64       | 254   | DS_READ_B96             |
| 68    | DS_DEC_U64       | 255   | DS_READ_B128            |

## 13.5. 向量内存缓冲区格式

有两种内存缓冲指示格式:

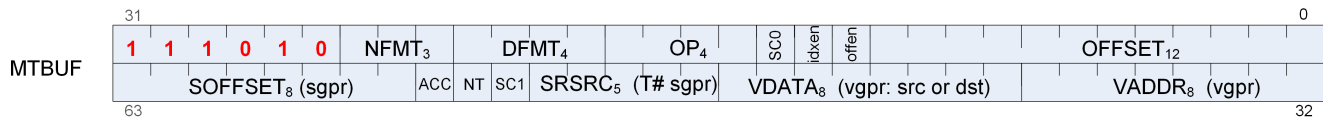
### MTBUF

键入缓冲访问( 指令定义了数据类型)

### MUBUF

未类型的缓冲访问(数据类型由缓冲/资源恒定定义)

### 13.5.1. MTBUF



- 格式MTBUF
- 说明内存已键入- 缓冲指令

表 96. MTBUF 字段

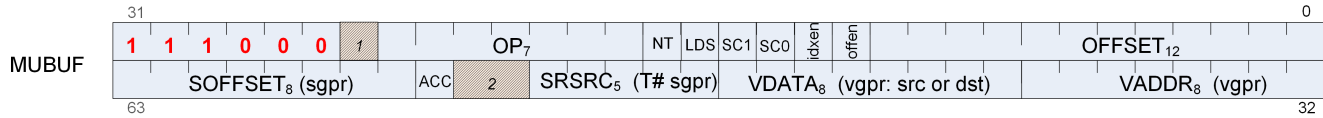
| 字段名称   | 位      | 格式或说明                       |
|--------|--------|-----------------------------|
| OFFSET | [11:0] | 请检查 date=中的日期值 (帮助)         |
| OFFEN  | [12]   | 1 = 电源擦除 VGPR, 0 = 地址擦除使用零  |
| IDXEN  | [13]   | 1 = 启用索引 VGPR, 0 = 地址索引使用 0 |
| SC0    | [14]   | 范围位 0                       |

| 字段名称     | 位       | 格式或说明                                                                                                                                                |
|----------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| OP       | [18:15] | Opcode,见下表。                                                                                                                                          |
| DFMT     | 22:19   | 内存缓冲器中的数据格式: 0 无效 1 8 2<br>16 3 8 8 4 32 5 16 16 6 10 11 11 11<br>8 10 10 2 9 2 10 10 10 8 8 8 11<br>32 32 12 16 16 16 13 32 32 32 14<br>32 32 32 32 |
| NFMT     | 25:23   | 内存中数据的数值格式: 0 = unorm, 1 = snorm, 2 = uscaled, 3 = sscaled, 4 = uint, 5 = sint, 6 = 保留, 7 = float。                                                   |
| ENCODING | [31:26] | 一定是: 111010 号                                                                                                                                        |
| VADDR    | [39:32] | VGPR 的地址用于提供地址的第一个组成部分(抵消或索引). 当同时使用索引和抵消时,索引在第一个 VGPR 中,在第二个中抵消.                                                                                    |
| VDATA    | [47:40] | VGPR 的地址, 用于提供写数据的第一部分或接收读数据的第一部分。                                                                                                                   |
| SRSRC    | [52:48] | SGPR 连续 4 个或 8 个 SGPRs 提供 V#(资源常数),由于必须和 4 对齐,所以缺少 2 个 LSB 的 SGPR 地址.                                                                                |
| SC1      | [53]    | 范围位 1                                                                                                                                                |
| NT       | [54]    | 非时间性                                                                                                                                                 |
| ACC      | [55]    | VDATA 是累加 VGPR                                                                                                                                       |
| SOFFSET  | [63:56] | 请检查 date=中的日期值 (帮助)                                                                                                                                  |

表 97. MTBUF Opcodes

| 操作码编号 | 名称                        | 操作码编号 | 名称                            |
|-------|---------------------------|-------|-------------------------------|
| 0     | TBUFFER_LOAD_FORMAT_X     | 8     | TBUFFER_LOAD_FORMAT_D16_X     |
| 1     | TBUFFER_LOAD_FORMAT_XY    | 9     | TBUFFER_LOAD_FORMAT_D16_XY    |
| 2     | TBUFFER_LOAD_FORMAT_XYZ   | 10    | TBUFFER_LOAD_FORMAT_D16_XYZ   |
| 3     | TBUFFER_LOAD_FORMAT_XYZW  | 11    | TBUFFER_LOAD_FORMAT_D16_XYZW  |
| 4     | TBUFFER_STORE_FORMAT_X    | 12    | TBUFFER_STORE_FORMAT_D16_X    |
| 5     | TBUFFER_STORE_FORMAT_XY   | 13    | TBUFFER_STORE_FORMAT_D16_XY   |
| 6     | TBUFFER_STORE_FORMAT_XYZ  | 14    | TBUFFER_STORE_FORMAT_D16_XYZ  |
| 7     | TBUFFER_STORE_FORMAT_XYZW | 15    | TBUFFER_STORE_FORMAT_D16_XYZW |

13.5.2. MUBUF



格式 MUBUF

说明 内存未类型- 缓冲指令

表 98. MUBUF 字段

| 字段名称     | 位       | 格式或说明                                                                 |
|----------|---------|-----------------------------------------------------------------------|
| OFFSET   | [11:0]  | 请检查 date=中的日期值 (帮助)                                                   |
| OFFEN    | [12]    | 1 = 电源擦除 VGPR, 0 = 地址擦除使用零                                            |
| IDXEN    | [13]    | 1 = 启用索引 VGPR, 0 = 地址索引使用 0                                           |
| SC0      | [14]    | 范围位 0                                                                 |
| SC1      | [15]    | 范围位 1                                                                 |
| LDS      | [16]    | 0 = 正常, 1 = 传输数据介于 LDS 与内存之间,而不是 VGPRs 与内存之间.                         |
| NT       | [17]    | 非时间性                                                                  |
| OP       | [24:18] | Opcode,见下表。                                                           |
| ENCODING | [31:26] | 一定是: 111000 号                                                         |
| VADDR    | [39:32] | VGPR 的地址用于提供地址的第一个组成部分(抵消或索引). 当同时使用索引和抵消时,索引在第一个 VGPR 中,在第二个中抵消.     |
| VDATA    | [47:40] | VGPR 的地址, 用于提供写数据的第一部分或接收读数据的第一部分。                                    |
| SRSRC    | [52:48] | SGPR 连续 4 个或 8 个 SGPRs 提供 V#(资源常数),由于必须和 4 对齐,所以缺少 2 个 LSB 的 SGPR 地址. |
| ACC      | [55]    | VDATA 是累加 VGPR                                                        |
| SOFFSET  | [63:56] | 请检查 date=中的日期值 (帮助)                                                   |

表 99. MUBUF Opcodes

| 操作码编号 | 名称                           | 操作码编号 | 名称                           |
|-------|------------------------------|-------|------------------------------|
| 0     | BUFFER_LOAD_FORMAT_X         | 37    | BUFFER_LOAD_SHORT_D16_HI     |
| 1     | BUFFER_LOAD_FORMAT_XY        | 38    | BUFFER_LOAD_FORMAT_D16_HI_X  |
| 2     | BUFFER_LOAD_FORMAT_XYZ       | 39    | BUFFER_STORE_FORMAT_D16_HI_X |
| 3     | BUFFER_LOAD_FORMAT_XYZW      | 40    | BUFFER_WBL2                  |
| 4     | BUFFER_STORE_FORMAT_X        | 41    | BUFFER_INV                   |
| 5     | BUFFER_STORE_FORMAT_XY       | 64    | BUFFER_ATOMIC_SWAP           |
| 6     | BUFFER_STORE_FORMAT_XYZ      | 65    | BUFFER_ATOMIC_CMPSWAP        |
| 7     | BUFFER_STORE_FORMAT_XYZW     | 66    | BUFFER_ATOMIC_ADD            |
| 8     | BUFFER_LOAD_FORMAT_D16_X     | 67    | BUFFER_ATOMIC_SUB            |
| 9     | BUFFER_LOAD_FORMAT_D16_XY    | 68    | BUFFER_ATOMIC_SMIN           |
| 10    | BUFFER_LOAD_FORMAT_D16_XYZ   | 69    | BUFFER_ATOMIC_UMIN           |
| 11    | BUFFER_LOAD_FORMAT_D16_XYZW  | 70    | BUFFER_ATOMIC_SMAX           |
| 12    | BUFFER_STORE_FORMAT_D16_X    | 71    | BUFFER_ATOMIC_UMAX           |
| 13    | BUFFER_STORE_FORMAT_D16_XY   | 72    | BUFFER_ATOMIC_AND            |
| 14    | BUFFER_STORE_FORMAT_D16_XYZ  | 73    | BUFFER_ATOMIC_OR             |
| 15    | BUFFER_STORE_FORMAT_D16_XYZW | 74    | BUFFER_ATOMIC_XOR            |

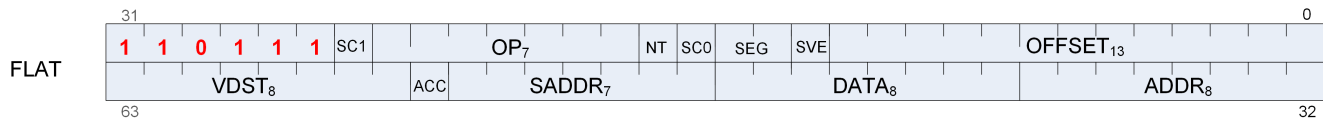
| 操作码编号 | 名称                        | 操作码编号 | 名称                       |
|-------|---------------------------|-------|--------------------------|
| 16    | BUFFER_LOAD_UBYTE         | 75    | BUFFER_ATOMIC_INC        |
| 17    | BUFFER_LOAD_SBYTE         | 76    | BUFFER_ATOMIC_DEC        |
| 18    | BUFFER_LOAD_USHORT        | 77    | BUFFER_ATOMIC_ADD_F32    |
| 19    | BUFFER_LOAD_SSHORT        | 78    | BUFFER_ATOMIC_PK_ADD_F16 |
| 20    | BUFFER_LOAD_DWORD         | 79    | BUFFER_ATOMIC_ADD_F64    |
| 21    | BUFFER_LOAD_DWORDX2       | 80    | BUFFER_ATOMIC_MIN_F64    |
| 22    | BUFFER_LOAD_DWORDX3       | 81    | BUFFER_ATOMIC_MAX_F64    |
| 23    | BUFFER_LOAD_DWORDX4       | 96    | BUFFER_ATOMIC_SWAP_X2    |
| 24    | BUFFER_STORE_BYTE         | 97    | BUFFER_ATOMIC_CMPSWAP_X2 |
| 25    | BUFFER_STORE_BYTE_D16_HI  | 98    | BUFFER_ATOMIC_ADD_X2     |
| 26    | BUFFER_STORE_SHORT        | 99    | BUFFER_ATOMIC_SUB_X2     |
| 27    | BUFFER_STORE_SHORT_D16_HI | 100   | BUFFER_ATOMIC_SMIN_X2    |
| 28    | BUFFER_STORE_DWORD        | 101   | BUFFER_ATOMIC_UMIN_X2    |
| 29    | BUFFER_STORE_DWORDX2      | 102   | BUFFER_ATOMIC_SMAX_X2    |
| 30    | BUFFER_STORE_DWORDX3      | 103   | BUFFER_ATOMIC_UMAX_X2    |
| 31    | BUFFER_STORE_DWORDX4      | 104   | BUFFER_ATOMIC_AND_X2     |
| 32    | BUFFER_LOAD_UBYTE_D16     | 105   | BUFFER_ATOMIC_OR_X2      |
| 33    | BUFFER_LOAD_UBYTE_D16_HI  | 106   | BUFFER_ATOMIC_XOR_X2     |
| 34    | BUFFER_LOAD_SBYTE_D16     | 107   | BUFFER_ATOMIC_INC_X2     |
| 35    | BUFFER_LOAD_SBYTE_D16_HI  | 108   | BUFFER_ATOMIC_DEC_X2     |
| 36    | BUFFER_LOAD_SHORT_D16     |       |                          |

## 13.6. Flat 格式

平坦内存指令分为三种：FLAT：每个工作项的内存地址可以位于全局内存、暂存（私有）内存或共享内存（LDS）。GLOBAL：与 FLAT 类似，但假定所有地址均指向全局内存。SCRATCH：与 FLAT 类似，但假定所有地址均指向暂存（私有）内存。

微码格式对每个都相同,只有 SEG(片段)字段的值不同.

### 13.6.1. FLAT



|    |           |
|----|-----------|
| 格式 | FLAT      |
| 说明 | FLAT 内存访问 |

表 100. FLAT 字段

| 字段名称   | 位      | 格式或说明                                                        |
|--------|--------|--------------------------------------------------------------|
| OFFSET | [12:0] | 地址偏移 Scratch, Global: 13 位签名字节偏移<br>FLAT: 12 位无符号偏移(MSB 被忽略) |



| 字段名称     | 位       | 格式或说明                                                                                                                                                                                    |
|----------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LDS      | [13]    | 0 = 正常, 1 = 传输数据介于 LDS 与内存之间,而不是 VGPRs 与内存之间.                                                                                                                                            |
| SEG      | [15:14] | 内存段(指令类型):0=平,1=刮,2=全局.                                                                                                                                                                  |
| SC0      | [16]    | 范围位 0                                                                                                                                                                                    |
| NT       | [17]    | 非时间性                                                                                                                                                                                     |
| OP       | [24:18] | Opcode. 参见下表 FLAT,SCRATCH 和 GLOBAL opcode.                                                                                                                                               |
| SC1      | [25]    | 范围位 1                                                                                                                                                                                    |
| ENCODING | [31:26] | 一定是: 110111 号                                                                                                                                                                            |
| ADDR     | [39:32] | VGPR, 持有地址或抵消。 对于 64 位地址, ADDR 拥有 LSB, ADDR+1 拥有 MSB。 对于一个 VGPR 有 32 位无符号的偏移。 对于 FLAT *: 指定地址。 对于 Global *和 SCRATCH *, SADR 是 0x7f: 指定地址。 对于 Global_*和 SCRATCH_*, SADR 不是 0x7f, 请指定一个偏移。 |
| DATA     | [47:40] | VGPR 提供数据.                                                                                                                                                                               |
| SADDR    | [54:48] | 标量 SGPR 提供了一个偏移地址(无符号)。 设置此字段为 0x7f 以禁用。 此字段的含义对 Scratch 和 Global 不同: FLAT: 未使用字节: 使用 SGPR 而不是 VGPR Global: 使用 SGPR 提供基址, VGPR 提供 32 位字节的偏移。                                             |
| ACC      | [55]    | VDATA 是累加 VGPR                                                                                                                                                                           |
| VDST     | [63:56] | 用于接收从内存返回到 VGPRs 的数据的目标 VGPR。                                                                                                                                                            |

表 101. FLAT Opcode

| 操作码编号 | 名称                      | 操作码编号 | 名称                      |
|-------|-------------------------|-------|-------------------------|
| 16    | FLAT_LOAD_UBYTE         | 69    | FLAT_ATOMIC_UMIN        |
| 17    | FLAT_LOAD_SBYTE         | 70    | FLAT_ATOMIC_SMAX        |
| 18    | FLAT_LOAD_USHORT        | 71    | FLAT_ATOMIC_UMAX        |
| 19    | FLAT_LOAD_SSHORT        | 72    | FLAT_ATOMIC_AND         |
| 20    | FLAT_LOAD_DWORD         | 73    | FLAT_ATOMIC_OR          |
| 21    | FLAT_LOAD_DWORDX2       | 74    | FLAT_ATOMIC_XOR         |
| 22    | FLAT_LOAD_DWORDX3       | 75    | FLAT_ATOMIC_INC         |
| 23    | FLAT_LOAD_DWORDX4       | 76    | FLAT_ATOMIC_DEC         |
| 24    | FLAT_STORE_BYTE         | 77    | FLAT_ATOMIC_ADD_F32     |
| 25    | FLAT_STORE_BYTE_D16_HI  | 78    | FLAT_ATOMIC_PK_ADD_F16  |
| 26    | FLAT_STORE_SHORT        | 79    | FLAT_ATOMIC_ADD_F64     |
| 27    | FLAT_STORE_SHORT_D16_HI | 80    | FLAT_ATOMIC_MIN_F64     |
| 28    | FLAT_STORE_DWORD        | 81    | FLAT_ATOMIC_MAX_F64     |
| 29    | FLAT_STORE_DWORDX2      | 82    | FLAT_ATOMIC_PK_ADD_BF16 |
| 30    | FLAT_STORE_DWORDX3      | 96    | FLAT_ATOMIC_SWAP_X2     |
| 31    | FLAT_STORE_DWORDX4      | 97    | FLAT_ATOMIC_CMPSWAP_X2  |
| 32    | FLAT_LOAD_UBYTE_D16     | 98    | FLAT_ATOMIC_ADD_X2      |
| 33    | FLAT_LOAD_UBYTE_D16_HI  | 99    | FLAT_ATOMIC_SUB_X2      |
| 34    | FLAT_LOAD_SBYTE_D16     | 100   | FLAT_ATOMIC_SMIN_X2     |
| 35    | FLAT_LOAD_SBYTE_D16_HI  | 101   | FLAT_ATOMIC_UMIN_X2     |
| 36    | FLAT_LOAD_SHORT_D16     | 102   | FLAT_ATOMIC_SMAX_X2     |
| 37    | FLAT_LOAD_SHORT_D16_HI  | 103   | FLAT_ATOMIC_UMAX_X2     |
| 64    | FLAT_ATOMIC_SWAP        | 104   | FLAT_ATOMIC_AND_X2      |



| 操作码编号 | 名称                  | 操作码编号 | 名称                 |
|-------|---------------------|-------|--------------------|
| 65    | FLAT_ATOMIC_CMPSWAP | 105   | FLAT_ATOMIC_OR_X2  |
| 66    | FLAT_ATOMIC_ADD     | 106   | FLAT_ATOMIC_XOR_X2 |
| 67    | FLAT_ATOMIC_SUB     | 107   | FLAT_ATOMIC_INC_X2 |
| 68    | FLAT_ATOMIC_SMIN    | 108   | FLAT_ATOMIC_DEC_X2 |

## 13.6.2. GLOBAL

表 102. GLOBAL Opcode

| 操作码编号 | 名称                        | 操作码编号 | 名称                        |
|-------|---------------------------|-------|---------------------------|
| 16    | GLOBAL_LOAD_UBYTE         | 67    | GLOBAL_ATOMIC_SUB         |
| 17    | GLOBAL_LOAD_SBYTE         | 68    | GLOBAL_ATOMIC_SMIN        |
| 18    | GLOBAL_LOAD_USHORT        | 69    | GLOBAL_ATOMIC_UMIN        |
| 19    | GLOBAL_LOAD_SSHORT        | 70    | GLOBAL_ATOMIC_SMAX        |
| 20    | GLOBAL_LOAD_DWORD         | 71    | GLOBAL_ATOMIC_UMAX        |
| 21    | GLOBAL_LOAD_DWORDX2       | 72    | GLOBAL_ATOMIC_AND         |
| 22    | GLOBAL_LOAD_DWORDX3       | 73    | GLOBAL_ATOMIC_OR          |
| 23    | GLOBAL_LOAD_DWORDX4       | 74    | GLOBAL_ATOMIC_XOR         |
| 24    | GLOBAL_STORE_BYTE         | 75    | GLOBAL_ATOMIC_INC         |
| 25    | GLOBAL_STORE_BYTE_D16_HI  | 76    | GLOBAL_ATOMIC_DEC         |
| 26    | GLOBAL_STORE_SHORT        | 77    | GLOBAL_ATOMIC_ADD_F32     |
| 27    | GLOBAL_STORE_SHORT_D16_HI | 78    | GLOBAL_ATOMIC_PK_ADD_F16  |
| 28    | GLOBAL_STORE_DWORD        | 79    | GLOBAL_ATOMIC_ADD_F64     |
| 29    | GLOBAL_STORE_DWORDX2      | 80    | GLOBAL_ATOMIC_MIN_F64     |
| 30    | GLOBAL_STORE_DWORDX3      | 81    | GLOBAL_ATOMIC_MAX_F64     |
| 31    | GLOBAL_STORE_DWORDX4      | 82    | GLOBAL_ATOMIC_PK_ADD_BF16 |
| 32    | GLOBAL_LOAD_UBYTE_D16     | 96    | GLOBAL_ATOMIC_SWAP_X2     |
| 33    | GLOBAL_LOAD_UBYTE_D16_HI  | 97    | GLOBAL_ATOMIC_CMPSWAP_X2  |
| 34    | GLOBAL_LOAD_SBYTE_D16     | 98    | GLOBAL_ATOMIC_ADD_X2      |
| 35    | GLOBAL_LOAD_SBYTE_D16_HI  | 99    | GLOBAL_ATOMIC_SUB_X2      |
| 36    | GLOBAL_LOAD_SHORT_D16     | 100   | GLOBAL_ATOMIC_SMIN_X2     |
| 37    | GLOBAL_LOAD_SHORT_D16_HI  | 101   | GLOBAL_ATOMIC_UMIN_X2     |
| 38    | GLOBAL_LOAD_LDS_UBYTE     | 102   | GLOBAL_ATOMIC_SMAX_X2     |
| 39    | GLOBAL_LOAD_LDS_SBYTE     | 103   | GLOBAL_ATOMIC_UMAX_X2     |
| 40    | GLOBAL_LOAD_LDS_USHORT    | 104   | GLOBAL_ATOMIC_AND_X2      |
| 41    | GLOBAL_LOAD_LDS_SSHORT    | 105   | GLOBAL_ATOMIC_OR_X2       |
| 42    | GLOBAL_LOAD_LDS_DWORD     | 106   | GLOBAL_ATOMIC_XOR_X2      |
| 64    | GLOBAL_ATOMIC_SWAP        | 107   | GLOBAL_ATOMIC_INC_X2      |
| 65    | GLOBAL_ATOMIC_CMPSWAP     | 108   | GLOBAL_ATOMIC_DEC_X2      |
| 66    | GLOBAL_ATOMIC_ADD         |       |                           |

## 13.6.3. SCRATCH

表 103. SCRATCH Opcode

| 操作码编号 | 名称                 | 操作码编号 | 名称                    |
|-------|--------------------|-------|-----------------------|
| 16    | SCRATCH_LOAD_UBYTE | 30    | SCRATCH_STORE_DWORDX3 |

| 操作码编号 | 名称                         | 操作码编号 | 名称                        |
|-------|----------------------------|-------|---------------------------|
| 17    | SCRATCH_LOAD_SBYTE         | 31    | SCRATCH_STORE_DWORDX4     |
| 18    | SCRATCH_LOAD_USHORT        | 32    | SCRATCH_LOAD_UBYTE_D16    |
| 19    | SCRATCH_LOAD_SSHORT        | 33    | SCRATCH_LOAD_UBYTE_D16_HI |
| 20    | SCRATCH_LOAD_DWORD         | 34    | SCRATCH_LOAD_SBYTE_D16    |
| 21    | SCRATCH_LOAD_DWORDX2       | 35    | SCRATCH_LOAD_SBYTE_D16_HI |
| 22    | SCRATCH_LOAD_DWORDX3       | 36    | SCRATCH_LOAD_SHORT_D16    |
| 23    | SCRATCH_LOAD_DWORDX4       | 37    | SCRATCH_LOAD_SHORT_D16_HI |
| 24    | SCRATCH_STORE_BYTE         | 38    | SCRATCH_LOAD_LDS_UBYTE    |
| 25    | SCRATCH_STORE_BYTE_D16_HI  | 39    | SCRATCH_LOAD_LDS_SBYTE    |
| 26    | SCRATCH_STORE_SHORT        | 40    | SCRATCH_LOAD_LDS_USHORT   |
| 27    | SCRATCH_STORE_SHORT_D16_HI | 41    | SCRATCH_LOAD_LDS_SSHORT   |
| 28    | SCRATCH_STORE_DWORD        | 42    | SCRATCH_LOAD_LDS_DWORD    |
| 29    | SCRATCH_STORE_DWORDX2      |       |                           |

## 排版补充译文

### 补译 1 · 原 PDF 第 3 页

Santa Clara, CA, 95054

圣克拉拉, 加利福尼亚州, 95054

### 补译 2 · 原 PDF 第 37 页 / 正文第 29 页

Sets SCC?

设置 SCC?

### 补译 3 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 0

寄存器 0

### 补译 4 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 1

寄存器 1

### 补译 5 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 0

寄存器 0

### 补译 6 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 1

寄存器 1

### 补译 7 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 0

寄存器 0

### 补译 8 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 1

寄存器 1

补译 9 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 2

寄存器 2

补译 10 · 原 PDF 第 54 页 / 正文第 46 页

Reg. 7

寄存器 7

补译 11 · 原 PDF 第 58 页 / 正文第 50 页

(normal)

(规格化)

补译 12 · 原 PDF 第 90 页 / 正文第 82 页

10.4. Global

10.4. Global 全局内存